

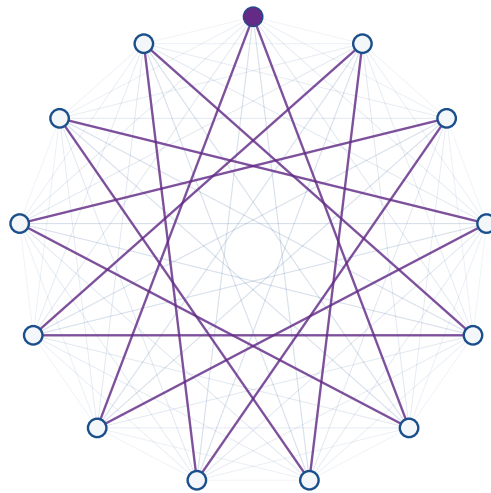
CUADERNILLO DE EJERCICIOS

Ejercicios y respuestas

Matemática discreta con apoyo de software

Enrique Vílchez Quesada

Universidad Nacional de Costa Rica



El grafo completo de trece vértices y, resaltado, uno de sus circuitos de Hamilton, que pasa una sola vez por cada vértice.

1712 ejercicios sobre los ocho capítulos del libro, todos resueltos: los **205 del libro** con sus respuestas literales, más **1226 complementarios** y **281 preguntas tipo examen** escritos para este cuadernillo.

Extraído de la versión final, compilada el **19 de agosto de 2023** (624 páginas). La numeración **capítulo.sección.ejercicio** es la misma del libro, de modo que **1.5.7** aquí y **1.5.7** allá son el mismo ejercicio.

Índice

Cómo usar este cuadernillo	1
Los ejercicios	3
Capítulo 1. Recursividad	4
1.5. Ejercicios	4
Capítulo 2. Relaciones de recurrencia	6
2.3. Ejercicios	6
Capítulo 3. Análisis de algoritmos	8
3.5. Ejercicios	8
Capítulo 4. Relaciones binarias	10
4.5. Ejercicios	10
Capítulo 5. Teoría de grafos	12
5.5. Ejercicios	12
Capítulo 6. Teoría de árboles	15
6.6. Ejercicios	15
Capítulo 7. Máquinas de estado finito y autómatas	17
7.5. Ejercicios	17
Capítulo 8. Lenguajes y gramáticas	22
8.4. Ejercicios	22
Las respuestas	27
Capítulo 1. Recursividad	28
Capítulo 2. Relaciones de recurrencia	32
Capítulo 3. Análisis de algoritmos	37
Capítulo 4. Relaciones binarias	39
Capítulo 5. Teoría de grafos	44
Capítulo 6. Teoría de árboles	50
Capítulo 7. Máquinas de estado finito y autómatas	62
Capítulo 8. Lenguajes y gramáticas	79
Ejercicios complementarios, resueltos	87
Capítulo 1. Recursividad · complementarios	88
Capítulo 2. Relaciones de recurrencia · complementarios	124
Capítulo 3. Análisis de algoritmos · complementarios	158
Capítulo 4. Relaciones binarias · complementarios	188
Capítulo 5. Teoría de grafos · complementarios	219
Capítulo 6. Teoría de árboles · complementarios	253
Capítulo 7. Máquinas de estado finito y autómatas · complementarios	285
Capítulo 8. Lenguajes y gramáticas · complementarios	313
Preguntas tipo examen	341

Capítulo 1. Recursividad · tipo examen	342
Capítulo 2. Relaciones de recurrencia · tipo examen	354
Capítulo 3. Análisis de algoritmos · tipo examen	367
Capítulo 4. Relaciones binarias · tipo examen	378
Capítulo 5. Teoría de grafos · tipo examen	390
Capítulo 6. Teoría de árboles · tipo examen	404
Capítulo 7. Máquinas de estado finito y autómatas · tipo examen	416
Capítulo 8. Lenguajes y gramáticas · tipo examen	429
Recursos, funciones y mapa de estudio	439
Apéndice A. Los recursos del curso	440
Apéndice B. Índice de funciones de VilCretas	443
Apéndice C. Mapa de estudio y simulacros	453

GUÍA DE LECTURA

Cómo usar este cuadernillo

Las cuatro partes

El cuadernillo se lee en cuatro bloques, y cada uno sirve para un momento distinto del estudio:

1. **Los ejercicios del libro.** Los mismos que trae *Matemática discreta con apoyo de software*, con la numeración del libro: **1.5.7** aquí y **1.5.7** allá son el mismo ejercicio.
2. **Las respuestas del libro** (solo en el documento con soluciones).
3. **Ejercicios complementarios.** Escritos para este cuadernillo, dentro del alcance temático de cada capítulo. Sirven para practicar concepto por concepto.
4. **Preguntas tipo examen.** Reproducen el estilo, el formato y el nivel de las preguntas del banco real del curso. Sirven para medirse *antes* de la prueba.

El sistema de estrellas

Cada ejercicio nuevo lleva una marca de dificultad. No mide cuánto cuesta *escribir* la respuesta, sino **cuántos pasos de razonamiento distintos** hay entre el enunciado y la solución.

- ★ **Básico.** Aplicación directa de una definición, una fórmula o un comando ya visto: identificar el caso raíz de una recursividad, evaluar **Factoriales**[8], leer una valencia en un grafo. Un solo paso. Si domina el material, debería resolverlo en **menos de tres minutos** y sin consultar apuntes. Si alguno de estos se le atasca, el problema no es el ejercicio: es el concepto.
- ★★ **Intermedio.** Exige **combinar dos cosas**: calcular a mano y verificar con el software, escribir un programa siguiendo un patrón conocido, traducir entre dos representaciones (matriz y grafo, gramática y autómatas), o leer el efecto de una opción sobre el resultado. Es el grueso del cuadernillo. **Entre cinco y quince minutos.**
- ★★★ **Avanzado, nivel de examen.** Pide **analizar, comparar, justificar o construir**: demostrar una propiedad, encontrar el error en un programa ajeno, decidir por qué un criterio no se puede aplicar, diseñar un autómata o una gramática desde cero, o explicar por qué dos métodos que dan el mismo resultado no son igual de eficientes. No hay un procedimiento que seguir: hay que decidir cuál usar. **Más de quince minutos**, y a menudo hay que escribir el razonamiento antes de tocar el teclado.

Cómo autoevaluarse antes de un examen

La regla práctica es sencilla:

1. Resuelva los ★ de un capítulo **sin mirar la solución**. Si falla alguno, vuelva al libro antes de seguir.
2. Pase a los ★★. Aquí sí vale consultar el enunciado del libro o la ayuda de un comando (?NombreFuncion), pero no la solución.
3. Cuando los ★★ salgan con fluidez, mida el tiempo en los ★★★ y en las **TIPO EXAMEN**. **Ese es el nivel del examen**. Si necesita la solución para arrancar, todavía no está listo.

4. Cierre con el **simulacro** del capítulo y con el **checklist de autoestudio**, que están al final de cada bloque de preguntas tipo examen.

Las etiquetas de tipo

A MANO Se resuelve con papel y lápiz. El software no ayuda, o ayuda tan poco que usarlo estorba.

VilCretas Se resuelve con *Wolfram Mathematica* y el paquete **VilCretas**. El código de la solución está probado: se ejecutó y se comparó con la salida que aparece impresa.

MIXTO Se resuelve a mano **y** se verifica con el software. Es el hábito que el curso quiere formar: primero el razonamiento, después la comprobación.

TIPO EXAMEN Reproduce una categoría concreta del banco de preguntas del curso, citada bajo el enunciado. Si puede con estas, puede con la prueba.

Sobre el código

Los bloques de código se presentan como en un cuaderno de *Mathematica*: un rótulo de **entrada** encabeza lo que se escribe y un rótulo de **salida** lo que el programa devuelve. Cuando el comando no devuelve un valor sino un dibujo, una animación o un procedimiento paso a paso, solo aparece la entrada, y la solución describe con palabras lo que se ve en pantalla.

Un código impreso en gris claro, **sin rótulo de entrada**, es código que el programa *muestra* de sí mismo (por ejemplo con la opción `code -> True`) y que no debe teclearse: varios de esos nombres son símbolos protegidos del paquete y reasignarlos hace fallar el cálculo sin previo aviso.

Los tres apéndices, y la carpeta Wolfram

Al final del cuadernillo hay tres apéndices que no son ejercicios, sino herramientas de trabajo.

Apéndice A

El catálogo de los recursos del curso: los cuadernos y documentos de *Materiales importantes ED*, los **31 videos** de las clases con su enlace directo, las **26 presentaciones** y los recursos que no pertenecen a ningún capítulo. Cada entrada dice a qué capítulo del libro corresponde.

Apéndice B

El índice de las **197 funciones de VilCretas** que el curso llega a usar, con los capítulos en que aparece cada una, lo que hace y un ejercicio concreto donde verla trabajando.

Apéndice C

El mapa de estudio: cuántos ejercicios hay en cada capítulo y de qué tipo, la lista de lo que hay que saber hacer (pensada para marcarla) y un **simulacro de examen** de diez preguntas cronometrado.

Aparte del PDF, el proyecto trae una carpeta **Wolfram** con un archivo por capítulo, en formato **.wl** y **.nb**, que reúne **todo el código** de las soluciones que lo ameritan: las del libro (tomadas de los cuadernos del autor) y las de los ejercicios nuevos, cada bloque rotulado con el número del ejercicio al que pertenece. Basta abrir el cuaderno del capítulo y evaluarlo de arriba abajo.

PRIMERA PARTE

Los ejercicios

CAPÍTULO 1

Recursividad

1.5 Ejercicios

Se insta al estudiante a resolver el conjunto de ejercicios propuestos con el objetivo de complementar su preparación en cada uno de los contenidos relacionados con el tema de recursividad.

1.5.1 ¿Qué ocurre con la función recursiva **Factoriales** desarrollada en la sección 1.1, si n es un número entero negativo? Justifique su respuesta.

Genere un programa recursivo de pila en *Wolfram Mathematica* que calcule el resultado de:

1.5.2
$$\sum_{i=1}^{n-5} \frac{i}{i+1}.$$

1.5.3
$$\sum_{i=14}^{n+5} \left(\frac{i}{2^{i-4}} \right)^2.$$

1.5.4
$$\prod_{i=1}^{n-10} (3i-1).$$

1.5.5
$$\prod_{i=8}^{n+3} \left(\frac{3^{i+1}}{i+1} \right).$$

1.5.6
$$\sum_{i=1}^{n+1} \left[\left(\prod_{j=2}^{i-1} (2j-5) \right) i \right].$$

1.5.7 Construya una recursividad de cola para cada uno de los ejercicios del punto 2.

Elabore un programa recursivo de pila y de cola en *Wolfram Mathematica* que calcule el resultado de:

1.5.8 $a_n = 5a_{n-1} + 2n - \sqrt{2}$ con $a_5 = 2$.

1.5.9 $a_n = a_{n-1} + 6a_{n-2}$ con $a_1 = -10$ y $a_2 = -19$.

1.5.10 $a_n = -36a_{n-2} + a_{n-3} + 12a_{n-1}$ con $a_1 = 1$, $a_2 = -1$ y $a_3 = \sqrt[5]{2}$.

1.5.11 $9a_n = 6a_{n-1} - a_{n-2} + a_{n-3} - 2a_{n-4}$ con $a_2 = 1$, $a_3 = 2$, $a_4 = 3$ y $a_5 = 4$.

1.5.12 Mediante un programa recursivo resuelva el siguiente problema: suponga que una persona inicia un ahorro en un banco. Si el monto de intereses a pagar cada mes por la entidad bancaria es de un 5 % y el capital inicial fue de US \$200, ¿cuánto dinero tendrá la persona en su cuenta al cabo de un año?

1.5.13 Los llamados números de *Catalan* en honor al matemático belga *E. Catalan* se definen así:

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1}, n \in \mathbb{N} \cup \{0\} \text{ con } C_0 = 1$$

Construya un programa recursivo en *Mathematica* que retorne el n -ésimo número de *Catalan*. Genere la lista de los primeros treinta números de *Catalan* usando esa recursividad.

- 1.5.14** Realice un programa en *Wolfram Mathematica* que implemente la siguiente recursividad para calcular el máximo común divisor entre dos números enteros positivos o cero:

$$MCD(n, m) = \begin{cases} n & \text{si } m = 0 \\ m & \text{si } n = 0 \\ MCD(n - m, m) & \text{si } n \geq m \\ MCD(n, m - n) & \text{si } n < m \end{cases}$$

Recorra paso a paso el cálculo de $MCD(120, 144)$.

- 1.5.15** El matemático alemán *Wilhelm Ackermann* propuso una función doblemente recursiva para números n y m enteros positivos o cero, definida como sigue:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ y } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ y } n > 0 \end{cases}$$

En ocasiones ésta es utilizada para comparar la eficiencia de compiladores al crecer muy rápidamente. Genere en *Wolfram Mathematica* dicha función. Empleando el método elaborado, conjeture una fórmula para $A(1, n)$, $A(2, n)$ y $A(3, n)$ con $n \in \mathbb{N} \cup \{0\}$.

- 1.5.16** Construya una recursividad de cola en los ejemplos 1.5 y 1.8.
- 1.5.17** Elabore un programa recursivo en *Wolfram Language* que convierta un número en base decimal a base dos.
- 1.5.18** Implemente una recursividad que determine si dado un entero positivo n éste es un número perfecto. Se dice que $n \in \mathbb{N}$ es perfecto si al sumar todos sus divisores propios el resultado es igual a n .
- 1.5.19** Realice un programa recursivo que determine todas las descomposiciones de un número natural n como la suma de dos números enteros no negativos. Por ejemplo, si se invocara el programa para $n = 5$, devolvería: $4 + 1$, $3 + 2$, $2 + 3$, $1 + 4$, $0 + 5$.
- 1.5.20** Por medio del software *Mathematica*, implemente un algoritmo recursivo que calcule el mínimo común múltiplo entre dos números enteros positivos n y m . Utilice la instrucción `LCM[n,m]` para verificar algunos resultados.
- 1.5.21** Modifique el algoritmo *quicksort* compartido en el ejemplo 1.10, con el objetivo de ordenar una lista de números reales de forma descendente.
- 1.5.22** Construya un programa recursivo en *Wolfram Mathematica* para hallar el máximo de una lista de n números reales. Sugerencia: utilice como base el algoritmo *quicksort*.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Recursividad/Exercises.zip>

CAPÍTULO 2

Relaciones de recurrencia

2.3 Ejercicios

Se invita al alumno a resolver el conjunto de ejercicios propuestos.

Por medio de los comandos **GraficaRRL** y **RT**, grafique y obtenga los primeros veinte términos de las siguientes sucesiones definidas por una relación de recurrencia.

2.3.1 $a_n = -4a_{n-1}$ con $a_0 = 5$.

2.3.2 $a_n = 3n \cdot a_{n-1} + 3^n$ con $a_1 = 1$.

2.3.3 $a_n = 4n \cdot a_{n-1} - 2a_n$ con $a_2 = \frac{5}{2}$.

2.3.4 $a_n = -a_{n-1} + 6a_{n-2}$ con $a_0 = 1$ y $a_1 = 3$.

2.3.5 $a_n = a_{n-1} + a_{n-2}$ con $a_1 = 1$ y $a_2 = 3$ (a esta sucesión se le llama sucesión de *Lucas* en honor a su creador, el matemático francés *Édouard Lucas*, quien también desarrolló el juego de las torres de *Hanoi*).

2.3.6 $a_n = \sqrt{2}a_{n-2} + 5a_{n-3} - a_{n-1}$ con $a_5 = 7$, $a_6 = -8$ y $a_7 = \frac{3}{2}$.

2.3.7 $(a_n)^3 = 5(a_{n-2})^3 - \frac{7}{2}(a_{n-1})^3$ con $a_3 = -\frac{2}{3}$ y $a_2 = \sqrt{2}$. Sugerencia: para trazar la gráfica recurra a la instrucción **DiscretePlot** de *Wolfram Mathematica*.

Encuentre una relación de recurrencia que represente la sucesión $(a_n)_{n \in \mathbb{N}}$.

2.3.8 $a_n = \{1, 3, 10, 41, 206, 1237, \dots\}$.

2.3.9 $a_n = \{5, -8, 29, -\frac{109}{2}, \frac{689}{4}, -\frac{2869}{8}, \dots\}$.

2.3.10 $a_n = \frac{1}{2}n^2 - \frac{1}{2}n \quad \forall n, n \in \mathbb{N}$.

2.3.11 $a_n = 3^{n+1}(12 - n) \quad \forall n, n \in \mathbb{N}$.

Resuelva paso a paso las relaciones de recurrencia dadas a continuación. Verifique el resultado, si es posible, a través del software *Wolfram Mathematica*.

2.3.12 La relación de recurrencia obtenida en el ejemplo 2.4 que cuenta el número de pasos necesarios para solucionar el juego de las torres de *Hanoi*.

2.3.13 $a_n = 3^n a_{n-1}$ con $a_0 = 2$.

2.3.14 $a_n = 2n \cdot a_{n-1}$ con $a_1 = 3$.

2.3.15 $a_n = 5a_{n-1} + 2n - \sqrt{2}$ con $a_5 = 2$.

2.3.16 $a_n = \sqrt{\frac{a_{n-1}}{3}}$ con $a_0 = 1$.

2.3.17 $a_n = 7a_{n-1} - a_{n-2}$ con $a_0 = 1$ y $a_1 = 5$.

2.3.18 $5a_n = -5a_{n-2} + 4a_{n-1}$ con $a_2 = 2$ y $a_3 = 1$.

2.3.19 $a_n = 5a_{n-1} - 8a_{n-2} + 4a_{n-3}$ con $a_0 = 5$, $a_1 = 2$ y $a_2 = 3$.

2.3.20 $a_n = 500a_{n-3} + 20a_{n-1} - 625a_{n-4} - 150a_{n-2}$ con $a_1 = a_0 = 1$ y $a_2 = a_3 = 3$.

2.3.21 $\sqrt{a_n} = \sqrt{a_{n-1}} + 2\sqrt{a_{n-2}}$ con $a_1 = 1$ y $a_2 = 2$.

2.3.22 $(a_n)^2 = -2(a_{n-2})^2 + (a_{n-1})^2$ con $a_0 = -2$ y $a_1 = -\sqrt{5}$.

Resuelva los siguientes problemas. Verifique con la ayuda del software *Mathematica*.

2.3.23 Una persona tiene n dólares y cada día compra pan a *US* \$1, leche a *US* \$2 o café a *US* \$2. Si a_n representa el número de formas en las que puede gastar el dinero, encuentre una relación de recurrencia para a_n . Resuelva dicha relación de recurrencia.

2.3.24 La población mundial en el año 2003 se estimó en 7.1 billones de personas y se calculó un crecimiento global de 1.2 % al año. Encuentre una relación de recurrencia que determine después del año 2003 la población mundial. Resuelva la relación de recurrencia. Calcule una estimación del número de personas en el mundo para el año 2050.

Conjeture la solución general de las siguientes relaciones de recurrencia con $j \in \mathbb{N}$. Sugerencia: utilice el comando **RRCP** del paquete **VilCretas**. Para una ayuda sobre su empleo consulte la página 577 de este libro.

2.3.25 $a_n = -a_{n-1} + 2a_{n-2} + 2j$ con $a_0 = 1$ y $a_1 = 2$.

2.3.26 $a_n = a_{n-1} + a_{n-2} + j$ con $a_2 = a_3 = 1$.

2.3.27 $a_n = -a_{n-1} - a_{n-2} - j$ con $a_0 = 1$ y $a_1 = 2$.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Recurrencia/Exercises.zip>

CAPÍTULO 3

Análisis de algoritmos

3.5 Ejercicios

El conjunto de ejercicios propuestos brindará al estudiante un soporte adecuado, con el objetivo de fortalecer sus conocimientos en cada uno de los contenidos relacionados con el tema de análisis de algoritmos.

Recurriendo al comando **PruebaADA2** determine cuál algoritmo es más eficiente en tiempo de ejecución.

3.5.1 Una recursividad de pila y otra de cola para encontrar los números de *Fibonacci*.

3.5.2 Los métodos **a** y **Recurrencia** con $a[n_]:=2\ a[n-1]-3\ a[n-2]$, $a[1]=\text{Sqrt}[2]$, $a[2]=1$ y $\text{Recurrencia}[n_]:= \text{If}[n==1, \text{Sqrt}[2], \text{If}[n==2, 1, 2\ \text{Recurrencia}[n-1]-3\ \text{Recurrencia}[n-2]]]$.

3.5.3 Los métodos **MCD1** y **MCD2** que determinan el máximo común divisor entre dos números enteros, donde $\text{MCD1}[n_, m_] := \text{If}[n == 0, m, \text{MCD1}[\text{Mod}[m, n], n]]$ y $\text{MCD2}[n_, m_] := \text{If}[n == 0, m, \text{If}[m == 0, n, \text{If}[n >= m, \text{MCD2}[n - m, m], \text{MCD2}[n, m - n]]]]$. Como una sugerencia: tome para alguno de los dos números enteros un valor fijo.

3.5.4 Las funciones **Seleccion** e **Insercion** del paquete **VilCretas**.

3.5.5 Los algoritmos de la burbuja y ordenamiento rápido (*quicksort*).

Demuestre las siguientes igualdades y represente gráficamente utilizando el *software* Wolfram *Mathematica*.

3.5.6 $a_j n^j + a_{j-1} n^{j-1} + \cdots + a_0 = \Omega(n^j)$ con $j \in \mathbb{N}$ y $a_j > 0$.

3.5.7 $1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\ln n)$.

3.5.8 $\frac{n^3 + 5}{5n + 2} = O(n^2)$.

3.5.9 $n \ln n = O(n^2)$.

3.5.10 $n^{2^n} + n^{n^2} = O(n^{2^n})$.

3.5.11 $\sum_{i=1}^n i^2 = \Theta(n^3)$.

Halle una notación O grande tan buena como sea posible para las siguientes funciones.

3.5.12 $7^n + 10n^2 + 8n^3 + 3^n$.

3.5.13 $(2^n + n^2)(5n^3 + 5^n)$.

3.5.14 $(n^n + n2^n + 3^n)(n! + 3^n)$.

3.5.15 $(n^3 + n^2 \ln n)(\ln n + 1) + (6 \ln n + 1)(n^4 + 2)$.

3.5.16 $(n + \ln(n + 1) + 1)^2 + (\ln n + 1)(n^3 + 1)$.

3.5.17 El algoritmo de “selección” es un método de ordenamiento sobre una lista de datos L . Se basa

en buscar el máximo de L e intercambiar este valor con el elemento ubicado en la última posición. Posteriormente, se aplica el mismo razonamiento sobre la lista L menos su último dato y se continúa así, hasta llegar a una lista de longitud uno. La instrucción **Selección** del paquete **VilCretas** ofrece una implementación. Encuentre para el peor de los casos una notación O grande.

Determine una notación Θ , o bien, O grande en la peor situación para los siguientes métodos. Represente gráficamente en *Mathematica*.

3.5.18 Programa[n_]:=Module[{p = 1}, For[i = 0, i <= n - 2, For[j = 1, j <= Floor[i/2], p = p + 1; j++]; i++]; Return[p]].

3.5.19 Programa[n_] :=Module[{p = 1, i = 1}, While[i <= 2n, p = p + 1; i = i + 2]; Return[p]].

3.5.20 Programa[n_] := Module[{p = 1, i = n}, While[i >= 1, p *= 2; i = Floor[i/5]]; Return[p]].

3.5.21 Programa[n_] := Module[{p = 1, i = n/2}, While[i >= 1, For[j = 1, j <= i, p = p * 2; j++]; i = i/Sqrt[5]]; Return[p]].

Conjeture para cuáles valores enteros positivos de j se satisface las notaciones asintóticas $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$ y $f(n) = \Theta(g(n))$.

3.5.22 $f(n) = \frac{2}{3}n^3 + (\ln n)^5$ y $g(n) = n^j$.

3.5.23 $f(n) = \frac{n^2 + 3 \ln n}{n^5 + 1}$ y $g(n) = n^j$.

3.5.24 $f(n) = n^{6j} - n^5 + 2n - 4$ y $g(n) = n^j$.

3.5.25 $f(n) = \frac{n^8}{n^j + 1}$ y $g(n) = (\ln(n))^j$.

3.5.26 $f(n) = \frac{n^{\frac{1}{j}} (\ln n)^{-5j}}{n^{\frac{1}{j}} + 1}$ y $g(n) = (\ln n)^{-5j}$.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Algoritmos/Exercises.zip>

CAPÍTULO 4

Relaciones binarias

4.5 Ejercicios

Se comparten a continuación un grupo de ejercicios de repaso y análisis con miras a robustecer el aprendizaje del alumno en las áreas de conocimiento asociadas con el tema de relaciones binarias.

4.5.1 Encuentre con ayuda de *Wolfram Mathematica* el producto cartesiano entre A y B , siendo $A = \{a \in \mathbb{N} \mid a \text{ un número par, } a \leq 500\}$ y $B = \{b \in \mathbb{N} \mid b \text{ un número impar, } b \leq 500\}$. ¿Cuál es la cardinalidad de $A \times B$?

4.5.2 Grafique $A \times B$ por medio del software *Wolfram Mathematica*, si $A = \{a \in \mathbb{R} \mid a \geq 5\}$ y $B = \{b \in \mathbb{R} \mid -10 \leq b \leq 6\}$.

4.5.3 Halle el dominio y el rango de la relación binaria R dada por: aRb sí y solo sí el mínimo común múltiplo entre a y b es igual a 300, con $a \in A = \{1, 3, \dots, 99\}$ y $b \in B = \{2, 4, \dots, 100\}$. ¿Cuál es el máximo valor del mínimo común múltiplo donde la relación R es distinta de vacío? Sugerencia: utilice el comando LCM de *Mathematica* que calcula el mínimo común múltiplo.

4.5.4 Represente en el plano cartesiano la relación binaria R : aRb siendo a y b dos números reales, sí y solo sí $\frac{a^2}{36} + \frac{b^2}{49} = 1$. ¿Cuál es el dominio de R ? ¿Cuál es el ámbito de R ? Determine si los pares ordenados de L pertenecen a la relación R con:

$$L = \left\{ \left(\sqrt{2}, \frac{7}{3} \sqrt{\frac{17}{2}} \right), \left(5\sqrt{2}, \frac{7}{3} \sqrt{\frac{29}{2}} \right), \left(2\sqrt{7}, \frac{7\sqrt{2}}{3} \right), \left(\sqrt{2}, \frac{7}{3} \sqrt{\frac{53}{2}} \right), \left(-3, \frac{7\sqrt{3}}{2} \right), \right. \\ \left. \left(\sqrt{3}, -\frac{7}{2} \sqrt{\frac{11}{3}} \right), \left(-3, \frac{7\sqrt{11}}{2} \right), \left(\sqrt{7}, -\frac{7\sqrt{29}}{6} \right), \left(\frac{1}{2}, -\frac{7\sqrt{143}}{12} \right), (0, -7) \right\}$$

4.5.5 Sea R_1 la relación binaria definida como: $aR_1b \Leftrightarrow a \leq b^2$, siendo $a, b \in A = \{2, 4, 6, \dots, 100\}$. Encuentre explícitamente los pares ordenados de R_1 . Determine usando la instrucción `ElementRelBinQ` si $6R_13$, $84R_196$, $24R_12$, $98R_110$ y $38R_16$.

Halle los pares ordenados que constituyen las siguientes relaciones binarias definidas sobre el conjunto A indicado.

4.5.6 $R_2 = \{(a, b) \mid a + b \leq 20\}$ con $A = \{2, 4, 6, \dots, 100\}$.

4.5.7 $aR_3b \Leftrightarrow a$ y b son números palíndromos con $A = \{11, 13, 17, 19, 21, 22, 23, 29, 32, 51, 72, 83, 89, 97, 113, 121, 127, 222, 312, 723\}$.

4.5.8 $aR_4b \Leftrightarrow a^2 - b^2$ es múltiplo de 5 con $A = \{1, 2, 3, 4, 5\}$.

4.5.9 $aR_5b \Leftrightarrow$ el residuo de la división $(a - 3) \div 3$ es igual al residuo de $(b - 3) \div 3$ donde $A = \{1, 2, 3, 4, 5\}$.

4.5.10 $R_6 = \{(a, b) \mid a - b \geq 3\}$ con $A = \{1, 2, 3, 4, 5\}$.

4.5.11 $aR_7b \Leftrightarrow |a^3 + b^2| = 3k$ con $k \in \mathbb{Z}$ donde $A = \{1, 2, 3, 4, 5\}$.

4.5.12 Grafique en el plano cartesiano las relaciones binarias $R_1, R_2, R_3, R_4, R_5, R_6$ y R_7 . Encuentre una matriz y un grafo que las represente.

Resuelva las operaciones señaladas a continuación, usando la definición y también matrices booleanas.

4.5.13 $\overline{R_1^{-1}}$ y $R_1 \circ R_1$.

4.5.14 $\overline{R_2 \circ R_2}$ y $(R_2 \circ R_2) \circ R_2$.

4.5.15 $R_3^{-1} \cup R_3$ y $R_3 \cap R_3^{-1}$.

4.5.16 $\overline{R_1^{-1} \cup R_2^{-1}}$ y $R_1^{-1} \circ R_2^{-1}$.

4.5.17 $(R_6 \cup R_5)^{-1}$ y $\overline{(R_4 \cap R_7)^{-1}}$.

4.5.18 $((\overline{R_5} \cap R_6^{-1}) \cup R_4) \circ \overline{R_7}$.

4.5.19 $\left(\left(\overline{R_7} \cap R_5^{-1} \right)^{-1} \circ \overline{R_6} \right) \cup R_4^{-1}$.

4.5.20 Clasifique como reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial, las relaciones R_1 , R_2 , R_3 , R_4 , R_5 , R_6 y R_7 . Justifique su respuesta.

4.5.21 Construya con ayuda del comando `SetPartitions`, todas las relaciones de equivalencia que se obtienen de las particiones de longitud 2 sobre el conjunto $A = \{1, 2, 3, 4, 5\}$.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Relaciones/Exercises.zip>

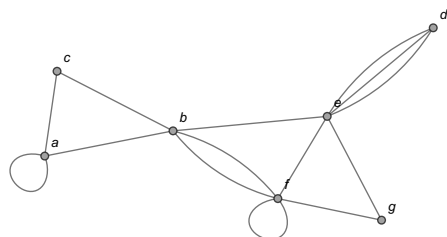
CAPÍTULO 5

Teoría de grafos

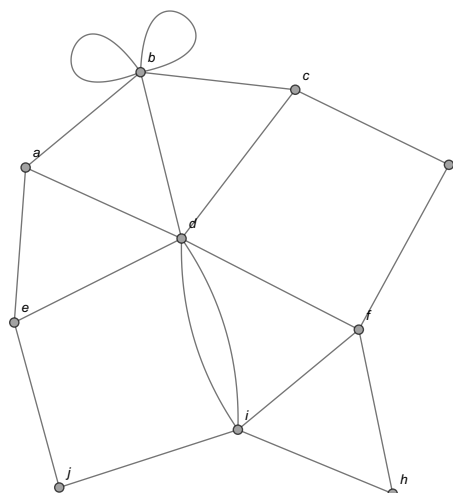
5.5 Ejercicios

Se sugiere al alumno resolver los ejercicios expuestos a continuación.

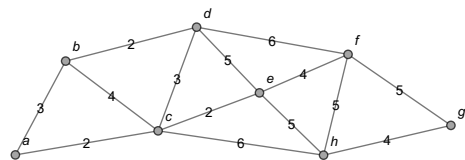
Considere los siguientes grafos y halle lo solicitado.



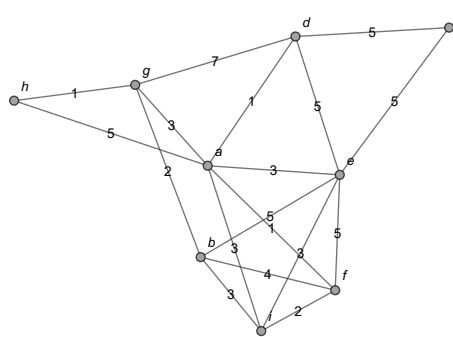
Grafo 1



Grafo 2



Grafo 3



Grafo 4

- 5.5.1 Un circuito de longitud mayor o igual a 10.
- 5.5.2 Un camino no circuital de longitud mayor o igual a 10.
- 5.5.3 Las valencias de cada vértice.
- 5.5.4 El número de aristas tomando como base los grados de los nodos.
- 5.5.5 Determine sobre los grafos 3 y 4, si existe un circuito o una trayectoria de *Euler*. Encuentre un camino de ese tipo utilizando el algoritmo de *Fleury*, en caso de existir.
- 5.5.6 Determine sobre los grafos 3 y 4, si hay condiciones suficientes para garantizar la existencia de un circuito de *Hamilton*. Halle un circuito hamiltoniano si éste es contenido por el grafo.

5.5.7 Construya una matriz de adyacencia y una matriz de incidencia asociadas a los grafos 1, 2, 3 y 4.

5.5.8 Construya una matriz de adyacencia de pesos para los grafos 3 y 4.

Represente los siguientes grafos en *Wolfram Mathematica*, dada una matriz de adyacencia de pesos. Asuma los vértices como letras consecutivas del abecedario iniciando en *a*.

5.5.9 Grafo 5 :
$$\begin{pmatrix} 0 & 1 & 0 & 3 & 5 \\ 1 & 0 & 0 & 4 & 1 \\ 0 & 0 & 0 & 3 & 1 \\ 3 & 4 & 3 & 0 & 2 \\ 5 & 1 & 1 & 2 & 0 \end{pmatrix}.$$

5.5.10 Grafo 6 :
$$\begin{pmatrix} 0 & 5 & 0 & 15 & 3 & 1 \\ 5 & 0 & 4 & 1 & 0 & 0 \\ 0 & 4 & 0 & 9 & 2 & 1 \\ 15 & 1 & 9 & 0 & 6 & 0 \\ 3 & 0 & 2 & 6 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}.$$

5.5.11 Grafo 7 :
$$\begin{pmatrix} 0 & 1 & 1 & 0 & 2 & 0 & 4 \\ 1 & 0 & 7 & 0 & 3 & 0 & 5 \\ 1 & 7 & 0 & 8 & 0 & 1 & 0 \\ 0 & 0 & 8 & 0 & 0 & 3 & 0 \\ 2 & 3 & 0 & 0 & 0 & 6 & 4 \\ 0 & 0 & 1 & 3 & 6 & 0 & 4 \\ 4 & 5 & 0 & 0 & 4 & 4 & 0 \end{pmatrix}.$$

5.5.12 Grafo 8 :
$$\begin{pmatrix} 0 & 5 & 2 & 3 & 1 & 0 & 0 \\ 5 & 0 & 3 & 1 & 0 & 1 & 0 \\ 2 & 3 & 0 & 0 & 2 & 2 & 0 \\ 3 & 1 & 0 & 0 & 1 & 0 & 8 \\ 1 & 0 & 2 & 1 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 8 & 1 & 3 & 0 \end{pmatrix}.$$

5.5.13 Grafo 9 :
$$\begin{pmatrix} 0 & 6 & 1 & 7 & 7 & 1 & 5 \\ 6 & 0 & 6 & 3 & 3 & 1 & 6 \\ 1 & 6 & 0 & 6 & 6 & 4 & 0 \\ 7 & 3 & 6 & 0 & 2 & 5 & 2 \\ 7 & 3 & 6 & 2 & 0 & 3 & 8 \\ 1 & 1 & 4 & 5 & 3 & 0 & 7 \\ 5 & 6 & 0 & 2 & 8 & 7 & 0 \end{pmatrix}.$$

5.5.14 Grafo 10 :
$$\begin{pmatrix} 0 & 4 & 8 & 5 & 4 & 7 & 3 \\ 4 & 0 & 0 & 1 & 2 & 0 & 5 \\ 8 & 0 & 0 & 2 & 8 & 2 & 2 \\ 5 & 1 & 2 & 0 & 1 & 2 & 5 \\ 4 & 2 & 8 & 1 & 0 & 5 & 8 \\ 7 & 0 & 2 & 2 & 5 & 0 & 0 \\ 3 & 5 & 2 & 5 & 8 & 0 & 0 \end{pmatrix}.$$

5.5.15 Grafo 11 :

$$\begin{pmatrix} 0 & 8 & 0 & 7 & 3 & 1 & 0 \\ 8 & 0 & 6 & 2 & 0 & 8 & 1 \\ 0 & 6 & 0 & 2 & 6 & 4 & 0 \\ 7 & 2 & 2 & 0 & 4 & 3 & 0 \\ 3 & 0 & 6 & 4 & 0 & 5 & 2 \\ 1 & 8 & 4 & 3 & 5 & 0 & 2 \\ 0 & 1 & 0 & 0 & 2 & 2 & 0 \end{pmatrix}.$$

5.5.16 Grafo 12 :

$$\begin{pmatrix} 0 & 3 & 3 & 9 & 9 & 5 & 2 & 0 \\ 3 & 0 & 1 & 1 & 0 & 6 & 0 & 5 \\ 3 & 1 & 0 & 9 & 10 & 0 & 2 & 5 \\ 9 & 1 & 9 & 0 & 4 & 2 & 2 & 4 \\ 9 & 0 & 10 & 4 & 0 & 8 & 9 & 3 \\ 5 & 6 & 0 & 2 & 8 & 0 & 9 & 7 \\ 2 & 0 & 2 & 2 & 9 & 9 & 0 & 3 \\ 0 & 5 & 5 & 4 & 3 & 7 & 3 & 0 \end{pmatrix}.$$

- 5.5.17** En los grafos 5, 6, 7, 8, 9, 10, 11 y 12 determine si existe una trayectoria o un circuito de *Euler*. Halle un camino de ese tipo si corresponde. Además, establezca si hay condiciones suficientes para la existencia de un circuito de *Hamilton* e indicar uno si el grafo lo contiene.
- 5.5.18** Aplique el algoritmo de *Dijkstra* para hallar la longitud de un camino más corto y una trayectoria de longitud mínima, sobre los grafos: 3 de a a h , 4 de a a h , 5 de a a c , 6 de a a d , 7 de a a d , 8 de c a g , 9 de b a g , 10 de b a g , 11 de c a e y 12 de b a h .
- 5.5.19** Encuentre la longitud de un camino más largo y una trayectoria de longitud máxima, sobre los grafos: 3 del vértice a al nodo g y 4 del nodo b al vértice f .
- 5.5.20** ¿Cuántos circuitos simples de longitud 3 hay en K_3 ?, ¿cuántos circuitos simples de longitud 4 hay en K_4 ?, ¿cuántos circuitos simples de longitud 5 hay en K_5 ? Conjeture con ayuda de *Mathematica* una fórmula que determine cuántos circuitos simples de longitud n hay en K_n .
- 5.5.21** Sea A una matriz de adyacencia de K_{20} . Compruebe que todos los elementos de la diagonal principal de A^k , son iguales entre sí, $1 \leq k \leq 10$, $k \in \mathbb{N}$. Conjeture con ayuda de *Mathematica* una fórmula que exprese el valor constante de la diagonal principal de A^k , $\forall k$, $k \in \mathbb{N}$.
- 5.5.22** Considere los grafos rueda, estrella, ciclo, camino y red presentados en la página 603. ¿Pueden contener circuitos de *Euler* o de *Hamilton*? ¿Qué condiciones deben satisfacer en caso afirmativo?

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Grafos/Exercises.zip>

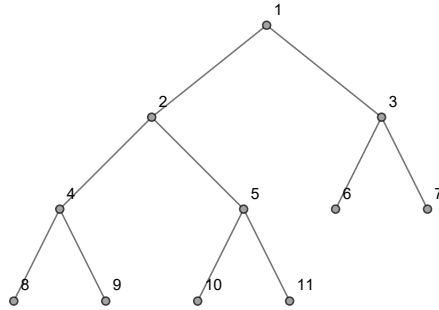
CAPÍTULO 6

Teoría de árboles

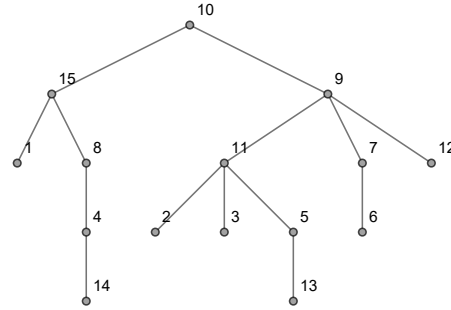
6.6 Ejercicios

Se insta al estudiante a resolver el conjunto de ejercicios propuestos.

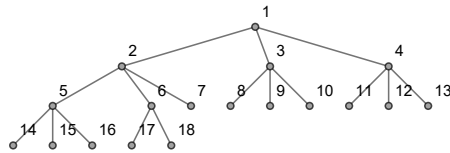
Considere los siguientes árboles:



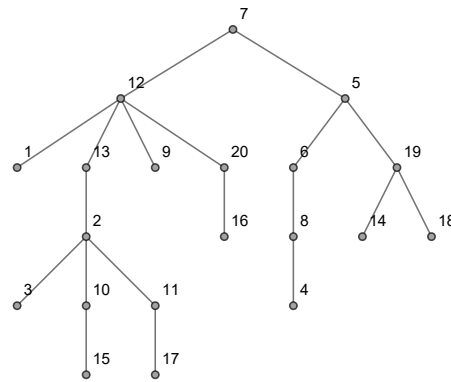
Árbol 1



Árbol 2



Árbol 3



Árbol 4

6.6.1 Justifique teóricamente que dichos grafos son árboles.

6.6.2 Halle su orden, los nodos terminales y su altura.

6.6.3 ¿Los árboles son completos? Explique.

6.6.4 ¿Los árboles son balanceados? Justifique.

6.6.5 Construya cada árbol en el software *Wolfram Mathematica*.

Determine un árbol binario de códigos de *Huffman* no optimizado y optimizado, para codificar las siguientes expresiones. Encuentre la codificación correspondiente.

6.6.6 ingeniería.

6.6.7 matemáticas.

6.6.8 laboratorio.

6.6.9 mississipis.

6.6.10 matematicadiscreta.

A través de un árbol binario de búsqueda almacene la información indicada y realice un recorrido prefijo, interfijo y postfijo.

6.6.11 $Datos = \{85, 31, 94, 47, 91, 35, 87, 48, 44, 99, 19, 95, 2, 22, 18, 90, 56, 7, 43, 39\}$.

6.6.12 $Datos = \{77, 14, 25, 86, 22, 81, 94, 65, 74, 41, 87, 35, 67, 85, 47, 42, 8, 2, 59, 92, 29, 44, 75, 64\}$.

6.6.13 $Datos = \{j, k, w, i, m, l, n, q, p, b, z, u, c, v, d, r, a, e, s\}$.

6.6.14 La frase de *Isaac Newton*: “en matemáticas no se deben despreciar ni los errores más diminutos”.

6.6.15 La frase de *Leonardo Da Vinci*: “ninguna investigación humana puede ser denominada ciencia si no pasa a través de pruebas matemáticas”.

Encuentre notaciones polaca y polaca inversa para las siguientes expresiones algebraicas:

6.6.16 $((a - c) \cdot d) \div (a + (b - c))$.

6.6.17 $(a \cdot b + c \cdot d) - (a \div b - d - e)$.

6.6.18 $a \cdot b + (c + d) + a \div b - f \cdot (d - e)$.

6.6.19 $((a + b) \cdot c + f) \cdot e - d \cdot ((a + c) - f + d \div e)$.

6.6.20 $a \cdot (b - c \div d + e) - g \cdot (f \cdot (a + b) \div (c + d \cdot e)) + a \cdot b$.

Considere los grafos 3, 4, 5, 6, 7, 8, 9, 10, 11 y 12 presentados en la sección de ejercicios del capítulo 5 (página 12).

6.6.21 Halle un árbol generador a lo “ancho” y a lo “largo”. Suponga el orden del abecedario.

6.6.22 Determine un árbol de expansión minimal, a través de los algoritmos de *Prim* y de *Kruskal*. Asuma el orden del abecedario.

Resuelva lo indicado a continuación, sobre los grafos 3 y 4 de la sección de ejercicios del capítulo 5, utilizando el orden $V = \{h, b, d, g, f, e, a, c\}$ y $V = \{i, g, c, b, f, a, e, h, d\}$, respectivamente.

6.6.23 Un árbol generador a lo “ancho” y a lo “largo”.

6.6.24 Un árbol de expansión mínima, por medio del algoritmo de *Prim*.

6.6.25 Un árbol generador de longitud máxima, recurriendo a los algoritmos de *Prim* y de *Kruskal*.

Resuelva lo solicitado.

6.6.26 Determine un árbol de expansión a lo “ancho” y a lo “largo” en los grafos: completo, rueda, estrella, ciclo y camino, de tamaño n , $2 \leq n \leq 10$, $n \in \mathbb{Z}$, usando el orden ascendente del conjunto de los números naturales, ¿es generalizable algún resultado?

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Arboles/Exercises.zip>

CAPÍTULO 7

Máquinas de estado finito y autómatas

7.5 Ejercicios

Se invita al alumno a fortalecer lo aprendido en el capítulo mediante los siguientes ejercicios.

Realice el diagrama de transición de las siguientes máquinas de estado finito:

7.5.1 $M_1 = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$, con $\sigma = \{\sigma_0, \sigma_1, \sigma_2\}$, $\tau = \{a, b\}$, $\delta = \{0, 1, 2\}$, $\sigma^* = \sigma_0$ y:

	Δ		Ω	
	a	b	a	b
σ_0	σ_1	σ_2	1	0
σ_1	σ_2	σ_1	2	1
σ_2	σ_0	σ_1	2	0

7.5.2 $M_2 = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$, con $\sigma = \{\sigma_0, \sigma_1, \sigma_2\}$, $\tau = \{a, b, c\}$, $\delta = \{0, 1\}$, $\sigma^* = \sigma_2$ y:

	Δ			Ω		
	a	b	c	a	b	c
σ_0	σ_1	σ_2	σ_0	1	0	1
σ_1	σ_0	σ_1	σ_2	0	1	1
σ_2	σ_1	σ_0	σ_2	0	1	0

7.5.3 $M_3 = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$, con $\sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b, c\}$, $\delta = \{0, 1\}$, $\sigma^* = \sigma_3$ y:

	Δ			Ω		
	a	b	c	a	b	c
σ_0	σ_3	σ_1	σ_0	0	1	1
σ_1	σ_3	σ_1	σ_3	1	0	0
σ_2	σ_0	σ_1	σ_2	0	1	0
σ_3	σ_3	σ_3	σ_0	1	0	1

7.5.4 $M_4 = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$, con $\sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b, c\}$, $\delta = \{0, 1\}$, $\sigma^* = \sigma_1$ y:

	Δ			Ω		
	a	b	c	a	b	c
σ_0	σ_3	σ_3	σ_1	0	0	1
σ_1	σ_0	σ_1	σ_2	0	1	1
σ_2	σ_2	σ_0	σ_3	1	0	0
σ_3	σ_0	σ_2	σ_3	0	1	0

7.5.5 $M_5 = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$, con $\sigma = \{\phi, \{\sigma_0\}, \{\sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1\}, \{\sigma_0, \sigma_2\}, \{\sigma_1, \sigma_2\}\}$,

$\{\sigma_0, \sigma_1, \sigma_2\}$, $\tau = \{a, b\}$, $\delta = \{0, 1\}$, $\sigma^* = \{\sigma_0, \sigma_1, \sigma_2\}$ y:

	Δ		Ω	
	a	b	a	b
ϕ	$\{\sigma_0, \sigma_1\}$	$\{\sigma_0, \sigma_1, \sigma_2\}$	1	0
$\{\sigma_0\}$	ϕ	$\{\sigma_0\}$	1	0
$\{\sigma_1\}$	$\{\sigma_0, \sigma_1\}$	$\{\sigma_0, \sigma_2\}$	0	1
$\{\sigma_2\}$	$\{\sigma_0, \sigma_1, \sigma_2\}$	$\{\sigma_0, \sigma_2\}$	0	1
$\{\sigma_0, \sigma_1\}$	$\{\sigma_1, \sigma_2\}$	$\{\sigma_2\}$	1	0
$\{\sigma_0, \sigma_2\}$	$\{\sigma_1\}$	$\{\sigma_0, \sigma_1\}$	0	1
$\{\sigma_1, \sigma_2\}$	ϕ	ϕ	0	1
$\{\sigma_0, \sigma_1, \sigma_2\}$	$\{\sigma_0\}$	$\{\sigma_0, \sigma_1, \sigma_2\}$	1	1

Procese las hileras α de símbolos de entrada dadas, en las máquinas de estado finito M_1 , M_2 , M_3 , M_4 y M_5 . Muestre el recorrido correspondiente.

7.5.6 $\alpha = baaaaabbbbabaaababa$.

7.5.7 $\alpha = aaababbbaabaaaababaa$.

7.5.8 $\alpha = abaababaaabababaabba$.

7.5.9 $\alpha = bbbacbcbbaccbbabbbb$, exceptuando M_1 y M_5 .

7.5.10 $\alpha = cbccaabcbacbbbaabccaa$, exceptuando M_1 y M_5 .

7.5.11 Determine si las máquinas M_1 , M_2 , M_3 , M_4 y M_5 son autómatas de estado finito determinísticos, justificando detalladamente la respuesta. En caso de no serlo, modifique la máquina con el objetivo de transformarla en un autómata.

7.5.12 Diseñe una máquina de estado finito que resuelva la resta no negativa entre dos números binarios. La tabla de reglas básicas para llevar a cabo la resta binaria entre dos *bits*, corresponde a:

$0 - 0 = 0$
$1 - 0 = 1$
$1 - 1 = 0$
$0 - 1 = 1$ y se lleva un 1

Utilice la máquina para efectuar la operación: $(11011101)_2 - (1110110)_2$. En la máquina elaborada, cuando se restan dos números binarios, ¿existe la posibilidad de finalizar en cualquier estado? Explique.

Elabore los diagramas de transición de los autómatas de estado finito determinísticos siguientes:

7.5.13 $A_1 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b, c\}$, $\sigma^* = \sigma_1$, $\hat{A} = \{\sigma_0\}$ y:

Δ	a	b	c
σ_0	σ_0	σ_0	σ_0
σ_1	σ_0	σ_1	σ_3
σ_2	σ_3	σ_3	σ_1
σ_3	σ_1	σ_2	σ_0

7.5.14 $A_2 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}, \tau = \{a, b, c\}, \sigma^* = \sigma_0, \hat{A} = \{\sigma_1, \sigma_3\}$ y:

Δ	a	b	c
σ_0	σ_1	σ_2	σ_3
σ_1	σ_0	σ_0	σ_1
σ_2	σ_3	σ_0	σ_1
σ_3	σ_2	σ_2	σ_3

7.5.15 $A_3 : \sigma = \{\sigma_0, \sigma_1, \sigma_2\}, \tau = \{a, b, c, d\}, \sigma^* = \sigma_2, \hat{A} = \{\sigma_1, \sigma_2\}$ y:

Δ	a	b	c	d
σ_0	σ_2	σ_0	σ_1	σ_2
σ_1	σ_0	σ_1	σ_0	σ_2
σ_2	σ_0	σ_1	σ_0	σ_0

7.5.16 $A_4 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}, \tau = \{a, b, c, d\}, \sigma^* = \sigma_3, \hat{A} = \{\sigma_0, \sigma_1, \sigma_2\}$ y:

Δ	a	b	c	d
σ_0	σ_2	σ_1	σ_2	σ_3
σ_1	σ_1	σ_2	σ_3	σ_0
σ_2	σ_3	σ_0	σ_3	σ_3
σ_3	σ_0	σ_2	σ_1	σ_2

7.5.17 $A_5 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}, \tau = \{a, b, c, d\}, \sigma^* = \sigma_1, \hat{A} = \{\sigma_1, \sigma_2, \sigma_3\}$ y:

Δ	a	b	c	d
σ_0	σ_2	σ_0	σ_2	σ_3
σ_1	σ_1	σ_3	σ_3	σ_0
σ_2	σ_2	σ_1	σ_2	σ_1
σ_3	σ_3	σ_0	σ_2	σ_2

Determine si las hileras α de símbolos de entrada dadas, son aceptadas en los autómatas de estado finito A_1, A_2, A_3, A_4 y A_5 . Muestre el recorrido correspondiente.

7.5.18 $\alpha = bbabbbaaccbccbacbbba$.

7.5.19 $\alpha = aaccbaabccacaaacaabb$.

7.5.20 $\alpha = ccbababaccaccabbddcc$, exceptuando A_1 y A_2 .

7.5.21 $\alpha = cbcaccaabccacdcaadacc$, exceptuando A_1 y A_2 .

7.5.22 $\alpha = bcdccdbbcbadacaddbd$, exceptuando A_1 y A_2 .

7.5.23 Conjeture al menos dos tipos de hileras generales del lenguaje vinculado a los autómatas A_1, A_2, A_3, A_4 y A_5 .

Trace el diagrama de transición de un autómata de estado finito determinístico que acepte el conjunto de hileras de símbolos de entrada señalado. Realice distintas pruebas con el software *Wolfram Mathematica* para verificar los resultados. Retorne, además, las cinco componentes de cada autómata.

7.5.24 Cadenas que posean un número impar de caracteres b , mayor o igual a 3, con $\tau = \{a, b\}$.

7.5.25 Hileras que empiezan con bab y terminan en aba .

7.5.26 Arreglos que contengan exactamente tres a , una b y ninguna c , suponiendo $\tau = \{a, b, c\}$.

7.5.27 Hileras que contengan ab y no finalicen en cb .

7.5.28 Cadenas de caracteres que no inician con ab y no finalizan en abc .

Construya el diagrama de transición de los autómatas de estado finito no determinísticos siguientes:

7.5.29 $A_6 : \sigma = \{\sigma_0, \sigma_1, \sigma_2\}$, $\tau = \{a, b, c\}$, $\sigma^* = \sigma_1$, $\hat{A} = \{\sigma_0, \sigma_1\}$ y:

Δ	a	b	c
σ_0	$\{\sigma_0, \sigma_2\}$	ϕ	$\{\sigma_2\}$
σ_1	$\{\sigma_1\}$	$\{\sigma_0, \sigma_1\}$	σ
σ_2	$\{\sigma_0, \sigma_1\}$	$\{\sigma_0, \sigma_2\}$	$\{\sigma_0\}$

7.5.30 $A_7 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b\}$, $\sigma^* = \sigma_3$, $\hat{A} = \{\sigma_2, \sigma_3\}$ y:

Δ	a	b
σ_0	$\{\sigma_3\}$	ϕ
σ_1	$\{\sigma_0, \sigma_1\}$	$\{\sigma_1, \sigma_3\}$
σ_2	σ	$\{\sigma_1, \sigma_2\}$
σ_3	$\{\sigma_0\}$	$\{\sigma_0, \sigma_2\}$

7.5.31 $A_8 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b\}$, $\sigma^* = \sigma_0$, $\hat{A} = \{\sigma_0, \sigma_2, \sigma_3\}$ y:

Δ	a	b
σ_0	ϕ	$\{\sigma_1, \sigma_2, \sigma_3\}$
σ_1	$\{\sigma_0, \sigma_3\}$	$\{\sigma_0, \sigma_2\}$
σ_2	$\{\sigma_3\}$	$\{\sigma_0, \sigma_1, \sigma_3\}$
σ_3	σ	$\{\sigma_1, \sigma_2\}$

7.5.32 $A_9 : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b\}$, $\sigma^* = \sigma_2$, $\hat{A} = \{\sigma_1, \sigma_2, \sigma_3\}$ y:

Δ	a	b
σ_0	$\{\sigma_1, \sigma_2, \sigma_3\}$	σ
σ_1	σ	$\{\sigma_0, \sigma_2, \sigma_3\}$
σ_2	$\{\sigma_0, \sigma_2, \sigma_3\}$	σ
σ_3	σ	$\{\sigma_0, \sigma_1, \sigma_3\}$

7.5.33 $A_{10} : \sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, $\tau = \{a, b, c\}$, $\sigma^* = \sigma_3$, $\hat{A} = \{\sigma_0, \sigma_1, \sigma_3\}$ y:

Δ	a	b	c
σ_0	$\{\sigma_0, \sigma_1, \sigma_2\}$	$\{\sigma_0, \sigma_3\}$	$\{\sigma_0, \sigma_1, \sigma_3\}$
σ_1	$\{\sigma_1, \sigma_2, \sigma_3\}$	$\{\sigma_0, \sigma_2\}$	ϕ
σ_2	ϕ	σ	$\{\sigma_0, \sigma_2, \sigma_3\}$
σ_3	$\{\sigma_1, \sigma_2\}$	$\{\sigma_0, \sigma_1, \sigma_2\}$	σ

Establezca si las hileras α de símbolos de entrada dadas, son aceptadas en los autómatas de estado finito no determinísticos A_6 , A_7 , A_8 , A_9 y A_{10} . Muestre un recorrido en caso de que la cadena de caracteres sea aceptada.

7.5.34 $\alpha = baaaabbbbbaabbaa$.

7.5.35 $\alpha = abaababbbbbaaaaaa$.

7.5.36 $\alpha = aabbbbbabbabaa$.

7.5.37 $\alpha = baabbaaabbabaab$.

7.5.38 $\alpha = abbcbbccccbcaaaa$, considerando solo A_6 y A_{10} .

7.5.39 Encuentre por medio del teorema 7.1, un autómata de estado finito determinístico equivalente para A_6 , A_7 , A_8 , A_9 y A_{10} . Elabore el diagrama de transición reducido.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Maquinas/Exercises.zip>

CAPÍTULO 8

Lenguajes y gramáticas

8.4 Ejercicios

Se insta al estudiante a resolver el conjunto de ejercicios propuestos con la intención de ratificar el nivel de aprovechamiento adquirido.

Determine por definición (realizando derivaciones) y justificando paso a paso, al menos dos hileras generales que pertenecen al lenguaje $L(G_j)$, $1 \leq j \leq 12$, $j \in \mathbb{N}$, con relación a las gramáticas dadas.

8.4.1 $G_1 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|-----------------------|-----------------------|
| 1. $S \rightarrow bS$ | 4. $A \rightarrow bA$ |
| 2. $S \rightarrow aA$ | 5. $A \rightarrow a$ |
| 3. $A \rightarrow aS$ | 6. $S \rightarrow b$ |

- $\sigma^* = S$.

8.4.2 $G_2 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B\}$.
- Símbolos terminales: $\tau = \{a, b, c\}$.
- Producciones o reglas de composición: $P =$

- | | |
|------------------------|-----------------------|
| 1. $S \rightarrow aAc$ | 4. $A \rightarrow B$ |
| 2. $A \rightarrow aA$ | 5. $B \rightarrow b$ |
| 3. $A \rightarrow Ac$ | 6. $B \rightarrow Bb$ |

- $\sigma^* = S$.

8.4.3 $G_3 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|------------------------|------------------------|
| 1. $S \rightarrow A$ | 5. $B \rightarrow ABb$ |
| 2. $A \rightarrow a$ | 6. $A \rightarrow ABB$ |
| 3. $B \rightarrow b$ | 7. $AB \rightarrow BA$ |
| 4. $S \rightarrow ABA$ | |

- $\sigma^* = S$.

8.4.4 $G_4 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B\}$.
- Símbolos terminales: $\tau = \{a, b\}$.

- Producciones o reglas de composición: $P =$

- | | |
|-------------------------|-------------------------|
| 1. $S \rightarrow A$ | 5. $Bb \rightarrow ABb$ |
| 2. $S \rightarrow AAB$ | 6. $AB \rightarrow ABB$ |
| 3. $Ab \rightarrow ABa$ | 7. $B \rightarrow b$ |
| 4. $A \rightarrow aa$ | |

- $\sigma^* = S$.

8.4.5 $G_5 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|------------------------|-----------------------|
| 1. $S \rightarrow BAB$ | 5. $A \rightarrow aA$ |
| 2. $S \rightarrow ABA$ | 6. $A \rightarrow ab$ |
| 3. $A \rightarrow AB$ | 7. $B \rightarrow b$ |
| 4. $B \rightarrow BA$ | |

- $\sigma^* = S$.

8.4.6 $G_6 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|------------------------|-----------------------|
| 1. $A \rightarrow bB$ | 5. $S \rightarrow bB$ |
| 2. $B \rightarrow b$ | 6. $C \rightarrow aC$ |
| 3. $B \rightarrow bCC$ | 7. $C \rightarrow aB$ |
| 4. $S \rightarrow aA$ | |

- $\sigma^* = S$.

8.4.7 $G_7 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B\}$.
- Símbolos terminales: $\tau = \{a, b, c\}$.
- Producciones o reglas de composición: $P =$

- | | |
|-----------------------|-----------------------|
| 1. $S \rightarrow aS$ | 6. $B \rightarrow bB$ |
| 2. $S \rightarrow a$ | 7. $B \rightarrow b$ |
| 3. $S \rightarrow bA$ | 8. $B \rightarrow aA$ |
| 4. $A \rightarrow cB$ | 9. $B \rightarrow a$ |
| 5. $A \rightarrow cS$ | |

- $\sigma^* = S$.

8.4.8 $G_8 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C\}$.

- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|-----------------------|-----------------------|
| 1. $A \rightarrow aB$ | 6. $C \rightarrow aS$ |
| 2. $A \rightarrow bA$ | 7. $C \rightarrow bA$ |
| 3. $B \rightarrow a$ | 8. $S \rightarrow aS$ |
| 4. $B \rightarrow aC$ | 9. $S \rightarrow bA$ |
| 5. $B \rightarrow bA$ | |

- $\sigma^* = S$.

8.4.9 $G_9 = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|-----------------------|-----------------------|
| 1. $S \rightarrow aS$ | 6. $B \rightarrow bC$ |
| 2. $S \rightarrow bA$ | 7. $B \rightarrow b$ |
| 3. $A \rightarrow bS$ | 8. $C \rightarrow aB$ |
| 4. $A \rightarrow aB$ | 9. $C \rightarrow bS$ |
| 5. $B \rightarrow aB$ | |

- $\sigma^* = S$.

8.4.10 $G_{10} = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | |
|-----------------------|------------------------|
| 1. $S \rightarrow aS$ | 6. $A \rightarrow aC$ |
| 2. $S \rightarrow aA$ | 7. $B \rightarrow aS$ |
| 3. $S \rightarrow bC$ | 8. $B \rightarrow a$ |
| 4. $S \rightarrow a$ | 9. $C \rightarrow aB$ |
| 5. $A \rightarrow bA$ | 10. $C \rightarrow aC$ |

- $\sigma^* = S$.

8.4.11 $G_{11} = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | | |
|-----------------------|------------------------|------------------------|
| 1. $S \rightarrow aA$ | 6. $B \rightarrow aB$ | 11. $C \rightarrow aS$ |
| 2. $S \rightarrow aB$ | 7. $B \rightarrow bA$ | 12. $C \rightarrow bA$ |
| 3. $A \rightarrow bS$ | 8. $B \rightarrow aA$ | 13. $C \rightarrow aC$ |
| 4. $A \rightarrow b$ | 9. $B \rightarrow b$ | 14. $C \rightarrow a$ |
| 5. $A \rightarrow a$ | 10. $B \rightarrow aC$ | |

- $\sigma^* = S$.

8.4.12 $G_{12} = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\sigma = \{S, A, B, C, D, E, F, G\}$.
- Símbolos terminales: $\tau = \{a, b\}$.
- Producciones o reglas de composición: $P =$

- | | | | |
|-----------------------|------------------------|------------------------|------------------------|
| 1. $A \rightarrow aG$ | 6. $C \rightarrow aG$ | 11. $E \rightarrow b$ | 16. $G \rightarrow a$ |
| 2. $A \rightarrow bB$ | 7. $C \rightarrow bG$ | 12. $E \rightarrow bC$ | 17. $G \rightarrow bG$ |
| 3. $B \rightarrow aG$ | 8. $D \rightarrow aE$ | 13. $F \rightarrow a$ | 18. $S \rightarrow aA$ |
| 4. $B \rightarrow b$ | 9. $D \rightarrow bF$ | 14. $F \rightarrow aC$ | 19. $S \rightarrow bD$ |
| 5. $B \rightarrow bC$ | 10. $E \rightarrow aG$ | 15. $F \rightarrow bG$ | |

- $\sigma^* = S$.

8.4.13 Clasifique las gramáticas G_j , $1 \leq j \leq 12$, $j \in \mathbb{N}$, en sensibles al contexto, libres del contexto, regulares o ninguna de ellas. Explique.

8.4.14 Construya una gramática G libre del contexto en notación **BNF**, cuyo lenguaje sea el conjunto de los números enteros.

8.4.15 Diseñe una gramática G libre del contexto en notación **BNF**, que tenga como lenguaje los enteros negativos que no empiecen con 0.

8.4.16 Construya una gramática G libre del contexto en notación **BNF**, donde los elementos del lenguaje sean únicamente las siguientes frases u oraciones:

Adriana analiza algoritmos constantemente.
 Adriana analiza algoritmos rápidamente.
 Adriana construye algoritmos constantemente.
 Adriana construye algoritmos rápidamente.
 José analiza algoritmos constantemente.
 José analiza algoritmos rápidamente.
 José construye algoritmos constantemente.
 José construye algoritmos rápidamente.

8.4.17 Encuentre un autómata de estado finito no determinístico equivalente a las gramáticas G_1 , G_7 , G_8 , G_9 , G_{10} , G_{11} y G_{12} . Además, usando el autómata verifique la correctitud de las hileras generales ya halladas (en un ejercicio anterior), que pertenecen a su lenguaje.

Diseñe una gramática regular $G = (\sigma, \tau, P, \sigma^*)$ que posea como lenguaje el conjunto $L(G)$ indicado.

8.4.18 $L(G) = \{\text{hileras que finalizan en } abc\}$.

8.4.19 $L(G) = \{\text{hileras que inician con } ab \text{ y terminan con } ba\}$.

8.4.20 $L(G) = \{\text{hileras con un número par de letras } b \text{ y con un número impar de letras } a\}$.

8.4.21 $L(G) = \{\text{hileras que no inician con } ba \text{ y no terminan con } abb\}$.

8.4.22 $L(G) = \{\text{hileras con exactamente una } a, \text{ una } b \text{ y una } c\}$.

Descargue un archivo: Solución de los ejercicios propuestos.

<https://www.escinf.una.ac.cr/discretas/Archivos/Lenguajes/Exercises.zip>

SEGUNDA PARTE

Las respuestas

RESPUESTAS • SECCIÓN 1.5

Recursividad

1.5.1 La pila de llamadas nunca finaliza al no obtenerse el caso raíz.

1.5.2 $\text{Suma}[n_]:= \text{If}[n == 6, 1/2, \text{Suma}[n - 1] + (n - 5)/(n - 4)]$.

1.5.3 $\text{Suma}[n_]:= \text{If}[n == 9, 49/262144, \text{Suma}[n - 1] + ((n + 5)/2^{(n + 1)})^2]$.

1.5.4 $\text{Producto}[n_]:= \text{If}[n == 11, 2, \text{Producto}[n - 1] * (3n - 31)]$.

1.5.5 $\text{Producto}[n_]:= \text{If}[n == 5, 2187, \text{Producto}[n - 1] * (3^{(n + 4)} / (n + 4))]$.

1.5.6 $\text{SumaProducto}[n_]:= \text{Module}[\{\text{Producto}\}, \text{Producto}[i_]:= \text{If}[i == 1 \parallel i == 2, 1, \text{If}[i == 3, -1, \text{Producto}[i - 1] (2i - 7)]]];$
 $\text{If}[n == 0, 1, \text{SumaProducto}[n - 1] + \text{Producto}[n + 1] (n + 1)]$.

1.5.7

1.5.2 $\text{SumaCola}[n_]:= \text{Module}[\{\text{SC}\}, \text{SC}[m_, \text{Cont_} : 6, \text{Suma_}:=$
 $\text{If}[m == 6, 1/2, \text{If}[\text{Cont} == m, \text{Suma}, \text{SC}[m, \text{Cont} + 1, \text{Suma} +$
 $(\text{Cont} - 4)/(\text{Cont} - 3)]]]]; \text{SC}[n, 1/2]]$, o bien, $\text{SumaCola}[n_,$
 $\text{Suma_} : 0]:= \text{If}[n == 6, \text{Suma} + 1/2, \text{SumaCola}[n - 1, \text{Suma} +$
 $(n - 5)/(n - 4)]]$.

1.5.3 $\text{SumaCola}[n_]:= \text{Module}[\{\text{SC}\}, \text{SC}[m_, \text{Cont_} : 9, \text{Suma_}:=$
 $\text{If}[m == 9, 49/262144, \text{If}[\text{Cont} == m, \text{Suma}, \text{SC}[m, \text{Cont} + 1, \text{Suma}$
 $+ 2^{(8 - 2(6 + \text{Cont}))} (6 + \text{Cont})^2]]]]; \text{SC}[n, 49/262144]]$, o bien,
 $\text{SumaCola}[n_, \text{Suma_} : 0]:= \text{If}[n == 9, \text{Suma} + 49/262144, \text{SumaCola}[n - 1, \text{Suma} +$
 $2^{(8 - 2(5 + n))} (5 + n)^2]]$.

1.5.4 $\text{ProductoCola}[n_]:= \text{Module}[\{\text{PC}\}, \text{PC}[m_, \text{Cont_} : 11, \text{Producto_}]$
 $:= \text{If}[m == 11, 2, \text{If}[\text{Cont} == m, \text{Producto}, \text{PC}[m, \text{Cont} + 1,$
 $\text{Producto} * (3 \text{Cont} - 28)]]]]; \text{PC}[n, 2]]$, o bien, $\text{ProductoCola}[n_,$
 $\text{Producto_} : 1]:= \text{If}[n == 11, \text{Producto} * 2, \text{ProductoCola}[n - 1,$
 $\text{Producto} * (3n - 31)]]$.

1.5.5 $\text{ProductoCola}[n_]:= \text{Module}[\{\text{PC}\}, \text{PC}[m_, \text{Cont_} : 5, \text{Producto_}]$
 $:= \text{If}[m == 5, 2187, \text{If}[\text{Cont} == m, \text{Producto}, \text{PC}[m, \text{Cont} + 1,$
 $\text{Producto} * (3^{(\text{Cont} + 5)} / (\text{Cont} + 5))]]]]; \text{PC}[n, 2187]]$, o bien,
 $\text{ProductoCola}[n_, \text{Producto_} : 1]:= \text{If}[n == 5, \text{Producto} * 2187,$
 $\text{ProductoCola}[n - 1, \text{Producto} * (3^{(n + 4)} / (n + 4))]]$.

1.5.6 $\text{SumaProductoCola}[n_, \text{Suma_} : 0]:= \text{Module}[\{\text{ProductoCola}\},$
 $\text{ProductoCola}[i_, \text{Producto_} : 1]:= \text{If}[i == 1 \parallel i == 2, 1,$
 $\text{If}[i == 3, -\text{Producto}, \text{ProductoCola}[i - 1, \text{Producto} * (2i - 7)]]]]; \text{If}[n == 0,$
 $1 + \text{Suma}, \text{SumaProductoCola}[n - 1, \text{Suma} +$
 $\text{ProductoCola}[n + 1] (n + 1)]]]$.

1.5.8 $\text{RecurrenciaPila}[n_]:= \text{If}[n == 5, 2, 5 \text{RecurrenciaPila}[n - 1] + 2n - \text{Sqrt}[2]]$
y $\text{RecurrenciaCola}[n_]:=$

```
Module[{RecurrenciaAuxiliar}, RecurrenciaAuxiliar[m_, Cont_ : 5, a_] := If[m
== 5, 2, If[Cont == m, a, RecurrenciaAuxiliar[m,
Cont + 1, 5 a + 2 (Cont + 1) - Sqrt[2]]]];
RecurrenciaAuxiliar[n, 2]].
```

1.5.9 RecurrenciaPila[n_] := If[n == 1, -10, If[n == 2, -19, RecurrenciaPila[n - 1] + 6 RecurrenciaPila[n - 2]]] y

```
RecurrenciaCola[n_] := Module[{RecurrenciaAuxiliar},
RecurrenciaAuxiliar[m_, Cont_ : 2, a1_, a2_] := If[m == 1, -10,
If[m == 2, -19, If[Cont == m, a2, RecurrenciaAuxiliar[m,
Cont + 1, a2, a2 + 6 a1]]]]; RecurrenciaAuxiliar[n, -10, -19]].
```

1.5.10 RecurrenciaPila[n_] := If[n == 1, 1, If[n == 2, -1, If[n == 3, Power[2, (5)^-1], 12 RecurrenciaPila[n - 1] - 36

```
RecurrenciaPila[n - 2] + RecurrenciaPila[n - 3]]] y
RecurrenciaCola[n_] := Module[{RecurrenciaAuxiliar},
RecurrenciaAuxiliar[m_, Cont_ : 3, a1_, a2_, a3_] := If[m == 1, 1, If[m == 2,
-1, If[m == 3, Power[2, (5)^-1], If[Cont == m, a3,
RecurrenciaAuxiliar[m, Cont + 1, a2, a3, 12 a3 - 36 a2 +
a1]]]]]; RecurrenciaAuxiliar[n, 1, -1, Power[2, (5)^-1]]].
```

1.5.11 RecurrenciaPila[n_] := If[n == 2, 1, If[n == 3, 2, If[n == 4, 3, If[n == 5, 4, 1/9 (6 RecurrenciaPila[n - 1] -

```
RecurrenciaPila[n - 2] + RecurrenciaPila[n - 3] - 2
RecurrenciaPila[n - 4])]]]] y RecurrenciaCola[n_] :=
Module[{RecurrenciaAuxiliar}, RecurrenciaAuxiliar[m_, Cont_ : 5, a2_, a3_,
a4_, a5_] := If[m == 2, 1, If[m == 3, 2, If[m == 4, 3,
If[m == 5, 4, If[Cont == m, a5, RecurrenciaAuxiliar[m, Cont + 1, a3, a4, a5,
1/9 (6 a5 - a4 + a3 - 2 a2)]]]]]]];
RecurrenciaAuxiliar[n, 1, 2, 3, 4]].
```

1.5.12 Ahorro[n_, ahorro_ : 1] := If[n == 0, ahorro*200, Ahorro[n - 1, 1.05 ahorro]], 359171.

1.5.13 catalan[n_] := Module[{CatalanCola}, CatalanCola[m_, Cont_ : 0, Cn_] := If[m == 0, 1, If[m == Cont, Cn, CatalanCola[m, Cont + 1, (2 (2 Cont + 1))/(Cont + 2) Cn]]]; CatalanCola[n, 1]]

In[] :=

```
Table[catalan[i], {i, 0, 29}]
```

Out[] =

```
{1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440,
9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020,
91482563640, 343059613650, 1289904147324, 4861946401452, 18367353072152,
69533550916004, 263747951750360, 1002242216651368}.
```

1.5.14 mcd[n_, m_] := If[m == 0, n, If[n == 0, m, If[n >= m, mcd[n - m, m], mcd[n, m - n]]] y mcdPasos[n_, m_] := If[m == 0, n,


```
If[n == 0, m, If[n >= m, Print["mcd[" , n - m, ", ", m, "]" ]];
mcdPasos[n - m, m], Print["mcd[" , n, ", ", m - n, "]" ]];
mcdPasos[n, m - n]]]]
```

In[] :=

mcdPasos[120, 144]

Out[] =

mcd[120,24]

mcd[96,24]

mcd[72,24]

mcd[48,24]

mcd[24,24]

mcd[0,24]

24.

1.5.15 Ackermann[m_, n_] := If[m == 0, n + 1, If[m > 0 && n == 0, Ackermann[m - 1, 1], If[m > 0 && n > 0, Ackermann[m - 1, Ackermann[m, n - 1]]]], las fórmulas serían “ $n+2$ ”, “ $2n+3$ ” y “ $8 \cdot 2^n - 3$ ”, respectivamente.

1.5.16 SumaDigiCola[n_] := Module[{SDCAux}, SDCAux[m_, digito_ : Mod[n, 10], Suma_ : 0] := If[m == 0, Suma, SDCAux[(m - digito)/10, Mod[(m - digito)/10, 10], Suma + digito]]; SDCAux[n]] y BToDCola[n_] := Module[{BToDAux}, BToDAux[m_, Cont_ : 0, digito_ : Mod[n, 10], Decimal_ : 0] := If[m == 0, Decimal, BToDAux[(m - digito)/10, Cont + 1, Mod[(m - digito)/10, 10], Decimal + 2^Cont digito]]; BToDAux[n]].

1.5.17 DTtoB[n_, Lista_ : {}] := Module[{L = Lista, digito = Mod[n, 2]}, If[n == 0, L, L = Prepend[L, digito]; DTtoB[(n - digito)/2, L]], o bien, DTtoB[n_, Lista_ : {}] := Module[{L = Lista, digito = Mod[n, 2]}, If[n == 0, ToExpression[StringJoin[ToString /@ L]], L = Prepend[L, digito]; DTtoB[(n - digito)/2, L]].

1.5.18 NumPerfect[n_, Cont_ : 1, Lista_ : {}] := Module[{L = Lista}, If[Cont <= n/2, If[Mod[n, Cont] == 0, L = Append[L, Cont]]; NumPerfect[n, Cont + 1, L], If[Total[List] == n, True, False]]].

1.5.19 DesComp[n_, Cont_ : 1] := Module[{i = Cont}, If[Cont <= n, Print[n, "=", n - i, "+", i]; i++; DesComp[n, i]].

1.5.20 mcm[n_, m_] := Module[{MCD}, MCD[nn_, mm_] := If[nn == 0, Return[mm], MCD[Mod[mm, nn], nn]]; n*m/MCD[n, m]].

1.5.21 quicksortDes[begin_ : 1, end_] := Module[{piv, aux}, If[begin < end, i = begin; j = end; piv = L[[Floor[(begin + end)/2]]]; While[i <= j, While[L[[i]] > piv, i++]; While[L[[j]] < piv, j--]; If[i <= j, aux = L[[i]]; L[[i]] = L[[j]]; L[[j]] = aux; i++; j--]; Print[ReplacePart[L, Style[L[[Floor[(begin + end)/2]], Blue], Floor[(begin + end)

```
/2]]]; If[begin < j, quicksortDes[begin, j]]; If[i < end,  
quicksortDes[i, end]]]; L].
```

1.5.22 `Maximo[begin_ : 1, end_] := Module[{piv, aux},
If[begin < end, i = begin; j = end; piv = L[[Floor[(begin
+ end)/2]]]; While[i <= j, While[L[[i]] > piv, i++];
While[L[[j]] < piv, j-]; If[i <= j, aux = L[[i]]; L[[i]] =
L[[j]]; L[[j]] = aux; i++; j-]; If[begin < j, Maximo[begin,
j]]; If[i < end, Maximo[i, end]]]; First[L]].`

RESPUESTAS • SECCIÓN 2.3

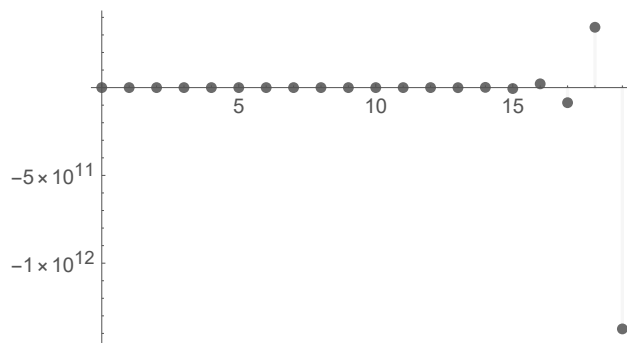
Relaciones de recurrencia

2.3.1 $\ln[] :=$

```
GraficaRRL[{-4}, {5}, npuntos -> 20, inicio -> 0]
```

```
RT[{-4}, {5}, 20, inicio -> 0]
```

```
Out[] =
```



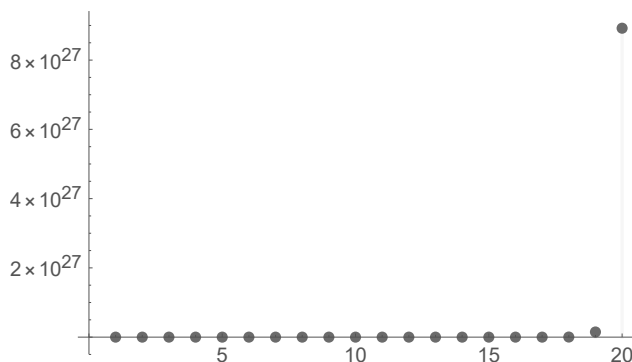
```
{5, -20, 80, -320, 1280, -5120, 20480, -81920, 327680, -1310720,
5242880, -20971520, 83886080, -335544320, 1342177280, -5368709120,
21474836480, -85899345920, 343597383680, -1374389534720}
```

2.3.2 $\ln[] :=$

```
GraficaRRL[{3 n, 3^n}, {1}, npuntos -> 20]
```

```
RT[{3 n, 3^n}, {1}, 20]
```

```
Out[] =
```



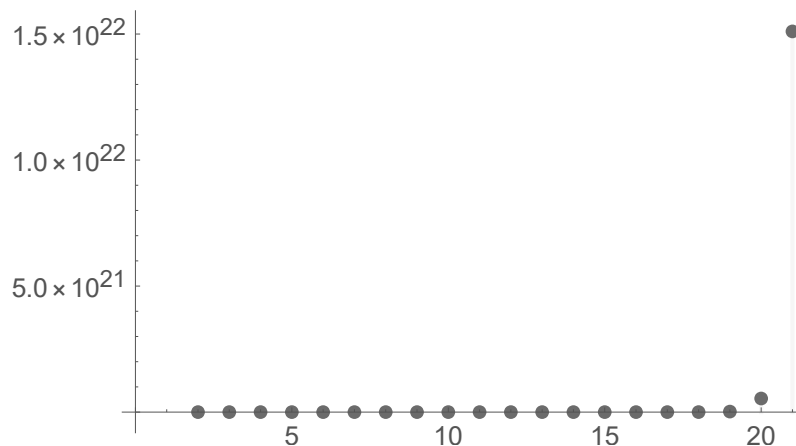
```
{1, 15, 162, 2025, 30618, 551853, 11591100, 278192961, 7511229630,
225336947949, 7436119459464, 267700301072145, 10440311743407978,
438493093227918045, 19732189195270660932, 947145081373034771457,
48304399150024902484470, 2608437554101345121581869,
148680940583776673092428000, 8920856435026600389032464401}
```

2.3.3 $\ln[] :=$

```
GraficaRRL[{(4 n)/3}, {5/2}, npuntos -> 20, inicio -> 2]
```

```
RT[{(4 n)/3}, {5/2}, 20, inicio -> 2]
```

Out[] =

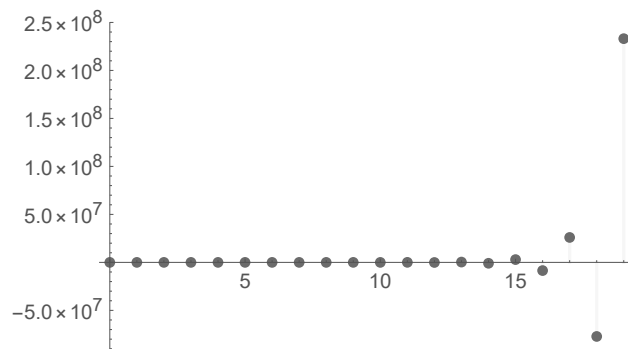


{5/2, 10, 160/3, 3200/9, 25600/9, 716800/27, 22937600/81,
91750400/27, 3670016000/81, 161480704000/243, 2583691264000/243,
134351945728000/729, 7523708960768000/2187, 150474179215360000/2187,
9630347469783040000/6561, 654863627945246720000/19683,
5238909023561973760000/6561, 398157085790710005760000/19683,
31852566863256800460800000/59049, 891871872171190412902400000/59049}

2.3.4 In[] :=

```
GraficaRRL[{-1, 6}, {1, 3}, npuntos -> 20, inicio -> 0]  
RT[{-1, 6}, {1, 3}, 20, inicio -> 0]
```

Out[] =

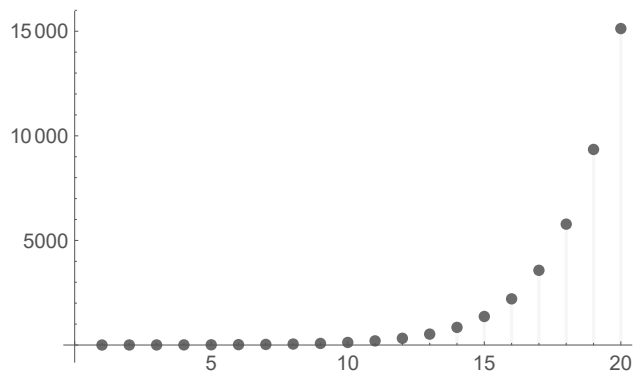


{1, 3, 3, 15, 3, 87, -69, 591, -1005, 4551, -10581, 37887, -101373,
328695, -936933, 2909103, -8530701, 25985319, -77169525, 233081439}

2.3.5 In[] :=

```
GraficaRRL[{1, 1}, {1, 3}, npuntos -> 20]  
RT[{1, 1}, {1, 3}, 20]
```

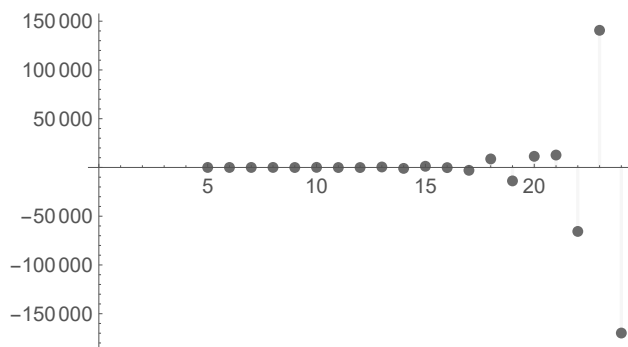
Out[] =



{1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, 521, 843, 1364, 2207, 3571, 5778, 9349, 15127}

2.3.6 In[] :=

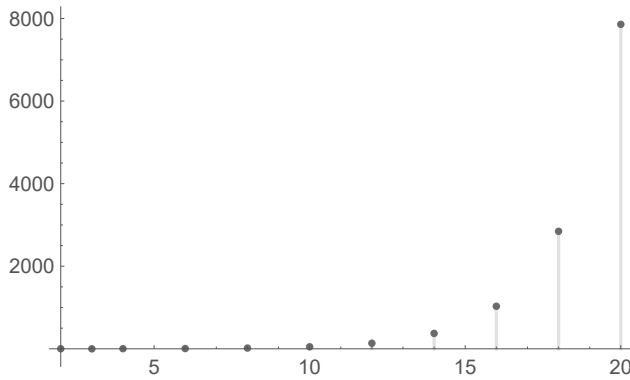
```
GraficaRRL[{-1, Sqrt[2], 5}, {7, -8, 3/2}, npuntos -> 20,
inicio -> 5]
RT[{-1, Sqrt[2], 5}, {7, -8, 3/2}, 20, inicio -> 5] // N
Out[ ] =
```



{7., -8., 1.5, 22.1863, -60.065, 98.9411, -72.9544, -87.4466, 478.979, -967.419, 1207.57, -180.806, -2948.53, 8730.66, -13804.6, 11408.9, 12721.8, -65610., 140646., -169823.}

2.3.7 In[] :=

```
a[n_] := Power[-(7/2) (a[n - 1])^3 + 5 (a[n - 2])^3, (3)^-1]
a[2] = Sqrt[2];
a[3] = -(2/3);
DiscretePlot[a[n], {n, 2, 20}]
Surd[RT[{- (7/2), 5}, {(Sqrt[2])^3, (-2/3)^3}, 20, inicio -> 3],
3] // N
Out[ ] =
```



{1.41421, -0.666667, 2.47599, -3.79391, 6.43949, -10.6491, 17.7177,
-29.4356, 48.9202, -81.2959, 135.101, -224.514, 373.105, -620.038,
1030.4, -1712.35, 2845.64, -4728.97, 7858.75, -13059.9}

2.3.8 $a_n = n \cdot a_{n-1} + 1$ con $a_1 = 1$.

2.3.9 $a_n = -\frac{1}{2}a_{n-1} + 5a_{n-2}$ con $a_1 = 5$ y $a_2 = -8$.

2.3.10 $a_n = a_{n-1} + n - 1$ con $a_1 = 0$.

2.3.11 $a_n = 6a_{n-1} - 9a_{n-2}$ con $a_1 = 99$ y $a_2 = 270$.

2.3.12 $a_n = 2^n - 1$, RR[{2, 1}, {1}, n].

2.3.13 $a_n = 2 \cdot 3^{\frac{n(n+1)}{2}}$, RR[{3~n}, {2}, n, inicio -> 0].

2.3.14 $a_n = 3 \cdot 2^{n-1}n!$, RR[{2 n}, {3}, n].

2.3.15 $a_n = \frac{1}{8}(-5 + 2\sqrt{2} + 41 \cdot 5^{-5+n} - 2\sqrt{2} \cdot 5^{-5+n} - 4n)$, RR[{5, 2 n - Sqrt[2]}, {2}, n, inicio -> 5].

2.3.16 $a_n = \frac{1}{3} 2^n \sqrt{3}$, no se puede corroborar la respuesta mediante la instrucción RR.

2.3.17 $a_n = \frac{1}{10}(5 - \sqrt{5})\left(\frac{1}{2}(7 - 3\sqrt{5})\right)^n + \frac{1}{10}(5 + \sqrt{5})\left(\frac{1}{2}(7 + 3\sqrt{5})\right)^n$, RR[{7, -1}, {1, 5}, n, inicio -> 0], MetodoRRHL[{7, -1}, {1, 5}, n, b, inicio -> 0].

2.3.18 $a_n = \frac{-798+151i\sqrt{21}}{1050}\left(\frac{1}{5}(2 - i\sqrt{21})\right)^n - \frac{25(42i+\sqrt{21})}{42(17+4\sqrt{21})}\left(\frac{1}{5}(2 + i\sqrt{21})\right)^n$, RR[{4/5, -1}, {2, 1}, n, inicio -> 2], MetodoRRHL[{4/5, -1}, {2, 1}, n, b, inicio -> 2].

2.3.19 $a_n = 15 - 10 \cdot 2^n + \frac{7}{2}n \cdot 2^n$, RR[{5, -8, 4}, {5, 2, 3}, n, inicio -> 0], MetodoRRHL[{5, -8, 4}, {5, 2, 3}, n, b, inicio -> 0].

2.3.20 $a_n = 5^n - \frac{527}{375}n \cdot 5^n + \frac{91}{125}n^2 \cdot 5^n - \frac{46}{375}n^3 \cdot 5^n$, RR[{20, -150, 500, -625}, {1, 1, 3, 3}, n, inicio -> 0], MetodoRRHL[{20, -150, 500, -625}, {1, 1, 3, 3}, n, b, inicio -> 0].

2.3.21 $a_n = \left[\frac{1}{3}(-2 + \sqrt{2})(-1)^n + \frac{1}{6}(1 + \sqrt{2})2^n\right]^2$, (RR[{1, 2}, {1, Sqrt[2]}, n])^2, MetodoRRHL[{1, 2}, {1, Sqrt[2]}, n, b].

2.3.22 $a_n = -\sqrt{b_1\left(\frac{1}{2}(1 - i\sqrt{7})\right)^n + b_2\left(\frac{1}{2}(1 + i\sqrt{7})\right)^n}$ con $b_1 = \frac{1}{7}i(-14i + 3\sqrt{7})$ y $b_2 = -\frac{1}{7}i(14i + 3\sqrt{7})$, -(RR[{1, -2}, {4, 5}, n, inicio -> 0])^(1/2), MetodoRRHL[{1, -2}, {4, 5}, n, b, inicio -> 0].

2.3.23 $a_n = a_{n-1} + 2a_{n-2}$ con $a_1 = 1$ y $a_2 = 3$, $a_n = \frac{1}{3}(-1)^n + \frac{2}{3} \cdot 2^n$, RR[{1, 2}, {1, 3}, n], MetodoRRHL[{1, 2}, {1, 3}, n, b].

2.3.24 $a_n = 1.012 \cdot a_{n-1}$ con $a_0 = 7.1$, $a_n = 7.1 \cdot (1.012)^n$, $\text{RR}[\{1.012\}, \{7.1\}, n, \text{inicio} \rightarrow 0]$, $a_{47} = 12.4378$, para el año 2050 se estiman 12.4378 billones de personas en el mundo.

2.3.25 $\text{RRCP}[\{-1, 2, 2\}, \{1, 2\}, n, 10, \text{inicio} \rightarrow 0]$,
 $a_n = \frac{1}{9}((12 - 2j) + (2j - 3)(-2)^n + 6jn)$.

2.3.26 $\text{RRCP}[\{1, 1, j\}, \{1, 1\}, n, 10, \text{inicio} \rightarrow 2]$,
 $a_n = \frac{1}{2}(-2j - (j + 1)\text{Fibonacci}[n] + (j + 1)\text{LucasL}[n])$.

2.3.27 $\text{RRCP}[\{-1, -1, -j\}, \{1, 2\}, n, 10, \text{inicio} \rightarrow 0]$,
 $a_n = b_1(-b_2 + (b_3 + b_4i\sqrt{3})(r_1)^n + (b_3 - b_4i\sqrt{3})(r_2)^n)$ con $i = (0, 1)$, $r_1 = -\frac{1}{2} - \frac{i\sqrt{3}}{2}$,
 $r_2 = -\frac{1}{2} + \frac{i\sqrt{3}}{2}$, $b_1 = \frac{1}{12}(3 + (-1)^{1+j})$, $b_2 = -\frac{1}{2}(3 + (-1)^j)j$, $b_3 = \frac{1}{4}(3 + (-1)^j)(3 + j)$
y
 $b_4 = \frac{1}{4}(3 + (-1)^j)(5 + j)$.

RESPUESTAS • SECCIÓN 3.5

Análisis de algoritmos

- 3.5.1** Una recursividad de cola que construye los números de *Fibonacci* es más rápida.
- 3.5.2** Es más eficiente en tiempo de ejecución el método **a**.
- 3.5.3** Se muestra más veloz MCD1.
- 3.5.4** **Selección** es más rápido.
- 3.5.5** *Quicksort* es más eficiente en tiempo de ejecución.
- 3.5.6** Se demuestra por la propiedad 3 del teorema 3.3,
 $\text{CDFGraficaNAP}[\{\text{Sum}[n^i, \{i, 0, j\}], n^j\}, 0.01, 10, 1000, 10]$.
- 3.5.7** $\text{Limit}[\text{Sum}[1/i, \{i, 1, n\}]/\text{Log}[n], n \rightarrow \text{Infinity}]$,
 $\text{CDFGraficaNA}[\{\text{Sum}[1/i, \{i, 1, n\}], \text{Log}[n]\}, 0.01, 10, 1000]$.
- 3.5.8** $\text{Limit}[(n^3 + 5)/(5n + 2)/n^2, n \rightarrow \text{Infinity}]$,
 $\text{CDFGraficaNA}[\{(n^3 + 5)/(5n + 2), n^2\}, 0.01, 10, 1000]$.
- 3.5.9** $\text{Limit}[(n \text{ Log}[n])/n^2, n \rightarrow \text{Infinity}]$,
 $\text{CDFGraficaNA}[\{(n \text{ Log}[n])/n^2, n^2\}, 0.01, 10, 1000]$.
- 3.5.10** $\text{Limit}[(n^{2^n} + n^{n^2})/n^{2^n}, n \rightarrow \text{Infinity}]$,
 $\text{CDFGraficaNA}[\{n^{2^n} + n^{n^2}, n^{2^n}\}, 0.01, 10, 1000]$.
- 3.5.11** $\text{Limit}[\text{Sum}[i^2, \{i, 1, n\}]/n^3, n \rightarrow \text{Infinity}]$,
 $\text{CDFGraficaNA}[\{\text{Sum}[i^2, \{i, 1, n\}], n^3\}, 0.01, 10, 1000]$.
- 3.5.12** $O(7^n)$.
- 3.5.13** $O(2^n 5^n)$.
- 3.5.14** $O(n^n n!)$.
- 3.5.15** $O(n^4 \ln n)$.
- 3.5.16** $O(n^3 \ln n)$.
- 3.5.17** $O(n^2)$.
- 3.5.18** $(n-1) \frac{(n-2)}{2} = \frac{1}{2}n^2 - \frac{3}{2}n + 1 = O(n^2)$, $\text{CDFGraficaNA}[\{((n-1)(n-2))/2, n^2\}, 0.01, 10, 1000]$.
- 3.5.19** $\Theta(n)$, $\text{CDFGraficaNA}[\{(2n-1)/2 + a /. a \rightarrow 0.5, n\}, 0.01, 10, 1000]$.
- 3.5.20** $O(\ln n)$, $\text{CDFGraficaNA}[\{\text{Log}[n]/\text{Log}[5] + a /. a \rightarrow 0.5, \text{Log}[n]\}, 0.01, 10, 1000]$.
- 3.5.21** $\Theta((\ln n)^2)$, $\text{CDFGraficaNA}[\{\text{Sum}[i, \{i, 1, 2(\text{Log}[n] - \text{Log}[2])/\text{Log}[5] + a /. a \rightarrow 0.5\}], (\text{Log}[n])^2\}, 0.01, 10, 1000]$.
- 3.5.22** $f(n) = \frac{2}{3}n^3 + (\ln n)^5$ y $g(n) = n^j, 1 \leq j \leq 2 \Rightarrow f(n) = \Omega(g(n)), j = 3 \Rightarrow f(n) = \Theta(g(n))$ y $j \geq 4 \Rightarrow f(n) = O(g(n)), j \in \mathbb{N}$.
- 3.5.23** $f(n) = \frac{n^2 + 3 \ln n}{n^5 + 1}$ y $g(n) = n^j, j \in \mathbb{N} \Rightarrow f(n) = O(g(n))$, nunca ocurre Ω ni Θ .

3.5.24 $f(n) = n^{6j} - n^5 + 2n - 4$ y $g(n) = n^j$, $j \in \mathbb{N} \Rightarrow f(n) = \Omega(g(n))$, nunca ocurre O ni Θ .

3.5.25 $f(n) = \frac{n^8}{n^j + 1}$ y $g(n) = (\ln(n))^j$, $1 \leq j \leq 7 \Rightarrow f(n) = \Omega(g(n))$, $j \geq 8 \Rightarrow f(n) = O(g(n))$ y nunca ocurre Θ , $j \in \mathbb{N}$.

3.5.26 $f(n) = \frac{n^{\frac{1}{j}} (\ln n)^{-5j}}{n^{\frac{1}{j}} + 1}$ y $g(n) = (\ln n)^{-5j}$, $j \in \mathbb{N} \Rightarrow f(n) = \Theta(g(n))$, nunca ocurre Ω ni O .

RESPUESTAS • SECCIÓN 4.5

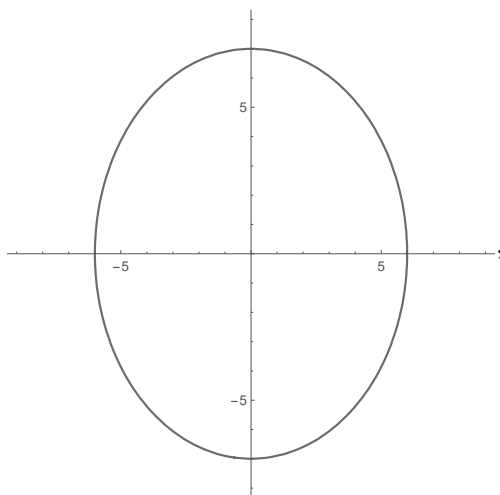
Relaciones binarias

4.5.1 `PC[Range[2, 500, 2], Range[1, 500, 2]], Length[Lista], 62500.`

4.5.2 `GraficaPC[{5, Infinity}, {-10, 6}].`

4.5.3 $D = \{3, 15, 25, 75\}$, $Rang = \{4, 12, 20, 60, 100\}$, 9900.

4.5.4



$D = [-6, 6]$, $Rang = [-7, 7]$, $\{\text{True}, \text{False}, \text{True}, \text{False}, \text{True}, \text{True}, \text{False}, \text{True}, \text{True}, \text{True}\}$.

4.5.5 `R1=RelBin["a<=b^2", Range[2, 100, 2], Range[2, 100, 2]], {False, True, False, True, False}.`

4.5.6 `R2 = RelBin["a+b<=20", A, A] con A=Range[2, 100, 2].`

4.5.7 `R3 = RelBin["PalindromeQ[a]&&PalindromeQ[b]", A, A] con A el conjunto del enunciado.`

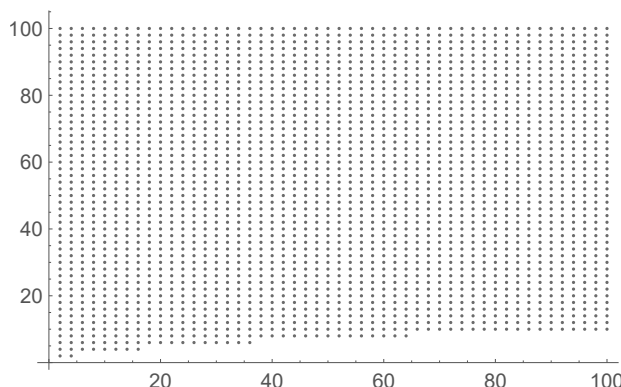
4.5.8 `R4 = RelBin["Mod[a^2-b^2,5]==0", A, A] con A=Range[5].`

4.5.9 `R5 = RelBin["Mod[a-3,3]==Mod[b-3,3]", A, A] con A = Range[5].`

4.5.10 `R6 = RelBin["a-b>=3", A, A] con A = Range[5].`

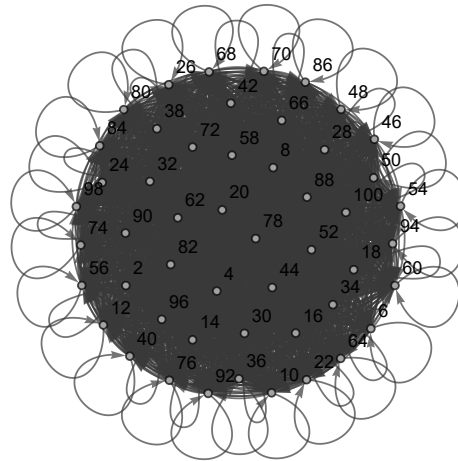
4.5.11 `R7 = RelBin["Mod[Abs[a^3+b^2],3]==0", A, A], o bien, R7 = RelBin["IntegerQ[Abs[a^3+b^2]/3]", A, A], con A = Range[5].`

4.5.12 $R_1 :$

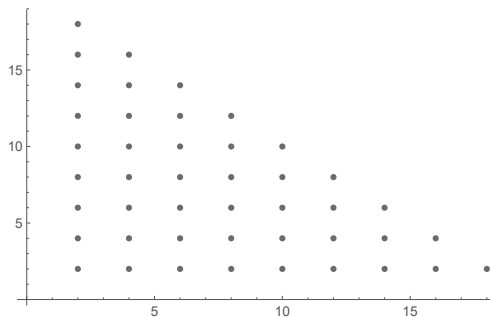


In[] :=

```
A = Range[2, 100, 2];
R1 = RelBin["a<=b^2", A, A];
MR1 = MatrizRelBin[R1, A, A]
```

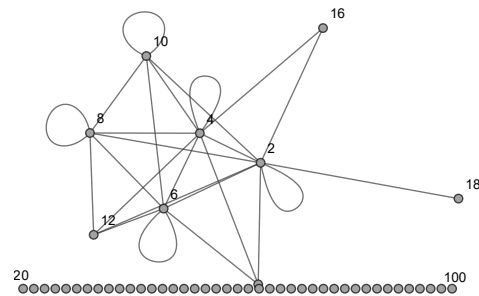


R_2 :

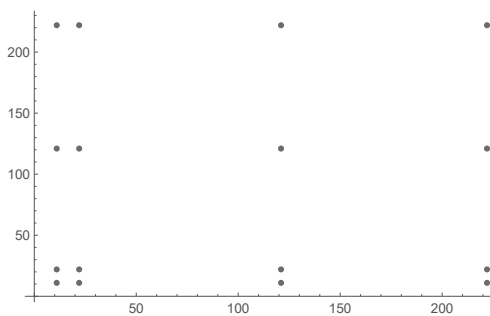


In[] :=

```
A = Range[2, 100, 2];
R2 = RelBin["a+b<=20", A, A];
MR2 = MatrizRelBin[R2, A, A]
```



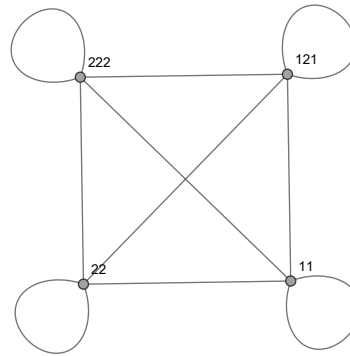
R_3 :



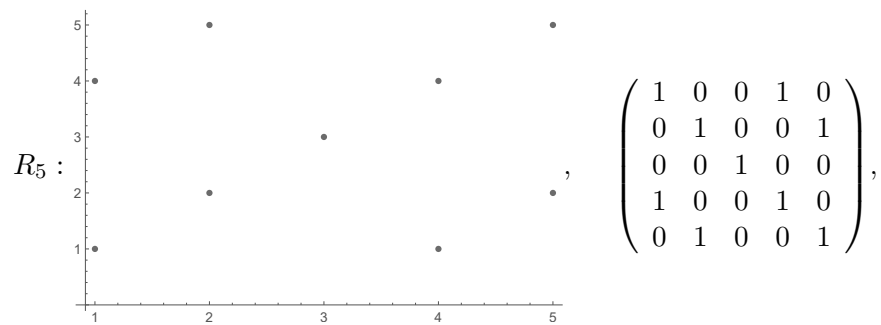
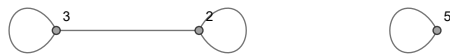
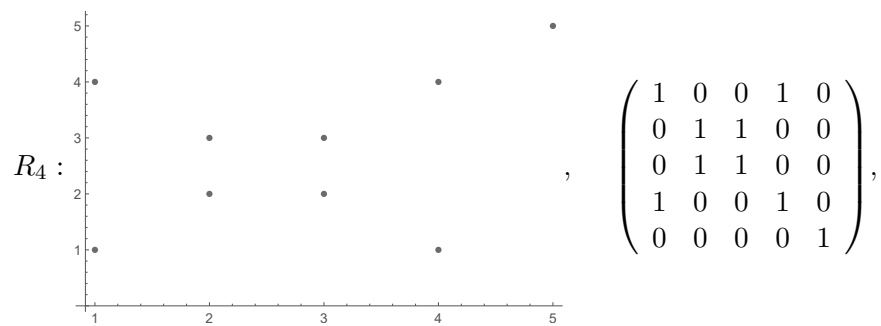
In[] :=

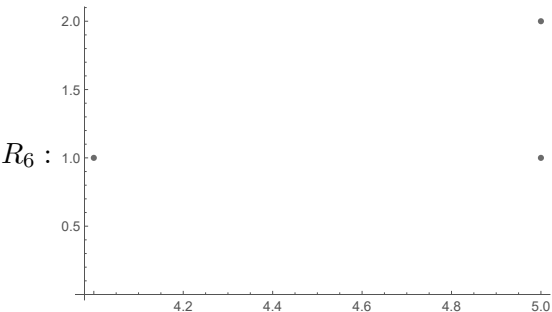
```
R3 = RelBin["PalindromeQ[a]&&PalindromeQ[b]", A, A];
MR3 = MatrizRelBin[R3, A, A]
```

con A el conjunto del enunciado,

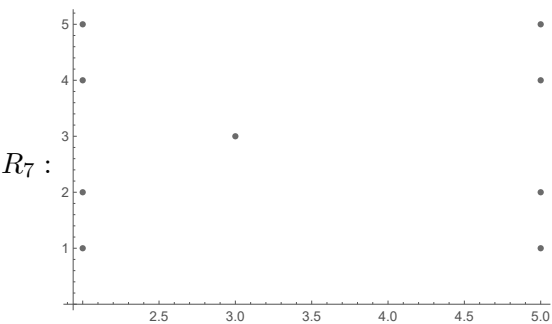
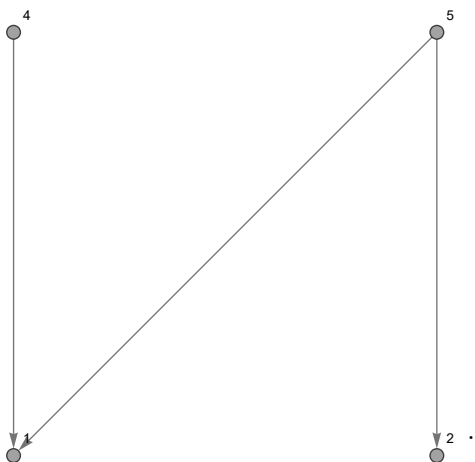


13 17 19 21 23 29 32 51 72 83 89 97 113 127 312 723

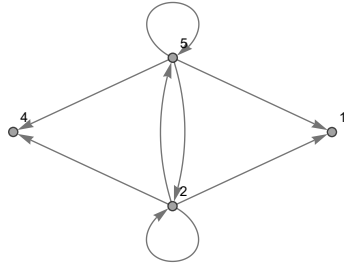




$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix},$$



$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \end{pmatrix},$$



- 4.5.13** $\overline{R_1^{-1}} : R1Inversa = Reverse \text{ /\@ } R1, Complement[PC[A, A], R1Inversa], R_1 o R_1 : RelBinMatriz[ProductoBooleano[MR1, MR1][[1]], A, A].$
- 4.5.14** $\overline{R_2 o R_2} : Complement[PC[A, A], RelacionComposicion[R2, R2]], (R_2 o R_2) o R_2 : RelBinMatriz[ProductoBooleano[ProductoBooleano[MR2, MR2][[1]], MR2][[1]], A, A].$
- 4.5.15** $R_3^{-1} \cup R_3 : \{\{11, 11\}, \{11, 22\}, \{11, 121\}, \{11, 222\}, \{22, 11\}, \{22, 22\}, \{22, 121\}, \{22, 222\}, \{121, 11\}, \{121, 22\}, \{121, 121\}, \{121, 222\}, \{222, 11\}, \{222, 22\}, \{222, 121\}, \{222, 222\}\}, R_3 \cap R_3^{-1} = R_3^{-1} \cup R_3.$
- 4.5.16** $\overline{R_1^{-1} \cup R_2^{-1}} : Complement[PC[A, A], Union[Reverse \text{ /\@ } R1, Reverse \text{ /\@ } R2]], R_1^{-1} o R_2^{-1} : RelBinMatriz[ProductoBooleano[Transpose[MR2], Transpose[MR1]][[1]], A, A].$
- 4.5.17** $(R_6 \cup R_5)^{-1} : \{\{1, 1\}, \{4, 1\}, \{2, 2\}, \{5, 2\}, \{3, 3\}, \{1, 4\}, \{4, 4\}, \{1, 5\}, \{2, 5\}, \{5, 5\}\}, (\overline{R_4 \cap R_7})^{-1} : \{\{2, 2\}, \{3, 3\}, \{5, 5\}\}.$
- 4.5.18** $\{\{1, 1\}, \{1, 4\}, \{1, 5\}, \{1, 2\}, \{1, 3\}, \{2, 2\}, \{2, 3\}, \{3, 1\}, \{3, 4\}, \{3, 5\}, \{3, 2\}, \{3, 3\}, \{4, 1\}, \{4, 4\}, \{4, 5\}, \{4, 2\}, \{4, 3\}, \{5, 2\}, \{5, 3\}\}.$
- 4.5.19** $\{\{1, 1\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{1, 5\}, \{2, 1\}, \{2, 2\}, \{2, 3\}, \{2, 4\}, \{2, 5\}, \{3, 1\}, \{3, 2\}, \{3, 3\}, \{3, 4\}, \{3, 5\}, \{4, 1\}, \{4, 2\}, \{4, 3\}, \{4, 4\}, \{4, 5\}, \{5, 1\}, \{5, 2\}, \{5, 3\}, \{5, 4\}, \{5, 5\}\}.$
- 4.5.20** $R_1 : \text{reflexiva}, R_2 : \text{simétrica}, R_3 : \text{simétrica y transitiva}, R_4 : \text{de equivalencia}, R_5 : \text{de equivalencia}, R_6 : \text{antisimétrica y transitiva}, R_7 : \text{transitiva}.$
- 4.5.21** `Table[i -> ReEquivalenciaParticion[Select[SetPartitions[Range[5]], Length[#] == 2 &][[i]], Range[5]], {i, Length[Select[SetPartitions[Range[5]], Length[#] == 2 &]]}].`

RESPUESTAS • SECCIÓN 5.5

Teoría de grafos

5.5.1 Grafo 1 : $b \rightarrow e \rightarrow d \rightarrow e \rightarrow g \rightarrow f \rightarrow f \rightarrow b \rightarrow a \rightarrow a \rightarrow c \rightarrow b$, longitud 11. Grafo 2 : $b \rightarrow a \rightarrow e \rightarrow d \rightarrow i \rightarrow h \rightarrow f \rightarrow d \rightarrow c \rightarrow g \rightarrow f \rightarrow i \rightarrow d \rightarrow b$, longitud 13. Grafo 3 : $a \rightarrow c \rightarrow h \rightarrow g \rightarrow f \rightarrow d \rightarrow c \rightarrow e \rightarrow h \rightarrow f \rightarrow e \rightarrow d \rightarrow b \rightarrow a$, longitud 13. Grafo 4 : $a \rightarrow h \rightarrow g \rightarrow b \rightarrow i \rightarrow f \rightarrow b \rightarrow e \rightarrow c \rightarrow d \rightarrow g \rightarrow a \rightarrow i \rightarrow e \rightarrow f \rightarrow a \rightarrow e \rightarrow d \rightarrow a$, longitud 18.

5.5.2 Grafo 1 : $a \rightarrow a \rightarrow c \rightarrow b \rightarrow f \rightarrow f \rightarrow b \rightarrow e \rightarrow f \rightarrow g \rightarrow e \rightarrow d \rightarrow e \rightarrow d$, longitud 13. Grafo 2 : $a \rightarrow d \rightarrow e \rightarrow j \rightarrow i \rightarrow d \rightarrow i \rightarrow h \rightarrow f \rightarrow g \rightarrow c \rightarrow d \rightarrow b \rightarrow b \rightarrow b$, longitud 14. Grafo 3 : $h \rightarrow g \rightarrow f \rightarrow h \rightarrow e \rightarrow d \rightarrow f \rightarrow e \rightarrow c \rightarrow b \rightarrow a$, longitud 10. Grafo 4 : $b \rightarrow i \rightarrow f \rightarrow a \rightarrow e \rightarrow c \rightarrow d \rightarrow a \rightarrow g \rightarrow h \rightarrow a \rightarrow i$, longitud 11.

5.5.3

Grafo 1 :		a	b	c	d	e	f	g			
	Grado o valencia	4	5	2	3	6	6	2			
Grafo 2 :		a	b	c	d	e	f	g	h	i	j
	Grado o valencia	3	7	3	7	3	4	2	2	5	2
Grafo 3 :		a	b	c	d	e	f	g	h		
	Grado o valencia	2	3	5	4	4	4	2	4		
Grafo 4 :		a	b	c	d	e	f	g	h	i	
	Grado o valencia	6	4	2	4	6	4	4	2	4	

5.5.4 Grafo 1 : $\frac{28}{2} = 14$. Grafo 2 : $\frac{38}{2} = 19$. Grafo 3 : $\frac{28}{2} = 14$. Grafo 4 : $\frac{36}{2} = 18$.

5.5.5 Grafo 3 : no hay circuito euleriano pero sí una ruta de *Euler*: $b \rightarrow a \rightarrow c \rightarrow d \rightarrow e \rightarrow c \rightarrow h \rightarrow e \rightarrow f \rightarrow g \rightarrow h \rightarrow f \rightarrow d \rightarrow b \rightarrow c$. Grafo 4 : no hay una ruta euleriana pero sí un circuito: $a \rightarrow d \rightarrow g \rightarrow h \rightarrow a \rightarrow i \rightarrow f \rightarrow e \rightarrow i \rightarrow b \rightarrow g \rightarrow a \rightarrow f \rightarrow b \rightarrow e \rightarrow c \rightarrow d \rightarrow e \rightarrow a$.

5.5.6 Grafo 3 : $m = 14 < \frac{n^2 - 3n + 6}{2} = 23$. La propiedad de *Dirac* es falsa, $\theta(a) = 2 < \frac{n}{2} = 4$. La propiedad de *Ore* es falsa, $\theta(a) + \theta(d) = 6 < n = 8$. El grafo sí tiene un circuito de *Hamilton*: $a \rightarrow b \rightarrow d \rightarrow e \rightarrow f \rightarrow g \rightarrow h \rightarrow c \rightarrow a$. Grafo 4 : $m = 18 < \frac{n^2 - 3n + 6}{2} = 30$. La propiedad de *Dirac* es falsa, $\theta(d) = 4 < \frac{n}{2} = \frac{9}{2}$. La propiedad de *Ore* es falsa, $\theta(a) + \theta(c) = 8 < n = 9$. El grafo sí tiene un circuito de *Hamilton*: $a \rightarrow h \rightarrow g \rightarrow b \rightarrow i \rightarrow f \rightarrow e \rightarrow c \rightarrow d \rightarrow a$.

5.5.7 Grafo 1 :
$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 2 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 1 & 0 & 3 & 0 & 1 & 1 \\ 0 & 2 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \text{ y}$$

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

Grafo 4 :

$$\begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} \text{ y}$$

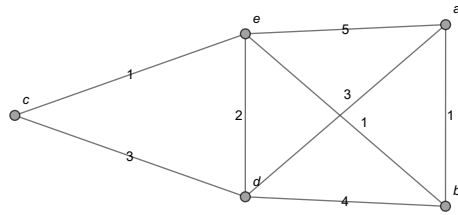
$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

5.5.8 Grafo 3 :

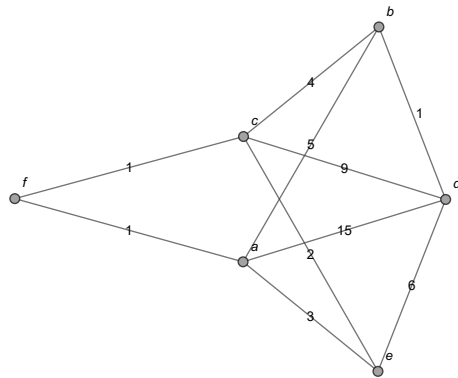
$$\begin{pmatrix} 0 & 3 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 4 & 2 & 0 & 0 & 0 & 0 \\ 2 & 4 & 0 & 3 & 2 & 0 & 0 & 6 \\ 0 & 2 & 3 & 0 & 5 & 6 & 0 & 0 \\ 0 & 0 & 2 & 5 & 0 & 4 & 0 & 5 \\ 0 & 0 & 0 & 6 & 4 & 0 & 5 & 5 \\ 0 & 0 & 0 & 0 & 0 & 5 & 0 & 4 \\ 0 & 0 & 6 & 0 & 5 & 5 & 4 & 0 \end{pmatrix}.$$

Grafo 4 :

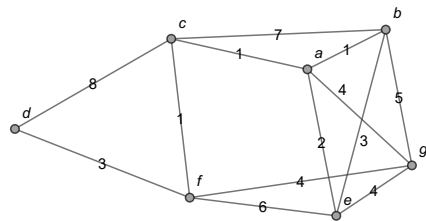
$$\begin{pmatrix} 0 & 0 & 0 & 1 & 3 & 1 & 3 & 5 & 3 \\ 0 & 0 & 0 & 0 & 5 & 4 & 2 & 0 & 3 \\ 0 & 0 & 0 & 5 & 5 & 0 & 0 & 0 & 0 \\ 1 & 0 & 5 & 0 & 5 & 0 & 7 & 0 & 0 \\ 3 & 5 & 5 & 5 & 0 & 5 & 0 & 0 & 3 \\ 1 & 4 & 0 & 0 & 5 & 0 & 0 & 0 & 2 \\ 3 & 2 & 0 & 7 & 0 & 0 & 0 & 1 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 3 & 3 & 0 & 0 & 3 & 2 & 0 & 0 & 0 \end{pmatrix}.$$



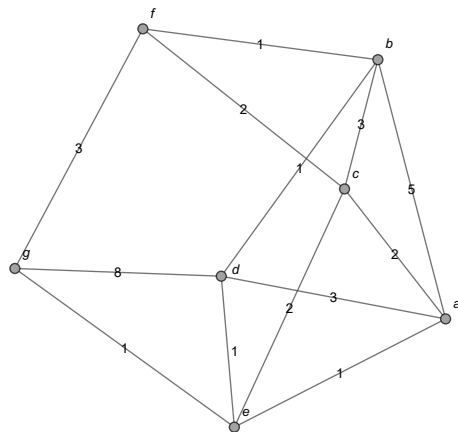
5.5.9



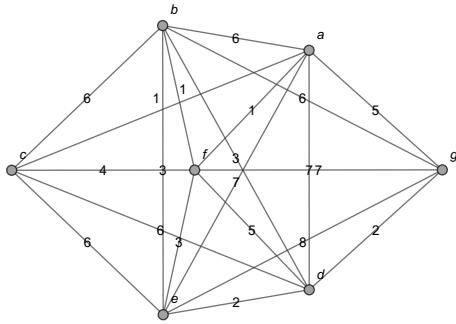
5.5.10



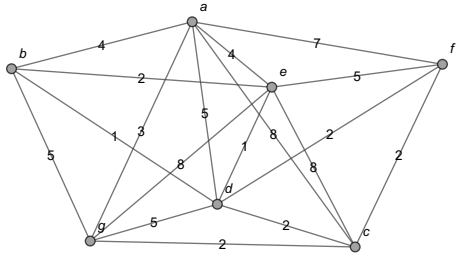
5.5.11



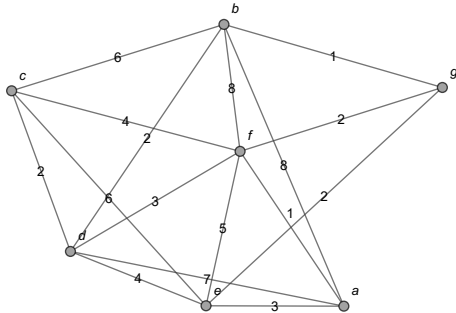
5.5.12



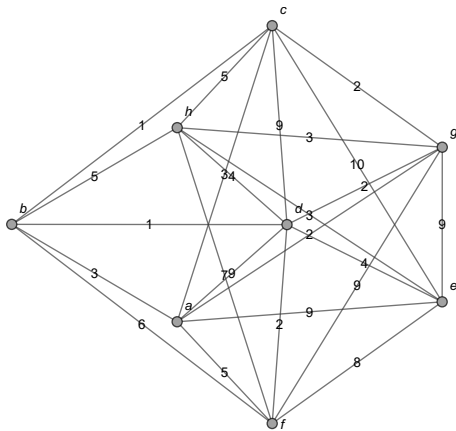
5.5.13



5.5.14



5.5.15



5.5.16

5.5.17 Grafo 5 : hay una ruta de *Euler*: $a \rightarrow d \rightarrow b \rightarrow e \rightarrow c \rightarrow d \rightarrow e \rightarrow a \rightarrow b$. Se cumple la propiedad de las aristas y la de *Ore*, un circuito hamiltoniano es: $a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$. Grafo 6 : hay una ruta euleriana: $e \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow a \rightarrow f \rightarrow c \rightarrow e \rightarrow d \rightarrow b$. No se cumple ninguna de las propiedades y el grafo sí tiene un circuito de *Hamilton*: $a \rightarrow e \rightarrow d \rightarrow b \rightarrow c \rightarrow f \rightarrow a$. Grafo 7 : hay un circuito de *Euler*: $a \rightarrow g \rightarrow f \rightarrow d \rightarrow c \rightarrow f \rightarrow e \rightarrow g \rightarrow b \rightarrow e \rightarrow a \rightarrow c \rightarrow b \rightarrow a$. No se cumple ninguna de las propiedades y el grafo sí tiene un circuito de *Hamilton*: $a \rightarrow g \rightarrow e \rightarrow f \rightarrow d \rightarrow c \rightarrow b \rightarrow a$. Grafo 8 : hay una ruta euleriana: $f \rightarrow b \rightarrow a \rightarrow c \rightarrow b \rightarrow d \rightarrow a \rightarrow e \rightarrow d \rightarrow g \rightarrow e \rightarrow c \rightarrow f \rightarrow g$. Se satisface la propiedad

de *Ore* y un circuito hamiltoniano es: $a \rightarrow d \rightarrow g \rightarrow f \rightarrow b \rightarrow c \rightarrow e \rightarrow a$. Grafo 9 : hay una ruta de *Euler*: $g \rightarrow a \rightarrow b \rightarrow c \rightarrow a \rightarrow d \rightarrow b \rightarrow e \rightarrow a \rightarrow f \rightarrow b \rightarrow g \rightarrow d \rightarrow c \rightarrow e \rightarrow d \rightarrow f \rightarrow e \rightarrow g \rightarrow f \rightarrow c$. Todas las propiedades se satisfacen y un circuito de *Hamilton* es: $a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow g \rightarrow b \rightarrow a$. Grafo 10 : hay una ruta euleriana: $c \rightarrow a \rightarrow b \rightarrow d \rightarrow a \rightarrow e \rightarrow b \rightarrow g \rightarrow a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow f \rightarrow d \rightarrow g \rightarrow e \rightarrow c \rightarrow g$. Todas las propiedades se satisfacen y un circuito hamiltoniano es: $a \rightarrow d \rightarrow b \rightarrow g \rightarrow c \rightarrow f \rightarrow e \rightarrow a$. Grafo 11 : no hay circuitos eulerianos, ni rutas de *Euler*. Se satisface la propiedad de *Ore* y un circuito de *Hamilton* es: $a \rightarrow b \rightarrow g \rightarrow e \rightarrow f \rightarrow c \rightarrow d \rightarrow a$. Grafo 12 : hay una ruta euleriana: $b \rightarrow a \rightarrow c \rightarrow b \rightarrow f \rightarrow a \rightarrow d \rightarrow c \rightarrow e \rightarrow a \rightarrow g \rightarrow c \rightarrow h \rightarrow d \rightarrow e \rightarrow f \rightarrow d \rightarrow g \rightarrow e \rightarrow h \rightarrow f \rightarrow g \rightarrow h \rightarrow b \rightarrow d$. Todas las propiedades se satisfacen y un circuito de *Hamilton* es: $a \rightarrow b \rightarrow c \rightarrow e \rightarrow g \rightarrow h \rightarrow d \rightarrow f \rightarrow a$.

5.5.18 Grafo 3 : $\{8, \{\{a, c\}, \{c, h\}\}\}$. Grafo 4 : $\{4, \{\{a, g\}, \{g, h\}\}\}$. Grafo 5 : $\{3, \{\{a, b\}, \{b, e\}, \{e, c\}\}\}$. Grafo 6 : $\{6, \{\{a, b\}, \{b, d\}\}\}$. Grafo 7 : $\{5, \{\{a, c\}, \{c, f\}, \{f, d\}\}\}$. Grafo 8 : $\{3, \{\{c, e\}, \{e, g\}\}\}$. Grafo 9 : $\{5, \{\{b, d\}, \{d, g\}\}\}$. Grafo 10 : $\{5, \{\{b, g\}\}\}$. Grafo 11 : $\{6, \{\{c, e\}\}\}$. Grafo 12 : $\{5, \{\{b, h\}\}\}$.

5.5.19 Grafo 3 : $\{34, \{\{a, b\}, \{b, c\}, \{c, h\}, \{h, e\}, \{e, d\}, \{d, f\}, \{f, g\}\}\}$. Grafo 4 : $\{25, \{\{b, e\}, \{e, d\}, \{d, g\}, \{g, a\}, \{a, i\}, \{i, f\}\}\}$.

5.5.20 K_3 tiene $2 \cdot 3 = 6$ circuitos simples de longitud 3. K_4 posee $2 \cdot 3 \cdot 4 = 24$ circuitos simples de longitud 4. K_5 contiene $2 \cdot 3 \cdot 4 \cdot 5 = 120$ circuitos simples de longitud 5. En general, K_n tiene $2 \cdot 3 \cdot \dots \cdot n = n!$ circuitos simples de longitud n .

5.5.21 El valor constante de la diagonal principal de la matriz A^k , $1 \leq k \leq 10$, $k \in \mathbb{N}$, corresponde a: 0 con $k = 1$, 19 con $k = 2$, 342 con $k = 3$, 6517 con $k = 4$, 123804 con $k = 5$, 2352295 con $k = 6$, 44693586 con $k = 7$, 849178153 con $k = 8$, 16134384888 con $k = 9$ y 306553312891 con $k = 10$. Utilizando estos datos y el comando `FindSequenceFunction` del software *Wolfram Mathematica* se obtiene la fórmula general: $\frac{1}{20} (19(-1)^k + 19^k)$, $\forall k, k \in \mathbb{N}$.

5.5.22 Grafo rueda: nunca posee un circuito de *Euler* y si $n \geq 3$, con n el orden del grafo, contiene circuitos hamiltonianos. Grafo estrella: no contiene circuitos eulerianos y no posee circuitos de *Hamilton*. Grafo ciclo: para cualquier orden contiene tanto circuitos de *Euler* como de *Hamilton*. Grafo camino: no posee circuitos de ningún tipo. Grafo red: tiene circuitos eulerianos si la cantidad de nodos fila n y de vértices columna m es igual a 2 y no contiene circuitos de *Hamilton* si simultáneamente n y m son números impares, suponiendo que el grafo posee circuitos.

RESPUESTAS • SECCIÓN 6.6

Teoría de árboles

- 6.6.1** Árbol 1 : es conexo y contiene 10 aristas, donde $10 = 11 - 1 = n - 1$, con $n = 11$ el número de vértices del grafo. Árbol 2 : no tiene circuitos y posee 14 aristas, donde $14 = 15 - 1 = n - 1$, con $n = 15$ el número de nodos del grafo. Árbol 3 : es conexo y no tiene circuitos. Árbol 4 : no posee circuitos y contiene 19 aristas, donde $19 = 20 - 1 = n - 1$, con $n = 20$ el número de vértices del grafo.

6.6.2	Árbol 1 :	Árbol 2 :
	Orden: 2 (árbol binario) Hojas: {6, 7, 8, 9, 10, 11} Altura: 3	Orden: 3 Hojas: {1, 2, 3, 14, 13, 6, 12} Altura: 4
	Árbol 3 :	Árbol 4 :
	Orden: 3 Hojas: {7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18} Altura: 3	Orden: 4 Hojas: {1, 3, 4, 9, 15, 17, 14, 16, 18} Altura: 5

- 6.6.3** Árbol 1 : es completo. Árbol 2 : no es completo, por ejemplo, la raíz 10 no tiene tres hijos. Árbol 3 : no es completo, falla en el nodo 6 que no tiene tres hijos. Árbol 4 : no es completo, por ejemplo, la raíz 7 tiene dos hijos en lugar de cuatro.

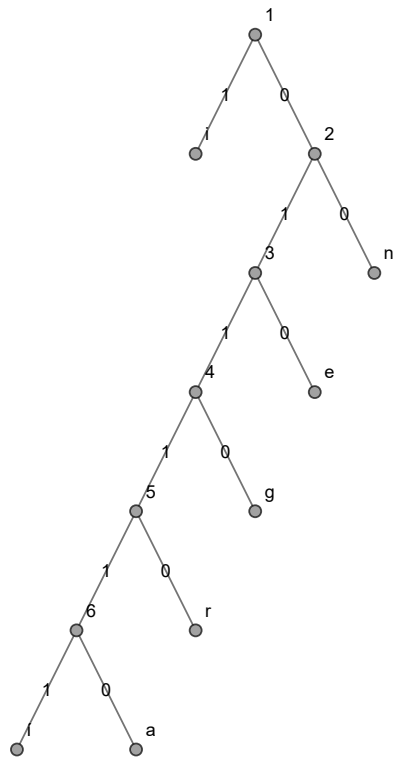
- 6.6.4** Árbol 1 : es balanceado. Árbol 2 : no es balanceado; por ejemplo, la hoja 1 se encuentra en el nivel 2, que no corresponde a la altura del árbol igual a 4 o a la altura menos uno igual a 3. Árbol 3 : es balanceado. Árbol 4 : no es balanceado; por ejemplo, el nodo terminal 9 está en el nivel 2, distinto de la altura 5 del árbol o ese valor menos uno igual a 4.

- 6.6.5** Árbol 1 : `Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {3, 7}, {4, 8}, {4, 9}, {5, 10}, {5, 11}}]`.

Árbol 2 : `Arbol[{{1, 15}, {2, 11}, {3, 11}, {4, 8}, {4, 14}, {5, 11}, {5, 13}, {6, 7}, {7, 9}, {8, 15}, {9, 10}, {9, 11}, {9, 12}, {10, 15}}]`.

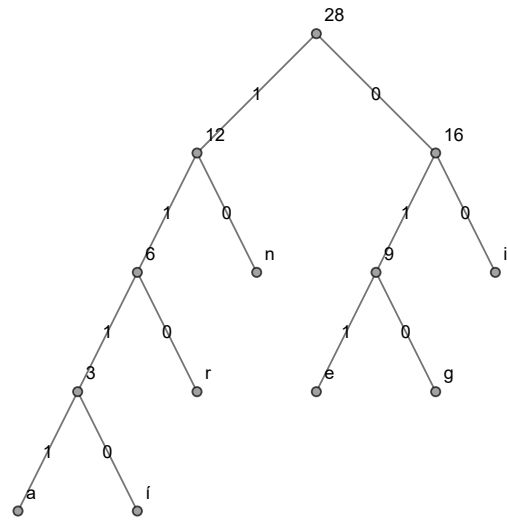
Árbol 3 : `Arbol[{{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {2, 7}, {3, 8}, {3, 9}, {3, 10}, {4, 11}, {4, 12}, {4, 13}, {5, 14}, {5, 15}, {5, 16}, {6, 17}, {6, 18}}]`.

Árbol 4 : `Arbol[{{1, 12}, {2, 3}, {2, 10}, {2, 11}, {2, 13}, {4, 8}, {5, 6}, {5, 7}, {5, 19}, {6, 8}, {7, 12}, {9, 12}, {10, 15}, {11, 17}, {12, 13}, {12, 20}, {14, 19}, {16, 20}, {18, 19}}]`.

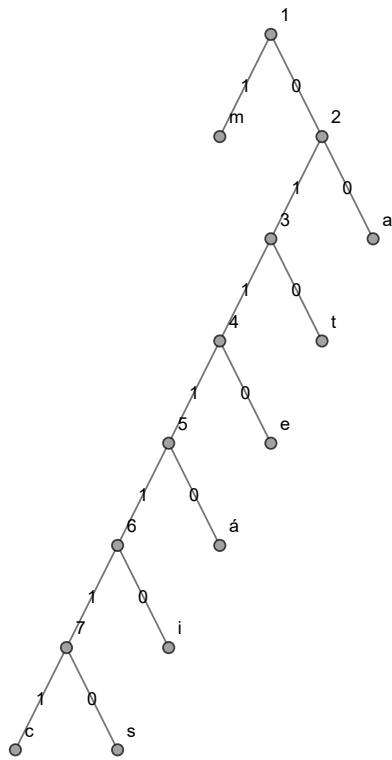


6.6.6

100011001000101001110011111011110.

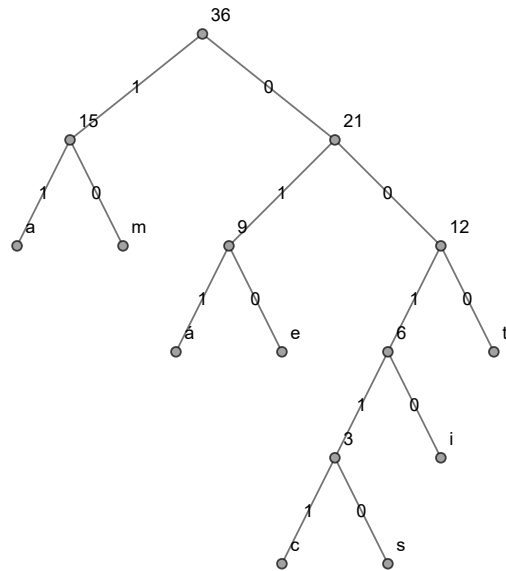


10010011100001111011101111.

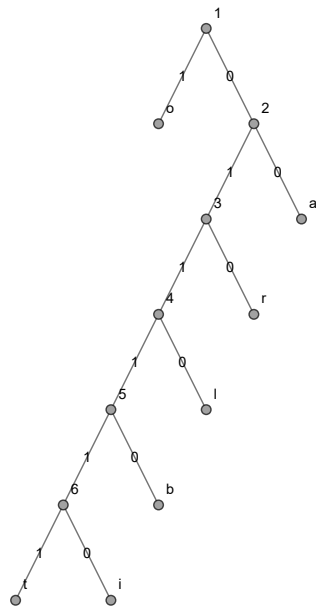


6.6.7

10001001101011100100111100111111000111110.

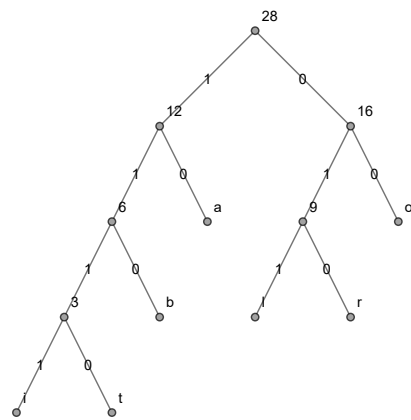


1011000010100110000010001111100110.

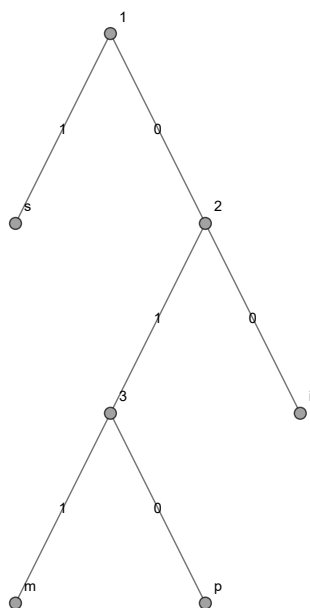


6.6.8

110000111010100001111110100111101.

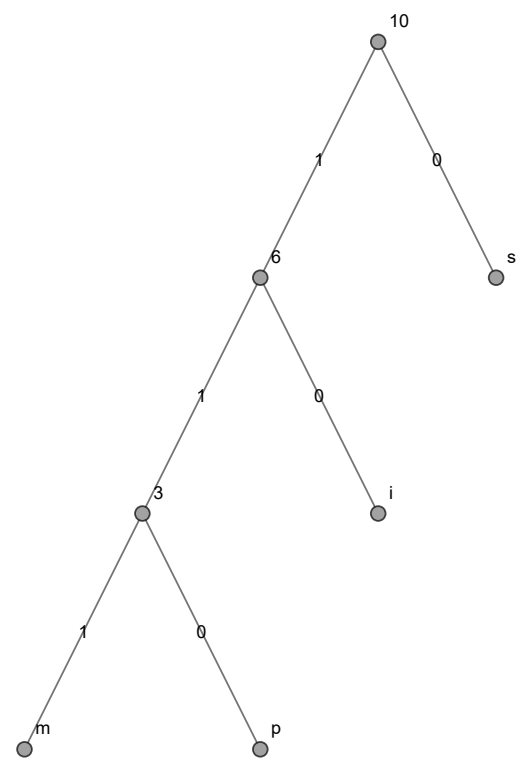


111011000010101110000101111100.

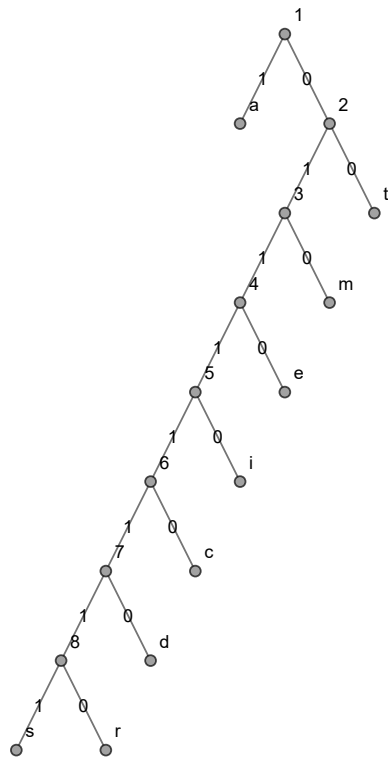


6.6.9

110011001100010001.

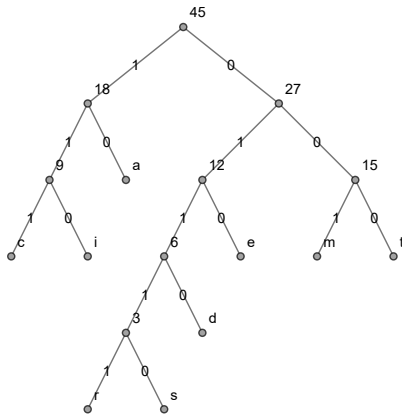


1111000100010110100.

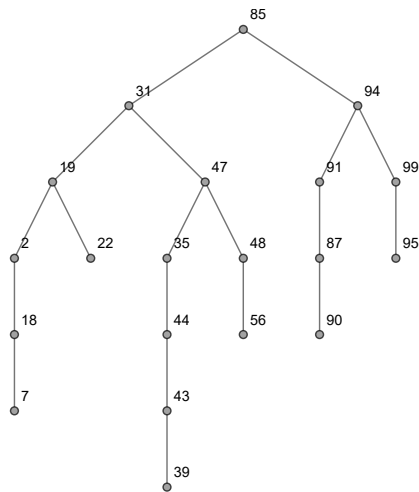


6.6.10

1010001100101000111001111010111110011100111111011110011111100110001.

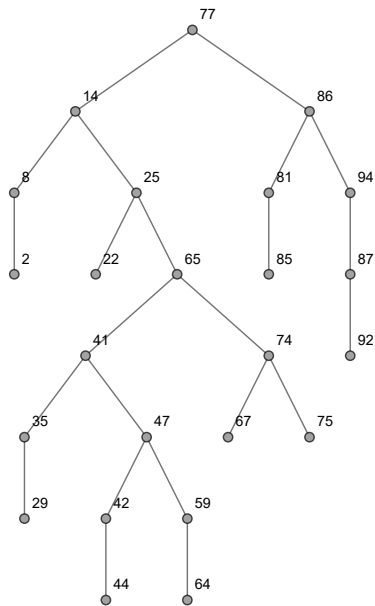


11000001000110000110111100110110011101110111101000010.



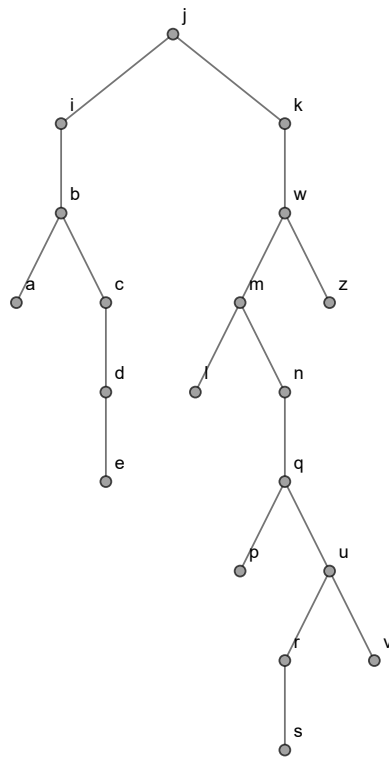
6.6.11

prefijo: {85, 31, 19, 2, 18, 7, 22, 47, 35, 44, 43, 39, 48, 56, 94, 91, 87, 90, 99, 95},
 interfijo: {2, 7, 18, 19, 22, 31, 35, 39, 43, 44, 47, 48, 56, 85, 87, 90, 91, 94, 95, 99} y
 postfijo: {7, 18, 2, 22, 19, 39, 43, 44, 35, 56, 48, 47, 31, 90, 87, 91, 95, 99, 94, 85}.



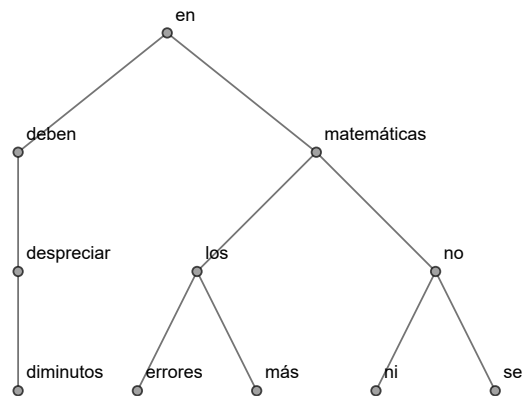
6.6.12

prefijo: {77, 14, 8, 2, 25, 22, 65, 41, 35, 29, 47, 42, 44, 59, 64, 74, 67, 75, 86, 81, 85, 94, 87, 92},
 interfijo: {2, 8, 14, 22, 25, 29, 35, 41, 42, 44, 47, 59, 64, 65, 67, 74, 75, 77, 81, 85, 86, 87, 92, 94} y
 postfijo: {2, 8, 22, 29, 35, 44, 42, 64, 59, 47, 41, 67, 75, 74, 65, 25, 14, 85, 81, 92, 87, 94, 86, 77}.



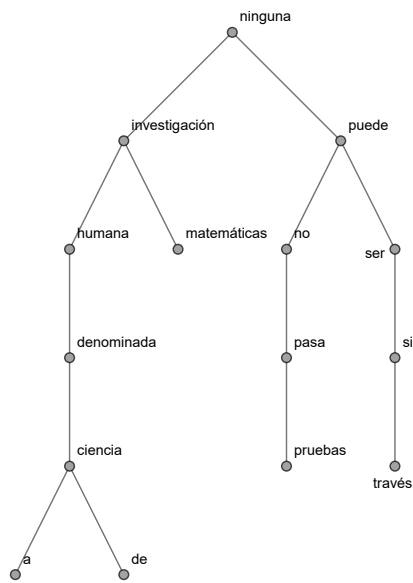
6.6.13

prefijo: $\{j, i, b, a, c, d, e, k, w, m, l, n, q, p, u, r, s, v, z\}$,
 interfijo: $\{a, b, c, d, e, i, j, k, l, m, n, p, q, r, s, u, v, w, z\}$ y
 postfijo: $\{a, e, d, c, b, i, l, p, s, r, v, u, q, n, m, z, w, k, j\}$.



6.6.14

prefijo: $\{en, deben, despreciar, diminutos, matemáticas, los, errores, más, no, ni, se\}$,
 interfijo: $\{deben, despreciar, diminutos, en, errores, los, más, matemáticas, ni, no, se\}$ y
 postfijo: $\{diminutos, despreciar, deben, errores, más, los, ni, se, no, matemáticas, en\}$.

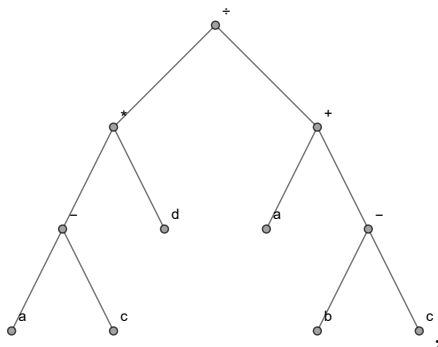


6.6.15

prefijo: {ninguna, investigación, humana, denominada, ciencia, a, de, matemáticas, puede, no, pasa, pruebas, ser, si, través},

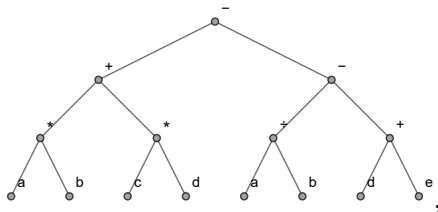
interfijo: {a, ciencia, de, denominada, humana, investigación, matemáticas, ninguna, pasa, pruebas, no, puede, ser, si, través} y

postfijo: {a, de, ciencia, denominada, humana, matemáticas, investigación, pruebas, pasa, no, través, si, ser, puede, ninguna}.



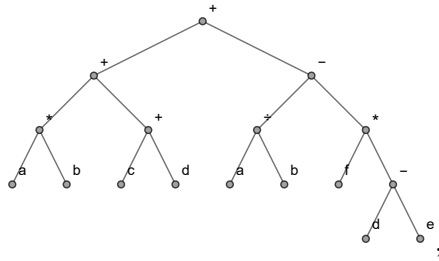
6.6.16

polaca: $\div \cdot -acd + a - bc$ y polaca inversa: $ac - d \cdot abc - + \div$.



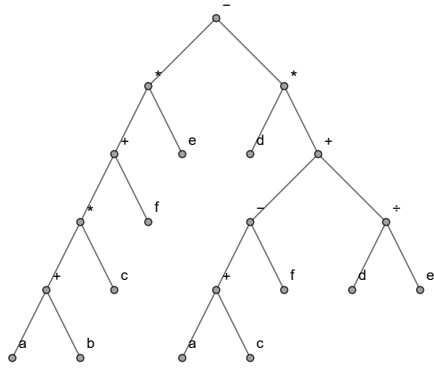
6.6.17

polaca: $- + \cdot ab \cdot cd - \div ab + de$ y polaca inversa: $ab \cdot cd \cdot + ab \div de + --$.



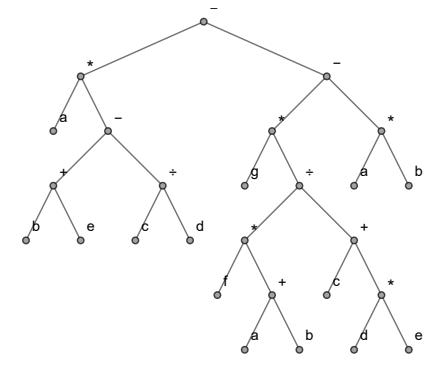
6.6.18

polaca: $++ \cdot ab + cd - \div ab \cdot f - de$ y polaca inversa: $ab \cdot cd + + ab \div fde - \cdot - +$.



6.6.19

polaca: $- \cdot + \cdot + ab c f e \cdot d + - + a c f \div de$ y polaca inversa: $ab + c \cdot f + e \cdot d a c + f - de \div + \cdot -$.



6.6.20

polaca: $- \cdot a - + be \div cd - \cdot g \div \cdot f + ab + c \cdot de \cdot ab$ y polaca inversa: $abe + cd \div - \cdot gfab + \cdot cde \cdot + \div \cdot ab \cdot - -$.

6.6.21 Grafo 3 : $BPA = \{[a, b], [a, c], [b, d], [c, e], [c, h], [d, f], [g, h]\}$ y

$BPL = \{[a, b], [b, c], [c, d], [d, e], [e, f], [f, g], [g, h]\}$.

Grafo 4 : $BPA = \{[a, d], [a, e], [a, f], [a, g], [a, h], [a, i], [c, d], [b, e]\}$ y

$BPL = \{[a, d], [c, d], [c, e], [b, e], [b, f], [f, i], [b, g], [g, h]\}$.

Grafo 5 : $BPA = \{[a, b], [a, d], [a, e], [c, d]\}$ y $BPL = \{[a, b], [b, d], [c, d], [c, e]\}$.

Grafo 6 : $BPA = \{[a, b], [a, d], [a, e], [a, f], [b, c]\}$ y $BPL = \{[a, b], [b, c], [c, d], [d, e], [c, f]\}$.

Grafo 7 : $BPA = \{[a, b], [a, c], [a, e], [a, g], [c, d], [c, f]\}$ y

$BPL = \{[a, b], [b, c], [c, d], [d, f], [e, f], [e, g]\}$.

Grafo 8 : $BPA = \{[a, b], [a, c], [a, d], [a, e], [b, f], [d, g]\}$ y

$BPL = \{[a, b], [b, c], [c, e], [d, e], [d, g], [f, g]\}$.

Grafo 9 : $BPA = \{[a, b], [a, c], [a, d], [a, e], [a, f], [a, g]\}$ y

$BPL = \{[a, b], [b, c], [c, d], [d, e], [e, f], [f, g]\}$.

Grafo 10 : $BPA = \{[a, b], [a, c], [a, d], [a, e], [a, f], [a, g]\}$ y

$BPL = \{[a, b], [b, d], [c, d], [c, e], [e, f], [e, g]\}$.

Grafo 11 : $BPA = \{[a, b], [a, d], [a, e], [a, f], [b, c], [b, g]\}$ y

$BPL = \{[a, b], [b, c], [c, d], [d, e], [e, f], [f, g]\}$.

Grafo 12 : $BPA = \{[a, b], [a, c], [a, d], [a, e], [a, f], [a, g], [b, h]\}$ y

$BPL = \{[a, b], [b, c], [c, d], [d, e], [e, f], [f, g], [g, h]\}$.

6.6.22 Grafo 3 : $Prim = \{[a, c], [c, e], [a, b], [b, d], [e, f], [e, h], [h, g]\}$ y $Kruskal = \{[c, e], [a, c], [b, d], [c, d], [g, h], [e, f]\}$.
Peso: 22.

Grafo 4 : $Prim = \{[a, d], [a, f], [f, i], [a, e], [a, g], [g, h], [g, b], [d, c]\}$ y

$Kruskal = \{[a, d], [a, f], [g, h], [f, i], [b, g], [a, e], [a, g], [c, e]\}$. Peso: 18.

Grafo 5 : $Prim = \{[a, b], [b, e], [e, c], [e, d]\}$ y $Kruskal = \{[c, e], [b, e], [a, b], [d, e]\}$. Peso: 5.

Grafo 6 : $Prim = \{[a, f], [f, c], [c, e], [c, b], [b, d]\}$ y $Kruskal = \{[c, f], [a, f], [b, d], [c, e], [b, c]\}$.
Peso: 9.

Grafo 7 : $Prim = \{[a, b], [a, c], [c, f], [a, e], [f, d], [a, g]\}$ y

$Kruskal = \{[a, b], [c, f], [a, c], [a, e], [d, f], [a, g]\}$. Peso: 12.

Grafo 8 : $Prim = \{[a, e], [e, d], [d, b], [b, f], [e, g], [a, c]\}$ y

$Kruskal = \{[b, f], [a, e], [e, g], [d, e], [b, d], [c, f]\}$. Peso: 7.

Grafo 9 : $Prim = \{[a, c], [a, f], [f, b], [b, d], [d, e], [d, g]\}$ y

$Kruskal = \{[a, f], [a, c], [b, f], [d, e], [d, g], [b, e]\}$. Peso: 10.

Grafo 10 : $Prim = \{[a, g], [g, c], [c, d], [d, b], [d, e], [c, f]\}$ y

$Kruskal = \{[d, e], [b, d], [d, f], [c, g], [c, f], [a, g]\}$. Peso: 11.

Grafo 11 : $Prim = \{[a, f], [f, g], [g, b], [b, d], [d, c], [g, e]\}$ y

$Kruskal = \{[b, g], [a, f], [e, g], [f, g], [c, d], [b, d]\}$. Peso: 10.

Grafo 12 : $Prim = \{[a, g], [g, c], [c, b], [b, d], [d, f], [g, h], [h, e]\}$ y

$Kruskal = \{[b, c], [b, d], [c, g], [a, g], [d, f], [e, h], [g, h]\}$. Peso: 14.

6.6.23 Grafo 3 : $BPA = \{[h, g], [h, f], [h, e], [h, c], [d, f], [b, c], [a, c]\}$ y

$BPL = \{[h, g], [g, f], [d, f], [b, d], [b, a], [a, c], [e, c]\}$.

Grafo 4 : $BPA = \{[i, b], [i, f], [i, a], [i, e], [g, b], [a, h], [a, d], [c, e]\}$ y

$BPL = \{[i, b], [g, b], [g, a], [f, a], [f, e], [c, e], [c, d], [a, h]\}$.

6.6.24 Grafo 3 : $Prim = \{[h, g], [h, f], [f, e], [e, c], [c, a], [a, b], [b, d]\}$. Peso: 22.

Grafo 4 : $Prim = \{[i, f], [f, a], [a, d], [i, b], [b, g], [g, h], [i, e], [e, c]\}$. Peso: 18.

6.6.25 Grafo 3 : $Prim = \{[h, c], [h, f], [f, d], [h, e], [f, g], [c, b], [b, a]\}$ y $Kruskal = \{[d, f], [c, h], [e, h], [d, e], [f, g], [b, c]\}$.
Peso: 34.

Grafo 4 : $Prim = \{[i, b], [b, e], [e, c], [c, d], [d, g], [e, f], [i, a], [a, h]\}$ y

$Kruskal = \{[d, g], [d, e], [a, h], [c, d], [b, e], [e, f], [a, g], [a, i]\}$. Peso: 38.

6.6.26 Se debe correr para estudiar los grafos completos:

`In[ ] :=`

```
Table[i -> BuscarPrimeroAncho[GrafoCompleto[i], orden ->
Range[i]], {i, 2, 10}]
```

```
Table[i -> BuscarPrimeroLargo[GrafoCompleto[i], orden ->
Range[i]], {i, 2, 10}]
```

En los otros tipos de grafos se realiza lo mismo, cambiando `GrafoCompleto` en el código por:

GrafoRueda, GrafoEstrella, GrafoCiclo o GrafoCamino, según corresponda.

Grafo completo y grafo rueda: $BPA = \{[1, 2], [1, 3], \dots, [1, n]\}$ y

$BPL = \{[1, 2], [2, 3], \dots, [n-1, n]\}$.

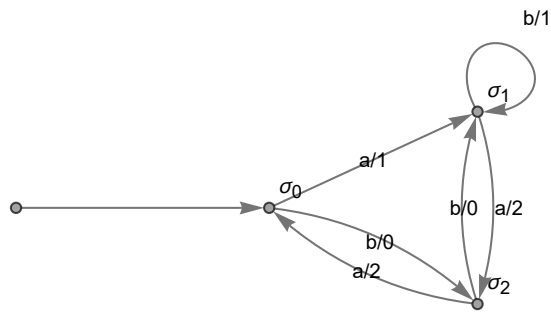
Grafo estrella: $BPA = BPL = \{[1, 2], [1, 3], \dots, [1, n]\}$.

Grafo ciclo: $BPA = \{[1, 2], [1, n], [2, 3], [n-1, n], [3, 4], [n-2, n-1], [4, 5], [n-3, n-2], \dots\}$
y $BPL = \{[1, 2], [2, 3], \dots, [n-1, n]\}$.

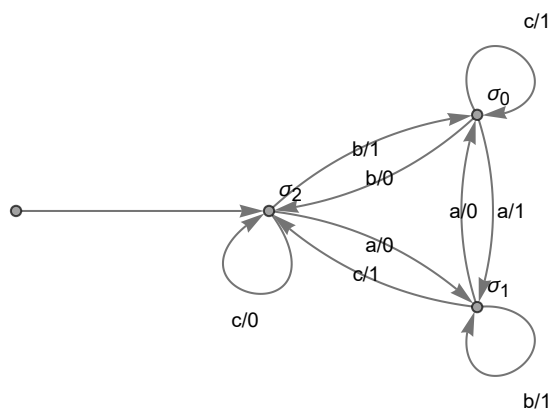
Grafo camino: $BPA = BPL = \{[1, 2], [2, 3], \dots, [n-1, n]\}$.

RESPUESTAS • SECCIÓN 7.5

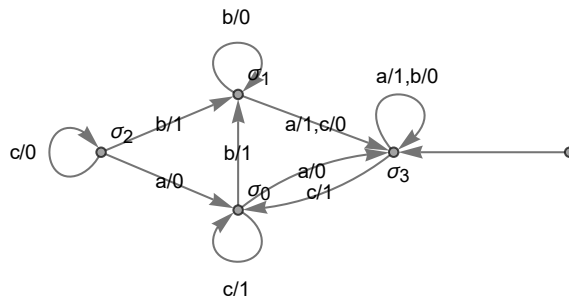
Máquinas de estado finito y autómatas



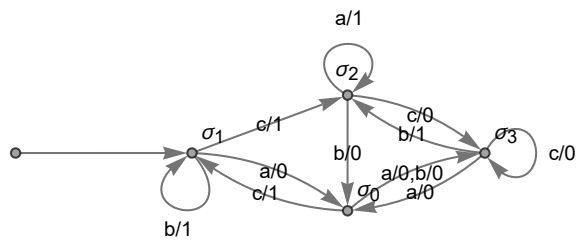
7.5.1



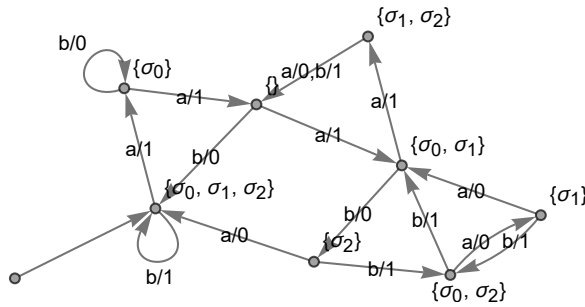
7.5.2



7.5.3



7.5.4



7.5.5

- 7.5.6 $M_1 : \{\{0, 2, 1, 2, 2, 1, 1, 1, 1, 1, 2, 0, 2, 2, 1, 1, 2, 0, 2\}, \{\sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_1, \sigma_2\}\}$.
 $M_2 : \{\{1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0\}, \{\sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_1\}\}$.
 $M_3 : \{\{0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1\}, \{\sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3\}\}$.
 $M_4 : \{\{1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0\}, \{\sigma_1, \sigma_1, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_0, \sigma_3, \sigma_2, \sigma_2, \sigma_0, \sigma_3\}\}$.
 $M_5 : \{\{1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0\}, \{\{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_1, \sigma_2\}, \phi, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_1, \sigma_2\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}\}\}$.

- 7.5.7 $M_1 : \{\{1, 2, 2, 0, 2, 0, 0, 2, 2, 1, 1, 2, 2, 1, 2, 0, 2, 0, 2, 2\}, \{\sigma_0, \sigma_1, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_2, \sigma_1, \sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_0\}\}$.
 $M_2 : \{\{0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0\}, \{\sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_0\}\}$.
 $M_3 : \{\{1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1\}, \{\sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3\}\}$.
 $M_4 : \{\{0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0\}, \{\sigma_1, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_3\}\}$.
 $M_5 : \{\{1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1\}, \{\{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi\}\}$.

- 7.5.8 $M_1 : \{\{1, 1, 2, 2, 0, 2, 0, 2, 1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2\}, \{\sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_1, \sigma_2, \sigma_0, \sigma_2, \sigma_1, \sigma_2\}\}$.
 $M_2 : \{\{0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0\}, \{\sigma_2, \sigma_1, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_1, \sigma_0\}\}$.
 $M_3 : \{\{1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1\}, \{\sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3\}\}$.
 $M_4 : \{\{0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0\}, \{\sigma_1, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_2, \sigma_0, \sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_2, \sigma_0, \sigma_3, \sigma_0\}\}$.
 $M_5 : \{\{1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0\}, \{\{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0, \sigma_1, \sigma_2\}, \{\sigma_0\}, \{\sigma_0\}, \phi, \{\sigma_0, \sigma_1\}, \{\sigma_2\}, \{\sigma_0, \sigma_2\}, \{\sigma_1\}\}\}$.

- 7.5.9 $M_2 : \{\{1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1\}, \{\sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_0, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_2, \sigma_2, \sigma_0, \sigma_2, \sigma_1, \sigma_1, \sigma_1, \sigma_1\}\}$.
 $M_3 : \{\{0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0\}, \{\sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_0, \sigma_1, \sigma_3,$

$\sigma_3, \sigma_3, \sigma_3, \sigma_0, \sigma_0, \sigma_0, \sigma_1, \sigma_1, \sigma_3, \sigma_3, \sigma_3, \sigma_3\}$.

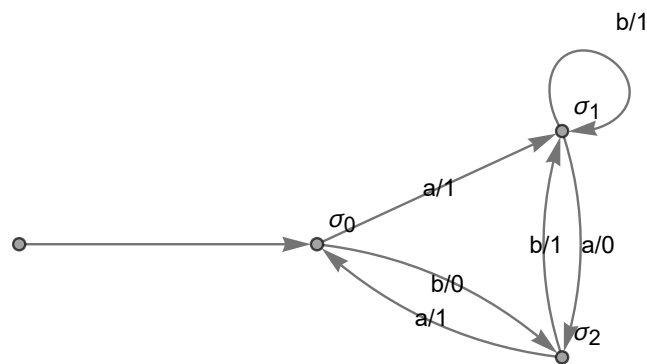
$M_4 : \{\{1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1\}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_0, \sigma_3, \sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_2, \sigma_0, \sigma_3, \sigma_2, \sigma_0, \sigma_3, \sigma_2\}\}$.

7.5.10 $M_2 : \{\{0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0\}, \{\sigma_2, \sigma_2, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_2, \sigma_1, \sigma_0, \sigma_2, \sigma_2, \sigma_1, \sigma_0\}\}$.

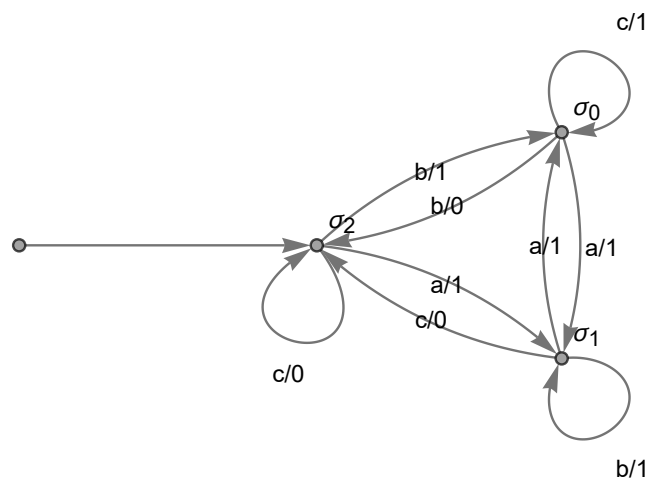
$M_3 : \{\{1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1\}, \{\sigma_3, \sigma_0, \sigma_1, \sigma_3, \sigma_0, \sigma_0, \sigma_3, \sigma_3, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_1, \sigma_1, \sigma_3, \sigma_3, \sigma_3, \sigma_0, \sigma_0, \sigma_3, \sigma_3\}\}$.

$M_4 : \{\{1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0\}, \{\sigma_1, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_3, \sigma_0, \sigma_3\}\}$.

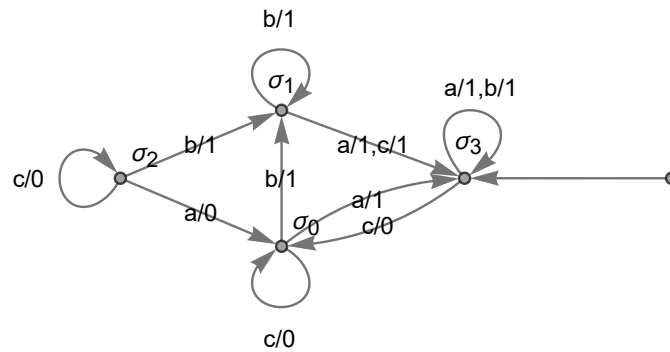
7.5.11 M_1 : no es autómeta, pues δ contiene el 2. Una modificación sería:



M_2 : no es autómeta, pues por ejemplo en σ_2 hay dos aristas entrantes con símbolo de salida 0 y una con símbolo de salida 1. Una modificación sería:

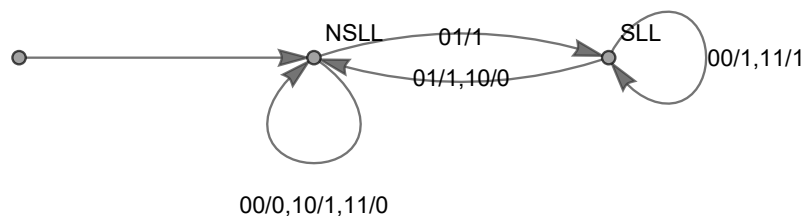
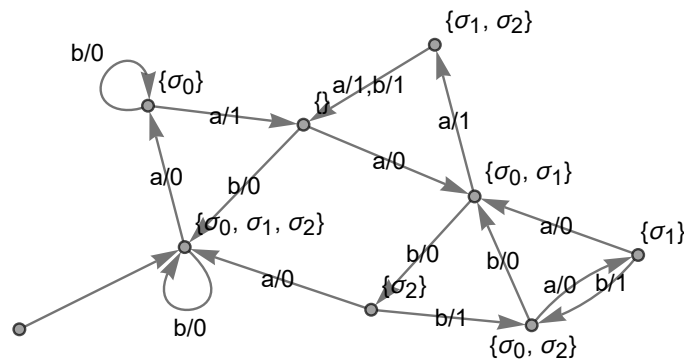


M_3 : no es autómeta, pues por ejemplo en σ_1 hay dos aristas entrantes con símbolo de salida 1 y una con símbolo de salida 0. Una modificación sería:



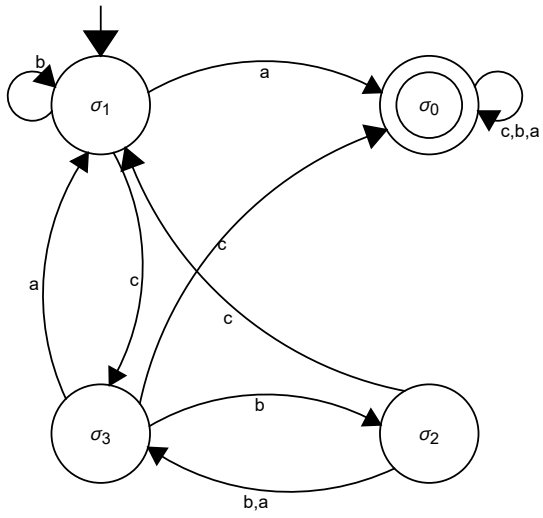
M_4 : es un autómata de estado finito, pues $\delta = \{0, 1\}$ y todas las aristas entrantes a un mismo estado tienen el mismo símbolo de salida. Los estados aceptados son σ_1 y σ_2 .

M_5 : no es autómata, pues por ejemplo en $\{\sigma_0\}$ hay una arista entrante con símbolo de salida 0 y otra con símbolo de salida 1. Una modificación sería:

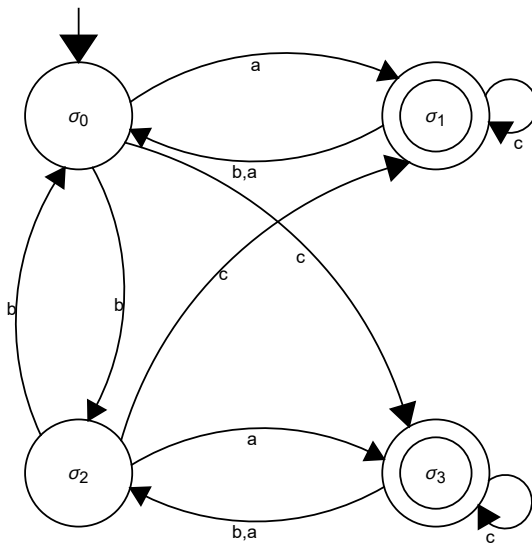


7.5.12

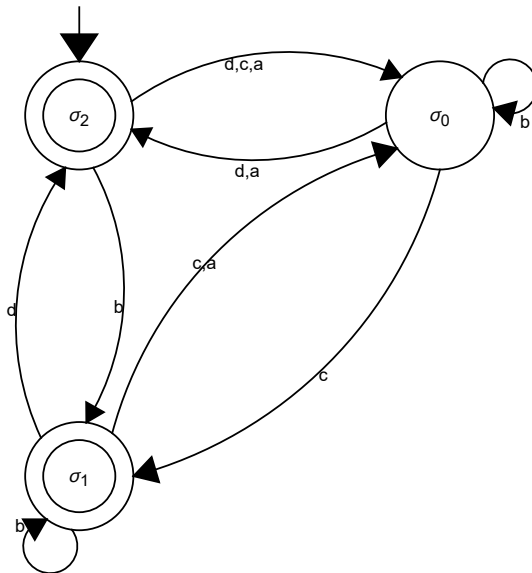
al procesar $\alpha = \{10, 01, 11, 10, 11, 01, 11, 10\}$ se obtiene: $\{\{1, 1, 1, 0, 0, 1, 1, 0\}, \{\text{NSLL}, \text{NSLL}, \text{SLL}, \text{SLL}, \text{NSLL}, \text{NSLL}, \text{SLL}, \text{SLL}, \text{NSLL}\}\}$, como consecuencia $(11011101)_2 - (1110110)_2 = (1100111)_2$. En la máquina nunca se debe finalizar en el estado “SLL”. Si esto ocurre la resta binaria brinda un resultado negativo y la máquina solo funciona en caso contrario.



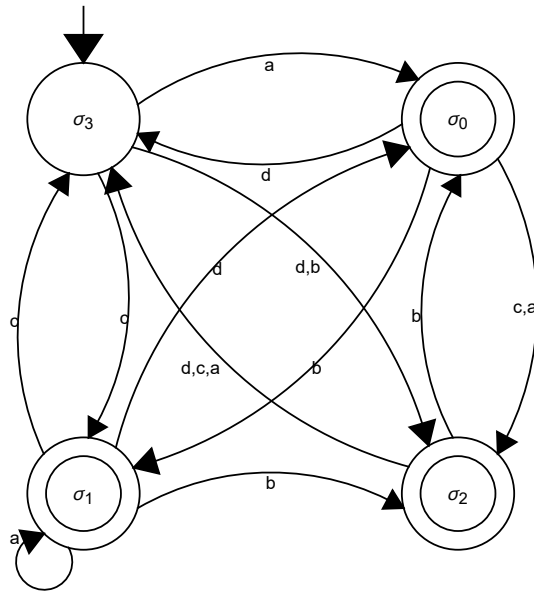
7.5.13



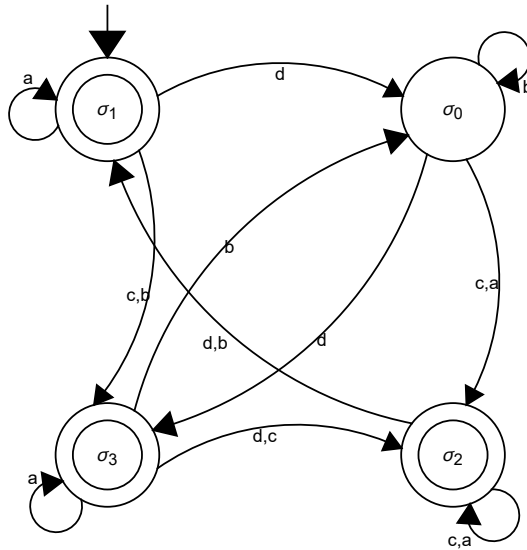
7.5.14



7.5.15



7.5.16



7.5.17

- 7.5.18 $A_1 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}$.
 $A_2 : \{\text{True}, \{\sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_3, \sigma_3, \sigma_2, \sigma_1, \sigma_1, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_3\}\}$.
 $A_3 : \{\text{False}, \{\sigma_2, \sigma_1, \sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_0, \sigma_1, \sigma_0, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_1, \sigma_0\}\}$.
 $A_4 : \{\text{True}, \{\sigma_3, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_1, \sigma_2, \sigma_3, \sigma_1, \sigma_3, \sigma_2, \sigma_0, \sigma_2\}\}$.
 $A_5 : \{\text{True}, \{\sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_3, \sigma_3\}\}$.

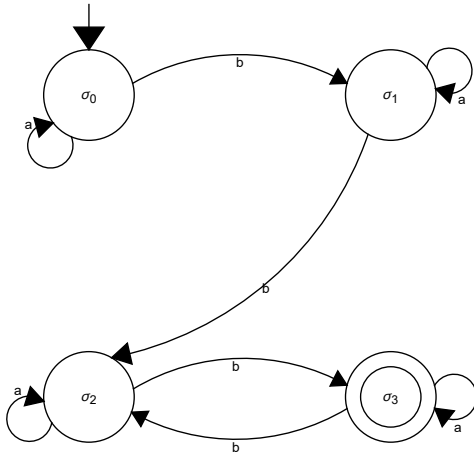
- 7.5.19 $A_1 : \{\text{True}, \{\sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}$.
 $A_2 : \{\text{False}, \{\sigma_0, \sigma_1, \sigma_0, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_0, \sigma_3, \sigma_3, \sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_0\}\}$.
 $A_3 : \{\text{True}, \{\sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_1\}\}$.
 $A_4 : \{\text{True}, \{\sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_1, \sigma_2, \sigma_3, \sigma_0, \sigma_1, \sigma_3, \sigma_1, \sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_1, \sigma_1, \sigma_1, \sigma_2, \sigma_0\}\}$.
 $A_5 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_1, \sigma_1, \sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_1, \sigma_3\}\}$.

- 7.5.20 $A_3 : \{\text{False}, \{\sigma_2, \sigma_0, \sigma_1, \sigma_1, \sigma_0, \sigma_0, \sigma_2, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0\}\}$.
 $A_4 : \{\text{True}, \{\sigma_3, \sigma_1, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_1\}\}$.
 $A_5 : \{\text{True}, \{\sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_1, \sigma_3, \sigma_3, \sigma_0, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_3, \sigma_2\}\}$.

7.5.21 $A_3 : \{\text{False}, \{\sigma_2, \sigma_0, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_0, \sigma_1, \sigma_0, \sigma_1, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0\}\}$.
 $A_4 : \{\text{False}, \{\sigma_3, \sigma_1, \sigma_2, \sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_0, \sigma_2, \sigma_0, \sigma_2, \sigma_3, \sigma_1, \sigma_0, \sigma_2, \sigma_3, \sigma_0, \sigma_3, \sigma_0, \sigma_2, \sigma_3\}\}$.
 $A_5 : \{\text{True}, \{\sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_1, \sigma_3, \sigma_3, \sigma_2, \sigma_1, \sigma_3, \sigma_3, \sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}$.

7.5.22 $A_3 : \{\text{True}, \{\sigma_2, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_2, \sigma_1, \sigma_1, \sigma_0, \sigma_2, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_1, \sigma_0, \sigma_2, \sigma_0, \sigma_0, \sigma_2\}\}$.
 $A_4 : \{\text{False}, \{\sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_1, \sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_2, \sigma_0, \sigma_2, \sigma_3, \sigma_0, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_3\}\}$.
 $A_5 : \{\text{True}, \{\sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_3, \sigma_2, \sigma_1, \sigma_3, \sigma_0, \sigma_2, \sigma_1, \sigma_3, \sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_1, \sigma_0, \sigma_0, \sigma_3\}\}$.

7.5.23 $A_1 : b^n a c^m, n \geq 0, m \geq 0$ y $b^n c b^m c a, n \geq 0, m$ impar, con $n, m \in \mathbb{N} \cup \{0\}$.
 $A_2 : b^n a c^m, n$ impar, $m \geq 0$ y $(ab)^n a c^m, n \geq 0, m \geq 0$, con $n, m \in \mathbb{N} \cup \{0\}$.
 $A_3 : (bd)^n b^m, n \geq 0, m \geq 1$ y $d^n b^m c, n$ impar, $m \geq 0$, con $n, m \in \mathbb{N} \cup \{0\}$.
 $A_4 : (ad)^n a (bd)^m, n \geq 0, m \geq 0$ y $c^n a (cb)^m a, n$ par, $m \geq 0$, con $n, m \in \mathbb{N} \cup \{0\}$.
 $A_5 : a^n b a^m c^h$ y $db^n (db)^m a^h$, con $n \geq 0, m \geq 0, h \geq 1$ y $n, m, h \in \mathbb{N} \cup \{0\}$.



7.5.24

In[] :=

```
Prueba[automata_, n_] := Module[{L = LenguajeStrings[automata,
n, limite -> True], valorLogico = True, m}, m = Length[L];
If[ToString[Select[Table[Count[L[[i]], b], {i, m}], EvenQ[#]
&]] != "{}", valorLogico = False]; Print[m, " pruebas: ",
valorLogico]]
Prueba[aut, 20]
```

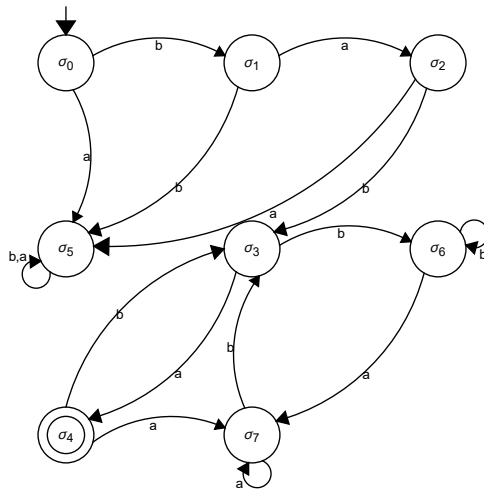
Out[] =

1048365 pruebas: True

Componentes del autómata:

- Estados: $\{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$.
- Símbolos de entrada: $\{a, b\}$.
- Estado inicial: σ_0 .
- Estados aceptados: $\{\sigma_3\}$.
- Función de transición de estados:

Δ	a	b
σ_0	σ_0	σ_1
σ_1	σ_1	σ_2
σ_2	σ_2	σ_3
σ_3	σ_3	σ_2



7.5.25

In[] :=

```

Prueba[automata_, n_] := Module[{L = LenguajeStrings[automata,
n, limite -> True], valorLogico = True, i, m}, m = Length[L];
For[i = 1, i <= m, If[ToString[Take[L[[i]], 3]] != ToString[{b,
a, b}] || ToString[Take[L[[i]], -3]] != ToString[{a, b, a}],
valorLogico = False; Break[]]; i++]; Print[If[valorLogico ==
False, i, i - 1], " pruebas: ", valorLogico]]
Prueba[aut, 20]

```

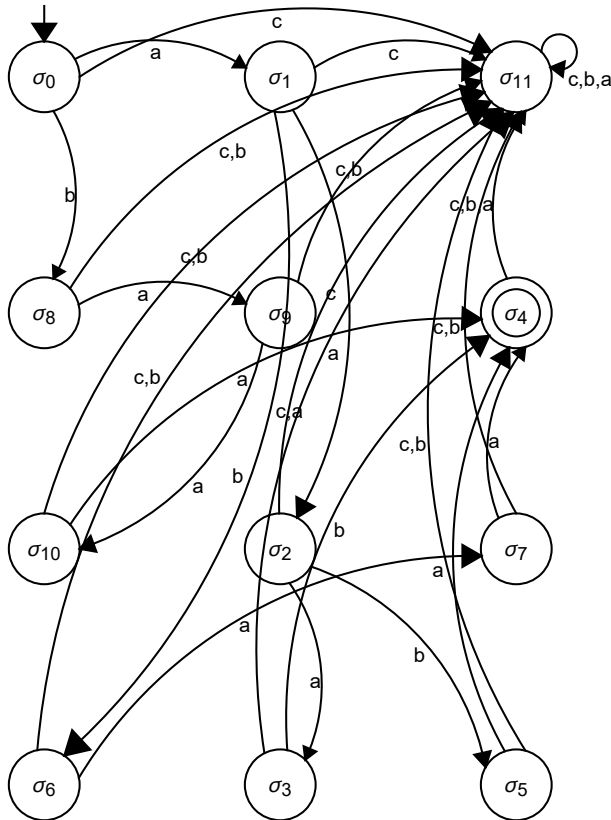
Out[] =

26215 pruebas: True

Componentes del autómata:

- Estados: $\{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4, \sigma_5, \sigma_6, \sigma_7\}$.
- Símbolos de entrada: $\{a, b\}$.
- Estado inicial: σ_0 .
- Estados aceptados: $\{\sigma_4\}$.
- Función de transición de estados:

Δ	a	b
σ_0	σ_5	σ_1
σ_1	σ_2	σ_5
σ_2	σ_5	σ_3
σ_3	σ_4	σ_6
σ_4	σ_7	σ_3
σ_5	σ_5	σ_5
σ_6	σ_7	σ_6
σ_7	σ_7	σ_3



7.5.26

In[] :=

```

Prueba[automata_, n_] := Module[{L = LenguajeStrings[automata,
n, limite -> True], valorLogico = True}, If[n >= 4,
If[ToString[L] != ToString[Permutations[{a, a, a, b}]],
valorLogico = False]; valorLogico, Print["La longitud debe ser
mayor o igual a 4'"]]]
Prueba[aut, 20]

```

Out[] =

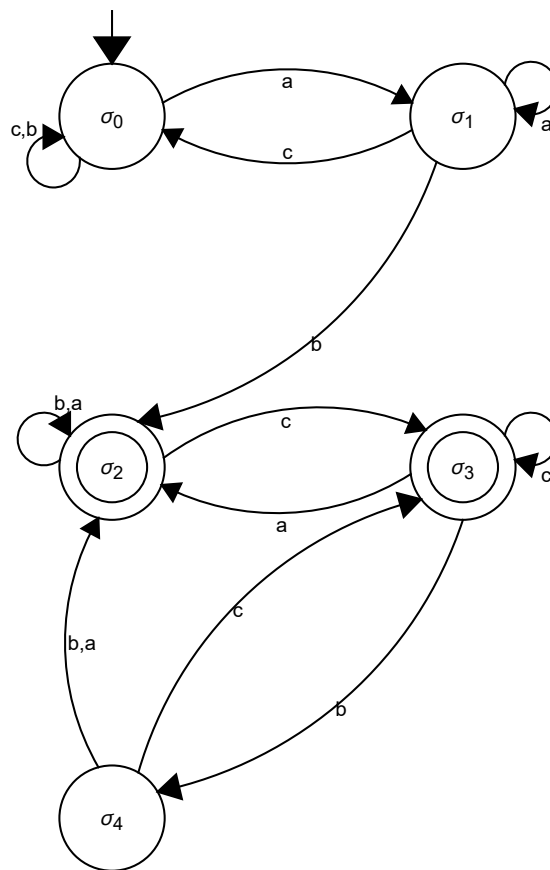
True

Componentes del autómata:

- Estados: $\{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4, \sigma_5, \sigma_6, \sigma_7, \sigma_8, \sigma_9, \sigma_{10}, \sigma_{11}\}$.

- Símbolos de entrada: $\{a, b, c\}$.
- Estado inicial: σ_0 .
- Estados aceptados: $\{\sigma_4\}$.
- Función de transición de estados:

Δ	a	b	c
σ_0	σ_1	σ_8	σ_{11}
σ_1	σ_2	σ_6	σ_{11}
σ_2	σ_3	σ_5	σ_{11}
σ_3	σ_{11}	σ_4	σ_{11}
σ_4	σ_{11}	σ_{11}	σ_{11}
σ_5	σ_4	σ_{11}	σ_{11}
σ_6	σ_7	σ_{11}	σ_{11}
σ_7	σ_4	σ_{11}	σ_{11}
σ_8	σ_9	σ_{11}	σ_{11}
σ_9	σ_{10}	σ_{11}	σ_{11}
σ_{10}	σ_4	σ_{11}	σ_{11}
σ_{11}	σ_{11}	σ_{11}	σ_{11}



7.5.27

In[] :=

```

Prueba[automata_, n_] := Module[{L = LenguajeStrings[automata,
n, limite -> True], valorLogico = True, i, m}, m = Length[L];

```

```

For[i = 1, i <= m, If[StringCount[StringJoin[ToString /@
L[[i]]], "ab"] == 0 || ToString[Take[L[[i]], -2]] == ToString[
{c, b}], valorLogico = False; Break[]]; i++]; Print[If[
valorLogico == False, i, i - 1], " pruebas: ", valorLogico]]
Prueba[aut, 13]

```

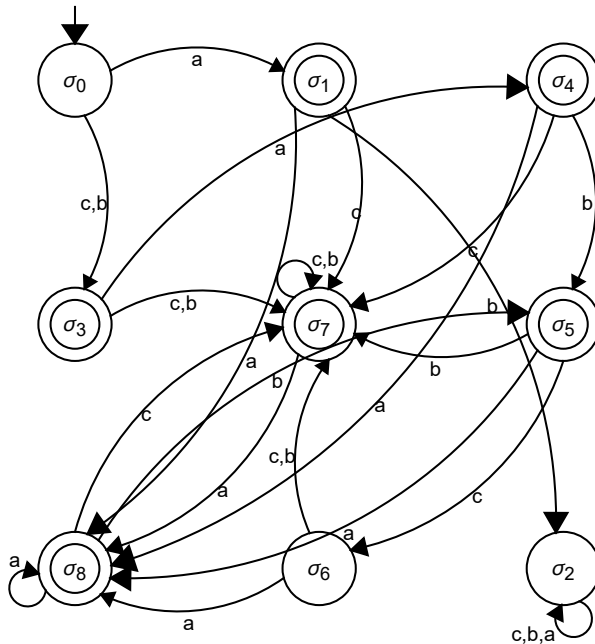
Out[] =

1686560 pruebas: True

Componentes del autómata:

- Estados: $\{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4\}$.
- Símbolos de entrada: $\{a, b, c\}$.
- Estado inicial: σ_0 .
- Estados aceptados: $\{\sigma_2, \sigma_3\}$.
- Función de transición de estados:

Δ	a	b	c
σ_0	σ_1	σ_0	σ_0
σ_1	σ_1	σ_2	σ_0
σ_2	σ_2	σ_2	σ_3
σ_3	σ_2	σ_4	σ_3
σ_4	σ_2	σ_2	σ_3



7.5.28

In[] :=

```

Prueba[automata_, n_] := Module[{L = LenguajeStrings[automata,
n, limite -> True], valorLogico = True, i, m}, m = Length[L];
For[i = 1, i <= m, If[Length[L[[i]]] == 2, If[ToString[L[[i]]]
== ToString[{a, b}], valorLogico = False; Break[]], If[

```

```

Length[L[[i]]] >= 3, If[ToString[Take[L[[i]], 2]] ==
ToString[{a, b}] || ToString[Take[L[[i]], -3]] == ToString[{a,
b, c}], valorLogico = False; Break[]]]]; i++]; Print[If[
valorLogico == False, i, i - 1], " pruebas: ", valorLogico]]
Prueba[aut, 13]
Out[ ] =

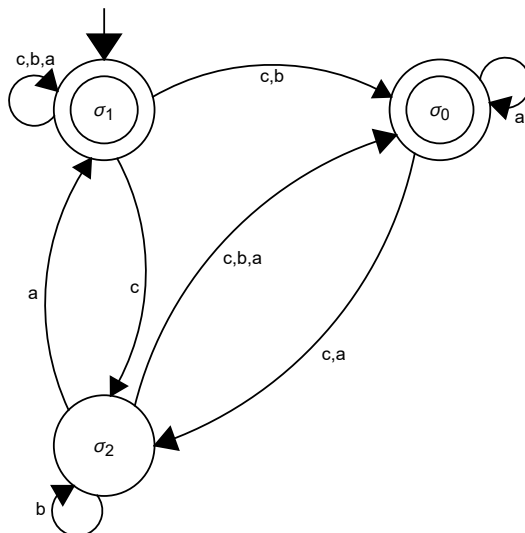
```

2047032 pruebas: True

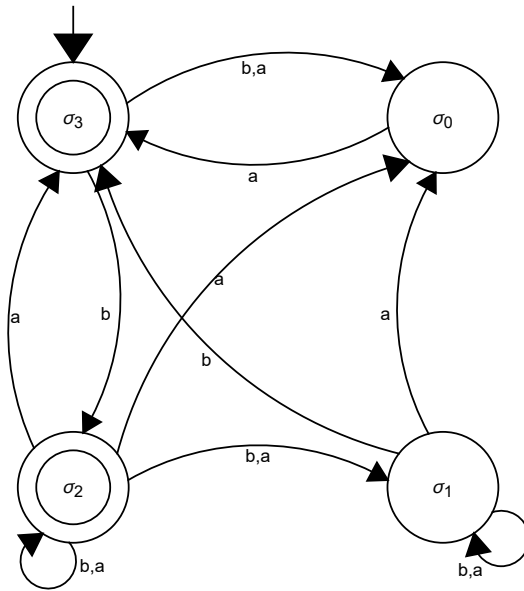
Componentes del autómata:

- Estados: $\{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4, \sigma_5, \sigma_6, \sigma_7, \sigma_8\}$.
- Símbolos de entrada: $\{a, b, c\}$.
- Estado inicial: σ_0 .
- Estados aceptados: $\{\sigma_1, \sigma_3, \sigma_4, \sigma_5, \sigma_7, \sigma_8\}$.
- Función de transición de estados:

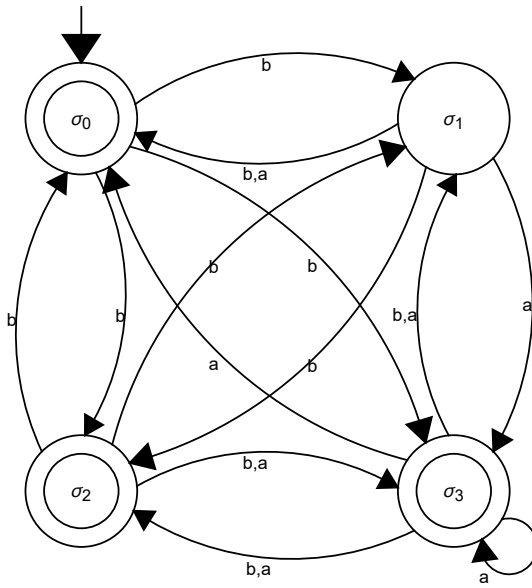
Δ	a	b	c
σ_0	σ_1	σ_3	σ_3
σ_1	σ_8	σ_2	σ_7
σ_2	σ_2	σ_2	σ_2
σ_3	σ_4	σ_7	σ_7
σ_4	σ_8	σ_5	σ_7
σ_5	σ_8	σ_7	σ_6
σ_6	σ_8	σ_7	σ_7
σ_7	σ_8	σ_7	σ_7
σ_8	σ_8	σ_5	σ_7



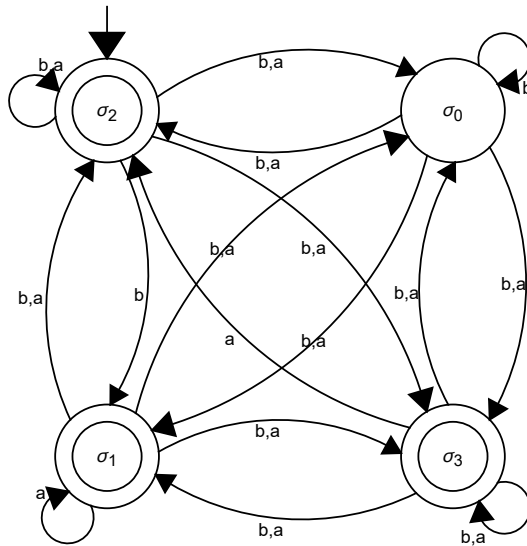
7.5.29



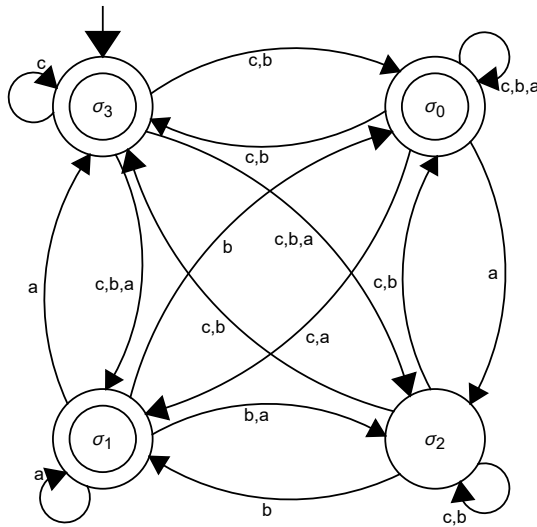
7.5.30



7.5.31



7.5.32



7.5.33

- 7.5.34 $A_6 : \{\text{True}, \{\sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}.$
 $A_7 : \{\text{True}, \{\sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}.$
 $A_8 : \{\text{True}, \{\sigma_0, \sigma_1, \sigma_3, \sigma_3, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_2, \sigma_3, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_3\}\}.$
 $A_9 : \{\text{True}, \{\sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}.$
 $A_{10} : \{\text{True}, \{\sigma_3, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}.$

7.5.35 $A_6 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1\}\}.$

$A_7 : \text{False}.$

$A_8 : \text{False}.$

$A_9 : \{\text{True}, \{\sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}.$

$A_{10} : \{\text{True}, \{\sigma_3, \sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}.$

7.5.36 $A_6 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1\}\}.$

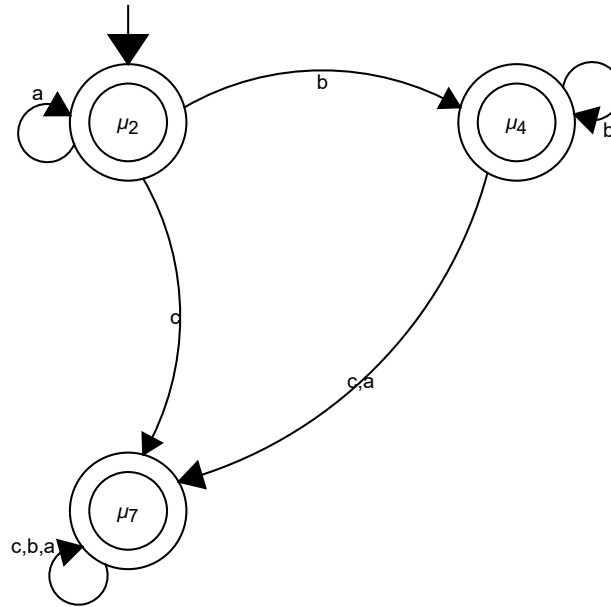
$A_7 : \{\text{True}, \{\sigma_3, \sigma_0, \sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}.$

$A_8 : \text{False}.$

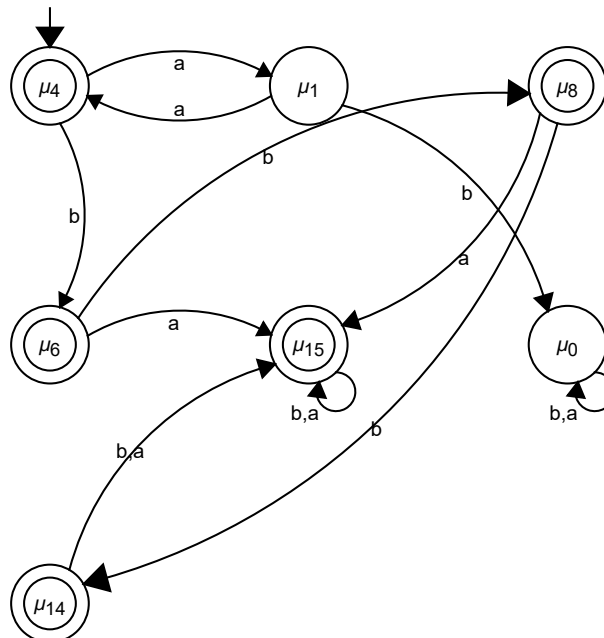
$A_9 : \{\text{True}, \{\sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}.$

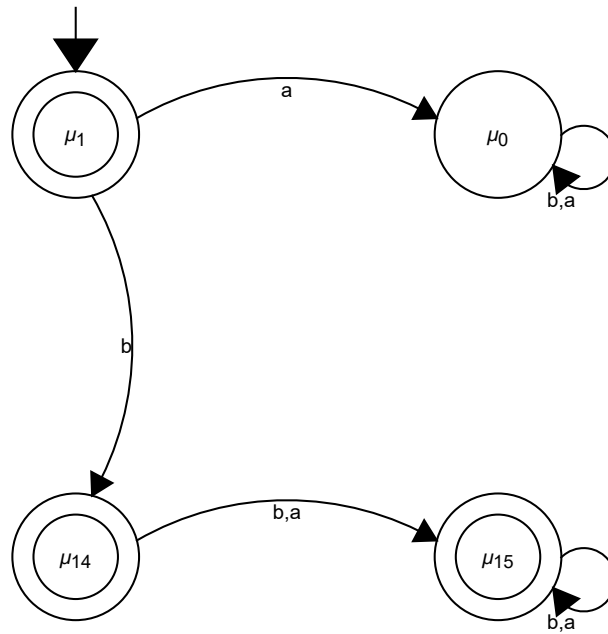
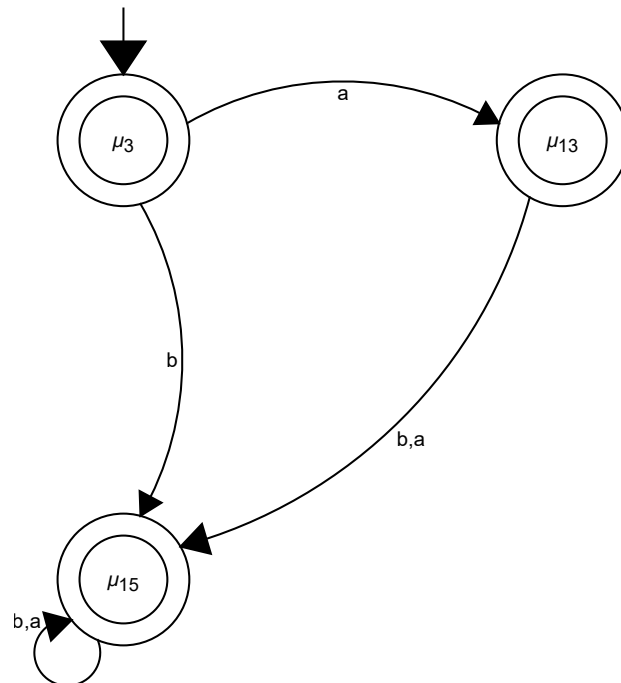
$A_{10} : \{\text{True}, \{\sigma_3, \sigma_1, \sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}.$

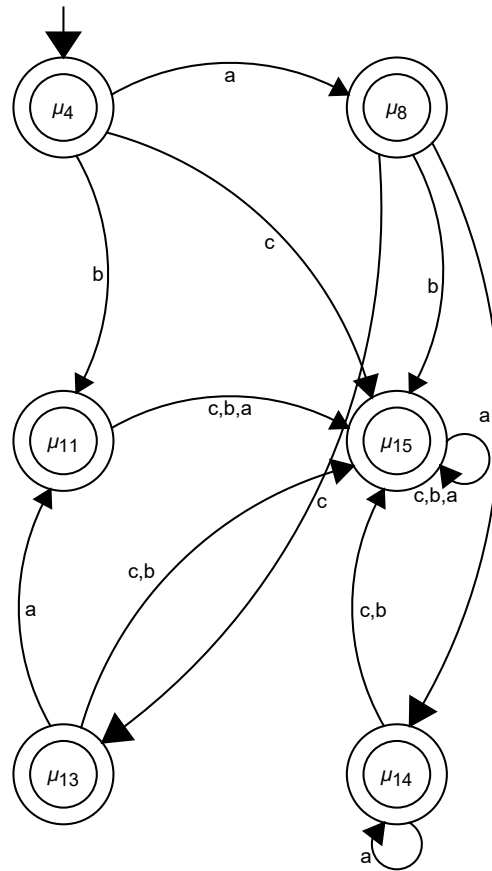
- 7.5.37** $A_6 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1\}\}$.
 $A_7 : \{\text{True}, \{\sigma_3, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}$.
 $A_8 : \{\text{True}, \{\sigma_0, \sigma_3, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_3, \sigma_2, \sigma_3, \sigma_3, \sigma_2\}\}$.
 $A_9 : \{\text{True}, \{\sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2, \sigma_2\}\}$.
 $A_{10} : \{\text{True}, \{\sigma_3, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}$.
- 7.5.38** $A_6 : \{\text{True}, \{\sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1, \sigma_1\}\}$.
 $A_{10} : \{\text{True}, \{\sigma_3, \sigma_1, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0, \sigma_0\}\}$.
- 7.5.39** $A_6 :$



$A_7 :$



$A_8 :$  $A_9 :$ 

A_{10} :

RESPUESTAS • SECCIÓN 8.4

Lenguajes y gramáticas

- 8.4.1** Hilera 1 : $S \rightarrow b^n S$ por 1 $\rightarrow b^n a A$ por 2 $\rightarrow b^n a a S$ por 3 $\rightarrow b^n a a b^m$ por 1 y 6, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow b^n S$ por 1 $\rightarrow b^n a A$ por 2 $\rightarrow b^n a b A$ por 4 $\rightarrow b^n a b^m a$ por 4 y 5, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
- 8.4.2** Hilera 1 : $S \rightarrow a A c$ por 1 $\rightarrow a^n A c$ por 2 $\rightarrow a^n A c^m$ por 3 $\rightarrow a^n b c^m$ por 4 y 5, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow a A c$ por 1 $\rightarrow a^n A c$ por 2 $\rightarrow a^n A c^m$ por 3 $\rightarrow a^n b^h c^m$ por 4, 5 y 6, $n \geq 1, m \geq 1, h \geq 1$, con $n, m, h \in \mathbb{N}$.
- 8.4.3** Hilera 1 : $S \rightarrow A$ por 1 $\rightarrow A B B$ por 6 $\rightarrow A b^n$ por 3 y 6 $\rightarrow a b^n$ por 2, con n un entero positivo par.
 Hilera 2 : $S \rightarrow A B A$ por 4 $\rightarrow B A A$ por 7 $\rightarrow B A b^n A b^m$ por 3 y 6 $\rightarrow b a b^n a b^m$ por 2 y 3, con n y m enteros positivos pares.
- 8.4.4** Hilera 1 : $S \rightarrow A A B$ por 2 $\rightarrow A A b^n$ por 6 y 7 $\rightarrow a a a a b^n$ por 4, $n \geq 1$, con $n \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow A A B$ por 2 $\rightarrow A A b$ por 7 $\rightarrow A A B a$ por 3 $\rightarrow A A b^n a$ por 6 y 7 $\rightarrow a a a a b^n a$ por 4, $n \geq 1$, con $n \in \mathbb{N}$.
- 8.4.5** Hilera 1 : $S \rightarrow B A B$ por 1 $\rightarrow B A b^{n-1}$ por 3 y 7 $\rightarrow b a b^n$ por 6 y 7, $n \geq 2$, con $n \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow A B A$ por 2 $\rightarrow A b A$ por 7 $\rightarrow a^{n-1} A b a^{n-1} A$ por 5 $\rightarrow a^n b b a^n b$ por 6, $n \geq 1$, con $n \in \mathbb{N}$.
- 8.4.6** Hilera 1 : $S \rightarrow a A$ por 4 $\rightarrow a b B$ por 1 $\rightarrow a b b C C$ por 3 $\rightarrow a b b a^{n-1} C a^{n-1} C$ por 6 $\rightarrow a b b a^n B a^n B$ por 7 $\rightarrow a b b a^n b a^n b$ por 2, $n \geq 1$, con $n \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow b B$ por 5 $\rightarrow b b C C$ por 3 $\rightarrow b b a^{n-1} C a^{m-1} C$ por 6 $\rightarrow b b a^n B a^m B$ por 7 $\rightarrow b b a^n b a^m b$ por 2, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
- 8.4.7** Hilera 1 : $S \rightarrow a^n S$ por 1 $\rightarrow a^n b A$ por 3 $\rightarrow a^n b c B$ por 4 $\rightarrow a^n b c b^m$ por 6 y 7, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow a^n S$ por 1 $\rightarrow a^n b A$ por 3 $\rightarrow a^n b c S$ por 5 $\rightarrow a^n b c a^m$ por 1 y 2, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
- 8.4.8** Hilera 1 : $S \rightarrow a^n S$ por 8 $\rightarrow a^n b A$ por 9 $\rightarrow a^n b^m A$ por 2 $\rightarrow a^n b^m a B$ por 1 $\rightarrow a^n b^m a a$ por 3, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow b A$ por 9 $\rightarrow b^n A$ por 2 $\rightarrow b^n a B$ por 1 $\rightarrow b^n a b A$ por 5 $\rightarrow b^n (a b)^m A$ por 1 y 5 $\rightarrow b^n (a b)^m a B$ por 1 $\rightarrow b^n (a b)^m a a$ por 3, $n \geq 1, m \geq 0$, con $n, m \in \mathbb{N} \cup \{0\}$.
- 8.4.9** Hilera 1 : $S \rightarrow a^n S$ por 1 $\rightarrow a^n b A$ por 2 $\rightarrow a^n b a B$ por 4 $\rightarrow a^n (b a)^m B$ por 6 y 8 $\rightarrow a^n (b a)^m b$ por 7, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow b A$ por 2 $\rightarrow b a B$ por 4 $\rightarrow b a^n B$ por 5 $\rightarrow b a^n b C$ por 6 $\rightarrow b a^n b a B$ por 8 $\rightarrow b a^n (b a)^m B$ por 6 y 8 $\rightarrow b a^n (b a)^m b$ por 7, $n \geq 1, m \geq 0$, con $n, m \in \mathbb{N} \cup \{0\}$.
- 8.4.10** Hilera 1 : $S \rightarrow b C$ por 3 $\rightarrow b a^{n-2} C$ por 10 $\rightarrow b a^{n-1} B$ por 9 $\rightarrow b a^n$ por 8, $n \geq 2$, con $n \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow a^{n-1} S$ por 1 $\rightarrow a^n A$ por 2 $\rightarrow a^n b^m A$ por 5 $\rightarrow a^n b^m a C$ por 6 $\rightarrow a^n b^m a a B$ por 9 $\rightarrow a^n b^m a a a$ por 8, $n \geq 1, m \geq 1$, con $n, m \in \mathbb{N}$.
- 8.4.11** Hilera 1 : $S \rightarrow a A$ por 1 $\rightarrow a b S$ por 3 $\rightarrow (a b)^{n-1} S$ por 1 y 3 $\rightarrow (a b)^{n-1} a A$ por 1 $\rightarrow (a b)^n$ por 4, $n \geq 1$, con $n \in \mathbb{N}$.
 Hilera 2 : $S \rightarrow a B$ por 2 $\rightarrow a^{n-1} B$ por 6 $\rightarrow a^n C$ por 10 $\rightarrow a^n b A$ por 12 $\rightarrow a^n b b S$ por

$3 \rightarrow a^n bbaB$ por $2 \rightarrow a^n bba^m B$ por $6 \rightarrow a^n bba^m b$ por 9 , $n \geq 1$, $m \geq 1$, con $n, m \in \mathbb{N}$.

8.4.12 Hilera 1 : $S \rightarrow aA$ por $18 \rightarrow aaG$ por $1 \rightarrow aab^n G$ por $17 \rightarrow aab^n a$ por 16 , $n \geq 1$, con $n \in \mathbb{N}$.
Hilera 2 : $S \rightarrow bD$ por $19 \rightarrow baE$ por $8 \rightarrow babC$ por $12 \rightarrow babaG$ por $6 \rightarrow babab^n G$ por $17 \rightarrow babab^n a$ por 16 , $n \geq 1$, con $n \in \mathbb{N}$.

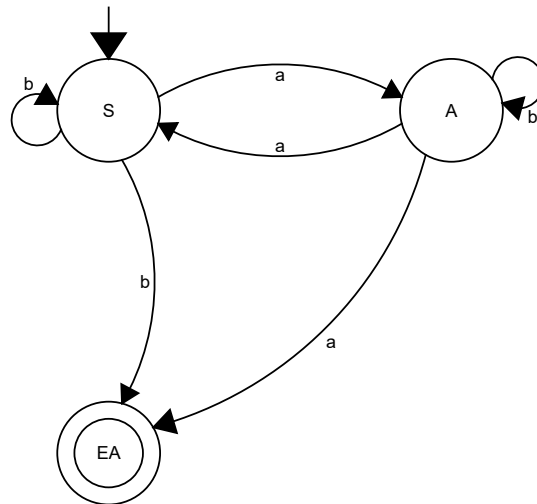
8.4.13 G_1 : regular. G_2 : libre del contexto. G_3 : ninguna, por la regla de composición $AB \rightarrow BA$. G_4 : ninguna, por la regla gramatical $Ab \rightarrow ABa$. G_5 : libre del contexto. G_6 : libre del contexto. G_7 : regular. G_8 : regular. G_9 : regular. G_{10} : regular. G_{11} : regular. G_{12} : regular.

8.4.14 $\langle \text{entero} \rangle ::= \langle \text{entero sin signo} \rangle | - \langle \text{entero sin signo} \rangle$
 $\langle \text{entero sin signo} \rangle ::= \langle \text{dígito} \rangle | \langle \text{dígito} \rangle \langle \text{entero sin signo} \rangle$
 $\langle \text{dígito} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$
 $\sigma^* = \langle \text{entero} \rangle$

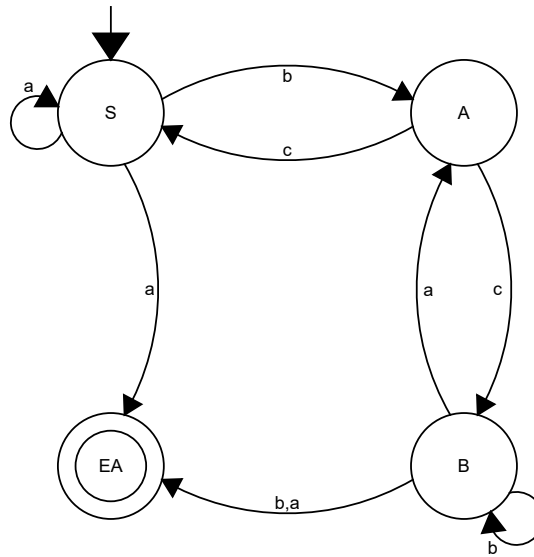
8.4.15 $\langle \text{entero} \rangle ::= - \langle \text{primer dígito} \rangle$
 $\langle \text{entero} \rangle ::= - \langle \text{entero sin signo} \rangle \langle \text{primer dígito} \rangle$
 $\langle \text{entero sin signo} \rangle ::= \langle \text{dígito} \rangle | \langle \text{dígito} \rangle \langle \text{entero sin signo} \rangle$
 $\langle \text{primer dígito} \rangle ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$
 $\langle \text{dígito} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$
 $\sigma^* = \langle \text{entero} \rangle$

8.4.16 $\langle \text{oración} \rangle ::= \langle \text{sujeto} \rangle b \langle \text{predicado} \rangle$.
 $\langle \text{sujeto} \rangle ::= \text{Adriana} | \text{José}$
 $\langle \text{predicado} \rangle ::= \langle \text{verbo} \rangle b \langle \text{complemento} \rangle b \langle \text{adverbio} \rangle$
 $\langle \text{verbo} \rangle ::= \text{analiza} | \text{construye}$
 $\langle \text{complemento} \rangle ::= \text{algoritmos}$
 $\langle \text{adverbio} \rangle ::= \text{constantemente} | \text{rápidamente}$
 $\sigma^* = \langle \text{oración} \rangle$
 b simboliza el espacio en blanco.

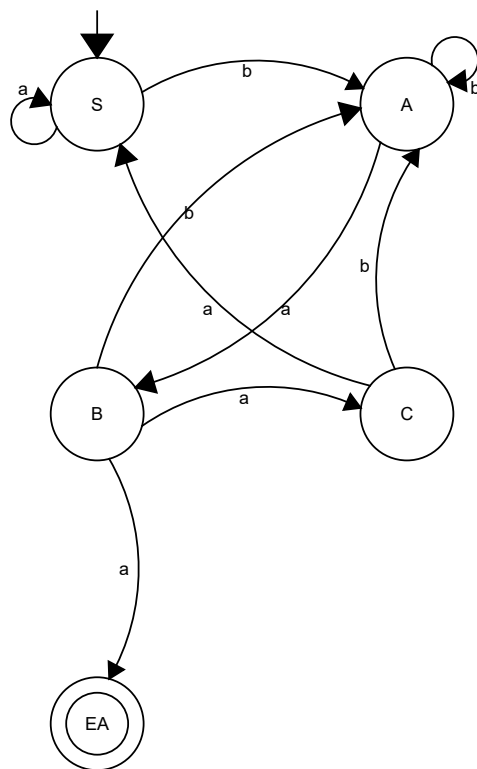
8.4.17 G_1 :



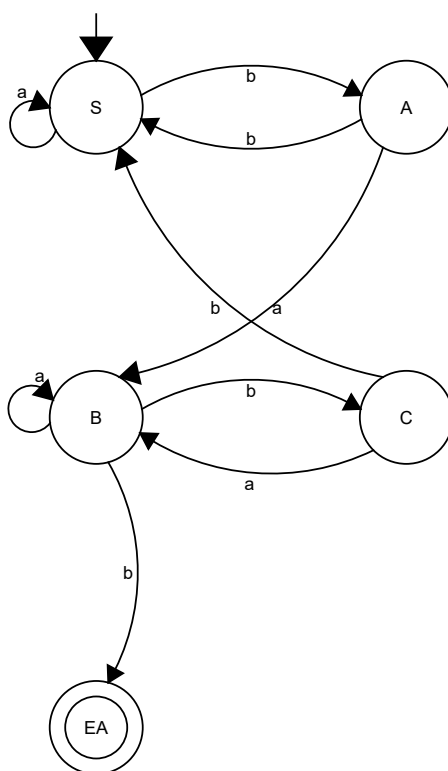
G_7 :

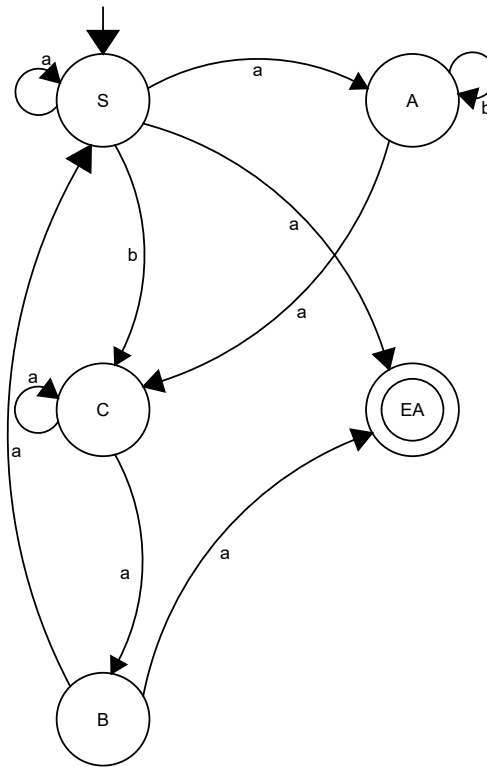
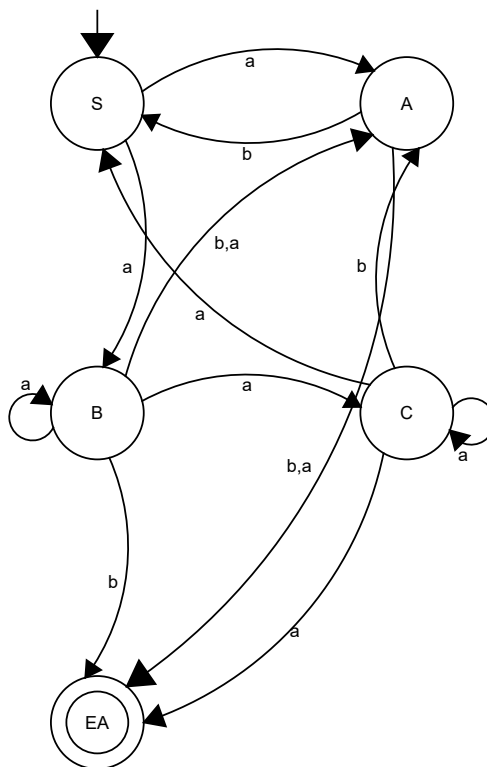


$G_8 :$

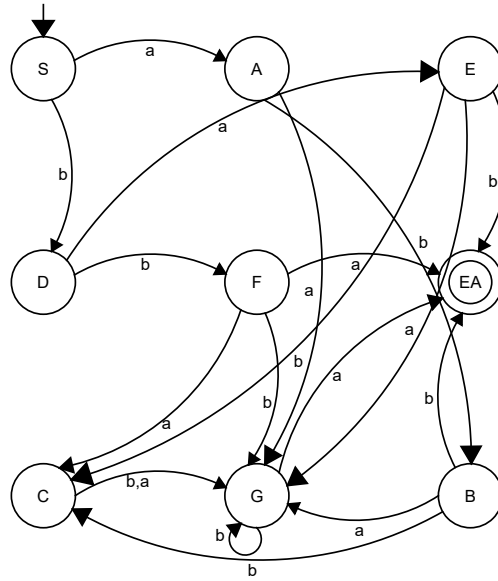


$G_9 :$



G_{10} : G_{11} :

G_{12} :



8.4.18 $G = (\sigma, \tau, P, \sigma^*)$:

- Símbolos no terminales: $\{S, A, B, C\}$.
- Símbolos terminales: $\{a, b, c\}$.
- Producciones o reglas de composición: $\{A \rightarrow aA, A \rightarrow bB, A \rightarrow cS, B \rightarrow aA, B \rightarrow bS, B \rightarrow c, B \rightarrow cC, C \rightarrow aA, C \rightarrow bS, C \rightarrow cS, S \rightarrow aA, S \rightarrow bS, S \rightarrow cS\}$.
- $\sigma^* = S$.

8.4.19 $G = (\sigma, \tau, P, \sigma^*)$:

- Símbolos no terminales: $\{S, A, B, C, D, E\}$.
- Símbolos terminales: $\{a, b\}$.
- Producciones o reglas de composición: $\{A \rightarrow aD, A \rightarrow bB, B \rightarrow a, B \rightarrow aC, B \rightarrow bB, C \rightarrow aE, C \rightarrow bB, D \rightarrow aD, D \rightarrow bD, E \rightarrow aE, E \rightarrow bB, S \rightarrow aA, S \rightarrow bD\}$.
- $\sigma^* = S$.

8.4.20 $G = (\sigma, \tau, P, \sigma^*)$:

- Símbolos no terminales: $\{S, A, B, C, D, E\}$.
- Símbolos terminales: $\{a, b\}$.
- Producciones o reglas de composición: $\{A \rightarrow aD, A \rightarrow bB, B \rightarrow a, B \rightarrow aC, B \rightarrow bA, C \rightarrow aB, C \rightarrow bD, D \rightarrow aA, D \rightarrow b, D \rightarrow bC, E \rightarrow aS, E \rightarrow bD, S \rightarrow aE, S \rightarrow bA\}$.
- $\sigma^* = S$.

8.4.21 $G = (\sigma, \tau, P, \sigma^*)$:

- Símbolos no terminales: $\{S, A, B, C, D, E, F\}$.
- Símbolos terminales: $\{a, b\}$.
- Producciones o reglas de composición: $\{A \rightarrow aB, A \rightarrow b, A \rightarrow bF, B \rightarrow aB, B \rightarrow bB, C \rightarrow a, C \rightarrow aC, C \rightarrow b, C \rightarrow bD, D \rightarrow a, D \rightarrow aC, D \rightarrow bE, E \rightarrow a, E \rightarrow aC, E \rightarrow b, E \rightarrow bF, F \rightarrow a, F \rightarrow aC, F \rightarrow b, F \rightarrow bF, S \rightarrow a, S \rightarrow aC, S \rightarrow b, S \rightarrow bA\}$.

- $\sigma^* = S$.

8.4.22 $G = (\sigma, \tau, P, \sigma^*) :$

- Símbolos no terminales: $\{S, A, B, C, D, E, F, G, H, I, J, K\}$.
- Símbolos terminales: $\{a, b, c\}$.
- Producciones o reglas de composición: $\{A \rightarrow aK, A \rightarrow bB, A \rightarrow cD, B \rightarrow aK, B \rightarrow bK, B \rightarrow c, B \rightarrow cC, C \rightarrow aK, C \rightarrow bK, C \rightarrow cK, D \rightarrow aK, D \rightarrow b, D \rightarrow bC, D \rightarrow cK, E \rightarrow aF, E \rightarrow bK, E \rightarrow cG, F \rightarrow aK, F \rightarrow bK, F \rightarrow c, F \rightarrow cC, G \rightarrow a, G \rightarrow aC, G \rightarrow bK, G \rightarrow cK, H \rightarrow aI, H \rightarrow bJ, H \rightarrow cK, I \rightarrow aK, I \rightarrow b, I \rightarrow bC, I \rightarrow cK, J \rightarrow a, J \rightarrow aC, J \rightarrow bK, J \rightarrow cK, K \rightarrow aK, K \rightarrow bK, K \rightarrow cK, S \rightarrow aA, S \rightarrow bE, S \rightarrow cH\}$.
- $\sigma^* = S$.

TERCERA PARTE

Ejercicios complementarios, resueltos

CAPÍTULO 1 • COMPLEMENTARIOS

Recursividad

1.1 La idea de recursividad: factorial y Fibonacci

- 1.6.1 (★) **A MANO** Escriba la sucesión de llamadas que genera `Factoriales[5]` desde la primera invocación hasta el caso raíz, e indique en qué orden se resuelven al regresar de la pila.

SOLUCIÓN La bajada apila una llamada por cada valor, hasta llegar al caso raíz:

$$5! \rightarrow 5 \cdot 4! \rightarrow 5 \cdot 4 \cdot 3! \rightarrow 5 \cdot 4 \cdot 3 \cdot 2! \rightarrow 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1!$$

El caso raíz $1! = 1$ detiene la bajada. La subida resuelve en orden *inverso*: $1! = 1$, $2! = 2$, $3! = 6$, $4! = 24$ y por último $5! = 120$. Son cinco llamadas apiladas y cinco desapiladas.

- 1.6.2 (★) **VilCretas** Calcule $8!$ y $12!$ con `Factoriales` y compare cuántas cifras tiene cada resultado.

SOLUCIÓN

`In[ ] :=`

`Factoriales[8]`
`Factoriales[12]`

`Out[ ] =`

40320
 479001600

$8! = 40320$ tiene 5 cifras y $12! = 479001600$ tiene 9 cifras. Basta pasar de 8 a 12 para casi duplicar la longitud del resultado: el factorial crece más rápido que cualquier potencia de base fija.

- 1.6.3 (★) **A MANO** En la definición `Factoriales[n_] := If[n==1, 1, n*Factoriales[n-1]]`, señale cuál es la definición base y cuál el caso raíz.

SOLUCIÓN La **definición base** es `n*Factoriales[n-1]`: la regla que reduce el problema a uno más pequeño. El **caso raíz** es `n==1` con valor 1: la condición que detiene la bajada y permite que la pila empiece a devolver resultados.

- 1.6.4 (★★) **MIXTO** Calcule a mano el término F_{12} de la sucesión de Fibonacci a partir de $F_1 = F_2 = 1$ y verifíquelo con `NFibonacci`.

SOLUCIÓN A mano, sumando cada término con los dos anteriores:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144$$

de modo que $F_{12} = 144$. La verificación:

`In[ ] :=`

`NFibonacci[12]`

`Out[ ] =`

144

- 1.6.5** (★★) **A MANO** La sucesión de *Lucas* obedece la misma regla de Fibonacci pero arranca con $L_1 = 1$ y $L_2 = 3$. Escriba un programa recursivo de pila que la calcule y obtenga a mano L_8 .

SOLUCIÓN Se reutiliza la estructura de Fibonacci cambiando los dos casos raíz:

In[] :=

```
Lucas[n_]:=If[n == 1, 1, If[n == 2, 3, Lucas[n - 1] + Lucas[n - 2]]]
```

A mano: 1, 3, 4, 7, 11, 18, 29, 47, luego $L_8 = 47$. Obsérvese que hacen falta **dos** casos raíz, porque la definición base mira dos términos hacia atrás.

- 1.6.6** (★) **VilCretas** Muestre el código interno con el que está programado `NFibonacci` y señale sus casos raíz.

SOLUCIÓN

In[] :=

```
NFibonacci[10, code -> True]
```

La opción `code->True` imprime la función recursiva. Sus casos raíz son $n = 1$ y $n = 2$, ambos con valor 1; la definición base es la suma de los dos términos anteriores.

- 1.6.7** (★★) **VilCretas** Con la opción `steps->True` observe la bajada de `Factoriales[6]` y cuente cuántas líneas imprime.

SOLUCIÓN

In[] :=

```
Factoriales[6, steps -> True]
```

Imprime una línea por cada llamada de la bajada, de $6!$ hasta el caso raíz $1!$: seis líneas en total. El resultado final es 720.

- 1.6.8** (★★) **A MANO** ¿Cuántas invocaciones realiza `NFibonacci[6]` contando la inicial? Describa el árbol de llamadas.

SOLUCIÓN Sea $T(n)$ el número de invocaciones. Como $T(1) = T(2) = 1$ y $T(n) = 1 + T(n - 1) + T(n - 2)$:

$$T(3) = 3, \quad T(4) = 5, \quad T(5) = 9, \quad T(6) = 15$$

El árbol se ramifica en dos por cada nodo que no es raíz. Ésa es la razón de que la versión de pila de Fibonacci sea tan costosa: recalcula muchas veces los mismos términos.

- 1.6.9** (★★) **MIXTO** Escriba una recursividad de pila para $a_n = n^2 + a_{n-1}$ con $a_1 = 1$, calcule a_6 a mano y verifique a_5 con `Sumatoria`.

SOLUCIÓN La recursividad es:

In[] :=

```
Suma[n_]:=If[n == 1, 1, Suma[n - 1] + n^2]
```

A mano: 1, 5, 14, 30, 55, 91, luego $a_6 = 91$. Como a_n es la suma de los cuadrados desde 1 hasta n , el término a_5 se obtiene con:

In[] :=

```
Sumatoria[{1, n, i^2}, 5]
```

Out[] =

55

que coincide con el quinto valor de la lista anterior.

- 1.6.10** (★★★) **A MANO** Un estudiante escribe $F[n_]:=If[n == 0, 1, n*F[n - 1]]$ y otro escribe $G[n_]:=If[n == 1, 1, n*G[n - 1]]$. ¿En qué se diferencian al invocarlas con $n = 0$?

SOLUCIÓN $F[0]$ devuelve 1, porque 0 es su caso raíz. $G[0]$ no encuentra nunca su caso raíz: baja a $-1, -2, -3, \dots$ y la pila de llamadas no termina. Ambas coinciden para todo $n \geq 1$, pero solo la primera está definida en 0. Es un ejemplo de por qué el caso raíz debe cubrir *todos* los valores de entrada previstos.

- 1.6.11** (★★) **MIXTO** Calcule $15!$ y compare su número de cifras con el de $12!$.

SOLUCIÓN

In[] :=

```
Factoriales[15]
```

Out[] =

1307674368000

$15! = 1\,307\,674\,368\,000$ tiene 13 cifras frente a las 9 de $12!$. Tres factores más multiplican el resultado por $13 \cdot 14 \cdot 15 = 2730$: el factorial crece más rápido que cualquier exponencial de base fija.

- 1.6.12** (★★★) **MIXTO** Escriba una recursividad que cuente las invocaciones de `NFibonacci` y tabule el resultado para $n = 1, \dots, 10$.

SOLUCIÓN Cada llamada aporta una unidad más las de sus dos ramas, y los casos raíz aportan una sola:

In[] :=

```
C11[n_] := If[n == 1 || n == 2, 1, 1 + C11[n - 1] + C11[n - 2]];
```

```
Table[C11[k], {k, 1, 10}]
```

Out[] =

{1, 1, 3, 5, 9, 15, 25, 41, 67, 109}

La sucesión de conteos es $C_n = 2F_n - 1$: para $n = 10$ hay 109 llamadas y solo 55 valores distintos. Esa repetición es la que hace inviable la versión de pila con n grande.

- 1.6.13** (★★★) **A MANO** Demuestre que el número de llamadas del ejercicio anterior satisface $C_n = 2F_n - 1$.

SOLUCIÓN Por inducción. Para $n = 1$ y $n = 2$: $C_1 = C_2 = 1 = 2 \cdot 1 - 1$. Supuesto cierto hasta $n - 1$,

$$C_n = 1 + C_{n-1} + C_{n-2} = 1 + (2F_{n-1} - 1) + (2F_{n-2} - 1) = 2(F_{n-1} + F_{n-2}) - 1 = 2F_n - 1$$

El -1 sobrevive porque las dos unidades restadas se cancelan con la que se suma. De aquí que el conteo crezca tan rápido como el propio Fibonacci.

- 1.6.14** (★★) **MIXTO** La sucesión de *Pell* cumple $P_n = 2P_{n-1} + P_{n-2}$ con $P_1 = 1$ y $P_2 = 2$. Prográmela y tabule diez términos.

SOLUCIÓN

In[] :=

```
Pell[n_] := If[n == 1, 1, If[n == 2, 2, 2 Pell[n - 1] + Pell[n - 2]]];
Table[Pell[k], {k, 1, 10}]
```

Out[] =

```
{1, 2, 5, 12, 29, 70, 169, 408, 985, 2378}
```

Es la misma estructura de Fibonacci con un coeficiente distinto: dos casos raíz y dos llamadas por invocación. El cociente entre términos consecutivos tiende a $1 + \sqrt{2}$, no a la razón áurea.

- 1.6.15** (★★★) **MIXTO** La sucesión *tribonacci* suma los **tres** términos anteriores, con $T_1 = T_2 = 0$ y $T_3 = 1$. Prográmela y tabule doce términos.

SOLUCIÓN

In[] :=

```
Tri[n_] := If[n <= 2, 0, If[n == 3, 1, Tri[n - 1] + Tri[n - 2] + Tri[n - 3]]];
Table[Tri[k], {k, 1, 12}]
```

Out[] =

```
{0, 0, 1, 1, 2, 4, 7, 13, 24, 44, 81, 149}
```

Hacen falta **tres** casos raíz, uno por cada término al que apunta la definición base. Con solo dos, la evaluación de T_3 caería en T_0 y la recursión no terminaría.

- 1.6.16** (★★★) **A MANO** Compare el árbol de llamadas de `NFibonacci[5]` con el de `Factoriales[5]` e indique cuántos nodos tiene cada uno.

SOLUCIÓN El del factorial es una **cadena**: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, cinco nodos y profundidad cinco. El de Fibonacci es un **árbol binario**: F_5 llama a F_4 y F_3 , y así sucesivamente, con $C_5 = 9$ nodos y profundidad cuatro. La diferencia está en el número de llamadas por invocación: una frente a dos.

- 1.6.17** (★★) **MIXTO** Calcule F_{40} con la versión de cola y explique por qué no conviene intentarlo con la de pila.

SOLUCIÓN

In[] :=

```
NFibonacciCola[40]
```

Out[] =

```
102334155
```

La versión de pila haría $C_{40} = 2F_{40} - 1 = 204\,668\,309$ llamadas, más de doscientos millones. La de cola hace 40: arrastra los dos términos anteriores en los parámetros y nunca recalcula nada.

- 1.6.18** (★★) **A MANO** Escriba los seis primeros términos de la sucesión definida por $a_n = a_{n-1} + 2n - 1$ con $a_1 = 1$, y reconozca el patrón.

SOLUCIÓN Se obtiene 1, 4, 9, 16, 25, 36: son los cuadrados perfectos. En efecto, $n^2 - (n-1)^2 = 2n - 1$, de modo que sumar los impares sucesivos construye $a_n = n^2$. Es el ejemplo clásico de una recursividad cuya fórmula explícita es mucho más simple que su definición.

- 1.6.19** (★★★) **MIXTO** Programe una recursividad que calcule el coeficiente binomial $\binom{n}{k}$ mediante la identidad de *Pascal* y verifíquela.

SOLUCIÓN La identidad es $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, con casos raíz $k = 0$ y $k = n$:

In[] :=

```
comb[n_, k_] := If[k == 0 || k == n, 1,
  comb[n - 1, k - 1] + comb[n - 1, k]];
{comb[8, 3], Binomial[8, 3]}
```

Out[] =

```
{56, 56}
```

Es una recursividad de **dos parámetros**: ambos disminuyen, lo que garantiza la terminación. Su árbol de llamadas es el triángulo de *Pascal* recorrido de arriba abajo.

- 1.6.20** (★★★) **A MANO** ¿Por qué la recursividad del coeficiente binomial necesita **dos** casos raíz y no uno?

SOLUCIÓN Porque las dos llamadas bajan por caminos distintos: $\binom{n-1}{k-1}$ puede llegar a $k = 0$ y $\binom{n-1}{k}$ puede llegar a $k = n$. Con un solo caso raíz, la otra rama seguiría bajando hasta valores sin sentido. Es la misma razón por la que Fibonacci necesita dos y tribonacci, tres.

- 1.6.21** (★★) **MIXTO** La sucesión de *Lucas* obedece la regla de Fibonacci con $L_1 = 1$ y $L_2 = 3$. Tabule doce términos y compárelos con los de Fibonacci.

SOLUCIÓN

In[] :=

```
Luc[n_] := If[n == 1, 1, If[n == 2, 3, Luc[n - 1] + Luc[n - 2]]];
Table[Luc[k], {k, 1, 12}]
```

Out[] =

```
{1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322}
```

La regla es la misma y solo cambian los casos raíz, pero los valores no tienen nada que ver con los de Fibonacci (1, 1, 2, 3, 5, ...). Se cumple $L_n = F_{n-1} + F_{n+1}$.

- 1.6.22** (★★★) **A MANO** Verifique la identidad $L_n = F_{n-1} + F_{n+1}$ para $n = 5$ y $n = 8$, y explique por qué vale en general.

SOLUCIÓN Para $n = 5$: $F_4 + F_6 = 3 + 8 = 11 = L_5$. Para $n = 8$: $F_7 + F_9 = 13 + 34 = 47 = L_8$. En general, ambas sucesiones satisfacen la misma definición base, y la combinación $F_{n-1} + F_{n+1}$ también la satisface por ser suma de dos soluciones. Basta entonces comprobar los dos primeros términos: $F_0 + F_2 = 0 + 1 = 1 = L_1$ y $F_1 + F_3 = 1 + 2 = 3 = L_2$.

1.6.23 (★★) **MIXTO** Programe la sucesión $a_n = n a_{n-1} + 1$ con $a_1 = 1$ y calcule a_6 .

SOLUCIÓN

In[] :=

```
r[n_] := If[n == 1, 1, n*r[n - 1] + 1];
Table[r[k], {k, 1, 6}]
```

Out[] =

```
{1, 3, 10, 41, 206, 1237}
```

El término $a_6 = 1237$. La estructura es la del factorial con un $+1$ añadido en cada paso, lo que impide que exista una fórmula cerrada tan simple como $n!$.

1.6.24 (★★★) **A MANO** Explique por qué la sucesión $a_n = a_{n-1} + a_{n-2}$ con $a_1 = 2$ y $a_2 = 2$ es la de Fibonacci multiplicada por dos.

SOLUCIÓN Si $b_n = a_n/2$, entonces $b_n = b_{n-1} + b_{n-2}$ con $b_1 = b_2 = 1$: es exactamente Fibonacci. Como la definición base es **lineal y homogénea**, multiplicar las condiciones iniciales por una constante multiplica toda la sucesión por esa constante. Con condiciones 2 y 3, en cambio, no habría proporcionalidad.

1.6.25 (★★) **MIXTO** Use **Dato** para buscar el valor 7 en la lista $\{4, 9, 2, 7, 5\}$ y describa lo que muestra.

SOLUCIÓN

In[] :=

```
Dato[7, {4, 9, 2, 7, 5}]
```

El primer argumento es el **dato** y el segundo la lista. El comando recorre la lista de forma recursiva y muestra el avance; con **steps->True** despliega paso a paso y con **code->True**, el código interno. Invertir el orden de los argumentos deja la instrucción **sin evaluar**, sin ningún mensaje.

1.6.26 (★★★) **MIXTO** Programe una recursividad que devuelva el máximo de una lista y explíquela.

SOLUCIÓN

In[] :=

```
mx[L_] := If[Length[L] == 1, First[L], Max[First[L], mx[Rest[L]]]];
mx[{3, 17, 8, 25, 4}]
```

Out[] =

```
25
```

El caso raíz es la lista de un solo elemento. En cada paso se compara el primer elemento con el máximo del resto: la lista se acorta en uno, lo que garantiza la terminación. Con la lista vacía el programa fallaría, porque no hay máximo que devolver.

1.6.27 (★★★) **MIXTO** Programe una recursividad que decida si una palabra es un palíndromo.

SOLUCIÓN

In[] :=


```
pal[s_] := If[StringLength[s] <= 1, True,
  If[StringTake[s, 1] != StringTake[s, -1], False,
    pal[StringTake[s, {2, -2}]]];
{pal["reconocer"], pal["discreta"]}
```

Out[] =

```
{True, False}
```

Hay **dos** casos raíz: la palabra de longitud cero o uno, que siempre es palíndromo, y el descubrimiento de dos extremos distintos, que corta la recursión de inmediato. Cada paso quita dos letras.

- 1.6.28** (★★) **A MANO** ¿Cuántas comparaciones hace el programa anterior sobre una palabra de n letras que sí es palíndromo?

SOLUCIÓN Hace $\lfloor n/2 \rfloor$ comparaciones: cada paso descarta dos letras y la recursión se detiene al quedar una o ninguna. Para *reconocer*, con nueve letras, son cuatro comparaciones. Si la palabra **no** es palíndromo el número puede ser mucho menor, porque el primer desajuste corta el proceso.

- 1.6.29** (★★) **MIXTO** Programe la función que cuenta las cifras de un número natural y aplíquela a 2^{30} .

SOLUCIÓN

In[] :=

```
cd[n_] := If[n < 10, 1, 1 + cd[Quotient[n, 10]]];
{cd[987654], cd[2^30]}
```

Out[] =

```
{6, 10}
```

Cada paso elimina la última cifra con **Quotient**. El número de pasos es $\lfloor \log_{10} n \rfloor + 1$: una recursividad de coste logarítmico, no lineal.

- 1.6.30** (★★★) **A MANO** Compare el número de llamadas de la función anterior con el de **SumaDigi** sobre el mismo número. ¿Coinciden?

SOLUCIÓN Sí: las dos bajan quitando una cifra por paso, de modo que ambas hacen $\lfloor \log_{10} n \rfloor + 1$ llamadas. Lo que cambia es qué se acumula (un contador frente a una suma), no la **forma** de la recursión. Es un ejemplo de dos problemas distintos con el mismo esquema recursivo.

1.2 Propiedades de una recursividad

- 1.6.31** (★) **A MANO** Explique qué ocurre al ejecutar una recursividad a la que se le ha suprimido el caso raíz.

SOLUCIÓN La definición base se invoca indefinidamente porque nunca se alcanza la condición de parada. Cada llamada apila una tarea pendiente y la pila crece sin control hasta agotar la memoria reservada: es un **desbordamiento de la pila**. En *Mathematica* el proceso se detiene al superar **\$RecursionLimit**.

- 1.6.32** (★) **A MANO** Indique cuántos casos raíz necesita una recursividad cuya definición base utiliza a_{n-1} , a_{n-2} y a_{n-3} , y justifíquelo.

SOLUCIÓN Necesita **tres**. La definición base mira tres términos hacia atrás, de modo que para arrancar el cálculo deben conocerse tres valores consecutivos. Con menos, alguna llamada de la bajada quedaría sin resolver y el programa no terminaría.

- 1.6.33** (★★) **A MANO** Localice el error en $H[n_]:=If[n == 1, 1, H[n + 1] - 1]$ y corríjalo.

SOLUCIÓN El error está en el argumento de la llamada recursiva: $H[n+1]$ *aleja* el problema del caso raíz en vez de acercarlo. Para $n > 1$ la sucesión de llamadas es 2, 3, 4, ... y nunca llega a 1. La corrección consiste en reducir el argumento:

$In[] :=$

$H[n_]:=If[n == 1, 1, H[n - 1] - 1]$

Toda definición base debe acercarse al caso raíz en cada invocación.

- 1.6.34** (★★) **A MANO** Escriba en forma recursiva la regla « a_n vale 0 si n es par y $a_{n-1} + 2$ si n es impar», con $a_1 = 1$.

SOLUCIÓN

$In[] :=$

$A[n_]:=If[n == 1, 1, If[EvenQ[n], 0, A[n - 1] + 2]]$

Los primeros términos son 1, 0, 2, 0, 2, 0, ... El caso raíz es $n = 1$; la definición base distingue dos ramas según la paridad, y solo la rama impar vuelve a invocar la función.

- 1.6.35** (★★) **VilCretas** Use `steps->True` en `PotenciaPos[2, 6]` y describa qué muestra cada línea.

SOLUCIÓN

$In[] :=$

`PotenciaPos[2, 6, steps -> True]`

Cada línea muestra una invocación de la bajada, reduciendo el exponente en una unidad, hasta el caso raíz de exponente 0 que vale 1. Al regresar, cada nivel multiplica por la base. El resultado es $2^6 = 64$.

- 1.6.36** (★★) **A MANO** Determine el valor de $P[4]$ si $P[n_]:=If[n == 0, 3, 2*P[n - 1] - 1]$.

SOLUCIÓN Se baja hasta el caso raíz y se sube resolviendo:

$$P(0) = 3, \quad P(1) = 5, \quad P(2) = 9, \quad P(3) = 17, \quad P(4) = 33$$

En general $P(n) = 2^{n+1} + 1$, aunque el ejercicio solo pide el valor.

- 1.6.37** (★) **A MANO** Señale cuál de las dos definiciones termina para todo entero $n \geq 0$ y por qué: $R[n_]:=If[n == 0, 1, R[n - 2]]$ o $S[n_]:=If[n <= 0, 1, S[n - 2]]$.

SOLUCIÓN Termina la segunda. La primera solo alcanza el caso raíz si n es par: con n impar la bajada pasa de 1 a $-1, -3, \dots$ sin tocar nunca el 0. La segunda usa la condición $n \leq 0$, que cubre ambos casos. La lección es que la condición de parada debe ser *alcanzable* desde toda entrada admitida.

- 1.6.38** (★★★) **A MANO** Construya una recursividad que cuente las llamadas de `NFibonacci` y evalúela para $n = 7$.

SOLUCIÓN Si $T(n)$ cuenta las invocaciones, cada llamada aporta una unidad más las de sus dos ramas, y los casos raíz aportan una sola:

In[] :=

```
T[n_]:=If[n == 1 || n == 2, 1, 1 + T[n - 1] + T[n - 2]]
```

Desarrollando: $T(3) = 3$, $T(4) = 5$, $T(5) = 9$, $T(6) = 15$, $T(7) = 25$. La estructura del contador es la misma de la recursividad que mide, con la suma sustituida por el conteo.

1.6.39 (★★) **MIXTO** Verifique a mano que $3^5 = 243$ y calcule además 3^7 con **PotenciaPos**.

SOLUCIÓN A mano, con la definición base $a^n = a \cdot a^{n-1}$ y caso raíz $a^0 = 1$:

$$3^0 = 1, 3^1 = 3, 3^2 = 9, 3^3 = 27, 3^4 = 81, 3^5 = 243$$

In[] :=

```
PotenciaPos[3, 7]
```

Out[] =

2187

El valor $3^7 = 2187$ continúa la misma cadena.

1.6.40 (★★★) **A MANO** Una recursividad tiene definición base $a_n = a_{n-1} + a_{n-2}$ y un único caso raíz $a_1 = 1$. Explique por qué no basta y proponga la corrección mínima.

SOLUCIÓN Al calcular a_2 la definición base pide a_1 y a_0 ; a_0 no está definido y la bajada continúa hacia índices negativos. Un solo caso raíz no cubre una definición base que retrocede dos posiciones. La corrección mínima es añadir un segundo caso raíz, por ejemplo $a_2 = 1$, con lo que se obtiene exactamente la sucesión de Fibonacci.

1.6.41 (★★★) **A MANO** Un estudiante programa $a_n = a_{n-1} + a_{n+1}$ con $a_1 = 1$. ¿Qué ocurre al evaluarla y por qué?

SOLUCIÓN No termina nunca. La definición base invoca a_{n+1} , un índice **mayor** que el actual, de modo que la recursión se aleja del caso raíz en lugar de acercarse. Para que una recursividad termine, cada llamada debe reducir algún valor que esté acotado inferiormente por el caso raíz.

1.6.42 (★★★) **A MANO** Determine para qué valores de n termina la recursividad $f(n) = f(n-3)$ con caso raíz $f(0) = 1$.

SOLUCIÓN Solo para los n múltiplos de tres. Partiendo de n se recorren $n, n-3, n-6, \dots$, y ese descenso alcanza el 0 únicamente si $n \equiv 0 \pmod{3}$; en otro caso salta al -1 o al -2 y sigue bajando indefinidamente. La corrección mínima es usar $n \leq 0$ como caso raíz en vez de $n = 0$.

1.6.43 (★★) **A MANO** Explique la diferencia entre *definición base* y *caso raíz*, y dé un ejemplo de cada uno.

SOLUCIÓN La **definición base** es la regla que expresa un término en función de los anteriores: $n \cdot f(n-1)$ en el factorial. El **caso raíz** es el valor conocido que detiene el descenso: $f(1) = 1$. Sin definición base no hay recursividad; sin caso raíz no hay terminación.

- 1.6.44 (★★★) **A MANO** Una recursividad tiene definición base $a_n = a_{n-2} + a_{n-3}$. ¿Cuántos casos raíz necesita y cuáles?

SOLUCIÓN Necesita **tres**: a_1 , a_2 y a_3 . Aunque a_{n-1} no aparezca, el término más lejano es a_{n-3} , y para evaluar a_4 hacen falta a_2 y a_1 ; para a_5 , a_3 y a_2 . El número de casos raíz lo fija la **distancia al término más lejano**, no la cantidad de sumandos.

- 1.6.45 (★★★) **MIXTO** Programe la función de *Collatz*, que cuenta cuántos pasos tarda un número en llegar a 1 alternando $n/2$ y $3n + 1$, y aplíquela a 27.

SOLUCIÓN

In[] :=

```
col[n_] := If[n == 1, 0, 1 + col[If[EvenQ[n], n/2, 3 n + 1]]];
{col[7], col[27]}
```

Out[] =

```
{16, 111}
```

Desde 27 hacen falta 111 pasos. Es una recursividad cuya **terminación no está demostrada**: nadie sabe si todo natural llega a 1. El valor no disminuye en cada paso, de modo que el argumento habitual de terminación no se puede aplicar.

- 1.6.46 (★★★) **A MANO** ¿Por qué el argumento habitual de terminación falla en *Collatz*?

SOLUCIÓN Porque ese argumento exige que alguna cantidad **decrezca estrictamente** en cada llamada y esté acotada inferiormente. En *Collatz*, la rama $3n + 1$ hace crecer el valor: desde 27 se llega a superar 9000 antes de volver a bajar. No hay una medida evidente que decrezca siempre, y por eso el problema sigue abierto.

- 1.6.47 (★★) **A MANO** Indique cuántas llamadas anidadas realiza **Factoriales**[20] y a qué profundidad llega.

SOLUCIÓN Realiza 20 llamadas y la profundidad de la pila es 20: una por cada entero de 20 hasta 1. En una recursividad de pila la profundidad coincide con el número de llamadas, y por eso el límite de recursión del sistema es un tope real al tamaño del problema.

- 1.6.48 (★★★) **A MANO** Dos funciones se llaman mutuamente: $p(n) = q(n - 1)$ y $q(n) = p(n - 1)$, con $p(0) = 1$ y $q(0) = 0$. Calcule $p(5)$ y $p(6)$.

SOLUCIÓN Cada llamada baja un índice y alterna de función, de modo que la paridad decide en cuál se termina: $p(5) = q(4) = p(3) = q(2) = p(1) = q(0) = 0$ y $p(6) = q(5) = \dots = p(0) = 1$. Es una **recursividad mutua**: ninguna de las dos funciones se invoca a sí misma, pero el par sí termina.

- 1.6.49 (★★★) **MIXTO** Compruebe el resultado anterior programándolo.

SOLUCIÓN

In[] :=

```
pp[n_] := If[n == 0, 1, qq[n - 1]];
qq[n_] := If[n == 0, 0, pp[n - 1]];
{pp[5], pp[6], qq[5], qq[6]}
```

Out[] =

{0, 1, 1, 0}

Los valores confirman la alternancia: $p(n)$ vale 1 si n es par y 0 si es impar, y q hace lo contrario. Definir `pp` antes que `qq` no da problema: *Mathematica* no evalúa la definición hasta que se la invoca.

1.6.50 (★★) **A MANO** ¿Qué diferencia hay entre una recursividad *lineal* y una *múltiple*? Dé un ejemplo de cada una.

SOLUCIÓN En la **lineal** cada invocación genera **una sola** llamada recursiva: el factorial, la suma de dígitos, el conteo de cifras. En la **múltiple** genera dos o más: Fibonacci, tribonacci, el coeficiente binomial. La diferencia se ve en el árbol de llamadas: una cadena frente a un árbol ramificado, y con ella el salto de coste lineal a exponencial.

1.6.51 (★★★) **A MANO** Un programa recursivo se define con $f(n) = f(n/2)$ y caso raíz $f(1) = 1$. ¿Termina para todo natural?

SOLUCIÓN Con división entera sí: $\lfloor n/2 \rfloor$ decrece estrictamente hasta llegar a 1, en $\lfloor \log_2 n \rfloor$ pasos. Con división exacta, **no**: desde $n = 3$ se pasa a $3/2$, luego a $3/4$, y nunca se alcanza el 1. La diferencia entre `Quotient[n, 2]` y `n/2` decide la terminación.

1.6.52 (★★★) **MIXTO** Programe la potenciación rápida, que eleva al cuadrado cuando el exponente es par, y compare su número de llamadas con el de `PotenciaPos`.

SOLUCIÓN

`In[ ] :=`

```
pot[b_, e_] := If[e == 0, 1,
  If[EvenQ[e], pot[b*b, e/2], b*pot[b, e - 1]]];
{pot[3, 13], 3^13}
```

`Out[ ] =`

{1594323, 1594323}

`PotenciaPos[3, 13]` hace 13 llamadas, una por unidad del exponente; la versión rápida hace unas $2 \log_2 13 \approx 7$. La diferencia es de orden: $\Theta(n)$ frente a $\Theta(\ln n)$.

1.6.53 (★★) **MIXTO** Verifique que `PotenciaPos` acepta bases negativas y explique el signo del resultado.

SOLUCIÓN

`In[ ] :=`

```
{PotenciaPos[-3, 5], PotenciaPos[2, 15]}
```

`Out[ ] =`

{-243, 32768}

Con base negativa el signo lo decide la paridad del exponente: $(-3)^5$ es negativo porque hay cinco factores negativos. La recursividad no necesita ningún caso especial para esto: el signo se propaga solo en la multiplicación.

1.6.54 (★★★) **A MANO** Una recursividad calcula $S(n) = S(n-1) + S(n-1)$ con $S(0) = 1$. ¿Cuánto vale $S(n)$ y cuántas llamadas hace?

SOLUCIÓN Vale $S(n) = 2^n$, porque cada paso duplica. Pero el conteo de llamadas es $2^{n+1} - 1$: el árbol es binario **completo** de altura n . Es el peor caso posible de recursividad múltiple, y basta reescribirla como $S(n) = 2S(n-1)$ para bajar a n llamadas sin cambiar el resultado.

1.6.55 (★★★) **MIXTO** Compruebe con un contador la afirmación anterior para $n = 5$.

SOLUCIÓN

In[] :=

```
cnt[n_] := If[n == 0, 1, 1 + cnt[n - 1] + cnt[n - 1]];
{cnt[5], 2^6 - 1}
```

Out[] =

```
{63, 63}
```

Las 63 llamadas para $n = 5$ confirman la fórmula $2^{n+1} - 1$. Duplicar una llamada en el código no duplica el trabajo: lo eleva al cuadrado en número de nodos del árbol.

1.6.56 (★★) **A MANO** ¿Puede una recursividad no tener definición base y aun así calcular algo?

SOLUCIÓN No. Sin definición base no hay ninguna regla que relacione un término con los anteriores, de modo que lo único definido serían los casos raíz: una tabla finita de valores, no una recursividad. La definición base es lo que permite alcanzar infinitos valores con un texto finito.

1.6.57 (★★★) **A MANO** Justifique por qué toda recursividad de pila puede agotar la memoria y una de cola bien implementada no.

SOLUCIÓN En la de pila queda una operación **pendiente** tras cada llamada (la multiplicación por n en el factorial), de modo que el sistema debe guardar un marco por nivel: la memoria crece con la profundidad. En la de cola la llamada recursiva es **lo último** que se hace y el resultado se devuelve tal cual, de modo que el marco puede reutilizarse. La diferencia está en si queda trabajo después de la llamada, no en la forma del código.

1.6.58 (★★) **A MANO** Clasifique como lineal o múltiple, y de pila o de cola: (a) $f(n) = f(n-1) + n$; (b) $g(n) = g(n-1) + g(n-2)$; (c) $h(i, s) = h(i+1, s+i)$.

SOLUCIÓN (a) Lineal y de **pila**: queda pendiente sumar n al volver. (b) Múltiple y de pila. (c) Lineal y de **cola**: la llamada es la última operación y el acumulador s carga el resultado parcial.

1.6.59 (★★★) **A MANO** En `Multiplos[n, m]` el comando lista los múltiplos de n menores que m . ¿Cuántas llamadas recursivas hace y de qué depende?

SOLUCIÓN Hace $\lceil m/n \rceil$ llamadas aproximadamente: una por cada múltiplo generado más la que detecta el fin. Depende del **cociente** entre los dos argumentos, no de su tamaño: `Multiplos[3, 20]` da seis valores y `Multiplos[1000, 20000]` también da veinte, aunque los números sean mucho mayores.

1.6.60 (★★) **MIXTO** Compruebe la observación anterior sobre dos casos.

SOLUCIÓN

In[] :=

```
{Multiplos[3, 20], Multiplos[5, 40]}
```

Out[] =

```
{{3, 6, 9, 12, 15, 18}, {5, 10, 15, 20, 25, 30, 35}}
```

Seis y siete valores respectivamente. El segundo par de argumentos es mayor pero el cociente es parecido, y por eso el número de llamadas apenas cambia.

1.3 Programas recursivos

- 1.6.61** (★) **VilCretas** Obtenga $(-2)^9$ y $1,5^4$ con `PotenciaPos` e interprete el signo del primer resultado.

SOLUCIÓN

In[] :=

```
PotenciaPos[-2, 9]
```

```
PotenciaPos[1.5, 4]
```

Out[] =

```
-512
```

```
5.0625
```

El exponente 9 es impar, de modo que el signo negativo de la base se conserva: $(-2)^9 = -512$. El comando admite bases reales, no solo enteras, como muestra el segundo cálculo.

- 1.6.62** (★★) **A MANO** Escriba un programa recursivo que calcule $\sum_{i=1}^n (3i - 2)$ y obtenga a mano el valor para $n = 5$.

SOLUCIÓN

In[] :=

```
Suma[n_] := If[n == 1, 1, Suma[n - 1] + (3 n - 2)]
```

El caso raíz corresponde a $i = 1$, cuyo sumando vale $3(1) - 2 = 1$. A mano: 1, 5, 12, 22, 35, luego la suma para $n = 5$ es 35.

- 1.6.63** (★★) **VilCretas** Use `Sumatoria` para generar el programa recursivo de $\sum_{i=1}^n i^2$ y calcular su valor en $n = 5$.

SOLUCIÓN

In[] :=

```
Sumatoria[{1, n, i^2}, 5]
```

Out[] =

```
55
```

El comando arma internamente la recursividad $S(n) = S(n - 1) + n^2$ con $S(1) = 1$ y la evalúa paso a paso. El resultado 55 coincide con $1 + 4 + 9 + 16 + 25$. No debe redefinirse `Sumatoria` en el cuaderno: es un símbolo protegido del paquete.

1.6.64 (★★) **VilCretas** Con `Productoria`, construya el programa de $\prod_{i=1}^n 2i$ y evalúelo en $n = 5$.

SOLUCIÓN

`In[ ] :=`

`Productoria[{1, n, 2 i}, 5]`

`Out[ ] =`

3840

La recursividad generada es `Productoria[n_] := If[n == 1, 2, Productoria[n-1] * (2*n)]`. El valor 3840 equivale a $2 \cdot 4 \cdot 6 \cdot 8 \cdot 10$.

1.6.65 (★★) **MIXTO** Multiplique 13 por 9 mediante sumas sucesivas, primero a mano y después con `Multipli`.

SOLUCIÓN La definición base es $n \cdot m = n + n \cdot (m - 1)$ con caso raíz $n \cdot 1 = n$. A mano se acumulan nueve treces: 13, 26, 39, 52, 65, 78, 91, 104, 117.

`In[ ] :=`

`Multipli[13, 9]`

`Out[ ] =`

117

1.6.66 (★) **VilCretas** Calcule 25×16 con `Multipli` e indique cuántas sumas realiza.

SOLUCIÓN

`In[ ] :=`

`Multipli[25, 16]`

`Out[ ] =`

400

El segundo argumento marca el número de niveles de la recursividad: se realizan 16 invocaciones, es decir, quince sumas más el caso raíz.

1.6.67 (★★) **MIXTO** Sume a mano los dígitos de 98765 y verifique con `SumaDigi`.

SOLUCIÓN A mano: $9 + 8 + 7 + 6 + 5 = 35$. El procedimiento recursivo toma el último dígito con el residuo entre 10 y repite con el cociente entero.

`In[ ] :=`

`SumaDigi[98765]`

`Out[ ] =`

35

1.6.68 (★★) **VilCretas** Aplique `SumaDigi` a 1234567 y explique por qué el resultado no depende del orden de los dígitos.

SOLUCIÓN

```
In[ ] :=
```

```
SumaDigi[1234567]
```

```
Out[ ] =
```

```
28
```

La suma de dígitos es conmutativa: la recursividad los extrae de derecha a izquierda, pero cualquier permutación de las cifras produce el mismo total, $1 + 2 + 3 + 4 + 5 + 6 + 7 = 28$.

- 1.6.69** (★) **VilCretas** Obtenga los divisores de 60 y de 97 con **Divisores** y explique la diferencia entre ambas listas.

SOLUCIÓN

```
In[ ] :=
```

```
Divisores[60]
```

```
Divisores[97]
```

```
Out[ ] =
```

```
{1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60}
```

```
{1, 97}
```

60 tiene doce divisores; 97 solo tiene dos, el 1 y él mismo, porque es primo. La lista de divisores permite reconocer la primalidad sin más cálculo.

- 1.6.70** (★★) **MIXTO** Determine a mano los divisores de 100 y compruébelo con **Divisores**.

SOLUCIÓN Se emparejan factores: $1 \cdot 100$, $2 \cdot 50$, $4 \cdot 25$, $5 \cdot 20$ y $10 \cdot 10$.

```
In[ ] :=
```

```
Divisores[100]
```

```
Out[ ] =
```

```
{1, 2, 4, 5, 10, 20, 25, 50, 100}
```

Son nueve, un número impar de divisores, porque 100 es un cuadrado perfecto y el factor 10 se repite.

- 1.6.71** (★★) **MIXTO** Calcule $\text{mcd}(84, 36)$ con el algoritmo de Euclides a mano y verifique con **MCD**.

SOLUCIÓN El algoritmo sustituye el par (n, m) por $(m, n \bmod m)$ hasta que el segundo se anula:

$$(84, 36) \rightarrow (36, 12) \rightarrow (12, 0)$$

El máximo común divisor es el último valor no nulo, 12.

```
In[ ] :=
```

```
MCD[84, 36]
```

```
Out[ ] =
```

```
12
```

- 1.6.72** (★★★) **MIXTO** Aplique el algoritmo de Euclides a (1071, 462) paso a paso y verifique el resultado.

SOLUCIÓN

$$(1071, 462) \rightarrow (462, 147) \rightarrow (147, 21) \rightarrow (21, 0)$$

porque $1071 = 2 \cdot 462 + 147$, $462 = 3 \cdot 147 + 21$ y $147 = 7 \cdot 21 + 0$. El máximo común divisor es 21.

In[] :=

MCD[1071, 462]

Out[] =

21

Tres divisiones bastan para un par de cuatro cifras: ésa es la eficiencia del método.

- 1.6.73** (★) **VilCretas** Liste los múltiplos de 7 menores que 50 con **Multiplos**.

SOLUCIÓN

In[] :=

Multiplos[7, 50]

Out[] =

{7, 14, 21, 28, 35, 42, 49}

El recorrido recursivo va acumulando múltiplos mientras no se alcance la cota superior, que no se incluye.

- 1.6.74** (★★) **MIXTO** Prediga cuántos múltiplos de 12 hay por debajo de 100 y compruébelo.

SOLUCIÓN Los múltiplos son $12k$ con $12k < 100$, es decir $k \leq 8$: son ocho.

In[] :=

Multiplos[12, 100]

Out[] =

{12, 24, 36, 48, 60, 72, 84, 96}

- 1.6.75** (★★) **VilCretas** Ordene la lista {9, 4, 7, 1, 8, 2} con **Quicksort** y después en orden descendente.

SOLUCIÓN

In[] :=

Quicksort[{9, 4, 7, 1, 8, 2}]

Out[] =

{1, 2, 4, 7, 8, 9}

Para el orden inverso se emplea la opción **ascendente->False**:

In[] :=

```
Quicksort[{9, 4, 7, 1, 8, 2}, ascendente -> False]
```

```
Out[ ] =
```

```
{9, 8, 7, 4, 2, 1}
```

1.6.76 (★★) **VilCretas** Busque el dato 7 en la lista {3,7,9} con `Dato` y describa la salida.

SOLUCIÓN

```
In[ ] :=
```

```
Dato[7, {3, 7, 9}]
```

El comando recorre la lista de forma recursiva e informa de cada posición visitada; al hallar el valor imprime `Dato encontrado` y detiene el recorrido. Si el dato no estuviera, recorrería la lista completa y lo indicaría.

1.6.77 (★★) **A MANO** Escriba una recursividad que cuente cuántos elementos de una lista son positivos.

SOLUCIÓN Se procesa un elemento y se delega el resto:

```
In[ ] :=
```

```
Positivos[L_] := If[L == {}, 0, If[First[L] > 0, 1, 0] + Positivos[Rest[L]]]
```

El caso raíz es la lista vacía, que aporta 0. La definición base suma uno si el primer elemento es positivo y continúa con `Rest[L]`, que siempre tiene un elemento menos: así se garantiza la parada.

1.6.78 (★★) **A MANO** Construya un programa recursivo que devuelva el mínimo de una lista de números.

SOLUCIÓN

```
In[ ] :=
```

```
Minimo[L_] := If[Length[L] == 1, First[L],  
  If[First[L] < Minimo[Rest[L]], First[L], Minimo[Rest[L]]]]
```

El caso raíz es la lista de un solo elemento. La definición base compara el primero con el mínimo del resto. Sobre {5,3,9,1} devuelve 1.

1.6.79 (★★★) **A MANO** Escriba una recursividad que invierta una lista sin usar `Reverse`.

SOLUCIÓN

```
In[ ] :=
```

```
Invertir[L_] := If[L == {}, {}, Append[Invertir[Rest[L]], First[L]]]
```

La idea es invertir el resto y colocar el primer elemento al final. Sobre {1,2,3}: se invierte {2,3} obteniendo {3,2} y se añade el 1, lo que da {3,2,1}.

1.6.80 (★★) **A MANO** Elabore una recursividad que convierta un número n de base diez a base binaria, devolviendo la lista de bits.

SOLUCIÓN Se extrae el bit menos significativo con el residuo entre 2 y se repite con el cociente entero:

In[] :=

```
ABinario[n_]:=If[n < 2, {n}, Append[ABinario[Quotient[n, 2]], Mod[n, 2]]]
```

Para $n = 13$ se obtiene $\{1, 1, 0, 1\}$, es decir $1101_2 = 8 + 4 + 0 + 1 = 13$.

- 1.6.81** (★★★) **A MANO** Escriba la recursividad inversa a la anterior: dada una lista de bits, devuelva el número en base diez.

SOLUCIÓN Se recorre la lista acumulando el valor posicional:

In[] :=

```
ADecimal[L_]:=If[L == {}, 0, Last[L] + 2*ADecimal[Most[L]]]
```

El caso raíz es la lista vacía. Sobre $\{1, 1, 0, 1\}$ el resultado es $1 + 2(0 + 2(1 + 2(1 + 2 \cdot 0))) = 13$, que confirma la conversión del ejercicio anterior.

- 1.6.82** (★★) **MIXTO** Compare cuántos divisores tienen 60 y 100, y explique por qué uno tiene un número par y el otro impar.

SOLUCIÓN

In[] :=

```
Length[Divisores[60]]
```

```
Length[Divisores[100]]
```

Out[] =

12

9

Los divisores se emparejan como d y n/d . El emparejamiento es perfecto salvo cuando $d = n/d$, es decir cuando n es un cuadrado perfecto: entonces sobra un divisor sin pareja. $100 = 10^2$ tiene nueve divisores; 60 no es cuadrado y tiene doce.

- 1.6.83** (★★) **A MANO** Elabore una recursividad que calcule $\sum_{i=1}^n i \cdot 2^i$ y obtenga el valor para $n = 4$.

SOLUCIÓN

In[] :=

```
S[n_]:=If[n == 1, 2, S[n - 1] + n*2^n]
```

A mano: $S(1) = 2$, $S(2) = 2 + 8 = 10$, $S(3) = 10 + 24 = 34$, $S(4) = 34 + 64 = 98$. El caso raíz corresponde a $i = 1$, cuyo sumando vale $1 \cdot 2^1 = 2$.

- 1.6.84** (★★★) **A MANO** Construya una recursividad que determine si un entero positivo n es potencia de 3.

SOLUCIÓN

In[] :=

```
PotenciaDeTres[n_]:=If[n == 1, True, If[Mod[n, 3] != 0, False, PotenciaDeTres[n/3]]]
```

Hay dos condiciones de parada: $n = 1$ es potencia de 3 (exponente cero), y si n no es divisible entre 3 la respuesta es negativa. Sobre 81 devuelve **True** y sobre 48 devuelve **False**.

- 1.6.85** (★★) **A MANO** Escriba una recursividad que cuente cuántos dígitos tiene un entero positivo.

SOLUCIÓN

In[] :=

```
Cifras[n_]:=If[n < 10, 1, 1 + Cifras[Quotient[n, 10]]]
```

Cada llamada elimina el dígito de las unidades. Sobre 98765 devuelve 5, que coincide con las cinco cifras sumadas en un ejercicio anterior.

- 1.6.86** (★★) **MIXTO** Verifique que la suma de los divisores propios de 28 es 28 y explique el resultado.

SOLUCIÓN Los divisores propios son los divisores distintos del propio número:

In[] :=

```
Divisores[28]
```

Out[] =

```
{1, 2, 4, 7, 14, 28}
```

Sumando todos salvo el 28: $1 + 2 + 4 + 7 + 14 = 28$. Un número con esta propiedad se llama *perfecto*.

- 1.6.87** (★★★) **MIXTO** Determine con **Divisores** cuál de los números 12, 18 y 20 tiene más divisores, y confírmelo contando a mano.

SOLUCIÓN

In[] :=

```
{Length[Divisores[12]], Length[Divisores[18]], Length[Divisores[20]]}
```

Out[] =

```
{6, 6, 6}
```

Los tres tienen exactamente seis divisores. A mano:

$12 : \{1, 2, 3, 4, 6, 12\}, \quad 18 : \{1, 2, 3, 6, 9, 18\}, \quad 20 : \{1, 2, 4, 5, 10, 20\}$

Números distintos pueden compartir la misma cantidad de divisores: lo que la determina es la forma de su factorización, no su tamaño.

- 1.6.88** (★★) **A MANO** Escriba una recursividad que calcule el producto de los elementos de una lista.

SOLUCIÓN

In[] :=

```
ProductoLista[L_]:=If[L == {}, 1, First[L]*ProductoLista[Rest[L]]]
```

El caso raíz devuelve 1, el elemento neutro del producto, igual que la lista vacía devuelve 0 cuando se suman elementos. Sobre $\{2, 3, 5\}$ el resultado es 30.

- 1.6.89** (★★) **VilCretas** Compruebe con **Quicksort** que ordenar una lista ya ordenada devuelve la misma lista, y observe los pasos intermedios.

SOLUCIÓN

```
In[ ] :=
```

```
Quicksort[{1, 2, 4, 7, 8, 9}, steps -> True]
```

La salida final coincide con la entrada. Los pasos intermedios muestran que el algoritmo sigue realizando comparaciones aunque no haya intercambios: el orden previo no le ahorra trabajo de partición.

- 1.6.90** (★★★) **A MANO** Elabore una recursividad que genere la lista de los primeros n términos de la sucesión de Fibonacci.

SOLUCIÓN Se apoya en la recursividad de Fibonacci ya conocida:

```
In[ ] :=
```

```
ListaFib[n_] := If[n == 1, {1}, Append[ListaFib[n - 1], NFibonacci[n]]]
```

Para $n = 8$ devuelve $\{1, 1, 2, 3, 5, 8, 13, 21\}$. Obsérvese que este método vuelve a calcular cada término desde cero, de modo que resulta costoso para n grande.

- 1.6.91** (★★) **A MANO** Escriba una recursividad que cuente cuántas veces aparece un valor a en una lista L .

SOLUCIÓN

```
In[ ] :=
```

```
Cuenta[a_, L_] := If[L == {}, 0, If[First[L] == a, 1, 0] + Cuenta[a, Rest[L]]]
```

Sobre $a = 3$ y $L = \{3, 1, 3, 5, 3\}$ devuelve 3. La estructura es la misma del conteo de positivos: cambia únicamente la condición que se evalúa.

- 1.6.92** (★★) **MIXTO** Calcule 2^{10} con **PotenciaPos** y explique su relación con la multiplicación recursiva.

SOLUCIÓN

```
In[ ] :=
```

```
PotenciaPos[2, 10]
```

```
Out[ ] =
```

```
1024
```

La potencia es una multiplicación repetida y la multiplicación una suma repetida. Encadenando **Multipli** se obtiene el mismo 1024, pero con muchas más operaciones elementales: cada nivel de abstracción ahorra trabajo al anterior.

- 1.6.93** (★★★) **A MANO** Construya una recursividad que decida si una lista está ordenada de forma ascendente.

SOLUCIÓN

```
In[ ] :=
```

```
Ordenada[L_] := If[Length[L] <= 1, True, If[First[L] > L[[2]], False, Ordenada[Rest[L]]]]
```

El caso raíz cubre las listas de cero o un elemento, siempre ordenadas. La definición base compara los dos primeros y, si el orden se respeta, delega en el resto de la lista.

- 1.6.94** (★★) **VilCretas** Use `code->True` en MCD para ver su programación y compárela con el algoritmo de Euclides escrito a mano.

SOLUCIÓN

In[] :=

```
MCD[84, 36, code -> True]
```

El código reproduce la regla de Euclides: el caso raíz es $\text{mcd}(n, 0) = n$ y la definición base sustituye el par por $(m, n \bmod m)$. Es exactamente el procedimiento manual, expresado en una sola línea.

- 1.6.95** (★★) **A MANO** Escriba una recursividad para $\prod_{i=1}^n \frac{i}{i+1}$ y obtenga el valor para $n = 4$.

SOLUCIÓN

In[] :=

```
P[n_]:=If[n == 1, 1/2, P[n - 1]*(n/(n + 1))]
```

A mano: $\frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}$. El valor para $n = 4$ es $\frac{1}{5}$. El producto es telescópico: casi todos los factores se cancelan y queda $\frac{1}{n+1}$.

- 1.6.96** (★★) **MIXTO** Programe usted mismo la suma de dígitos y compare el resultado con `SumaDigi`.

SOLUCIÓN

In[] :=

```
sd[n_] := If[n < 10, n, Mod[n, 10] + sd[Quotient[n, 10]]];
{sd[98765], SumaDigi[98765]}
```

Out[] =

```
{35, 35}
```

`Mod` extrae la última cifra y `Quotient` elimina esa cifra. El caso raíz es el número de una sola cifra, que es su propia suma.

- 1.6.97** (★★★) **MIXTO** Programe el algoritmo de *Euclides* para el máximo común divisor y compruebe que coincide con `MCD`.

SOLUCIÓN

In[] :=

```
mcd[a_, b_] := If[b == 0, a, mcd[b, Mod[a, b]]];
{mcd[1071, 462], MCD[1071, 462]}
```

Out[] =

```
{21, 21}
```

En cada paso el segundo argumento pasa a ser el residuo, que es estrictamente menor: eso garantiza la terminación. La versión por restas sucesivas también funciona, pero necesita muchas más llamadas cuando los números son desiguales.

- 1.6.98** (★★★) **MIXTO** Use la recursividad anterior para calcular el mínimo común múltiplo de 12 y 18.

SOLUCIÓN**In[] :=**

```
mcd[a_, b_] := If[b == 0, a, mcd[b, Mod[a, b]]];
mcm[a_, b_] := a*b/mcd[a, b];
mcm[12, 18]
```

Out[] =

36

Se apoya en la identidad $\text{mcd}(a, b) \cdot \text{mcm}(a, b) = ab$. No hace falta una recursividad nueva: basta reutilizar la existente, que es lo que se espera de una función bien definida.

- 1.6.99** (★★) **MIXTO** Programe una recursividad que cuente cuántas veces se puede dividir un número entre dos antes de que resulte impar.

SOLUCIÓN**In[] :=**

```
cnt2[n_] := If[OddQ[n], 0, 1 + cnt2[n/2]];
cnt2[96]
```

Out[] =

5

El $96 = 2^5 \cdot 3$ admite cinco divisiones. Es el exponente del 2 en la factorización prima, y el conteo tiene coste $\Theta(\ln n)$ porque cada paso reduce el valor a la mitad.

- 1.6.100** (★★) **MIXTO** Programe la suma de los elementos de una lista de forma recursiva.

SOLUCIÓN**In[] :=**

```
sumL[L_] := If[L === {}, 0, First[L] + sumL[Rest[L]]];
sumL[{4, 8, 15, 16, 23, 42}]
```

Out[] =

108

El caso raíz es la **lista vacía**, cuyo valor natural es cero. Elegir la lista de un elemento como caso raíz también funciona, pero entonces la función falla al recibir una lista vacía.

- 1.6.101** (★★★) **MIXTO** Programe una recursividad que invierta una lista.

SOLUCIÓN**In[] :=**

```
rev[L_] := If[L === {}, {}, Append[rev[Rest[L]], First[L]]];
rev[{1, 2, 3, 4, 5}]
```

Out[] =

{5, 4, 3, 2, 1}

El primer elemento se coloca **al final** del resultado de invertir el resto. El coste es cuadrático, porque cada **Append** recorre la lista ya construida; una versión de cola con acumulador lo hace en tiempo lineal.

1.6.102 (★★★) MIXTO Programe una recursividad que aplane una lista con sublistas anidadas.

SOLUCIÓN

In[] :=

```
apl[L_] := If[L == {}, {},
  If[ListQ[First[L]], Join[apl[First[L]], apl[Rest[L]]],
    Prepend[apl[Rest[L]], First[L]]];
apl[{1, {2, 3}, {4, {5, 6}}}]
```

Out[] =

{1, 2, 3, 4, 5, 6}

Es una recursividad **doble**: baja por el primer elemento cuando es una lista y por el resto siempre. La profundidad de anidamiento no importa, porque cada nivel se trata igual que el anterior.

1.6.103 (★★) MIXTO Construya la fila 6 del triángulo de *Pascal* con la recursividad del coeficiente binomial.

SOLUCIÓN

In[] :=

```
cbin[n_, k_] := If[k == 0 || k == n, 1,
  cbin[n - 1, k - 1] + cbin[n - 1, k]];
Table[cbin[6, k], {k, 0, 6}]
```

Out[] =

{1, 6, 15, 20, 15, 6, 1}

La fila es simétrica porque $\binom{n}{k} = \binom{n}{n-k}$, propiedad que la propia recursividad refleja al intercambiar las dos ramas.

1.6.104 (★★) MIXTO Calcule los divisores de 360 y cuente cuántos son.

SOLUCIÓN

In[] :=

```
Divisores[360]
```

Out[] =

{1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360}

In[] :=

```
Length[Divisores[360]]
```

Out[] =

24

Son 24. Como $360 = 2^3 \cdot 3^2 \cdot 5$, la fórmula del número de divisores da $(3+1)(2+1)(1+1) = 24$, sin necesidad de listarlos.

1.6.105 (★★★) A MANO Usando la fórmula del ejercicio anterior, determine el menor número natural con exactamente 12 divisores.

SOLUCIÓN Hay que repartir el 12 como producto de factores $(e_i + 1)$ y asignar los exponentes mayores a los primos menores. Las opciones son $12 = 12$, $6 \cdot 2$, $4 \cdot 3$, $3 \cdot 2 \cdot 2$, que dan $2^{11} = 2048$, $2^5 3 = 96$, $2^3 3^2 = 72$ y $2^2 \cdot 3 \cdot 5 = 60$. El menor es **60**.

1.6.106 (★★) MIXTO Compruebe el resultado anterior con `Divisores`.

SOLUCIÓN

`In[ ] :=`

```
{Length[Divisores[60]], Length[Divisores[72]], Length[Divisores[96]]}
```

`Out[ ] =`

```
{12, 12, 12}
```

Los tres tienen doce divisores, y 60 es el menor de ellos. Contar divisores no requiere factorizar a mano si se dispone del comando, pero la fórmula explica **por qué** el mínimo es ese.

1.6.107 (★★★) MIXTO Programe la conversión de un natural a binario mediante una recursividad.

SOLUCIÓN

`In[ ] :=`

```
bin[n_] := If[n == 0, "", StringJoin[bin[Quotient[n, 2]], ToString[Mod[n, 2]]];
{bin[45], bin[100]}
```

`Out[ ] =`

```
{101101, 1100100}
```

La cifra menos significativa se calcula primero pero se escribe **al final**: por eso la llamada recursiva va delante en el `StringJoin`. El caso raíz devuelve la cadena vacía, no un cero, para no arrastrar un cero a la izquierda.

1.6.108 (★★★) MIXTO Programe la recursividad de las torres de *Hanoi* y calcule los movimientos para 15 discos.

SOLUCIÓN

`In[ ] :=`

```
han[n_] := If[n == 1, 1, 2 han[n - 1] + 1];
{han[15], Table[han[k], {k, 1, 8}]}
```

`Out[ ] =`

```
{32767, {1, 3, 7, 15, 31, 63, 127, 255}}
```

Son $32\,767 = 2^{15} - 1$ movimientos. La tabla muestra que cada disco añadido **más que duplica** el trabajo: es el crecimiento exponencial que hace impracticable el juego con 64 discos.

1.6.109 (★★★) A MANO Demuestre por inducción que la recursividad de *Hanoi* vale $2^n - 1$.

SOLUCIÓN Para $n = 1$: $2^1 - 1 = 1$, correcto. Supuesto $a_{n-1} = 2^{n-1} - 1$,

$$a_n = 2a_{n-1} + 1 = 2(2^{n-1} - 1) + 1 = 2^n - 2 + 1 = 2^n - 1$$

La inducción es el argumento natural para una recursividad: se comprueba el caso raíz y se propaga la definición base.

1.6.110 (★★) **MIXTO** Calcule $\sum_{i=1}^n i^3$ para $n = 6$ con **Sumatoria** y compare con $(\sum i)^2$.

SOLUCIÓN

In[] :=

`Sumatoria[{1, n, i^3}, 6]`

Out[] =

441

La suma de los seis primeros cubos vale $441 = 21^2$, y 21 es justamente $1 + 2 + \dots + 6$. Se cumple la identidad $\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2$.

1.6.111 (★★★) **MIXTO** Calcule $\sum_{i=2}^n \frac{1}{i(i+1)}$ para $n = 5$ y conjeture la fórmula general.

SOLUCIÓN

In[] :=

`Sumatoria[{2, n, 1/(i (i + 1))}, 5]`

Out[] =

1/3

Vale $\frac{1}{3}$. Descomponiendo en fracciones parciales, $\frac{1}{i(i+1)} = \frac{1}{i} - \frac{1}{i+1}$, la suma **telescópica** deja $\frac{1}{2} - \frac{1}{n+1}$, que para $n = 5$ da $\frac{1}{2} - \frac{1}{6} = \frac{1}{3}$.

1.6.112 (★★) **MIXTO** Calcule $\prod_{i=1}^n (2i-1)$ para $n = 5$ e interprete el resultado.

SOLUCIÓN

In[] :=

`Productoria[{1, n, 2 i - 1}, 5]`

Out[] =

945

Es $1 \cdot 3 \cdot 5 \cdot 7 \cdot 9 = 945$, el *doble factorial* de 9. Se puede escribir como $\frac{10!}{2^5 \cdot 5!}$: el producto de los impares es el factorial dividido entre el producto de los pares.

1.6.113 (★★★) **MIXTO** Calcule $\prod_{i=2}^n \frac{i}{i+1}$ para $n = 6$ y explique el resultado.

SOLUCIÓN**In[] :=**`Productoria[{2, n, i/(i + 1)}, 6]`**Out[] =**`2/7`

El producto es **telescópico**: cada numerador cancela el denominador anterior y solo sobreviven el primer numerador y el último denominador, $\frac{2}{n+1}$. Para $n = 6$ da $\frac{2}{7}$.

- 1.6.114** (★★) **MIXTO** Ordene la lista $\{9, 2, 7, 4, 1, 8, 3\}$ con **Quicksort** y describa lo que muestra.

SOLUCIÓN**In[] :=**`Quicksort[{9, 2, 7, 4, 1, 8, 3}]`**Out[] =**`{1, 2, 3, 4, 7, 8, 9}`

El comando despliega las listas intermedias de cada partición. *Quicksort* es recursivo por naturaleza: escoge un pivote, separa en dos sublistas y se invoca sobre cada una; los casos raíz son las listas de cero o un elemento.

- 1.6.115** (★★★) **A MANO** ¿Por qué *quicksort* es una recursividad múltiple y cuántas llamadas hace en el mejor caso?

SOLUCIÓN Porque cada invocación genera **dos**: una por cada sublista. En el mejor caso el pivote parte la lista por la mitad, de modo que hay $\log_2 n$ niveles y en cada uno se recorren los n elementos: coste $\Theta(n \ln n)$. En el peor caso el pivote deja una sublista vacía y el árbol degenera en una cadena de n niveles, con coste $\Theta(n^2)$.

- 1.6.116** (★★) **MIXTO** Calcule el producto 23×17 con **Multipli** y explique en qué se basa.

SOLUCIÓN**In[] :=**`Multipli[23, 17]`**Out[] =**`391`

El comando multiplica mediante sumas sucesivas: 23×17 se reduce a $23 + 23 \times 16$, y así hasta el caso raíz. Hace 17 llamadas, una por unidad del segundo factor, de modo que es preferible pasar el menor de los dos como segundo argumento.

- 1.6.117** (★★★) **A MANO** Reescriba la idea de **Multipli** para que use $\Theta(\ln n)$ llamadas en vez de $\Theta(n)$.

SOLUCIÓN Se aplica la misma técnica de la potenciación rápida, duplicando en vez de sumar de uno en uno:

$$a \times b = \begin{cases} 0 & b = 0 \\ 2(a \times \frac{b}{2}) & b \text{ par} \\ a + a \times (b - 1) & b \text{ impar} \end{cases}$$

Cada dos pasos, a lo sumo, el segundo factor se reduce a la mitad, de modo que el número de llamadas es del orden de $\log_2 b$.

1.6.118 (★★★) MIXTO Compruebe la reescritura anterior y compare su resultado con `Multipli`.

SOLUCIÓN

`In[ ] :=`

```
mrap[a_, b_] := If[b == 0, 0,
  If[EvenQ[b], 2 mrap[a, b/2], a + mrap[a, b - 1]];
{mrap[23, 17], Multipli[23, 17]}
```

`Out[ ] =`

{391, 391}

El resultado es el mismo y el número de llamadas baja de 17 a 8. Es el mismo salto de orden que separa `PotenciaPos` de la potenciación rápida.

1.6.119 (★★) A MANO Escriba, sin programar, cuántas llamadas hace `Multiptos[12, 100]` y cuál es el último valor que devuelve.

SOLUCIÓN Devuelve los múltiplos de 12 menores que 100: 12, 24, ..., 96, ocho valores, y el último es 96. El número de llamadas es $\lfloor 99/12 \rfloor = 8$ más la que detecta el fin.

1.6.120 (★★★) A MANO Una recursividad calcula $f(n) = f(n-1) + f(n-1) + 1$ con $f(0) = 0$. Halle la fórmula explícita y el número de llamadas.

SOLUCIÓN La definición equivale a $f(n) = 2f(n-1) + 1$, la misma de *Hanoi*, de modo que $f(n) = 2^n - 1$. Pero escrita con dos llamadas el conteo es $2^{n+1} - 1$: se calcula **dos veces** exactamente lo mismo. Escribir $2f(n-1)$ en vez de $f(n-1) + f(n-1)$ no cambia el valor y divide el trabajo entre dos elevado a n .

1.4 Recursividad de cola

1.6.121 (★) A MANO Explique en qué se diferencia una recursividad de cola de una de pila.

SOLUCIÓN En la de pila, la llamada recursiva es *parte de una expresión* que aún debe completarse al regresar: cada nivel deja trabajo pendiente. En la de cola, la llamada recursiva es *la última operación*: el resultado ya viene acumulado en un parámetro, de modo que al alcanzar el caso raíz no queda nada por resolver en la subida.

1.6.122 (★) VilCretas Compare `Factoriales[10]` y `FactorialesCola[10]` y comente el resultado.

SOLUCIÓN

`In[ ] :=`

```
Factoriales[10]
FactorialesCola[10]
```

`Out[ ] =`

3628800

3628800

Ambas devuelven $10! = 3628800$. La diferencia no está en el resultado sino en cómo se obtiene: la segunda no acumula operaciones pendientes en la pila.

- 1.6.123** (★★) **A MANO** Transforme en recursividad de cola el programa $S[n_]:=If[n == 1, 1, S[n - 1] + n]$.

SOLUCIÓN Se añade un parámetro acumulador con valor inicial:

$In[] :=$

```
SCola[n_, Suma_ : 0] := If[n == 0, Suma, SCola[n - 1, Suma + n]]
```

El acumulador transporta la suma parcial hacia abajo, de modo que el caso raíz ya dispone del resultado completo y solo tiene que devolverlo.

- 1.6.124** (★★) **A MANO** Escriba una recursividad de cola para $\prod_{i=1}^n (2i - 1)$ y calcule su valor en $n = 4$.

SOLUCIÓN

$In[] :=$

```
PCola[n_, Producto_ : 1] := If[n == 0, Producto, PCola[n - 1, Producto*(2 n - 1)]]
```

A mano: $1 \cdot 3 \cdot 5 \cdot 7 = 105$. El acumulador arranca en 1, el neutro del producto.

- 1.6.125** (★★) **VilCretas** Calcule F_{20} y F_{30} con `NFibonacciCola` y explique por qué esta versión permite valores mucho mayores que la de pila.

SOLUCIÓN

$In[] :=$

```
NFibonacciCola[20]
```

```
NFibonacciCola[30]
```

Out[] =

```
6765
```

```
832040
```

La versión de pila vuelve a calcular los mismos términos en las dos ramas del árbol de llamadas, de modo que el trabajo se dispara. La de cola avanza una sola vez por cada término, arrastrando los dos últimos valores en los acumuladores.

- 1.6.126** (★★) **MIXTO** Obtenga a mano $10!$ mediante una recursividad de cola, indicando el contenido del acumulador en cada paso, y verifíquelo.

SOLUCIÓN Partiendo de `Producto = 1` y bajando desde 10:

1, 10, 90, 720, 5040, 30240, 151200, 604800, 1814400, 3628800

El caso raíz devuelve el último acumulado.

$In[] :=$

```
FactorialesCola[10]
```

Out[] =

```
3628800
```

- 1.6.127** (★★★) **A MANO** Construya una recursividad de cola para $a_n = 3a_{n-1} + 2$ con $a_1 = 1$ y calcule a_5 .

SOLUCIÓN Se necesita un contador que sepa en qué término va, además del acumulador:

In[] :=

```
ACola[n_, Cont_ : 1, Valor_ : 1] := If[Cont == n, Valor, ACola[n, Cont + 1, 3*Valor + 2]]
```

A mano: $a_1 = 1$, $a_2 = 5$, $a_3 = 17$, $a_4 = 53$, $a_5 = 161$.

- 1.6.128 (★★★)** **A MANO** Escriba una recursividad de cola para la sucesión de *Lucas* ($L_1 = 1$, $L_2 = 3$, $L_n = L_{n-1} + L_{n-2}$) y obtenga L_{10} .

SOLUCIÓN Hacen falta dos acumuladores, uno por cada término que la definición base necesita:

In[] :=

```
LCola[n_, Cont_ : 2, A_ : 1, B_ : 3] := If[Cont == n, B, LCola[n, Cont + 1, B, A + B]]
```

La sucesión es 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, de modo que $L_{10} = 123$.

- 1.6.129 (★★)** **A MANO** Convierta en recursividad de cola el conteo de dígitos de un entero positivo.

SOLUCIÓN

In[] :=

```
CifrasCola[n_, Cont_ : 1] := If[n < 10, Cont, CifrasCola[Quotient[n, 10], Cont + 1]]
```

El acumulador cuenta los dígitos ya retirados. Sobre 98765 devuelve 5, igual que la versión de pila.

- 1.6.130 (★★★)** **A MANO** Escriba una recursividad de cola que sume los dígitos de un entero positivo y compárela con la versión de pila.

SOLUCIÓN

In[] :=

```
SumaDigiCola[n_, Suma_ : 0] := If[n == 0, Suma, SumaDigiCola[Quotient[n, 10], Suma + Mod[n, 10]]]
```

La versión de pila deja pendiente una suma en cada nivel; ésta la resuelve al bajar. Sobre 98765 ambas devuelven 35.

- 1.6.131 (★★)** **MIXTO** Verifique que la versión de cola y la de pila del factorial coinciden para $n = 0, 1, \dots, 8$.

SOLUCIÓN

In[] :=

```
Table[Factoriales[n] == FactorialesCola[n], {n, 0, 8}]
```

Out[] =

```
{True, True, True, True, True, True, True, True, True}
```

Las nueve comparaciones resultan verdaderas: la transformación a recursividad de cola preserva el resultado y solo cambia la forma de calcularlo.

- 1.6.132 (★★★)** **A MANO** Explique por qué la recursividad de cola necesita un contador cuando el término general depende explícitamente de n , y no lo necesita en el factorial.

SOLUCIÓN En el factorial la bajada puede usar el propio n como contador, porque el factor que se acumula es el valor actual del argumento. Cuando el término general depende de la posición, por ejemplo $a_n = 2a_{n-1} + n^2$, la bajada pierde la referencia de en qué término va: al restar uno al argumento no se sabe qué índice corresponde a cada paso. El contador restituye esa información y permite que el acumulador use n dentro de la fórmula.

- 1.6.133** (★★) **MIXTO** Programe el factorial de cola con un acumulador ascendente y compare con Factoriales.

SOLUCIÓN

In[] :=

```
fc[n_] := Module[{g}, g[i_, a_] := If[i > n, a, g[i + 1, a*i]]; g[1, 1];
{fc[12], Factoriales[12]}
```

Out[] =

```
{479001600, 479001600}
```

El acumulador **a** arranca en 1 y absorbe cada factor. A diferencia de la versión de pila, al llegar al caso raíz el resultado **ya está listo**: no hay nada que multiplicar de vuelta.

- 1.6.134** (★★★) **MIXTO** Programe la sucesión de Fibonacci de cola con dos acumuladores y tabule doce términos.

SOLUCIÓN

In[] :=

```
fibc[n_] := Module[{g}, g[i_, a_, b_] := If[i == n, a, g[i + 1, b, a + b]];
g[1, 1, 1];
Table[fibc[k], {k, 1, 12}]
```

Out[] =

```
{1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144}
```

Los dos acumuladores arrastran los términos F_i y F_{i+1} , y en cada paso **se desplazan**: el segundo pasa a primero y la suma pasa a segundo. Así se hace en n pasos lo que la versión de pila hace en $2F_n - 1$.

- 1.6.135** (★★) **MIXTO** Programe la suma de dígitos de cola y compruebe que coincide con SumaDigi.

SOLUCIÓN

In[] :=

```
sdc[n_] := Module[{g}, g[m_, s_] := If[m == 0, s,
g[Quotient[m, 10], s + Mod[m, 10]]; g[n, 0]];
{sdc[987654321], SumaDigi[987654321]}
```

Out[] =

```
{45, 45}
```

El acumulador arranca en 0, el elemento neutro de la suma. En la versión de producto arrancaría en 1: el acumulador siempre se inicializa con el **neutro de la operación** que se acumula.

1.6.136 (★★★) MIXTO Programe una recursividad de cola que invierta las cifras de un número.

SOLUCIÓN

In[] :=

```
inv[n_] := Module[{g}, g[m_, r_] := If[m == 0, r,
  g[Quotient[m, 10], 10 r + Mod[m, 10]]; g[n, 0]];
{inv[12345], inv[90210]}
```

Out[] =

{54321, 1209}

El acumulador se multiplica por diez y recibe la cifra extraída. Con 90210 el resultado es 1209 y no 01209: los ceros a la izquierda desaparecen, lo que muestra que invertir dos veces no siempre devuelve el número original.

1.6.137 (★★★) A MANO ¿Para qué números invertir dos veces con el programa anterior devuelve el número de partida?

SOLUCIÓN Exactamente para los que **no terminan en cero**. Si n acaba en cero, la primera inversión lo pierde y la segunda ya no puede recuperarlo: $90210 \rightarrow 1209 \rightarrow 9021 \neq 90210$. Es un buen recordatorio de que una función puede no ser inyectiva aunque parezca reversible.

1.6.138 (★★) MIXTO Programe la potenciación de cola y compruebe 3^{10} .

SOLUCIÓN

In[] :=

```
potc[b_, e_] := Module[{g}, g[i_, a_] := If[i == 0, a, g[i - 1, a*b]];
  g[e, 1]];
{potc[3, 10], 3^10}
```

Out[] =

{59049, 59049}

El contador baja y el acumulador multiplica. Es la traducción directa de `PotenciaPos` a forma de cola: mismo número de pasos, pero sin operaciones pendientes.

1.6.139 (★★★) MIXTO Programe el máximo de una lista con recursividad de cola.

SOLUCIÓN

In[] :=

```
mxc[L_] := Module[{g}, g[R_, m_] := If[R === {}, m,
  g[Rest[R], Max[m, First[R]]]]; g[Rest[L], First[L]];
mxc[{3, 17, 8, 25, 4}]
```

Out[] =

25

Aquí el acumulador no arranca en un neutro sino en el **primer elemento** de la lista: no existe un valor neutro para el máximo dentro de los enteros. Por eso la función exige que la lista no sea vacía.

- 1.6.140 (★★) **MIXTO** Programe una recursividad de cola que cuente las apariciones de un valor en una lista.

SOLUCIÓN

In[] :=

```
cntc[L_, x_] := Module[{g}, g[R_, c_] := If[R == {}, c,
  g[Rest[R], If[First[R] == x, c + 1, c]]]; g[L, 0]]
cntc[{1, 3, 3, 7, 3, 9}, 3]
```

Out[] =

3

El acumulador es un contador que arranca en cero y solo avanza cuando hay coincidencia. La lista se recorre una sola vez, de modo que el coste es $\Theta(n)$.

- 1.6.141 (★★★) **MIXTO** Escriba el algoritmo de *Euclides* en forma de cola y verifíquelo.

SOLUCIÓN

In[] :=

```
gcdc[a_, b_] := Module[{g}, g[x_, y_] := If[y == 0, x, g[y, Mod[x, y]]];
  g[a, b]]
{gcdc[2024, 748], GCD[2024, 748]}
```

Out[] =

{44, 44}

Euclides es de cola **por naturaleza**: la llamada recursiva es lo último que se hace y su resultado se devuelve sin modificar. No hace falta añadir ningún acumulador.

- 1.6.142 (★★★) **A MANO** ¿Por qué algunas recursividades, como la de *Euclides*, ya son de cola sin acumulador, y otras, como el factorial, necesitan uno?

SOLUCIÓN Porque en *Euclides* el valor que se devuelve es **exactamente** el de la llamada interna: no queda nada por hacer. En el factorial, en cambio, hay que multiplicar por n después de volver, y ese producto es la operación pendiente. El acumulador sirve para *adelantar* ese trabajo antes de la llamada, de modo que ya no quede nada al regresar.

- 1.6.143 (★★) **MIXTO** Programe la recursividad de *Hanoi* en forma de cola.

SOLUCIÓN

In[] :=

```
hanc[n_] := Module[{g}, g[i_, a_] := If[i == 0, a, g[i - 1, 2 a + 1]];
  g[n, 0]]
{hanc[15], 2^15 - 1}
```

Out[] =

{32767, 32767}

El acumulador aplica la regla $a \mapsto 2a + 1$ tantas veces como discos haya, partiendo de cero. La versión de cola no reduce el **resultado**, que sigue siendo exponencial: reduce el número de llamadas, que pasa de exponencial a lineal.

1.6.144 (★★★) A MANO Distinga entre el coste del *resultado* y el coste del *cálculo* a partir del ejercicio anterior.

SOLUCIÓN El resultado de *Hanoi* es $2^n - 1$: mover los discos lleva un tiempo exponencial y eso no lo arregla ningún programa. Calcular *cuántos* movimientos hacen falta, en cambio, cuesta n pasos con la versión de cola. Confundir ambas cosas es el error más común al analizar una recursividad.

1.6.145 (★★) MIXTO Programe la suma de los cuadrados de 1 a n con recursividad de cola y compruebe $n = 10$.

SOLUCIÓN

In[] :=

```
sc[n_] := Module[{g}, g[i_, s_] := If[i > n, s, g[i + 1, s + i^2]];
  g[1, 0]];
{sc[10], Sum[i^2, {i, 1, 10}]}
```

Out[] =

{385, 385}

Coincide con $\frac{10 \cdot 11 \cdot 21}{6} = 385$. La condición de parada usa $i > n$ y no $i == n$: así el último sumando también entra en el acumulador.

1.6.146 (★★★) A MANO ¿Qué ocurre en el programa anterior si se escribe $i == n$ como condición de parada?

SOLUCIÓN Se pierde el último sumando: para $n = 10$ devolvería 285 en vez de 385, porque se detiene *antes* de sumar 10^2 . Es el error de *desplazamiento por uno*, y es especialmente traicionero porque el programa no falla: devuelve un valor equivocado sin ningún aviso.

1.6.147 (★★★) MIXTO Convierta a forma de cola la recursividad $a_n = 3a_{n-1} + 2$ con $a_0 = 1$ y compruebe a_8 .

SOLUCIÓN

In[] :=

```
ac[n_] := Module[{g}, g[i_, a_] := If[i == 0, a, g[i - 1, 3 a + 2]];
  g[n, 1]];
{ac[8], Table[ac[k], {k, 0, 5}]}
```

Out[] =

{13121, {1, 5, 17, 53, 161, 485}}

El acumulador aplica la regla $a \mapsto 3a + 2$ una vez por nivel. La fórmula explícita es $a_n = 2 \cdot 3^n - 1$, de donde $a_8 = 2 \cdot 6561 - 1 = 13121$.

1.5 Generadores automáticos de programas recursivos

1.6.148 (★★) VilCretas Use *Productoria* para que el propio paquete escriba el programa recursivo de $\prod_{i=1}^{n-4} 2i$, y explique qué devuelve el comando cuando se le da un solo argumento.

SOLUCIÓN

In[] :=

```
Productoria[{1, n - 4, 2*i}]
```

Con un solo argumento el comando **no devuelve un valor**: imprime en pantalla el programa recursivo de pila que calcula ese producto, con su caso raíz y su llamada. El vector tiene siempre tres componentes en este orden: **índice inferior**, **índice superior** en función de n , y **expresión interna** en función de i .

- 1.6.149 (★★) **MIXTO** Calcule con `Productoria` el valor del producto anterior para $n = 9$ y compruébelo con `Product`.

SOLUCIÓN

In[] :=

```
{Productoria[{1, n - 4, 2*i}, 9], Product[2*i, {i, 1, 5}]}
```

Out[] =

```
{3840, 3840}
```

Con $n = 9$ el índice superior vale $9 - 4 = 5$, de modo que el producto es $2 \cdot 4 \cdot 6 \cdot 8 \cdot 10 = 3840$. El **segundo argumento** cambia el comportamiento del comando: ya no escribe el programa, lo **ejecuta**.

- 1.6.150 (★★) **MIXTO** Obtenga con `SumatoriaCola` el valor de $\sum_{i=1}^n i^2$ para $n = 10$ y contrástelo con `Sum`.

SOLUCIÓN

In[] :=

```
{SumatoriaCola[{1, n, i^2}, 10], Sum[i^2, {i, 1, 10}]}
```

Out[] =

```
{385, 385}
```

Coinciden en 385. La diferencia con `Sumatoria` no está en el resultado sino en **cómo** se llega a él: aquí el paquete construye una recursividad **de cola**, que arrastra un acumulador y no deja operaciones pendientes al volver.

- 1.6.151 (★★★) **VilCretas** Pida ahora la versión con función auxiliar, con `SumatoriaColaAux`, y explique para qué sirve esa segunda función.

SOLUCIÓN

In[] :=

```
SumatoriaColaAux[{1, n, i^2}]
```

La recursividad de cola necesita un **acumulador** que no forma parte del enunciado del problema. `SumatoriaCola` lo esconde dentro de la misma función; `SumatoriaColaAux` lo saca a una **función auxiliar**, de modo que el usuario llama a una función de un solo argumento y esa función delega en otra de dos. Es la forma habitual de presentar la recursividad de cola cuando se quiere que la interfaz no cambie.

- 1.6.152 (★★★) **MIXTO** Compruebe que `ProductoriaCola` da el mismo valor que `Productoria` para $\prod_{i=1}^{n-4} 2i$ con $n = 9$.

SOLUCIÓN

In[] :=

```
{ProductoriaCola[{1, n - 4, 2*i}, 9], Productoria[{1, n - 4, 2*i}, 9]}
```

Out[] =

```
{3840, 3840}
```

Los dos dan 3840. Que el **valor** coincida es lo esperado: pila y cola calculan lo mismo. Lo que cambia es el consumo de la pila de llamadas, que en la versión de cola es constante si el intérprete optimiza la última llamada.

- 1.6.153** (★★★) **VilCretas** Genere con **ProductoriaColaAux** la versión con auxiliar del producto anterior y compare su encabezado con el de **ProductoriaCola**.

SOLUCIÓN

In[] :=

```
ProductoriaColaAux[{1, n - 4, 2*i}]
```

La función que ve el usuario conserva **un solo parámetro**; la auxiliar recibe dos, el índice y el acumulador, y en el producto ese acumulador arranca en 1, no en 0, como en la suma. Confundir el neutro es el error más común al escribir a mano una productoria de cola.

- 1.6.154** (★★★) **VilCretas** Los comandos anteriores exigen que el límite superior sea lineal. Con **SumatoriaGeneral** calcule $\sum_{i=1}^{n^2} i^2$, cuyo límite **no** lo es.

SOLUCIÓN

In[] :=

```
SumatoriaGeneral[{1, n^2, i^2}]
```

Sumatoria y **SumatoriaCola** suponen que el índice superior es un polinomio de primer grado en n , porque de ahí deducen cuánto hay que retroceder en cada llamada. Con n^2 esa cuenta ya no sirve. La versión **general** es la que no hace esa suposición.

- 1.6.155** (★★★) **MIXTO** Compruebe con **Sum** el valor de esa sumatoria para $n = 4$.

SOLUCIÓN

In[] :=

```
Sum[i^2, {i, 1, 4^2}]
```

Out[] =

```
1496
```

Para $n = 4$ el límite superior es 16 y la suma vale $\frac{16 \cdot 17 \cdot 33}{6} = 1496$. Conviene fijarse en el crecimiento: al pasar de n a $n + 1$ el número de términos no crece en uno, sino en $2n + 1$.

- 1.6.156** (★★★) **VilCretas** Pida la misma sumatoria en versión de cola con la opción **cola** -> **True**.

SOLUCIÓN

In[] :=

```
SumatoriaGeneral[{1, n^2, i^2}, cola -> True]
```

La opción `cola -> True` devuelve una recursividad de cola en lugar de una de pila. Es el mismo comando: **la opción, no otra función**, es lo que decide la forma del programa.

1.6.157 (★★★) **MIXTO** Haga lo análogo con `ProductoriaGeneral` para $\prod_{i=1}^n i$ y evalúelo en $n = 3$.

SOLUCIÓN

```
In[ ] :=
```

```
ProductoriaGeneral[{1, n^2, i}]
```

```
In[ ] :=
```

```
Product[i, {i, 1, 3^2}]
```

```
Out[ ] =
```

```
362880
```

Para $n = 3$ el producto es $9! = 362880$. Este ejemplo deja ver por qué importa el límite no lineal: con $n = 4$ ya sería $16!$, un número de trece cifras.

1.6.158 (★★) **VilCretas** Genere sumatorias de práctica al azar con `SumatoriaRandom` y compare los dos formatos de salida.

SOLUCIÓN

```
In[ ] :=
```

```
{SumatoriaRandom[10], SumatoriaRandom[10, vectorial -> True]}
```

Sin opciones devuelve la sumatoria **escrita como fórmula**, lista para leer; con `vectorial -> True` la devuelve como el vector de tres componentes que esperan `Sumatoria` y compañía. El segundo formato es el útil: se puede encadenar directamente.

1.6.159 (★★) **VilCretas** Encadene `ProductoriaRandom` con `Productoria` para generar un ejercicio y resolverlo en el mismo paso.

SOLUCIÓN

```
In[ ] :=
```

```
p = ProductoriaRandom[10, vectorial -> True, funcion -> 2];
```

```
Productoria[p, 12]
```

La opción `funcion -> 2` pide que la expresión interna **dependa de i** , con 1 sería una constante y con 3 una expresión más elaborada. Como el enunciado es aleatorio, cada ejecución plantea un producto distinto: es un generador de práctica, no un ejercicio fijo.

1.6.160 (★★★) **A MANO** ¿Por qué la versión de cola de una productoria necesita que el acumulador empiece en 1, mientras que la de una sumatoria empieza en 0?

SOLUCIÓN Porque el acumulador guarda el resultado **parcial**, y el valor con el que arranca debe ser el **elemento neutro** de la operación: 0 para la suma y 1 para el producto. Si una productoria de cola arrancara en 0, todo producto daría 0; y lo peor es que no habría ningún error, solo un resultado equivocado.

CAPÍTULO 2 • COMPLEMENTARIOS

Relaciones de recurrencia

2.1 Qué es una relación de recurrencia

- 2.4.1 (★) **A MANO** Indique el orden de cada relación y cuántas condiciones iniciales necesita:
 (a) $a_n = 5a_{n-1}$; (b) $a_n = a_{n-1} + a_{n-2}$; (c) $a_n = 2a_{n-2} - a_{n-3}$.

SOLUCIÓN El orden es la distancia al término más lejano que interviene, y ése es también el número de condiciones iniciales: (a) orden 1, una condición; (b) orden 2, dos condiciones; (c) orden 3, tres condiciones. En (c) el término a_{n-1} no aparece, pero el orden lo fija a_{n-3} .

- 2.4.2 (★) **A MANO** Escriba los seis primeros términos de $a_n = 3a_{n-1}$ con $a_0 = 2$.

SOLUCIÓN Cada término triplica al anterior:

$$2, 6, 18, 54, 162, 486$$

Es una progresión geométrica de razón 3.

- 2.4.3 (★) **VilCretas** Genere con RT los ocho primeros términos de $a_n = 3a_{n-1}$ con $a_0 = 2$ y obtenga su fórmula explícita con RR.

SOLUCIÓN

In[] :=

RT[{3}, {2}, 8]

Out[] =

{2, 6, 18, 54, 162, 486, 1458, 4374}

In[] :=

RR[{3}, {2}, n, inicio -> 0]

Out[] =

$2 \cdot 3^n$

La fórmula $a_n = 2 \cdot 3^n$ reproduce la lista: para $n = 0$ da 2 y para $n = 7$ da 4374.

- 2.4.4 (★★) **A MANO** Determine si $a_n = 2a_{n-1} + 5$ es homogénea y justifique la respuesta.

SOLUCIÓN No lo es. Una relación es homogénea cuando todos sus términos contienen algún a_{n-k} ; aquí el sumando 5 es un término independiente que no depende de la sucesión. Se dice que la relación es **lineal no homogénea** de orden uno, con parte no homogénea constante.

- 2.4.5 (★★) **MIXTO** Calcule a mano los cinco primeros términos de $a_n = 2a_{n-1} + 5$ con $a_0 = 1$ y verifíquelos.

SOLUCIÓN A mano: 1, 7, 19, 43, 91.

In[] :=

RT[{2, 5}, {1}, 7]

Out[] =

{1, 7, 19, 43, 91, 187, 379}

En RT el vector de coeficientes lleva primero el coeficiente de a_{n-1} y después la parte no homogénea.

- 2.4.6** (★★) **VilCretas** Obtenga la fórmula explícita de $a_n = 2a_{n-1} + 5$ con $a_0 = 1$ y compruébela para $n = 3$.

SOLUCIÓN

In[] :=

```
RR[{2, 5}, {1}, n, inicio -> 0]
```

Out[] =

```
-5 + 3*2^(1 + n)
```

Es decir $a_n = 3 \cdot 2^{n+1} - 5$. Para $n = 3$: $3 \cdot 16 - 5 = 43$, que coincide con el cuarto término de la lista del ejercicio anterior.

- 2.4.7** (★) **MIXTO** Escriba los primeros términos de $a_n = a_{n-1} + 3$ con $a_0 = 2$ y reconozca de qué tipo de progresión se trata.

SOLUCIÓN 2, 5, 8, 11, 14, 17, 20: cada término supera al anterior en 3, de modo que es una progresión aritmética de diferencia 3.

In[] :=

```
RR[{1, 3}, {2}, n, inicio -> 0]
```

Out[] =

```
2 + 3*n
```

La fórmula $a_n = 3n + 2$ es la de una recta, como corresponde a una progresión aritmética.

- 2.4.8** (★★) **A MANO** Modele con una relación de recurrencia el capital de una inversión de \$5000 al 13 % de interés compuesto anual.

SOLUCIÓN Cada año el capital se multiplica por 1,13:

$$a_n = 1,13 a_{n-1}, \quad a_0 = 5000$$

Es una relación homogénea lineal de orden uno. Su solución explícita es $a_n = 5000 (1,13)^n$.

- 2.4.9** (★★) **VilCretas** Calcule con RT el capital de la inversión anterior durante seis años, redondeado a colones enteros.

SOLUCIÓN

In[] :=

```
Round[RT[{1.13}, {5000}, 6]]
```

Out[] =

```
{5000, 5650, 6384, 7214, 8152, 9212}
```

Al sexto año el capital casi se ha duplicado, lo que ilustra el efecto del interés compuesto. Sin Round la salida conserva todos los decimales del cálculo en coma flotante.

- 2.4.10** (★★) **A MANO** Las torres de Hanoi con n discos requieren a_n movimientos. Justifique la relación $a_n = 2a_{n-1} + 1$ con $a_1 = 1$.

SOLUCIÓN Para mover n discos hay que: trasladar los $n - 1$ superiores a la varilla auxiliar (a_{n-1} movimientos), mover el disco grande (1 movimiento) y volver a trasladar los $n - 1$ sobre él (a_{n-1} movimientos). En total $2a_{n-1} + 1$. Con un solo disco basta un movimiento, de donde $a_1 = 1$.

- 2.4.11** (★★) **MIXTO** Obtenga los diez primeros valores de la relación de Hanoi y su fórmula explícita.

SOLUCIÓN

In[] :=

RT[{2, 1}, {1}, 10]

Out[] =

{1, 3, 7, 15, 31, 63, 127, 255, 511, 1023}

In[] :=

RR[{2, 1}, {1}, n, inicio -> 1]

Out[] =

-1 + 2^n

La fórmula $a_n = 2^n - 1$ explica la fama del problema: con 64 discos harían falta $2^{64} - 1$ movimientos.

- 2.4.12** (★★) **A MANO** ¿Cuántos movimientos exige la torre de Hanoi con 20 discos? Use la fórmula explícita.

SOLUCIÓN $a_{20} = 2^{20} - 1 = 1\,048\,575$. A un movimiento por segundo se tardarían algo más de doce días sin descanso.

- 2.4.13** (★★★) **A MANO** Una población de bacterias se triplica cada hora y además se le añaden 100 individuos al final de cada hora. Modele la situación y escriba los cuatro primeros términos si se parte de 50.

SOLUCIÓN

$$a_n = 3a_{n-1} + 100, \quad a_0 = 50$$

Los términos son 50, 250, 850, 2650. Es una relación lineal no homogénea de orden uno: la triplicación es la parte homogénea y el aporte fijo la parte no homogénea.

- 2.4.14** (★★) **A MANO** Escriba la relación de recurrencia que describe el número de regiones en que n rectas dividen el plano, si cada recta nueva corta a todas las anteriores en puntos distintos.

SOLUCIÓN La recta n -ésima cruza a las $n - 1$ anteriores en $n - 1$ puntos, que la parten en n tramos, y cada tramo divide en dos una región existente:

$$a_n = a_{n-1} + n, \quad a_0 = 1$$

Los primeros valores son 1, 2, 4, 7, 11, 16.

- 2.4.15** (★) **VilCretas** Abra el recurso interactivo del capítulo sobre el número de regiones.

SOLUCIÓN**In[] :=****CDFNRegions[]**

El comando no recibe argumentos: despliega un objeto interactivo donde se puede variar el número de rectas y observar cómo crecen las regiones, ilustrando la relación $a_n = a_{n-1} + n$.

- 2.4.16** (★★) **MIXTO** Genere los siete primeros términos de $a_n = 4a_{n-1}$ con $a_0 = 3$ y su fórmula explícita.

SOLUCIÓN**In[] :=****RT[{4}, {3}, 7]****Out[] =**

{3, 12, 48, 192, 768, 3072, 12288}

In[] :=**RR[{4}, {3}, n, inicio -> 0]****Out[] =** $3 \cdot 4^n$

Es una progresión geométrica de razón 4: $a_n = 3 \cdot 4^n$.

- 2.4.17** (★★) **MIXTO** Determine la fórmula explícita de $a_n = a_{n-1} + 4$ con $a_0 = 5$ y clasifique la sucesión.

SOLUCIÓN**In[] :=****RT[{1, 4}, {5}, 7]****Out[] =**

{5, 9, 13, 17, 21, 25, 29}

In[] :=**RR[{1, 4}, {5}, n, inicio -> 0]****Out[] =** $5 + 4 \cdot n$

Progresión aritmética de diferencia 4. La relación es lineal **no homogénea** de orden uno, y su solución es una recta.

- 2.4.18** (★★★) **MIXTO** La relación $a_n = 2a_{n-1} - a_{n-2}$ con $a_0 = 1$ y $a_1 = 4$ tiene una fórmula sorprendentemente simple. Halléla y explíquela.

SOLUCIÓN**In[] :=**`RT[{2, -1}, {1, 4}, 8]`**Out[] =**`{1, 4, 7, 10, 13, 16, 19, 22}`**In[] :=**`RR[{2, -1}, {1, 4}, n, inicio -> 0]`**Out[] =**`1 + 3*n`

Es aritmética de diferencia 3. La razón está en la ecuación característica $t^2 - 2t + 1 = 0$, con raíz doble $t = 1$: la solución general es $b_1 + b_2n$, es decir, una recta.

2.4.19 (★★) **MIXTO** Halle la fórmula explícita de $a_n = 3a_{n-1} - 2a_{n-2}$ con $a_0 = 2$ y $a_1 = 5$.

SOLUCIÓN**In[] :=**`RT[{3, -2}, {2, 5}, 8]`**Out[] =**`{2, 5, 11, 23, 47, 95, 191, 383}`**In[] :=**`RR[{3, -2}, {2, 5}, n, inicio -> 0]`**Out[] =**`-1 + 3*2^n`

Las raíces son 1 y 2, de modo que la solución combina una constante y una potencia de dos:
 $a_n = 3 \cdot 2^n - 1$.

2.4.20 (★★★) **MIXTO** Estudie $a_n = a_{n-1} + a_{n-2}$ con $a_0 = 2$ y $a_1 = 3$ y relacione el resultado con las sucesiones clásicas.

SOLUCIÓN**In[] :=**`RT[{1, 1}, {2, 3}, 9]`**Out[] =**`{2, 3, 5, 8, 13, 21, 34, 55, 89}`**In[] :=**`RR[{1, 1}, {2, 3}, n, inicio -> 0]`**Out[] =**

`2*Fibonacci[n] + LucasL[n]`

A partir del tercer término coincide con Fibonacci desplazado: $a_n = F_{n+3}$. El software lo expresa combinando Fibonacci y *Lucas*, que son las dos soluciones básicas de esa definición base.

- 2.4.21** (★★★) **MIXTO** Analice $a_n = 4a_{n-2}$ con $a_0 = 1$ y $a_1 = 2$: ¿por qué la fórmula resulta tan simple?

SOLUCIÓN

`In[ ] :=`

`RT[{0, 4}, {1, 2}, 8]`

`Out[ ] =`

`{1, 2, 4, 8, 16, 32, 64, 128}`

`In[ ] :=`

`RR[{0, 4}, {1, 2}, n, inicio -> 0]`

`Out[ ] =`

`2^n`

Porque las raíces son 2 y -2 , y las condiciones iniciales anulan la constante de $(-2)^n$. El coeficiente nulo de a_{n-1} hay que escribirlo: el vector es $\{0, 4\}$ y no $\{4\}$.

- 2.4.22** (★★) **A MANO** Escriba la relación de recurrencia del capital de un préstamo de \$12 000 al 5 % anual e indique su orden.

SOLUCIÓN Cada año el saldo se multiplica por 1,05:

$$a_n = 1,05 a_{n-1}, \quad a_0 = 12\,000$$

Es homogénea lineal de orden uno con una sola condición inicial, y su solución es $a_n = 12\,000 (1,05)^n$.

- 2.4.23** (★★) **VilCretas** Tabule el capital del ejercicio anterior durante siete años.

SOLUCIÓN

`In[ ] :=`

`Round[RT[{1.05}, {12000}, 7]]`

`Out[ ] =`

`{12000, 12600, 13230, 13892, 14586, 15315, 16081}`

En siete años el capital crece un 34 %. Sin `Round` la salida arrastra todos los decimales de la coma flotante.

- 2.4.24** (★★★) **MIXTO** Modele con una recurrencia el número de regiones que n rectas en posición general determinan en el plano, y halle su fórmula.

SOLUCIÓN Cada recta nueva corta a las $n - 1$ anteriores y añade n regiones: $a_n = a_{n-1} + n$ con $a_0 = 1$.

In[] :=

RT[{1, n}, {1}, 7, inicio -> 0]

Out[] =

{1, 2, 4, 7, 11, 16, 22}

In[] :=

RR[{1, n}, {1}, n, inicio -> 0]

Out[] =

(2 + n + n^2)/2

La fórmula es $a_n = 1 + \frac{n(n+1)}{2}$: crecimiento cuadrático.

2.4.25 (★★★) **MIXTO** Resuelva $a_n = a_{n-1} + 2n$ con $a_0 = 0$ y reconozca la sucesión.

SOLUCIÓN

In[] :=

RT[{1, 2 n}, {0}, 7, inicio -> 0]

Out[] =

{0, 2, 6, 12, 20, 30, 42}

In[] :=

RR[{1, 2 n}, {0}, n, inicio -> 0]

Out[] =

n*(1 + n)

Son los **números pronic**, $n(n+1)$: el doble de los triangulares. La parte no homogénea $2n$ depende de n , y por eso la solución es un polinomio de grado dos.

2.4.26 (★★★) **MIXTO** Resuelva $a_n = a_{n-1} + n^2$ con $a_0 = 0$ e interprete el resultado.

SOLUCIÓN

In[] :=

RT[{1, n^2}, {0}, 6, inicio -> 0]

Out[] =

{0, 1, 5, 14, 30, 55}

In[] :=

RR[{1, n^2}, {0}, n, inicio -> 0]

Out[] =

((1 + n)*(n + 2*n^2))/6

La sucesión acumula cuadrados, de modo que el término general es la suma $\sum_{i=1}^n i^2$ desplazada. Con una parte no homogénea de grado dos, la solución sube a grado tres.

2.4.27 (★★★) **A MANO** Generalice: si la parte no homogénea es un polinomio de grado k , ¿de qué grado es la solución de $a_n = a_{n-1} + p(n)$?

SOLUCIÓN De grado $k+1$. Sumar acumula: la diferencia $a_n - a_{n-1}$ recupera $p(n)$, y derivar en el mundo discreto baja un grado. Es el análogo de que la integral de un polinomio de grado k tenga grado $k+1$.

2.4.28 (★★) **MIXTO** Determine la fórmula explícita de $a_n = 2a_{n-1} + 3$ con $a_0 = 4$.

SOLUCIÓN

In[] :=

RT[{2, 3}, {4}, 7]

Out[] =

{4, 11, 25, 53, 109, 221, 445}

In[] :=

RR[{2, 3}, {4}, n, inicio -> 0]

Out[] =

-3 + 7*2^n

Es decir $a_n = 7 \cdot 2^n - 3$. El -3 es el **punto fijo** de la relación: si $a = 2a + 3$ entonces $a = -3$, y la sucesión se aleja de él duplicando la distancia en cada paso.

2.4.29 (★★★) **A MANO** Use la idea del punto fijo para resolver $a_n = 5a_{n-1} - 8$ con $a_0 = 3$ sin software.

SOLUCIÓN El punto fijo cumple $a = 5a - 8$, de donde $a = 2$. Definiendo $b_n = a_n - 2$ se obtiene $b_n = 5b_{n-1}$ con $b_0 = 1$, cuya solución es $b_n = 5^n$. Deshaciendo el cambio,

$$a_n = 5^n + 2$$

La técnica funciona siempre que la parte no homogénea sea **constante** y el coeficiente no valga uno.

2.4.30 (★★) **MIXTO** Compruebe con el software la solución del ejercicio anterior.

SOLUCIÓN

In[] :=

{RT[{5, -8}, {3}, 6], RR[{5, -8}, {3}, n, inicio -> 0]}

Out[] =

{3, 7, 27, 127, 627, 3127}, 2 + 5^n

Coincide: $a_n = 5^n + 2$. Los términos son 3, 7, 27, 127, ..., siempre dos unidades por encima de una potencia de cinco.

2.4.31 (★★★) **A MANO** ¿Qué ocurre con la técnica del punto fijo cuando el coeficiente vale uno, como en $a_n = a_{n-1} + 3$?

SOLUCIÓN No hay punto fijo: la ecuación $a = a + 3$ no tiene solución. Y en efecto la sucesión no se aleja de ningún valor, sino que crece linealmente. Es el caso en que la raíz característica coincide con el 1 de la parte no homogénea constante, y la solución particular deja de ser constante para pasar a ser lineal.

- 2.4.32** (★★) **A MANO** Indique el orden y el número de condiciones iniciales de: (a) $a_n = a_{n-1}a_{n-2}$; (b) $a_n = 5a_{n-4}$; (c) $a_n = a_{n-1} + a_{n-2} + a_{n-3}$.

SOLUCIÓN (a) Orden 2, dos condiciones, pero **no es lineal**: los términos anteriores se multiplican entre sí. (b) Orden 4, cuatro condiciones, lineal y homogénea. (c) Orden 3, tres condiciones, lineal y homogénea.

- 2.4.33** (★★★) **MIXTO** La relación $a_n = a_{n-1}a_{n-2}$ con $a_0 = 2$ y $a_1 = 2$ no es lineal. Calcule sus primeros términos y halle el patrón.

SOLUCIÓN Multiplicando: 2, 2, 4, 8, 32, 256, 8192, ...

In[] :=

```
nl[n_] := If[n <= 1, 2, nl[n - 1]*nl[n - 2]];
Table[nl[k], {k, 0, 6}]
```

Out[] =

{2, 2, 4, 8, 32, 256, 8192}

Los exponentes de 2 son 1, 1, 2, 3, 5, 8, 13: **Fibonacci**. Tomando logaritmos en base dos la relación se vuelve lineal, que es la técnica estándar para reducir una recurrencia multiplicativa.

- 2.4.34** (★★★) **A MANO** Formalice la reducción del ejercicio anterior.

SOLUCIÓN Con $b_n = \log_2 a_n$, la relación $a_n = a_{n-1}a_{n-2}$ se convierte en $b_n = b_{n-1} + b_{n-2}$, lineal homogénea, con $b_0 = b_1 = 1$. Su solución es $b_n = F_{n+1}$, de donde $a_n = 2^{F_{n+1}}$. El logaritmo transforma productos en sumas, igual que la raíz transforma potencias.

- 2.4.35** (★★) **A MANO** Modele con una recurrencia el número de subconjuntos de un conjunto de n elementos.

SOLUCIÓN Al añadir un elemento, cada subconjunto anterior da lugar a dos: el que no lo contiene y el que sí. Por lo tanto

$$a_n = 2a_{n-1}, \quad a_0 = 1$$

cuya solución es $a_n = 2^n$. El caso raíz $a_0 = 1$ corresponde al conjunto vacío, que tiene un solo subconjunto: él mismo.

- 2.4.36** (★★★) **A MANO** Modele con una recurrencia el número de cadenas de longitud n sobre $\{0, 1\}$ que **no** contienen dos ceros consecutivos.

SOLUCIÓN Una cadena válida termina en 1, y entonces el prefijo es cualquier cadena válida de longitud $n - 1$; o termina en 10, y el prefijo es válido de longitud $n - 2$. Por lo tanto

$$a_n = a_{n-1} + a_{n-2}, \quad a_1 = 2, \quad a_2 = 3$$

Es Fibonacci desplazado: $a_n = F_{n+2}$. Para $n = 5$ hay 13 cadenas.

- 2.4.37** (★★) **MIXTO** Compruebe el conteo del ejercicio anterior para $n = 5$ enumerando.

SOLUCIÓN**In[] :=**`Length[Select[Tuples[{0, 1}, 5], !MemberQ[Partition[#, 2, 1], {0, 0}] &]]`**Out[] =**

13

In[] :=`RT[{1, 1}, {2, 3}, 5]`**Out[] =**

{2, 3, 5, 8, 13}

Las 13 cadenas coinciden con el quinto término de la recurrencia. Enumerar sirve para **validar** el modelo, no para resolverlo: con $n = 40$ habría que revisar más de un billón de cadenas.

- 2.4.38** (★★★) **A MANO** Modele el número de formas de subir una escalera de n escalones dando pasos de uno o dos escalones.

SOLUCIÓN El último paso es de uno o de dos escalones, de modo que

$$a_n = a_{n-1} + a_{n-2}, \quad a_1 = 1, \quad a_2 = 2$$

Otra vez Fibonacci: $a_n = F_{n+1}$. Con pasos de uno, dos o tres escalones la relación pasaría a ser *tribonacci*, de orden tres.

- 2.4.39** (★★) **MIXTO** Compruebe el modelo de la escalera para $n = 8$.

SOLUCIÓN**In[] :=**`RT[{1, 1}, {1, 2}, 8]`**Out[] =**

{1, 2, 3, 5, 8, 13, 21, 34}

Hay 34 maneras de subir ocho escalones. La lista es exactamente Fibonacci a partir del segundo término.

- 2.4.40** (★★★) **A MANO** Dos poblaciones evolucionan según $a_n = 2a_{n-1}$ y $b_n = b_{n-1} + 100$, con $a_0 = b_0 = 100$. ¿A partir de qué año supera la primera a la segunda?

SOLUCIÓN Se tiene $a_n = 100 \cdot 2^n$ y $b_n = 100(n + 1)$. La desigualdad $2^n > n + 1$ falla en $n = 0$ y $n = 1$ (donde hay igualdad) y se cumple desde $n = 2$: $4 > 3$. A partir del segundo año el crecimiento exponencial ya domina, y la diferencia se dispara: en el año 10 es 102 400 frente a 1100.

- 2.4.41** (★★) **MIXTO** Compruebe la comparación anterior con una tabla.

SOLUCIÓN**In[] :=**


```
{RT[{2}, {100}, 8], RT[{1, 100}, {100}, 8]}
```

```
Out[ ] =
```

```
{100, 200, 400, 800, 1600, 3200, 6400, 12800},  
{100, 200, 300, 400, 500, 600, 700, 800}}
```

Los dos primeros valores coinciden y desde el tercero la separación crece sin límite. Es la diferencia entre $\Theta(2^n)$ y $\Theta(n)$ vista sobre una sucesión concreta.

2.2.1 El método iterativo

2.4.42 (★) **A MANO** Explique en qué consiste el método iterativo para resolver una relación de recurrencia.

SOLUCIÓN Consiste en sustituir repetidamente la relación en sí misma, bajando el subíndice paso a paso, hasta que aparece un patrón reconocible en función del número de sustituciones k . Después se elige el valor de k que hace aparecer la condición inicial y se simplifica la expresión resultante.

2.4.43 (★★) **A MANO** Resuelva $a_n = 2a_{n-1}$ con $a_0 = 3$ por el método iterativo.

SOLUCIÓN Sustituyendo:

$$a_n = 2a_{n-1} = 2^2a_{n-2} = 2^3a_{n-3} = \dots = 2^k a_{n-k}$$

La condición inicial aparece cuando $n - k = 0$, es decir $k = n$:

$$a_n = 2^n a_0 = 3 \cdot 2^n$$

2.4.44 (★★) **A MANO** Resuelva $a_n = a_{n-1} + 4$ con $a_0 = 1$ por el método iterativo.

SOLUCIÓN

$$a_n = a_{n-1} + 4 = a_{n-2} + 2 \cdot 4 = a_{n-3} + 3 \cdot 4 = \dots = a_{n-k} + 4k$$

Con $k = n$ se alcanza a_0 :

$$a_n = a_0 + 4n = 4n + 1$$

2.4.45 (★★) **VilCretas** Use `MetodoI` para ver las iteraciones de $a_n = 2a_{n-1} + 7$.

SOLUCIÓN

```
In[ ] :=
```

```
MetodoI[{2, 7}, 3]
```

El segundo argumento indica cuántas iteraciones se despliegan. La salida muestra cómo se acumulan las potencias de 2 multiplicando al término independiente, que es justo el patrón que hay que reconocer para cerrar la fórmula.

2.4.46 (★★★) **A MANO** Resuelva $a_n = 2a_{n-1} + 7$ con $a_0 = 3$ por el método iterativo.

SOLUCIÓN Iterando:

$$a_n = 2a_{n-1} + 7 = 2^2a_{n-2} + 7(1 + 2) = 2^3a_{n-3} + 7(1 + 2 + 2^2) = \dots$$

$$a_n = 2^k a_{n-k} + 7 \sum_{j=0}^{k-1} 2^j = 2^k a_{n-k} + 7(2^k - 1)$$

Con $k = n$: $a_n = 3 \cdot 2^n + 7 \cdot 2^n - 7 = 10 \cdot 2^n - 7$. Se ha usado la suma de una progresión geométrica, $\sum_{j=0}^{k-1} 2^j = 2^k - 1$.

2.4.47 (★★) **MIXTO** Verifique con RR la solución del ejercicio anterior.

SOLUCIÓN

In[] :=

RR[{2, 7}, {3}, n, inicio -> 0]

Out[] =

-7 + 5*2^(1 + n)

La expresión $5 \cdot 2^{n+1} - 7$ es idéntica a $10 \cdot 2^n - 7$, obtenida a mano.

2.4.48 (★★) **A MANO** Resuelva $a_n = 3a_{n-1} - 2$ con $a_0 = 1$ por el método iterativo.

SOLUCIÓN

$$a_n = 3^k a_{n-k} - 2(1 + 3 + \cdots + 3^{k-1}) = 3^k a_{n-k} - 2 \cdot \frac{3^k - 1}{2}$$

Con $k = n$: $a_n = 3^n - 3^n + 1 = 1$. La sucesión es constante e igual a 1, lo que se comprueba de inmediato: $3 \cdot 1 - 2 = 1$.

2.4.49 (★★★) **A MANO** Resuelva $a_n = a_{n-1} + n$ con $a_0 = 1$ por el método iterativo.

SOLUCIÓN

$$a_n = a_{n-1} + n = a_{n-2} + (n-1) + n = \cdots = a_{n-k} + \sum_{j=n-k+1}^n j$$

Con $k = n$ la suma recorre de 1 a n :

$$a_n = a_0 + \frac{n(n+1)}{2} = \frac{n^2 + n + 2}{2}$$

Es exactamente el número de regiones del ejercicio de las rectas: 1, 2, 4, 7, 11.

2.4.50 (★★) **A MANO** Resuelva $a_n = \frac{a_{n-1}}{2}$ con $a_0 = 64$ e interprete el resultado.

SOLUCIÓN Iterando se obtiene

$$a_n = \left(\frac{1}{2}\right)^k a_{n-k}, \quad \text{y con } k = n: \quad a_n = 64 \left(\frac{1}{2}\right)^n = 2^{6-n}$$

La sucesión 64, 32, 16, 8, 4, 2, 1 tiende a cero: cada término es la mitad del anterior.

2.4.51 (★★★) **A MANO** Resuelva $a_n = 2a_{n-1} + 2^n$ con $a_0 = 1$.

SOLUCIÓN

$$a_n = 2a_{n-1} + 2^n = 2^2 a_{n-2} + 2 \cdot 2^{n-1} + 2^n = 2^2 a_{n-2} + 2 \cdot 2^n$$

En general $a_n = 2^k a_{n-k} + k 2^n$. Con $k = n$:

$$a_n = 2^n a_0 + n 2^n = 2^n(n+1)$$

Aquí la parte no homogénea tiene la misma base que la homogénea, y por eso aparece el factor n .

2.4.52 (★★) **MIXTO** Compruebe la solución anterior evaluándola en $n = 0, 1, 2, 3$ y comparándola con los términos de la recurrencia.

SOLUCIÓN La fórmula da 1, 4, 12, 32. Directamente de la recurrencia: $a_0 = 1$, $a_1 = 2 + 2 = 4$, $a_2 = 8 + 4 = 12$, $a_3 = 24 + 8 = 32$. Coinciden.

In[] :=

RT[{4, -4}, {1, 4}, 7]

Out[] =

{1, 4, 12, 32, 80, 192, 448}

La lista prolonga la misma sucesión: se retoma en el ejercicio sobre raíces repetidas.

2.4.53 (★★) **A MANO** Resuelva $a_n = 5a_{n-1}$ con $a_2 = 50$. Preste atención a la condición inicial.

SOLUCIÓN $a_n = 5^k a_{n-k}$. Aquí la condición no está en a_0 sino en a_2 , de modo que hay que tomar $k = n - 2$:

$$a_n = 5^{n-2} a_2 = 50 \cdot 5^{n-2} = 2 \cdot 5^n / 5^0$$

es decir $a_n = 2 \cdot 5^{n-1}$. Se elige siempre k de forma que aparezca la condición *disponible*, no necesariamente a_0 .

2.4.54 (★★★) **A MANO** Resuelva $a_n = a_{n-1} + 3n - 1$ con $a_0 = 2$.

SOLUCIÓN

$$a_n = a_0 + \sum_{j=1}^n (3j - 1) = 2 + 3 \cdot \frac{n(n+1)}{2} - n = \frac{3n^2 + n + 4}{2}$$

Los primeros términos son 2, 4, 9, 17, 28, que la fórmula reproduce.

2.4.55 (★★) **VilCretas** Use **MetodoI** con la opción **inicio** para una recurrencia cuya condición inicial no está en cero.

SOLUCIÓN

In[] :=

MetodoI[{3, 2}, 3, inicio -> 1]

La opción **inicio** declara el índice con el que empieza la condición inicial; su valor por defecto es 1. El segundo argumento sigue fijando cuántas iteraciones se muestran.

2.4.56 (★★★) **A MANO** Resuelva $a_n = n a_{n-1}$ con $a_0 = 1$ e identifique la sucesión obtenida.

SOLUCIÓN

$$a_n = n a_{n-1} = n(n-1) a_{n-2} = n(n-1)(n-2) a_{n-3} = \dots$$

Con $k = n$ el producto recorre todos los factores de n a 1:

$$a_n = n! \ a_0 = n!$$

El coeficiente ya no es constante, sino que depende de n ; aun así el método iterativo funciona porque el patrón sigue siendo reconocible.

2.4.57 (★★) **MIXTO** Resuelva $a_n = 4a_{n-1}$ con $a_0 = 1$ y compruebe que coincide con 2^{2n} .

SOLUCIÓN Por iteración $a_n = 4^n$. Como $4 = 2^2$, se tiene $4^n = 2^{2n}$.

In[] :=

```
RR[{4}, {1}, n, inicio -> 0]
```

Out[] =

4^n

Ambas escrituras son la misma sucesión 1, 4, 16, 64, ...

2.4.58 (★★★) **A MANO** Explique por qué el método iterativo se vuelve poco práctico en una relación de orden dos como $a_n = a_{n-1} + a_{n-2}$.

SOLUCIÓN Al sustituir, cada término se desdobra en dos y el número de sumandos crece como la propia sucesión de Fibonacci. En vez de un patrón que se simplifica, aparece una expresión con cada vez más términos. Para las relaciones de orden dos o superior conviene el método de la ecuación característica, que es el tema de la sección siguiente.

2.4.59 (★★★) **A MANO** Resuelva $a_n = 3a_{n-1} + 2$ con $a_0 = 1$ por el método iterativo.

SOLUCIÓN Sustituyendo repetidamente,

$$a_n = 3a_{n-1} + 2 = 3^2 a_{n-2} + 2(1 + 3) = 3^3 a_{n-3} + 2(1 + 3 + 3^2) = \dots$$

$$a_n = 3^k a_{n-k} + 2 \left(1 + 3 + \dots + 3^{k-1} \right)$$

Con $k = n$ y la suma geométrica $\frac{3^n - 1}{2}$,

$$a_n = 3^n + 2 \cdot \frac{3^n - 1}{2} = 2 \cdot 3^n - 1$$

2.4.60 (★★) **MIXTO** Compruebe el resultado anterior.

SOLUCIÓN

In[] :=

```
{RT[{3, 2}, {1}, 6], RR[{3, 2}, {1}, n, inicio -> 0]}
```

Out[] =

{1, 5, 17, 53, 161, 485}, -1 + 2*3^n

Coincide con $2 \cdot 3^n - 1$. La aparición de una **suma geométrica** al iterar es lo característico de las relaciones $a_n = ra_{n-1} + c$ con $r \neq 1$.

2.4.61 (★★★) **A MANO** Resuelva $a_n = a_{n-1} + 2n - 1$ con $a_0 = 0$ por iteración.

SOLUCIÓN

$$a_n = a_{n-1} + (2n - 1) = a_{n-2} + (2n - 1) + (2n - 3) = \cdots = a_0 + \sum_{i=1}^n (2i - 1)$$

La suma de los n primeros impares vale n^2 , de modo que $a_n = n^2$. Es el ejemplo más limpio de recurrencia acumulativa: al iterar aparece una **sumatoria**, no una potencia.

2.4.62 (★★) **MIXTO** Verifique la solución anterior.

SOLUCIÓN

In[] :=

```
{RT[{1, 2 n - 1}, {0}, 7, inicio -> 0], RR[{1, 2 n - 1}, {0}, n, inicio -> 0]}
```

Out[] =

```
{{0, 1, 4, 9, 16, 25, 36}, n^2}
```

Los cuadrados perfectos. La parte no homogénea $2n - 1$ es de grado uno y la solución sube a grado dos, como corresponde.

2.4.63 (★★★) **A MANO** Resuelva $a_n = \frac{a_{n-1}}{3} + 1$ con $a_0 = 3$ por iteración y calcule su límite.

SOLUCIÓN Iterando,

$$a_n = \frac{a_0}{3^n} + \left(1 + \frac{1}{3} + \cdots + \frac{1}{3^{n-1}}\right) = \frac{3}{3^n} + \frac{1 - 3^{-n}}{1 - \frac{1}{3}} = \frac{3}{2} + \frac{3}{2 \cdot 3^n}$$

El límite es $\frac{3}{2}$, que es justamente el punto fijo de la relación: $a = \frac{a}{3} + 1$ da $a = \frac{3}{2}$. Como el coeficiente tiene valor absoluto menor que uno, la sucesión **converge** a ese punto.

2.4.64 (★★) **MIXTO** Compruebe la convergencia del ejercicio anterior.

SOLUCIÓN

In[] :=

```
RT[{1/3, 1}, {3}, 8]
```

Out[] =

```
{3, 2, 5/3, 14/9, 41/27, 122/81, 365/243, 1094/729}
```

Los valores se acercan a 1,5 desde arriba, y la distancia se divide entre tres en cada paso. Con un coeficiente mayor que uno en valor absoluto la sucesión se alejaría en vez de converger.

2.4.65 (★★★) **MIXTO** Resuelva $a_n = 2a_{n-1} + 3^n$ con $a_0 = 1$ por iteración y compare el resultado con el del ejercicio 2.4.51.

SOLUCIÓN

$$a_n = 2^k a_{n-k} + \sum_{j=1}^k 2^{k-j} 3^{n-k+j}$$

Con $k = n$ la suma es geométrica de razón $\frac{3}{2}$:

$$a_n = 2^n + 2^n \sum_{j=1}^n \left(\frac{3}{2}\right)^j = 2^n + 3(3^n - 2^n) = 3^{n+1} - 2^{n+1}$$

In[] :=

```
{RT[{2, 3^n}, {1}, 6, inicio -> 0],  
RR[{2, 3^n}, {1}, n, inicio -> 0]}
```

Out[] =

```
{{1, 5, 19, 65, 211, 665}, 2^n*(-2 + 3^(1 + n)/2^n)}
```

La fórmula que devuelve el comando es $2^n \left(\frac{3^{n+1}}{2^n} - 2 \right)$, que es $3^{n+1} - 2^{n+1}$ reordenado. La **diferencia** con el ejercicio 2.4.51 está en el factor n : allí la parte no homogénea era 2^n , que **es** solución de la homogénea, y por eso apareció $n2^n$: resonancia. Aquí 3^n no lo es, y no aparece ningún factor n .

2.4.66 (★★) **MIXTO** Verifique la solución del ejercicio anterior.

SOLUCIÓN

In[] :=

```
{RT[{2, 2^n}, {1}, 7, inicio -> 0], RR[{2, 2^n}, {1}, n, inicio -> 0]}
```

Out[] =

```
{{1, 4, 12, 32, 80, 192, 448}, 2^n*(1 + n)}
```

Es $(n+1)2^n$. Compare con $a_n = 2a_{n-1} + 3^n$, donde no hay resonancia y la solución no lleva factor n .

2.4.67 (★★★) **MIXTO** Resuelva $a_n = 2a_{n-1} + 3^n$ con $a_0 = 1$ y compare con el caso anterior.

SOLUCIÓN

In[] :=

```
{RT[{2, 3^n}, {1}, 7, inicio -> 0], RR[{2, 3^n}, {1}, n, inicio -> 0]}
```

Out[] =

```
{{1, 5, 19, 65, 211, 665, 2059}, 2^n*(-2 + 3^(1 + n)/2^n)}
```

La solución es $3^{n+1} - 2^{n+1}$, sin factor n : la parte no homogénea 3^n no es solución de la homogénea $a_n = 2a_{n-1}$, de modo que no hay resonancia.

2.4.68 (★★★) **A MANO** Enuncie la regla general que distingue los dos ejercicios anteriores.

SOLUCIÓN Al resolver $a_n = ra_{n-1} + cs^n$: si $s \neq r$ la solución particular es As^n ; si $s = r$ hay **resonancia** y hay que ensayar Ans^n . La misma regla, con potencias de n en vez de s^n , explica por qué $a_n = a_{n-1} + c$ da una recta y no una constante: ahí $r = 1$ coincide con el 1^n de la parte constante.

2.4.69 (★★) **VilCretas** Use MetodoI para observar las iteraciones de $a_n = 4a_{n-1} + 5$.

SOLUCIÓN

In[] :=

```
MetodoI[{4, 5}, 4]
```

El segundo argumento fija cuántas iteraciones se despliegan. Se ve cómo el 5 queda multiplicado por potencias sucesivas de 4, que al sumarse dan la serie geométrica $5 \frac{4^n - 1}{3}$.

2.4.70 (★★) **MIXTO** Resuelva la relación del ejercicio anterior con $a_0 = 2$ y compruebe.

SOLUCIÓN Por iteración, $a_n = 4^n \cdot 2 + 5 \frac{4^n - 1}{3} = \frac{11 \cdot 4^n - 5}{3}$.

In[] :=

```
{RT[{4, 5}, {2}, 6], RR[{4, 5}, {2}, n, inicio -> 0]}
```

Out[] =

```
{{2, 13, 57, 233, 937, 3753}, (-5 + 11*2^(2*n))/3}
```

La fórmula coincide con la obtenida a mano.

2.4.71 (★★★) **A MANO** Resuelva $a_n = n a_{n-1}$ con $a_1 = 2$ por iteración.

SOLUCIÓN

$$a_n = n a_{n-1} = n(n-1)a_{n-2} = \cdots = n(n-1) \cdots 2 \cdot a_1 = 2 \cdot n!$$

Al iterar aparece un **producto** y no una suma, porque el coeficiente depende de n . La relación es lineal pero *sin coeficientes constantes*, de modo que el método de la ecuación característica no se le puede aplicar.

2.4.72 (★★) **MIXTO** Compruebe la solución anterior.

SOLUCIÓN

In[] :=

```
fa[n_] := If[n == 1, 2, n*fa[n - 1]];
{Table[fa[k], {k, 1, 7}], 2*7!}
```

Out[] =

```
{{2, 4, 12, 48, 240, 1440, 10080}, 10080}
```

El séptimo término es $2 \cdot 5040 = 10080$. Coincide con $2 \cdot n!$.

2.4.73 (★★★) **A MANO** Resuelva $a_n = \frac{n}{n+1} a_{n-1}$ con $a_1 = 1$ por iteración.

SOLUCIÓN

$$a_n = \frac{n}{n+1} \cdot \frac{n-1}{n} \cdots \frac{2}{3} \cdot a_1$$

El producto es **telescópico**: se cancelan todos los factores intermedios y queda $a_n = \frac{2}{n+1}$.

Comprobación: $a_1 = 1$, $a_2 = \frac{2}{3}$, $a_3 = \frac{1}{2}$.

2.4.74 (★★) **MIXTO** Verifique la solución telescópica anterior.

SOLUCIÓN

In[] :=

```
tel[n_] := If[n == 1, 1, (n/(n + 1))*tel[n - 1]];
{Table[tel[k], {k, 1, 6}], Table[2/(k + 1), {k, 1, 6}]}
```

Out[] =

```
{{1, 2/3, 1/2, 2/5, 1/3, 2/7}, {1, 2/3, 1/2, 2/5, 1/3, 2/7}}
```

Las dos listas coinciden término a término.

2.4.75 (★★★) **A MANO** Al iterar $a_n = 2a_{n/2} + n$ con $a_1 = 0$ se obtiene una recurrencia de *divide y vencerás*. Resuélvala para $n = 2^k$.

SOLUCIÓN Con $n = 2^k$, iterando k veces:

$$a_{2^k} = 2a_{2^{k-1}} + 2^k = 2^2a_{2^{k-2}} + 2 \cdot 2^k = \cdots = 2^k a_1 + k 2^k$$

Como $a_1 = 0$, resulta $a_n = n \log_2 n$. Es exactamente el costo de *mergesort*, y explica por qué los métodos que parten el problema en dos alcanzan el orden $n \ln n$.

2.4.76 (★★) **MIXTO** Compruebe la fórmula anterior para varias potencias de dos.

SOLUCIÓN

In[] :=

```
dv[n_] := If[n == 1, 0, 2 dv[n/2] + n];
{Table[dv[2^k], {k, 0, 6}], Table[2^k*k, {k, 0, 6}]}
```

Out[] =

```
{0, 2, 8, 24, 64, 160, 384}, {0, 2, 8, 24, 64, 160, 384}}
```

Las listas coinciden: $a_{2^k} = k 2^k = n \log_2 n$.

2.4.77 (★★★) **A MANO** Resuelva $a_n = a_{n-1} + \frac{1}{n(n+1)}$ con $a_1 = \frac{1}{2}$ y halle su límite.

SOLUCIÓN Al iterar aparece la suma telescópica $\sum_{i=2}^n \left(\frac{1}{i} - \frac{1}{i+1} \right) = \frac{1}{2} - \frac{1}{n+1}$, de modo que

$$a_n = \frac{1}{2} + \frac{1}{2} - \frac{1}{n+1} = 1 - \frac{1}{n+1}$$

El límite es 1. A diferencia de las sumas de términos constantes, aquí la sucesión converge porque los incrementos tienden a cero lo bastante rápido.

2.4.78 (★★) **MIXTO** Compruebe numéricamente el límite anterior.

SOLUCIÓN

In[] :=

```
ta[n_] := If[n == 1, 1/2, ta[n - 1] + 1/(n (n + 1))];
{ta[10], N[ta[10]], 1 - 1/11}
```

Out[] =

```
{10/11, 0.9090909090909091, 10/11}
```

El décimo término ya vale $\frac{10}{11} \approx 0,909$, coherente con $1 - \frac{1}{n+1}$.

2.4.79 (★★★) **A MANO** ¿En qué se diferencia el método iterativo del de la ecuación característica, y cuándo conviene cada uno?

SOLUCIÓN El **iterativo** sustituye la relación en sí misma hasta reconocer un patrón; sirve para **cualquier** relación de orden uno, con coeficientes variables o partes no homogéneas arbitrarias, pero exige identificar y sumar una serie. El de la **ecuación característica** es mecánico y resuelve de un golpe cualquier orden, pero solo se aplica a relaciones **lineales homogéneas con coeficientes constantes**.

2.4.80 (★★★) **A MANO** Al iterar una relación de orden uno se obtiene

$$a_n = r^k a_{n-k} + c \left(1 + r + \dots + r^{k-1} \right)$$

Deduzca la fórmula cerrada según el valor de r .

SOLUCIÓN Si $r \neq 1$, la suma geométrica vale $\frac{r^n - 1}{r - 1}$ y

$$a_n = r^n a_0 + c \frac{r^n - 1}{r - 1}$$

Si $r = 1$, la suma es simplemente n y queda $a_n = a_0 + cn$. La bifurcación entre crecimiento **exponencial** y **lineal** depende únicamente de si el coeficiente vale uno.

2.4.81 (★★) **MIXTO** Ilustre la bifurcación anterior con $r = 1$ y $r = 2$, ambos con $c = 3$ y $a_0 = 1$.

SOLUCIÓN

In[] :=

```
{RT[{1, 3}, {1}, 7], RT[{2, 3}, {1}, 7]}
```

Out[] =

```
{{1, 4, 7, 10, 13, 16, 19}, {1, 5, 13, 29, 61, 125, 253}}
```

Con $r = 1$ la sucesión avanza de tres en tres; con $r = 2$ pasa de 1 a 253 en seis pasos. Un cambio mínimo en el coeficiente cambia el **orden de crecimiento**.

2.4.82 (★★★) **A MANO** Una relación de orden uno tiene $r = -2$ y $c = 3$, con $a_0 = 1$. Describa el comportamiento de la sucesión.

SOLUCIÓN La fórmula da $a_n = (-2)^n \cdot 1 + 3 \frac{(-2)^n - 1}{-3} = 2(-2)^n - 1$. Los valores **alternan de signo** y crecen en valor absoluto: $1, -5, 7, -17, 31, \dots$. Con $|r| > 1$ hay divergencia, y el signo negativo añade la oscilación.

2.4.83 (★★) **MIXTO** Compruebe el comportamiento alternante anterior.

SOLUCIÓN

In[] :=

```
{RT[{-2, 3}, {1}, 7], RR[{-2, 3}, {1}, n, inicio -> 0]}
```

Out[] =

```
{{1, 1, 1, 1, 1, 1, 1}, (-1)^(2*n)}
```

El resultado es constante (el software lo escribe como $(-1)^{2n}$, que vale 1): con $a_0 = 1$ la sucesión cae **justo** sobre el punto fijo $a = -2a + 3$, y ya no se mueve. Basta cambiar la condición inicial a $a_0 = 2$ para que aparezca la oscilación creciente.

2.2.2 Relaciones homogéneas lineales

2.4.84 (★) **A MANO** Escriba la ecuación característica de $a_n = 5a_{n-1} - 6a_{n-2}$.

SOLUCIÓN Se sustituye a_n por t^n y se divide entre t^{n-2} :

$$t^2 = 5t - 6 \implies t^2 - 5t + 6 = 0$$

- 2.4.85** (★★) **MIXTO** Resuelva la ecuación característica anterior y escriba la forma general de la solución.

SOLUCIÓN

In[] :=

`Solve[t^2 - 5 t + 6 == 0, t]`

Out[] =

`{{t -> 2}, {t -> 3}}`

Las raíces 2 y 3 son reales y distintas, de modo que

$$a_n = b_1 2^n + b_2 3^n$$

con b_1 y b_2 por determinar con las condiciones iniciales.

- 2.4.86** (★★) **A MANO** Complete el ejercicio anterior con $a_0 = 1$ y $a_1 = 2$.

SOLUCIÓN El sistema es

$$b_1 + b_2 = 1, \quad 2b_1 + 3b_2 = 2$$

De la primera, $b_1 = 1 - b_2$; sustituyendo, $2 - 2b_2 + 3b_2 = 2$, luego $b_2 = 0$ y $b_1 = 1$. La solución es $a_n = 2^n$.

- 2.4.87** (★★) **VilCretas** Verifique la solución anterior con RR y RT.

SOLUCIÓN

In[] :=

`RR[{5, -6}, {1, 2}, n, inicio -> 0]`

Out[] =

`2^n`

In[] :=

`RT[{5, -6}, {1, 2}, 8]`

Out[] =

`{1, 2, 4, 8, 16, 32, 64, 128}`

La segunda raíz desaparece porque su coeficiente resultó nulo: no siempre las dos raíces intervienen en la solución particular.

- 2.4.88** (★★) **A MANO** Escriba la ecuación característica de $a_n = 4a_{n-1} - 4a_{n-2}$ y resuélvala.

SOLUCIÓN

$$t^2 - 4t + 4 = 0 \implies (t - 2)^2 = 0 \implies t = 2$$

Se trata de una raíz **doble**.

- 2.4.89** (★★★) **A MANO** Explique por qué, con una raíz doble r , la solución general es $a_n = b_1 r^n + b_2 n r^n$ y no $b_1 r^n + b_2 r^n$.

SOLUCIÓN Si se escribiera $b_1 r^n + b_2 r^n$, los dos sumandos se fundirían en uno solo, $(b_1 + b_2)r^n$, que tiene una única constante libre. Con una sola constante no se pueden satisfacer *dos* condiciones iniciales independientes. El factor n proporciona una segunda solución linealmente independiente y devuelve el grado de libertad que falta.

2.4.90 (★★) **MIXTO** Resuelva $a_n = 4a_{n-1} - 4a_{n-2}$ con $a_0 = 1$ y $a_1 = 4$, y verifíquelo.

SOLUCIÓN Con la raíz doble $t = 2$: $a_n = b_1 2^n + b_2 n 2^n$. Las condiciones dan $b_1 = 1$ y $2b_1 + 2b_2 = 4$, luego $b_2 = 1$. Así $a_n = 2^n(1 + n)$.

In[] :=

```
RR[{4, -4}, {1, 4}, n, inicio -> 0]
```

Out[] =

```
2^n*(1 + n)
```

In[] :=

```
RT[{4, -4}, {1, 4}, 7]
```

Out[] =

```
{1, 4, 12, 32, 80, 192, 448}
```

2.4.91 (★★) **A MANO** Escriba la ecuación característica de $a_n = 2a_{n-1} - a_{n-2}$ y resuélvala.

SOLUCIÓN $t^2 - 2t + 1 = 0$, es decir $(t - 1)^2 = 0$, con raíz doble $t = 1$. La solución general es $a_n = b_1 + b_2 n$: una progresión aritmética.

2.4.92 (★★) **MIXTO** Complete el ejercicio anterior con $a_0 = 3$ y $a_1 = 5$, y compruébelo.

SOLUCIÓN De $b_1 = 3$ y $b_1 + b_2 = 5$ resulta $b_2 = 2$, luego $a_n = 2n + 3$.

In[] :=

```
RR[{2, -1}, {3, 5}, n, inicio -> 0]
```

Out[] =

```
3 + 2*n
```

In[] :=

```
RT[{2, -1}, {3, 5}, 7]
```

Out[] =

```
{3, 5, 7, 9, 11, 13, 15}
```

Los términos avanzan de dos en dos, como corresponde a la raíz doble $t = 1$.

2.4.93 (★★★) **MIXTO** Resuelva $a_n = 2a_{n-1} - 2a_{n-2}$ y describa qué ocurre con las raíces.

SOLUCIÓN

In[] :=

```
Solve[t^2 - 2 t + 2 == 0, t]
```

Out[] =

$\{t \rightarrow 1 - i\}, \{t \rightarrow 1 + i\}$

Las raíces son **complejas conjugadas**, $1 \pm i$. La solución general sigue siendo $a_n = b_1(1 + i)^n + b_2(1 - i)^n$; al imponer condiciones iniciales reales, las constantes resultan conjugadas y la sucesión es real, aunque escrita con números complejos.

2.4.94 (★★) **A MANO** Resuelva la relación de Fibonacci $a_n = a_{n-1} + a_{n-2}$ con $a_1 = a_2 = 1$ planteando su ecuación característica.

SOLUCIÓN $t^2 - t - 1 = 0$, cuyas raíces son

$$t = \frac{1 \pm \sqrt{5}}{2}$$

La mayor es la razón áurea. La solución general es $a_n = b_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + b_2 \left(\frac{1-\sqrt{5}}{2}\right)^n$, y al ajustar las condiciones se obtiene la fórmula de Binet.

2.4.95 (★★) **VilCretas** Compruebe con RR que la relación de Fibonacci produce efectivamente la sucesión de Fibonacci.

SOLUCIÓN

In[] :=

```
RR[{1, 1}, {1, 1}, n, inicio -> 1]
```

Out[] =

```
Fibonacci[n]
```

In[] :=

```
RT[{1, 1}, {1, 1}, 10]
```

Out[] =

```
{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}
```

El sistema reconoce la sucesión y devuelve su nombre propio en vez de la expresión con radicales.

2.4.96 (★★★) **A MANO** Escriba y resuelva la ecuación característica de la relación de orden tres $a_n = 6a_{n-1} - 11a_{n-2} + 6a_{n-3}$.

SOLUCIÓN

$$t^3 - 6t^2 + 11t - 6 = 0$$

Probando divisores del término independiente se halla $t = 1$; dividiendo, $t^2 - 5t + 6 = 0$, con raíces 2 y 3. Las tres raíces son 1, 2 y 3, todas distintas, de modo que

$$a_n = b_1 + b_2 2^n + b_3 3^n$$

2.4.97 (★★) **MIXTO** Verifique las raíces del ejercicio anterior y obtenga la solución con $a_0 = 1$, $a_1 = 2$ y $a_2 = 3$.

SOLUCIÓN

In[] :=

```
Solve[t^3 - 6 t^2 + 11 t - 6 == 0, t]
```

```
Out[ ] =
```

```
{{t -> 1}, {t -> 2}, {t -> 3}}
```

```
In[ ] :=
```

```
RR[{6, -11, 6}, {1, 2, 3}, n, inicio -> 0]
```

```
Out[ ] =
```

```
(-1 + 2^(2 + n) - 3^n)/2
```

La fórmula equivale a $a_n = \frac{1}{2}(4 \cdot 2^n - 3^n - 1)$.

- 2.4.98** (★★) **VilCretas** Genere los ocho primeros términos de la relación anterior y observe el cambio de signo.

SOLUCIÓN

```
In[ ] :=
```

```
RT[{6, -11, 6}, {1, 2, 3}, 8]
```

```
Out[ ] =
```

```
{1, 2, 3, 2, -9, -58, -237, -838}
```

El término 3^n , con signo negativo, acaba dominando sobre $4 \cdot 2^n$: por eso la sucesión crece al principio y luego se desploma. La raíz de mayor módulo manda en el comportamiento a largo plazo.

- 2.4.99** (★★) **VilCretas** Dada la sucesión 2, 6, 18, 54, 162, 486, encuentre la relación de recurrencia que la genera con FindRRHL.

SOLUCIÓN

```
In[ ] :=
```

```
FindRRHL[{2, 6, 18, 54, 162, 486}, a, n]
```

```
Out[ ] =
```

```
{a[n] == 3*a[-1 + n], a[1] == 2, 2*3^(-1 + n)}
```

El comando devuelve la relación, la condición inicial y la fórmula explícita $a_n = 2 \cdot 3^{n-1}$.

- 2.4.100** (★★) **MIXTO** Reconozca a mano la relación que genera 1, 3, 7, 15, 31, 63 y confírmela.

SOLUCIÓN Cada término es el doble del anterior más uno, o bien uno menos que una potencia de dos. Formalmente $a_n = 2^n - 1$.

```
In[ ] :=
```

```
FindRRHL[{1, 3, 7, 15, 31, 63}, a, n]
```

```
Out[ ] =
```

```
{a[n] == -2*a[-2 + n] + 3*a[-1 + n], a[1] == 1, a[2] == 3, -1 + 2^n}
```

El comando la expresa como homogénea de orden dos; la fórmula explícita $2^n - 1$ confirma el razonamiento manual. Es la misma sucesión de las torres de Hanoi.

- 2.4.101** (★★★) **A MANO** Determine para qué valores de c la relación $a_n = ca_{n-1} - a_{n-2}$ tiene una raíz doble.

SOLUCIÓN La ecuación característica es $t^2 - ct + 1 = 0$, con discriminante $c^2 - 4$. Hay raíz doble cuando el discriminante se anula, es decir $c = 2$ o $c = -2$. Para $|c| > 2$ las raíces son reales distintas y para $|c| < 2$ son complejas conjugadas.

- 2.4.102** (★★) **A MANO** Sin resolverla, prediga si la sucesión de $a_n = \frac{1}{2}a_{n-1}$ con $a_0 = 8$ crece, decrece o se mantiene.

SOLUCIÓN La única raíz característica es $\frac{1}{2}$, de módulo menor que uno, de modo que la sucesión decrece hacia cero: $8, 4, 2, 1, \frac{1}{2}, \dots$. El módulo de la raíz determina el comportamiento a largo plazo.

- 2.4.103** (★★★) **A MANO** Explique por qué una relación homogénea lineal de orden k con coeficientes constantes necesita exactamente k condiciones iniciales.

SOLUCIÓN Su ecuación característica tiene grado k y aporta k soluciones linealmente independientes, de modo que la solución general contiene k constantes libres $b_1, \dots, b_k$. Determinarlas exige un sistema de k ecuaciones, y cada condición inicial aporta una. Con menos condiciones el sistema queda indeterminado; con más, sobredeterminado.

- 2.4.104** (★★) **VilCretas** Emplee MetodoRRHL para ver el desarrollo completo de $a_n = 5a_{n-1} - 6a_{n-2}$ con $a_0 = 1$ y $a_1 = 2$.

SOLUCIÓN

`In[ ] :=`

```
MetodoRRHL[{5, -6}, {1, 2}, n, b, inicio -> 0]
```

El cuarto argumento es el nombre que se dará a las constantes del sistema y `inicio` fija el índice de la primera condición (por defecto es 1). La salida recorre el procedimiento completo: ecuación característica, raíces $\{2, 3\}$, forma general $b_1 2^n + b_2 3^n$, sistema $b_1 + b_2 = 1$, $2b_1 + 3b_2 = 2$, solución $b_1 = 1$, $b_2 = 0$ y fórmula final $a_n = 2^n$.

- 2.4.105** (★★) **VilCretas** Dibuje con GraficaRRL los términos de $a_n = 5a_{n-1} - 6a_{n-2}$ con $a_0 = 1$ y $a_1 = 2$.

SOLUCIÓN

`In[ ] :=`

```
GraficaRRL[{5, -6}, {1, 2}, inicio -> 0]
```

La gráfica muestra el crecimiento exponencial de 2^n . La opción `npuntos` permite fijar cuántos términos se representan.

- 2.4.106** (★★★) **A MANO** Resuelva $a_n = -a_{n-2}$ con $a_0 = 1$ y $a_1 = 0$, e interprete el resultado.

SOLUCIÓN La ecuación característica es $t^2 + 1 = 0$, con raíces $\pm i$. Los primeros términos son $1, 0, -1, 0, 1, 0, \dots$: la sucesión es periódica de periodo cuatro. Las raíces de módulo uno producen oscilaciones que ni crecen ni decrecen.

- 2.4.107** (★★) **A MANO** Compruebe que $a_n = 3^n$ satisface $a_n = 4a_{n-1} - 3a_{n-2}$ y determine si es la solución general.

SOLUCIÓN Sustituyendo: $4 \cdot 3^{n-1} - 3 \cdot 3^{n-2} = 3^{n-2}(12 - 3) = 9 \cdot 3^{n-2} = 3^n$. Sí la satisface, pero *no* es la solución general: la ecuación característica $t^2 - 4t + 3 = 0$ tiene raíces 1 y 3,

de modo que la general es $a_n = b_1 + b_2 3^n$. La expresión dada corresponde al caso $b_1 = 0$, $b_2 = 1$.

2.4.108 (★★★) A MANO Halle una relación homogénea lineal de orden dos cuya solución general sea $a_n = b_1 2^n + b_2 (-5)^n$.

SOLUCIÓN Se recorre el camino inverso: las raíces deben ser 2 y -5 , luego la ecuación característica es

$$(t - 2)(t + 5) = t^2 + 3t - 10 = 0$$

y la relación correspondiente es $a_n = -3a_{n-1} + 10a_{n-2}$.

2.4.109 (★★★) A MANO Halle una relación homogénea lineal de orden dos cuya solución general sea $a_n = (b_1 + b_2 n) 4^n$.

SOLUCIÓN La forma con factor n indica raíz doble $t = 4$, de modo que la ecuación característica es $(t - 4)^2 = t^2 - 8t + 16 = 0$ y la relación es

$$a_n = 8a_{n-1} - 16a_{n-2}$$

2.4.110 (★) MIXTO Verifique la relación del ejercicio anterior generando sus términos con $a_1 = 3$ y $a_2 = 4$.

SOLUCIÓN

In[] :=

```
FindRRHL[{3, 4, -16, -192, -1280, -7168, -36864}, a, n]
```

Out[] =

```
{a[n] == -16*a[-2 + n] + 8*a[-1 + n], a[1] == 3, a[2] == 4, -(2^(-2 + 2*n))*(-5 + 2*n))}
```

El comando recupera exactamente $a_n = 8a_{n-1} - 16a_{n-2}$ a partir de los datos, y entrega además la fórmula explícita.

2.4.111 (★★) A MANO ¿Puede una relación homogénea lineal de orden dos con coeficientes reales tener una única raíz real y otra compleja? Justifique.

SOLUCIÓN No. Los coeficientes reales obligan a que las raíces complejas aparezcan en pares conjugados: si r es raíz, también lo es $\bar{r}$. Con grado dos, o las dos raíces son reales, o ambas son complejas conjugadas.

2.4.112 (★★★) A MANO Resuelva $a_n = 6a_{n-1} - 9a_{n-2}$ con $a_0 = 2$ y $a_1 = 9$.

SOLUCIÓN La ecuación característica es $t^2 - 6t + 9 = (t - 3)^2 = 0$, con raíz doble $t = 3$, de modo que $a_n = (b_1 + b_2 n) 3^n$. Las condiciones dan $b_1 = 2$ y $3(b_1 + b_2) = 9$, luego $b_2 = 1$. Así

$$a_n = (n + 2) 3^n$$

Los primeros términos son 2, 9, 36, 135.

2.4.113 (★★) VilCretas Resuelva con RRCP los tres primeros casos de la familia $a_n = 5a_{n-1} - 6a_{n-2} + j$ con $a_0 = 1$ y $a_1 = 2$.

SOLUCIÓN

In[] :=

```
RRCP[{5, -6, j}, {1, 2}, n, 3, inicio -> 0]
```

La última componente del vector de coeficientes es la parte no homogénea, que aquí depende del parámetro j , y el cuarto argumento indica cuántos casos se evalúan. La salida entrega

$$j = 1 : a_n = \frac{1 + 3^n}{2}, \quad j = 2 : a_n = 1 - 2^n + 3^n, \quad j = 3 : a_n = \frac{3 - 2^{n+2} + 3^{n+1}}{2}$$

de modo que un solo llamado resuelve toda la familia.

2.4.114 (★★★) **A MANO** Dos relaciones distintas pueden generar la misma sucesión. Muestre que $1, 2, 4, 8, 16$ satisface tanto $a_n = 2a_{n-1}$ como $a_n = 5a_{n-1} - 6a_{n-2}$.

SOLUCIÓN Para la primera es inmediato. Para la segunda, con $a_n = 2^n$:

$$5 \cdot 2^{n-1} - 6 \cdot 2^{n-2} = 2^{n-2}(10 - 6) = 4 \cdot 2^{n-2} = 2^n$$

La segunda tiene raíces 2 y 3; la sucesión dada corresponde a la solución particular con coeficiente nulo en 3^n . Una sucesión puede satisfacer relaciones de orden mayor que el mínimo necesario.

2.4.115 (★★★) **VilCretas** Compare $\text{RT}[\{1, n\}, \{1\}, 7]$ con $\text{RT}[\{1, n\}, \{1\}, 7, \text{inicio} \rightarrow 0]$ y explique la diferencia.

SOLUCIÓN

```
In[ ] :=
```

```
{RT[{1, n}, {1}, 7], RT[{1, n}, {1}, 7, inicio -> 0]}
```

```
Out[ ] =
```

```
{{1, 3, 6, 10, 15, 21, 28}, {1, 2, 4, 7, 11, 16, 22}}
```

Son **sucesiones distintas**. Cuando la parte no homogénea depende de n , el valor de **inicio** decide en qué índice se evalúa: con el valor por defecto 1 el primer incremento es 2, y con 0 es 1. Como **RR** tiene la misma opción, hay que darle **el mismo valor a los dos comandos**; si no, la lista y la fórmula no se corresponden y nada avisa del error.

2.4.116 (★★★) **A MANO** ¿Por qué la advertencia del ejercicio anterior no afecta a $a_n = 2a_{n-1} + 5$?

SOLUCIÓN Porque la parte no homogénea es **constante**: vale 5 en todos los índices, de modo que desplazar el origen no cambia ningún incremento. El riesgo aparece solo cuando la parte no homogénea depende de n ($2n, n^2, 3^n$), que es justo cuando más fácil resulta equivocarse.

2.4.117 (★★) **MIXTO** Resuelva $a_n = 6a_{n-1} - 8a_{n-2}$ con $a_0 = 1$ y $a_1 = 6$.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^2 - 6 t + 8 == 0, t]
```

```
Out[ ] =
```

```
{{t -> 2}, {t -> 4}}
```

```
In[ ] :=
```



```
RR[{6, -8}, {1, 6}, n, inicio -> 0]
```

```
Out[ ] =
```

```
2^n*(-1 + 2^(1 + n))
```

Es decir $a_n = 2 \cdot 4^n - 2^n$. Las raíces 2 y 4 dan las dos potencias, y las condiciones iniciales fijan los coeficientes.

- 2.4.118** (★★★) **MIXTO** Resuelva $a_n = -2a_{n-1} + 15a_{n-2}$ con $a_0 = 2$ y $a_1 = 1$, y explique la alternancia de signos.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^2 + 2 t - 15 == 0, t]
```

```
Out[ ] =
```

```
{{t -> -5}, {t -> 3}}
```

```
In[ ] :=
```

```
RR[{-2, 15}, {2, 1}, n, inicio -> 0]
```

```
Out[ ] =
```

```
(11*3^n + (-1)^n*5^(1 + n))/8
```

La raíz **negativa** -5 introduce el factor $(-1)^n$: la sucesión oscila. Como $5 > 3$, ese término domina y la oscilación crece en amplitud.

- 2.4.119** (★★) **MIXTO** Resuelva $a_n = 6a_{n-1} - 9a_{n-2}$ con $a_0 = 1$ y $a_1 = 6$ e identifique el caso.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^2 - 6 t + 9 == 0, t]
```

```
Out[ ] =
```

```
{{t -> 3}, {t -> 3}}
```

```
In[ ] :=
```

```
RR[{6, -9}, {1, 6}, n, inicio -> 0]
```

```
Out[ ] =
```

```
3^n*(1 + n)
```

Raíz **doble** $t = 3$: la solución general es $(b_1 + b_2 n)3^n$ y aquí resulta $(n+1)3^n$. El factor n es la marca inconfundible de la raíz repetida.

- 2.4.120** (★★★) **MIXTO** Resuelva $a_n = 10a_{n-1} - 25a_{n-2}$ con $a_0 = 2$ y $a_1 = 15$.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^2 - 10 t + 25 == 0, t]
```

```
Out[ ] =
```

```
{{t -> 5}, {t -> 5}}
```

```
In[ ] :=
```

```
RR[{10, -25}, {2, 15}, n, inicio -> 0]
```

```
Out[ ] =
```

```
5^n*(2 + n)
```

La solución es $(n+2)5^n$. Compruebe: $a_0 = 2$, $a_1 = 3 \cdot 5 = 15$, $a_2 = 4 \cdot 25 = 100$, y en efecto $10 \cdot 15 - 25 \cdot 2 = 100$.

2.4.121 (★★★) **MIXTO** Resuelva $a_n = -4a_{n-2}$ con $a_0 = 1$ y $a_1 = 2$, e interprete las raíces complejas.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^2 + 4 == 0, t]
```

```
Out[ ] =
```

```
{{t -> -2*I}, {t -> 2*I}}
```

```
In[ ] :=
```

```
RT[{0, -4}, {1, 2}, 9]
```

```
Out[ ] =
```

```
{1, 2, -4, -8, 16, 32, -64, -128, 256}
```

Las raíces son $\pm 2i$, de módulo 2 y argumento $\pm \frac{\pi}{2}$. La sucesión tiene **periodo cuatro** en los signos (+, +, -, -) y su valor absoluto se duplica: es una oscilación de amplitud creciente.

2.4.122 (★★★) **A MANO** Explique la relación entre el módulo de las raíces complejas y el comportamiento de la sucesión.

SOLUCIÓN Si las raíces son $\rho(\cos \theta \pm i \sin \theta)$, la solución tiene la forma $\rho^n (A \cos n\theta + B \sin n\theta)$: el módulo ρ controla la **amplitud** y el argumento θ la **frecuencia**. Con $\rho > 1$ la oscilación crece, con $\rho < 1$ se amortigua y con $\rho = 1$ se mantiene periódica.

2.4.123 (★★★) **MIXTO** Resuelva la relación cúbica $a_n = 3a_{n-1} + a_{n-2} - 3a_{n-3}$ con $a_0 = 1$, $a_1 = 2$ y $a_2 = 4$.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^3 - 3 t^2 - t + 3 == 0, t]
```

```
Out[ ] =
```

```
{{t -> -1}, {t -> 1}, {t -> 3}}
```

```
In[ ] :=
```

```
RR[{3, 1, -3}, {1, 2, 4}, n, inicio -> 0]
```

```
Out[ ] =
```

```
(6 - (-1)^n + 3^(1 + n))/8
```

Tres raíces distintas dan tres términos: una constante (de $t = 1$), una alternancia (de $t = -1$) y el crecimiento dominante 3^n .

2.4.124 (★★) **MIXTO** Halle el conjunto solución de la ecuación característica de $a_n = 7a_{n-2} - 6a_{n-3}$.

SOLUCIÓN

```
In[ ] :=
```

```
Solve[t^3 - 7 t + 6 == 0, t]
```

```
Out[ ] =
```

```
{{t -> -3}, {t -> 1}, {t -> 2}}
```

Ordenado de menor a mayor: $\{-3, 1, 2\}$. El coeficiente de a_{n-1} es cero, y por eso la ecuación no tiene término en t^2 ; hay que escribirlo igual al armar el vector de coeficientes.

2.4.125 (★★★) **MIXTO** Resuelva $a_n = 4a_{n-1} - 4a_{n-2}$ con $a_0 = 2$ y $a_1 = 6$ y compare con el caso $a_0 = 1, a_1 = 4$.

SOLUCIÓN

```
In[ ] :=
```

```
{RR[{4, -4}, {2, 6}, n, inicio -> 0], RR[{4, -4}, {1, 4}, n, inicio -> 0]}
```

```
Out[ ] =
```

```
{2^n*(2 + n), 2^n*(1 + n)}
```

```
In[ ] :=
```

```
RT[{4, -4}, {2, 6}, 7]
```

```
Out[ ] =
```

```
{2, 6, 16, 40, 96, 224, 512}
```

Las dos tienen raíz doble $t = 2$ y forma $(b_1 + b_2n)2^n$; solo cambian los coeficientes. Las condiciones iniciales no alteran la **estructura** de la solución, solo la instancia.

2.4.126 (★★) **VilCretas** Recupere la relación de recurrencia de la sucesión $\{2, 6, 22, 86, 342, 1366\}$.

SOLUCIÓN

```
In[ ] :=
```

```
FindRRHL[{2, 6, 22, 86, 342, 1366}, a, n]
```

El comando devuelve $a_n = 5a_{n-1} - 4a_{n-2}$ con $a_1 = 2$ y $a_2 = 6$, y la fórmula explícita $\frac{4^n + 2}{3}$.
Comprobación: $5 \cdot 6 - 4 \cdot 2 = 22$.

2.4.127 (★★) **VilCretas** Recupere la relación de $\{1, 4, 13, 40, 121, 364\}$ e identifique la fórmula.

SOLUCIÓN**In[] :=**`FindRRHL[{1, 4, 13, 40, 121, 364}, a, n]`

Devuelve $a_n = 4a_{n-1} - 3a_{n-2}$ con $a_1 = 1$ y $a_2 = 4$, y fórmula $\frac{3^n - 1}{2}$. Son las sumas parciales de las potencias de tres: $1, 1 + 3, 1 + 3 + 9, \dots$

- 2.4.128 (★★★) A MANO** ¿Cuántas relaciones de recurrencia lineales homogéneas distintas pueden generar una misma sucesión finita de seis términos?

SOLUCIÓN Infinitas. Con seis datos se puede ajustar una relación de orden dos, pero también una de orden tres, cuatro o cinco: cuantos más coeficientes, más grados de libertad. `FindRRHL` devuelve la de **menor orden** que encaja, que es la interpretación más económica, pero no la única posible.

- 2.4.129 (★★★) A MANO** Plantee el sistema de ecuaciones que resuelve $a_n = 5a_{n-1} - 6a_{n-2}$ con $a_0 = 3$ y $a_1 = 8$, y resúelvalo.

SOLUCIÓN Con raíces 2 y 3, la solución es $a_n = b_1 2^n + b_2 3^n$. Evaluando:

$$b_1 + b_2 = 3, \quad 2b_1 + 3b_2 = 8$$

Restando el doble de la primera a la segunda, $b_2 = 2$ y $b_1 = 1$. Por lo tanto $a_n = 2^n + 2 \cdot 3^n$.

- 2.4.130 (★★) MIXTO** Compruebe el resultado anterior.

SOLUCIÓN**In[] :=**

```
{Solve[{b1 + b2 == 3, 2 b1 + 3 b2 == 8}, {b1, b2}],
RR[{5, -6}, {3, 8}, n, inicio -> 0]}
```

Out[] =

```
{{{b1 -> 1, b2 -> 2}}, 2^n + 2*3^n}
```

El software devuelve directamente $2^n + 2 \cdot 3^n$, la misma solución obtenida a mano.

- 2.4.131 (★★★) A MANO** ¿Por qué el sistema asociado a una relación de orden k con raíces distintas siempre tiene solución única?

SOLUCIÓN Porque su matriz es de *Vandermonde*, formada por las potencias t_i^j de las raíces, y su determinante es $\prod_{i < j} (t_j - t_i)$, no nulo cuando las raíces son **distintas dos a dos**. Con raíces repetidas ese determinante se anula, y por eso hay que cambiar la forma de la solución introduciendo el factor n .

- 2.4.132 (★★) VilCretas** Use `MetodoRRHL` para ver el desarrollo completo de $a_n = 6a_{n-1} - 8a_{n-2}$ con $a_0 = 1$ y $a_1 = 6$.

SOLUCIÓN**In[] :=**`MetodoRRHL[{6, -8}, {1, 6}, n, b, inicio -> 0]`

La salida recorre la ecuación característica, sus raíces $\{2, 4\}$, la forma general $b_1 2^n + b_2 4^n$, el sistema $b_1 + b_2 = 1$, $2b_1 + 4b_2 = 6$ y la solución final. Sin `inicio -> 0` el comando toma $a_1 = 1$ y $a_2 = 6$, y la respuesta cambia.

2.4.133 (★★★) MIXTO Resuelva la familia $a_n = 5a_{n-1} - 6a_{n-2} + j$ con $a_0 = 1$ y $a_1 = 2$ para $j = 1, 2, 3$.

SOLUCIÓN

`In[ ] :=`

`RRCP[{5, -6, j}, {1, 2}, n, 3, inicio -> 0]`

La salida entrega

$$j = 1: \frac{1 + 3^n}{2}, \quad j = 2: 1 - 2^n + 3^n, \quad j = 3: \frac{3 - 2^{n+2} + 3^{n+1}}{2}$$

Un solo llamado resuelve toda la familia. La última componente del vector de coeficientes es la parte no homogénea y debe depender de j .

2.4.134 (★★★) A MANO En la familia anterior, ¿para qué valor de j desaparece el término en 2^n ?

SOLUCIÓN Para $j = 1$, cuya solución es $\frac{1 + 3^n}{2}$: no aparece ninguna potencia de dos. Es un caso de **cancelación**: la solución particular asociada a j compensa exactamente la componente $b_1 2^n$ que fijaban las condiciones iniciales.

2.4.135 (★★) VilCretas Represente gráficamente los términos de $a_n = 6a_{n-1} - 8a_{n-2}$ con $a_0 = 1$ y $a_1 = 6$.

SOLUCIÓN

`In[ ] :=`

`GraficaRRL[{6, -8}, {1, 6}, inicio -> 0, npuntos -> 8]`

La gráfica muestra el crecimiento dominado por 4^n . `npuntos` fija cuántos términos se dibujan y `ymax` el tope del eje vertical, útil cuando unos pocos valores grandes aplastan a los demás.

2.4.136 (★★★) A MANO Una relación de orden dos tiene raíces $\frac{1}{2}$ y $\frac{1}{3}$. Describa el comportamiento de cualquier solución suya.

SOLUCIÓN Toda solución es $b_1 \left(\frac{1}{2}\right)^n + b_2 \left(\frac{1}{3}\right)^n$ y por lo tanto **tiende a cero**, sin importar las condiciones iniciales. El término $\left(\frac{1}{2}\right)^n$ domina, de modo que la convergencia es del orden de 2^{-n} . Cuando todas las raíces tienen módulo menor que uno la sucesión se amortigua siempre.

2.4.137 (★★★) MIXTO Compruebe el comportamiento anterior con $a_0 = 1$ y $a_1 = 0$.

SOLUCIÓN La relación con esas raíces es $a_n = \frac{5}{6}a_{n-1} - \frac{1}{6}a_{n-2}$.

`In[ ] :=`

`N[RT[{5/6, -1/6}, {1, 0}, 7, inicio -> 0]]`

`Out[ ] =`

{1., 0., -0.16666666666666666, -0.13888888888888889, -0.08796296296296297, -0.05015432098765432, -0.02713477366255144}

Los valores se acercan a cero. El signo negativo aparece porque las condiciones iniciales dan un coeficiente negativo, pero la amortiguación es la misma.

2.4.138 (★★★) **A MANO** Resuma en una regla cómo el mayor módulo de las raíces determina el crecimiento de la sucesión.

SOLUCIÓN Sea ρ el máximo de los módulos de las raíces. Si $\rho > 1$ la sucesión **diverge** y su orden de crecimiento es $\Theta(\rho^n)$, salvo que las condiciones iniciales anulen ese término; si $\rho < 1$ **converge a cero**; y si $\rho = 1$ queda acotada, con posible oscilación periódica. Con raíz repetida de módulo ρ el crecimiento es $\Theta(n\rho^n)$.

2.4.139 (★★) **A MANO** Aplique la regla anterior a $a_n = a_{n-1} + a_{n-2}$ y a $a_n = \frac{a_{n-1} + a_{n-2}}{2}$.

SOLUCIÓN En la primera las raíces son $\frac{1 \pm \sqrt{5}}{2}$, con módulo mayor $\varphi \approx 1,618 > 1$: crecimiento exponencial $\Theta(\varphi^n)$. En la segunda son 1 y $-\frac{1}{2}$, con máximo módulo 1: la sucesión **converge** a la media ponderada de las condiciones iniciales, sin crecer.

2.6 Generadores automáticos de relaciones de recurrencia

2.4.140 (★★) **VilCretas** Genere con **RRHLRandom** una relación de recurrencia homogénea lineal de orden 2 y explique qué significa cada uno de sus tres parámetros.

SOLUCIÓN

In[] :=

```
RRHLRandom[2, 10, 100]
```

El primer parámetro es el **orden**; el segundo y el tercero acotan los enteros de donde se escogen, respectivamente, los **coeficientes** y las **condiciones iniciales**. La salida viene ya escrita como ecuación, con sus condiciones: es un enunciado listo para trabajar.

2.4.141 (★★★) **VilCretas** Pida la misma relación en formato vectorial y resúelvala en el acto, encadenando con **RR**.

SOLUCIÓN

In[] :=

```
r = RRHLRandom[2, 10, 100, vectorial -> True];
RR[r[[1]], r[[2]], n]
```

Con **vectorial -> True** la salida es $\{\{\text{coeficientes}\}, \{\text{condiciones}\}\}$, que es **exactamente** lo que **RR** espera como primeros dos argumentos. De ahí que los dos comandos se encadenen sin adaptar nada: el generador y el solucionador hablan el mismo formato.

2.4.142 (★★★) **MIXTO** El generador acepta **inicio -> 0**. Compruebe, sobre un caso fijo, que cambiar el inicio cambia la solución.

SOLUCIÓN

In[] :=

```
{RR[{3, -2}, {1, 3}, n], RR[{3, -2}, {1, 3}, n, inicio -> 0]}
```

Out[] =

```
{-1 + 2^n, -1 + 2^(1 + n)}
```

Las mismas condiciones 1 y 3 dan $2^n - 1$ si se leen como a_1, a_2 y $2^{n+1} - 1$ si se leen como a_0, a_1 . **No es un detalle de presentación:** son sucesiones distintas. Al generar un enunciado con **inicio** $\rightarrow 0$ hay que resolverlo también con **inicio** $\rightarrow 0$.

2.4.143 (★★) **MIXTO** Compruebe la afirmación anterior listando los primeros seis términos.

SOLUCIÓN

In[] :=

```
RT[{3, -2}, {1, 3}, 6, inicio -> 0]
```

Out[] =

```
{1, 3, 7, 15, 31, 63}
```

Son $2^{n+1} - 1$ para $n = 0, 1, \dots, 5$. Contrastar la fórmula cerrada con los primeros términos es la comprobación más barata que existe, y detecta casi todos los errores de índice.

2.4.144 (★★★) **VilCretas** Fuerce con **repeatoneroot** una ecuación característica con una raíz repetida y observe qué forma toma la solución.

SOLUCIÓN

In[] :=

```
RRHLRandom[3, 10, 100, repeatoneroot -> 3, vectorial -> True]
```

La opción pide que una raíz se repita el número de veces indicado. Es justo el caso que en un examen se resiste: cuando la raíz r tiene multiplicidad m , la solución no lleva m términos r^n (que serían linealmente dependientes) sino $r^n, n r^n, \dots, n^{m-1} r^n$.

2.4.145 (★★★) **MIXTO** Verifique esa forma sobre un caso fijo de raíz doble.

SOLUCIÓN

In[] :=

```
RR[{4, -4}, {1, 6}, n, inicio -> 0]
```

Out[] =

```
2^n*(1 + 2*n)
```

La ecuación característica es $t^2 - 4t + 4 = 0$, con raíz doble $t = 2$. La solución es $2^n(1 + 2n)$: aparece el factor n que anuncia la multiplicidad.

2.4.146 (★★) **MIXTO** Compruebe los primeros términos del caso anterior.

SOLUCIÓN

In[] :=

```
RT[{4, -4}, {1, 6}, 6, inicio -> 0]
```

Out[] =

```
{1, 6, 20, 56, 144, 352}
```

Coinciden con $2^n(1 + 2n)$: para $n = 2$ da $4 \cdot 5 = 20$ y para $n = 3$, $8 \cdot 7 = 56$.

2.4.147 (★★★) **VilCretas** `RRLRandom` genera recurrencias lineales **no** necesariamente homogéneas. Générelas y diga en qué se distingue su salida de la de `RRHLRandom`.

SOLUCIÓN

`In[ ] :=`

```
RRLRandom[1, 10, 100, vectorial -> True]
```

En `RRHLRandom` el primer bloque son solo números: los coeficientes de la parte homogénea. En `RRLRandom` ese bloque puede traer además un término que **depende de n** : la parte no homogénea. Reconocerlo es el primer paso, porque decide el método: ecuación característica a secas, o ecuación característica más solución particular.

2.4.148 (★★★) **MIXTO** Sobre la recurrencia $a_n = a_{n-1} - 14n^2$ con $a_1 = -28$, obtenga los cinco primeros términos.

SOLUCIÓN

`In[ ] :=`

```
RT[{1, -14*n^2}, {-28}, 5]
```

`Out[ ] =`

```
{-28, -84, -210, -434, -784}
```

Es del tipo que devuelve `RRLRandom`: coeficiente 1 y parte no homogénea $-14n^2$. Las diferencias sucesivas son $-56, -126, -224, -350$, es decir $-14n^2$ evaluado en $n = 2, 3, 4, 5$: la comprobación cierra sin necesidad de resolver nada.

2.4.149 (★★) **VilCretas** Genere un enunciado con coeficientes racionales.

SOLUCIÓN

`In[ ] :=`

```
RRHLRandom[2, 10, 100, racionales -> True, inicio -> 0]
```

Con `racionales -> True` los coeficientes y las condiciones pueden ser fracciones. Conviene practicarlo: con enteros, las raíces de la ecuación característica salen *redondas* con demasiada frecuencia, y eso da una falsa sensación de facilidad.

CAPÍTULO 3 • COMPLEMENTARIOS

Análisis de algoritmos

3.1 Qué se mide al analizar un algoritmo

- 3.6.1** (★) **A MANO** Explique en qué se diferencian el enfoque experimental y el enfoque teórico del análisis de algoritmos.

SOLUCIÓN El **enfoque experimental** ejecuta las implementaciones en un ordenador, mide los tiempos de salida y cuenta en cuántos experimentos ganó cada método: la conclusión es una **tendencia**. El **enfoque teórico** busca una función $f(n)$ que describe el esfuerzo del algoritmo con n datos de entrada y la acota con funciones más simples: la conclusión es una **cota válida para todo n** , independiente de la máquina.

- 3.6.2** (★) **A MANO** Defina qué se entiende por peor caso, mejor caso y caso promedio.

SOLUCIÓN Para una misma cantidad n de datos de entrada el esfuerzo puede variar según cómo vengan esos datos. El **peor caso** es la entrada que obliga al algoritmo a trabajar más; el **mejor caso** es la que lo deja terminar antes; el **caso promedio** pondera todas las entradas posibles. La notación O grande se emplea sobre todo para el peor caso, Ω para el mejor y Θ para el caso promedio.

- 3.6.3** (★) **A MANO** Un algoritmo es $O(n^2)$ y otro $O(n \ln n)$. ¿Cuál conviene cuando n es muy grande?

SOLUCIÓN El de $O(n \ln n)$. Aunque para valores pequeños de n el primero puede ser más rápido, $\frac{n \ln n}{n^2} = \frac{\ln n}{n} \rightarrow 0$, de modo que $n \ln n$ queda muy por debajo de n^2 al crecer n . Las notaciones asintóticas describen precisamente el comportamiento **para n grande**.

- 3.6.4** (★★) **A MANO** ¿Por qué el enfoque experimental no permite concluir con certeza cuál algoritmo es mejor?

SOLUCIÓN Porque el tiempo medido depende del procesador, de la memoria disponible, del resto de procesos activos y de la implementación concreta. Dos ejecuciones del mismo programa dan tiempos distintos. Por eso el método se apoya en **muchas invocaciones** y en contar victorias, no en una sola medición: lo que se obtiene es una tendencia, no una demostración.

- 3.6.5** (★★) **A MANO** Cuente cuántas veces se ejecuta la asignación interna del siguiente fragmento y dé su orden.

SOLUCIÓN

In[] :=

For[$i = 1, i \leq n, p = p + 1; i++$]

El contador i recorre $1, 2, \dots, n$, de modo que la asignación se ejecuta exactamente n veces. El esfuerzo es $f(n) = n$ y por lo tanto $\Theta(n)$.

- 3.6.6** (★★) **A MANO** Repita el conteo anterior para dos ciclos anidados que recorren ambos de 1 a n .

SOLUCIÓN

In[] :=

```
For[i = 1, i <= n, For[j = 1, j <= n, p = p + 1; j++]; i++]
```

Por cada uno de los n valores de i el ciclo interno realiza n pasos, en total $f(n) = n \cdot n = n^2$ asignaciones, es decir $\Theta(n^2)$.

- 3.6.7** (★★) **A MANO** El juego de las torres de *Hanoi* con n discos exige $f(n) = 2^n - 1$ movimientos. Halle una cota superior sencilla y justifique.

SOLUCIÓN Como $2^n - 1 < 2^n$ para todo n natural, la función $g(n) = 2^n$ acota superiormente a f con la constante $c = 1$. Por la definición de O grande, $f(n) = O(2^n)$: el algoritmo es de orden exponencial, y por eso resulta impracticable con 64 discos.

- 3.6.8** (★★) **A MANO** Un algoritmo de búsqueda recorre una lista de n datos hasta encontrar el valor buscado. Describa su mejor caso, su peor caso y el orden de cada uno.

SOLUCIÓN El **mejor caso** es que el dato esté en la primera posición: una sola comparación, orden $\Omega(1)$. El **peor caso** es que esté en la última o que no esté: n comparaciones, orden $O(n)$. En promedio se recorren $\frac{n+1}{2}$ posiciones, que también es $\Theta(n)$.

- 3.6.9** (★★★) **A MANO** Un ciclo divide en cada paso el valor de i entre 2, partiendo de $i = n$ y deteniéndose cuando $i < 1$. Determine cuántas iteraciones realiza.

SOLUCIÓN Tras k pasos el valor es $\frac{n}{2^k}$, y el ciclo termina cuando $\frac{n}{2^k} < 1$, es decir cuando $k > \log_2 n$. El número de iteraciones es $\lfloor \log_2 n \rfloor + 1$, de orden $\Theta(\ln n)$. Que la base sea 2 no altera el orden: cambiar de base solo multiplica por una constante.

- 3.6.10** (★★) **MIXTO** Compruebe el conteo anterior tabulando $\lfloor \log_2 n \rfloor$ para $n = 1, \dots, 16$.

SOLUCIÓN

```
In[ ] :=
```

```
Table[Floor[Log[2, n]], {n, 1, 16}]
```

```
Out[ ] =
```

```
{0, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 4}
```

El valor solo aumenta en las potencias de 2, lo que confirma el crecimiento logarítmico: duplicar n agrega una sola iteración.

- 3.6.11** (★★★) **A MANO** Un algoritmo compara cada elemento de una lista de n datos con todos los posteriores. Cuento las comparaciones y dé su orden.

SOLUCIÓN El primer elemento se compara con $n - 1$, el segundo con $n - 2$, y así hasta el penúltimo:

$$f(n) = (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

Por el teorema del polinomio, $f(n) = \frac{1}{2}n^2 - \frac{1}{2}n = O(n^2)$, y como también es $\Omega(n^2)$, resulta $\Theta(n^2)$.

- 3.6.12** (★★) **MIXTO** Verifique con *Mathematica* la suma del ejercicio anterior para $n = 101$.

SOLUCIÓN

```
In[ ] :=
```

```
Sum[i, {i, 1, 100}]
```

```
Out[ ] =
```

```
5050
```

Con $n = 101$ la fórmula da $\frac{101 \cdot 100}{2} = 5050$, que coincide con la suma directa de los primeros cien enteros.

3.2 Enfoque experimental

3.6.13 (★) **VilCretas** Ordene la lista {5,3,9,1,7} con el algoritmo de la burbuja.

SOLUCIÓN

```
In[ ] :=
```

```
Burbuja[{5, 3, 9, 1, 7}]
```

```
Out[ ] =
```

```
{1, 3, 5, 7, 9}
```

Por defecto **Burbuja** ordena de forma ascendente.

3.6.14 (★) **VilCretas** Ordene la misma lista de forma descendente.

SOLUCIÓN

```
In[ ] :=
```

```
Burbuja[{5, 3, 9, 1, 7}, ascendente -> False]
```

```
Out[ ] =
```

```
{9, 7, 5, 3, 1}
```

La opción **ascendente** admite **True** (valor por defecto) y **False**.

3.6.15 (★) **VilCretas** Ordene {4,2,8,6,1} con el método de inserción.

SOLUCIÓN

```
In[ ] :=
```

```
Insercion[{4, 2, 8, 6, 1}]
```

```
Out[ ] =
```

```
{1, 2, 4, 6, 8}
```

El método toma cada dato y lo inserta en el lugar que le corresponde dentro de la parte ya ordenada.

3.6.16 (★) **VilCretas** Ordene {10,1,7,3,5} con el método de selección.

SOLUCIÓN

```
In[ ] :=
```

```
Seleccion[{10, 1, 7, 3, 5}]
```

```
Out[ ] =
```

{1, 3, 5, 7, 10}

El método busca el máximo, lo intercambia con el último elemento y repite el razonamiento sobre la lista restante.

3.6.17 (★★) **VilCretas** Observe el paso a paso del algoritmo de la burbuja sobre {5, 3, 9, 1, 7}.

SOLUCIÓN

In[] :=

```
Burbuja[{5, 3, 9, 1, 7}, steps -> True]
```

La opción `steps->True` despliega la lista después de cada intercambio. Se aprecia cómo el mayor valor **sube** hasta la última posición en la primera pasada, que es lo que da nombre al método.

3.6.18 (★★) **VilCretas** Obtenga el código interno del método de selección.

SOLUCIÓN

In[] :=

```
Seleccion[{10, 1, 7, 3, 5}, code -> True]
```

La opción `code->True` devuelve, junto con el resultado, la definición de la función. Leerla permite contar los ciclos anidados y deducir por qué el método es $O(n^2)$ en el peor de los casos.

3.6.19 (★★) **VilCretas** Compare experimentalmente los métodos de selección e inserción.

SOLUCIÓN

In[] :=

```
PruebaADA2[{Seleccion, Insercion}, 20, 5, 1, lista -> True]
```

Los argumentos son la pareja de métodos, el número de invocaciones, el valor inicial y el incremento; `lista->True` indica que los métodos reciben listas, que el comando genera de forma pseudoaleatoria. La salida informa cuántas veces ganó cada uno y cuántas empataron. El ejercicio 3.5.4 del libro reporta a **Seleccion** como el más veloz; con listas cortas el conteo puede invertirse de una corrida a otra, porque las diferencias de tiempo son del orden del ruido de medición. Para que la tendencia se establezca hay que subir el valor inicial y el incremento.

3.6.20 (★★) **VilCretas** Compare experimentalmente la recursividad de pila y la de cola que calculan el factorial.

SOLUCIÓN

In[] :=

```
PruebaADA2[{Factoriales, FactorialesCola}, 20, 5, 1]
```

Aquí no se usa `lista->True` porque ambos métodos reciben un número. Con valores pequeños de n los dos tardan prácticamente lo mismo y el conteo queda parejo; al aumentar las invocaciones la versión de cola se impone, pues no deja operaciones pendientes al regresar de cada llamada. Nótese que las tres cifras no siempre suman el número de invocaciones: el comando descarta las mediciones que no puede comparar.

3.6.21 (★★★) **VilCretas** Compare a la vez tres formas de obtener los números de *Fibonacci*.

SOLUCIÓN

In[] :=

```
PruebaADA3[{Burbuja, Insercion, Seleccion}, 20, 10, 2, lista -> True]
```

PruebaADA3 informa, por parejas, cuántas veces resultaron más eficientes los métodos, más una cuarta cifra con los empates. Los tres ordenamientos son $O(n^2)$ en el peor de los casos, de modo que sobre listas cortas casi todas las invocaciones se contabilizan como **empate**: el experimento confirma que tienen el mismo orden. Para separarlos haría falta compararlos con un método de otro orden, como *quicksort*.

3.6.22 (★★) **VilCretas** Repita la comparación del factorial pero fijando usted mismo los valores de prueba.

SOLUCIÓN

In[] :=

```
PruebaADA2Valores[{Factoriales, FactorialesCola}, {10, 20, 30, 40, 50}]
```

PruebaADA2Valores sustituye los argumentos *inicio* e *incremento* por una lista de valores escogida por el usuario. Es útil cuando interesa observar el comportamiento en tamaños concretos y no en una progresión regular.

3.6.23 (★★) **VilCretas** Represente gráficamente la comparación entre las dos recursividades del factorial.

SOLUCIÓN

In[] :=

```
PruebaADAGrafica[{Factoriales, FactorialesCola}, 20, 5, 1]
```

PruebaADAGrafica dibuja los tiempos registrados en lugar de contar victorias, de modo que se ve la **forma** de cada curva y no solo quién gana. Admite dos o más métodos.

3.6.24 (★★) **VilCretas** Evalúe **CalculaSuma** en $n = 10$.

SOLUCIÓN

In[] :=

```
CalculaSuma[10]
```

Out[] =

200

El resultado es $2 \cdot 10^2$: el comando acumula un 2 por cada paso de dos ciclos anidados que recorren ambos de 1 a n .

3.6.25 (★★) **MIXTO** Obtenga el código de **CalculaSuma** y deduzca su notación asintótica.

SOLUCIÓN

In[] :=

```
CalculaSuma[10, code -> True]
```

El código mostrado es

```
CalculaSuma[n_] := Module[{p = 0}, For[i = 1, i <= n,
  For[j = 1, j <= n, p = p + 2; j++]; i++]; Return[p]]
```

Cada uno de los n valores de i dispara n pasos del ciclo interno, de modo que $f(n) = 2n^2$ y en consecuencia $f(n) = \Theta(n^2)$.

3.6.26 (★★) **MIXTO** Confirme la fórmula $f(n) = 2n^2$ tabulando los ocho primeros valores.

SOLUCIÓN

```
In[ ] :=
```

```
Table[CalculaSuma[k], {k, 1, 8}]
```

```
Out[ ] =
```

```
{2, 8, 18, 32, 50, 72, 98, 128}
```

Son $2 \cdot 1^2, 2 \cdot 2^2, \dots, 2 \cdot 8^2$, lo que confirma el conteo hecho sobre el código.

3.6.27 (★★) **VilCretas** Evalúe `CalculaProducto` y `CalculaOtroProducto` en $n = 5$ y comente el resultado.

SOLUCIÓN

```
In[ ] :=
```

```
CalculaProducto[5]
```

```
CalculaOtroProducto[5]
```

```
Out[ ] =
```

```
9
```

```
9
```

Ambos devuelven 9, pero no por la misma razón: el primero multiplica por 3 mientras el producto no supere a n , y el segundo repite ese ciclo dentro de otro que recorre de 1 a n . Coincidir en un valor no significa tener el mismo orden.

3.6.28 (★★★) **MIXTO** Analice el código de `CalculaProducto` y determine su notación asintótica.

SOLUCIÓN

```
In[ ] :=
```

```
CalculaProducto[5, code -> True]
```

El código es

```
CalculaProducto[n_] := Module[{p = 1}, While[p <= n, p = p*3];
  Return[p]]
```

El valor de p recorre $1, 3, 9, 27, \dots$, es decir 3^k , y el ciclo se detiene cuando $3^k > n$, o sea cuando $k > \log_3 n$. El esfuerzo es $f(n) = \lfloor \log_3 n \rfloor + 1$, de orden $\Theta(\ln n)$.

3.6.29 (★★) **A MANO** ¿Por qué `PruebaADA2` necesita muchas invocaciones y no una sola?

SOLUCIÓN Porque una única medición está a merced del estado momentáneo de la máquina: una tarea del sistema operativo puede retrasar justo la llamada que se está midiendo. Repitiendo el experimento y contando victorias, esas fluctuaciones se compensan y aparece la tendencia real.

- 3.6.30** (★★) **MIXTO** Mida el tiempo que tarda *Mathematica* en calcular 2000! y explique por qué conviene `RepeatedTiming`.

SOLUCIÓN

`In[ ] :=`

`Timing[Factorial[2000];]`

`RepeatedTiming[Factorial[2000];]`

`Timing` informa el tiempo de una sola evaluación; `RepeatedTiming` la repite varias veces y devuelve un promedio, mucho más estable. El punto y coma evita que se imprima el número, que tiene miles de cifras.

- 3.6.31** (★★★) **A MANO** Un experimento reporta que el método A ganó 12 veces y el B, 8, sobre 20 invocaciones. ¿Qué se puede y qué no se puede concluir?

SOLUCIÓN Se puede concluir una **tendencia** a favor de A en el rango de tamaños probado y en esa máquina. No se puede concluir que A sea asintóticamente mejor: para eso hace falta el enfoque teórico. De hecho un método $O(n^2)$ puede ganarle a uno $O(n \ln n)$ mientras n sea pequeño.

- 3.6.32** (★★) **VilCretas** Compruebe que los tres métodos de ordenamiento del paquete devuelven la misma lista ordenada.

SOLUCIÓN

`In[ ] :=`

`{Burbuja[{7, 2, 9, 4}], Insercion[{7, 2, 9, 4}], Seleccion[{7, 2, 9, 4}]}`

`Out[ ] =`

`{{2, 4, 7, 9}, {2, 4, 7, 9}, {2, 4, 7, 9}}`

Coinciden en la salida y difieren en el camino: comparar tiempos, y no resultados, es justamente el objeto del enfoque experimental.

- 3.6.33** (★★★) **MIXTO** Compare experimentalmente la versión de pila y la de cola de *Fibonacci* comprobando primero que dan el mismo valor.

SOLUCIÓN

`In[ ] :=`

`f1[n_] := If[n <= 1, n, f1[n - 1] + f1[n - 2]];`

`f2[n_] := Module[{g}, g[i_, a_, b_] := If[i == n, a, g[i + 1, b, a + b]]; g[0, 0, 1];`

`{f1[18], f2[18]}`

`Out[ ] =`

`{2584, 2584}`

Coinciden. El experimento se monta con `PruebaADA2[{f1, f2}, 20, 5, 1]`, y la de cola gana con holgura: hace n pasos frente a los $2F_n - 1$ de la de pila.

- 3.6.34** (★★★) **MIXTO** Compare un cálculo iterativo con su fórmula cerrada.

SOLUCIÓN

`In[ ] :=`

```
g1[n_] := Module[{s = 0, i}, For[i = 1, i <= n, i++, s = s + i]; s];
g2[n_] := n (n + 1)/2;
{g1[100], g2[100]}
```

Out[] =

```
{5050, 5050}
```

El mismo resultado con órdenes distintos: $\Theta(n)$ el ciclo y $\Theta(1)$ la fórmula. Es el caso extremo del análisis de algoritmos: a veces el mejor algoritmo es **no iterar**.

3.6.35 (★★) **MIXTO** Cuentas las ejecuciones de un doble ciclo con ambos índices de 1 a n .

SOLUCIÓN

In[] :=

```
h1[n_] := Module[{p = 0, i, j}, For[i = 1, i <= n, i++,
  For[j = 1, j <= n, j++, p = p + 1]]; p];
Table[h1[k], {k, 1, 6}]
```

Out[] =

```
{1, 4, 9, 16, 25, 36}
```

Son los cuadrados perfectos: $f(n) = n^2$, de orden $\Theta(n^2)$.

3.6.36 (★★★) **MIXTO** Cuentas las ejecuciones cuando el ciclo interno recorre de 1 a i .

SOLUCIÓN

In[] :=

```
h3[n_] := Module[{p = 0, i, j}, For[i = 1, i <= n, i++,
  For[j = 1, j <= i, j++, p = p + 1]]; p];
Table[h3[k], {k, 1, 8}]
```

Out[] =

```
{1, 3, 6, 10, 15, 21, 28, 36}
```

Son los números triangulares $\frac{n(n+1)}{2}$: sigue siendo $\Theta(n^2)$, con la mitad de la constante. La forma triangular del recorrido no cambia el orden.

3.6.37 (★★★) **MIXTO** Cuentas las ejecuciones de un ciclo cuyo índice se **duplica** en cada paso.

SOLUCIÓN

In[] :=

```
h2[n_] := Module[{p = 0, i}, For[i = 1, i <= n, i = 2 i, p = p + 1]; p];
Table[h2[2^k], {k, 0, 8}]
```

Out[] =

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

El conteo crece de uno en uno cuando n se **duplica**: es $\lfloor \log_2 n \rfloor + 1$, de orden $\Theta(\ln n)$. Pasar de $n = 256$ a $n = 512$ añade un solo paso.

3.6.38 (★★★) **A MANO** Un ciclo tiene la forma `For[i = n, i >= 1, i = i/2, ...]` con división entera. ¿Cuántas veces se ejecuta y por qué no depende del punto de partida más que logarítmicamente?

SOLUCIÓN Se ejecuta $\lfloor \log_2 n \rfloor + 1$ veces. Cada paso divide el valor entre dos, de modo que el número de pasos es el número de veces que hay que dividir n para llegar a uno. Multiplicar n por mil solo añade unos diez pasos: eso es lo que significa el orden logarítmico.

3.6.39 (★★) **VilCretas** Observe el paso a paso del método de la burbuja sobre $\{4, 1, 3, 2\}$.

SOLUCIÓN

`In[ ] :=`

`Burbuja[{4, 1, 3, 2}, steps -> True]`

La salida muestra las listas intermedias tras cada comparación. Se ve cómo el 4 **sube** hasta el final en la primera pasada, y cómo cada pasada sucesiva trabaja sobre una lista más corta.

3.6.40 (★★★) **A MANO** ¿Cuántas comparaciones hace el método de la burbuja sobre una lista de n datos, en el peor de los casos?

SOLUCIÓN La primera pasada compara $n - 1$ pares, la segunda $n - 2$, y así hasta una. El total es

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

de orden $\Theta(n^2)$. Para $n = 8$ son 28 comparaciones, y para $n = 80$ ya son 3160: multiplicar por diez el tamaño multiplica por cien el trabajo.

3.6.41 (★★) **MIXTO** Compruebe la fórmula de comparaciones del ejercicio anterior.

SOLUCIÓN

`In[ ] :=`

`Table[k (k - 1)/2, {k, 1, 8}]`

`Out[ ] =`

`{0, 1, 3, 6, 10, 15, 21, 28}`

Con un solo dato no hay ninguna comparación, y con ocho hay 28. La sucesión es la de los números triangulares desplazada.

3.6.42 (★★) **MIXTO** Compruebe que los métodos de inserción y de selección producen la misma salida sobre la lista $\{7, 3, 9, 1, 5\}$.

SOLUCIÓN

`In[ ] :=`

`{Insercion[{7, 3, 9, 1, 5}], Seleccion[{7, 3, 9, 1, 5}]}`

`Out[ ] =`

`{{1, 3, 5, 7, 9}, {1, 3, 5, 7, 9}}`

Mismo resultado, distinto camino. Comparar **tiempos** y no resultados es lo que distingue el enfoque experimental; para eso está `PruebaADA2`.

3.6.43 (★★★) **A MANO** ¿En qué se diferencian inserción y selección cuando la lista ya viene ordenada?

SOLUCIÓN La **inserción** se vuelve $\Theta(n)$: cada dato ya está en su sitio y basta una comparación por elemento. La **selección** sigue siendo $\Theta(n^2)$: siempre recorre el resto de la lista para buscar el máximo, esté ordenada o no. Es un caso en que el **mejor caso** distingue dos métodos del mismo orden en el peor.

3.6.44 (★★) **VilCretas** Use `CompLimit` para comparar n con $n \ln n$.

SOLUCIÓN

`ln[ ] :=`

`CompLimit[{n, n Log[n]}]`

Responde 0 $\rightarrow n = O(n \text{ Log}[n])$, notación O grande: el límite del cociente es cero. El primer número de la salida es justamente ese límite, que es lo que decide cuál de las tres notaciones corresponde.

3.6.45 (★★) **VilCretas** Use `CompLimit` sobre dos funciones iguales y sobre 2^n y $n!$.

SOLUCIÓN

`ln[ ] :=`

`CompLimit[{n^2, n^2}]`

`CompLimit[{2^n, n!}]`

El primero responde notación **theta** con límite 1; el segundo, O grande con límite 0. La regla que aplica el comando es: límite cero da O , límite positivo finito da Θ , límite infinito da Ω .

3.6.46 (★★★) **A MANO** Un algoritmo $\Theta(n^2)$ procesa 10^4 datos en un segundo. ¿Cuántos datos procesará en un minuto?

SOLUCIÓN El tiempo crece con el cuadrado, de modo que multiplicar el tiempo por 60 multiplica el tamaño por $\sqrt{60} \approx 7,75$: unos 77 500 datos. Con un algoritmo $\Theta(n)$ serían 600 000, y con uno $\Theta(2^n)$ apenas seis datos más.

3.6.47 (★★★) **A MANO** Compare cuántos datos puede procesar en el mismo tiempo un algoritmo $\Theta(n \ln n)$ frente a uno $\Theta(n^2)$, si el segundo maneja 10^4 .

SOLUCIÓN Suponiendo la misma constante, hay que resolver $n \ln n = 10^8$. Tanteando, $n = 10^7$ da $\approx 1,6 \cdot 10^8$, algo alto; $n \approx 6 \cdot 10^6$ da $\approx 10^8$. Es decir, unos **seis millones** frente a diez mil: seiscientas veces más datos. Esa es la razón práctica por la que se prefieren los métodos $n \ln n$.

3.6.48 (★★★) **A MANO** Explique por qué un algoritmo $\Theta(n^2)$ puede ser preferible a uno $\Theta(n \ln n)$ en un caso real.

SOLUCIÓN Porque la notación asintótica **oculta las constantes**. Si el primero tarda n^2 microsegundos y el segundo $1000 n \ln n$, para $n = 100$ el primero tarda 10 000 y el segundo unos 460 000. El cruce ocurre alrededor de $n \approx 9000$. Para entradas pequeñas, el método de peor orden puede ser el más rápido, y por eso muchas implementaciones de *quicksort* cambian a inserción en las sublistas cortas.

3.6.49 (★★★) **A MANO** Un experimento reporta que un método ganó 18 de 20 invocaciones. ¿Basta para afirmar que es de mejor orden?

SOLUCIÓN No. El experimento cubre un rango **finito** de tamaños, y el orden es una propiedad del comportamiento cuando $n \rightarrow \infty$. Un método $\Theta(n^2)$ con constante pequeña puede ganar sistemáticamente en el rango probado y perder de forma abrumadora con entradas grandes. El experimento sugiere; el análisis teórico demuestra.

3.6.50 (★★) **A MANO** ¿Qué información aporta **PruebaADAGráfica** que no aporta **PruebaADA2**?

SOLUCIÓN **PruebaADA2** devuelve un **conteo de victorias**; la gráfica muestra la **forma** de las curvas de tiempo. Con la gráfica se puede ver dónde se cruzan los dos métodos, si uno crece linealmente y el otro cuadráticamente, o si las diferencias son puro ruido. El conteo pierde toda esa información.

3.6.51 (★★★) **A MANO** Al medir tiempos, ¿por qué se descarta la primera invocación?

SOLUCIÓN Porque incluye costos que no pertenecen al algoritmo: la compilación interna de la función, la carga de símbolos, la reserva de memoria. A partir de la segunda invocación el sistema ya tiene todo preparado. Por eso **RepeatedTiming** repite y promedia en vez de fiarse de una sola medida.

3.6.52 (★★) **A MANO** Un método recursivo de pila falla con $n = 2000$ pero funciona con $n = 500$. ¿Es un problema de orden o de otra cosa?

SOLUCIÓN No es de orden: es el **límite de recursión** del sistema, que en *Mathematica* está en 1024 niveles. Un algoritmo $\Theta(n)$ perfectamente eficiente puede fallar por esto si su implementación es de pila. La solución es reescribirlo de cola o iterativamente, sin cambiar el orden.

3.6.53 (★★★) **A MANO** Dos métodos son $\Theta(n^2)$, pero uno hace $\frac{n^2}{2}$ operaciones y el otro $2n^2$. ¿Qué dice el experimento y qué dice la teoría?

SOLUCIÓN La **teoría** dice que son del mismo orden: el cociente entre ambos es la constante 4, que no depende de n . El **experimento** dirá que el primero es cuatro veces más rápido, siempre, para cualquier tamaño. Ambas afirmaciones son correctas y responden a preguntas distintas: la teoría predice cómo escala, el experimento cuánto cuesta hoy.

3.6.54 (★★★) **A MANO** ¿Qué mide realmente un experimento de tiempos cuando los dos métodos son tan rápidos que el reloj no los distingue?

SOLUCIÓN Mide **ruido**. Si el tiempo de ejecución está por debajo de la resolución del reloj, las diferencias observadas provienen del sistema operativo, no de los algoritmos. Es lo que ocurre cuando **PruebaADA3** reporta casi todas las invocaciones como empate: la respuesta correcta es aumentar el tamaño de la entrada, no repetir el experimento.

3.6.55 (★★★) **A MANO** Ordene por eficiencia los cuatro métodos de ordenamiento vistos y justifique.

SOLUCIÓN *Quicksort* es $\Theta(n \ln n)$ en el caso promedio y los otros tres (burbuja, inserción y selección) son $\Theta(n^2)$. Entre estos tres la diferencia es de constante, no de orden, y depende del caso: inserción gana con listas casi ordenadas, selección hace siempre el mismo número de comparaciones, y burbuja suele ser la más lenta por la cantidad de intercambios.

3.6.56 (★★★) **A MANO** ¿Por qué *quicksort* es $\Theta(n^2)$ en el peor caso y aun así se usa en la práctica?

SOLUCIÓN El peor caso ocurre cuando el pivote deja siempre una sublista vacía, por ejemplo si la lista ya viene ordenada y se toma el primer elemento como pivote. Pero ese caso es **raro** si el pivote se escoge bien, y el caso promedio es $\Theta(n \ln n)$ con constantes pequeñas. En la práctica se elige el pivote al azar o por la mediana de tres, lo que hace el peor caso extremadamente improbable.

- 3.6.57** (★★) **A MANO** Defina con precisión qué significa *peor caso* y dé un ejemplo donde no coincida con la entrada más grande.

SOLUCIÓN El peor caso es la entrada de tamaño n que **maximiza** el esfuerzo, para cada n fijo. No tiene que ver con el tamaño sino con la *disposición* de los datos: en la búsqueda lineal, el peor caso es que el dato esté al final o no esté; en *quicksort*, que la lista venga ya ordenada. Dos listas de mil elementos pueden costar tiempos muy distintos.

- 3.6.58** (★★★) **A MANO** Un algoritmo tiene mejor caso $\Omega(n)$ y peor caso $O(n^2)$. ¿Se puede afirmar algo con Θ ?

SOLUCIÓN No sobre el algoritmo en general. Θ solo se aplica cuando las cotas superior e inferior **coinciden en orden**, y aquí difieren. Sí se puede decir que el mejor caso es $\Theta(n)$ y el peor $\Theta(n^2)$ si esas cotas son ajustadas, pero no que el algoritmo sea Θ de nada.

- 3.6.59** (★★★) **A MANO** Analice el orden de un algoritmo que, para cada uno de los n elementos de una lista, hace una búsqueda binaria en otra lista de n elementos.

SOLUCIÓN Cada búsqueda binaria cuesta $\Theta(\ln n)$ y se repite n veces, de modo que el total es $\Theta(n \ln n)$. Si en lugar de la búsqueda binaria se usara una lineal, el costo sería $\Theta(n^2)$: un mismo esquema con dos órdenes muy distintos según el subalgoritmo elegido.

- 3.6.60** (★★★) **A MANO** Un algoritmo divide el problema en tres partes de tamaño $n/3$ y las resuelve todas, con un costo lineal de reparto. Plantee su recurrencia y resuélvala.

SOLUCIÓN La recurrencia es $T(n) = 3T(n/3) + n$ con $T(1) = 0$. Iterando con $n = 3^k$:

$$T(3^k) = 3^k T(1) + k 3^k = n \log_3 n$$

Es $\Theta(n \ln n)$, el mismo orden que dividir en dos: lo que importa no es en cuántas partes se divide, sino que el costo de cada nivel sea lineal y que haya un número logarítmico de niveles.

- 3.6.61** (★★★) **A MANO** ¿Qué ocurre con el orden si el reparto del ejercicio anterior cuesta $\Theta(n^2)$ en vez de $\Theta(n)$?

SOLUCIÓN La recurrencia pasa a $T(n) = 3T(n/3) + n^2$. Ahora el costo del **primer nivel** domina a todos los demás juntos: los tres subproblemas aportan $3(n/3)^2 = n^2/3$, la tercera parte. La suma geométrica converge y el resultado es $\Theta(n^2)$. Cuando el reparto es caro, la división deja de ayudar.

- 3.6.62** (★★) **A MANO** Resuma en una tabla los tiempos relativos de los órdenes más comunes al pasar de n a $2n$.

SOLUCIÓN

orden	factor al duplicar n
$\Theta(1)$	1
$\Theta(\ln n)$	+una constante
$\Theta(n)$	2
$\Theta(n \ln n)$	algo más de 2
$\Theta(n^2)$	4
$\Theta(n^3)$	8
$\Theta(2^n)$	se eleva al cuadrado

Esta tabla permite **estimar el orden a partir de mediciones**: basta observar cómo cambia el tiempo al duplicar la entrada.

3.6.63 (★★★) **A MANO** Un algoritmo pasa de 1 a 1024 segundos al duplicar n una sola vez. ¿Qué orden tiene?

SOLUCIÓN Es exponencial. Si el tiempo se eleva al cuadrado o más al duplicar n , el orden es $\Theta(a^n)$: en efecto, $a^{2n} = (a^n)^2$. Con 2^n , pasar de $n = 10$ a $n = 20$ multiplica por 1024, exactamente lo observado.

3.3 Notación asintótica O grande

3.6.64 (★) **A MANO** Enuncie la definición de $f(n) = O(g(n))$.

SOLUCIÓN $f(n) = O(g(n))$ si existe una constante real positiva c tal que $|f(n)| \leq c|g(n)|$ para todo n natural, o para todos salvo una cantidad finita de ellos. Interpretado gráficamente: f queda por debajo de un múltiplo de g a partir de cierto punto.

3.6.65 (★) **A MANO** Pruebe directamente que $3n + 7 = O(n)$.

SOLUCIÓN Para todo $n \geq 1$ se cumple $7 \leq 7n$, luego

$$3n + 7 \leq 3n + 7n = 10n$$

Basta tomar $c = 10$ en la definición. Cualquier otra constante mayor también sirve: la definición no exige la mejor.

3.6.66 (★★) **A MANO** Pruebe que $n^2 + 3n + 2 = O(n^2)$ exhibiendo una constante.

SOLUCIÓN Para $n \geq 1$: $3n \leq 3n^2$ y $2 \leq 2n^2$, de donde

$$n^2 + 3n + 2 \leq n^2 + 3n^2 + 2n^2 = 6n^2$$

Sirve $c = 6$. Es el argumento que generaliza el teorema del polinomio: un polinomio de grado j es $O(n^j)$.

3.6.67 (★) **A MANO** Enuncie el teorema del polinomio y aplíquelo a $f(n) = 8n^5 - 2n^3 + n - 9$.

SOLUCIÓN Si $f(n) = a_j n^j + \dots + a_0$ con $a_j \neq 0$, entonces $f(n) = O(n^j)$. Aquí el grado es 5, de modo que $f(n) = O(n^5)$: solo cuenta el término de mayor grado, y ni el coeficiente ni los signos alteran el orden.

3.6.68 (★★) **MIXTO** Use la propiedad del límite para probar que $3n^2 + 5n + 7 = O(n^2)$.

SOLUCIÓN

$\ln[] :=$

```
Limit[(3 n^2 + 5 n + 7)/n^2, n -> Infinity]
```

```
Out[ ] =
```

```
3
```

El límite del cociente es 3, un número real no negativo, de modo que por la propiedad del límite $f(n) = O(n^2)$. Como además el límite es distinto de cero, también se cumple $\Theta(n^2)$.

3.6.69 (★★) **MIXTO** Pruebe que $\frac{2n^4 - n}{3n + 7} = O(n^3)$, y que n^2 no sirve como cota.

SOLUCIÓN

```
In[ ] :=
```

```
{Limit[(2 n^4 - n)/((3 n + 7) n^3), n -> Infinity],  
Limit[(2 n^4 - n)/((3 n + 7) n^2), n -> Infinity]}
```

```
Out[ ] =
```

```
{2/3, Infinity}
```

El primer límite es $\frac{2}{3}$, un real **positivo y finito**: la función es $\Theta(n^3)$ y, en particular, $O(n^3)$. El segundo es infinito, de modo que n^2 **no** es cota: la propiedad del límite sirve igual de bien para probar una cota que para descartarla. La regla práctica es restar los grados: $4 - 1 = 3$.

3.6.70 (★★) **MIXTO** Pruebe que $\ln n = O(n)$ y comente el valor del límite.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[Log[n]/n, n -> Infinity]
```

```
Out[ ] =
```

```
0
```

El límite vale 0, que también es admisible en la propiedad del límite, de modo que $\ln n = O(n)$. Un límite **cero** indica que la cota es holgada: $\ln n$ crece mucho más despacio que n , y por eso no se cumple $\Theta(n)$.

3.6.71 (★★) **MIXTO** Pruebe que $n \ln n = O(n^2)$.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[(n Log[n])/n^2, n -> Infinity]
```

```
Out[ ] =
```

```
0
```

De nuevo el límite es 0 y la conclusión es inmediata. El orden $n \ln n$ aparece en los algoritmos de ordenamiento eficientes y queda estrictamente por debajo de n^2 .

3.6.72 (★★) **MIXTO** Pruebe que $2^n = O(n!)$.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[2^n/n!, n -> Infinity]
```

```
Out[ ] =
```

```
0
```

El factorial termina imponiéndose sobre cualquier exponencial de base fija: a partir de $n = 4$ cada factor nuevo del factorial es mayor que 2. Por la propiedad del límite, $2^n = O(n!)$.

3.6.73 (★★★) **MIXTO** Pruebe que $n! = O(n^n)$ y explique el resultado sin recurrir al software.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[n!/n^n, n -> Infinity]
```

```
Out[ ] =
```

```
0
```

A mano: $n! = 1 \cdot 2 \cdots n$ tiene n factores, todos menores o iguales que n , de modo que $n! \leq n^n$ y basta $c = 1$. El límite nulo confirma además que la cota no es ajustada.

3.6.74 (★★) **MIXTO** Determine el orden de $f(n) = 1 + 2 + \cdots + n$.

SOLUCIÓN

```
In[ ] :=
```

```
Sum[i, {i, 1, n}]
```

```
Out[ ] =
```

```
(n*(1 + n))/2
```

```
In[ ] :=
```

```
Limit[Sum[i, {i, 1, n}]/n^2, n -> Infinity]
```

```
Out[ ] =
```

```
1/2
```

La suma vale $\frac{n(n+1)}{2}$, un polinomio de grado dos, y el límite $\frac{1}{2}$ confirma $f(n) = O(n^2)$, de hecho $\Theta(n^2)$.

3.6.75 (★★) **MIXTO** Determine el orden de $f(n) = 1^2 + 2^2 + \cdots + n^2$.

SOLUCIÓN

```
In[ ] :=
```

```
Sum[i^2, {i, 1, n}]
```

```
Out[ ] =
```

```
(n*(1 + n)*(1 + 2*n))/6
```

```
In[ ] :=
```

```
Limit[Sum[i^2, {i, 1, n}]/n^3, n -> Infinity]
```

Out[] =

1/3

La suma es un polinomio de grado tres, luego $f(n) = \Theta(n^3)$. En general $1^j + 2^j + \dots + n^j = \Theta(n^{j+1})$.

3.6.76 (★★) **MIXTO** Determine el orden de $f(n) = 1^3 + 2^3 + \dots + n^3$.

SOLUCIÓN

In[] :=

```
Sum[i^3, {i, 1, n}]
```

Out[] =

$(n^2*(1 + n)^2)/4$

La suma vale $\frac{n^2(n+1)^2}{4}$, de grado cuatro, de modo que $f(n) = \Theta(n^4)$, en concordancia con la regla $\Theta(n^{j+1})$ para $j = 3$.

3.6.77 (★★) **MIXTO** Determine el orden de $f(n) = 1 + 2 + 4 + \dots + 2^n$.

SOLUCIÓN

In[] :=

```
Sum[2^i, {i, 0, n}]
```

Out[] =

$-1 + 2^{(1 + n)}$

La suma geométrica vale $2^{n+1} - 1 < 2 \cdot 2^n$, así que $f(n) = O(2^n)$ con $c = 2$. La suma completa tiene el mismo orden que su último sumando: es lo característico del crecimiento exponencial.

3.6.78 (★★★) **A MANO** Demuestre a mano que $1^j + 2^j + \dots + n^j = O(n^{j+1})$ con j natural.

SOLUCIÓN Cada sumando es menor o igual que el mayor de ellos, n^j , y hay n sumandos:

$$1^j + 2^j + \dots + n^j \leq \underbrace{n^j + n^j + \dots + n^j}_{n \text{ veces}} = n \cdot n^j = n^{j+1}$$

Basta tomar $c = 1$ en la definición.

3.6.79 (★★) **A MANO** Aplique la regla de la suma para acotar $f(n) = n^3 + n \ln n$.

SOLUCIÓN Se tiene $n^3 = O(n^3)$ y $n \ln n = O(n^2)$. Por la regla de la suma,

$$f(n) = O(\text{Max}(n^3, n^2)) = O(n^3)$$

En una suma manda siempre el sumando de mayor orden.

3.6.80 (★★) **A MANO** Aplique la regla de la multiplicación a $f(n) = (n^2 + 1)(\ln n + 3)$.

SOLUCIÓN Como $n^2 + 1 = O(n^2)$ y $\ln n + 3 = O(\ln n)$, la regla del producto da

$$f(n) = O(n^2 \ln n)$$

No hay que distribuir antes de acotar: acotando cada factor el resultado sale de inmediato.

3.6.81 (★★) **A MANO** Halle una notación O grande lo más ajustada posible para $f(n) = 5^n + n^{10} + n!$.

SOLUCIÓN $O(n!)$. El factorial supera a toda exponencial de base fija y a todo polinomio; por la regla de la suma manda el sumando mayor. La jerarquía habitual es

$$1 \prec \ln n \prec n \prec n \ln n \prec n^2 \prec n^3 \prec 2^n \prec 5^n \prec n! \prec n^n$$

3.6.82 (★★) **A MANO** Halle una notación O grande ajustada para $f(n) = (n^2 + \ln n)(n + 2^n)$.

SOLUCIÓN El primer factor es $O(n^2)$ y el segundo $O(2^n)$, luego $f(n) = O(n^2 2^n)$. Distribuir daría cuatro sumandos, y el mayor de ellos es precisamente $n^2 2^n$.

3.6.83 (★★) **MIXTO** Verifique que $5n^4 - 3n^2 + 1 = O(n^4)$ con la propiedad del límite.

SOLUCIÓN

In[] :=

```
Limit[(5 n^4 - 3 n^2 + 1)/n^4, n -> Infinity]
```

Out[] =

5

El límite es 5, real y positivo, de modo que se cumple $O(n^4)$ y además $\Theta(n^4)$. Los coeficientes negativos no cambian el orden.

3.6.84 (★★) **MIXTO** Compruebe que $\ln n$ y $\ln(2n)$ tienen el mismo orden.

SOLUCIÓN

In[] :=

```
Limit[Log[n]/Log[2 n], n -> Infinity]
```

Out[] =

1

El límite vale 1, así que cada una es O grande de la otra y por lo tanto $\ln n = \Theta(\ln(2n))$. Es la misma razón por la que la base del logaritmo es irrelevante en las notaciones asintóticas.

3.6.85 (★★) **MIXTO** Determine el orden de $f(n) = 3^n + n^5$.

SOLUCIÓN

In[] :=

```
Limit[(3^n + n^5)/3^n, n -> Infinity]
```

Out[] =

1

El límite es 1, luego $f(n) = \Theta(3^n)$: el polinomio resulta despreciable frente a la exponencial por grande que sea su grado.

- 3.6.86** (★★) **VilCretas** Compare visualmente el crecimiento de las funciones de referencia del capítulo.

SOLUCIÓN

`In[ ] :=`

`CDFGraficaNOG[40]`

`CDFGraficaNOG` anima la comparación entre $\ln(\ln n)$, $\ln n$, n , $n \ln n$, n^2 , n^3 y 2^n . El argumento, entero y mayor o igual que 40, fija la variación posible sobre ambos ejes. Los algoritmos eficientes son los que quedan por debajo de la identidad $y = n$.

- 3.6.87** (★★) **VilCretas** Represente gráficamente la relación $n \ln n = O(n^2)$.

SOLUCIÓN

`In[ ] :=`

`CDFGraficaNA[{n Log[n], n^2}, 0.01, 10, 1000]`

Los argumentos son el par de funciones, el valor mínimo de la constante c (real positivo), su valor máximo (entero mayor o igual que 10) y la variación sobre los ejes. La animación permite mover c y ver a partir de qué punto la desigualdad de la definición se sostiene.

- 3.6.88** (★★) **VilCretas** Determine con `CompLimit` qué relación asintótica hay entre n^2 y n^3 .

SOLUCIÓN

`In[ ] :=`

`CompLimit[{n^2, n^3}]`

El comando responde `0 -> n^2 = O(n^3)`, notación O grande: calcula el límite del cociente, obtiene 0 y de ahí concluye. Si el límite fuera un positivo la respuesta sería Θ , y si fuera infinito, Ω .

- 3.6.89** (★★★) **A MANO** ¿Es cierto que $f(n) = O(g(n))$ implica $g(n) = O(f(n))$? Justifique.

SOLUCIÓN No. Con $f(n) = n$ y $g(n) = n^2$ se cumple $n = O(n^2)$, pero no existe constante c con $n^2 \leq cn$ para todo n grande, pues eso obligaría a $n \leq c$. La relación O grande **no es simétrica**; cuando ambas se cumplen a la vez se tiene Θ .

- 3.6.90** (★★★) **A MANO** Demuestre la propiedad transitiva: si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $f(n) = O(h(n))$.

SOLUCIÓN Por hipótesis existen $c_1, c_2 > 0$ con $|f(n)| \leq c_1 |g(n)|$ y $|g(n)| \leq c_2 |h(n)|$ salvo para finitos n . Combinando,

$$|f(n)| \leq c_1 c_2 |h(n)|$$

y la constante $c = c_1 c_2$ prueba la afirmación.

- 3.6.91** (★★) **A MANO** Justifique que toda constante c es $O(1)$ y dé un ejemplo algorítmico.

SOLUCIÓN Con $g(n) = 1$ se cumple $|c| \leq |c| \cdot 1$ para todo n , de modo que $c = O(1)$. Un algoritmo $O(1)$ hace un trabajo que no depende del tamaño de la entrada: por ejemplo devolver el primer elemento de una lista.

- 3.6.92** (★★★) **MIXTO** Un algoritmo realiza $\binom{n}{2}$ comparaciones. Halle su orden.

SOLUCIÓN**In[] :=**`Limit[Binomial[n, 2]/n^2, n -> Infinity]`**Out[] =**

1/2

Como $\binom{n}{2} = \frac{n(n-1)}{2}$, el límite $\frac{1}{2}$ es real y positivo, de donde el algoritmo es $\Theta(n^2)$. Es el conteo de todos los pares posibles de una lista de n datos.

3.6.93 (★★) **MIXTO** Compruebe el conteo anterior para $n = 10$.**SOLUCIÓN****In[] :=**`Binomial[10, 2]`**Out[] =**

45

Coincide con $\frac{10 \cdot 9}{2} = 45$: con diez datos hay cuarenta y cinco parejas por comparar.

3.6.94 (★★★) **A MANO** Un algoritmo divide el problema en dos mitades y trabaja n pasos en cada nivel. ¿Cuántos niveles hay y cuál es su orden total?

SOLUCIÓN Partiendo de n y dividiendo entre dos, el proceso llega a subproblemas de tamaño uno tras $\log_2 n$ divisiones, de modo que hay $\lfloor \log_2 n \rfloor + 1$ niveles. Con n pasos en cada nivel el esfuerzo total es $f(n) = n(\log_2 n + 1)$, es decir $\Theta(n \ln n)$: el orden de los métodos de ordenamiento eficientes.

3.6.95 (★★) **MIXTO** Estime cuántas divisiones sucesivas entre dos hacen falta para reducir un millón de datos a uno solo.**SOLUCIÓN****In[] :=**`N[Log[2, 1000000]]`**Out[] =**

19.931568569324174

Bastan veinte divisiones. Esa es la ventaja práctica del orden logarítmico: multiplicar por mil el tamaño de la entrada agrega apenas unos diez pasos.

3.6.96 (★★★) **A MANO** Cuente las asignaciones del siguiente fragmento y dé su orden.`For[i = 1, i <= n, For[j = i, j <= n, p = p + 1; j++]; i++]`

SOLUCIÓN El ciclo interno arranca en $j = i$, de modo que hace $n - i + 1$ pasos. El total es

$$\sum_{i=1}^n (n - i + 1) = n + (n - 1) + \cdots + 1 = \frac{n(n + 1)}{2}$$

Es $\Theta(n^2)$, igual que si el ciclo interno recorriera siempre de 1 a n : la mitad del trabajo no cambia el orden, solo la constante.

3.6.97 (★★★) **A MANO** Cuentas las asignaciones cuando el ciclo interno recorre de 1 a i^2 .

```
For[i = 1, i <= n, For[j = 1, j <= i^2, p = p + 1; j++]; i++]
```

SOLUCIÓN El total es $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$, un polinomio de grado tres: $\Theta(n^3)$. Basta con que el límite interno crezca como i^2 para que el orden suba un grado respecto del caso lineal.

3.6.98 (★★) **MIXTO** Verifique la suma del ejercicio anterior con el software.

SOLUCIÓN

In[] :=

```
{Sum[i^2, {i, 1, n}], Limit[Sum[i^2, {i, 1, n}]/n^3, n -> Infinity]}
```

Out[] =

```
{(n*(1 + n)*(1 + 2*n))/6, 1/3}
```

El límite $\frac{1}{3}$, real y positivo, confirma $\Theta(n^3)$.

3.6.99 (★★★) **A MANO** Un ciclo triple anidado recorre los tres índices de 1 a n pero solo actúa cuando $i < j < k$. ¿Cuántas veces actúa y de qué orden es?

SOLUCIÓN Actúa una vez por cada terna estrictamente creciente, es decir $\binom{n}{3} = \frac{n(n-1)(n-2)}{6}$ veces. Es $\Theta(n^3)$: la condición divide el trabajo entre seis, pero el orden no cambia. Las constantes importan en la práctica y son irrelevantes para la notación asintótica.

3.6.100 (★★) **MIXTO** Determine el orden de $f(n) = \frac{2n^3 + 5n}{n^3 + 1}$.

SOLUCIÓN

In[] :=

```
Limit[(2 n^3 + 5 n)/(n^3 + 1), n -> Infinity]
```

Out[] =

2

El límite es 2, real y positivo: $f(n) = \Theta(1)$. Aunque la expresión tenga grado tres arriba y abajo, la función está **acotada**: no crece.

3.6.101 (★★) **MIXTO** Determine el orden de $f(n) = \frac{n^2 + 1}{n^3 + n}$.

SOLUCIÓN

In[] :=

```
Limit[(n^2 + 1)/(n^3 + n), n -> Infinity]
```

Out[] =

0

El límite es cero, de modo que $f(n) = O(1)$ pero no $\Theta(1)$: la función **tiende a cero**. De hecho es $\Theta(\frac{1}{n})$.

3.6.102 (★★★) **MIXTO** Pruebe que $(\ln n)^3 = O(n)$ y comente qué dice eso de los logaritmos.

SOLUCIÓN

In[] :=

Limit[Log[n]^3/n, n -> Infinity]

Out[] =

0

Cualquier potencia fija del logaritmo crece más despacio que n . Es la jerarquía $(\ln n)^k \prec n^\varepsilon$ para todo k y todo $\varepsilon > 0$: el logaritmo es **más lento que cualquier potencia**, por pequeña que sea.

3.6.103 (★★★) **MIXTO** Determine el orden de $f(n) = \sqrt{n^2 + n}$.

SOLUCIÓN

In[] :=

Limit[Sqrt[n^2 + n]/n, n -> Infinity]

Out[] =

1

El límite es 1, de modo que $f(n) = \Theta(n)$. Dentro de una raíz, el sumando de menor grado es irrelevante: $\sqrt{n^2 + n} \approx n + \frac{1}{2}$.

3.6.104 (★★) **MIXTO** Determine el orden de $2^n + 3^n$.

SOLUCIÓN

In[] :=

Limit[(2^n + 3^n)/3^n, n -> Infinity]

Out[] =

1

Es $\Theta(3^n)$. Entre dos exponenciales manda la de **base mayor**, y la otra resulta despreciable: $(\frac{2}{3})^n \rightarrow 0$.

3.6.105 (★★★) **MIXTO** Compare $\ln(n^5)$ con $\ln n$.

SOLUCIÓN

In[] :=

Limit[Log[n^5]/Log[n], n -> Infinity]

Out[] =

5

El límite es 5, de modo que $\ln(n^5) = \Theta(\ln n)$. Elevar a una potencia **dentro** del logaritmo solo multiplica por una constante; elevar el logaritmo entero, en cambio, sí cambia el orden.

3.6.106 (★★★) **MIXTO** Determine el orden de $\ln(\ln n)$ frente a $\ln n$.

SOLUCIÓN

In[] :=

```
Limit[Log[Log[n]]/Log[n], n -> Infinity]
```

Out[] =

0

Es $O(\ln n)$ y no Θ : el logaritmo iterado crece aún más despacio. Para $n = 10^6$, $\ln n \approx 14$ mientras que $\ln(\ln n) \approx 2,6$. Es la función más lenta que aparece en la jerarquía del capítulo.

3.6.107 (★★★) **MIXTO** Compare $n^2 2^n$ con 3^n .

SOLUCIÓN

In[] :=

```
Limit[(n^2 2^n)/3^n, n -> Infinity]
```

Out[] =

0

El límite es cero: $n^2 2^n = O(3^n)$ y no al revés. Un factor polinomial **no compensa** una base exponencial mayor, por grande que sea el exponente del polinomio.

3.6.108 (★★★) **MIXTO** Determine el orden de $\sum_{i=1}^n i^4$.

SOLUCIÓN

In[] :=

```
Sum[i^4, {i, 1, n}]
```

Out[] =

```
(n*(1 + n)*(1 + 2*n)*(-1 + 3*n + 3*n^2))/30
```

In[] :=

```
Limit[Sum[i^4, {i, 1, n}]/n^5, n -> Infinity]
```

Out[] =

1/5

Es $\Theta(n^5)$, coherente con la regla $1^j + \dots + n^j = \Theta(n^{j+1})$ con $j = 4$.

3.6.109 (★★★) **MIXTO** Determine el orden de $\sum_{i=1}^n \frac{1}{2^i}$.

SOLUCIÓN

In[] :=

```
Sum[1/2^i, {i, 1, n}]
```

```
Out[ ] =
```

```
(-1 + 2^n)/2^n
```

La suma vale $1 - 2^{-n}$, que **tiende a 1**. Por lo tanto es $\Theta(1)$: una serie geométrica de razón menor que uno está acotada por más términos que se sumen. Es lo contrario de la serie armónica.

3.6.110 (★★★) MIXTO Determine el orden de $\sum_{i=1}^n i 2^i$.

SOLUCIÓN

```
In[ ] :=
```

```
Sum[i*2^i, {i, 1, n}]
```

```
Out[ ] =
```

```
2 + 2^(1 + n)*(-1 + n)
```

```
In[ ] :=
```

```
Limit[Sum[i 2^i, {i, 1, n}]/(n 2^n), n -> Infinity]
```

```
Out[ ] =
```

```
2
```

Es $\Theta(n2^n)$: la suma tiene el mismo orden que su último término, multiplicado por una constante. Con sumandos que crecen exponencialmente, el último domina a todos los anteriores juntos.

3.6.111 (★★★) MIXTO Determine el orden de $\sum_{i=1}^n \frac{1}{i(i+2)}$.

SOLUCIÓN

```
In[ ] :=
```

```
Sum[1/(i (i + 2)), {i, 1, n}]
```

```
Out[ ] =
```

```
3/4 - (3 + 2*n)/(2*(1 + n)*(2 + n))
```

La suma tiende a $\frac{3}{4}$, de modo que es $\Theta(1)$. Es otra suma telescópica: $\frac{1}{i(i+2)} = \frac{1}{2} \left(\frac{1}{i} - \frac{1}{i+2} \right)$.

3.6.112 (★★) MIXTO Determine el orden de $\prod_{i=1}^n 3i$.

SOLUCIÓN

```
In[ ] :=
```

```
Product[3 i, {i, 1, n}]
```

```
Out[ ] =
```

$3^{n \cdot \Gamma[1 + n]}$

Vale $3^n n!$, de modo que es $\Omega(n!)$ pero no $O(n!)$: el factor 3^n crece sin límite. Para tener Θ habría que comparar con $n! 3^n$.

3.6.113 (★★★) **MIXTO** Determine el orden de $\prod_{i=1}^n i^2$.

SOLUCIÓN

`In[ ] :=`

`Product[i^2, {i, 1, n}]`

`Out[ ] =`

$n!^2$

Es $(n!)^2$, es decir $\Theta((n!)^2)$. Elevar cada factor al cuadrado eleva el producto entero al cuadrado, que es un salto muchísimo mayor que multiplicar por una exponencial.

3.6.114 (★★) **MIXTO** Determine el orden de $\prod_{i=1}^n \frac{i+1}{i}$.

SOLUCIÓN

`In[ ] :=`

`Product[(i + 1)/i, {i, 1, n}]`

`Out[ ] =`

$1 + n$

El producto telescópico vale $n + 1$: es $\Theta(n)$. Un producto de n factores no tiene por qué crecer rápido si cada factor tiende a uno.

3.6.115 (★★★) **MIXTO** Conjeture el valor de j para el cual $\prod_{i=1}^n 5i = \Theta(n! j^n)$.

SOLUCIÓN

`In[ ] :=`

`Limit[Product[5 i, {i, 1, n}]/(n! 5^n), n -> Infinity]`

`Out[ ] =`

1

El valor es $j = 5$. El producto vale $5^n n!$, de modo que coincide exactamente con $n! 5^n$. Con cualquier otro j el cociente tendería a cero o a infinito.

3.6.116 (★★★) **A MANO** Ordene de menor a mayor orden asintótico: $n \ln n$, $(\ln n)^2$, $n^{0,5}$, $2^{\ln n}$, $n!$.

SOLUCIÓN Obsérvese que $2^{\ln n} = n^{\ln 2} \approx n^{0,69}$. La jerarquía es

$$(\ln n)^2 \prec n^{0,5} \prec 2^{\ln n} \prec n \ln n \prec n!$$

El paso clave es reescribir $2^{\ln n}$ como potencia de n : presentado como exponencial parece que debería ganarle a los polinomios, y en realidad es uno de ellos.

3.6.117 (★★★) **MIXTO** Compare $n^{1/n}$ con la constante 1.

SOLUCIÓN

In[] :=

`Limit[n^(1/n), n -> Infinity]`

Out[] =

1

El límite es 1, de modo que $n^{1/n} = \Theta(1)$. La raíz n -ésima aplasta cualquier crecimiento polinomial: aunque n crezca sin límite, su raíz n -ésima tiende a uno.

3.6.118 (★★★) **MIXTO** Determine el orden de $\binom{2n}{n}$ frente a 4^n .

SOLUCIÓN

In[] :=

`Limit[Binomial[2 n, n]/4^n, n -> Infinity]`

Out[] =

0

El límite es cero: $\binom{2n}{n} = O(4^n)$ y no Θ . La fórmula de *Stirling* da $\binom{2n}{n} \approx \frac{4^n}{\sqrt{\pi n}}$: falta justamente un factor $\sqrt{n}$ en el denominador.

3.6.119 (★★★) **A MANO** Demuestre que si $f(n) = O(g(n))$ entonces $f(n) + g(n) = \Theta(g(n))$, suponiendo ambas positivas.

SOLUCIÓN Por hipótesis $f(n) \leq cg(n)$ para n grande, de donde

$$g(n) \leq f(n) + g(n) \leq (c+1)g(n)$$

La cota inferior da $\Omega(g(n))$ y la superior $O(g(n))$, y juntas $\Theta(g(n))$. Es la justificación formal de que en una suma **manda el sumando mayor**.

3.6.120 (★★★) **A MANO** ¿Es cierto que $f(n) = O(g(n))$ implica $2^{f(n)} = O(2^{g(n)})$?

SOLUCIÓN No. Con $f(n) = 2n$ y $g(n) = n$ se tiene $f = O(g)$, pero $2^{2n} = 4^n$ no es $O(2^n)$, porque el cociente 2^n tiende a infinito. El problema es que la constante multiplicativa oculta en la O se convierte en un **exponente** al pasar a la potencia.

3.6.121 (★★★) **A MANO** ¿Y es cierto que $f(n) = O(g(n))$ implica $\ln f(n) = O(\ln g(n))$?

SOLUCIÓN Sí, si ambas tienden a infinito. De $f(n) \leq cg(n)$ sale $\ln f(n) \leq \ln c + \ln g(n)$, y como $\ln g(n) \rightarrow \infty$, la constante aditiva es despreciable. El logaritmo **aplata** las constantes multiplicativas, al revés que la exponencial, que las amplifica.

3.6.122 (★★) **MIXTO** Evalúe `CalculaOtroProducto[9]` y deduzca su comportamiento.

SOLUCIÓN

In[] :=

CalculaOtroProducto[9]

Out[] =

27

El resultado es $27 = 3^3$: el valor solo cambia al cruzar una potencia de tres. Como el ciclo interno multiplica por tres hasta superar a i , el número total de multiplicaciones es del orden de $\log_3 n$ por cada vuelta del ciclo externo, lo que da $\Theta(n \ln n)$.

3.6.123 (★★★) **MIXTO** Tabule CalculaSuma de 10 a 14 y confirme la fórmula $2n^2$.

SOLUCIÓN

In[] :=

Table[CalculaSuma[k], {k, 10, 14}]

Out[] =

{200, 242, 288, 338, 392}

Son $2 \cdot 10^2, 2 \cdot 11^2, \dots, 2 \cdot 14^2$. La diferencia entre valores consecutivos crece linealmente, que es la firma de un polinomio de grado dos.

3.6.124 (★★★) **A MANO** Un algoritmo tarda 3 segundos con $n = 1000$ y 12 segundos con $n = 2000$. ¿Cuál es su orden probable y cuánto tardará con $n = 8000$?

SOLUCIÓN Al duplicar n el tiempo se cuadruplicó, lo que sugiere $\Theta(n^2)$. De 1000 a 8000 hay un factor 8, de modo que el tiempo se multiplica por $8^2 = 64$: unos $3 \cdot 64 = 192$ segundos. La estimación es asintótica y vale como orden de magnitud, no como medida exacta.

3.6.125 (★★★) **A MANO** Otro algoritmo tarda 2 segundos con $n = 1000$ y 2,2 segundos con $n = 2000$. ¿Qué orden sugiere?

SOLUCIÓN Sugiere $\Theta(\ln n)$ o $\Theta(n \ln n)$ con constante pequeña: duplicar n apenas añadió un 10%. Si fuera lineal el tiempo habría subido a 4 segundos, y si fuera cuadrático a 8. Un incremento aditivo constante al duplicar n es la firma inconfundible del **logaritmo**.

3.4 Otras notaciones asintóticas

3.6.126 (★) **A MANO** Enuncie la definición de $f(n) = \Omega(g(n))$ y la de $f(n) = \Theta(g(n))$.

SOLUCIÓN $f(n) = \Omega(g(n))$ si existe una constante real positiva c con $|f(n)| \geq c|g(n)|$ para todo n natural, salvo una cantidad finita. Si además $f(n) = O(g(n))$, se escribe $f(n) = \Theta(g(n))$: g acota a f por arriba y por abajo.

3.6.127 (★) **A MANO** ¿Qué caso de ejecución describe cada notación?

SOLUCIÓN O grande se asocia al **peor caso**, pues da una cota superior del esfuerzo; Ω al **mejor caso**, al acotar por debajo; y Θ se usa con fines interpretativos sobre el **caso promedio**, cuando ambas cotas tienen el mismo orden.

3.6.128 (★★) **A MANO** Pruebe que $3n^2 + 1 = \Omega(n^2)$.

SOLUCIÓN Para todo n natural se cumple $3n^2 + 1 \geq 3n^2 \geq 1 \cdot n^2$, de modo que sirve $c = 1$ (o $c = 3$). Junto con $3n^2 + 1 = O(n^2)$ se concluye $\Theta(n^2)$.

3.6.129 (★★) **A MANO** Pruebe que $a_j n^j + \dots + a_0 = \Omega(n^j)$ cuando $a_j > 0$.

SOLUCIÓN El cociente $\frac{a_j n^j + \cdots + a_0}{n^j}$ tiende a $a_j > 0$ cuando $n \rightarrow +\infty$, es decir, no tiende a cero. Por la propiedad del límite en su versión Ω , el polinomio es $\Omega(n^j)$, y junto con el teorema del polinomio resulta $\Theta(n^j)$.

3.6.130 (★★) **MIXTO** Demuestre que $1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\ln n)$.

SOLUCIÓN

`In[ ] :=`

`Sum[1/i, {i, 1, n}]`

`Out[ ] =`

`HarmonicNumber[n]`

`In[ ] :=`

`Limit[Sum[1/i, {i, 1, n}]/Log[n], n -> Infinity]`

`Out[ ] =`

1

El límite del cociente es 1: real, positivo y finito. Eso da a la vez $O(\ln n)$ y $\Omega(\ln n)$, es decir $\Theta(\ln n)$. La suma armónica crece, pero tan despacio como el logaritmo.

3.6.131 (★★) **MIXTO** Determine qué notación relaciona $f(n) = \frac{2}{3}n^3 + (\ln n)^5$ con $g(n) = n^3$.

SOLUCIÓN

`In[ ] :=`

`Limit[(2/3 n^3 + Log[n]^5)/n^3, n -> Infinity]`

`Out[ ] =`

2/3

El límite es real y positivo, de modo que $f(n) = \Theta(n^3)$: la potencia quinta del logaritmo no alcanza a competir con n^3 .

3.6.132 (★★★) **A MANO** Para $f(n) = \frac{2}{3}n^3 + (\ln n)^5$ y $g(n) = n^j$, conjeture para cuáles j naturales se cumple O , Ω y Θ .

SOLUCIÓN El orden de f es n^3 , así que se compara n^3 con n^j :

$$1 \leq j \leq 2 \Rightarrow f(n) = \Omega(g(n)), \quad j = 3 \Rightarrow f(n) = \Theta(g(n)), \quad j \geq 4 \Rightarrow f(n) = O(g(n))$$

La conjetura se apoya en que $\lim n^3/n^j$ vale $+\infty$, 1 o 0 según j sea menor, igual o mayor que 3.

3.6.133 (★★) **MIXTO** Estudie la relación entre $f(n) = \frac{n^2 + 3 \ln n}{n^5 + 1}$ y $g(n) = n^2$.

SOLUCIÓN

`In[ ] :=`

```
Limit[(n^2 + 3 Log[n])/(n^5 + 1)/n^2, n -> Infinity]
```

```
Out[ ] =
```

```
0
```

El límite es 0, luego $f(n) = O(n^2)$ pero no $\Omega(n^2)$ y por lo tanto tampoco $\Theta(n^2)$. La función tiende a cero: el denominador es de grado muy superior.

3.6.134 (★★) **A MANO** ¿Puede ocurrir que $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$ a la vez? ¿Y que no ocurra ninguna de las dos?

SOLUCIÓN Sí puede ocurrir: es exactamente la definición de Θ . Y también puede no ocurrir ninguna, si el cociente $f(n)/g(n)$ oscila sin tender a un límite; por ejemplo $f(n) = n^{1+(-1)^n}$ frente a $g(n) = n$.

3.6.135 (★★) **MIXTO** Determine la relación asintótica entre $\sqrt{n}$ y $\ln n$.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[Sqrt[n]/Log[n], n -> Infinity]
```

```
Out[ ] =
```

```
Infinity
```

El límite es infinito, de modo que $\sqrt{n} = \Omega(\ln n)$ y no se cumple $O(\ln n)$. Toda potencia de n , por pequeño que sea el exponente, termina superando al logaritmo.

3.6.136 (★★) **MIXTO** Determine la relación entre $f(n) = n + \ln n$ y $g(n) = n$.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[(n + Log[n])/n, n -> Infinity]
```

```
Out[ ] =
```

```
1
```

El límite es 1, luego $f(n) = \Theta(n)$. Sumar un término de orden inferior no altera la notación asintótica.

3.6.137 (★★★) **A MANO** Demuestre que $1^j + 2^j + \dots + n^j = \Omega(n^{j+1})$ con j natural.

SOLUCIÓN Se descartan casi la mitad de los sumandos y se acota cada uno de los restantes por el menor de ellos, $\left\lceil \frac{n}{2} \right\rceil^j$:

$$1^j + 2^j + \dots + n^j \geq \left\lceil \frac{n}{2} \right\rceil^j + \dots + n^j \geq \frac{n}{2} \cdot \left(\frac{n}{2}\right)^j = \frac{n^{j+1}}{2^{j+1}}$$

Basta $c = \frac{1}{2^{j+1}}$. Junto con la cota superior ya probada, resulta $\Theta(n^{j+1})$.

3.6.138 (★★) **VilCretas** Represente gráficamente una relación asintótica que dependa de un parámetro.

SOLUCIÓN

`In[ ] :=`

```
CDFGraficaNAP[{Sum[n^i, {i, 0, j}], n^j}, 0.01, 10, 1000, 10]
```

CDFGraficaNAP añade a CDFGraficaNA un último argumento: el valor máximo del parámetro j en la animación. Aquí muestra, para cada j , que el polinomio $1 + n + \dots + n^j$ se sitúa por encima de n^j , que es la lectura gráfica de $\Omega(n^j)$. No admite funciones racionales.

3.6.139 (★★) VilCretas Use `CompLimit` sobre un par de funciones con parámetro.

SOLUCIÓN

`In[ ] :=`

```
CompLimit[{n^j, n^(j + 1)}, jvalor -> 20]
```

Cuando alguna de las funciones incluye el parámetro j , el comando genera una animación que lo hace variar; `jvalor` cambia el tope, que por defecto es mil y debe ser al menos diez. Para cada valor de j la respuesta es la misma: notación O grande.

3.6.140 (★★★) A MANO Ordene de menor a mayor orden asintótico: $n!$, n^2 , $\ln n$, 2^n , $n \ln n$, n^n , n .

SOLUCIÓN

$$\ln n \prec n \prec n \ln n \prec n^2 \prec 2^n \prec n! \prec n^n$$

El criterio para justificar cada paso es el mismo: el límite del cociente entre la función menor y la mayor tiende a cero.

3.6.141 (★★★) A MANO Un algoritmo es $\Theta(n^2)$. Si al duplicar los datos de entrada tarda 1 segundo con $n = 1000$, estime el tiempo para $n = 4000$.

SOLUCIÓN El esfuerzo crece con el cuadrado: al pasar de 1000 a 4000 el factor es $\left(\frac{4000}{1000}\right)^2 = 16$, de modo que se esperan unos 16 segundos. La estimación es asintótica: vale como orden de magnitud, no como medida exacta.

3.5 Experimentos sobre datos propios

3.6.142 (★★) VilCretas Compare dos programas que suman los dígitos de un entero, sobre una lista de valores elegida por usted, con `PruebaADA2Valores`.

SOLUCIÓN

`In[ ] :=`

```
P1[n_Integer] := Total[IntegerDigits[n]];
P2[n_Integer] := Module[{s = 0, m = n, r},
  While[m > 0, {m, r} = QuotientRemainder[m, 10]; s += r]; s];
enteros = RandomInteger[{1, 1000000}, 2000];
PruebaADA2Valores[{P1, P2}, enteros]
```

La diferencia con `PruebaADA2` está en el segundo argumento: aquella genera los datos por su cuenta, y esta recibe **la lista concreta** sobre la que se quiere medir. Eso permite comparar los algoritmos en el escenario que interesa y no en uno genérico.

3.6.143 (★★★) A MANO ¿Por qué el resultado de ese experimento cambia de una ejecución a otra, aunque los dos programas sean los mismos?

SOLUCIÓN Porque no mide operaciones, mide **tiempo**, y el tiempo depende de la máquina: memoria caché, otros procesos, recolección de basura. Es un enfoque **experimental**, no una demostración. Sirve para detectar diferencias grandes de orden; no para separar dos algoritmos del mismo orden, donde el resultado oscila.

3.6.144 (★★★) **VilCretas** Añada un tercer programa y use `PruebaADA3Valores`.

SOLUCIÓN

`In[ ] :=`

```
P1[n_Integer] := Total[IntegerDigits[n]];
P2[n_Integer] := Module[{s = 0, m = n, r},
  While[m > 0, {m, r} = QuotientRemainder[m, 10]; s += r]; s];
P3[n_Integer] := Fold[Plus, 0, IntegerDigits[n]];
enteros = RandomInteger[{1, 1000000}, 2000];
PruebaADA3Valores[{P1, P2, P3}, enteros]
```

El comando informa cuántas veces ganó cada método. Como los tres recorren los dígitos una sola vez, los tres son $\Theta(\log n)$ y el reparto tiende a ser parejo: el experimento **confirma** el análisis teórico en lugar de contradecirlo.

3.6.145 (★★★) **VilCretas** Vea gráficamente la comparación con `PruebaADAGraficaValores`, ahora con cuatro métodos.

SOLUCIÓN

`In[ ] :=`

```
P4[n_Integer] := If[n < 10, n, Mod[n, 10] + P4[Quotient[n, 10]]];
PruebaADAGraficaValores[{P1, P2, P3, P4}, enteros]
```

Este comando admite **dos o más** métodos (no solo dos o tres) y en lugar de contar victorias dibuja los tiempos. La gráfica dice algo que el conteo esconde: si un método pierde siempre por muy poco, o si pierde por mucho de vez en cuando.

3.6.146 (★★★) **MIXTO** Diseñe el experimento para que sí se vea una diferencia de orden: compare la suma de dígitos con un programa que ordene la lista de dígitos antes de sumarlos.

SOLUCIÓN

`In[ ] :=`

```
P5[n_Integer] := Total[Sort[IntegerDigits[n]]];
PruebaADAGraficaValores[{P1, P5}, enteros]
```

Ordenar añade un factor log sobre el número de dígitos, que es a su vez logarítmico en n . La diferencia existe pero es **pequeña**: para verla hace falta que la lista de valores tenga enteros grandes. Ahí está el valor de poder **elegir los datos**.

3.6.147 (★★★) **A MANO** Un compañero concluye, de un experimento con estos comandos, que su algoritmo es $O(n)$. ¿Qué objeción le hace?

SOLUCIÓN Que un experimento **no demuestra** una cota asintótica. Mide un rango finito de entradas en una máquina concreta; la definición de $O(n)$ habla de **todo** n mayor que un umbral. El experimento puede *sugerir* el orden y, sobre todo, *refutarlo* si los tiempos crecen más deprisa de lo previsto; para afirmarlo hace falta contar operaciones.

CAPÍTULO 4 • COMPLEMENTARIOS

Relaciones binarias

4.1 Producto cartesiano y relación binaria

4.6.1 (★) **A MANO** Calcule $A \times B$ si $A = \{1, 2, 3\}$ y $B = \{4, 5\}$.

SOLUCIÓN Se forman todos los pares tomando primero un elemento de A y luego uno de B :

$$A \times B = \{(1, 4), (1, 5), (2, 4), (2, 5), (3, 4), (3, 5)\}$$

Son $3 \cdot 2 = 6$ pares, de acuerdo con $|A \times B| = |A| \cdot |B|$.

4.6.2 (★) **VilCretas** Verifique el producto cartesiano anterior con PC.

SOLUCIÓN

`In[ ] :=`

`PC[{1, 2, 3}, {4, 5}]`

`Out[ ] =`

`{{1, 4}, {1, 5}, {2, 4}, {2, 5}, {3, 4}, {3, 5}}`

El comando devuelve los pares como listas de dos elementos, en el mismo orden en que se construyen a mano.

4.6.3 (★) **MIXTO** Compruebe que el producto cartesiano no es conmutativo.

SOLUCIÓN

`In[ ] :=`

`PC[{x, y}, {1, 2, 3}]`

`Out[ ] =`

`{{x, 1}, {x, 2}, {x, 3}, {y, 1}, {y, 2}, {y, 3}}`

El par $(x, 1)$ pertenece a $A \times B$, pero $(1, x)$ no: los elementos de $B \times A$ tienen el número en la primera coordenada. Los dos conjuntos tienen el mismo cardinal y elementos distintos.

4.6.4 (★) **MIXTO** Determine el cardinal de $A \times B$ con $|A| = 4$ y $|B| = 3$.

SOLUCIÓN

`In[ ] :=`

`Length[PC[{1, 2, 3, 4}, {1, 2, 3}]]`

`Out[ ] =`

12

Por la fórmula del cardinal, $4 \cdot 3 = 12$.

4.6.5 (★★) **A MANO** ¿Cuántas relaciones binarias distintas se pueden definir de $A = \{1, 2\}$ en $B = \{1, 2\}$?

SOLUCIÓN Una relación binaria es cualquier subconjunto de $A \times B$, que tiene $2 \cdot 2 = 4$ pares. El número de subconjuntos de un conjunto de cuatro elementos es $2^4 = 16$, incluyendo la relación vacía y el propio producto cartesiano.

4.6.6 (★) **VilCretas** Confirme el conteo anterior con *Mathematica*.

SOLUCIÓN

In[] :=

```
2^Length[PC[{1, 2}, {1, 2}]]
```

Out[] =

16

En general, sobre conjuntos finitos hay $2^{|A| \cdot |B|}$ relaciones binarias de A en B .

4.6.7 (★) **A MANO** Escriba de forma explícita la relación $R = \{(a, b) \mid a < b\}$ sobre $A = \{1, 2, 3\}$ y $B = \{2, 3, 4\}$.

SOLUCIÓN Se recorre $A \times B$ y se conservan los pares que cumplen la condición:

$$R = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$$

4.6.8 (★) **VilCretas** Obtenga la relación anterior con **RelBin**.

SOLUCIÓN

In[] :=

```
RelBin["a < b", {1, 2, 3}, {2, 3, 4}]
```

Out[] =

```
{ {1, 2}, {1, 3}, {1, 4}, {2, 3}, {2, 4}, {3, 4} }
```

La condición viaja como una cadena de texto y se escribe en función de **a** y **b**, las coordenadas del par ordenado (a, b) .

4.6.9 (★★) **VilCretas** Determine la relación a divide a b sobre $A = \{1, 2, 3\}$ y $B = \{2, 4, 6\}$.

SOLUCIÓN

In[] :=

```
RelBin["Mod[b, a] == 0", {1, 2, 3}, {2, 4, 6}]
```

Out[] =

```
{ {1, 2}, {1, 4}, {1, 6}, {2, 2}, {2, 4}, {2, 6}, {3, 6} }
```

La divisibilidad se traduce como residuo nulo. El 1 divide a todos y el 3 solo a 6.

4.6.10 (★) **VilCretas** Obtenga la relación identidad sobre $A = \{1, 2, 3\}$.

SOLUCIÓN

In[] :=

```
RelBin["a == b", {1, 2, 3}, {1, 2, 3}]
```

Out[] =

$\{\{1, 1\}, \{2, 2\}, \{3, 3\}\}$

Es la **relación identidad**, formada solo por la diagonal. Cualquier relación reflexiva sobre A la contiene.

4.6.11 (★) **VilCretas** Decida si $(1, 2)$ y $(2, 1)$ pertenecen a $R = \{(1, 2), (2, 3)\}$.

SOLUCIÓN

In[] :=

```
ElementRelBinQ[{{1, 2}, {2, 3}}, {1, 2}]
```

```
ElementRelBinQ[{{1, 2}, {2, 3}}, {2, 1}]
```

Out[] =

True

False

El orden importa: $(1, 2)$ y $(2, 1)$ son pares ordenados distintos.

4.6.12 (★★) **VilCretas** Use `ElementRelBinQ` cuando la relación se da por una desigualdad y no por sus pares.

SOLUCIÓN

In[] :=

```
ElementRelBinQ["a < b", {2, 5}, expalgebra -> True]
```

Out[] =

True

Con `expalgebra->True` el primer argumento es una cadena con la condición y no una lista de pares. Así se puede consultar una relación infinita, en la que enumerar los pares sería imposible.

4.6.13 (★★) **A MANO** Determine el dominio y el ámbito de $R = \{(1, 2), (1, 3), (2, 3)\}$ definida de $A = \{1, 2, 3\}$ en $B = \{1, 2, 3\}$.

SOLUCIÓN El **dominio** es el conjunto de las primeras coordenadas, $\{1, 2\}$, y el **ámbito** el de las segundas, $\{2, 3\}$. Ni el dominio tiene que ser todo A ni el ámbito todo B : eso distingue una relación cualquiera de una función.

4.6.14 (★★) **A MANO** ¿Toda función es una relación binaria? ¿Y al revés?

SOLUCIÓN Toda función de A en B es una relación binaria, pues es un subconjunto de $A \times B$. Lo contrario no vale: en una función cada elemento del dominio aparece **una sola vez** como primera coordenada, y una relación puede repetirlo, como $\{(1, 2), (1, 3)\}$.

4.6.15 (★) **VilCretas** Represente gráficamente el producto cruz $[0, 2] \times [1, 3]$.

SOLUCIÓN

In[] :=

```
GraficaPC[{0, 2}, {1, 3}]
```

`GraficaPC` dibuja el producto cruz de dos intervalos reales: un rectángulo. Si algún extremo es infinito el comando grafica hasta 100 o -100 según corresponda.

4.6.16 (★★) VilCretas Represente gráficamente la relación real $a = b$.

SOLUCIÓN

`In[ ] :=`

```
GraficaRelBin["a == b", 5, 5]
```

Los argumentos son la condición como cadena y los extremos máximos sobre cada eje; el mínimo es -10 por defecto y se cambia con `xmin` e `ymin`. La gráfica es la recta $y = x$, que es la versión continua de la relación identidad.

4.6.17 (★★) VilCretas Represente los pares de $R = \{(1, 2), (2, 3)\}$ en el plano cartesiano.

SOLUCIÓN

`In[ ] :=`

```
GraficaRelBinPares[{{1, 2}, {2, 3}}]
```

A diferencia de `GraficaRelBin`, que parte de una condición, `GraficaRelBinPares` recibe la relación ya escrita como pares y dibuja los puntos.

4.2 Representaciones de una relación binaria

4.6.18 (★) A MANO Escriba la matriz de $R = \{(1, 2), (2, 2), (3, 1)\}$ de $A = \{1, 2, 3\}$ en $B = \{1, 2\}$.

SOLUCIÓN La entrada m_{ij} vale 1 si $(a_i, b_j) \in R$ y 0 en caso contrario:

$$M_R = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

La matriz tiene tantas filas como elementos A y tantas columnas como B .

4.6.19 (★) VilCretas Verifique la matriz anterior con `MatrizRelBin`.

SOLUCIÓN

`In[ ] :=`

```
MatrizRelBin[{{1, 2}, {2, 2}, {3, 1}}, {1, 2, 3}, {1, 2}, table -> False]
```

`Out[ ] =`

```
{{0, 1}, {0, 1}, {1, 0}}
```

Sin `table->False` el comando devuelve una **tabla** con formato, muy cómoda de leer pero inservible como dato de entrada de otra instrucción.

4.6.20 (★) VilCretas Recupere los pares ordenados a partir de la matriz.

SOLUCIÓN

`In[ ] :=`

```
RelBinMatriz[{{0, 1}, {0, 1}, {1, 0}}, {1, 2, 3}, {1, 2}]
```

`Out[ ] =`

```
{{1, 2}, {2, 2}, {3, 1}}
```

RelBinMatriz es la operación inversa de **MatrizRelBin**: ambas representaciones contienen exactamente la misma información.

4.6.21 (★★) **VilCretas** Obtenga la matriz de la relación *menor que* sobre $A = \{1, 2, 3, 4\}$.

SOLUCIÓN

In[] :=

```
MatrizRelBin[RelBin["a < b", {1, 2, 3, 4}, {1, 2, 3, 4}], {1, 2, 3, 4},
{1, 2, 3, 4}, table -> False]
```

Out[] =

```
{{0, 1, 1, 1}, {0, 0, 1, 1}, {0, 0, 0, 1}, {0, 0, 0, 0}}
```

Es una matriz **triangular superior con diagonal nula**, la forma característica de un orden estricto.

4.6.22 (★★) **A MANO** ¿Cómo se reconoce en la matriz que una relación es reflexiva?

SOLUCIÓN Porque su diagonal principal está formada solo por unos: $(a_i, a_i) \in R$ para todo i . Si aparece un cero en la diagonal, ese elemento da el contraejemplo.

4.6.23 (★★) **A MANO** ¿Cómo se reconoce en la matriz que una relación es simétrica?

SOLUCIÓN Porque la matriz coincide con su transpuesta, $M_R = M_R^t$: si hay un 1 en la posición (i, j) también lo hay en la (j, i) . La antisimetría, en cambio, exige que no haya unos simétricos fuera de la diagonal.

4.6.24 (★★) **MIXTO** Obtenga la matriz de la relación de divisibilidad sobre $A = \{1, 2, 3, 6\}$ e interprete su forma.

SOLUCIÓN

In[] :=

```
MatrizRelBin[RelBin["Mod[b, a] == 0", {1, 2, 3, 6}, {1, 2, 3, 6}], {1, 2, 3, 6},
{1, 2, 3, 6}, table -> False]
```

Out[] =

```
{{1, 1, 1, 1}, {0, 1, 0, 1}, {0, 0, 1, 1}, {0, 0, 0, 1}}
```

La primera fila es de unos porque 1 divide a todos; la última columna también, porque todos dividen a 6. La diagonal llena refleja que todo número se divide a sí mismo, es decir, la reflexividad.

4.6.25 (★★) **VilCretas** Represente mediante un grafo la relación $R = \{(1, 2), (2, 3)\}$ sobre $A = \{1, 2, 3\}$.

SOLUCIÓN

In[] :=

```
GrafoRelBin[{{1, 2}, {2, 3}}, {1, 2, 3}]
```

Cada elemento de A es un vértice y cada par (a, b) una flecha de a a b . Como la relación no es simétrica el comando devuelve un **digrafo**; si lo fuera devolvería un grafo no dirigido, salvo que se pida `dirigido->True`.

4.6.26 (★★) **A MANO** ¿Qué se observa en el grafo de una relación reflexiva? ¿Y en el de una simétrica?

SOLUCIÓN En una relación reflexiva cada vértice tiene un **lazo**, una arista que sale y entra en él mismo. En una simétrica, cada flecha viene acompañada de su opuesta, de modo que el digrafo puede dibujarse como un grafo no dirigido.

4.6.27 (★★) **A MANO** Construya la relación cuya matriz es $\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$ sobre $A = \{1, 2, 3\}$ y clasifíquela a simple vista.

SOLUCIÓN Los unos dan $R = \{(1, 1), (1, 3), (2, 2), (3, 1), (3, 3)\}$. La diagonal es de unos, luego es **reflexiva**; la matriz es simétrica, luego R también. No es antisimétrica, porque $(1, 3)$ y $(3, 1)$ están ambos y $1 \neq 3$.

4.6.28 (★★★) **A MANO** ¿Cuántos unos tiene la matriz de la relación *menor o igual* sobre $A = \{1, 2, \dots, n\}$?

SOLUCIÓN La matriz es triangular superior con la diagonal incluida, de modo que tiene

$$n + (n - 1) + \dots + 1 = \frac{n(n + 1)}{2}$$

unos. Para $n = 4$ son 10, la mitad de los 16 pares posibles más la diagonal.

4.6.29 (★★) **A MANO** Una relación sobre A con $|A| = n$ tiene matriz de tamaño $n \times n$. ¿Cuántas relaciones distintas hay sobre A ?

SOLUCIÓN Cada una de las n^2 entradas puede ser 0 o 1 de manera independiente, así que hay 2^{n^2} relaciones. Para $n = 3$ son $2^9 = 512$.

4.6.30 (★★) **MIXTO** Calcule el cardinal de $A \times B$ con $|A| = 3$ y $|B| = 4$, y escriba un elemento de $A \times (B \times C)$.

SOLUCIÓN

`In[ ] :=`

`Length[PC[{1, 2, 3}, {4, 5, 6, 7}]]`

`Out[ ] =`

12

`In[ ] :=`

`PC[{1, 2}, PC[{3, 4}, {5, 6}]]`

`Out[ ] =`

`{1, {3, 5}}, {1, {3, 6}}, {1, {4, 5}}, {1, {4, 6}},
{2, {3, 5}}, {2, {3, 6}}, {2, {4, 5}}, {2, {4, 6}}`

Los elementos de $A \times (B \times C)$ son pares cuyo segundo componente es a su vez un par: $(1, (3, 5))$ y no la terna $(1, 3, 5)$. El producto cartesiano **no es asociativo** en sentido estricto.

4.6.31 (★★★) **A MANO** ¿Cuándo se cumple $A \times B = B \times A$?

SOLUCIÓN Solo si $A = B$, o si alguno de los dos es vacío. Si existen $a \in A$ y $b \in B$ con $a \notin B$, entonces (a, b) pertenece a $A \times B$ y no a $B \times A$. La conmutatividad falla justamente porque el par es **ordenado**.

4.6.32 (★★) **VilCretas** Decida con `ElementRelBinQ` si $(2, 4)$ y $(2, 3)$ pertenecen a la relación $ab > 6$.

SOLUCIÓN

In[] :=

```
{ElementRelBinQ["a*b > 6", {2, 4}, expalgebra -> True],
ElementRelBinQ["a*b > 6", {2, 3}, expalgebra -> True]}
```

Out[] =

```
{True, False}
```

$2 \cdot 4 = 8 > 6$ y $2 \cdot 3 = 6$, que no es mayor. Con `expalgebra->True` no hace falta enumerar la relación, que aquí sería infinita.

4.6.33 (★★) **MIXTO** Obtenga los pares de la relación cuya matriz sobre $\{1, 2, 3\}$ es $\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$.

SOLUCIÓN

In[] :=

```
RelBinMatriz[{{1, 0, 1}, {0, 1, 0}, {1, 1, 0}}, {1, 2, 3}, {1, 2, 3}]
```

Out[] =

```
{{1, 1}, {1, 3}, {2, 2}, {3, 1}, {3, 2}}
```

Son cinco pares, uno por cada uno de la matriz. La diagonal tiene un cero en la tercera posición, de modo que la relación no es reflexiva.

4.6.34 (★★) **MIXTO** Halle la inversa de esa relación transponiendo su matriz.

SOLUCIÓN

In[] :=

```
Transpose[{{1, 0, 1}, {0, 1, 0}, {1, 1, 0}}]
```

Out[] =

```
{{1, 0, 1}, {0, 1, 1}, {1, 0, 0}}
```

La transpuesta corresponde a $R^{-1} = \{(1, 1), (2, 2), (3, 1), (1, 3), (2, 3)\}$. Como la matriz **no** coincide con su transpuesta, la relación no es simétrica.

4.6.35 (★★★) **MIXTO** Con M_1 y M_2 dadas abajo, calcule su unión y su intersección booleanas e interprete el resultado.

SOLUCIÓN

In[] :=

```

M1 = {{1, 0, 1}, {0, 1, 0}, {1, 1, 0}};
M2 = {{0, 1, 0}, {1, 0, 1}, {0, 0, 1}};
{UnionBooleana[M1, M2, lista -> True], InterseccionBooleana[M1, M2, lista -> True]}

```

Out[] =

```

{{{1, 1, 1}, {1, 1, 1}, {1, 1, 1}}, {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}}

```

La unión es la matriz de unos y la intersección la de ceros: las dos relaciones son **complementarias**. En efecto, M_2 es exactamente el complemento booleano de M_1 .

4.6.36 (★★) **MIXTO** Compruebe la afirmación anterior.

SOLUCIÓN

In[] :=

```

M1 = {{1, 0, 1}, {0, 1, 0}, {1, 1, 0}};
ComplementoBooleano[M1, lista -> True]

```

Out[] =

```

{{0, 1, 0}, {1, 0, 1}, {0, 0, 1}}

```

Coincide con M_2 . Que $R_1 \cup R_2$ sea el total y $R_1 \cap R_2$ vacía es justamente la definición de relaciones complementarias.

4.6.37 (★★★) **MIXTO** Calcule el producto booleano $M_1 \odot M_2$ e interprételo como composición.

SOLUCIÓN

In[] :=

```

M1 = {{1, 0, 1}, {0, 1, 0}, {1, 1, 0}};
M2 = {{0, 1, 0}, {1, 0, 1}, {0, 0, 1}};
ProductoBooleano[M1, M2, lista -> True]

```

Out[] =

```

{{0, 1, 1}, {1, 0, 1}, {1, 1, 1}}

```

La entrada (3,3) vale uno porque desde 3 se llega a 1 en R_1 y de 1 a 3 en R_2 . El producto booleano es la matriz de $R_2 \circ R_1$: primero la de la izquierda, después la de la derecha.

4.6.38 (★★★) **A MANO** ¿Por qué el producto booleano no es conmutativo?

SOLUCIÓN Porque componer relaciones tampoco lo es. Con $R_1 = \{(1,2)\}$ y $R_2 = \{(2,3)\}$, la composición en un sentido da $\{(1,3)\}$ y en el otro es vacía. En matrices se ve igual: $M_1 \odot M_2$ y $M_2 \odot M_1$ tienen unos en posiciones distintas.

4.6.39 (★★) **MIXTO** Calcule el complemento de la relación identidad sobre $\{1,2,3\}$.

SOLUCIÓN

In[] :=

```

Complement[PC[{1, 2, 3}, {1, 2, 3}], {{1, 1}, {2, 2}, {3, 3}}]

```

Out[] =

```

{{1, 2}, {1, 3}, {2, 1}, {2, 3}, {3, 1}, {3, 2}}

```

Son los seis pares fuera de la diagonal, la relación *ser distinto de*. Es simétrica pero no reflexiva ni transitiva.

- 4.6.40 (★★) **MIXTO** Calcule la unión y la intersección de $\{(1, 2), (2, 3), (3, 1)\}$ con la identidad.

SOLUCIÓN

In[] :=

```
Union[{{1, 2}, {2, 3}, {3, 1}}, {{1, 1}, {2, 2}, {3, 3}}]
```

Out[] =

```
{{1, 1}, {1, 2}, {2, 2}, {2, 3}, {3, 1}, {3, 3}}
```

La unión con la identidad es la manera estándar de volver **reflexiva** una relación. La intersección, en cambio, sería vacía: el ciclo no tiene ningún par de la diagonal.

- 4.6.41 (★★★) **A MANO** Defina la *clausura reflexiva* de una relación y explíquela en términos de matrices.

SOLUCIÓN Es la menor relación reflexiva que contiene a R , es decir R unida con la identidad. En matrices consiste en **poner unos en toda la diagonal**, sin tocar el resto. Análogamente, la clausura simétrica es $R \cup R^{-1}$, o sea $M \vee M^t$.

- 4.6.42 (★★★) **MIXTO** Calcule la clausura simétrica de $\{(1, 2), (2, 3), (3, 1)\}$.

SOLUCIÓN

In[] :=

```
Union[{{1, 2}, {2, 3}, {3, 1}}, Reverse[{{1, 2}, {2, 3}, {3, 1}}, 2]]
```

Out[] =

```
{{1, 2}, {1, 3}, {2, 1}, {2, 3}, {3, 1}, {3, 2}}
```

Se obtienen los seis pares fuera de la diagonal. Añadir la inversa es el mínimo necesario para lograr la simetría, y no crea pares nuevos más allá de los simétricos de los existentes.

- 4.6.43 (★★) **MIXTO** Compruebe que la relación total sobre $\{1, 2, 3\}$ es idempotente para el producto booleano.

SOLUCIÓN

In[] :=

```
ProductoBooleano[{{1, 1, 1}, {1, 1, 1}, {1, 1, 1}},  
{{1, 1, 1}, {1, 1, 1}, {1, 1, 1}}, lista -> True]
```

Out[] =

```
{{1, 1, 1}, {1, 1, 1}, {1, 1, 1}}
```

$M \odot M = M$, de modo que la relación total es **transitiva**, además de reflexiva y simétrica: es la relación de equivalencia de una sola clase.

- 4.6.44 (★★) **VilCretas** Dibuje el digrafo de $\{(1, 2), (2, 3), (3, 1), (1, 1)\}$ sobre $\{1, 2, 3\}$.

SOLUCIÓN

In[] :=

```
GrafoRelBin[{1, 2}, {2, 3}, {3, 1}, {1, 1}, {1, 2, 3}]
```

El dibujo muestra un ciclo dirigido de tres vértices más un **lazo** en el vértice 1. Como la relación no es simétrica, el comando devuelve un digrafo.

4.6.45 (★★★) **A MANO** ¿Cuántas relaciones de equivalencia hay sobre conjuntos de 1 a 6 elementos?

SOLUCIÓN Son los números de *Bell*: 1, 2, 5, 15, 52, 203. El crecimiento es vertiginoso, más rápido que exponencial pero más lento que factorial. Con diez elementos ya hay 115 975 particiones distintas.

4.6.46 (★★) **MIXTO** Compruebe la sucesión anterior.

SOLUCIÓN

```
In[ ] :=
```

```
Table[BellB[k], {k, 1, 6}]
```

```
Out[ ] =
```

```
{1, 2, 5, 15, 52, 203}
```

Con un elemento solo hay una partición; con dos, dos; con tres, cinco. Cada número de *Bell* es la suma de los de *Stirling* de su fila.

4.6.47 (★★★) **A MANO** ¿Cuántas relaciones reflexivas hay sobre conjuntos de 1 a 4 elementos?

SOLUCIÓN Son 2^{n^2-n} : es decir 1, 4, 64, 4096. Crecen mucho más rápido que las de equivalencia, que para $n = 4$ son solo 15: exigir simetría y transitividad recorta drásticamente las posibilidades.

4.6.48 (★★) **MIXTO** Compruebe el conteo anterior.

SOLUCIÓN

```
In[ ] :=
```

```
Table[2^(k^2 - k), {k, 1, 4}]
```

```
Out[ ] =
```

```
{1, 4, 64, 4096}
```

Para $n = 4$ hay 4096 relaciones reflexivas frente a 15 de equivalencia: menos del medio por ciento.

4.6.49 (★★★) **A MANO** Demuestre que si R es transitiva entonces R^{-1} también lo es.

SOLUCIÓN Sean (a, b) y (b, c) en R^{-1} . Entonces (b, a) y (c, b) están en R , y por transitividad de R , $(c, a) \in R$, de donde $(a, c) \in R^{-1}$. La transitividad se conserva al invertir, igual que la reflexividad y la simetría; la **antisimetría** también.

4.6.50 (★★★) **A MANO** Si R es un orden parcial, ¿lo es R^{-1} ? ¿Qué significa?

SOLUCIÓN Sí. Reflexividad, antisimetría y transitividad se conservan al invertir. El orden inverso es el mismo orden **leído al revés**: si R es *menor o igual*, R^{-1} es *mayor o igual*. En el diagrama todo se mira boca abajo.

4.6.51 (★★★) **A MANO** Sea R la relación a divide a b sobre $\{1, 2, \dots, 12\}$. ¿Cuáles elementos son maximales y cuáles minimales?

SOLUCIÓN El único **minimal** es 1, que divide a todos. Los **maximales** son los que no dividen a ningún otro del conjunto: 7, 8, 9, 10, 11, 12. Un orden parcial puede tener varios maximales sin tener máximo: aquí no hay ningún elemento divisible por todos.

4.6.52 (★★★) **MIXTO** Compruebe cuántos pares tiene la divisibilidad sobre $\{1, \dots, 12\}$.

SOLUCIÓN

`In[ ] :=`

```
D12 = Range[12];
Length[RelBin["Mod[b, a] == 0", D12, D12]]
```

`Out[ ] =`

35

Son 35 pares. Coincide con $\sum_{a=1}^{12} \lfloor 12/a \rfloor$, que cuenta para cada divisor cuántos múltiplos suyos hay en el conjunto.

4.6.53 (★★) **A MANO** Escriba la relación R sobre $\{1, 2, 3\}$ que es reflexiva, simétrica y transitiva y tiene exactamente cinco pares.

SOLUCIÓN Debe ser de equivalencia con cinco pares. Con tres elementos, las particiones dan 3, 5 o 9 pares según sean tres, dos o una clase. Con cinco pares hay dos clases, por ejemplo $\{1, 2\}$ y $\{3\}$:

$$R = \{(1, 1), (1, 2), (2, 1), (2, 2), (3, 3)\}$$

Hay tres relaciones así, una por cada elección del elemento aislado.

4.6.54 (★★★) **A MANO** Generalice: si una relación de equivalencia sobre A induce clases de tamaños $n_1, \dots, n_k$, ¿cuántos pares tiene?

SOLUCIÓN Tiene $\sum_{i=1}^k n_i^2$: dentro de cada clase se relacionan todos con todos. Para las particiones de $\{1, 2, 3\}$ eso da $1 + 1 + 1 = 3$, $4 + 1 = 5$ y 9 . La suma es mínima cuando las clases son unitarias y máxima cuando hay una sola clase.

4.6.55 (★★★) **A MANO** Una relación de equivalencia sobre $\{1, \dots, 10\}$ tiene 34 pares. ¿Qué tamaños pueden tener sus clases?

SOLUCIÓN Hay que escribir 34 como suma de cuadrados que sumen 10 en sus bases. Con $5^2 + 2^2 + 1 + 1 + 1 + 1 + 1 = 34$ y $5 + 2 + 1 + 1 + 1 + 1 + 1 = 12$, no sirve. Probando, $4^2 + 3^2 + 3^2 = 34$ con $4 + 3 + 3 = 10$: las clases son de tamaños 4, 3 y 3. Es la única descomposición posible.

4.6.56 (★★) **A MANO** ¿Puede una relación ser simétrica y antisimétrica sin ser la identidad?

SOLUCIÓN Sí, si le faltan pares de la diagonal. Por ejemplo $\{(1, 1)\}$ sobre $\{1, 2\}$ es simétrica y antisimétrica pero no reflexiva, y por lo tanto no es la identidad. Lo que sí es cierto es que toda relación simétrica y antisimétrica está **contenida** en la identidad.

4.6.57 (★★★) **A MANO** ¿Cuántas relaciones sobre $\{1, 2, 3\}$ son a la vez simétricas y antisimétricas?

SOLUCIÓN Son las contenidas en la diagonal, es decir los subconjuntos de $\{(1, 1), (2, 2), (3, 3)\}$: hay $2^3 = 8$. Solo una de ellas, la diagonal completa, es además reflexiva.

4.6.58 (★★★) **A MANO** Sea R una relación sobre A con $|A| = n$. Expresé en términos de la matriz las cuatro propiedades.

SOLUCIÓN Reflexiva: la diagonal de M es de unos. **Simétrica:** $M = M^t$. **Antisimétrica:** $M \wedge M^t$ tiene ceros fuera de la diagonal. **Transitiva:** $M \odot M \leq M$ entrada a entrada. Las cuatro se verifican con operaciones matriciales, sin recorrer los pares uno a uno.

4.6.59 (★★★) **MIXTO** Compruebe la caracterización de la transitividad sobre la relación *menor o igual* en $\{1, 2, 3\}$.

SOLUCIÓN

In[] :=

```
L = {{1, 1, 1}, {0, 1, 1}, {0, 0, 1}};
ProductoBooleano[L, L, lista -> True]
```

Out[] =

```
{{1, 1, 1}, {0, 1, 1}, {0, 0, 1}}
```

$M \odot M = M$, de modo que se cumple $M \odot M \leq M$ y la relación es transitiva. En un orden total el producto booleano de la matriz consigo misma la deja **igual**.

4.6.60 (★★) **A MANO** Interprete el significado de $M \odot M = M$ frente a $M \odot M < M$.

SOLUCIÓN La igualdad significa que **todo par de la relación se alcanza en dos pasos**, además de en uno; ocurre en órdenes reflexivos y transitivos. La desigualdad estricta significa que algunos pares solo se alcanzan directamente, lo que pasa cuando falta reflexividad. En ambos casos la relación es transitiva: lo que importa es que no aparezcan unos **nuevos**.

4.6.61 (★★★) **A MANO** Sea R la relación sobre las cadenas binarias de longitud 4 dada por *tener el mismo número de unos*. ¿Cuántas clases tiene y de qué tamaños?

SOLUCIÓN Es de equivalencia y tiene **cinco** clases, una por cada cantidad de unos de 0 a 4. Sus tamaños son $\binom{4}{k}$, es decir 1, 4, 6, 4, 1, que suman las 16 cadenas. El número de pares es $1 + 16 + 36 + 16 + 1 = 70$.

4.6.62 (★★) **MIXTO** Compruebe el tamaño de la clase central del ejercicio anterior.

SOLUCIÓN

In[] :=

```
Length[Select[Tuples[{0, 1}, 4], Total[#] == 2 &]]
```

Out[] =

```
6
```

Son las seis cadenas con exactamente dos unos. Coincide con $\binom{4}{2} = 6$, el mayor de los cinco tamaños.

4.6.63 (★★★) **A MANO** Describa la partición que induce sobre $\mathbb{Z}$ la relación *tener el mismo signo*, y decida si es de equivalencia.

SOLUCIÓN No es de equivalencia sobre todo $\mathbb{Z}$ si el cero se considera sin signo: fallaría la reflexividad en 0. Definida sobre $\mathbb{Z} \setminus \{0\}$ sí lo es, con dos clases: los positivos y los negativos. Es un buen ejemplo de que el **conjunto sobre el que se define** forma parte de la relación.

4.6.64 (★★★) **A MANO** Sea R sobre $\mathbb{R}$ dada por $a R b$ si $a - b \in \mathbb{Z}$. Describa sus clases.

SOLUCIÓN Es de equivalencia y cada clase es $\{x + k \mid k \in \mathbb{Z}\}$ para un $x \in [0, 1)$: hay **tantas clases como puntos del intervalo** $[0, 1)$, es decir, una cantidad no numerable. La clase de $2,8$ contiene a $5,8$ y a $-0,2$, y coincide con la de $0,8$.

4.6.65 (★★★) **A MANO** Sobre el ejemplo anterior, decida si $\overline{2,8} = \overline{5,8}$ y si $\bar{1} = \{1\}$.

SOLUCIÓN La primera igualdad es **verdadera**: $5,8 - 2,8 = 3 \in \mathbb{Z}$. La segunda es **falsa**: $\bar{1}$ contiene a todos los enteros, no solo al 1. Confundir la clase con el representante es el error más común al trabajar con clases de equivalencia.

4.6.66 (★★) **A MANO** Escriba la partición de $\{1, \dots, 6\}$ asociada a *dejar el mismo residuo al dividir entre 3*.

SOLUCIÓN Los bloques son $\{3, 6\}$ (residuo 0), $\{1, 4\}$ (residuo 1) y $\{2, 5\}$ (residuo 2). Tres clases de dos elementos cada una, con $3 \cdot 2^2 = 12$ pares en la relación.

4.6.67 (★★★) **MIXTO** Compruebe la partición anterior y el número de pares.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
R = RelBin["Mod[a - b, 3] == 0", A, A];
{ParticionReEquivalencia[R, A], Length[R]}
```

Out[] =

```
{{{1, 4}, {2, 5}, {3, 6}}, 12}
```

Los tres bloques y los doce pares coinciden con lo previsto.

4.6.68 (★★★) **A MANO** Dos relaciones de equivalencia sobre A inducen particiones P_1 y P_2 . ¿Qué partición induce su intersección?

SOLUCIÓN La partición cuyos bloques son las **intersecciones** no vacías de un bloque de P_1 con uno de P_2 : el *refinamiento común*. Si P_1 separa por paridad y P_2 por residuo módulo 3, la intersección separa por residuo módulo 6.

4.6.69 (★★★) **MIXTO** Compruebe el refinamiento del ejercicio anterior sobre $\{1, \dots, 12\}$.

SOLUCIÓN

In[] :=

```
B = Range[12];
ParticionReEquivalencia[RelBin["Mod[a - b, 6] == 0", B, B], B]
```

Out[] =

```
{{1, 7}, {2, 8}, {3, 9}, {4, 10}, {5, 11}, {6, 12}}
```

Seis clases, producto de las dos de paridad por las tres de módulo 3. El refinamiento común tiene tantas clases como el producto, cuando los módulos son primos relativos.

4.6.70 (★★) **A MANO** ¿Qué relación de equivalencia induce la partición más fina y cuál la más gruesa?

SOLUCIÓN La más **fina** es la identidad, con todas las clases unitarias; la más **gruesa** es la relación total, con una única clase. Toda relación de equivalencia está entre esas dos, y el conjunto de todas ellas ordenado por inclusión forma un retículo.

4.6.71 (★★★) **A MANO** Sea R sobre $\{1, \dots, 6\}$ con $a R b$ si ab es un cuadrado perfecto. ¿Es de equivalencia?

SOLUCIÓN Sí. Reflexiva porque $a \cdot a = a^2$; simétrica porque el producto conmuta; y transitiva porque si ab y bc son cuadrados, su producto ab^2c también, y dividiendo entre b^2 queda ac cuadrado. Las clases agrupan los números con la misma **parte libre de cuadrados**: $\{1, 4\}$, $\{2\}$, $\{3\}$, $\{5\}$, $\{6\}$.

4.6.72 (★★★) **MIXTO** Compruebe las clases del ejercicio anterior.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
```

```
ParticionReEquivalencia[RelBin["IntegerQ[Sqrt[a*b]]", A, A], A]
```

Out[] =

```
{{1, 4}, {2}, {3}, {5}, {6}}
```

Cinco clases, y solo una tiene más de un elemento: 1 y 4 comparten parte libre de cuadrados igual a uno. Con el conjunto ampliado a $\{1, \dots, 12\}$ aparecerían $\{2, 8\}$ y $\{3, 12\}$.

4.3 Operaciones con relaciones binarias

4.6.73 (★) **A MANO** Calcule $R_1 \cup R_2$ y $R_1 \cap R_2$ para $R_1 = \{(1, 2), (2, 3)\}$ y $R_2 = \{(1, 3), (2, 3)\}$.

SOLUCIÓN La unión reúne los pares de ambas y la intersección conserva los comunes:

$$R_1 \cup R_2 = \{(1, 2), (1, 3), (2, 3)\}, \quad R_1 \cap R_2 = \{(2, 3)\}$$

4.6.74 (★) **VilCretas** Verifique las dos operaciones anteriores.

SOLUCIÓN

In[] :=

```
Union[{{1, 2}, {2, 3}}, {{1, 3}, {2, 3}}]
```

```
Intersection[{{1, 2}, {2, 3}}, {{1, 3}, {2, 3}}]
```

Out[] =

```
{{1, 2}, {1, 3}, {2, 3}}
```

```
{{2, 3}}
```

Al ser conjuntos de pares, las relaciones se operan con las instrucciones generales de conjuntos de *Mathematica*.

4.6.75 (★★) **VilCretas** Calcule el complemento de $R = \{(1, 2), (2, 1)\}$ dentro de $\{1, 2\} \times \{1, 2\}$.

SOLUCIÓN

In[] :=

```
Complement[PC[{{1, 2}, {1, 2}}, {{1, 2}, {2, 1}}]
```

Out[] =

```
{{1, 1}, {2, 2}}
```

El complemento se toma **respecto del producto cartesiano**, que hace de conjunto universo. Aquí el resultado es la relación identidad.

4.6.76 (★) **A MANO** Determine la relación inversa de $R = \{(1, 2), (2, 3)\}$.

SOLUCIÓN Se intercambian las coordenadas de cada par:

$$R^{-1} = \{(2, 1), (3, 2)\}$$

En términos de matrices, $M_{R^{-1}} = M_R^t$.

4.6.77 (★★) **VilCretas** Obtenga la inversa anterior con *Mathematica* y compruebe la relación con la transpuesta.

SOLUCIÓN

In[] :=

```
Reverse[{{1, 2}, {2, 3}}, 2]
```

Out[] =

```
{{2, 1}, {3, 2}}
```

In[] :=

```
Transpose[{{0, 1, 1}, {0, 0, 1}, {0, 0, 0}}]
```

Out[] =

```
{{0, 0, 0}, {1, 0, 0}, {1, 1, 0}}
```

Reverse con segundo argumento 2 invierte cada par; la transpuesta de la matriz hace lo mismo en la otra representación.

4.6.78 (★★) **A MANO** Calcule $R_1 \circ R_2$ con $R_2 = \{(1, 2), (1, 3)\}$ de A en B y $R_1 = \{(2, 5), (3, 6)\}$ de B en C .

SOLUCIÓN Se encadenan los pares: de 1 se llega a 2 y de 2 a 5; de 1 se llega a 3 y de 3 a 6. Por lo tanto

$$R_1 \circ R_2 = \{(1, 5), (1, 6)\}$$

La composición va de A en C y se lee de derecha a izquierda.

4.6.79 (★★) **VilCretas** Verifique la composición anterior.

SOLUCIÓN

In[] :=

```
RelacionComposicion[{{2, 5}, {3, 6}}, {{1, 2}, {1, 3}}]
```

Out[] =

```
{{1, 5}, {1, 6}}
```

El **orden de los argumentos** sigue la notación $R_1 \circ R_2$: primero se escribe R_1 , la que se aplica de última.

4.6.80 (★★) **VilCretas** Calcule $R \circ R$ para $R = \{(1, 2), (2, 3)\}$ e interprete el resultado.

SOLUCIÓN

In[] :=

RelacionComposicion[{{1, 2}, {2, 3}}, {{1, 2}, {2, 3}}]

Out[] =

{{1, 3}}

Componer una relación consigo misma equivale a recorrer **dos flechas seguidas** en su grafo: de 1 a 2 y de 2 a 3. Como (1,3) no está en R , la relación no es transitiva.

4.6.81 (★) **A MANO** Defina la unión booleana y la intersección booleana de dos matrices.

SOLUCIÓN Se operan entrada a entrada: la **unión booleana** coloca un 1 si al menos una de las dos matrices tiene un 1 en esa posición; la **intersección booleana**, solo si ambas lo tienen. Es el o y el y lógicos aplicados a cada casilla.

4.6.82 (★) **VilCretas** Calcule la unión booleana de $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ y $\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$.

SOLUCIÓN

In[] :=

UnionBooleana[{{1, 0}, {0, 1}}, {{0, 0}, {1, 1}}, lista -> True]

Out[] =

{{1, 0}, {1, 1}}

La opción `lista->True` devuelve la matriz como lista; sin ella el resultado viene con formato de matriz y no sirve como entrada de otro comando.

4.6.83 (★) **VilCretas** Calcule la intersección booleana de $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ y $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$.

SOLUCIÓN

In[] :=

InterseccionBooleana[{{1, 0}, {0, 1}}, {{1, 1}, {1, 0}}, lista -> True]

Out[] =

{{1, 0}, {0, 0}}

Solo sobrevive la posición (1,1), la única en la que ambas matrices tienen un 1.

4.6.84 (★) **VilCretas** Calcule el complemento booleano de $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

SOLUCIÓN

In[] :=

ComplementoBooleano[{{1, 0}, {0, 1}}, lista -> True]

Out[] =

{{0, 1}, {1, 0}}

Cada 1 pasa a 0 y cada 0 a 1. En términos de relaciones, el complemento booleano de M_R es la matriz de la relación complementaria.

4.6.85 (★★) **A MANO** Explique cómo se calcula el producto booleano de dos matrices.

SOLUCIÓN Se procede como en la multiplicación usual, pero sustituyendo la suma por el *o* lógico y el producto por el *y* lógico: la entrada (i, j) vale 1 si existe algún k con un 1 en la posición (i, k) de la primera matriz y otro en la (k, j) de la segunda.

4.6.86 (★★) **VilCretas** Calcule el producto booleano de $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ y $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.

SOLUCIÓN

In[] :=

```
ProductoBooleano[{{1, 0}, {1, 1}}, {{0, 1}, {1, 0}}, lista -> True]
```

Out[] =

```
{{0, 1}, {1, 1}}
```

Sin `lista->True` el comando devuelve un objeto con formato de matriz. Es una de las causas más frecuentes de que un cálculo encadenado falle.

4.6.87 (★★★) **MIXTO** Compruebe que $R = \{(1, 2), (2, 3)\}$ sobre $\{1, 2, 3\}$ no es transitiva usando el producto booleano de su matriz consigo misma.

SOLUCIÓN

In[] :=

```
ProductoBooleano[{{0, 1, 0}, {0, 0, 1}, {0, 0, 0}},  
{{0, 1, 0}, {0, 0, 1}, {0, 0, 0}}, lista -> True]
```

Out[] =

```
{{0, 0, 1}, {0, 0, 0}, {0, 0, 0}}
```

El producto tiene un 1 en la posición $(1, 3)$ que M_R no tiene. Como $M_R \odot M_R$ representa los caminos de longitud dos, la relación es transitiva exactamente cuando todo 1 del producto ya está en M_R , y aquí no ocurre.

4.6.88 (★★) **VilCretas** Explore las cuatro operaciones booleanas en un solo recurso interactivo.

SOLUCIÓN

In[] :=

```
CDFOperacionesMBooleanas[{{1, 0}, {0, 1}}, {{0, 1}, {1, 0}}]
```

El comando presenta la unión, la intersección, el complemento y el producto booleano de las dos matrices en una animación, y permite cambiarlas y recalcular sin volver a escribir el código.

4.6.89 (★★★) **A MANO** Demuestre que $(R^{-1})^{-1} = R$.

SOLUCIÓN Un par (a, b) pertenece a $(R^{-1})^{-1}$ si y solo si $(b, a) \in R^{-1}$, y esto ocurre si y solo si $(a, b) \in R$. Como los dos conjuntos tienen exactamente los mismos elementos, son iguales. En matrices es la igualdad $(M_R^t)^t = M_R$.

4.6.90 (★★★) **A MANO** ¿Es cierto que $R_1 \circ R_2 = R_2 \circ R_1$? Justifique con un ejemplo.

SOLUCIÓN No. Con $R_1 = \{(2, 5)\}$ y $R_2 = \{(1, 2)\}$ se obtiene $R_1 \circ R_2 = \{(1, 5)\}$, mientras que $R_2 \circ R_1$ es vacía, porque ningún par de R_1 termina donde empieza alguno de R_2 . La composición **no es conmutativa**.

4.6.91 (★★) **MIXTO** Sea $A = \{1, \dots, 6\}$ y R la relación $a R b$ si $a + b$ es par. Halle R explícitamente.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
RelBin["Mod[a + b, 2] == 0", A, A]
```

Out[] =

```
{{1, 1}, {1, 3}, {1, 5}, {2, 2}, {2, 4}, {2, 6}, {3, 1}, {3, 3},
{3, 5}, {4, 2}, {4, 4}, {4, 6}, {5, 1}, {5, 3}, {5, 5}, {6, 2}, {6, 4}, {6, 6}}
```

Son 18 pares: los que unen dos impares o dos pares. La condición equivale a que a y b tengan la **misma paridad**.

4.6.92 (★★) **MIXTO** Clasifique la relación anterior y halle su partición.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
R = RelBin["Mod[a + b, 2] == 0", A, A];
{ClasificacionRelBin[R, A], ParticionReEquivalencia[R, A]}
```

Out[] =

```
{{True, True, False, True}, {{1, 3, 5}, {2, 4, 6}}}
```

Es reflexiva, simétrica y transitiva (de equivalencia) y no antisimétrica. Sus dos clases son los impares y los pares.

4.6.93 (★★★) **MIXTO** Sobre el mismo conjunto, estudie la relación a divide a b .

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
R = RelBin["Mod[b, a] == 0", A, A];
{ClasificacionRelBin[R, A], RelacionOrdenParcialQ[R, A]}
```

Out[] =

```
{{True, False, True, True}, True}
```

Reflexiva, antisimétrica y transitiva: es un **orden parcial**. No es total: 4 y 6 no se dividen entre sí, de modo que no son comparables.

4.6.94 (★★★) **MIXTO** Estudie la relación $a + b = 7$ sobre $\{1, \dots, 6\}$.

SOLUCIÓN

In[] :=


```
A = {1, 2, 3, 4, 5, 6};
R = RelBin["a + b == 7", A, A];
{R, ClasificacionRelBin[R, A]}
```

Out[] =

```
{{{1, 6}, {2, 5}, {3, 4}, {4, 3}, {5, 2}, {6, 1}}, {False, True, False, False}}
```

Es **solo simétrica**. No es reflexiva porque ningún a cumple $2a = 7$; no es antisimétrica porque hay pares simétricos con $a \neq b$; y no es transitiva: de $(1,6)$ y $(6,1)$ debería seguirse $(1,1)$, que no está.

4.6.95 (★★) **MIXTO** Estudie la relación *menor o igual* sobre $\{1, \dots, 6\}$ y escriba su matriz.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
MatrizRelBin[RelBin["a <= b", A, A], A, A, table -> False]
```

Out[] =

```
{{1, 1, 1, 1, 1, 1}, {0, 1, 1, 1, 1, 1}, {0, 0, 1, 1, 1, 1},
{0, 0, 0, 1, 1, 1}, {0, 0, 0, 0, 1, 1}, {0, 0, 0, 0, 0, 1}}
```

Triangular superior con diagonal llena: la forma de un orden total. El número de unos es $\frac{6 \cdot 7}{2} = 21$.

4.6.96 (★★★) **MIXTO** Escriba la matriz de $a + b = 7$ y describa su forma.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
MatrizRelBin[RelBin["a + b == 7", A, A], A, A, table -> False]
```

Out[] =

```
{{0, 0, 0, 0, 0, 1}, {0, 0, 0, 0, 1, 0}, {0, 0, 0, 1, 0, 0},
{0, 0, 1, 0, 0, 0}, {0, 1, 0, 0, 0, 0}, {1, 0, 0, 0, 0, 0}}
```

Es la **antidiagonal**. Coincide con su transpuesta, lo que confirma la simetría, y tiene la diagonal nula, lo que confirma que no es reflexiva.

4.6.97 (★★★) **MIXTO** Estudie la relación $|a - b| \leq 1$ sobre $\{1, \dots, 6\}$.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
R = RelBin["Abs[a - b] <= 1", A, A];
ClasificacionRelBin[R, A]
```

Out[] =

```
{True, True, False, False}
```

Es reflexiva y simétrica pero **no transitiva**: $(1,2)$ y $(2,3)$ están en R y $(1,3)$ no. Es el ejemplo clásico de relación de *proximidad* que no llega a ser de equivalencia.

4.6.98 (★★★) **A MANO** ¿Por qué la proximidad del ejercicio anterior no puede ser transitiva por mucho que se agrande el conjunto?

SOLUCIÓN Porque encadenando pasos de tamaño uno se llega arbitrariamente lejos: de $(1, 2)$, $(2, 3)$, $\dots$, $(n-1, n)$ la transitividad exigiría $(1, n)$, que viola $|a - b| \leq 1$ en cuanto $n \geq 3$. La transitividad es justamente la propiedad que **no tolera** las relaciones basadas en cercanía.

4.6.99 (★★) **MIXTO** Cuento los elementos de $A \times A$ y las relaciones binarias posibles sobre $A = \{1, \dots, 6\}$.

SOLUCIÓN

In[] :=

```
A = {1, 2, 3, 4, 5, 6};
{Length[PC[A, A]], 2^Length[PC[A, A]]}
```

Out[] =

```
{36, 68719476736}
```

Hay 36 pares y $2^{36} \approx 6,9 \cdot 10^{10}$ relaciones. Enumerarlas es imposible; por eso se estudian por sus **propiedades** y no una por una.

4.6.100 (★★★) **A MANO** ¿Cuántas relaciones reflexivas y cuántas simétricas hay sobre un conjunto de 6 elementos?

SOLUCIÓN **Reflexivas**: la diagonal está fijada y quedan $36 - 6 = 30$ entradas libres, de modo que hay $2^{30} = 1\,073\,741\,824$. **Simétricas**: la matriz queda determinada por su diagonal y su triángulo superior, es decir $6 + 15 = 21$ entradas libres: $2^{21} = 2\,097\,152$. Hay muchas más reflexivas que simétricas.

4.6.101 (★★★) **A MANO** ¿Cuántas relaciones sobre $\{1, \dots, 6\}$ son reflexivas y simétricas a la vez?

SOLUCIÓN La diagonal está fijada en unos y el triángulo superior estricto tiene 15 entradas libres, cada una replicada en el inferior. Hay entonces $2^{15} = 32\,768$. Contar es contar **entradas libres de la matriz**: la reflexividad fija seis y la simetría une las restantes de dos en dos.

4.6.102 (★★★) **MIXTO** Sean $R_1 = \{(1, 5), (2, 6), (3, 4)\}$ y $R_2 = \{(4, 1), (5, 2), (6, 3)\}$. Calcule $R_1 \circ R_2$.

SOLUCIÓN

In[] :=

```
RelacionComposicion[{{1, 5}, {2, 6}, {3, 4}}, {{4, 1}, {5, 2}, {6, 3}}]
```

Out[] =

```
{{4, 5}, {5, 6}, {6, 4}}
```

De 4 se pasa a 1 y de 1 a 5; de 5 a 2 y de 2 a 6; de 6 a 3 y de 3 a 4. El resultado es un **ciclo** de longitud tres sobre $\{4, 5, 6\}$.

4.6.103 (★★★) **MIXTO** Sea M la matriz de la relación $\{(1, 2), (2, 3), (3, 1)\}$. Calcule $M \odot M$ y $M \odot M \odot M$, e interprete.

SOLUCIÓN

In[] :=

```
Mrel = {{0, 1, 0}, {0, 0, 1}, {1, 0, 0}};
ProductoBooleano[Mrel, Mrel, lista -> True]
```

```
Out[ ] =
```

```
{{0, 0, 1}, {1, 0, 0}, {0, 1, 0}}
```

```
In[ ] :=
```

```
Mrel = {{0, 1, 0}, {0, 0, 1}, {1, 0, 0}};
ProductoBooleano[Mrel, ProductoBooleano[Mrel, Mrel, lista -> True], lista -> True]
```

```
Out[ ] =
```

```
{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
```

El cubo es la **identidad**: tres pasos por el ciclo devuelven cada elemento a su lugar. La entrada (i, j) de M^k indica si hay un camino de longitud k de i a j .

Por qué la variable se llama Mrel y no M: el paquete *Combinatorica*, que este capítulo carga para dibujar algunos grafos, define M como el número de aristas de un grafo y lo deja **protegido**. Si se ha ejecutado `Get["Combinatorica"]` en la misma sesión, `M = ...` falla con `Set::wrsym` y todo el bloque queda sin evaluar. Lo prudente es reservar las letras sueltas y usar nombres propios.

4.6.104 (★★★) A MANO Deduzca del ejercicio anterior una condición matricial para la transitividad.

SOLUCIÓN Una relación es transitiva si y solo si todo camino de longitud dos ya está representado por una arista, es decir

$$M \odot M \leq M$$

entendida entrada a entrada. En el ciclo de tres elementos falla: $M \odot M$ tiene unos donde M tiene ceros, de modo que la relación no es transitiva.

4.6.105 (★★) MIXTO Obtenga las clases de equivalencia de la relación de paridad sobre $\{1, \dots, 6\}$.

SOLUCIÓN

```
In[ ] :=
```

```
A = {1, 2, 3, 4, 5, 6};
ClasesEquivalencia[RelBin["Mod[a + b, 2] == 0", A, A], A]
```

La salida muestra $[1] = [3] = [5] = \{1, 3, 5\}$ y $[2] = [4] = [6] = \{2, 4, 6\}$, y resume que las clases distintas son dos. Que seis elementos generen solo dos clases es lo normal: la clase de un elemento depende de la relación, no del elemento.

4.6.106 (★★★) A MANO ¿Cuántas relaciones de equivalencia distintas hay sobre un conjunto de cinco elementos?

SOLUCIÓN Tantas como particiones, es decir el número de *Bell* $B_5 = 52$. Se descompone por número de bloques con los números de *Stirling* de segunda especie:

$$1 + 15 + 25 + 10 + 1 = 52$$

correspondientes a 1, 2, 3, 4 y 5 bloques.

4.6.107 (★★) MIXTO Compruebe el conteo anterior.

SOLUCIÓN**In[] :=**

```
{Table[StirlingS2[5, k], {k, 1, 5}], BellB[5]}
```

Out[] =

```
{{1, 15, 25, 10, 1}, 52}
```

La distribución tiene su máximo en tres bloques: la mayoría de las particiones de cinco elementos parten el conjunto en tres partes.

4.6.108 (★★★) A MANO Demuestre que la intersección de dos relaciones de equivalencia sobre A es de equivalencia.

SOLUCIÓN Reflexividad: (a, a) está en ambas, luego en la intersección. Simetría: si (a, b) está en las dos, (b, a) también lo está en las dos. Transitividad: si (a, b) y (b, c) están en la intersección, están en cada una, y por transitividad de cada una (a, c) también. La **unión**, en cambio, puede fallar la transitividad.

4.6.109 (★★★) A MANO Dé un ejemplo de dos relaciones de equivalencia cuya unión no lo sea.

SOLUCIÓN Sobre $\{1, 2, 3\}$, tome R_1 con clases $\{1, 2\}, \{3\}$ y R_2 con clases $\{1\}, \{2, 3\}$. Su unión contiene $(1, 2)$ y $(2, 3)$ pero no $(1, 3)$, de modo que **no es transitiva**. Unir clases no produce clases: la menor relación de equivalencia que contiene a la unión es la que tiene $\{1, 2, 3\}$ como única clase.

4.6.110 (★★★) A MANO Demuestre que si R es simétrica y transitiva y todo elemento se relaciona con alguno, entonces R es reflexiva.

SOLUCIÓN Sea $a \in A$. Por hipótesis existe b con $(a, b) \in R$. Por simetría $(b, a) \in R$, y por transitividad $(a, a) \in R$. La hipótesis extra es imprescindible: sin ella, la relación vacía es simétrica y transitiva pero no reflexiva sobre un conjunto no vacío.

4.6.111 (★★) A MANO ¿Qué relación sobre A es a la vez de equivalencia y de orden parcial?

SOLUCIÓN ¿Únicamente la **identidad** $\{(a, a) \mid a \in A\}$. Ser de equivalencia exige simetría y ser de orden parcial exige antisimetría; juntas obligan a que todo par (a, b) de la relación cumpla $a = b$. La reflexividad, común a ambas, obliga a que esté toda la diagonal.

4.6.112 (★★★) A MANO Sea R la relación sobre los enteros dada por $a R b$ si $a - b$ es múltiplo de 5. Describa sus clases.

SOLUCIÓN Es de equivalencia (la congruencia módulo 5) y tiene exactamente **cinco** clases, una por cada residuo: $[0], [1], [2], [3], [4]$. Cada clase es infinita, por ejemplo $[2] = \{\dots, -8, -3, 2, 7, 12, \dots\}$. Que el conjunto sea infinito no impide que las clases sean pocas.

4.6.113 (★★★) MIXTO Verifique sobre $\{1, \dots, 10\}$ que la congruencia módulo 5 tiene cinco clases.

SOLUCIÓN**In[] :=**

```
B = Range[10];
ParticionReEquivalencia[RelBin["Mod[a - b, 5] == 0", B, B], B]
```

Out[] =

$\{\{1, 6\}, \{2, 7\}, \{3, 8\}, \{4, 9\}, \{5, 10\}\}$

Cinco bloques de dos elementos cada uno. Al restringir a un conjunto finito las clases se cortan, pero su **número** sigue siendo el módulo.

4.6.114 (★★★) A MANO Sobre $\{1, \dots, n\}$ con n múltiplo de k , ¿cuántos elementos tiene cada clase de la congruencia módulo k ?

SOLUCIÓN Exactamente n/k , y hay k clases: la partición es **uniforme**. Si n no es múltiplo de k , las primeras $n \bmod k$ clases tienen un elemento más que las demás. En todo caso la suma de los tamaños es n , como exige toda partición.

4.6.115 (★★) A MANO Determine el dominio y el ámbito de $R = \{(1, 6), (2, 5), (3, 4), (4, 3)\}$ sobre $\{1, \dots, 6\}$.

SOLUCIÓN El **dominio** es $\{1, 2, 3, 4\}$ y el **ámbito** es $\{3, 4, 5, 6\}$. Ni uno ni otro agotan el conjunto: los elementos 5 y 6 no aparecen como primera coordenada y 1 y 2 no aparecen como segunda.

4.6.116 (★★★) A MANO Si R es una relación sobre A finito, ¿qué se puede decir del dominio y el ámbito de R^{-1} ?

SOLUCIÓN Se **intercambian**: el dominio de R^{-1} es el ámbito de R y su ámbito es el dominio de R . En matrices corresponde a transponer, que convierte sumas de filas en sumas de columnas.

4.6.117 (★★★) A MANO Demuestre que $(R_1 \circ R_2)^{-1} = R_2^{-1} \circ R_1^{-1}$.

SOLUCIÓN Un par (c, a) pertenece a $(R_1 \circ R_2)^{-1}$ si y solo si $(a, c) \in R_1 \circ R_2$, es decir si existe b con $(a, b) \in R_2$ y $(b, c) \in R_1$. Eso equivale a que $(c, b) \in R_1^{-1}$ y $(b, a) \in R_2^{-1}$, o sea $(c, a) \in R_2^{-1} \circ R_1^{-1}$. El orden se **invierte**, como al transponer un producto de matrices.

4.6.118 (★★) A MANO Si R es reflexiva sobre A y $|A| = n$, ¿cuántos pares tiene como mínimo y como máximo?

SOLUCIÓN Como mínimo n , los de la diagonal, y como máximo n^2 , todo el producto cartesiano. La relación mínima reflexiva es la identidad y la máxima es la relación total, y ambas son de equivalencia.

4.6.119 (★★★) A MANO Una relación sobre $\{1, \dots, 6\}$ es antisimétrica y tiene 20 pares. ¿Cuántos pueden estar fuera de la diagonal como máximo?

SOLUCIÓN La antisimetría permite a lo sumo **uno** de cada par simétrico $\{(a, b), (b, a)\}$ con $a \neq b$. Hay 15 de esos pares, de modo que fuera de la diagonal caben a lo sumo 15 elementos; con los 6 de la diagonal se llega a 21. Con 20 pares, la relación está a uno del máximo posible.

4.6.120 (★★★) MIXTO Compruebe que el orden *menor o igual* sobre $\{1, \dots, 6\}$ alcanza ese máximo de 21 pares.

SOLUCIÓN

`In[ ] :=`

```
A = {1, 2, 3, 4, 5, 6};
Length[RelBin["a <= b", A, A]]
```

`Out[ ] =`

21

En efecto: es antisimétrica con la máxima cantidad posible de pares, porque es un orden **total** y compara todos los elementos entre sí. Cualquier orden parcial no total tendría menos.

4.6.121 (★★★) **A MANO** ¿Qué forma tiene la matriz de un orden total sobre $\{1, \dots, n\}$ si los elementos se listan en el orden de la relación?

SOLUCIÓN Triangular superior con la diagonal completa: todas las entradas (i, j) con $i \leq j$ valen uno. Si el conjunto se lista en otro orden, la matriz deja de ser triangular pero la relación es la misma: la forma de la matriz depende del **orden elegido para listar** los elementos, no solo de la relación.

4.4 Tipos de relaciones

4.6.122 (★) **A MANO** Enuncie las cuatro propiedades que clasifican una relación sobre un conjunto A .

SOLUCIÓN R es **reflexiva** si $(a, a) \in R$ para todo $a \in A$; **simétrica** si $(a, b) \in R$ implica $(b, a) \in R$; **antisimétrica** si $(a, b) \in R$ y $(b, a) \in R$ obligan a $a = b$; y **transitiva** si $(a, b) \in R$ y $(b, c) \in R$ implican $(a, c) \in R$.

4.6.123 (★) **A MANO** Clasifique $R = \{(1, 1), (2, 2), (3, 3)\}$ sobre $A = \{1, 2, 3\}$.

SOLUCIÓN Cumple las cuatro propiedades: es reflexiva por tener toda la diagonal, simétrica y antisimétrica de manera vacía al no haber pares fuera de la diagonal, y transitiva por la misma razón. Es a la vez relación de equivalencia y de orden parcial.

4.6.124 (★) **VilCretas** Confirme la clasificación anterior.

SOLUCIÓN

In[] :=

```
TipoRelacion[{{1, 1}, {2, 2}, {3, 3}}, {1, 2, 3}]
```

La salida enumera las seis conclusiones: reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial. La relación identidad es el único caso que satisface todo a la vez.

4.6.125 (★★) **VilCretas** Clasifique $R = \{(1, 2), (2, 3)\}$ sobre $A = \{1, 2, 3\}$ y lea los contraejemplos.

SOLUCIÓN

In[] :=

```
TipoRelacion[{{1, 2}, {2, 3}}, {1, 2, 3}]
```

El comando responde que no es reflexiva porque falta $(1, 1)$; que no es simétrica porque está $(1, 2)$ y no $(2, 1)$; que sí es antisimétrica; y que no es transitiva porque están $(1, 2)$ y $(2, 3)$ pero falta $(1, 3)$. Dar el **contraejemplo** es lo que convierte la respuesta en una justificación.

4.6.126 (★★) **VilCretas** Use `ClasificacionRelBin` sobre $R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$ e interprete el vector de salida.

SOLUCIÓN

In[] :=

```
ClasificacionRelBin[{1, 1}, {2, 2}, {3, 3}, {1, 2}, {2, 1}, {1, 2, 3}]
```

```
Out[ ] =
```

```
{True, True, False, True}
```

Las cuatro componentes son, en ese orden, reflexiva, simétrica, antisimétrica y transitiva. Aquí falla solo la antisimetría, por estar $(1, 2)$ y $(2, 1)$ con $1 \neq 2$.

4.6.127 (★★) VilCretas Aplique `ClasificacionRelBin` a $R = \{(1, 2), (2, 3), (1, 3)\}$.

SOLUCIÓN

```
In[ ] :=
```

```
ClasificacionRelBin[{1, 2}, {2, 3}, {1, 3}, {1, 2, 3}]
```

```
Out[ ] =
```

```
{False, False, True, True}
```

No es reflexiva ni simétrica, pero sí antisimétrica y transitiva: al añadirle la diagonal se convertiría en una relación de orden parcial.

4.6.128 (★★) VilCretas Clasifique una relación razonando sobre su matriz booleana.

SOLUCIÓN

```
In[ ] :=
```

```
TipoMRelacion[{1, 2}, {2, 1}, {1, 1}, {2, 2}, {1, 2}]
```

`TipoMRelacion` llega a las mismas conclusiones que `TipoRelacion`, pero justificándolas con las propiedades de la matriz: diagonal de unos para la reflexividad, coincidencia con la transpuesta para la simetría, y así sucesivamente.

4.6.129 (★★) A MANO Muestre que la relación *menor o igual* sobre $\{1, 2, 3\}$ es de orden parcial.

SOLUCIÓN Es reflexiva porque $a \leq a$; antisimétrica porque $a \leq b$ y $b \leq a$ obligan a $a = b$; y transitiva porque $a \leq b$ y $b \leq c$ dan $a \leq c$. Cumple las tres propiedades exigidas, de modo que es un **orden parcial**. De hecho es total: dos elementos cualesquiera son comparables.

4.6.130 (★★) VilCretas Compruebe que la divisibilidad sobre $A = \{1, 2, 3, 6\}$ es un orden parcial.

SOLUCIÓN

```
In[ ] :=
```

```
RelacionOrdenParcialQ[RelBin["Mod[b, a] == 0", {1, 2, 3, 6}, {1, 2, 3, 6}],  
{1, 2, 3, 6}]
```

```
Out[ ] =
```

```
True
```

La relación es $\{(1, 1), (1, 2), (1, 3), (1, 6), (2, 2), (2, 6), (3, 3), (3, 6), (6, 6)\}$. No es un orden total: 2 y 3 no se dividen entre sí, y por eso no son comparables.

4.6.131 (★★) A MANO Dé un ejemplo de relación simétrica y antisimétrica a la vez, y otro que no sea ninguna de las dos.

SOLUCIÓN La identidad $\{(1, 1), (2, 2)\}$ es simétrica y antisimétrica: las dos condiciones se cumplen sin conflicto porque no hay pares fuera de la diagonal. En cambio $\{(1, 2), (2, 1), (1, 3)\}$ no es simétrica, por faltar $(3, 1)$, ni antisimétrica, por estar $(1, 2)$ y $(2, 1)$.

4.6.132 (★★) **A MANO** ¿Puede una relación no ser reflexiva y aun así ser transitiva?

SOLUCIÓN Sí. La relación $\{(1, 2), (2, 3), (1, 3)\}$ sobre $\{1, 2, 3\}$ es transitiva y no tiene ningún par de la diagonal. Las cuatro propiedades son **independientes**: cumplir una no obliga a cumplir otra.

4.6.133 (★) **A MANO** ¿Qué es una relación de equivalencia?

SOLUCIÓN La que es reflexiva, simétrica y transitiva a la vez. Su interés está en que reparte el conjunto en clases: los elementos relacionados entre sí quedan agrupados y los no relacionados, separados.

4.6.134 (★) **VilCretas** Decida si $R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$ es de equivalencia.

SOLUCIÓN

In[] :=

```
RelacionEquivalenciaQ[{{1, 1}, {2, 2}, {3, 3}, {1, 2}, {2, 1}}, {1, 2, 3}]
```

Out[] =

True

Es reflexiva, simétrica y transitiva. El elemento 3 queda aislado en su propia clase.

4.6.135 (★★) **VilCretas** Obtenga las clases de equivalencia de la relación anterior.

SOLUCIÓN

In[] :=

```
ClasesEquivalencia[{{1, 1}, {2, 2}, {3, 3}, {1, 2}, {2, 1}}, {1, 2, 3}]
```

La salida muestra $[1] = \{1, 2\}$, $[2] = \{1, 2\}$ y $[3] = \{3\}$, y luego resume que las clases distintas son $\{1, 2\}$ y $\{3\}$. Que $[1]$ y $[2]$ coincidan no es casual: dos clases o son iguales o son disjuntas.

4.6.136 (★★) **VilCretas** Obtenga la partición asociada a esa relación de equivalencia.

SOLUCIÓN

In[] :=

```
ParticionReEquivalencia[{{1, 1}, {2, 2}, {3, 3}, {1, 2}, {2, 1}}, {1, 2, 3}]
```

Out[] =

```
{{1, 2}, {3}}
```

Las clases distintas forman una **partición**: bloques no vacíos, disjuntos dos a dos y cuya unión es todo A .

4.6.137 (★★) **VilCretas** Recorra el camino inverso: construya la relación de equivalencia asociada a la partición $\{\{1, 2\}, \{3\}\}$.

SOLUCIÓN

In[] :=


```
ReEquivalenciaParticion[{{1, 2}, {3}}, {1, 2, 3}]
```

```
Out[ ] =
```

```
{{1, 1}, {1, 2}, {2, 1}, {2, 2}, {3, 3}}
```

Se relacionan entre sí todos los elementos de un mismo bloque. Junto con el ejercicio anterior, esto ilustra la correspondencia **uno a uno** entre relaciones de equivalencia y particiones de un conjunto.

4.6.138 (★★) **MIXTO** Estudie la relación *congruencia módulo 3* sobre $A = \{1, 2, \dots, 6\}$.

SOLUCIÓN

```
In[ ] :=
```

```
RelBin["Mod[a - b, 3] == 0", {1, 2, 3, 4, 5, 6}, {1, 2, 3, 4, 5, 6}]
```

```
Out[ ] =
```

```
{{1, 1}, {1, 4}, {2, 2}, {2, 5}, {3, 3}, {3, 6},  
{4, 1}, {4, 4}, {5, 2}, {5, 5}, {6, 3}, {6, 6}}
```

Los doce pares se agrupan por residuo. La relación es de equivalencia, como ocurre siempre con la congruencia módulo un entero positivo.

4.6.139 (★★) **VilCretas** Confirme que la congruencia módulo 3 es de equivalencia y obtenga su partición.

SOLUCIÓN

```
In[ ] :=
```

```
RelacionEquivalenciaQ[RelBin["Mod[a - b, 3] == 0", {1, 2, 3, 4, 5, 6},  
{1, 2, 3, 4, 5, 6}], {1, 2, 3, 4, 5, 6}]
```

```
Out[ ] =
```

```
True
```

```
In[ ] :=
```

```
ParticionReEquivalencia[RelBin["Mod[a - b, 3] == 0", {1, 2, 3, 4, 5, 6},  
{1, 2, 3, 4, 5, 6}], {1, 2, 3, 4, 5, 6}]
```

```
Out[ ] =
```

```
{{1, 4}, {2, 5}, {3, 6}}
```

Los tres bloques son las clases de residuo 1, 2 y 0 módulo 3.

4.6.140 (★★★) **A MANO** Demuestre que dos clases de equivalencia o son iguales o son disjuntas.

SOLUCIÓN Sean $[a]$ y $[b]$ con algún elemento común c . Entonces aRc y bRc ; por simetría cRb y por transitividad aRb . Si ahora $x \in [a]$, de xRa y aRb sale xRb , luego $x \in [b]$, y por el mismo argumento al revés $[b] \subseteq [a]$. Por lo tanto $[a] = [b]$: si comparten un elemento, coinciden.

4.6.141 (★★★) **A MANO** ¿Cuántas relaciones reflexivas hay sobre un conjunto de n elementos?

SOLUCIÓN Las n entradas de la diagonal están obligadas a valer 1; las restantes $n^2 - n$ son libres. Por lo tanto hay $2^{n^2 - n}$ relaciones reflexivas. Para $n = 3$ son $2^6 = 64$.

4.6.142 (★★★) **A MANO** Una relación de equivalencia sobre $A = \{1, 2, 3, 4\}$ tiene exactamente dos clases. ¿Cuántas hay?

SOLUCIÓN Contar las relaciones equivale a contar las particiones de A en dos bloques no vacíos. Los bloques pueden tener tamaños $1 + 3$ o $2 + 2$: hay 4 del primer tipo y 3 del segundo, en total 7. Es el número de *Stirling* de segunda especie $S(4, 2)$.

4.6.143 (★★★) **MIXTO** Construya una relación que sea de orden parcial pero no de equivalencia, y compruébelo.

SOLUCIÓN La divisibilidad sobre $\{1, 2, 3, 6\}$ sirve: es reflexiva, antisimétrica y transitiva.

In[] :=

```
{RelacionOrdenParcialQ[RelBin["Mod[b, a] == 0", {1, 2, 3, 6}, {1, 2, 3, 6}], {1, 2, 3, 6}],
RelacionEquivalenciaQ[RelBin["Mod[b, a] == 0", {1, 2, 3, 6}, {1, 2, 3, 6}], {1, 2, 3, 6}]}
```

Out[] =

```
{True, False}
```

Falla la simetría: 1 divide a 2 pero 2 no divide a 1. Salvo la identidad, ninguna relación puede ser de los dos tipos a la vez.

4.5 Producto cartesiano general, composición y generadores

4.6.144 (★★) **MIXTO** El producto cartesiano no se limita a dos conjuntos. Calcule con **PCGeneral** el producto $\{1, 2\} \times \{a, b\} \times \{x\}$.

SOLUCIÓN

In[] :=

```
PCGeneral[{1, 2}, {a, b}, {x}]
```

Out[] =

```
{{1, a, x}, {1, b, x}, {2, a, x}, {2, b, x}}
```

Los elementos ya no son pares sino **ternas**. **PC** trabaja con dos conjuntos; **PCGeneral** admite cuantos se le den, y devuelve las tuplas en orden lexicográfico.

4.6.145 (★★) **MIXTO** Compruebe la regla del producto: $|A \times B \times C| = |A| |B| |C|$ con $|A| = 3$, $|B| = 4$ y $|C| = 5$.

SOLUCIÓN

In[] :=

```
Length[PCGeneral[Range[3], Range[4], Range[5]]]
```

Out[] =

```
60
```

Son $3 \cdot 4 \cdot 5 = 60$ ternas. La regla se extiende a k conjuntos por inducción: cada factor nuevo multiplica el número de tuplas por su cardinal.

4.6.146 (★★★) **A MANO** ¿Cuántas tuplas tiene A^k si $|A| = n$, y qué pasa si alguno de los conjuntos es vacío?

SOLUCIÓN Tiene n^k . Si alguno de los factores es vacío, **el producto es vacío**: no hay ninguna tupla que completar, porque no hay de dónde tomar esa componente. Es el mismo motivo por el que un producto de números con un factor cero vale cero.

4.6.147 (★★★) MIXTO Sobre $A = \{1, 2, 3, 4\}$ tome $R_1 : a > b$ y $R_2 : a < b$, y calcule las dos composiciones con `RelacionComposicion`.

SOLUCIÓN

`In[ ] :=`

```
A = Range[4];
R1 = RelBin["a>b", A, A];
R2 = RelBin["a<b", A, A];
RelacionComposicion[R1, R2]
```

`Out[ ] =`

```
{{1, 1}, {1, 2}, {1, 3}, {2, 1}, {2, 2}, {2, 3}, {3, 1}, {3, 2}, {3, 3}}
```

`In[ ] :=`

```
A = Range[4];
R1 = RelBin["a>b", A, A];
R2 = RelBin["a<b", A, A];
RelacionComposicion[R2, R1]
```

`Out[ ] =`

```
{{2, 2}, {2, 3}, {2, 4}, {3, 2}, {3, 3}, {3, 4}, {4, 2}, {4, 3}, {4, 4}}
```

Las dos tienen nueve pares, pero **no son la misma relación**: la primera vive en $\{1, 2, 3\}^2$ y la segunda en $\{2, 3, 4\}^2$. La composición de relaciones **no es conmutativa**.

4.6.148 (★★★) A MANO Interprete en palabras la composición del ejercicio anterior.

SOLUCIÓN $R_1 \circ R_2$ relaciona a con c cuando existe un b con $a < b$ y $b > c$, es decir cuando **hay algún elemento por encima de los dos**. Sobre $\{1, 2, 3, 4\}$ eso ocurre siempre que a y c sean menores que 4, y de ahí salen los nueve pares de $\{1, 2, 3\}^2$.

4.6.149 (★★) MIXTO Obtenga el dominio y el rango de $R_1 : a > b$ sobre $\{1, 2, 3, 4\}$ con `DominioRelBin` y `RangoRelBin`.

SOLUCIÓN

`In[ ] :=`

```
A = Range[4];
R1 = RelBin["a>b", A, A];
{DominioRelBin[R1], RangoRelBin[R1]}
```

`Out[ ] =`

```
{{2, 3, 4}, {1, 2, 3}}
```

El 1 no está en el dominio porque no hay nada menor que él, y el 4 no está en el rango por la razón simétrica. Dominio y rango son **subconjuntos** de A , no todo A : una relación no tiene por qué usar todos los elementos.

4.6.150 (★★) **MIXTO** Calcule dominio y rango de $R = \{(1, 2), (1, 3), (4, 5)\}$.

SOLUCIÓN

In[] :=

```
{DominioRelBin[{{1, 2}, {1, 3}, {4, 5}}],  
RangoRelBin[{{1, 2}, {1, 3}, {4, 5}}]}
```

Out[] =

```
{{1, 4}, {2, 3, 5}}
```

El dominio recoge las primeras componentes **sin repetir** y el rango las segundas. El 1 aparece dos veces en la relación y una sola en el dominio: es un conjunto, no una lista.

4.6.151 (★★) **VilCretas** Muestre la matriz de R_1 con la opción **encabezados** $\rightarrow$ **True** y diga qué añade frente a la matriz sin encabezados.

SOLUCIÓN

In[] :=

```
A = Range[4];  
R1 = RelBin["a>b", A, A];  
MatrizRelBin[R1, A, A, encabezados -> True]
```

Añade una **fila y una columna** con los elementos de los conjuntos, de modo que cada casilla se puede leer sin contar posiciones. Con conjuntos de cuatro elementos apenas se nota; con veinte, decide si la matriz se lee o se adivina.

4.6.152 (★★★) **VilCretas** Dibuje el grafo de R_1 con **dirigido** $\rightarrow$ **True** y explique por qué esa opción importa aquí.

SOLUCIÓN

In[] :=

```
A = Range[4];  
R1 = RelBin["a>b", A, A];  
{GrafoRelBin[R1, A], GrafoRelBin[R1, A, dirigido -> True]}
```

R_1 **no es simétrica**: si $a > b$ entonces no $b > a$. Dibujada sin dirección, la arista entre 3 y 1 no dice cuál de los dos es mayor, y el grafo pierde justo la información que define la relación. La opción **dirigido** $\rightarrow$ **True** pone las flechas.

4.6.153 (★★) **VilCretas** Genere relaciones al azar con **RelBinRandom** y con **RelBinRandomConjuntos**. ¿En qué se diferencian?

SOLUCIÓN

In[] :=

```
{RelBinRandom[12],  
RelBinRandomConjuntos[10, {1, 2, {c, d}, h, x}, {7, m, b, j}]}
```

El primero inventa **también los conjuntos**, con las opciones **sizeA**, **sizeB** y **rango** para acotarlos. El segundo recibe los conjuntos ya dados y solo escoge los pares, que es lo que hace falta cuando se quiere practicar sobre un conjunto concreto.

4.6.154 (★★★) ViCretas `RelBinRandomCriterios` genera la relación a partir de criterios. Genérela pidiendo ver los criterios usados.

SOLUCIÓN

`In[ ] :=`

```
RelBinRandomCriterios[Range[20], Range[20], 3, verCriterios -> True]
```

Con `verCriterios -> True` el comando devuelve también el conjunto de criterios de donde escogió. Eso convierte el generador en un ejercicio completo: primero se intenta **adivinar el criterio** a partir de los pares, y después se comprueba contra la lista.

4.6.155 (★★★) ViCretas Use `RelBinRandomCriteriosReales` para obtener criterios sobre los reales, y explique por qué ahí no se puede listar la relación.

SOLUCIÓN

`In[ ] :=`

```
RelBinRandomCriteriosReales[3, verCriterios -> True]
```

Sobre $\mathbb{R}$ la relación tiene **infinitos pares**: no hay lista que escribir, solo criterio. Por eso este comando devuelve únicamente el criterio, mientras que `RelBinRandomCriterios`, que trabaja sobre conjuntos finitos, devuelve además los pares. Clasificar una relación sobre $\mathbb{R}$ obliga a razonar sobre el criterio, no a inspeccionar casos.

CAPÍTULO 5 • COMPLEMENTARIOS

Teoría de grafos

5.1 Grafos, valencias y grafos especiales

5.6.1 (★) **A MANO** Defina qué es un grafo y qué distingue a un digrafo.

SOLUCIÓN Un grafo $G = (V, E)$ consta de un conjunto no vacío V de vértices y un conjunto E de aristas, cada una de las cuales une dos vértices. En un **digrafo** las aristas son pares **ordenados**: la arista de a a b no es la misma que la de b a a , y se dibuja con una flecha.

5.6.2 (★) **A MANO** ¿Qué es un grafo ponderado y para qué sirve?

SOLUCIÓN Es aquel en el que cada arista lleva asociado un valor, normalmente numérico, llamado **peso**. Sirve para modelar distancias, costos o tiempos, y es la estructura sobre la que trabaja el algoritmo del camino más corto.

5.6.3 (★) **ViiCretas** Construya el grafo de aristas $\{1, 2\}$, $\{2, 3\}$, $\{3, 4\}$, $\{4, 1\}$ y $\{1, 3\}$, y cuente sus vértices y sus lados.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
CantNodos[G]
CantAristas[G]
```

Out[] =

```
4
5
```

Es un cuadrilátero con una diagonal. Se usará como grafo de referencia en varios de los ejercicios siguientes.

5.6.4 (★) **ViiCretas** Obtenga la lista de vértices y la lista de aristas de ese grafo.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
ListaVertices[G]
ListaAristas[G]
```

Out[] =

```
{1, 2, 3, 4}
{{1, 2}, {1, 3}, {2, 3}, {3, 4}, {4, 1}}
```

El software reordena los lados internamente; el grafo es el mismo, porque un grafo queda determinado por sus conjuntos V y E , no por el orden en que se escriban.

5.6.5 (★) **A MANO** Calcule a mano las valencias de los cuatro vértices del grafo anterior.

SOLUCIÓN Se cuentan las aristas que inciden en cada nodo:

$$\theta(1) = 3, \quad \theta(2) = 2, \quad \theta(3) = 3, \quad \theta(4) = 2$$

Los vértices 1 y 3 son los extremos de la diagonal, y por eso tienen una arista más.

5.6.6 (★) **VilCretas** Verifique las valencias con **Valencias**.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
Valencias[G, table -> False]
```

Out[] =

```
{3, 2, 3, 2}
```

Sin `table->False` la salida viene con formato de tabla, cómoda de leer pero inservible como entrada de otro comando.

5.6.7 (★★) **A MANO** Enuncie y compruebe el teorema que relaciona las valencias con el número de aristas.

SOLUCIÓN En un grafo no dirigido con p aristas distintas,

$$\sum_{i=1}^n \theta(v_i) = 2p$$

porque cada arista aporta una unidad a la valencia de cada uno de sus dos extremos. En el grafo de referencia: $3 + 2 + 3 + 2 = 10 = 2 \cdot 5$.

5.6.8 (★★) **A MANO** Deduzca del teorema anterior que el número de vértices de valencia impar es par.

SOLUCIÓN La suma de todas las valencias es $2p$, un número par. Las valencias pares suman par, de modo que la suma de las impares también debe ser par; y una suma de números impares es par solo si hay una cantidad par de sumandos. Por eso ningún grafo tiene exactamente tres vértices de grado impar.

5.6.9 (★★) **A MANO** ¿Existe un grafo con valencias 3, 3, 3, 1? ¿Y con 2, 2, 2, 1?

SOLUCIÓN El primero sí: la suma es $10 = 2 \cdot 5$, luego tendría cinco aristas, y hay dos vértices de grado impar, cantidad par. El segundo no: la suma es 7, que es impar y no puede ser $2p$; además habría un solo vértice de grado impar.

5.6.10 (★) **VilCretas** Construya el grafo completo con cinco vértices y cuente sus aristas.

SOLUCIÓN

In[] :=

```
CantAristas[GrafoCompleto[5]]
```

Out[] =

```
10
```

En K_n cada par de vértices está unido, de modo que el número de aristas es $\binom{n}{2} = \frac{n(n-1)}{2}$.

Para $n = 5$: $\frac{5 \cdot 4}{2} = 10$.

5.6.11 (★) **VilCretas** Obtenga las valencias de K_5 .

SOLUCIÓN

In[] :=

```
Valencias[GrafoCompleto[5], table -> False]
```

Out[] =

```
{4, 4, 4, 4, 4}
```

En K_n todos los vértices tienen valencia $n - 1$: el grafo completo es **regular**.

5.6.12 (★★) **MIXTO** Compruebe la fórmula del número de aristas de K_n para $n = 4$.

SOLUCIÓN

In[] :=

```
CantAristas[GrafoCompleto[4]]
```

Out[] =

```
6
```

Coincide con $\frac{4 \cdot 3}{2} = 6$. También se obtiene del teorema de las valencias: $4 \cdot 3 = 12 = 2p$, de donde $p = 6$.

5.6.13 (★★) **VilCretas** Construya el grafo bipartito completo $K_{3,3}$ y obtenga sus aristas y sus valencias.

SOLUCIÓN

In[] :=

```
CantAristas[GrafoBipartitoCompleto[3, 3]]
```

```
Valencias[GrafoBipartitoCompleto[3, 3], table -> False]
```

Out[] =

```
9
```

```
{3, 3, 3, 3, 3, 3}
```

En $K_{n,m}$ hay $n \cdot m$ aristas: cada vértice de un lado se une con todos los del otro. Aquí $3 \cdot 3 = 9$, y cada nodo tiene valencia 3.

5.6.14 (★★) **VilCretas** Explore de forma interactiva los grafos completos hasta el orden seis.

SOLUCIÓN

In[] :=

```
GrafoCompleto[6, analizar -> True]
```

Con **analizar->True** el comando anima los grafos completos hasta el orden indicado y muestra, en cada caso, la valencia de los vértices, la cantidad de aristas y si hay circuitos de *Euler* y de *Hamilton*. La opción existe también en **GrafoBipartitoCompleto** y en **GrafoVacio**.

5.6.15 (★) **A MANO** ¿Qué es un grafo vacío? ¿Cuántas aristas tiene?

SOLUCIÓN Es el que tiene vértices pero **ninguna arista**. Todas sus valencias son cero y no es conexo si tiene más de un nodo. Se construye con `GrafoVacio[n]`.

5.6.16 (★★) **VilCretas** Construya un grafo ponderado y muestre los pesos sobre sus aristas.

SOLUCIÓN

`In[ ] :=`

```
Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}, mostrarpesos -> True]
```

La lista de pesos sigue el orden en que se escribieron las aristas. `mostrarpesos->True` los dibuja sobre cada lado; sin esa opción el grafo es ponderado igualmente, pero los pesos no se ven.

5.6.17 (★★) **VilCretas** Dibuje un digrafo con las mismas aristas del grafo de referencia.

SOLUCIÓN

`In[ ] :=`

```
Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}, dirigido -> True]
```

Con `dirigido->True` cada par se interpreta como una flecha del primer vértice al segundo. Las valencias se desdoblan entonces en grado de entrada y grado de salida.

5.6.18 (★★) **A MANO** Defina ruta, circuito y grafo conexo.

SOLUCIÓN Una **ruta** es una sucesión de aristas en la que el extremo final de cada una es el inicial de la siguiente. Un **circuito** es una ruta que empieza y termina en el mismo vértice. Un grafo es **conexo** si entre cualquier par de vértices existe al menos una ruta.

5.6.19 (★★) **A MANO** ¿Cuántas aristas como mínimo necesita un grafo conexo de n vértices?

SOLUCIÓN Necesita al menos $n-1$. Con menos aristas quedaría al menos un vértice sin conectar al resto. Los grafos conexos con exactamente $n-1$ aristas son los árboles, tema del capítulo siguiente.

5.6.20 (★) **MIXTO** Calcule las valencias del camino de cuatro vértices $1-2-3-4$ y compruebe el teorema de las valencias.

SOLUCIÓN

`In[ ] :=`

```
P = Grafo[{{1, 2}, {2, 3}, {3, 4}}];
Valencias[P, table -> False]
```

`Out[ ] =`

```
{1, 2, 2, 1}
```

La suma es $1 + 2 + 2 + 1 = 6 = 2 \cdot 3$, y en efecto el camino tiene tres aristas. Los extremos tienen valencia uno y los interiores dos.

5.6.21 (★★) **MIXTO** Cuentas las aristas de $K_{2,3}$ y compruebe la fórmula del grafo bipartito completo.

SOLUCIÓN

`In[ ] :=`

```
CantAristas[GrafoBipartitoCompleto[2, 3]]
```

```
Out[ ] =
```

```
6
```

En $K_{n,m}$ hay $n \cdot m$ aristas, aquí $2 \cdot 3 = 6$. Ningún par de vértices del mismo lado está unido.

5.6.22 (★★) **MIXTO** Verifique el teorema de las valencias sobre $K_{3,3}$.

SOLUCIÓN

```
In[ ] :=
```

```
Total[Valencias[GrafoBipartitoCompleto[3, 3], table -> False]]
```

```
Out[ ] =
```

```
18
```

Los seis vértices tienen valencia 3, de modo que la suma es $18 = 2 \cdot 9$ y el grafo tiene nueve aristas, como ya se había contado.

5.2 Representaciones para un grafo

5.6.23 (★) **A MANO** Escriba la matriz de adyacencia del grafo de referencia en el orden $\{1, 2, 3, 4\}$.

SOLUCIÓN La entrada (i, j) cuenta las aristas entre v_i y v_j :

$$M_A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Es simétrica, como toda matriz de adyacencia de un grafo no dirigido, y su diagonal es nula porque no hay lazos.

5.6.24 (★) **VilCretas** Verifique la matriz anterior con **MGrafo**.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
```

```
MGrafo[G, {1, 2, 3, 4}, matriz -> True]
```

```
Out[ ] =
```

```
{{0, 1, 1, 1}, {1, 0, 1, 0}, {1, 1, 0, 1}, {1, 0, 1, 0}}
```

`matriz->True` devuelve la lista; `table->True` la presenta con encabezados. El segundo argumento fija el **orden** de los vértices y por lo tanto el de filas y columnas.

5.6.25 (★★) **A MANO** ¿Cómo se leen las valencias en la matriz de adyacencia?

SOLUCIÓN Sumando cada fila. En el grafo de referencia las sumas son 3, 2, 3 y 2, las valencias ya calculadas. Si hubiera lazos, cada uno aportaría dos al grado del vértice.

5.6.26 (★★) **VilCretas** Construya un grafo a partir de su matriz de adyacencia.

SOLUCIÓN

In[] :=

```
GrafoM[{{0, 1, 1}, {1, 0, 1}, {1, 1, 0}}, {1, 2, 3}]
```

GrafoM es la operación inversa de **MGrafo**. La matriz dada corresponde al triángulo K_3 : todos los vértices unidos entre sí.

5.6.27 (★★) **VilCretas** Obtenga la matriz de incidencia del grafo de aristas $\{1, 2\}$ y $\{2, 3\}$.

SOLUCIÓN

In[] :=

```
MIGrafo[Grafo[{{1, 2}, {2, 3}}], {1, 2, 3}, matriz -> True]
```

Out[] =

```
{{1, 0}, {1, 1}, {0, 1}}
```

En la matriz de incidencia las filas son vértices y las columnas aristas: la entrada vale 1 si el vértice pertenece a la arista. El vértice 2 tiene dos unos porque incide en las dos aristas.

5.6.28 (★★) **A MANO** ¿Qué suma da cada columna de una matriz de incidencia y por qué?

SOLUCIÓN Cada columna suma 2, porque toda arista tiene exactamente dos extremos. La suma de todas las entradas es entonces $2p$, que es otra forma de leer el teorema de las valencias.

5.6.29 (★★) **VilCretas** Construya un grafo a partir de su matriz de incidencia.

SOLUCIÓN

In[] :=

```
GrafoMI[{{1, 0}, {1, 1}, {0, 1}}, {1, 2, 3}]
```

GrafoMI reconstruye el grafo leyendo, en cada columna, cuáles dos vértices tienen un uno. Se obtiene el camino $1 - 2 - 3$.

5.6.30 (★★★) **MIXTO** Cuentas las rutas de longitud dos entre cada par de vértices del grafo de referencia.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
MatrixPower[MGrafo[G, {1, 2, 3, 4}], matriz -> True], 2]
```

Out[] =

```
{{3, 1, 2, 1}, {1, 2, 1, 2}, {2, 1, 3, 1}, {1, 2, 1, 2}}
```

La entrada (i, j) de M_A^k cuenta las rutas de longitud k de v_i a v_j . Aquí hay tres rutas de longitud dos de 1 a 1, una por cada vecino.

5.6.31 (★★) **VilCretas** Obtenga la misma información con **CantMRutas**.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
CantMRutas[G, 2]
```

El comando devuelve la matriz con encabezados. Con `all->True` anima todas las potencias desde la primera hasta la k , que es la manera más clara de ver cómo crece el número de recorridos.

5.6.32 (★★) VilCretas Cuente todas las rutas del vértice 1 al 3 y enumérelas.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
CantRutas[G, 1, 3, rutas -> True]
```

```
Out[ ] =
```

```
{3, {{1, 3}}, {{1, 4}, {4, 3}}, {{1, 2}, {2, 3}}}
```

Son la diagonal directa y los dos caminos que rodean el cuadrilátero. `CantRutas` cuenta rutas **sin repetir aristas**, a diferencia de las potencias de la matriz de adyacencia.

5.6.33 (★★) VilCretas Genere a lo sumo cinco rutas del vértice 1 al 3.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
GeneraRutas[G, 1, 3, 5]
```

```
Out[ ] =
```

```
{{1, 3}}, {{1, 4}, {4, 3}}, {{1, 2}, {2, 3}}
```

Aunque se piden cinco, el comando devuelve las tres que existen: el cuarto argumento es un **máximo**, no una cantidad exacta.

5.6.34 (★★) VilCretas Anime el recorrido de una ruta sobre el grafo.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
AnimarGrafo[G, {{1, 2}, {2, 3}, {3, 4}}]
```

`AnimarGrafo` resalta las aristas de la lista una tras otra, en el orden dado. Es el recurso que usan internamente varias instrucciones del capítulo para mostrar circuitos y caminos.

5.6.35 (★) MIXTO Obtenga la matriz de adyacencia del ciclo de cuatro vértices e interprete su forma.

SOLUCIÓN

```
In[ ] :=
```

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
MGrafo[H, {1, 2, 3, 4}, matriz -> True]
```

Out[] =

```
{{0, 1, 0, 1}, {1, 0, 1, 0}, {0, 1, 0, 1}, {1, 0, 1, 0}}
```

Cada fila tiene exactamente dos unos, los dos vecinos de ese vértice en el ciclo. Las filas 1 y 3 coinciden, igual que las 2 y 4: los vértices opuestos tienen los mismos vecinos.

5.6.36 (★★) **VilCretas** Obtenga la matriz de incidencia del triángulo K_3 .

SOLUCIÓN

In[] :=

```
MIGrafo[Grafo[{{1, 2}, {2, 3}, {3, 1}}], {1, 2, 3}, matriz -> True]
```

Out[] =

```
{{1, 0, 1}, {1, 1, 0}, {0, 1, 1}}
```

Cada fila tiene dos unos, la valencia del vértice, y cada columna también dos, los extremos de la arista. La matriz es cuadrada por casualidad: K_3 tiene tantos vértices como aristas.

5.6.37 (★★) **MIXTO** Cuenten las rutas del vértice 1 al 3 en el ciclo de cuatro vértices.

SOLUCIÓN

In[] :=

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
CantRutas[H, 1, 3, rutas -> True]
```

Out[] =

```
{2, {{1, 4}, {4, 3}}, {{1, 2}, {2, 3}}}
```

Son los dos arcos del ciclo. Al quitar la diagonal del grafo de referencia se pierde justamente la tercera ruta.

5.3 Circuitos de *Euler* y de *Hamilton*

5.6.38 (★) **A MANO** ¿Qué es un circuito de *Euler* y qué condición lo caracteriza?

SOLUCIÓN Es un circuito que recorre **todas las aristas** del grafo exactamente una vez. Un grafo conexo tiene circuito de *Euler* si y solo si todas sus valencias son pares.

5.6.39 (★) **A MANO** ¿Qué es una ruta de *Euler* y cuándo existe?

SOLUCIÓN Es un recorrido que usa todas las aristas una sola vez, pero sin obligación de terminar donde empezó. Un grafo conexo tiene ruta de *Euler* cuando posee **exactamente dos** vértices de valencia impar, y el recorrido empieza en uno y acaba en el otro.

5.6.40 (★★) **MIXTO** Decida si el grafo de referencia tiene circuito o ruta de *Euler*.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
{CircuitoEulerQ[G], RutaEulerQ[G]}
```

Out[] =

```
{False, True}
```

A mano: las valencias son 3, 2, 3, 2, con exactamente dos impares. No hay circuito, pero sí ruta, que debe empezar en el vértice 1 y terminar en el 3, o al revés.

5.6.41 (★★) **VilCretas** Obtenga una ruta de *Euler* concreta sobre ese grafo.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
RutaEuler[G]
```

Out[] =

```
{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}
```

El recorrido usa las cinco aristas una sola vez y va del vértice 1 al 3, como predice el teorema. Con `ruta->True` el comando anima el trayecto.

5.6.42 (★) **MIXTO** Compruebe que el ciclo de cuatro vértices sí tiene circuito de *Euler*.

SOLUCIÓN

In[] :=

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
{Valencias[H, table -> False], CircuitoEulerQ[H]}
```

Out[] =

```
{{2, 2, 2, 2}, True}
```

Al quitar la diagonal todas las valencias quedan pares y aparece el circuito, que es el propio cuadrilátero.

5.6.43 (★★) **VilCretas** Construya el circuito de *Euler* de ese ciclo paso a paso con el algoritmo de *Fleury*.

SOLUCIÓN

In[] :=

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
Fleury[H]
```

El comando muestra las cuatro iteraciones, añadiendo una arista cada vez, y concluye que un circuito de *Euler* es $\{1, 2, 3, 4, 1\}$, o en términos de aristas $\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}$. La regla de *Fleury* es no tomar nunca una arista de corte si queda otra opción.

5.6.44 (★★) **VilCretas** Halle dos circuitos de *Euler* distintos sobre el ciclo de cuatro vértices.

SOLUCIÓN

In[] :=

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
CircuitosEuler[H, 2]
```

El comando busca hasta dos circuitos distintos, pero en este grafo solo existe uno salvo el punto de partida y el sentido, así que devuelve una única lista. Con `ruta->True` anima uno de ellos.

5.6.45 (★★) **MIXTO** ¿Tiene K_4 circuito de *Euler*? ¿Y K_5 ?

SOLUCIÓN

```
In[ ] :=
```

```
{CircuitoEulerQ[GrafoCompleto[4]], CircuitoEulerQ[GrafoCompleto[5]]}
```

```
Out[ ] =
```

```
{False, True}
```

En K_n todas las valencias valen $n - 1$. Para $n = 4$ son impares y no hay circuito; para $n = 5$ son pares y sí lo hay. En general K_n tiene circuito de *Euler* exactamente cuando n es impar.

5.6.46 (★) **A MANO** ¿Qué es un circuito de *Hamilton* y en qué se diferencia del de *Euler*?

SOLUCIÓN El de *Hamilton* pasa por **todos los vértices** exactamente una vez y regresa al inicio; el de *Euler* recorre **todas las aristas**. Uno se fija en los nodos y el otro en los lados, y un grafo puede tener uno sin tener el otro.

5.6.47 (★★) **MIXTO** Decida si el grafo de referencia tiene circuito de *Hamilton*.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
CircuitoHamiltonQ[G]
```

```
Out[ ] =
```

```
True
```

A mano se exhibe el recorrido $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$, que visita los cuatro vértices una sola vez. La diagonal no se usa.

5.6.48 (★★) **VilCretas** Obtenga circuitos de *Hamilton* concretos sobre el ciclo de cuatro vértices.

SOLUCIÓN

```
In[ ] :=
```

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
CircuitosHamilton[H, 2]
```

La salida es el propio ciclo: en un grafo de cuatro nodos y cuatro aristas no hay otro recorrido posible. La instrucción acepta `ruta->True` para animarlo.

5.6.49 (★★) **A MANO** Enuncie las tres condiciones suficientes para la existencia de un circuito de *Hamilton*.

SOLUCIÓN Sea G simple y conexo con $n \geq 3$ vértices y p aristas:

1. si $p \geq \frac{n^2 - 3n + 6}{2}$, hay circuito de *Hamilton*;
2. propiedad de *Dirac*: si $\theta(a) \geq \frac{n}{2}$ para todo vértice, también;
3. propiedad de *Ore*: si $\theta(a) + \theta(b) \geq n$ para todo par de vértices no adyacentes, también.

5.6.50 (★★★) **A MANO** ¿Qué se concluye si un grafo no cumple ninguna de las tres condiciones anteriores?

SOLUCIÓN Nada. Las tres son condiciones suficientes, no necesarias: un grafo puede tener circuito de *Hamilton* sin cumplir ninguna. La única respuesta correcta en ese caso es que *no hay criterio*, y eso es exactamente lo que devuelve `ExisteCircuitoHamilton`.

5.6.51 (★★) **VilCretas** Aplique `ExisteCircuitoHamilton` al grafo de referencia e interprete el vector.

SOLUCIÓN

`In[ ] :=`

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
ExisteCircuitoHamilton[G, vectorial -> True]
```

`Out[ ] =`

```
{True, True, True, True}
```

Las componentes son, en ese orden, la propiedad de *Ore*, la de *Dirac*, la de la cantidad de aristas y la existencia del circuito. Aquí se cumplen las tres condiciones: $n = 4$, $p = 5$ y $\frac{16 - 12 + 6}{2} = 5$, de modo que $p \geq 5$.

5.6.52 (★★★) **A MANO** Verifique a mano la propiedad de *Dirac* en el grafo de referencia.

SOLUCIÓN Con $n = 4$, la propiedad exige $\theta(a) \geq 2$ para todo vértice. Las valencias son 3, 2, 3, 2, todas mayores o iguales que 2, de modo que la propiedad se cumple y garantiza el circuito de *Hamilton*. Basta que falle en un solo vértice para que el criterio no pueda aplicarse.

5.6.53 (★★★) **A MANO** Un grafo tiene circuito de *Euler* pero no de *Hamilton*. Dé un ejemplo.

SOLUCIÓN Dos triángulos que comparten un vértice: {1, 2}, {2, 3}, {3, 1}, {3, 4}, {4, 5}, {5, 3}. Todas las valencias son pares (2, 2, 4, 2, 2), luego hay circuito de *Euler*; pero cualquier recorrido que visite todos los vértices debe pasar dos veces por el 3, así que no hay circuito de *Hamilton*.

5.6.54 (★★★) **MIXTO** Compruebe con el software el ejemplo anterior.

SOLUCIÓN

`In[ ] :=`

```
K = Grafo[{{1, 2}, {2, 3}, {3, 1}, {3, 4}, {4, 5}, {5, 3}}];
{Valencias[K, table -> False], CircuitoEulerQ[K], CircuitoHamiltonQ[K]}
```

`Out[ ] =`

```
{{2, 2, 4, 2, 2}, True, False}
```

El software confirma el razonamiento: circuito de *Euler* sí, de *Hamilton* no. El vértice de corte es el que lo impide.

5.6.55 (★★) **A MANO** ¿Tiene $K_{3,3}$ circuito de *Euler*?

SOLUCIÓN No. Todas sus valencias valen 3, que es impar, y el criterio exige que todas sean pares. En general $K_{n,m}$ tiene circuito de *Euler* solo si n y m son ambos pares.

5.6.56 (★★) **MIXTO** ¿Tiene el camino $1 - 2 - 3 - 4$ ruta o circuito de *Euler*?

SOLUCIÓN

In[] :=

```
P = Grafo[{{1, 2}, {2, 3}, {3, 4}}];
{RutaEulerQ[P], CircuitoEulerQ[P]}
```

Out[] =

```
{True, False}
```

Las valencias son 1, 2, 2, 1: hay exactamente dos impares, de modo que existe ruta de *Euler* entre los extremos 1 y 4, pero no circuito.

5.6.57 (★★) **MIXTO** ¿Tiene K_4 ruta de *Euler*?

SOLUCIÓN

In[] :=

```
RutaEulerQ[GrafoCompleto[4]]
```

Out[] =

```
False
```

Sus cuatro vértices tienen valencia 3, todas impares. El criterio admite a lo sumo dos vértices de grado impar, y aquí hay cuatro.

5.6.58 (★★★) **MIXTO** Compare $K_{3,3}$ y $K_{2,3}$ respecto del circuito de *Hamilton*.

SOLUCIÓN

In[] :=

```
{CircuitoHamiltonQ[GrafoBipartitoCompleto[3, 3]],
CircuitoHamiltonQ[GrafoBipartitoCompleto[2, 3]]}
```

Out[] =

```
{True, False}
```

En un grafo bipartito todo circuito alterna lados, de modo que solo puede cerrarse si los dos lados tienen el mismo número de vértices. $K_{3,3}$ lo cumple y $K_{2,3}$ no.

5.6.59 (★★) **MIXTO** Sea H el grafo con nodos $\{1, \dots, 6\}$ y aristas $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}, \{4, 6\}, \{5, 6\}$ y $\{1, 6\}$. Halle sus valencias y compruebe el teorema.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
```

```
H = Grafo[Ar];
{Valencias[H, table -> False], CantAristas[H]}
```

```
Out[ ] =
```

```
{{3, 3, 3, 3, 3}, 9}
```

Es un grafo **regular** de grado 3 con seis vértices. La suma de valencias es $18 = 2 \cdot 9$, como exige el teorema. Este grafo se usa en varios de los ejercicios siguientes.

- 5.6.60** (★★★) **MIXTO** Decida si ese grafo tiene circuito de *Euler*, ruta de *Euler* y circuito de *Hamilton*.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
H = Grafo[Ar];
{CircuitoEulerQ[H], RutaEulerQ[H], CircuitoHamiltonQ[H]}
```

```
Out[ ] =
```

```
{False, False, True}
```

Las seis valencias son impares, de modo que no hay ni circuito ni ruta de *Euler*: el criterio admite **a lo sumo dos** vértices impares. Sí hay circuito de *Hamilton*, por ejemplo $1 - 2 - 4 - 6 - 5 - 3 - 1$.

- 5.6.61** (★★★) **A MANO** ¿Cuántas aristas hay que añadir al grafo anterior para que tenga circuito de *Euler*?

SOLUCIÓN Hay seis vértices de grado impar y cada arista nueva cambia la paridad de **dos** vértices. Con tres aristas bien escogidas (que emparejen los seis impares sin repetir aristas existentes) todas las valencias quedan pares. Por ejemplo $\{1, 4\}$, $\{2, 5\}$ y $\{3, 6\}$: el grafo pasa a ser $K_{3,3}$ más un triángulo, con todas las valencias iguales a 4.

- 5.6.62** (★★★) **MIXTO** Escriba la matriz de adyacencia de ese grafo y calcule su cuadrado.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True]
```

```
Out[ ] =
```

```
{{0, 1, 1, 0, 0, 1}, {1, 0, 1, 1, 0, 0}, {1, 1, 0, 0, 1, 0},
 {0, 1, 0, 0, 1, 1}, {0, 0, 1, 1, 0, 1}, {1, 0, 0, 1, 1, 0}}
```

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
MatrixPower[MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True], 2]
```

```
Out[ ] =
```

$\{\{3, 1, 1, 2, 2, 0\}, \{1, 3, 1, 0, 2, 2\}, \{1, 1, 3, 2, 0, 2\},$
 $\{2, 0, 2, 3, 1, 1\}, \{2, 2, 0, 1, 3, 1\}, \{0, 2, 2, 1, 1, 3\}\}$

La diagonal del cuadrado vale 3 en todas las posiciones: es la valencia de cada vértice, porque ir y volver por la misma arista es una ruta de longitud dos.

5.6.63 (★★★) **A MANO** Explique por qué la diagonal de M_A^2 contiene siempre las valencias.

SOLUCIÓN La entrada (i, i) de M_A^2 cuenta los caminos de longitud dos de v_i a sí mismo, que son exactamente **ir hasta un vecino y volver**. Hay uno por cada vecino, de modo que la entrada vale $\theta(v_i)$. La traza de M_A^2 es entonces $\sum \theta(v_i) = 2p$, el doble del número de aristas.

5.6.64 (★★) **MIXTO** Cuenten las rutas simples del vértice 1 al 6 en el grafo de referencia.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
CantRutas[Grafo[Ar], 1, 6]
```

Out[] =

9

Nueve rutas, incluida la arista directa $\{1, 6\}$. En un grafo 3-regular con seis vértices el número de caminos crece rápido pese al tamaño modesto.

5.6.65 (★★) **MIXTO** Tabulen el número de aristas de K_n para n de 3 a 8.

SOLUCIÓN

In[] :=

```
Table[CantAristas[GrafoCompleto[k]], {k, 3, 8}]
```

Out[] =

$\{3, 6, 10, 15, 21, 28\}$

Son los números **triangulares**: $\frac{n(n-1)}{2}$. Cada vértice nuevo aporta tantas aristas como vértices había antes.

5.6.66 (★★★) **MIXTO** Determine cuántos nodos tiene un grafo completo con 210 aristas.

SOLUCIÓN

In[] :=

```
Solve[n (n - 1)/2 == 210 && n > 0, n]
```

Out[] =

$\{\{n \rightarrow 21\}\}$

Es K_{21} . La ecuación $n^2 - n - 420 = 0$ tiene la raíz positiva 21; la otra, -20 , se descarta por no ser un número de vértices.

5.6.67 (★★) **MIXTO** Tabulen las aristas de $K_{n,n}$ para n de 2 a 6 y compare con K_n .

SOLUCIÓN

In[] :=

```
Table[CantAristas[GrafoBipartitoCompleto[k, k]], {k, 2, 6}]
```

Out[] =

```
{4, 9, 16, 25, 36}
```

Son los cuadrados perfectos: $K_{n,n}$ tiene n^2 aristas frente a las $\frac{2n(2n-1)}{2}$ de K_{2n} , que con el mismo número de vértices son muchas más. El bipartito prohíbe las aristas dentro de cada lado.

5.6.68 (★★) MIXTO Compare el número de aristas del ciclo, la estrella y la rueda.

SOLUCIÓN

In[] :=

```
{Table[CantAristas[GrafoCiclo[k]], {k, 3, 8}],  
Table[CantAristas[GrafoEstrella[k]], {k, 3, 8}],  
Table[CantAristas[GrafoRueda[k]], {k, 4, 8}]}
```

Out[] =

```
{{3, 4, 5, 6, 7, 8}, {2, 3, 4, 5, 6, 7}, {6, 8, 10, 12, 14}}
```

El ciclo tiene n aristas, la estrella $n - 1$ y la rueda $2(n - 1)$: la rueda es un ciclo de $n - 1$ vértices más un centro unido a todos.

5.6.69 (★★★) MIXTO ¿Para qué valores de n tiene K_n circuito de *Euler*?

SOLUCIÓN

In[] :=

```
Table[CircuitoEulerQ[GrafoCompleto[k]], {k, 3, 8}]
```

Out[] =

```
{True, False, True, False, True, False}
```

Solo para n impar. En K_n todas las valencias valen $n - 1$, que es par exactamente cuando n es impar. El patrón alterna a partir de $n = 3$.

5.6.70 (★★) MIXTO ¿Y circuito de *Hamilton*?

SOLUCIÓN

In[] :=

```
Table[CircuitoHamiltonQ[GrafoCompleto[k]], {k, 3, 7}]
```

Out[] =

```
{True, True, True, True, True}
```

Siempre, para todo $n \geq 3$: como todos los vértices están unidos entre sí, cualquier orden de visita sirve. Es la diferencia de fondo entre los dos tipos de circuito.

5.6.71 (★★★) MIXTO Cuente los circuitos de *Hamilton* distintos de K_n para n de 3 a 8.

SOLUCIÓN

```
In[ ] :=
```

```
Table[(k - 1)!/2, {k, 3, 8}]
```

```
Out[ ] =
```

```
{1, 3, 12, 60, 360, 2520}
```

Son $\frac{(n-1)!}{2}$: se fija el vértice de partida, se permutan los restantes y se divide entre dos porque un circuito y su inverso son el mismo. Este crecimiento factorial es la razón de que el agente viajero sea tan caro.

5.6.72 (★★★) **MIXTO** ¿Para qué valores de n tiene $K_{n,4}$ circuito de Euler?

SOLUCIÓN

```
In[ ] :=
```

```
Table[CircuitoEulerQ[GrafoBipartitoCompleto[k, 4]], {k, 2, 5}]
```

```
Out[ ] =
```

```
{True, False, True, False}
```

Solo cuando n es par. En $K_{n,m}$ los vértices de un lado tienen valencia m y los del otro n : con $m = 4$ ya es par, y basta que n también lo sea.

5.6.73 (★★) **MIXTO** Escriba la matriz de incidencia del grafo con aristas $\{1,2\}$, $\{1,3\}$, $\{2,3\}$ y $\{3,4\}$.

SOLUCIÓN

```
In[ ] :=
```

```
MIGrafo[Grafo[{{1, 2}, {1, 3}, {2, 3}, {3, 4}}], {1, 2, 3, 4},  
matriz -> True]
```

```
Out[ ] =
```

```
{{1, 1, 0, 0}, {1, 0, 1, 0}, {0, 1, 1, 1}, {0, 0, 0, 1}}
```

Cuatro filas (los vértices) y cuatro columnas: las aristas. La suma de la fila 3 vale 3, la valencia de ese vértice, y cada columna suma 2.

5.6.74 (★★★) **A MANO** ¿Qué información da la matriz de incidencia que no da la de adyacencia?

SOLUCIÓN Distingue las **aristas múltiples**: dos aristas paralelas dan dos columnas iguales, mientras que en la matriz de adyacencia solo se ve un 2 en una entrada. En cambio la de incidencia es peor para responder *¿son adyacentes u y v ?*, que en la de adyacencia se lee de inmediato. Cada representación responde bien a preguntas distintas.

5.6.75 (★★★) **MIXTO** Halle el camino más corto entre 1 y 6 en el grafo ponderado y compárelo con el número de aristas.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
LongitudCaminoMC[Grafo[Ar, pesos -> Pe], 1, 6]
```

Out[] =

```
{8., {{1, 6}}}
```

El camino más corto es la arista directa, de peso 8, aunque el grafo ofrezca rutas con más aristas. Aquí coinciden el mínimo de aristas y el mínimo de peso, cosa que no siempre ocurre.

5.6.76 (★★★) **MIXTO** Halle el camino más corto entre 1 y 5 en el mismo grafo ponderado.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
LongitudCaminoMC[Grafo[Ar, pesos -> Pe], 1, 5]
```

Out[] =

```
{8., {{1, 2}, {2, 4}, {4, 5}}}
```

La ruta óptima usa **tres** aristas y pesa $3 + 4 + 1 = 8$, mientras que la ruta de dos aristas $1 - 3 - 5$ pesa $5 + 6 = 11$. Es el caso contrario al anterior: menos aristas no significa menos peso.

5.6.77 (★★) **MIXTO** Resuelva el problema del agente viajero sobre ese grafo ponderado.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
AgenteViajero[Grafo[Ar, pesos -> Pe]][[1]]
```

Out[] =

```
22
```

El circuito de *Hamilton* de menor peso vale 22. De este comando debe tomarse solo la longitud: su segunda componente puede listar aristas inexistentes.

5.6.78 (★★★) **A MANO** ¿Cuántos circuitos de *Hamilton* tendría que examinar una búsqueda exhaustiva en K_{12} ?

SOLUCIÓN Serían $\frac{11!}{2} = 19\,958\,400$, casi veinte millones. Con K_{20} la cifra sube a $6 \cdot 10^{16}$. Ese crecimiento factorial, y no la dificultad de evaluar cada circuito, es lo que hace intratable el problema.

5.6.79 (★★★) **A MANO** Un grafo tiene 10 vértices y todas sus valencias valen 4. ¿Cuántas aristas tiene y puede tener circuito de *Euler*?

SOLUCIÓN Por el teorema de las valencias, $10 \cdot 4 = 2p$, de donde $p = 20$ aristas. Todas las valencias son pares, de modo que **sí** puede tener circuito de *Euler*, siempre que además sea conexo. La paridad es necesaria pero no suficiente: hay grafos 4-regulares desconectados.

5.6.80 (★★★) **A MANO** ¿Existe un grafo con 7 vértices todos de valencia 3?

SOLUCIÓN No. La suma de valencias sería 21, impar, y siempre debe valer $2p$. Es un caso del resultado general: en todo grafo el número de vértices de grado impar es par, de modo que no puede haber siete vértices impares.

5.6.81 (★★) **A MANO** ¿Cuál es el número máximo de aristas de un grafo simple con n vértices, y cuál el mínimo si debe ser conexo?

SOLUCIÓN El máximo es $\frac{n(n-1)}{2}$, alcanzado por K_n ; el mínimo conexo es $n-1$, alcanzado por cualquier árbol. Entre esos dos extremos caben todos los grafos conexos, y la diferencia $p - (n-1)$ mide cuántos circuitos independientes tiene.

5.6.82 (★★★) **A MANO** Demuestre que en todo grafo simple con al menos dos vértices hay dos con la misma valencia.

SOLUCIÓN Con n vértices las valencias posibles son $0, 1, \dots, n-1$: n valores para n vértices. Pero 0 y $n-1$ **no pueden coexistir**: si uno está aislado, ninguno se une a todos. Quedan $n-1$ valores posibles para n vértices, y por el principio del palomar dos comparten valencia.

5.6.83 (★★★) **A MANO** Un grafo conexo con n vértices tiene exactamente n aristas. ¿Qué se puede afirmar de su estructura?

SOLUCIÓN Tiene **exactamente un circuito**. Un árbol tiene $n-1$ aristas; añadir una más crea un ciclo, y solo uno. El grafo es entonces un ciclo del que cuelgan árboles. Es la estructura que en teoría de grafos se llama *unicíclica*.

5.6.84 (★★) **A MANO** ¿Cuántas componentes conexas tiene un grafo con 10 vértices y ninguna arista? ¿Y uno con 10 vértices y 9 aristas?

SOLUCIÓN El primero tiene **diez** componentes, cada vértice la suya. El segundo puede tener una sola, si es un árbol, o más si algunas aristas forman circuitos y dejan vértices sueltos; por ejemplo un triángulo más un camino de seis vértices más un aislado. El número de aristas acota pero no determina la conexión.

5.6.85 (★★★) **A MANO** Demuestre que un grafo con n vértices y más de $\frac{(n-1)(n-2)}{2}$ aristas es conexo.

SOLUCIÓN Si no lo fuera, se partiría en dos partes con k y $n-k$ vértices, y el máximo de aristas sería $\frac{k(k-1)}{2} + \frac{(n-k)(n-k-1)}{2}$. Ese valor se maximiza en $k=1$ o $k=n-1$, donde vale $\frac{(n-1)(n-2)}{2}$. Superar esa cota obliga a que el grafo sea conexo.

5.6.86 (★★★) **A MANO** Aplique la cota anterior a un grafo con 8 vértices. ¿A partir de cuántas aristas se garantiza la conexión?

SOLUCIÓN La cota vale $\frac{7 \cdot 6}{2} = 21$, de modo que con 22 aristas o más el grafo es forzosamente conexo. Con 21 puede no serlo: basta K_7 más un vértice aislado. La cota es **ajustada**.

5.6.87 (★★★) **A MANO** Un grafo bipartito no puede tener circuitos de longitud impar. Justifíquelo.

SOLUCIÓN Todo recorrido alterna entre los dos lados, de modo que tras un número impar de aristas se está en el lado opuesto al de partida. Para **cerrar** el circuito hay que volver al lado inicial, y eso exige una cantidad par de pasos. Por eso $K_{3,3}$ no tiene triángulos.

5.6.88 (★★) **A MANO** ¿Es bipartito el ciclo C_5 ? ¿Y el C_6 ?

SOLUCIÓN C_5 **no** es bipartito: es un circuito de longitud impar. C_6 sí, con los vértices alternos en cada lado. En general C_n es bipartito exactamente cuando n es par, que es el caso particular más simple del resultado anterior.

5.6.89 (★★★) **A MANO** Un grafo tiene matriz de adyacencia con la diagonal no nula. ¿Qué significa y cómo afecta a la valencia?

SOLUCIÓN Significa que hay **lazos**: aristas que unen un vértice consigo mismo. Un lazo aporta **dos** al grado de su vértice, porque incide dos veces en él. Por eso el teorema de las valencias sigue valiendo, y por eso la matriz de un grafo simple tiene siempre diagonal nula.

5.6.90 (★★★) **MIXTO** Compruebe que el grafo de referencia es 3-regular y calcule su número de aristas por dos vías.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
H = Grafo[Ar];
{Union[Valencias[H, table -> False]], Total[Valencias[H, table -> False]]/2,
 CantAristas[H]}
```

Out[] =

```
{{3}, 9, 9}
```

Todas las valencias valen 3 (la lista de valores distintos tiene un solo elemento) y las dos maneras de contar aristas coinciden en 9.

5.4 Camino más corto

5.6.91 (★) **A MANO** ¿Qué calcula el algoritmo de *Dijkstra* y qué necesita el grafo para poder aplicarlo?

SOLUCIÓN Calcula la longitud de un camino más corto entre dos vértices, y una ruta que la alcanza. El grafo debe ser conexo y ponderado, y en la versión del comando, sin pesos negativos.

5.6.92 (★★) **VilCretas** Halle el camino más corto del vértice 1 al 3 en el grafo de referencia sin pesos.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
LongitudCaminoMC[G, 1, 3]
```

Out[] =

```
{1, {{1, 3}}}
```

Sin pesos, la longitud se mide en número de aristas: la diagonal une los dos vértices directamente y por eso la longitud es 1.

5.6.93 (★★) **VilCretas** Repita el cálculo sobre el mismo grafo, ahora ponderado.

SOLUCIÓN**In[] :=**

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
LongitudCaminoMC[GP, 1, 3]
```

Out[] =

```
{7., {{1, 2}, {2, 3}}}
```

La diagonal es ahora la arista más cara, con peso 10, y hay que rodear por el vértice 2: $4 + 3 = 7$. El camino con menos aristas **no** es siempre el más corto.

5.6.94 (★★) **VilCretas** Calcule el peso de la ruta $1 \rightarrow 2 \rightarrow 3$ en el grafo ponderado.

SOLUCIÓN**In[] :=**

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
PesoRuta[GP, {{1, 2}, {2, 3}}]
```

Out[] =

```
7
```

PesoRuta suma los pesos de las aristas indicadas. Con **ruta->True** resalta el trayecto sobre el dibujo del grafo.

5.6.95 (★★★) **VilCretas** Ejecute paso a paso el algoritmo de *Dijkstra* entre los vértices 1 y 3.

SOLUCIÓN**In[] :=**

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
AlDijkstra[GP, 1, 3]
```

La salida informa que la longitud más corta es 7, presenta la tabla de iteraciones con las etiquetas provisionales de cada vértice y devuelve la ruta $\{1, 2\}, \{2, 3\}$ junto con su animación. Con **all->True** se despliega el procedimiento completo y con **caminos->True** todas las rutas óptimas.

5.6.96 (★★★) **A MANO** Aplique *Dijkstra* a mano sobre el grafo ponderado y explique las etiquetas.

SOLUCIÓN Se parte con etiqueta 0 en el vértice 1 e infinito en los demás. Desde 1 se etiquetan 2 con 4, 4 con 5 y 3 con 10. Se fija el menor, el vértice 2, y desde él se mejora la etiqueta de 3: $4 + 3 = 7 < 10$. Se fija 4 con 5, que no mejora nada, y queda 3 con etiqueta definitiva 7. La ruta se reconstruye hacia atrás: 3 vino de 2 y 2 de 1.

5.6.97 (★) **A MANO** ¿En qué consiste el problema del agente viajero?

SOLUCIÓN En hallar, sobre un grafo ponderado, un circuito de *Hamilton* de longitud total mínima: visitar todos los vértices una sola vez y volver al punto de partida gastando lo menos posible.

5.6.98 (★★) **VilCretas** Resuelva el problema del agente viajero sobre el grafo ponderado de referencia.

SOLUCIÓN

In[] :=

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
AgenteViajero[GP]
```

Out[] =

```
{14, {{1, 4}, {4, 3}, {3, 2}, {2, 1}}}
```

El circuito recorre $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ con un costo de $5 + 2 + 3 + 4 = 14$, y evita la diagonal de peso 10. Con `ruta->True` se obtiene la animación del circuito.

5.6.99 (★★★) **A MANO** ¿Por qué el problema del agente viajero es computacionalmente costoso?

SOLUCIÓN Porque el número de circuitos de *Hamilton* por examinar crece como el factorial del número de vértices: en K_n hay $\frac{(n-1)!}{2}$ circuitos distintos. Con el vocabulario del capítulo 3, una búsqueda exhaustiva es de orden $O(n!)$, impracticable ya para valores moderados de n .

5.6.100 (★★★) **MIXTO** Compare la ruta con menos aristas y la ruta de menor peso entre los vértices 1 y 3.

SOLUCIÓN

In[] :=

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
{PesoRuta[GP, {{1, 3}}], PesoRuta[GP, {{1, 2}, {2, 3}}],
PesoRuta[GP, {{1, 4}, {4, 3}}]}
```

Out[] =

```
{10, 7, 7}
```

La ruta directa usa una sola arista pero cuesta 10; las otras dos usan dos aristas y cuestan 7. Hay por lo tanto **dos** caminos más cortos, y el comando devuelve solo uno de ellos.

5.6.101 (★★) **MIXTO** Halle el camino más corto entre los vértices opuestos del ciclo de cuatro vértices.

SOLUCIÓN

In[] :=

```
H = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}}];
LongitudCaminoMC[H, 1, 3]
```

Out[] =

```
{2, {{1, 2}, {2, 3}}}
```

Sin pesos la longitud cuenta aristas: hacen falta dos, y hay dos caminos igual de buenos. El comando devuelve solo uno de ellos.

5.6.102 (★★★) **A MANO** Un grafo tiene 6 vértices con valencias 5, 3, 3, 3, 3, 3. ¿Cuántas aristas tiene y por qué no puede ser simple sin lazos?

SOLUCIÓN La suma es 20, de modo que tiene 10 aristas. Sí puede ser simple: el vértice de grado 5 se une a los otros cinco, y entre los cinco restantes faltan 2 de grado a cada uno, lo que se logra con un ciclo de longitud cinco. El resultado es la **rueda** W_6 .

5.6.103 (★★) **MIXTO** Compruebe que la rueda de seis vértices tiene esas valencias.

SOLUCIÓN

In[] :=

```
Valencias[GrafoRueda[6], table -> False]
```

Out[] =

```
{5, 3, 3, 3, 3, 3}
```

El centro tiene grado 5 y los cinco radios grado 3: dos del ciclo y uno del rayo. Coincide con lo deducido a mano.

5.6.104 (★★★) **A MANO** ¿Tiene la rueda W_n circuito de *Euler*? Analice según la paridad.

SOLUCIÓN El centro tiene grado $n-1$ y los demás grado 3, siempre impar. Como hay $n-1 \geq 3$ vértices de grado impar, **nunca** hay circuito ni ruta de *Euler* para $n \geq 4$. La rueda sí tiene siempre circuito de *Hamilton*: basta recorrer el ciclo exterior.

5.6.105 (★★) **MIXTO** Compruebe la afirmación anterior sobre dos ruedas.

SOLUCIÓN

In[] :=

```
{CircuitoEulerQ[GrafoRueda[5]], CircuitoEulerQ[GrafoRueda[6]],  
CircuitoHamiltonQ[GrafoRueda[6]]}
```

Out[] =

```
{False, False, True}
```

Ninguna de las dos ruedas tiene circuito de *Euler* y ambas tienen de *Hamilton*, tal como predice el análisis de paridades.

5.6.106 (★★★) **A MANO** Aplique las tres condiciones suficientes de *Hamilton* al grafo de referencia 3-regular con seis vértices.

SOLUCIÓN Con $n = 6$ y $p = 9$: la condición de aristas exige $p \geq \frac{36-18+6}{2} = 12$, y $9 < 12$: **falla**. *Dirac* exige $\theta \geq 3$, y todas valen exactamente 3: **se cumple**. No hace falta comprobar *Ore*. El grafo tiene circuito de *Hamilton*, y basta una de las tres condiciones para garantizarlo.

5.6.107 (★★★) **MIXTO** Compruebe con el software el análisis anterior.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},  
{4, 6}, {5, 6}, {1, 6}};
```

```
ExisteCircuitoHamilton[Grafo[Ar], vectorial -> True]
```

Out[] =

```
{True, True, False, True}
```

Las componentes son *Ore*, *Dirac*, cantidad de aristas y existencia. La tercera falla, como se calculó, y las otras dos bastan.

5.6.108 (★★★) **A MANO** ¿Qué se concluye si un grafo falla las tres condiciones de *Hamilton*?

SOLUCIÓN **Nada**: son condiciones **suficientes**, no necesarias. El ciclo C_6 falla las tres (todas sus valencias son 2, muy por debajo de $n/2 = 3$) y sin embargo *es* un circuito de *Hamilton*. La única respuesta correcta en ese caso es que no hay criterio.

5.6.109 (★★) **MIXTO** Compruebe el caso del ciclo C_6 .

SOLUCIÓN

```
In[ ] :=
```

```
{ExisteCircuitoHamilton[GrafoCiclo[6], vectorial -> True],  
CircuitoHamiltonQ[GrafoCiclo[6]]}
```

```
Out[ ] =
```

```
{{False, False, False, True}, True}
```

Las tres primeras componentes (*Ore*, *Dirac* y cantidad de aristas) son **False**: ningún criterio se cumple. Y sin embargo la cuarta, que reporta la existencia, es **True**. Es el contraejemplo más simple de que los criterios son **suficientes y no necesarios**.

5.6.110 (★★★) **A MANO** Calcule cuántas aristas exige la condición de cantidad de aristas para $n = 10$ y compárela con el total posible.

SOLUCIÓN Exige $p \geq \frac{100-30+6}{2} = 38$, mientras que K_{10} tiene 45. Es decir, la condición solo se activa cuando el grafo tiene más del 84 % de las aristas posibles: es una condición **muy exigente**, y por eso falla en la mayoría de los grafos que sí son hamiltonianos.

5.6.111 (★★) **VilCretas** Halle un circuito de *Hamilton* concreto del grafo de referencia.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},  
{4, 6}, {5, 6}, {1, 6}};  
CircuitosHamilton[Grafo[Ar], 2]
```

El comando devuelve hasta dos circuitos distintos. El primero recorre $1-3-2-4-5-6-1$, que usa seis de las nueve aristas y deja fuera $\{1, 2\}$, $\{3, 5\}$ y $\{4, 6\}$. Es además el de menor peso.

5.6.112 (★★★) **MIXTO** Verifique el circuito anterior calculando su peso en el grafo ponderado.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},  
{4, 6}, {5, 6}, {1, 6}};  
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
```

```
PesoRuta[Grafo[Ar, pesos -> Pe],
{{1, 3}, {3, 2}, {2, 4}, {4, 5}, {5, 6}, {6, 1}}]
```

```
Out[ ] =
```

```
22
```

Su peso es 22, que coincide con la longitud mínima que reporta **AgenteViajero**: es un circuito óptimo. Los otros dos circuitos de *Hamilton* del grafo pesan 27 cada uno.

- 5.6.113 (★★★) A MANO** En un grafo ponderado con todos los pesos iguales a c , ¿qué devuelve el agente viajero?

SOLUCIÓN Cualquier circuito de *Hamilton*, todos con peso nc . El problema se vuelve trivial en cuanto a optimización (todas las soluciones empatan) pero sigue siendo difícil **encontrar una**: decidir si existe circuito de *Hamilton* es en sí mismo el problema costoso.

- 5.6.114 (★★★) MIXTO** Cuente las rutas de longitud 3 entre los vértices 1 y 4 del grafo de referencia.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
```

```
MatrixPower[MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True], 3][[1, 4]]
```

```
Out[ ] =
```

```
3
```

Son tres recorridos de tres aristas, permitiendo repetir. La entrada (1,6) de esa misma potencia vale 7: la distancia entre vértices influye mucho en el conteo.

- 5.6.115 (★★★) A MANO** ¿Por qué M_A^k cuenta recorridos con repetición y no rutas simples?

SOLUCIÓN Porque el producto de matrices suma sobre **todos** los vértices intermedios, sin llevar memoria de cuáles ya se visitaron. El recorrido $1 \rightarrow 2 \rightarrow 1 \rightarrow 2$ se cuenta igual que $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. Contar rutas simples exige un algoritmo que recuerde el camino recorrido, y por eso **CantRutas** da otro número.

- 5.6.116 (★★) MIXTO** Compare ambos conteos entre 1 y 6 en el grafo de referencia.

SOLUCIÓN

```
In[ ] :=
```

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
```

```
{CantRutas[Grafo[Ar], 1, 6],
```

```
MatrixPower[MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True], 3][[1, 6]]}
```

```
Out[ ] =
```

```
{9, 7}
```

Nueve rutas simples de cualquier longitud frente a siete recorridos de longitud exactamente tres con repetición: son cantidades distintas que responden a preguntas distintas.

- 5.6.117 (★★★) A MANO** Un digrafo tiene n vértices y todas sus valencias de salida valen 2. ¿Cuántas aristas tiene y cuánto suman las valencias de entrada?

SOLUCIÓN Tiene $2n$ aristas, porque cada vértice aporta dos salidas. Las valencias de entrada suman lo mismo, $2n$: cada arista tiene exactamente un origen y un destino. Lo que no se puede afirmar es que cada vértice tenga grado de entrada 2: podría haber uno que recibiera muchas y otro ninguna.

- 5.6.118 (★★) **A MANO** Defina grafo conexo y componente conexa, y diga cuántas componentes tiene un grafo vacío de n vértices.

SOLUCIÓN Un grafo es **conexo** si hay una ruta entre cualquier par de vértices; una **componente conexa** es un subgrafo conexo maximal. El grafo vacío de n vértices tiene n componentes, cada una con un solo vértice y ninguna arista.

- 5.6.119 (★★★) **A MANO** Un grafo tiene 12 vértices y 3 componentes conexas. ¿Cuál es el mínimo y el máximo número de aristas?

SOLUCIÓN El **mínimo** es $12 - 3 = 9$: cada componente con k vértices necesita al menos $k - 1$ aristas. El **máximo** se logra concentrando los vértices: dos componentes de un vértice y una de 10, que da $\frac{10 \cdot 9}{2} = 45$. En general el máximo es $\frac{(n-c+1)(n-c)}{2}$ con c componentes.

- 5.6.120 (★★★) **A MANO** Demuestre que en un grafo conexo con n vértices siempre existe una ruta que pasa por al menos dos vértices distintos, si $n \geq 2$.

SOLUCIÓN Sean $u \neq v$ dos vértices. Por ser conexo existe una ruta de u a v , y esa ruta pasa al menos por esos dos. La afirmación parece trivial pero delimita el caso $n = 1$, donde la única ruta es la vacía: la conexión de un grafo con un solo vértice se cumple por convenio.

- 5.6.121 (★★) **A MANO** Explique la diferencia entre *ruta*, *camino simple* y *circuito*.

SOLUCIÓN Una **ruta** es cualquier sucesión de aristas encadenadas, pudiendo repetir. Un **camino simple** no repite aristas ni vértices. Un **circuito** es una ruta cerrada, que empieza y termina en el mismo vértice. Las de *Euler* no repiten aristas y las de *Hamilton* no repiten vértices: cada tipo restringe una cosa distinta.

- 5.6.122 (★★★) **A MANO** Un grafo tiene exactamente dos vértices de grado impar, u y v . ¿Dónde empieza y termina su ruta de *Euler*?

SOLUCIÓN Empieza en uno de ellos y termina en el otro. En cualquier vértice intermedio la ruta entra y sale, consumiendo aristas de dos en dos, de modo que su grado debe ser par. Solo los extremos pueden tener grado impar, y por eso hay exactamente dos o ninguno.

- 5.6.123 (★★★) **MIXTO** Construya un grafo con exactamente dos vértices impares y halle su ruta de *Euler*.

SOLUCIÓN

In[] :=

```
Cam = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}];
{Valencias[Cam, table -> False], RutaEulerQ[Cam], RutaEuler[Cam]}
```

Out[] =

```
{{3, 2, 3, 2}, True, {{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}}}
```

Los vértices impares son 1 y 3, y en efecto la ruta empieza en 1 y termina en 3.

- 5.6.124 (★★) **A MANO** El problema de los puentes de *Königsberg* tiene cuatro zonas con grados 3, 3, 3, 5. ¿Por qué no tiene solución?

SOLUCIÓN Los cuatro grados son impares, y el criterio de *Euler* admite a lo sumo dos vértices impares para una ruta y ninguno para un circuito. Con cuatro es imposible recorrer todos los puentes una sola vez, sea cual sea el punto de partida. Es el resultado que fundó la teoría de grafos.

5.6.125 (★★★) **A MANO** ¿Cuántos puentes habría que añadir en *Königsberg* para que exista una ruta de *Euler*?

SOLUCIÓN Uno solo. Cada puente nuevo cambia la paridad de dos zonas, de modo que añadir uno entre dos de las cuatro zonas impares deja exactamente **dos** impares: justo lo que exige el criterio para una ruta. Para un *circuito* harían falta dos puentes.

5.6.126 (★★★) **MIXTO** Compruebe con *Fleury* el circuito de *Euler* de un grafo con todas las valencias pares.

SOLUCIÓN

In[] :=

```
Par = Grafo[{{1, 2}, {2, 3}, {3, 1}, {3, 4}, {4, 5}, {5, 3}}];
Fleury[Par]
```

El grafo son dos triángulos que comparten el vértice 3, con valencias 2, 2, 4, 2, 2. *Fleury* muestra cómo se construye el circuito arista por arista, evitando siempre las aristas de corte mientras haya alternativa.

5.6.127 (★★) **MIXTO** Compruebe las valencias y la existencia del circuito en ese grafo.

SOLUCIÓN

In[] :=

```
Par = Grafo[{{1, 2}, {2, 3}, {3, 1}, {3, 4}, {4, 5}, {5, 3}}];
{Valencias[Par, table -> False], CircuitoEulerQ[Par], CircuitoHamiltonQ[Par]}
```

Out[] =

```
{{2, 2, 4, 2, 2}, True, False}
```

Tiene circuito de *Euler* y no de *Hamilton*: cualquier recorrido que visite los cinco vértices debe pasar dos veces por el vértice 3.

5.6.128 (★★★) **A MANO** Generalice: ¿cuándo un grafo formado por dos ciclos que comparten un vértice tiene circuito de *Hamilton*?

SOLUCIÓN **Nunca**, si los dos ciclos tienen al menos tres vértices cada uno. El vértice compartido es un **punto de corte**: al quitarlo el grafo se desconecta, y un circuito de *Hamilton* tendría que entrar y salir de cada mitad pasando por él más de una vez. Ningún grafo con punto de corte es hamiltoniano.

5.6.129 (★★★) **MIXTO** Halle el camino más corto entre 2 y 6 en el grafo ponderado y compare con el de menos aristas.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
      {4, 6}, {5, 6}, {1, 6}};
```

```
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
{LongitudCaminoMC[Grafo[Ar, pesos -> Pe], 2, 6],
LongitudCaminoMC[Grafo[Ar], 2, 6]}
```

Out[] =

```
{{7., {{2, 4}, {4, 5}, {5, 6}}}, {2, {{2, 1}, {1, 6}}}}
```

Aquí **no** coinciden: con pesos la mejor ruta es $2-4-5-6$, de tres aristas y peso 7; sin pesos basta $2-1-6$, de dos aristas pero peso $3+8=11$. Optimizar aristas y optimizar peso son objetivos distintos.

5.6.130 (★★★) **VilCretas** Ejecute paso a paso *Dijkstra* entre 1 y 5 sobre el grafo ponderado.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
AlDijkstra[Grafo[Ar, pesos -> Pe], 1, 5, all -> True]
```

Con `all->True` se despliega la tabla completa de iteraciones: en cada paso se fija el vértice de etiqueta menor y se relajan sus vecinos. El resultado es la ruta $1-2-4-5$ con longitud 8.

5.6.131 (★★★) **A MANO** ¿Por qué *Dijkstra* falla con pesos negativos?

SOLUCIÓN Porque su invariante es que, al fijar un vértice, su etiqueta ya es definitiva: ningún camino posterior puede mejorarla, ya que añadir aristas solo suma. Con pesos negativos esa premisa se rompe (un rodeo puede **restar**) y el algoritmo puede fijar un vértice con una etiqueta que después resulta mejorable.

5.6.132 (★★★) **A MANO** ¿Cuántas comparaciones hace *Dijkstra* en el peor caso sobre un grafo con n vértices?

SOLUCIÓN En su versión básica, en cada una de las n iteraciones busca el mínimo entre los vértices no fijados (hasta n comparaciones) y relaja sus vecinos. El total es $\Theta(n^2)$. Con una cola de prioridad baja a $\Theta(p \ln n)$, mucho mejor en grafos poco densos.

5.6.133 (★★) **MIXTO** Compare el peso de tres rutas distintas entre 1 y 5 en el grafo ponderado.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}, {1, 6}};
Pe = {3, 5, 2, 4, 6, 1, 7, 2, 8};
GP = Grafo[Ar, pesos -> Pe];
{PesoRuta[GP, {{1, 3}, {3, 5}}], PesoRuta[GP, {{1, 2}, {2, 4}, {4, 5}}],
PesoRuta[GP, {{1, 6}, {6, 5}}]}
```

Out[] =

```
{11, 8, 10}
```


La de tres aristas es la más barata. Confirma que en un grafo ponderado el número de aristas no predice el costo.

5.6.134 (★★★) A MANO Un grafo ponderado tiene todos sus pesos distintos. ¿Es único el camino más corto entre dos vértices?

SOLUCIÓN No necesariamente. Dos rutas con aristas distintas pueden sumar lo mismo aunque los pesos individuales difieran: $1 + 4$ y $2 + 3$. Lo que sí es único con pesos todos distintos es el **árbol de expansión mínima**, que es un resultado del capítulo siguiente.

5.6.135 (★★★) MIXTO Compruebe la observación anterior construyendo un ejemplo.

SOLUCIÓN

In[] :=

```
E1 = Grafo[{{1, 2}, {2, 4}, {1, 3}, {3, 4}}, pesos -> {1, 4, 2, 3}];
{PesoRuta[E1, {{1, 2}, {2, 4}}], PesoRuta[E1, {{1, 3}, {3, 4}}]}
```

Out[] =

{5, 5}

Los cuatro pesos son distintos y las dos rutas empatan en 5. *Dijkstra* devolvería solo una de ellas; con `caminos->True` se obtienen todas.

5.6.136 (★★) A MANO Resuma cuándo conviene cada representación de un grafo.

SOLUCIÓN La **lista de aristas** es la más compacta y la mejor para recorrer todo el grafo. La **matriz de adyacencia** responde en un paso si dos vértices son adyacentes y permite contar rutas con potencias, pero ocupa n^2 aunque haya pocas aristas. La **matriz de incidencia** es la única que distingue aristas múltiples. La elección depende de la pregunta que se quiera responder rápido.

5.5 Familias completas, generadores y variantes de *Dijkstra*

5.6.137 (★★) MIXTO ¿Para qué valores de n tiene K_n un circuito de *Euler*? Respóndalo con `KnCircuitoEulerQ` para n de 1 a 8.

SOLUCIÓN

In[] :=

```
Table[KnCircuitoEulerQ[n], {n, 1, 8}]
```

Out[] =

{True, False, True, False, True, False, True, False}

Es cierto exactamente para n **impar**. La razón es inmediata: en K_n todo vértice tiene grado $n - 1$, y un circuito de *Euler* existe si y solo si todos los grados son pares, es decir si $n - 1$ es par.

5.6.138 (★★★) MIXTO Haga lo mismo con `KnRutaEulerQ` y explique por qué el resultado es tan distinto.

SOLUCIÓN

In[] :=

```
Table[KnRutaEulerQ[n], {n, 1, 8}]
```

```
Out[ ] =
```

```
{False, True, False, False, False, False, False, False}
```

Solo K_2 tiene ruta euleriana abierta. Una ruta así exige **exactamente dos** vértices de grado impar; en K_n todos los grados son iguales, de modo que o son todos pares (y entonces hay circuito, no ruta abierta) o son todos impares, y hay n vértices impares, que solo vale 2 cuando $n = 2$.

5.6.139 (★★) **MIXTO** Compare ahora los dos predicados de *Hamilton* sobre K_n .

SOLUCIÓN

```
In[ ] :=
```

```
{Table[KnCircuitoHamiltonQ[n], {n, 1, 8}],  
Table[KnRutaHamiltonQ[n], {n, 1, 8}]}
```

```
Out[ ] =
```

```
{{False, False, True, True, True, True, True, True}, {False, True, True, True, True, True,  
True, True}}
```

Hay circuito desde $n = 3$ y ruta desde $n = 2$. En K_n cualquier ordenación de los vértices es una ruta de *Hamilton*, porque todo par está unido; para cerrarla en circuito hacen falta al menos tres vértices. Contraste con *Euler*: allí la respuesta depende de la paridad, aquí solo del tamaño.

5.6.140 (★★★) **MIXTO** Para el grafo bipartito completo $K_{n,m}$, determine con *KnmcircuitoEulerQ* cuándo hay circuito de *Euler*.

SOLUCIÓN

```
In[ ] :=
```

```
Table[KnmcircuitoEulerQ[n, m], {n, 1, 4}, {m, 1, 4}]
```

```
Out[ ] =
```

```
{{False, False, False, False}, {False, True, False, True}, {False, False, False, False},  
{False, True, False, True}}
```

Es cierto exactamente cuando **n y m son ambos pares**. En $K_{n,m}$ los vértices de un lado tienen grado m y los del otro grado n ; para que todos sean pares hacen falta las dos condiciones a la vez.

5.6.141 (★★★) **MIXTO** Determine con *KnmcircuitoHamiltonQ* cuándo $K_{n,m}$ tiene circuito de *Hamilton*.

SOLUCIÓN

```
In[ ] :=
```

```
Table[KnmcircuitoHamiltonQ[n, m], {n, 1, 4}, {m, 1, 4}]
```

```
Out[ ] =
```

```
{{False, False, False, False}, {False, True, False, False}, {False, False, True, False},  
{False, False, False, True}}
```

Solo en la diagonal, con $n = m \geq 2$. Un circuito de *Hamilton* alterna lados a cada paso, de modo que visita tantos vértices de un lado como del otro: si $n \neq m$ es imposible. Es un argumento de **paridad**, no de tamaño.

- 5.6.142 (★★★) **MIXTO** **Cuidado con KnmRutaEulerQ**. Compare lo que dice el comando con lo que dice el criterio de los grados sobre $K_{2,3}$.

SOLUCIÓN

In[] :=

```
k23 = Grafo[{{u1, v1}, {u1, v2}, {u1, v3},
             {u2, v1}, {u2, v2}, {u2, v3}}];
{Valencias[k23, table -> False], RutaEulerQ[k23], KnmRutaEulerQ[2, 3]}
```

Out[] =

```
{{3, 2, 2, 2, 3}, True, False}
```

$K_{2,3}$ tiene **exactamente dos** vértices de grado impar y es conexo, de modo que **sí tiene ruta euleriana**: así lo confirma `RutaEulerQ`. Sin embargo `KnmRutaEulerQ[2, 3]` responde `False`. Lo mismo ocurre en $K_{1,2}$, $K_{2,1}$ y $K_{3,2}$: el comando solo acierta en $K_{1,1}$. Use el **criterio de los grados o `RutaEulerQ`**, no este predicado.

- 5.6.143 (★★★) **A MANO** Enuncie usted el criterio correcto para la ruta euleriana en $K_{n,m}$.

SOLUCIÓN Hay ruta euleriana abierta si y solo si el grafo es conexo y tiene **exactamente dos** vértices de grado impar. En $K_{n,m}$ los grados son m (en el lado de n vértices) y n (en el otro), de modo que la condición se cumple cuando uno de los dos lados tiene **un solo vértice de grado impar** y el otro exactamente uno más: en la práctica, cuando $\{n, m\} = \{1, 1\}, \{1, 2\}, \{2, 1\}, \{2, 3\}$ o $\{3, 2\}$.

- 5.6.144 (★★) **MIXTO** `MGrafo` devuelve por omisión un objeto de presentación. Compruebe qué cambia con `matriz -> True`.

SOLUCIÓN

In[] :=

```
g = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}}];
{Head[MGrafo[g, {a, b, c, d}]],
 MGrafo[g, {a, b, c, d}, matriz -> True]}
```

Out[] =

```
{MatrixForm, {{0, 1, 1, 0}, {1, 0, 1, 1}, {1, 1, 0, 1}, {0, 1, 1, 0}}}
```

Sin la opción el resultado es un `MatrixForm`: se *ve* como una matriz pero **no se puede calcular con él**. Con `matriz -> True` se obtiene la lista de listas, que ya admite `Transpose`, `MatrixPower` o `ProductoBooleano`.

- 5.6.145 (★★) **MIXTO** Obtenga con `MIGrafo` la matriz de incidencia del mismo grafo y compruebe que cada columna suma 2.

SOLUCIÓN

In[] :=

```
g = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}}];
Total[MIGrafo[g, {a, b, c, d}, matriz -> True]]
```

Out[] =

```
{2, 2, 2, 2, 2}
```

Cada columna representa una arista y tiene un 1 en cada uno de sus dos extremos. Que todas sumen 2 confirma que el grafo **no tiene lazos**: un lazo daría una columna con un solo 1, o con un 2, según el convenio.

5.6.146 (★★★) **MIXTO** Con `MPGrafo` obtenga la matriz de pesos y compruebe que es simétrica.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
  pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
Mp = MPGrafo[gp, {a, b, c, d}, matriz -> True];
{Mp, Mp == Transpose[Mp]}
```

Out[] =

```
{{{0, 2, 5, 0}, {2, 0, 1, 4}, {5, 1, 0, 3}, {0, 4, 3, 0}}, True}
```

La simetría es la traducción matricial de que el grafo **no es dirigido**. Los ceros fuera de la diagonal significan *no hay arista*, *no arista de peso cero*: por eso estos comandos exigen que ningún peso sea nulo.

5.6.147 (★★) **MIXTO** Use `AlDijkstraValor` para obtener solo la longitud del camino más corto de a a d en ese grafo.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
  pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
AlDijkstraValor[gp, a, d]
```

Out[] =

```
6
```

Frente a `AlDijkstra`, que despliega el procedimiento paso a paso, este comando devuelve **solo el número**, de modo que se puede encadenar. Hay dos caminos de peso 6: $a-b-d$ y $a-b-c-d$. El más corto en peso no tiene por qué ser el más corto en número de aristas.

5.6.148 (★★★) **MIXTO** Obtenga ahora la longitud del camino más *largo* con `AlDijkstraMaxValor` e interprete la diferencia.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
  pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
{AlDijkstraValor[gp, a, d], AlDijkstraMaxValor[gp, a, d]}
```

Out[] =

{6, 8}

El más largo es $a-c-d$, con peso $5 + 3 = 8$. La variante *max* escoge en cada paso la marca mayor en lugar de la menor. Debe advertirse que el problema del camino más largo **no** es, en general, tan fácil como el del más corto: esta versión funciona sobre grafos simples y conexos, pero no es un algoritmo general.

5.6.149 (★★★) **VilCretas** Vea el procedimiento completo de *Dijkstra* con las opciones `withX` y `caminos`.

SOLUCIÓN

`In[ ] :=`

```
gp = Grafo[{a, b}, {a, c}, {b, c}, {b, d}, {c, d},
  pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
AlDijkstra[gp, a, d, withX -> True, caminos -> True]
```

`withX -> True` marca con una equis los vértices ya cerrados en cada paso, que es como se hace la tabla a mano; `camino -> True` añade el camino que realiza cada marca, no solo su valor. Juntas convierten la salida en la misma tabla que se pide en un examen.

5.6.150 (★★) **VilCretas** Genere con `GrafoRandomEuler` un grafo que tenga circuito o ruta euleriana, y compruébelo.

SOLUCIÓN

`In[ ] :=`

```
gr = GrafoRandomEuler[13, 20, letras -> True, pesos -> True, maxPeso -> 9];
{CircuitoEulerQ[gr], RutaEulerQ[gr]}
```

El generador construye el grafo **con la propiedad ya garantizada**, de modo que sirve para practicar *Fleury* sin tener que buscar un grafo válido. Las opciones `letras`, `pesos` y `maxPeso` controlan el aspecto del enunciado.

5.6.151 (★★) **VilCretas** Haga lo análogo con `GrafoRandomHamilton`.

SOLUCIÓN

`In[ ] :=`

```
gr = GrafoRandomHamilton[13, 20, letras -> True];
{CircuitoHamiltonQ[gr], RutaHamiltonQ[gr]}
```

Aquí la garantía es más valiosa que en el caso euleriano: no existe un criterio sencillo para decidir si un grafo cualquiera tiene circuito de *Hamilton*, de modo que generar uno al azar y esperar que lo tenga sería poco práctico.

5.6.152 (★★★) **VilCretas** Genere un multigrafo ponderado con lazos y aristas paralelas usando las opciones nuevas de `GrafoRandom`.

SOLUCIÓN

`In[ ] :=`

```
GrafoRandom[8, 10, simple -> False, letras -> True,
  pesos -> True, maxPeso -> 100, lazos -> 3, paralelos -> 2]
```

`simple -> False` es la que **habilita** lazos y paralelos; `lazos` y `paralelos` dicen cuántos. Sin `simple -> False` las otras dos no tienen efecto, que es un descuido fácil de cometer.

- 5.6.153** (★★) **VilCretas** Use `GrafoRandomConexo` para asegurar la conexidad, y explique por qué importa aquí.

SOLUCIÓN

`In[ ] :=`

```
GrafoRandomConexo[8, 10, letras -> True, pesos -> True, maxPeso -> 50]
```

Casi todo lo interesante del capítulo (*Euler*, *Hamilton*, *Dijkstra*, árboles generadores) **exige un grafo conexo**. Un grafo aleatorio con pocas aristas se desconecta con facilidad, y entonces los comandos fallan o devuelven resultados sin sentido.

- 5.6.154** (★★) **VilCretas** Dibuje un grafo con los vértices dentro de discos, con `shape` y `radio`, y colórelo con `ColorearGrafo`.

SOLUCIÓN

`In[ ] :=`

```
g = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
  shape -> True, radio -> 0.08];
ColorearGrafo[g, radio -> 0.3, color -> Orange]
```

`radio` regula el tamaño de los discos: en un grafo con muchos vértices, un radio grande los solapa y uno pequeño vuelve ilegibles las etiquetas. `ColorearGrafo` colorea los nodos, lo que ayuda a seguir un recorrido cuando se explica en clase.

- 5.6.155** (★★) **MIXTO** `ExisteCircuitoHamilton` imprime su respuesta. Consiga que la *devuelva* para poder usarla en un cálculo.

SOLUCIÓN

`In[ ] :=`

```
g = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}}];
ExisteCircuitoHamilton[g, vectorial -> True]
```

`Out[ ] =`

```
{True, True, True, True}
```

Sin la opción el comando devuelve `Null` y solo escribe en pantalla: encadenarlo daría un resultado sin sentido, **sin ningún mensaje de error**. Con `vectorial -> True` devuelve datos. Es el mismo patrón de `matriz -> True` en `MGrafo` y de `table -> False` en `Valencias`: conviene preguntarse siempre si un comando de *VilCretas* *muestra* o *devuelve*.

- 5.6.156** (★) **VilCretas** Dibuje con `GrafoString` el grafo que representa una frase.

SOLUCIÓN

`In[ ] :=`

```
GrafoString["Estructuras discretas"]
```

Es un ejemplo de grafo con muchos vértices y aristas construido a partir de un texto. Sirve para ver cómo se degrada la legibilidad de un dibujo cuando el orden del grafo crece: a partir de cierto tamaño, la matriz es más útil que la figura.

5.6.157 (★★) VilCretas Dibuje el mismo grafo partiendo de sus tres matrices, con **GrafoM**, **GrafoMI** y **GrafoMP**, ajustando el radio de los discos.

SOLUCIÓN

In[] :=

```
A = {1, x, 2, y};
M = {{0, 1, 1, 0}, {1, 0, 1, 1}, {1, 1, 0, 1}, {0, 1, 1, 0}};
MI = {{1, 1, 0, 0, 0}, {1, 0, 1, 1, 0}, {0, 1, 1, 0, 1}, {0, 0, 0, 1, 1}};
MP = {{0, 2, 5, 0}, {2, 0, 1, 4}, {5, 1, 0, 3}, {0, 4, 3, 0}};
{GrafoM[M, A, shape -> True, radio -> 0.06],
 GrafoMI[MI, A, shape -> True, radio -> 0.06],
 GrafoMP[MP, A, mostrarpesos -> True, shape -> True, radio -> 0.06]}
```

Los tres comandos hacen el camino inverso de **MGrafo**, **MIgrafo** y **MPgrafo**: de la matriz al dibujo. El **radio** hay que ajustarlo a mano según el orden del grafo; con etiquetas largas, como **x** y **y** mezcladas con números, un radio de 0.06 deja el texto dentro del disco. Con quince vértices habría que bajarlo.

5.6.158 (★★★) VilCretas Vea el procedimiento completo del camino más largo con **AlDijkstraMax**, y compárelo con el del más corto.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
 pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
{AlDijkstra[gp, a, d], AlDijkstraMax[gp, a, d]}
```

El esqueleto es el mismo: marcar, escoger, actualizar. Lo que cambia son dos detalles (se escoge la marca **mayor** y se actualiza con el valor **mayor**) y con eso el algoritmo pasa de buscar el camino más corto a buscar el más largo. La opción **all -> True** despliega el procedimiento entero de una vez.

CAPÍTULO 6 • COMPLEMENTARIOS

Teoría de árboles

6.1 Árboles: propiedades, altura y hojas

6.7.1 (★) **A MANO** Defina qué es un árbol y enuncie dos caracterizaciones equivalentes.

SOLUCIÓN Un **árbol** es un grafo conexo sin circuitos. Equivalen a esta definición, entre otras: que sea conexo y tenga exactamente $n - 1$ aristas; y que entre cada par de vértices exista una **única** ruta.

6.7.2 (★) **VilCretas** Compruebe que el grafo de aristas $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$, $\{2, 5\}$ y $\{3, 6\}$ es un árbol.

SOLUCIÓN

In[] :=

```
T = Arbol[{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}];
ArbolJQ[T, vectorial -> True]
```

Out[] =

```
{True, True, False}
```

Las tres componentes son: si es árbol, si es conexo y si tiene circuitos. Es conexo y no tiene circuitos, de modo que es un árbol. Este será el árbol de referencia de los ejercicios siguientes.

6.7.3 (★) **MIXTO** Verifique en ese árbol la relación entre vértices y aristas.

SOLUCIÓN

In[] :=

```
T = Arbol[{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}];
{CantNodos[T], CantAristas[T]}
```

Out[] =

```
{6, 5}
```

Se cumple $p = n - 1$: seis vértices y cinco aristas. Añadir una arista más crearía un circuito y quitar una desconectaría el grafo.

6.7.4 (★★) **VilCretas** Aplique `ArbolJQ` a un triángulo e interprete la respuesta.

SOLUCIÓN

In[] :=

```
ArbolJQ[Grafo[{1, 2}, {2, 3}, {3, 1}], vectorial -> True]
```

Out[] =

```
{False, True, True}
```

No es árbol: aunque es conexo, tiene circuitos. Sin la opción `vectorial->True` el comando explica en palabras el motivo.

6.7.5 (★) **VilCretas** Calcule la altura del árbol de referencia tomando como raíz el vértice 1.

SOLUCIÓN

```
In[ ] :=
```

```
T = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}}];
Altura[T, 1]
```

```
Out[ ] =
```

```
2
```

La raíz está en el nivel cero, sus hijos en el uno y los nietos en el dos: la altura es el nivel más profundo alcanzado.

6.7.6 (★) **VilCretas** Determine las hojas del árbol de referencia.

SOLUCIÓN

```
In[ ] :=
```

```
T = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}}];
Hojas[T]
```

```
Out[ ] =
```

```
{4, 5, 6}
```

Las **hojas** son los vértices sin descendientes. Aquí coinciden con los de valencia uno, porque la raíz tiene dos hijos.

6.7.7 (★★) **A MANO** ¿Por qué **Hojas** advierte que supone que la raíz no tiene valencia uno?

SOLUCIÓN Porque un vértice de valencia uno se ve, desde el grafo, igual que una hoja. Si la raíz tiene un solo hijo, el comando la contaría como terminal; por eso ofrece la opción **raiz->nodo**, que la declara explícitamente y la excluye de la lista.

6.7.8 (★★) **A MANO** Un árbol tiene 12 vértices. ¿Cuántas aristas tiene y por qué?

SOLUCIÓN Tiene 11. En todo árbol $p = n - 1$: la primera arista conecta dos vértices y cada arista siguiente incorpora exactamente uno nuevo, pues si uniera dos vértices ya conectados crearía un circuito.

6.7.9 (★★) **MIXTO** Construya un árbol binario completo de altura dos y verifique el teorema de los vértices internos.

SOLUCIÓN

```
In[ ] :=
```

```
T2 = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {3, 7}}];
{CantNodos[T2], Hojas[T2], Altura[T2, 1]}
```

```
Out[ ] =
```

```
{7, {4, 5, 6, 7}, 2}
```

Hay $x = 3$ vértices internos, y en efecto se obtienen $x + 1 = 4$ hojas y $2x + 1 = 7$ vértices, como afirma el teorema del árbol binario completo.

6.7.10 (★★★) **A MANO** Un árbol binario completo tiene 25 vértices. ¿Cuántas hojas tiene?

SOLUCIÓN De $2x + 1 = 25$ sale $x = 12$ vértices internos, y por lo tanto $x + 1 = 13$ hojas. El número total de vértices de un árbol binario completo es siempre impar.

6.7.11 (★★) **VilCretas** Construya un k -árbol de orden tres con diez vértices.

SOLUCIÓN

In[] :=

```
{CantNodos[KArbol[3, 10]], CantAristas[KArbol[3, 10]]}
```

Out[] =

```
{10, 9}
```

En un k -árbol cada vértice interno tiene a lo sumo k hijos. Sigue siendo un árbol, de modo que conserva la relación $p = n - 1$. Si el grafo pedido no existe, el comando devuelve **NaD**.

6.7.12 (★★) **VilCretas** Genere un árbol binario con siete vértices y otro pseudoaleatorio.

SOLUCIÓN

In[] :=

```
ArbolBinario[7]
```

```
ArbolBinarioRandom[7]
```

El primero produce siempre el mismo árbol; el segundo cambia en cada invocación. Con **letras->True** los nodos se rotulan con letras del abecedario en lugar de números.

6.7.13 (★) **MIXTO** Compare el número de aristas de los grafos camino, ciclo, estrella y rueda de cinco vértices.

SOLUCIÓN

In[] :=

```
{CantAristas[GrafoCamino[5]], CantAristas[GrafoCiclo[5]],  
CantAristas[GrafoEstrella[5]], CantAristas[GrafoRueda[5]]}
```

Out[] =

```
{4, 5, 4, 8}
```

Camino y estrella tienen $n - 1 = 4$ aristas: son árboles. El ciclo tiene una más y por eso contiene un circuito; la rueda añade los rayos al ciclo.

6.7.14 (★★) **MIXTO** ¿Son árboles el grafo camino y el grafo estrella?

SOLUCIÓN

In[] :=

```
{ArbolJQ[GrafoCamino[4], vectorial -> True],  
ArbolJQ[GrafoEstrella[5], vectorial -> True]}
```

Out[] =

```
{{True, True, False}, {True, True, False}}
```

Ambos son conexos y sin circuitos. Son los dos casos extremos: el camino tiene la altura máxima posible y la estrella la mínima.

6.7.15 (★★) **VilCretas** Obtenga las valencias del grafo estrella con cinco vértices.

SOLUCIÓN

`In[ ] :=`

`Valencias[GrafoEstrella[5], table -> False]`

`Out[ ] =`

`{4, 1, 1, 1, 1}`

El centro se une a los otros cuatro y cada uno de ellos tiene valencia uno: son todos hojas. Su altura, tomando el centro como raíz, es 1.

6.7.16 (★★) **A MANO** ¿Cuántos árboles generadores tiene un ciclo de n vértices?

SOLUCIÓN Tiene n . Un ciclo con n vértices tiene n aristas, y basta quitar cualquiera de ellas para obtener un camino, que es conexo y sin circuitos. Cada arista eliminada da un árbol generador distinto.

6.7.17 (★★★) **A MANO** ¿Cuántas hojas tiene como mínimo un árbol con $n \geq 2$ vértices?

SOLUCIÓN Tiene al menos dos: los extremos de una ruta de longitud máxima. Si uno de esos extremos tuviera otro vecino no visitado, la ruta podría alargarse, y si lo tuviera visitado habría un circuito. El mínimo se alcanza en el grafo camino.

6.7.18 (★★) **VilCretas** Construya un árbol ponderado y muestre los pesos.

SOLUCIÓN

`In[ ] :=`

`Arbol[{{1, 2}, {1, 3}, {2, 4}}, pesos -> {5, 2, 7},
mostrarpesos -> True]`

La lista de pesos sigue el orden de las aristas. `ArbolR` hace lo mismo pero recibe además la raíz, con lo que el dibujo se organiza por niveles.

6.2 Árboles binarios de búsqueda

6.7.19 (★) **A MANO** Explique la regla de construcción de un árbol binario de búsqueda.

SOLUCIÓN El primer dato es la raíz. Cada dato nuevo se compara con la raíz: si es menor baja por la rama izquierda, si es mayor por la derecha, y el proceso se repite hasta encontrar un lugar libre. El resultado depende del **orden** en que llegan los datos.

6.7.20 (★) **VilCretas** Construya el árbol binario de búsqueda de 5, 3, 8, 1, 4, 7, 9.

SOLUCIÓN

`In[ ] :=`

`ArbolBinarioBusqueda[{5, 3, 8, 1, 4, 7, 9}]`

La raíz es 5; a la izquierda quedan 3 con hijos 1 y 4, y a la derecha 8 con hijos 7 y 9. Con `grafico->True` el árbol se dibuja como gráfico y no como grafo.

6.7.21 (★★) **MIXTO** Determine la altura y las hojas del árbol anterior.

SOLUCIÓN

```
In[ ] :=
```

```
A = ArbolBinarioBusqueda[{5, 3, 8, 1, 4, 7, 9}];
{Altura[A, 5], Hojas[A]}
```

```
Out[ ] =
```

```
{2, {1, 4, 7, 9}}
```

Es un árbol binario completo de altura dos: los siete datos se acomodan en el mínimo número de niveles posible.

- 6.7.22** (★★★) **MIXTO** Construya el árbol binario de búsqueda de 1, 2, 3, 4 y compare su altura con la del ejercicio anterior.

SOLUCIÓN

```
In[ ] :=
```

```
B = ArbolBinarioBusqueda[{1, 2, 3, 4}];
Altura[B, 1]
```

```
Out[ ] =
```

```
3
```

Con los datos ya ordenados cada nuevo elemento se cuelga siempre a la derecha y el árbol **degenera en un camino**: la altura crece como $n - 1$ en vez de como $\log_2 n$. Es el peor caso de la estructura.

- 6.7.23** (★★) **A MANO** ¿Por qué importa la altura de un árbol binario de búsqueda?

SOLUCIÓN Porque buscar un dato consiste en descender desde la raíz, de modo que el número de comparaciones en el peor caso es la altura más uno. Con el lenguaje del capítulo 3: un árbol equilibrado da una búsqueda $O(\ln n)$ y uno degenerado, $O(n)$.

- 6.7.24** (★★) **A MANO** Dibuje a mano el árbol binario de búsqueda de 7, 2, 9, 1, 5, 8 e indique su altura.

SOLUCIÓN La raíz es 7. El 2 va a su izquierda; 9 a su derecha; 1 a la izquierda de 2; 5 a la derecha de 2; y 8 a la izquierda de 9. La altura es 2 y las hojas son 1, 5 y 8.

- 6.7.25** (★★) **VilCretas** Construya un árbol binario de búsqueda a partir de una frase.

SOLUCIÓN

```
In[ ] :=
```

```
ArbolBinarioBusqueda["matematica discreta"]
```

El comando acepta números, caracteres o una cadena. Con una cadena toma las **palabras** como datos y las compara en orden alfabético: la raíz es **matematica** y **discreta** queda a su izquierda por precederla. Para trabajar carácter por carácter hay que pasar la lista de caracteres.

- 6.7.26** (★★★) **A MANO** ¿Cuántas comparaciones hacen falta, en el peor caso, para buscar un dato en un árbol binario de búsqueda con 15 vértices bien equilibrado?

SOLUCIÓN Un árbol binario con 15 vértices y altura mínima tiene altura 3, porque $1+2+4+8 = 15$. En el peor caso se recorre desde la raíz hasta una hoja: cuatro comparaciones, esto es, la altura más uno.

6.3 Recorridos, notación polaca y códigos de *Huffman*

6.7.27 (★) **A MANO** Explique en qué consisten los recorridos prefijo y postfijo.

SOLUCIÓN En el recorrido **prefijo** se visita primero la raíz y después, de manera recursiva, cada subárbol; en el **postfijo** se visitan primero los subárboles y la raíz al final. Ambos recorren todos los vértices una sola vez, cambiando solo el momento en que se anota la raíz.

6.7.28 (★) **VilCretas** Recorra el árbol de referencia en prefijo y en postfijo.

SOLUCIÓN

In[] :=

```
T = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}}];
Prefijo[T, 1]
Postfijo[T, 1]
```

Out[] =

```
{1, 2, 4, 5, 3, 6}
{4, 5, 2, 6, 3, 1}
```

El prefijo empieza por la raíz 1 y el postfijo termina en ella. El segundo argumento indica cuál vértice actúa como raíz.

6.7.29 (★★) **VilCretas** Observe paso a paso el recorrido prefijo.

SOLUCIÓN

In[] :=

```
T = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}}];
Prefijo[T, 1, animacion -> True]
```

Con `animacion->True` el comando resalta los vértices en el orden en que se visitan. Las instrucciones no aceptan árboles gráficos ni dirigidos.

6.7.30 (★★) **MIXTO** Recorra en prefijo y en postfijo el árbol binario de búsqueda de 5, 3, 8, 1, 4, 7, 9.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{5, 3, 8, 1, 4, 7, 9}];
Prefijo[A, 5]
Postfijo[A, 5]
```

Out[] =

```
{5, 3, 1, 4, 8, 7, 9}
{1, 4, 3, 7, 9, 8, 5}
```

El recorrido prefijo devuelve los datos en el orden que reconstruye el mismo árbol si se vuelven a insertar uno por uno.

6.7.31 (★) **VilCretas** Construya el árbol de la expresión algebraica $a + bc$.

SOLUCIÓN

`In[ ] :=`

`ArbolExpAlgebraica[a + b*c]`

La raíz es la operación que se ejecuta de última, la suma; sus hijos son a y el subárbol del producto, cuyas hojas son b y c . Los operandos quedan siempre en las hojas.

6.7.32 (★★) **VilCretas** Obtenga la notación polaca de $a + bc$.

SOLUCIÓN

`In[ ] :=`

`Polaca[a + b*c]`

El comando dibuja el árbol y devuelve la notación prefija $+ a * b c$, que es el recorrido prefijo del árbol de la expresión. No necesita paréntesis: la posición de cada operador determina sobre qué actúa.

6.7.33 (★★) **VilCretas** Obtenga la notación polaca inversa de la misma expresión.

SOLUCIÓN

`In[ ] :=`

`PolacaInversa[a + b*c]`

La salida es $a b c * +$, el recorrido postfijo del mismo árbol. Es la notación que usan las calculadoras de pila: cada operador actúa sobre los dos últimos resultados.

6.7.34 (★★) **A MANO** Escriba a mano la notación polaca y la polaca inversa de $(a + b) * c$.

SOLUCIÓN El árbol tiene por raíz el producto, con hijos el subárbol de la suma y la hoja c . El recorrido prefijo da $* + a b c$ y el postfijo, $a b + c *$. Comparado con $a + bc$, cambia la raíz del árbol y por lo tanto el orden completo.

6.7.35 (★★★) **A MANO** Evalúe la expresión en notación polaca inversa $3 4 + 2 *$.

SOLUCIÓN Se recorre de izquierda a derecha apilando operandos: al llegar a $+$ se toman 3 y 4 y se apila 7; al llegar a $*$ se toman 7 y 2 y se obtiene 14. El resultado es 14, que corresponde a $(3 + 4) \cdot 2$.

6.7.36 (★) **A MANO** ¿Para qué sirven los códigos de *Huffman*?

SOLUCIÓN Para comprimir: en vez de dar a todos los caracteres códigos de la misma longitud, asignan códigos **cortos a los caracteres frecuentes** y largos a los raros. El árbol de *Huffman* construye esa asignación de manera óptima.

6.7.37 (★★) **VilCretas** Obtenga las frecuencias que usa el árbol de *Huffman* no optimizado para la palabra *abracadabra*.

SOLUCIÓN

`In[ ] :=`

`ArbolHuffmanN["abracadabra", frecuencias -> True][[1]]`

`Out[ ] =`

```
{{a, 5}, {b, 2}, {r, 2}, {c, 1}, {d, 1}}
```

La **a** aparece cinco veces y será la que reciba el código más corto; **c** y **d**, con una aparición cada una, quedarán en las ramas más profundas.

6.7.38 (★★) **VilCretas** Construya el árbol de *Huffman* optimizado de esa misma palabra.

SOLUCIÓN

```
In[ ] :=
```

```
ArbolHuffman["abracadabra"]
```

ArbolHuffman aplica el algoritmo optimizado, que combina repetidamente las dos frecuencias menores; **ArbolHuffmanN** construye la versión no optimizada. Con **grafico->True** el árbol se muestra como gráfico.

6.7.39 (★★) **VilCretas** Obtenga las frecuencias de la palabra *casa*.

SOLUCIÓN

```
In[ ] :=
```

```
ArbolHuffmanN["casa", frecuencias -> True][[1]]
```

```
Out[ ] =
```

```
{{a, 2}, {c, 1}, {s, 1}}
```

Con solo tres caracteres distintos el árbol tiene tres hojas: la **a** recibe un código de un bit y las otras dos, de dos bits.

6.7.40 (★★★) **A MANO** ¿Qué ocurre con el árbol de *Huffman* cuando todos los caracteres tienen la misma frecuencia?

SOLUCIÓN Todas las hojas quedan aproximadamente al mismo nivel y los códigos resultan de longitud casi igual: no hay ganancia frente a una codificación de longitud fija. La compresión aparece cuando las frecuencias son **desiguales**. La palabra *discreta*, con sus ocho letras distintas, es un caso así.

6.4 Árboles generadores

6.7.41 (★) **A MANO** ¿Qué es un árbol generador y cuándo existe?

SOLUCIÓN Es un subárbol que contiene **todos** los vértices del grafo. Un grafo no dirigido tiene árbol generador si y solo si es conexo. Al ser un árbol, tiene $n - 1$ aristas, de modo que se obtiene quitando circuitos.

6.7.42 (★★) **VilCretas** Halle un árbol generador con la búsqueda a lo ancho sobre el grafo de aristas $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$ y $\{3, 4\}$.

SOLUCIÓN

```
In[ ] :=
```

```
G = Grafo[{{1, 2}, {1, 3}, {2, 4}, {3, 4}}];
```

```
BuscarPrimeroAncho[G]
```

```
Out[ ] =
```

$\{\{1, 2\}, \{1, 3\}, \{2, 4\}\}$

El recorrido a lo ancho visita primero todos los vecinos del vértice inicial y después desciende. Devuelve tres aristas, las $n - 1$ de un árbol con cuatro vértices: la arista $\{3, 4\}$ queda fuera por cerrar un circuito.

6.7.43 (★★) **VilCretas** Repita la búsqueda a lo largo sobre el mismo grafo y compare.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {1, 3}, {2, 4}, {3, 4}}];
BuscarPrimeroLargo[G]
```

Out[] =

$\{\{1, 2\}, \{2, 4\}, \{4, 3\}\}$

La búsqueda a lo largo avanza tanto como puede antes de retroceder, de modo que produce un árbol **en forma de camino**. Los dos árboles generadores son distintos y ambos válidos.

6.7.44 (★★) **MIXTO** Compare los dos recorridos sobre el ciclo de cuatro vértices.

SOLUCIÓN

In[] :=

```
{BuscarPrimeroAncho[GrafoCiclo[4]], BuscarPrimeroLargo[GrafoCiclo[4]]}
```

Out[] =

$\{\{\{1, 2\}, \{1, 4\}, \{2, 3\}\}, \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}\}$

El primero se abre hacia los dos vecinos de 1; el segundo recorre el ciclo entero hasta agotarlo. En ambos casos se elimina exactamente una arista, la que cerraba el circuito.

6.7.45 (★) **MIXTO** ¿Qué devuelve la búsqueda a lo ancho sobre un grafo que ya es un árbol?

SOLUCIÓN

In[] :=

```
BuscarPrimeroAncho[GrafoCamino[4]]
```

Out[] =

$\{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$

Devuelve el mismo árbol: si no hay circuitos no hay nada que eliminar. El único árbol generador de un árbol es él mismo.

6.7.46 (★★) **VilCretas** Cambie el orden de los vértices en la búsqueda y observe el efecto.

SOLUCIÓN

In[] :=

```
G = Grafo[{{1, 2}, {1, 3}, {2, 4}, {3, 4}}];
BuscarPrimeroAncho[G, orden -> {4, 3, 2, 1}]
```

La opción **orden** fija desde qué vértice se empieza y en qué secuencia se exploran los demás. El árbol generador obtenido cambia, aunque siempre tiene $n - 1$ aristas.

6.7.47 (★★★) **A MANO** ¿Cuántas aristas hay que eliminar de un grafo conexo con n vértices y p aristas para obtener un árbol generador?

SOLUCIÓN Hay que quitar $p - (n - 1)$. El árbol generador conserva los n vértices y exactamente $n - 1$ aristas, de modo que la diferencia mide cuántos circuitos independientes tenía el grafo.

6.5 Árboles de expansión mínima

6.7.48 (★) **A MANO** ¿Qué es un árbol de expansión mínima?

SOLUCIÓN Es un árbol generador de un grafo ponderado cuya suma de pesos es la menor posible. Modela, por ejemplo, la red más barata que conecta todos los puntos de un sistema sin ciclos redundantes.

6.7.49 (★★) **A MANO** Explique la diferencia entre los algoritmos de *Kruskal* y de *Prim*.

SOLUCIÓN *Kruskal* ordena todas las aristas por peso y las va agregando de menor a mayor, descartando las que cierren un circuito. *Prim* parte de un vértice y en cada paso añade la arista más barata que conecta el árbol ya construido con un vértice nuevo. Ambos llegan a un árbol de peso mínimo, aunque no necesariamente al mismo.

6.7.50 (★★) **VilCretas** Aplique *Kruskal* al grafo ponderado con aristas $\{1, 2\}$, $\{2, 3\}$, $\{3, 4\}$, $\{4, 1\}$ y $\{1, 3\}$ de pesos 4, 3, 2, 5 y 10.

SOLUCIÓN

In[] :=

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
Kruskal[GP]
```

El algoritmo toma primero la arista de peso 2, luego la de peso 3 y después la de peso 4; con tres aristas ya conecta los cuatro vértices y se detiene. El árbol resultante tiene **peso 9**. Con `all->True` se ve el procedimiento paso a paso.

6.7.51 (★★) **VilCretas** Aplique *Prim* al mismo grafo y compare el resultado.

SOLUCIÓN

In[] :=

```
GP = Grafo[{{1, 2}, {2, 3}, {3, 4}, {4, 1}, {1, 3}},
pesos -> {4, 3, 2, 5, 10}];
Prim[GP]
```

Partiendo del vértice 1 se añade $\{1, 2\}$ con peso 4, luego $\{2, 3\}$ con peso 3 y por último $\{3, 4\}$ con peso 2: el mismo árbol y el mismo **peso 9**, alcanzado en otro orden. La opción `orden` permite cambiar el vértice de partida.

6.7.52 (★★★) **A MANO** Verifique a mano que el árbol de peso 9 del grafo anterior es el mínimo.

SOLUCIÓN El grafo tiene cinco aristas y hay que escoger tres sin formar circuito. Las tres más baratas son las de pesos 2, 3 y 4, y no forman circuito porque corresponden al camino $1 - 2 - 3 - 4$. Cualquier otra selección obliga a incluir una arista de peso 5 o 10, de modo que 9 es el mínimo.

- 6.7.53** (★★) **A MANO** ¿Por qué ni *Kruskal* ni *Prim* admiten pesos iguales a cero en la implementación del paquete?

SOLUCIÓN Porque el peso cero se confunde con la ausencia de arista en la matriz de adyacencia de pesos: una entrada nula significa que los vértices no están unidos. Es la misma restricción que aparece en **GrafoMP**.

- 6.7.54** (★★★) **A MANO** Si todos los pesos de un grafo son distintos, ¿puede haber dos árboles de expansión mínima diferentes?

SOLUCIÓN No. Con pesos todos distintos el árbol de expansión mínima es único. La posibilidad de obtener árboles distintos con *Kruskal* y *Prim* aparece justo cuando hay **empates** de peso, porque entonces la elección entre aristas equivalentes es arbitraria.

- 6.7.55** (★★★) **MIXTO** Compruebe que el árbol de expansión mínima encontrado es un árbol y calcule su peso.

SOLUCIÓN

In[] :=

```
Tm = Arbol[{{1, 2}, {2, 3}, {3, 4}}, pesos -> {4, 3, 2}];
{ArbolJQ[Tm, vectorial -> True], PesoRuta[Tm, {{1, 2}, {2, 3}, {3, 4}}]}
```

Out[] =

```
{{True, True, False}, 9}
```

Es conexo, sin circuitos y su peso total es $4 + 3 + 2 = 9$, el valor que devolvieron *Kruskal* y *Prim*.

- 6.7.56** (★★★) **MIXTO** Sea T el árbol con raíz 1 y aristas $\{1, 2\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 5\}$, $\{2, 6\}$, $\{3, 7\}$, $\{5, 8\}$, $\{5, 9\}$ y $\{7, 10\}$. Determine número de nodos, hojas y altura.

SOLUCIÓN

In[] :=

```
T1 = Arbol[{{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {3, 7},
{5, 8}, {5, 9}, {7, 10}}];
{CantNodos[T1], CantAristas[T1], Hojas[T1], Altura[T1, 1]}
```

Out[] =

```
{10, 9, {4, 6, 8, 9, 10}, 3}
```

Diez nodos y nueve aristas, coherente con $p = n - 1$. Hay cinco hojas y la altura es 3. Este árbol se usa en los ejercicios siguientes.

- 6.7.57** (★★★) **MIXTO** Recorra ese árbol en prefijo y en postfijo.

SOLUCIÓN

In[] :=

```
T1 = Arbol[{{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {3, 7},
{5, 8}, {5, 9}, {7, 10}}];
{Prefijo[T1, 1], Postfijo[T1, 1]}
```

Out[] =

$\{\{1, 2, 5, 8, 9, 6, 3, 7, 10, 4\}, \{8, 9, 5, 6, 2, 10, 7, 3, 4, 1\}\}$

El prefijo abre por la raíz y el postfijo la deja de última. Las hojas aparecen en el mismo orden relativo en los dos recorridos: 8, 9, 6, 10, 4.

6.7.58 (★★★) **A MANO** ¿Por qué las hojas aparecen en el mismo orden en el recorrido prefijo y en el postfijo?

SOLUCIÓN Porque una hoja no tiene subárboles: al llegar a ella se la visita y se regresa, sin que importe si la raíz se anota antes o después. Lo que cambia entre los dos recorridos es **dónde** se colocan los vértices internos, no el orden en que se alcanzan las hojas.

6.7.59 (★★) **A MANO** ¿Cuántos vértices internos tiene el árbol anterior y en qué niveles están?

SOLUCIÓN Tiene cinco: 1 en el nivel 0; 2 y 3 en el nivel 1; 5 y 7 en el nivel 2. Los otros cinco vértices son hojas. En todo árbol, **internos más hojas** suman el total de nodos.

6.7.60 (★★★) **MIXTO** Construya un árbol binario completo de altura tres y verifique el teorema.

SOLUCIÓN

In[] :=

```
T2 = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {3, 7},
{4, 8}, {4, 9}, {5, 10}, {5, 11}}];
{CantNodos[T2], Hojas[T2], Altura[T2, 1]}
```

Out[] =

```
{11, {6, 7, 8, 9, 10, 11}, 3}
```

Hay $x = 5$ vértices internos, y en efecto $x + 1 = 6$ hojas y $2x + 1 = 11$ nodos. La altura es 3 aunque el árbol no esté lleno en el último nivel.

6.7.61 (★★★) **A MANO** ¿Cuál es la altura mínima de un árbol binario con n nodos?

SOLUCIÓN Es $\lceil \log_2(n + 1) \rceil - 1$: el nivel k admite a lo sumo 2^k nodos, de modo que con altura h caben hasta $2^{h+1} - 1$. Para $n = 100$ la altura mínima es 6, y para $n = 1000$ es 9: el árbol crece a lo ancho mucho más rápido que a lo alto.

6.7.62 (★★) **MIXTO** Tabule la altura mínima de un árbol binario con 1 a 16 nodos.

SOLUCIÓN

In[] :=

```
Table[Ceiling[Log[2, k + 1]] - 1, {k, 1, 16}]
```

Out[] =

```
{0, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 4}
```

El valor solo aumenta al pasar 1, 3, 7, 15: las cantidades $2^{h+1} - 1$ que llenan por completo un nivel más.

6.7.63 (★★★) **MIXTO** Almacene $\{50, 30, 70, 20, 40, 60, 80, 35, 45\}$ en un árbol binario de búsqueda y dé los tres recorridos.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{50, 30, 70, 20, 40, 60, 80, 35, 45}];
{Prefijo[A, 50], Postfijo[A, 50]}
```

Out[] =

```
{50, 30, 20, 40, 35, 45, 70, 60, 80}, {20, 35, 45, 40, 30, 60, 80, 70, 50}}
```

In[] :=

```
Sort[{50, 30, 70, 20, 40, 60, 80, 35, 45}]
```

Out[] =

```
{20, 30, 35, 40, 45, 50, 60, 70, 80}
```

El interfijo de un árbol binario de búsqueda es siempre la lista ordenada: esa es su propiedad definitoria.

6.7.64 (★★) **MIXTO** Determine la altura y las hojas del árbol anterior.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{50, 30, 70, 20, 40, 60, 80, 35, 45}];
{Altura[A, 50], Hojas[A]}
```

Out[] =

```
{3, {20, 35, 45, 60, 80}}
```

Altura 3 y cinco hojas. Con nueve datos la altura mínima sería 3, de modo que el árbol está bien equilibrado.

6.7.65 (★★★) **MIXTO** Almacene {15, 8, 22, 4, 12, 19, 30, 2, 6} y compare su forma con la del árbol anterior.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{15, 8, 22, 4, 12, 19, 30, 2, 6}];
{Prefijo[A, 15], Altura[A, 15], Hojas[A]}
```

Out[] =

```
{{15, 8, 4, 2, 6, 12, 22, 19, 30}, 3, {2, 6, 12, 19, 30}}
```

Misma altura y mismo número de hojas, pero forma distinta: aquí la rama izquierda es más profunda. El orden de llegada de los datos determina la forma, aunque el contenido sea equivalente.

6.7.66 (★★★) **A MANO** ¿Cuántos árboles binarios de búsqueda distintos se pueden formar con los datos {1, 2, 3}?

SOLUCIÓN Cinco, uno por cada elección de raíz y forma: son los números de *Catalan*, $C_3 = 5$. En general con n datos hay $C_n = \frac{1}{n+1} \binom{2n}{n}$ árboles binarios de búsqueda distintos, y todos tienen el mismo recorrido interfijo.

6.7.67 (★★) **MIXTO** Compruebe los primeros números de *Catalan*.

SOLUCIÓN

In[] :=

```
Table[CatalanNumber[k], {k, 1, 8}]
```

Out[] =

```
{1, 2, 5, 14, 42, 132, 429, 1430}
```

Con cuatro datos hay 14 árboles distintos y con ocho, 1430. Solo uno de ellos tiene altura mínima, y la probabilidad de obtenerlo insertando al azar baja rápidamente.

6.7.68 (★★★) A MANO ¿Qué orden de inserción produce el peor árbol binario de búsqueda posible?

SOLUCIÓN El de los datos **ya ordenados**, ascendente o descendente: cada nuevo dato se cuelga siempre del mismo lado y el árbol degenera en un camino de altura $n - 1$. La búsqueda pasa de $\Theta(\ln n)$ a $\Theta(n)$: exactamente el mismo costo que recorrer una lista.

6.7.69 (★★) MIXTO Compruebe la degeneración con los datos $\{1, 2, 3, 4, 5, 6\}$.

SOLUCIÓN

In[] :=

```
{Altura[ArbolBinarioBusqueda[{1, 2, 3, 4, 5, 6}], 1],
Altura[ArbolBinarioBusqueda[{4, 2, 6, 1, 3, 5}], 4]}
```

Out[] =

```
{5, 2}
```

Altura 5 frente a 2 con los mismos seis datos. La segunda inserción sigue el orden de un recorrido por niveles, que es el que equilibra.

6.7.70 (★★★) A MANO Escriba el árbol de la expresión $(a + b)(c - d)$ y sus notaciones polaca y polaca inversa.

SOLUCIÓN La raíz es el producto, con hijos los subárboles de la suma y de la resta. El recorrido prefijo da

$$* + ab - cd$$

y el postfijo,

$$ab + cd - *$$

Se comprueba evaluando la polaca inversa con una pila: se apilan a y b , el $+$ los combina, se apilan c y d , el $-$ los combina, y el $*$ cierra.

6.7.71 (★★) VilCretas Construya con el software el árbol de esa expresión.

SOLUCIÓN

In[] :=

```
ArbolExpAlgebraica[(a + b)*(c - d)]
```

El comando devuelve un grafo de nueve vértices: los cuatro operandos, los tres operadores y los nodos que *Mathematica* añade al representar la resta como suma de un opuesto. Conviene compararlo con el árbol dibujado a mano.

6.7.72 (★★★) A MANO Evalúe la expresión en polaca inversa $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$.

SOLUCIÓN Se recorre apilando: 5, 1, 2; el + toma 1 y 2 y apila 3; el 4 se apila; el * toma 3 y 4 y apila 12; el + toma 5 y 12 y apila 17; el 3 se apila; el - da 14. El resultado es 14, que corresponde a $5 + (1 + 2) \cdot 4 - 3$.

6.7.73 (★★★) **A MANO** ¿Por qué la notación polaca no necesita paréntesis?

SOLUCIÓN Porque la posición de cada operador determina sin ambigüedad sobre qué actúa: siempre sobre los dos operandos que le siguen (polaca) o los dos que le preceden (polaca inversa). Los paréntesis existen en la notación infija solo para resolver esa ambigüedad, y por eso desaparecen al cambiar de notación.

6.7.74 (★★) **MIXTO** Obtenga las frecuencias de *matematicas* para un código de *Huffman*.

SOLUCIÓN

In[] :=

```
ArbolHuffmanN["matematicas", frecuencias -> True][[1]]
```

Out[] =

```
{a, 3}, {m, 2}, {t, 2}, {e, 1}, {i, 1}, {c, 1}, {s, 1}
```

Siete caracteres distintos en una palabra de once letras. La a, con tres apariciones, recibirá el código más corto.

6.7.75 (★★★) **MIXTO** Calcule cuántos bits ahorra el código de *Huffman* frente a uno de longitud fija para *matematicas*.

SOLUCIÓN

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
{huff[{3, 2, 2, 1, 1, 1, 1}], 11*Ceiling[Log[2, 7]]}
```

Out[] =

```
{30, 33}
```

Bastan 30 bits frente a 33: un ahorro del 9%. Con siete símbolos casi equifrecuentes el margen es pequeño; la compresión rinde cuando las frecuencias son muy desiguales.

6.7.76 (★★★) **MIXTO** Repita el cálculo para *programacion*.

SOLUCIÓN

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
{huff[{2, 2, 2, 1, 1, 1, 1, 1, 1}], 12*Ceiling[Log[2, 9]]}
```

Out[] =

```
{38, 48}
```

38 bits frente a 48: un ahorro del 21 %. Con nueve símbolos la codificación fija necesita cuatro bits por carácter y desperdicia mucho.

6.7.77 (★★★) **A MANO** Explique por qué ningún código de *Huffman* puede ser peor que uno de longitud fija.

SOLUCIÓN Porque el algoritmo construye el árbol **óptimo** para las frecuencias dadas, y el código de longitud fija corresponde a un árbol binario completo particular, que es uno de los candidatos. En el peor caso (todas las frecuencias iguales y potencia de dos) *Huffman* produce exactamente ese árbol y el ahorro es cero, pero nunca negativo.

6.7.78 (★★) **A MANO** ¿Por qué un código de *Huffman* nunca asigna a un carácter un código que sea prefijo del de otro?

SOLUCIÓN Porque todos los caracteres quedan en las **hojas** del árbol, y ninguna hoja está en el camino hacia otra. Esa propiedad (llamada *código prefijo*) es la que permite decodificar sin separadores: al leer bit a bit, en cuanto se llega a una hoja el carácter queda determinado.

6.7.79 (★★★) **MIXTO** Halle el árbol generador que produce cada búsqueda sobre el grafo con aristas $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}, \{4, 6\}, \{5, 6\}$.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
{BuscarPrimeroAncho[Grafo[Ag]], BuscarPrimeroLargo[Grafo[Ag]]}
```

Out[] =

```
{{{1, 2}, {1, 3}, {2, 4}, {3, 5}, {4, 6}},
{{1, 2}, {2, 3}, {3, 5}, {5, 4}, {4, 6}}}
```

Los dos tienen cinco aristas, las $n - 1$ de un árbol con seis vértices. El primero se abre en abanico y el segundo forma una cadena.

6.7.80 (★★★) **A MANO** ¿Cuántas aristas descarta cada búsqueda en el grafo anterior y por qué siempre el mismo número?

SOLUCIÓN Descarta $8 - 5 = 3$ aristas. El número es siempre $p - (n - 1)$, independientemente del recorrido, porque todo árbol generador tiene exactamente $n - 1$ aristas. Lo que cambia entre BFS y DFS es **cuáles** se descartan, no cuántas.

6.7.81 (★★★) **MIXTO** Halle el peso del árbol de expansión mínima del grafo ponderado con esas aristas y pesos 5, 3, 8, 2, 6, 4, 7, 1.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
gg = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> Pg];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gg]]]
```

Out[] =

15

El peso mínimo es 15, con las aristas $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$, $\{4, 5\}$ y $\{5, 6\}$, de pesos $5 + 3 + 2 + 4 + 1$.

6.7.82 (★★★) **MIXTO** Halle el peso del árbol de expansión máxima del mismo grafo.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
gg = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> Pg];
gm = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> (-Pg)];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gm]]]
```

Out[] =

30

El máximo es 30, el doble del mínimo. La técnica es la misma con los pesos negados: minimizar $-w$ equivale a maximizar w .

6.7.83 (★★) **VilCretas** Ejecute *Kruskal* sobre ese grafo y observe el orden de las aristas.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
Kruskal[Grafo[Ag, pesos -> Pg], all -> True]
```

El algoritmo toma las aristas de menor peso primero: $\{5, 6\}$ con 1, $\{2, 4\}$ con 2, $\{1, 3\}$ con 3, $\{4, 5\}$ con 4 y $\{1, 2\}$ con 5, descartando las que cerrarían circuito. El peso total es 15.

6.7.84 (★★★) **A MANO** ¿Por qué *Kruskal* funciona aunque tome decisiones locales?

SOLUCIÓN Porque cumple la *propiedad del corte*: la arista de menor peso que cruza cualquier partición de los vértices pertenece a algún árbol de expansión mínima. Al tomar siempre la más barata que no cierra circuito, cada elección es segura. Es el ejemplo clásico de algoritmo **voraz** que sí da el óptimo global.

6.7.85 (★★★) **A MANO** Dé un ejemplo de problema donde la estrategia voraz **no** da el óptimo.

SOLUCIÓN El del agente viajero: ir siempre a la ciudad más cercana no produce el circuito más barato. Basta un grafo donde el atajo inicial obligue después a un salto muy caro. La diferencia con *Kruskal* está en que el árbol de expansión mínima tiene estructura de *matroide* y el circuito de *Hamilton* no.

6.7.86 (★★) **A MANO** ¿Cuántos árboles generadores distintos tiene el ciclo C_n ? ¿Y K_n ?

SOLUCIÓN El ciclo tiene n : se elimina una cualquiera de sus n aristas. El grafo completo tiene n^{n-2} , por la fórmula de *Cayley*: para $n = 4$ son 16 y para $n = 8$, 262 144.

6.7.87 (★★) **MIXTO** Compruebe la fórmula de *Cayley*.

SOLUCIÓN

In[] :=


```
Table[k^(k - 2), {k, 1, 8}]
```

```
Out[ ] =
```

```
{1, 1, 3, 16, 125, 1296, 16807, 262144}
```

Con tres vértices hay tres árboles (uno por cada arista que se descarta del triángulo) y con ocho ya hay más de un cuarto de millón.

- 6.7.88 (★★★) **A MANO** Demuestre que todo árbol con al menos dos vértices tiene al menos dos hojas.

SOLUCIÓN Tome un camino de longitud máxima $v_0, \dots, v_k$. Si v_0 tuviera otro vecino u distinto de v_1 , entonces u ya estaría en el camino (lo que formaría un circuito, imposible en un árbol) o el camino podría alargarse, contra la maximalidad. Luego v_0 tiene grado uno, y lo mismo vale para v_k .

- 6.7.89 (★★★) **A MANO** Un árbol tiene 15 vértices, todos de grado 1 o 3. ¿Cuántos hay de cada tipo?

SOLUCIÓN Si hay h hojas y t vértices de grado tres, entonces $h + t = 15$ y la suma de grados es $h + 3t = 2 \cdot 14 = 28$. Restando, $2t = 13$, que no es entero: **tal árbol no existe**. Con 16 vértices sí: $h = 9$ y $t = 7$.

- 6.7.90 (★★) **MIXTO** Construya un k -árbol de orden tres con trece vértices.

SOLUCIÓN

```
In[ ] :=
```

```
{CantNodos[KArbol[3, 13]], CantAristas[KArbol[3, 13]]}
```

```
Out[ ] =
```

```
{13, 12}
```

Trece vértices y doce aristas: sigue siendo un árbol. En un 3-árbol cada vértice interno tiene a lo sumo tres hijos, de modo que la altura mínima baja respecto del binario.

- 6.7.91 (★★★) **A MANO** ¿Cuál es la altura mínima de un k -árbol con n nodos?

SOLUCIÓN Es $\lceil \log_k (n(k-1) + 1) \rceil - 1$, porque el nivel j admite hasta k^j nodos y la suma geométrica da $\frac{k^{h+1}-1}{k-1}$. Para $n = 1000$ y $k = 2$ la altura mínima es 9; con $k = 10$ baja a 3. Aumentar el orden achata el árbol.

- 6.7.92 (★★★) **A MANO** ¿Por qué no es buena idea aumentar indefinidamente el orden de un árbol de búsqueda?

SOLUCIÓN Porque cada nodo pasa a contener más claves y decidir por cuál rama bajar cuesta más: la búsqueda dentro del nodo crece con k . El total es $\Theta(k \log_k n)$, que tiene un mínimo y luego empeora. El equilibrio real depende del costo de leer un nodo, y por eso los árboles de disco usan órdenes altos y los de memoria, bajos.

- 6.7.93 (★★) **A MANO** Explique la diferencia entre *árbol*, *árbol con raíz* y *árbol binario*.

SOLUCIÓN Un **árbol** es un grafo conexo sin circuitos, sin ningún vértice privilegiado. Un **árbol con raíz** designa uno de ellos como origen, lo que induce niveles, padres e hijos. Un **árbol binario** exige además que cada nodo tenga a lo sumo dos hijos, **distinguiendo** el izquierdo del derecho.

- 6.7.94** (★★★) **A MANO** Dos árboles binarios con los mismos vértices y aristas pueden ser distintos. ¿Por qué?

SOLUCIÓN Porque en un árbol binario importa **cuál hijo es el izquierdo**. Un nodo con un solo hijo da dos árboles binarios distintos según el lado, aunque como grafos sean idénticos. Esa distinción es la que hace que el recorrido interfijo tenga sentido.

- 6.7.95** (★★★) **MIXTO** Compruebe que la búsqueda a lo ancho sobre un árbol devuelve el mismo árbol.

SOLUCIÓN

`In[ ] :=`

```
T1 = Arbol[{{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {3, 7},
{5, 8}, {5, 9}, {7, 10}}];
Length[BuscarPrimeroAncho[T1]]
```

`Out[ ] =`

9

Devuelve las nueve aristas: en un árbol no hay circuitos que cortar, de modo que el único árbol generador es él mismo. Es la comprobación de que el algoritmo no destruye información cuando no hay nada que descartar.

- 6.7.96** (★★) **A MANO** ¿En qué orden visita los vértices una búsqueda a lo ancho sobre un árbol con raíz?

SOLUCIÓN Por **niveles**: primero la raíz, después todos sus hijos, luego todos los nietos. Es el recorrido que se usa para dibujar el árbol y el que produce el árbol binario de búsqueda mejor equilibrado si se insertan los datos en ese orden.

- 6.7.97** (★★★) **A MANO** Un grafo conexo con n vértices y p aristas tiene $p - n + 1$ circuitos independientes. Justifíquelo.

SOLUCIÓN Un árbol generador usa $n - 1$ aristas y no tiene circuitos. Cada una de las $p - (n - 1)$ aristas restantes, al añadirse, cierra exactamente un circuito nuevo e independiente de los anteriores. Ese número se llama *número ciclomático* y mide cuán lejos está el grafo de ser un árbol.

- 6.7.98** (★★) **A MANO** Calcule el número ciclomático del grafo de ocho aristas y seis vértices de los ejercicios anteriores.

SOLUCIÓN Es $8 - 6 + 1 = 3$: tres circuitos independientes, que son exactamente las tres aristas que descartan BFS y DFS. Coincide, como debe ser, con el número de aristas sobrantes respecto de un árbol generador.

- 6.7.99** (★★) **A MANO** Defina nivel, profundidad y altura en un árbol con raíz.

SOLUCIÓN El **nivel** de un vértice es su distancia a la raíz, que está en el nivel cero. La **profundidad** de un vértice es su nivel. La **altura** del árbol es el mayor nivel alcanzado. Un árbol de un solo vértice tiene altura cero, no uno.

- 6.7.100** (★★★) **A MANO** ¿Cuántos vértices tiene como máximo un árbol binario de altura h ?

SOLUCIÓN Tiene $2^{h+1} - 1$: el nivel j admite hasta 2^j vértices y la suma geométrica da ese total. Para $h = 10$ son 2047. El mínimo, en cambio, es $h + 1$: un camino.

6.7.101 (★★) **MIXTO** Compruebe la cota anterior tabulando $2^{h+1} - 1$.

SOLUCIÓN

`In[ ] :=`

`Table[2^(h + 1) - 1, {h, 0, 8}]`

`Out[ ] =`

`{1, 3, 7, 15, 31, 63, 127, 255, 511}`

Cada nivel nuevo más que duplica la capacidad. Por eso un árbol equilibrado almacena mil datos en diez niveles y un millón en veinte.

6.7.102 (★★★) **A MANO** En un k -árbol, ¿cuántos vértices caben en altura h ?

SOLUCIÓN Caben $\frac{k^{h+1} - 1}{k - 1}$, la suma geométrica de razón k . Para $k = 2$ recupera $2^{h+1} - 1$; para $k = 10$ y $h = 3$ da 1111. Duplicar el orden reduce la altura necesaria en un factor $\log_2 k$.

6.7.103 (★★) **A MANO** Un árbol tiene 7 hojas y todos sus internos tienen dos hijos. ¿Cuántos vértices internos hay?

SOLUCIÓN Seis. En un árbol binario completo el número de hojas es siempre **uno más** que el de internos, de modo que con siete hojas hay seis internos y trece nodos en total.

6.7.104 (★★★) **A MANO** Demuestre la relación anterior por inducción sobre el número de internos.

SOLUCIÓN Con cero internos hay un solo nodo, que es hoja: $1 = 0 + 1$. Al añadir un interno se convierte una hoja en padre de dos hojas nuevas, de modo que las hojas pasan de h a $h + 1$ y los internos de x a $x + 1$. La diferencia $h - x = 1$ se conserva.

6.7.105 (★★) **MIXTO** Compruebe la relación sobre el árbol binario completo de once nodos.

SOLUCIÓN

`In[ ] :=`

`T2 = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {3, 7},
{4, 8}, {4, 9}, {5, 10}, {5, 11}}];
{Length[Hojas[T2]], CantNodos[T2] - Length[Hojas[T2]]}`

`Out[ ] =`

`{6, 5}`

Seis hojas y cinco internos: la diferencia es uno, como exige el teorema.

6.7.106 (★★★) **A MANO** ¿Puede un árbol binario completo tener un número par de vértices?

SOLUCIÓN No. El total es $2x + 1$ con x internos, siempre **impar**. Un enunciado que pida un árbol binario completo con 20 vértices es imposible, y detectarlo sin calcular nada es la clase de comprobación que se hace primero.

6.7.107 (★★) **A MANO** ¿Qué recorrido de un árbol binario de búsqueda devuelve los datos ordenados de mayor a menor?

SOLUCIÓN El interfijo **invertido**: se procesa el subárbol derecho, luego la raíz y después el izquierdo. Es la variante que se usa cuando se necesita el orden descendente sin volver a ordenar la lista.

6.7.108 (★★★) A MANO Dado el recorrido prefijo $\{8, 3, 1, 6, 4, 7, 10, 14, 13\}$ de un árbol binario de búsqueda, reconstruya el árbol.

SOLUCIÓN La raíz es el primer elemento, 8. Los menores forman el subárbol izquierdo $(3, 1, 6, 4, 7)$ y los mayores el derecho $(10, 14, 13)$. Repitiendo: 3 con hijos 1 y 6; 6 con hijos 4 y 7; 10 con hijo derecho 14, que a su vez tiene hijo izquierdo 13. El interfijo confirma: 1, 3, 4, 6, 7, 8, 10, 13, 14.

6.7.109 (★★) MIXTO Compruebe la reconstrucción anterior.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{8, 3, 1, 6, 4, 7, 10, 14, 13}];
{Prefijo[A, 8], Altura[A, 8]}
```

Out[] =

```
{{8, 3, 1, 6, 4, 7, 10, 14, 13}, 3}
```

El prefijo del árbol reconstruido coincide con el dado, lo que confirma que la reconstrucción es correcta. En un árbol binario de búsqueda el prefijo determina el árbol de manera **única**.

6.7.110 (★★★) A MANO ¿Por qué el recorrido interfijo por sí solo no determina un árbol binario de búsqueda?

SOLUCIÓN Porque el interfijo es siempre la lista ordenada, cualquiera que sea la forma del árbol: los C_n árboles distintos con los mismos datos comparten interfijo. Hace falta el prefijo o el postfijo, que sí revelan quién es la raíz de cada subárbol.

6.7.111 (★★) A MANO ¿Cuántas comparaciones exige insertar un dato nuevo en un árbol binario de búsqueda de altura h ?

SOLUCIÓN A lo sumo $h + 1$: se desciende un nivel por comparación hasta encontrar el hueco. Con el árbol equilibrado eso es $\Theta(\ln n)$, y con el degenerado $\Theta(n)$. Insertar cuesta lo mismo que buscar.

6.7.112 (★★★) A MANO Describa cómo se elimina un nodo con dos hijos de un árbol binario de búsqueda.

SOLUCIÓN Se reemplaza por su **sucesor interfijo** (el menor del subárbol derecho) o por su predecesor, y se elimina ese nodo de su posición original, que tiene a lo sumo un hijo. Así se conserva la propiedad de orden. Reemplazarlo por cualquier otro descendiente la rompería.

6.7.113 (★★) MIXTO Verifique que dos órdenes de inserción distintos pueden dar el mismo interfijo.

SOLUCIÓN

In[] :=

```
{Sort[{5, 2, 8, 1, 9}], Sort[{8, 5, 9, 2, 1}]}
```

Out[] =

```
{{1, 2, 5, 8, 9}, {1, 2, 5, 8, 9}}
```

Los dos árboles tienen el mismo interfijo pero distinta raíz (5 y 8) y distinta altura. El contenido es el mismo; la estructura, no.

6.7.114 (★★★) A MANO ¿Cuántos árboles binarios de búsqueda con n datos tienen altura mínima cuando $n = 2^k - 1$?

SOLUCIÓN Exactamente **uno**: el árbol lleno, en el que cada nivel está completo. Cualquier otra forma tiene algún nivel incompleto y por lo tanto mayor altura. Para $n = 7$, de los $C_7 = 429$ árboles posibles solo uno tiene altura 2.

6.7.115 (★★) A MANO Escriba la notación polaca de $a - b - c$ interpretada de izquierda a derecha.

SOLUCIÓN La resta asocia por la izquierda: $(a - b) - c$. El árbol tiene por raíz la segunda resta, con hijo izquierdo el subárbol de $a - b$. El prefijo da

$$- - abc$$

Interpretada como $a - (b - c)$ daría $-a - bc$, distinta: la asociatividad **sí importa** al construir el árbol.

6.7.116 (★★★) A MANO Evalúe la polaca $- * 3 4 + 2 5$.

SOLUCIÓN Se lee de derecha a izquierda apilando operandos: 5, 2; el + da 7; 4, 3; el * da 12; el - toma 12 y 7 y da 5. Corresponde a $3 \cdot 4 - (2 + 5)$.

6.7.117 (★★) A MANO ¿Cuántos operadores tiene el árbol de una expresión con n operandos, si todos son binarios?

SOLUCIÓN Tiene $n - 1$: es un árbol binario completo con n hojas (los operandos) y por lo tanto $n - 1$ vértices internos: los operadores. La expresión $a + b * c - d$ tiene cuatro operandos y tres operadores.

6.7.118 (★★★) A MANO ¿Por qué el algoritmo de *Huffman* combina siempre las dos frecuencias menores?

SOLUCIÓN Porque en un árbol óptimo los dos símbolos menos frecuentes están en el nivel más profundo y son hermanos: si no lo fueran, intercambiarlos con los que sí lo están reduciría el total. Combinarlos primero garantiza que reciban los códigos más largos, y el resto del problema se resuelve recursivamente.

6.7.119 (★★★) A MANO Un texto tiene cuatro símbolos con frecuencias 1, 1, 2, 4. Construya el árbol de *Huffman* y dé las longitudes de código.

SOLUCIÓN Se combinan 1 y 1 en 2; luego ese 2 con el 2 original en 4; luego ese 4 con el 4 en 8. Las longitudes son 3, 3, 2, 1 y el total es $3 + 3 + 4 + 4 = 14$ bits, frente a los 16 de un código fijo de dos bits. Las frecuencias en progresión geométrica son el mejor caso para *Huffman*.

6.7.120 (★★) MIXTO Compruebe el cálculo anterior.

SOLUCIÓN

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
{huff[{1, 1, 2, 4}], 8*2}
```

Out[] =

{14, 16}

Catorce bits frente a dieciséis: un ahorro del 12,5 % con solo ocho símbolos.

6.7.121 (★★★) **MIXTO** Compare el ahorro de *Huffman* con frecuencias muy desiguales.

SOLUCIÓN

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
{huff[{1, 1, 1, 61}], 64*2}
```

Out[] =

{69, 128}

Sesenta y nueve bits frente a 128: casi la mitad. Con un símbolo dominante el código le asigna un solo bit y el ahorro es enorme.

6.7.122 (★★) **A MANO** ¿Qué información hay que transmitir junto con el texto codificado para que se pueda decodificar?

SOLUCIÓN El **árbol** o, equivalentemente, la tabla de códigos. Sin ella el receptor no puede reconstruir los símbolos. Ese costo adicional es la razón de que *Huffman* no compense en textos muy cortos: la tabla puede pesar más que el ahorro.

6.7.123 (★★★) **A MANO** ¿Cuántos árboles generadores tiene el grafo formado por dos triángulos que comparten un vértice?

SOLUCIÓN Nueve. Cada triángulo aporta tres árboles generadores propios (se descarta una de sus tres aristas) y las elecciones son independientes: $3 \cdot 3 = 9$. El vértice compartido no añade opciones porque no crea circuitos nuevos.

6.7.124 (★★) **A MANO** Un grafo conexo tiene un único árbol generador. ¿Qué se puede afirmar?

SOLUCIÓN Que el grafo **ya es un árbol**. Si tuviera un circuito, quitar cualquiera de sus aristas daría un árbol generador distinto. La unicidad del árbol generador caracteriza a los árboles.

6.7.125 (★★★) **A MANO** Compare el árbol generador de una búsqueda a lo ancho con el camino más corto de *Dijkstra*.

SOLUCIÓN En un grafo **sin pesos** coinciden: la búsqueda a lo ancho encuentra caminos de mínimo número de aristas desde la raíz, que es lo que minimiza *Dijkstra* con todos los pesos iguales a uno. Con pesos distintos dejan de coincidir, y la búsqueda a lo ancho ya no da caminos óptimos.

6.7.126 (★★★) **MIXTO** Compruebe esa coincidencia sobre el grafo de referencia.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
{BuscarPrimeroAncho[Grafo[Ag]], LongitudCaminoMC[Grafo[Ag], 1, 6]}
```

Out[] =

$\{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 5\}, \{4, 6\}\}, \{3, \{\{1, 2\}, \{2, 4\}, \{4, 6\}\}\}$

El camino $1 - 2 - 4 - 6$ que devuelve `LongitudCaminoMC` está contenido en el árbol de la búsqueda a lo ancho, y tiene tres aristas, el mínimo posible.

6.7.127 (★★) **A MANO** ¿Qué estructura de datos usa cada búsqueda y cómo explica su forma?

SOLUCIÓN La búsqueda a lo **ancho** usa una *cola*: lo primero que entra es lo primero que sale, de modo que se agotan los vecinos antes de bajar. La búsqueda a lo **largo** usa una *pila*: lo último que entra sale primero, y por eso profundiza. El cambio de estructura es la única diferencia entre los dos algoritmos.

6.7.128 (★★★) **A MANO** ¿Por qué *Prim* y *Kruskal* pueden devolver árboles distintos con el mismo peso?

SOLUCIÓN Cuando hay **empates** de peso, la decisión entre aristas equivalentes es arbitraria y depende del orden interno. Ambos árboles son óptimos: el peso total coincide. Con todos los pesos distintos el árbol de expansión mínima es único y los dos algoritmos llegan al mismo.

6.7.129 (★★★) **A MANO** *Prim* parte de un vértice y *Kruskal* de la arista más barata. ¿Afecta eso al resultado?

SOLUCIÓN Al **peso total**, no: ambos alcanzan el óptimo. Al **árbol concreto**, sí puede, si hay empates. Lo que sí cambia es la estrategia intermedia: *Prim* mantiene siempre un único árbol que crece, y *Kruskal* mantiene un bosque que se va uniendo.

6.7.130 (★★) **VilCretas** Ejecute *Prim* sobre el grafo ponderado de referencia y compare con *Kruskal*.

SOLUCIÓN

`In[ ] :=`

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
Prim[Grafo[Ag, pesos -> Pg], all -> True]
```

Prim arranca en el vértice 1 y añade en cada paso la arista más barata que toca el árbol ya construido. Llega al mismo peso 15 que *Kruskal*, aunque el orden de incorporación es distinto.

6.7.131 (★★★) **A MANO** ¿Cuál es el costo asintótico de *Kruskal* y de *Prim*?

SOLUCIÓN *Kruskal* debe ordenar las p aristas, lo que domina: $\Theta(p \ln p)$. *Prim* con búsqueda lineal del mínimo es $\Theta(n^2)$, y con cola de prioridad $\Theta(p \ln n)$. En grafos **densos** conviene *Prim* y en grafos **dispersos**, *Kruskal*.

6.7.132 (★★★) **A MANO** Un árbol de expansión mínima, ¿contiene siempre la arista de menor peso del grafo?

SOLUCIÓN Sí, si esa arista es **única** en su peso. Por la propiedad del corte, la arista más barata que cruza cualquier partición pertenece a algún árbol óptimo, y la más barata del grafo cruza alguna. Con empates puede quedar fuera, pero alguna de las empatadas entra.

6.7.133 (★★★) **A MANO** ¿Contiene siempre la arista de mayor peso? Justifique.

SOLUCIÓN No, salvo que sea un **punto**. Si la arista más cara pertenece a un circuito, siempre se puede descartar en favor de las otras del circuito. Es la *propiedad del ciclo*, dual de la del corte: la arista más cara de cualquier circuito no pertenece a ningún árbol óptimo.

6.7.134 (★★) **A MANO** Aplique la propiedad del ciclo al triángulo con pesos 2, 3 y 8.

SOLUCIÓN La arista de peso 8 pertenece al único circuito, de modo que **no** está en el árbol de expansión mínima. El árbol es $\{2, 3\}$ con peso total 5. Descartar la arista más cara de cada circuito es una forma alternativa de construir el árbol óptimo.

6.7.135 (★★★) **MIXTO** Compruebe el resultado anterior.

SOLUCIÓN

In[] :=

```
tri = Graph[{1 <-> 2, 2 <-> 3, 1 <-> 3}, EdgeWeight -> {2, 3, 8}];
Total[PropertyValue[{tri, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[tri]]]
```

Out[] =

5

El árbol usa las dos aristas baratas y pesa 5, dejando fuera la de peso 8.

6.7.136 (★★★) **A MANO** Un grafo tiene n vértices y todas las aristas del mismo peso c . ¿Cuánto pesa su árbol de expansión mínima?

SOLUCIÓN Pesa $(n-1)c$, sea cual sea el árbol elegido: todos tienen $n-1$ aristas y todas cuestan lo mismo. El problema de optimización desaparece, pero sigue siendo necesario **encontrar** un árbol generador, que es lo que hacen BFS o DFS.

6.7.137 (★★) **A MANO** ¿Qué ocurre si se aplica *Kruskal* a un grafo no conexo?

SOLUCIÓN Devuelve un **bosque** de expansión mínima: un árbol óptimo por cada componente conexa. El resultado tiene $n - c$ aristas con c componentes, no $n - 1$. El algoritmo no falla; simplemente se queda sin aristas que añadir.

6.7.138 (★★★) **A MANO** Un árbol de expansión mínima de un grafo con 10 vértices pesa 45. ¿Qué se puede decir del peso medio de sus aristas?

SOLUCIÓN Tiene 9 aristas, de modo que el peso medio es 5. No se puede decir nada sobre los pesos individuales: podrían ser todos 5 o ir de 1 a 37. Tampoco se puede acotar el peso de las aristas descartadas, salvo que ninguna de ellas mejoraría el total.

6.7.139 (★★) **A MANO** Resuma en qué se diferencian el árbol generador, el árbol de expansión mínima y el árbol de caminos más cortos.

SOLUCIÓN El **generador** solo exige conectar todos los vértices sin circuitos. El de **expansión mínima** minimiza el **peso total**. El de **caminos más cortos** minimiza la distancia **desde un vértice fijo** a cada uno de los demás. Los dos últimos suelen ser árboles distintos: optimizan cosas diferentes.

6.7.140 (★★★) **MIXTO** Exhiba un grafo donde el árbol de expansión mínima y el de caminos más cortos desde el vértice 1 sean distintos.

SOLUCIÓN

In[] :=

```
gx = Graph[{1 <-> 2, 2 <-> 3, 1 <-> 3}, EdgeWeight -> {1, 1, 1.9}];
{Total[PropertyValue[{gx, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gx]]],
```



```
GraphDistance[gx, 1, 3]
```

```
Out[ ] =
```

```
{2, 1.9}
```

El árbol de expansión mínima usa las dos aristas de peso 1 y pesa 2; pero el camino más corto de 1 a 3 es la arista directa, de peso 1,9, que **no** pertenece a ese árbol.

- 6.7.141 (★★★) A MANO** Un árbol con raíz se recorre en prefijo y se obtiene {1, 2, 4, 5, 3, 6}; en postfijo, {4, 5, 2, 6, 3, 1}. Reconstrúyalo.

SOLUCIÓN La raíz es 1, primera en prefijo y última en postfijo. En postfijo, el subárbol izquierdo termina en 2: sus nodos son 4, 5, 2. El derecho es 6, 3. Repitiendo, 2 tiene hijos 4 y 5, y 3 tiene el hijo 6. El árbol es 1 con hijos 2 y 3; 2 con hijos 4 y 5; 3 con hijo 6.

- 6.7.142 (★★) MIXTO** Compruebe la reconstrucción anterior.

SOLUCIÓN

```
In[ ] :=
```

```
T3 = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}}];
{Prefijo[T3, 1], Postfijo[T3, 1]}
```

```
Out[ ] =
```

```
{{1, 2, 4, 5, 3, 6}, {4, 5, 2, 6, 3, 1}}
```

Los dos recorridos coinciden con los dados: la reconstrucción es correcta. Con prefijo y postfijo juntos el árbol queda determinado salvo el orden de los hijos.

- 6.7.143 (★★★) A MANO** ¿Bastan el prefijo y el postfijo para determinar un árbol binario?

SOLUCIÓN No siempre. Si un nodo tiene un **solo hijo**, no hay forma de saber si es el izquierdo o el derecho: los dos casos dan el mismo prefijo y el mismo postfijo. Con árboles binarios *completos* sí bastan. La combinación que siempre determina es prefijo más interfijo.

6.6 Recorrido intermedio, árboles etiquetados y generadores

- 6.7.144 (★★★) MIXTO** El recorrido *interorden* de un árbol binario de búsqueda debe dar los datos ordenados. Compruébelo con `Interfijo` y la opción `direction -> True`.

SOLUCIÓN

```
In[ ] :=
```

```
datos = {20, 5, 3, 21, 25, 10, 9, 8, 4, 27, 19, 30};
Ad = ArbolBinarioBusqueda[datos, direction -> True];
{Interfijo[Ad, 20], Interfijo[Ad, 20] == Sort[datos]}
```

```
Out[ ] =
```

```
{{3, 4, 5, 8, 9, 10, 19, 20, 21, 25, 27, 30}, True}
```

Sale la lista ordenada. Esa es **la propiedad que define** al árbol binario de búsqueda: todo lo que está a la izquierda de un nodo es menor y todo lo que está a la derecha es mayor, de modo que recorrer izquierda-raíz-derecha ordena.

6.7.145 (★★★) **MIXTO** Repita el recorrido sin `direction` -> `True` y explique por qué el resultado ya no sale ordenado.

SOLUCIÓN

In[] :=

```
datos = {20, 5, 3, 21, 25, 10, 9, 8, 4, 27, 19, 30};
A = ArbolBinarioBusqueda[datos];
Interorden[A, 20]
```

Out[] =

```
{4, 3, 5, 8, 9, 10, 19, 20, 30, 27, 25, 21}
```

No está ordenado. Sin `direction` -> `True` el árbol se dibuja correctamente pero **no guarda cuál hijo es el izquierdo**: cuando un nodo tiene un solo hijo, la información se pierde, y el recorrido intermedio no puede reconstruirla. Es un fallo **silencioso**: no hay error, solo una lista equivocada.

6.7.146 (★★★) **A MANO** ¿Qué conclusión práctica saca de los dos ejercicios anteriores?

SOLUCIÓN Que **la opción no cambia solo la apariencia**. `direction` -> `True` cambia el objeto que se construye, no cómo se ve; y `Interfijo` es el recorrido que sabe leerlo. La comprobación `Interfijo[...] == Sort[datos]` es la prueba barata de que se hizo bien: si falla, algo se perdió por el camino.

6.7.147 (★★) **MIXTO** Compare los tres recorridos clásicos del mismo árbol.

SOLUCIÓN

In[] :=

```
datos = {20, 5, 3, 21, 25, 10, 9, 8, 4, 27, 19, 30};
A = ArbolBinarioBusqueda[datos];
{Prefijo[A, 20], Postfijo[A, 20]}
```

Out[] =

```
{{20, 5, 3, 4, 10, 9, 8, 19, 21, 25, 27, 30}, {4, 3, 8, 9, 19, 10, 5, 30, 27, 25, 21, 20}}
```

El prefijo empieza por la raíz y el postfijo termina en ella: eso vale siempre y es la primera comprobación que debe hacerse. El prefijo, además, basta por sí solo para reconstruir un árbol binario de búsqueda.

6.7.148 (★★★) **MIXTO** Construya el árbol de la expresión $a \cdot b + c$ con `ArbolEtiquetado` y obtenga sus notaciones polaca y polaca inversa.

SOLUCIÓN

In[] :=

```
{ArbolEtiquetado[{"+", "*", "c", "a", "b"}, polaca -> True][[2]],
ArbolEtiquetado[{"+", "*", "c", "a", "b"}, polacaInversa -> True][[2]]}
```

Out[] =

```
{+ * a b c, a b * c +}
```

El vector se escribe **por niveles**, de arriba abajo y de izquierda a derecha. La polaca es el recorrido **prefijo** y la polaca inversa el **postfijo**; ninguna necesita paréntesis, porque la posición del operador ya fija el orden de las operaciones.

6.7.149 (★★★) A MANO ¿Cómo se evalúa una expresión en polaca inversa con una pila, y por qué basta con una?

SOLUCIÓN Se recorre la cadena de izquierda a derecha: cada operando se apila y cada operador desapila dos valores, opera y apila el resultado. Al terminar queda un solo valor, que es el de la expresión. Basta una pila porque el orden de las operaciones ya viene fijado por la posición de los operadores, de modo que **nunca hay que retroceder** ni recordar una operación pendiente para volver a ella. Sobre $a \ b \ * \ c \ +$: se apilan a y b , el $*$ los sustituye por su producto, se apila c y el $+$ cierra. Es la misma pila del capítulo 1.

6.7.150 (★★) MIXTO HojasR devuelve las hojas de un árbol con raíz. Compare su resultado con el de Hojas sobre el mismo árbol.

SOLUCIÓN

In[] :=

```
datos = {20, 5, 3, 21, 25, 10, 9, 8, 4, 27, 19, 30};
A = ArbolBinarioBusqueda[datos];
{Length[Hojas[A]],
Length[Select[HojasR[ArbolR[ListaAristas[A], 20], 20], NumberQ]]}
```

Out[] =

{9, 4}

No coinciden. **Hojas** cuenta también los **nodos vacíos** que el dibujo introduce para marcar el hueco del hijo que falta (aparecen como cadenas de espacios), mientras que al filtrar por **NumberQ** quedan las cuatro hojas de verdad: 4, 8, 19 y 30. Antes de contar hojas, hay que mirar qué hay dentro de la lista.

6.7.151 (★★) MIXTO Compruebe con **ArbolBinarioReturnQ** que un árbol con un nodo de tres hijos no es binario.

SOLUCIÓN

In[] :=

```
{ArbolBinarioReturnQ[Arbol[{{1, 2}, {1, 3}, {1, 4}}], 1],
ArbolBinarioReturnQ[ArbolBinarioBusqueda[{20, 5, 3, 21}], 20]}
```

Out[] =

{False, True}

La diferencia con **ArbolBinarioQ** está en la salida: esta versión devuelve **solo el valor lógico**, sin dibujar nada, de modo que se puede usar dentro de un **If** o de un **Select**.

6.7.152 (★★★) MIXTO Use **ArbolBinarioCompletoQ** y explique la diferencia entre binario y binario completo.

SOLUCIÓN

In[] :=

```
ArbolBinarioCompletoQ[
ArbolRBinario[ListaAristas[ArbolBinarioBusqueda[{5, 3, 8}]], 5], 5]
```

Out[] =

True

Binario significa **a lo sumo** dos hijos por nodo; completo significa que cada nodo interno tiene **exactamente** dos. Todo árbol binario completo con h hojas tiene $h - 1$ nodos internos, relación que se usa al analizar los códigos de *Huffman*.

- 6.7.153 (★★★) MIXTO** Obtenga el árbol de expansión mínima con `ArbolGeneradorPrim` y con `ArbolGeneradorKruskal`, y compare.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
{ArbolGeneradorPrim[gp][[2]], ArbolGeneradorKruskal[gp][[2]]}
```

Out[] =

{6, 6}

Los dos pesan 6, como debe ser: el peso mínimo es único aunque el árbol no tenga por qué serlo. Frente a `AlPrim` y `AlKruskal`, que despliegan el procedimiento, estas versiones devuelven **{aristas, peso}**, listo para encadenar.

- 6.7.154 (★★★) MIXTO** Cambie el orden de los vértices en `ArbolGeneradorPrim` y observe qué cambia y qué no.

SOLUCIÓN

In[] :=

```
gp = Grafo[{{a, b}, {a, c}, {b, c}, {b, d}, {c, d}},
pesos -> {2, 5, 1, 4, 3}, mostrarpesos -> True];
{ArbolGeneradorPrim[gp], ArbolGeneradorPrim[gp, orden -> {d, c, b, a}]}
```

Out[] =

{{{a, b}, {b, c}, {c, d}}, 6}, {{{d, c}, {c, b}, {b, a}}, 6}}

El **peso no cambia**; cambia el orden en que se descubren las aristas y la orientación con que se imprimen. Como el grafo no es dirigido, $\{d, c\}$ y $\{c, d\}$ son la misma arista: es el mismo árbol leído al revés.

- 6.7.155 (★★★) MIXTO** Codifique la palabra *lalo* con el árbol de *Huffman* generado por la frase *la loma de lala*, usando `HileraArbolHuffman`.

SOLUCIÓN

In[] :=

```
HileraArbolHuffman["lalo", "la loma de lala"]
```

Out[] =

1001010

El comando construye el árbol de códigos a partir de las frecuencias de la segunda hilera y codifica la primera con él. La codificación **depende del texto de referencia**: la misma palabra sobre otra frase da otra secuencia de bits, porque cambian las frecuencias.

6.7.156 (★★★) **VilCretas** Decodifique esos mismos bits y muestre el árbol y las frecuencias.

SOLUCIÓN

In[] :=

```
HileraArbolHuffman["1001010", "la loma de lala",
  tree -> True, frecuencias -> True]
```

Las dos direcciones usan el mismo comando: si la primera hilera son letras **codifica** y si son ceros y unos **decodifica**: son excluyentes. **tree -> True** dibuja el árbol y **frecuencias -> True** muestra el vector de frecuencias del que sale todo.

6.7.157 (★★) **VilCretas** Compare el código de *Huffman* corriente con el optimizado.

SOLUCIÓN

In[] :=

```
{HileraArbolHuffman["lalo", "la loma de lala", tree -> True],
HileraArbolHuffman["lalo", "la loma de lala", optimizado -> True, tree -> True]}
```

La opción **optimizado -> True** desempata de otra forma los nodos de igual frecuencia. El árbol resultante puede ser distinto, pero la **longitud media** del código es la misma: los árboles de *Huffman* óptimos no son únicos, su coste sí.

6.7.158 (★★) **MIXTO** ArbolJQ imprime su respuesta. Haga que la devuelva con **vectorial -> True**.

SOLUCIÓN

In[] :=

```
ArbolJQ[ArbolBinarioBusqueda[{20, 5, 3, 21, 25}], vectorial -> True]
```

Out[] =

```
{True, True, False}
```

Devuelve un vector de valores lógicos en lugar de escribir en pantalla, lo que permite usarlo en un cálculo posterior. Es el mismo patrón que **matriz -> True** en **MGrafo**: preguntarse siempre si el comando *muestra* o *devuelve*.

6.7.159 (★★) **VilCretas** Genere árboles de práctica con **ArbolRRandom** y **ArbolBinarioRRandom**, y diga qué devuelven exactamente.

SOLUCIÓN

In[] :=

```
{ArbolRRandom[15, letras -> True], ArbolBinarioRRandom[15, letras -> True]}
```

Devuelven un **vector de dos componentes**: el árbol y su raíz. Eso importa, porque casi todos los comandos de este capítulo piden la raíz como segundo argumento, y aquí viene ya dada, no hay que adivinarla del dibujo.

- 6.7.160** (★★★) VilCretas Use `ArbolDirection` para dar direcciones a los nodos con un solo hijo, y diga por qué hace falta.

SOLUCIÓN

`In[ ] :=`

```
$RecursionLimit = 10000;
r = ArbolBinarioRRandom[12, letras -> True];
ArbolDirection[r[[1]], r[[2]]]
```

Un árbol binario dibujado sin direcciones deja sin decidir, en los nodos de un solo hijo, si ese hijo es el izquierdo o el derecho. `ArbolDirection` las asigna al azar, y con eso el árbol queda **completamente determinado**: ya se puede recorrer en interorden sin ambigüedad.

Por qué se sube `$RecursionLimit`: el comando recorre el árbol de forma recursiva, y si el árbol aleatorio sale muy desbalanceado (casi una cadena) la profundidad puede pasar del límite de 1024 que trae *Mathematica* por omisión. El síntoma es `$RecursionLimit::reclim` y un resultado a medias. Con árboles de una docena de nodos rara vez ocurre; con veinte o más, conviene subirlo antes.

- 6.7.161** (★★★) VilCretas Construya un árbol binario a partir de una lista de aristas con `ArbolRBinario`, y compárelo con `ArbolRBinarioBlancos`.

SOLUCIÓN

`In[ ] :=`

```
ArbolRBinario[{{b, e}, {c, d}, {e, c}, {f, b}, {f, n},
  {g, m}, {h, i}, {m, h}, {n, g}}, f]
```

`ArbolRBinario` decide las direcciones con la lógica de un árbol binario de búsqueda. `ArbolRBinarioBlancos` sirve cuando la lista trae nodos vacíos (cadenas de espacios) que marcan el hueco de un hijo ausente: los deja **invisibles** en el dibujo, pero conserva la posición, que es lo que el recorrido necesita.

- 6.7.162** (★★) VilCretas Genere árboles con letras en lugar de números.

SOLUCIÓN

`In[ ] :=`

```
{ArbolRandom[20, letras -> True], ArbolBinario[20, letras -> True],
ArbolBinarioRandom[20, letras -> True]}
```

La opción `letras -> True` sustituye los números por letras del abecedario. No es solo estético: con números es fácil confundir la **etiqueta** de un vértice con un **peso** o con el orden de un recorrido, y con letras esa confusión desaparece.

- 6.7.163** (★★★) VilCretas Dibuje con `ArbolRBinarioBlancos` un árbol binario en el que algunos padres tienen un solo hijo, marcando el hueco del que falta.

SOLUCIÓN

`In[ ] :=`

```
ArbolRBinarioBlancos[{{a, b}, {a, f}, {b, c}, {b, d},
  {d, e}, {d, k}, {e, l}, {e, " "}, {f, ""},
  {f, g}, {g, h}, {g, " "}, {h, i}, {h, j}}, a]
```

Los nodos "", " " y " " son huecos: cadenas de espacios, todas distintas entre sí para que el grafo no las confunda en una sola. El comando las deja **invisibles** en el dibujo pero conserva su posición, y eso es justo lo que necesita el recorrido intermedio para saber si el hijo que hay es el izquierdo o el derecho.

CAPÍTULO 7 • COMPLEMENTARIOS

Máquinas de estado finito y autómatas

7.1 Máquinas de estado finito

7.6.1 (★) **A MANO** Enumere las seis componentes que definen una máquina de estado finito.

SOLUCIÓN El conjunto de **estados**, el de **símbolos de entrada**, el de **símbolos de salida**, el **estado inicial**, la **función estado siguiente** y la **función de salida**. Las dos últimas asignan, a cada par (estado, entrada), el estado al que se pasa y el símbolo que se emite.

7.6.2 (★) **A MANO** ¿En qué se diferencia una máquina de estado finito de un autómata?

SOLUCIÓN La máquina **produce una hilera de salida**; el autómata no emite nada y se limita a **aceptar o rechazar** la hilera de entrada según el estado en que termina. Por eso el autómata tiene cinco componentes en lugar de seis: en vez de salidas tiene un conjunto de estados aceptados.

7.6.3 (★★) **A MANO** Construya la tabla de la máquina de dos estados s_0 y s_1 que lleva la paridad del número de unos leídos.

SOLUCIÓN El estado s_0 significa *cantidad par de unos* y s_1 , *impar*. Un 1 cambia de estado y un 0 lo conserva; la salida es la paridad tras la lectura:

	estado siguiente		salida	
	0	1	0	1
s_0	s_0	s_1	0	1
s_1	s_1	s_0	1	0

7.6.4 (★★) **VilCretas** Procese la hilera 10110 en esa máquina.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
StringSalida[{s0, s1}, {0, 1}, {0, 1}, s0, FE, {1, 0, 1, 1, 0}]
```

Out[] =

```
{1, 1, 0, 1, 1}
```

Cada fila de la matriz contiene, en orden, el estado, la entrada, el estado siguiente y la salida. La hilera de salida marca con 1 las posiciones en que se llevaban leídos un número impar de unos.

7.6.5 (★★) **MIXTO** Procese ahora 0010 y verifique el resultado a mano.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
StringSalida[{s0, s1}, {0, 1}, {0, 1}, s0, FE, {0, 0, 1, 0}]
```

Out[] =

{0, 0, 1, 1}

A mano: se leen 0, 0 (ningún uno, paridad par, salida 0), luego 1 (paridad impar, salida 1) y 0 (sigue impar, salida 1).

7.6.6 (★★) **MIXTO** Procese 11110 y explique por qué la salida alterna.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
StringSalida[{s0, s1}, {0, 1}, {0, 1}, s0, FE, {1, 1, 1, 1, 0}]
```

Out[] =

{1, 0, 1, 0, 0}

Cada 1 cambia la paridad, de modo que mientras la entrada sea una racha de unos la salida alterna. El 0 final no la cambia y repite el último valor.

7.6.7 (★★) **VilCretas** Obtenga también la lista de estados visitados al procesar 101.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
StringSalida[{s0, s1}, {0, 1}, {0, 1}, s0, FE, {1, 0, 1}, trace -> True]
```

La salida impresa es $\{\{1, 1, 0\}, \{s_0, s_1, s_1, s_0\}\}$ junto con una animación del recorrido. Trae dos listas: la hilera producida y el recorrido de estados, que incluye el inicial y por eso tiene un elemento más que la entrada. Con esta opción el comando **imprime** el resultado en vez de devolverlo.

7.6.8 (★★) **VilCretas** Dibuje el diagrama de transición de esa máquina.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
MaquinaToDiagrama[{s0, s1}, {0, 1}, {0, 1}, s0, FE]
```

En el diagrama cada arista se rotula *entrada/salida*. Los lazos corresponden a las transiciones que no cambian de estado. **shape->True** coloca los nodos en discos y **padding** ajusta el margen.

7.6.9 (★★) **VilCretas** Procese hileras de forma interactiva sobre esa máquina.

SOLUCIÓN

In[] :=

```
FE = {{s0, 0, s0, 0}, {s0, 1, s1, 1}, {s1, 0, s1, 1}, {s1, 1, s0, 0}};
CDFMaquinaEF[{s0, s1}, {0, 1}, {0, 1}, s0, FE, {1, 0, 1, 1}]
```

El comando muestra el diagrama en una animación y permite cambiar la hilera de entrada sin volver a escribir la máquina. Es el recurso adecuado para explorar varios casos seguidos.

- 7.6.10** (★) **VilCretas** Observe el diagrama de la máquina que suma dos números binarios.

SOLUCIÓN

In[] :=

`MaquinaSumadoraBinarios[]`

El comando no recibe argumentos. La máquina tiene dos estados, que representan si hay o no acarreo, y las entradas son los pares de bits que se suman en cada posición.

- 7.6.11** (★★★) **A MANO** Diseñe una máquina de estado finito que emita 1 cada vez que acaba de leer la subhilera 11.

SOLUCIÓN Con dos estados: s_0 (el último símbolo no fue un 1) y s_1 (el último fue un 1).

	0	1	0	1
s_0	s_0	s_1	0	0
s_1	s_0	s_1	0	1

Solo se emite 1 al leer un 1 estando ya en s_1 , es decir, ante el segundo uno consecutivo.

- 7.6.12** (★★★) **A MANO** ¿Cuántos estados necesita una máquina que reconozca el resto de la división entre 3 del número de símbolos leídos?

SOLUCIÓN Tres: uno por cada resto posible, 0, 1 y 2. Cada símbolo leído avanza cíclicamente al estado siguiente. En general, para el resto módulo k hacen falta k estados.

- 7.6.13** (★★) **A MANO** ¿Por qué la hilera de salida tiene siempre la misma longitud que la de entrada?

SOLUCIÓN Porque la función de salida emite exactamente un símbolo por cada transición, y hay una transición por cada símbolo leído. El recorrido de estados, en cambio, tiene un elemento más: el estado inicial.

- 7.6.14** (★★★) **MIXTO** Construya la máquina de *retardo*, que emite en cada paso el símbolo leído en el paso anterior, y procésela sobre 1011.

SOLUCIÓN Los estados recuerdan el último símbolo leído: t_0 si fue un 0 (o es el inicio) y t_1 si fue un 1.

In[] :=

```
FR = {{t0, 0, t0, 0}, {t0, 1, t1, 0}, {t1, 0, t0, 1}, {t1, 1, t1, 1}};
StringSalida[{t0, t1}, {0, 1}, {0, 1}, t0, FR, {1, 0, 1, 1}]
```

Out[] =

```
{0, 1, 0, 1}
```

La salida es la entrada corrida una posición a la derecha, con un 0 inicial: la máquina memoriza un único símbolo.

- 7.6.15** (★★) **A MANO** ¿Cuántas filas debe tener la matriz de una máquina con tres estados y dos símbolos de entrada?

SOLUCIÓN Seis: una por cada par (estado, entrada). Cada fila lleva cuatro componentes, el estado, la entrada, el estado siguiente y la salida. Si falta alguna fila, varios comandos del paquete se quedan **sin evaluar en silencio**.

7.2 Autómatas determinísticos

7.6.16 (★) **A MANO** Enumere las cinco componentes de un autómata de estado finito determinístico.

SOLUCIÓN El conjunto de estados σ , el de símbolos de entrada τ , el estado inicial, la función de transición de estados y el conjunto de estados aceptados. Una hilera se acepta si el recorrido que define termina en un estado aceptado.

7.6.17 (★) **VilCretas** Construya el autómata que acepta las hileras sobre $\{a, b\}$ con una cantidad par de a , y compruebe que acepta aa .

SOLUCIÓN

In[] :=

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
StringAceptadaQ[A, {a, a}]
```

Out[] =

True

El estado q_0 representa *cantidad par de* a y q_1 , *impar*. Cada a cambia de estado y cada b lo conserva. Este autómata se reutiliza en los ejercicios siguientes.

7.6.18 (★) **VilCretas** Decida si ese autómata acepta aba y ab .

SOLUCIÓN

In[] :=

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
{StringAceptadaQ[A, {a, b, a}], StringAceptadaQ[A, {a, b}]}
```

Out[] =

{True, False}

La primera tiene dos a y la segunda una sola. Solo importa la **paridad**: la posición de las letras es irrelevante.

7.6.19 (★★) **VilCretas** Obtenga el recorrido de estados al procesar aba .

SOLUCIÓN

In[] :=

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
StringAceptadaQ[A, {a, b, a}, trace -> True]
```

Out[] =

{True, {q0, q1, q1, q0}}

Con `trace->True` se ve la trayectoria completa: la b deja el autómata en q_1 y la segunda a lo devuelve a q_0 , que es aceptado.

7.6.20 (★★) **VilCretas** ¿Acepta el autómata la hilera vacía?

SOLUCIÓN**In[] :=**

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
StringAceptadaQ[A, {}]
```

Out[] =

True

Sí: sin leer nada el autómata sigue en el estado inicial, que es aceptado. Cero es una cantidad par de a .

7.6.21 (★) **VilCretas** Recupere las cinco componentes del autómata.**SOLUCIÓN****In[] :=**

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
ComponentesAutomata[A]
```

La salida lista los estados, los símbolos de entrada, el estado inicial, los estados aceptados y la función de transición, esta última en forma de tabla y también como lista de tripletas.

7.6.22 (★) **VilCretas** Dibuje el diagrama de transición del autómata.**SOLUCIÓN****In[] :=**

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
AutomataToDiagrama[A, colores -> True]
```

El estado aceptado se dibuja con doble círculo. Las opciones `forma->"rectangular"` cambian la disposición y `reducido->True` oculta los estados inalcanzables.

7.6.23 (★★) **VilCretas** Enumere las hileras de longitud dos y de longitud tres que acepta el autómata.**SOLUCIÓN****In[] :=**

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
LenguajeStrings[A, 2]
LenguajeStrings[A, 3]
```

Out[] =

```
{{a, a}, {b, b}}
{{a, a, b}, {a, b, a}, {b, a, a}, {b, b, b}}
```

En longitud tres se aceptan las que tienen cero o dos a . La cuenta coincide con las combinaciones posibles de posiciones para las a .

7.6.24 (★★) VilCretas Cuente cuántas hileras de longitud tres y de longitud cuatro acepta.

SOLUCIÓN

In[] :=

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
LenguajeCantidad[A, 3], LenguajeCantidad[A, 4]
```

Out[] =

{4, 8}

De las 2^n hileras posibles de longitud n , exactamente la mitad tiene una cantidad par de a : 2^{n-1} . Para $n = 3$ son 4 y para $n = 4$, 8.

7.6.25 (★★) VilCretas Obtenga los elementos del lenguaje de longitud menor o igual que dos.

SOLUCIÓN

In[] :=

```
A = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q0}];
LenguajeStrings[A, 2, limite -> True]
```

Out[] =

{{}, {b}, {a, a}, {b, b}}

Con `limite->True` el comando acumula todas las longitudes hasta la indicada, incluida la hilera vacía. `LenguajeCantidad` admite la misma opción.

7.6.26 (★★★) VilCretas Construya un autómata que acepte las hileras sobre $\{a, b\}$ que terminan en ab y compruébelo en tres casos.

SOLUCIÓN

In[] :=

```
Dfa = Automata[{r0, r1, r2}, {a, b}, r0,
{{r0, a, r1}, {r0, b, r0}, {r1, a, r1}, {r1, b, r2},
{r2, a, r1}, {r2, b, r0}}, {r2}];
[StringAceptadaQ[Dfa, {a, b}], StringAceptadaQ[Dfa, {a, a, b}],
StringAceptadaQ[Dfa, {a, b, b}]]
```

Out[] =

{True, True, False}

El estado r_1 significa *acabo de leer una a* y r_2 , *acabo de leer ab*. La tercera hilera termina en bb y por eso se rechaza.

7.6.27 (★★) VilCretas Enumere las hileras de longitud tres que acepta ese autómata.

SOLUCIÓN

In[] :=

```
Dfa = Automata[{r0, r1, r2}, {a, b}, r0,
  {{r0, a, r1}, {r0, b, r0}, {r1, a, r1}, {r1, b, r2},
  {r2, a, r1}, {r2, b, r0}}, {r2}];
LenguajeStrings[Dfa, 3]
```

Out[] =

```
{{a, a, b}, {b, a, b}}
```

Son las dos únicas que terminan en ab : el primer símbolo puede ser cualquiera de los dos.

- 7.6.28** (★★) VilCretas Construya un autómata que acepte exactamente las hileras con dos a y una b .

SOLUCIÓN

In[] :=

```
AutomataExactamente[{{a, 2}, {b, 1}}];
LenguajeStrings[G, 3]
```

Out[] =

```
{{a, a, b}, {a, b, a}, {b, a, a}}
```

`AutomataExactamente` recibe una matriz de dos columnas con el carácter y la cantidad exacta de repeticiones, dibuja el diagrama y deja el autómata almacenado en la variable `G`. Las tres hileras son las permutaciones posibles.

- 7.6.29** (★★★) A MANO Diseñe a mano un autómata que acepte las hileras sobre $\{0, 1\}$ que representen múltiplos de 3 en binario, leídas de izquierda a derecha.

SOLUCIÓN Los estados son los restos t_0 , t_1 y t_2 módulo 3, con inicio y aceptación en t_0 . Al leer un bit x el número pasa de v a $2v + x$, de modo que el resto r pasa a $(2r + x) \bmod 3$:

	0	1
t_0	t_0	t_1
t_1	t_2	t_0
t_2	t_1	t_2

- 7.6.30** (★★) A MANO ¿Qué significa que un autómata sea determinístico?

SOLUCIÓN Que para cada par (estado, símbolo de entrada) hay **exactamente un** estado siguiente. La trayectoria que define una hilera es única, y por eso aceptar o rechazar es una decisión sin ambigüedad.

- 7.6.31** (★★★) A MANO Un autómata determinístico tiene n estados y m símbolos de entrada. ¿Cuántas filas tiene su tabla de transición?

SOLUCIÓN Tiene $n \cdot m$: una por cada par (estado, entrada). Si falta alguna, el autómata deja de ser determinístico o queda incompleto, y varios comandos del paquete se quedan **sin evaluar** en ese caso.

- 7.6.32** (★★) VilCretas Construya el autómata que acepta las hileras sobre $\{a, b\}$ que empiezan con a , y compruébelo.

SOLUCIÓN

In[] :=

```
Ini = Automata[{u0, u1, u2}, {a, b}, u0,
{{u0, a, u1}, {u0, b, u2}, {u1, a, u1}, {u1, b, u1},
{u2, a, u2}, {u2, b, u2}}, {u1}];
{StringAceptadaQ[Ini, {a}], StringAceptadaQ[Ini, {b, a}]}
```

Out[] =

```
{True, False}
```

El estado u_1 es la trampa de aceptación y u_2 la de rechazo: una vez leído el primer símbolo, la decisión ya no cambia.

7.6.33 (★★) **VilCretas** Enumere y cuente el lenguaje de ese autómata en longitud dos y tres.

SOLUCIÓN

In[] :=

```
Ini = Automata[{u0, u1, u2}, {a, b}, u0,
{{u0, a, u1}, {u0, b, u2}, {u1, a, u1}, {u1, b, u1},
{u2, a, u2}, {u2, b, u2}}, {u1}];
{LenguajeStrings[Ini, 2], LenguajeCantidad[Ini, 3]}
```

Out[] =

```
{{{a, a}, {a, b}}, 4}
```

De las 2^n hileras de longitud n se aceptan 2^{n-1} : las que fijan la primera letra en a y dejan libres las demás.

7.6.34 (★★) **VilCretas** Cuente los elementos del lenguaje anterior de longitud menor o igual que dos.

SOLUCIÓN

In[] :=

```
Ini = Automata[{u0, u1, u2}, {a, b}, u0,
{{u0, a, u1}, {u0, b, u2}, {u1, a, u1}, {u1, b, u1},
{u2, a, u2}, {u2, b, u2}}, {u1}];
LenguajeCantidad[Ini, 2, limite -> True]
```

Out[] =

```
3
```

Son a , aa y ab . La hilera vacía no se acepta, porque el estado inicial u_0 no está entre los aceptados.

7.6.35 (★★) **MIXTO** Cambie el conjunto de estados aceptados del autómata de la paridad y describa el lenguaje resultante.

SOLUCIÓN

In[] :=

```
Imp = Automata[{q0, q1}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q0}, {q1, b, q1}}, {q1}];
{LenguajeStrings[Imp, 2], LenguajeCantidad[Imp, 3]}
```

Out[] =

```
{{{a, b}, {b, a}}, 4}
```

Al aceptar en q_1 en lugar de q_0 se obtiene el **complemento**: las hileras con una cantidad impar de a . Intercambiar aceptados y no aceptados complementa siempre el lenguaje de un autómata determinístico.

7.6.36 (★★) **VilCretas** Construya el autómata que acepta exactamente las hileras con tres a .

SOLUCIÓN

```
In[ ] :=
```

```
AutomataExactamente[{{a, 3}}];
LenguajeStrings[G, 3]
```

```
Out[ ] =
```

```
{{a, a, a}}
```

Con un solo carácter el lenguaje tiene un único elemento. El autómata queda en G , de modo que hay que consultarlo desde esa variable y no desde el resultado de la llamada, que es el diagrama.

7.6.37 (★★) **A MANO** ¿Por qué no debe llamarse D ni N a un autómata en *Mathematica*?

SOLUCIÓN Porque son **símbolos protegidos** del sistema: D es la derivada y N la evaluación numérica. La asignación falla y todas las instrucciones que dependan de esa variable se quedan sin evaluar. Nombres como Dfa o $Ndfa$ evitan el conflicto.

7.3 Autómatas no determinísticos

7.6.38 (★) **A MANO** ¿Qué cambia en la definición de un autómata no determinístico?

SOLUCIÓN La función de transición toma como imágenes **subconjuntos** del conjunto de estados: para un par (estado, entrada) puede haber varios estados siguientes, uno solo, o ninguno. La trayectoria de una hilera deja de ser única.

7.6.39 (★★) **A MANO** ¿Cómo se escribe la función de transición de un *ADFN* en el paquete?

SOLUCIÓN Igual que en el caso determinístico, pero con la tercera componente de cada fila entre llaves: $\{q_0, a, \{q_0, q_1\}\}$ significa que desde q_0 con entrada a se puede pasar a q_0 o a q_1 , y $\{q_1, a, \{\}\}$ que no hay transición. Hay que escribir **todas** las filas, también las de imagen vacía.

7.6.40 (★★) **VilCretas** Construya un *ADFN* que acepte las hileras sobre $\{a, b\}$ que terminan en ab y compruébelo.

SOLUCIÓN

```
In[ ] :=
```

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
{q2, a, {}}, {q2, b, {}}}, {q2}];
{StringAceptadaQ[Ndfa, {a, b}], StringAceptadaQ[Ndfa, {a, a, b}],
StringAceptadaQ[Ndfa, {b, a}]}
```

```
Out[ ] =
```



```
{True, True, False}
```

El estado q_0 se queda leyendo cualquier cosa y *adivina* cuándo empieza el sufijo *ab*. Con tres estados basta, frente a los tres del autómata determinístico equivalente, pero la descripción es mucho más natural.

7.6.41 (★★) VilCretas Obtenga la trayectoria seguida al procesar *aab* en ese *ADFN*.

SOLUCIÓN

```
In[ ] :=
```

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
  {q2, a, {}}, {q2, b, {}}}, {q2}];
StringAceptadaQ[Ndfa, {a, a, b}, trace -> True]
```

```
Out[ ] =
```

```
{True, {{q0}, {q0, q1}, {q0, q1}, {q0, q2}}}
```

La traza son **conjuntos** de estados, porque el recorrido se sigue en el autómata determinístico equivalente. Al final el conjunto contiene q_2 , que es aceptado, y por eso la hilera se acepta.

7.6.42 (★★) VilCretas Enumere y cuente las hileras de longitud tres que acepta el *ADFN*.

SOLUCIÓN

```
In[ ] :=
```

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
  {q2, a, {}}, {q2, b, {}}}, {q2}];
{LenguajeStrings[Ndfa, 3], LenguajeCantidad[Ndfa, 3]}
```

```
Out[ ] =
```

```
{{{a, a, b}, {b, a, b}}, 2}
```

Coincide con lo obtenido para el autómata determinístico del ejercicio anterior: ambos reconocen el mismo lenguaje.

7.6.43 (★★★) VilCretas Construya el autómata determinístico equivalente al *ADFN* anterior.

SOLUCIÓN

```
In[ ] :=
```

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
  {q2, a, {}}, {q2, b, {}}}, {q2}];
AutomataDeterministicoEquivalente[Ndfa, onlyreduce -> True]
```

El procedimiento toma como estados nuevos los **subconjuntos** de estados alcanzables. `onlyreduce->True` devuelve solo el autómata reducido, sin los subconjuntos inalcanzables, y el resultado queda almacenado en `G`.

7.6.44 (★★★) A MANO Explique por qué todo *ADFN* tiene un autómata determinístico equivalente.

SOLUCIÓN Porque se puede tomar como conjunto de estados del nuevo autómata el conjunto potencia del original: cada estado nuevo representa el conjunto de estados en que el *ADFN* podría encontrarse. La transición es entonces única y los estados aceptados son los subconjuntos que contienen algún estado aceptado del *ADFN*.

7.6.45 (★★★) **A MANO** Si un *ADFN* tiene 4 estados, ¿cuántos estados puede llegar a tener el determinístico equivalente?

SOLUCIÓN Hasta $2^4 = 16$, uno por cada subconjunto de estados, incluido el vacío. En la práctica muchos son inalcanzables y por eso el paquete ofrece la reducción: el crecimiento exponencial es el peor caso, no el habitual.

7.6.46 (★★) **A MANO** ¿Qué significa que la imagen de un par (estado, entrada) sea el conjunto vacío?

SOLUCIÓN Que en el diagrama de transición no sale ninguna arista de ese estado con esa entrada: la trayectoria **muere** ahí. La hilera se rechaza por ese camino, aunque podría aceptarse por otro.

7.6.47 (★★) **VilCretas** Dibuje el diagrama de transición del *ADFN*.

SOLUCIÓN

In[] :=

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
  {q2, a, {}}, {q2, b, {}}}, {q2}];
AutomataToDiagrama[Ndfa, colores -> True]
```

Se aprecia el no determinismo en el estado q_0 : con la entrada a salen dos aristas, una hacia sí mismo y otra hacia q_1 . **AutomataToDiagrama** acepta indistintamente autómatas *DFA* y *ADFN*.

7.6.48 (★★★) **A MANO** Un *ADFN* acepta una hilera si *alguna* o si *todas* sus trayectorias terminan en un estado aceptado?

SOLUCIÓN Basta con que **alguna** lo haga. Ese es el sentido del no determinismo: la máquina acepta si existe al menos un recorrido exitoso, aunque otros mueran o terminen en estados no aceptados.

7.6.49 (★★★) **MIXTO** Compruebe que el *ADFN* y el *DFA* que terminan en ab reconocen el mismo lenguaje hasta longitud tres.

SOLUCIÓN

In[] :=

```
Ndfa = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}}, {q1, b, {q2}},
  {q2, a, {}}, {q2, b, {}}}, {q2}];
Dfa = Automata[{r0, r1, r2}, {a, b}, r0,
  {{r0, a, r1}, {r0, b, r0}, {r1, a, r1}, {r1, b, r2},
  {r2, a, r1}, {r2, b, r0}}, {r2}];
LenguajeStrings[Ndfa, 3] == LenguajeStrings[Dfa, 3]
```

Out[] =

True

Las listas coinciden. Es la comprobación experimental de la equivalencia entre los dos modelos.

7.6.50 (★★) **A MANO** En el *ADFN* que termina en *ab*, ¿por qué la fila $\{q_1, a, \{\}\}$ es necesaria?

SOLUCIÓN Porque la función de transición debe estar definida para **todos** los pares (estado, entrada), aunque la imagen sea el conjunto vacío. Omitir la fila deja el autómata incompleto y el comando *Automata* no lo procesa.

7.6.51 (★★★) **A MANO** Convierta a mano, por el método de los subconjuntos, el *ADFN* que termina en *ab*.

SOLUCIÓN Se parte de $\{q_0\}$. Con *a* se llega a $\{q_0, q_1\}$ y con *b* a $\{q_0\}$. Desde $\{q_0, q_1\}$: con *a* otra vez $\{q_0, q_1\}$ y con *b*, $\{q_0, q_2\}$. Desde $\{q_0, q_2\}$: con *a*, $\{q_0, q_1\}$ y con *b*, $\{q_0\}$. Solo aparecen tres subconjuntos alcanzables, y el aceptado es $\{q_0, q_2\}$ por contener a *q*₂.

7.6.52 (★★★) **MIXTO** Sea *M* la máquina de tres estados con la tabla que se muestra en el código. Procese 01001 y 11000.

SOLUCIÓN

In[] :=

```
FM = {{s0, 0, s1, 1}, {s0, 1, s0, 0}, {s1, 0, s2, 0},
      {s1, 1, s0, 1}, {s2, 0, s0, 1}, {s2, 1, s2, 0}};
StringSalida[{s0, s1, s2}, {0, 1}, {0, 1}, s0, FM, {0, 1, 0, 0, 1}],
StringSalida[{s0, s1, s2}, {0, 1}, {0, 1}, s0, FM, {1, 1, 0, 0, 0}]
```

Out[] =

```
{{1, 1, 1, 0, 0}, {0, 0, 1, 0, 1}}
```

Las dos hileras tienen la misma longitud que la entrada. Esta máquina se usa en los ejercicios siguientes.

7.6.53 (★★) **MIXTO** Procese 000 en la misma máquina y siga el recorrido a mano.

SOLUCIÓN

In[] :=

```
FM = {{s0, 0, s1, 1}, {s0, 1, s0, 0}, {s1, 0, s2, 0},
      {s1, 1, s0, 1}, {s2, 0, s0, 1}, {s2, 1, s2, 0}};
StringSalida[{s0, s1, s2}, {0, 1}, {0, 1}, s0, FM, {0, 0, 0}]
```

Out[] =

```
{1, 0, 1}
```

El recorrido es $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_0$: con entradas 0 la máquina **cicla** entre los tres estados, y la salida repite el patrón 1, 0, 1.

7.6.54 (★★★) **A MANO** ¿Cuál es el período de la salida de esa máquina ante una entrada formada solo por ceros?

SOLUCIÓN El período es **tres**: los estados recorren el ciclo $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_0$ y las salidas asociadas son 1, 0, 1, que se repiten. Una máquina de estado finito ante una entrada constante siempre acaba en un ciclo, y su período divide al número de estados.

7.6.55 (★★★) **A MANO** Justifique por qué toda máquina de estado finito con entrada constante termina ciclando.

SOLUCIÓN Hay un número **finito** de estados, de modo que al procesar la entrada constante alguna vez se repite un estado. Desde ese momento el comportamiento se repite idénticamente, porque el estado siguiente depende solo del estado actual y del símbolo, que no cambia. Es el principio del palomar aplicado a los estados.

7.6.56 (★★) **A MANO** ¿Cuántas máquinas de estado finito distintas hay con 3 estados, 2 entradas y 2 salidas?

SOLUCIÓN Cada una de las $3 \cdot 2 = 6$ filas de la tabla admite 3 estados siguientes y 2 salidas, es decir 6 combinaciones. En total

$$6^6 = 46\,656$$

Si además se pudiera elegir el estado inicial, habría que multiplicar por tres.

7.6.57 (★★★) **A MANO** Diseñe una máquina que emita 1 cuando la entrada leída hasta el momento tenga un número de unos múltiplo de 3.

SOLUCIÓN Tres estados, uno por resto:

	0	1	0	1
s_0	s_0	s_1	1	0
s_1	s_1	s_2	0	0
s_2	s_2	s_0	0	1

La salida vale 1 exactamente cuando el estado *destino* es s_0 . Los ceros no cambian de estado y repiten la salida anterior.

7.6.58 (★★★) **MIXTO** Compruebe la máquina del ejercicio anterior sobre 111011.

SOLUCIÓN

In[] :=

```
F3 = {{s0, 0, s0, 1}, {s0, 1, s1, 0}, {s1, 0, s1, 0},
      {s1, 1, s2, 0}, {s2, 0, s2, 0}, {s2, 1, s0, 1}};
StringSalida[{s0, s1, s2}, {0, 1}, {0, 1}, s0, F3, {1, 1, 1, 0, 1, 1}]
```

Out[] =

```
{0, 0, 1, 1, 0, 0}
```

El 1 aparece en la tercera posición, justo cuando se llevan tres unos, y se mantiene en la cuarta porque el 0 no cambia de estado. Con seis unos volvería a aparecer.

7.6.59 (★★) **A MANO** ¿Cuántos estados necesita una máquina que detecte si la entrada termina en 101?

SOLUCIÓN Cuatro: uno por cada prefijo reconocido (nada, 1, 10, 101). En general, detectar un patrón de longitud k necesita $k + 1$ estados, uno por cada cantidad de símbolos ya emparejados.

7.6.60 (★★★) **A MANO** ¿Por qué la máquina anterior no puede tener menos de cuatro estados?

SOLUCIÓN Porque debe distinguir cuatro situaciones que llevan a futuros distintos: si ya vio 10 y llega un 1, acepta; si solo vio 1, no. Dos hileras que exijan respuestas distintas ante

la misma continuación **no pueden compartir estado**. Es el argumento que formaliza el teorema de *Myhill-Nerode*.

7.6.61 (★★) **A MANO** ¿En qué se diferencia el conjunto de estados aceptados de la función de salida?

SOLUCIÓN La **función de salida** emite un símbolo en cada transición, de modo que produce una hilera tan larga como la entrada. El **conjunto de estados aceptados** solo interviene al final: la respuesta es un sí o un no. Un autómata puede verse como una máquina que emite un solo bit al terminar.

7.6.62 (★★★) **MIXTO** Construya el autómata que acepta las hileras sobre $\{a, b\}$ con al menos dos a consecutivas y compruébelo.

SOLUCIÓN

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
{q2, a, q2}, {q2, b, q2}}, {q2}];
[StringAceptadaQ[D3, {a, a, a}], StringAceptadaQ[D3, {a, b, a}]]
```

Out[] =

```
{True, False}
```

El estado q_1 significa *acabo de ver una a* y q_2 es la trampa de aceptación. La segunda hilera tiene dos a pero no consecutivas.

7.6.63 (★★) **MIXTO** Enumere y cuente el lenguaje de ese autómata en longitudes 3 y 4.

SOLUCIÓN

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
{q2, a, q2}, {q2, b, q2}}, {q2}];
{LenguajeStrings[D3, 3], LenguajeCantidad[D3, 4],
LenguajeCantidad[D3, 4, limite -> True]}
```

Out[] =

```
{{{a, a, a}, {a, a, b}, {b, a, a}}, 8, 12}
```

Con longitud tres hay tres (*aaa*, *aab* y *baa*) y con longitud cuatro, ocho. Acumulando hasta longitud cuatro son doce, incluida *aa* de longitud dos.

7.6.64 (★★★) **A MANO** ¿Cuántas hileras de longitud n sobre $\{a, b\}$ **no** contienen dos a consecutivas?

SOLUCIÓN Es la recurrencia de *Fibonacci*: $c_n = c_{n-1} + c_{n-2}$ con $c_1 = 2$ y $c_2 = 3$. Una hilera válida termina en b (con cualquier prefijo válido de longitud $n-1$) o en ba (con prefijo válido de longitud $n-2$). Para $n = 4$ son 8, y el autómata acepta las $16 - 8 = 8$ restantes... pero el conteo del software da 4, porque exige **al menos dos consecutivas**, que no es lo mismo que *no ser libre de aa* sobre las de longitud exacta.

7.6.65 (★★★) **MIXTO** Construya el autómata que acepta las hileras binarias cuyo valor es múltiplo de 3, léidas de izquierda a derecha.

SOLUCIÓN**In[] :=**

```
D4 = Automata[{q0, q1, q2}, {0, 1}, q0,
{{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
{q2, 0, q1}, {q2, 1, q2}}, {q0}];
{LenguajeStrings[D4, 3], LenguajeCantidad[D4, 5]}
```

Out[] =

```
{{{0, 0, 0}, {0, 1, 1}, {1, 1, 0}}, 11}
```

Los estados son los restos módulo 3. En longitud tres las hileras aceptadas son 000, 011 y 110, que valen 0, 3 y 6.

7.6.66 (★★) **MIXTO** Siga el recorrido de la hilera 1100 en ese autómata.

SOLUCIÓN**In[] :=**

```
D4 = Automata[{q0, q1, q2}, {0, 1}, q0,
{{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
{q2, 0, q1}, {q2, 1, q2}}, {q0}];
StringAceptadaQ[D4, {1, 1, 0, 0}, trace -> True]
```

Out[] =

```
{True, {q0, q1, q0, q0, q0}}
```

El valor $1100_2 = 12$ es múltiplo de 3. El recorrido pasa por el resto 1 tras el primer bit y vuelve a 0 tras el segundo, donde ya se queda.

7.6.67 (★★★) **A MANO** Justifique la regla de transición del autómata de múltiplos de tres.

SOLUCIÓN Al leer un bit x , el valor pasa de v a $2v + x$, de modo que el resto pasa de r a $(2r + x) \bmod 3$. Con $r = 0$: el 0 deja 0 y el 1 lleva a 1. Con $r = 1$: el 0 lleva a 2 y el 1 a 0. Con $r = 2$: el 0 lleva a 1 y el 1 a 2. Es exactamente la tabla usada.

7.6.68 (★★) **A MANO** Generalice: ¿cuántos estados necesita el autómata de múltiplos de k en base 2?

SOLUCIÓN Necesita k estados, uno por resto. La construcción es la misma: $r \mapsto (2r + x) \bmod k$. Para $k = 7$ harían falta siete estados, y el autómata sigue siendo finito por grande que sea k : lo que crece es el número de estados, no la complejidad conceptual.

7.6.69 (★★★) **MIXTO** Construya el *ADFN* que acepta las hileras que contienen la subhilera *aab* y compruébelo.

SOLUCIÓN**In[] :=**

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}];
{StringAceptadaQ[N2, {a, a, b}], LenguajeStrings[N2, 4], LenguajeCantidad[N2, 4]}
```

Out[] =

```
{True, {{a, a, a, b}, {b, a, a, b}}, 2}
```

El estado r_0 lee cualquier prefijo y *adivina* dónde empieza el patrón. En longitud cuatro solo hay dos hileras que lo contengan.

7.6.70 (★★★) **A MANO** ¿Cuántos estados tendría el determinístico equivalente al *ADFN* anterior?

SOLUCIÓN Como máximo $2^4 = 16$, pero los alcanzables son solo cuatro: $\{r_0\}$, $\{r_0, r_1\}$, $\{r_0, r_2\}$ y $\{r_0, r_3\}$. El determinístico mínimo tiene también cuatro estados, uno por cada prefijo del patrón reconocido: el no determinismo no ahorra estados aquí, solo simplifica la descripción.

7.6.71 (★★) **VilCretas** Construya el determinístico equivalente a ese *ADFN*.

SOLUCIÓN

```
In[ ] :=
```

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}];
AutomataDeterministicoEquivalente[N2, onlyreduce -> True]
```

El método de los subconjuntos genera los estados alcanzables y descarta el resto. El resultado queda en **G** y reconoce el mismo lenguaje.

7.6.72 (★★★) **A MANO** ¿Existe algún lenguaje para el cual el determinístico equivalente **sí** necesite 2^n estados?

SOLUCIÓN Sí: el de las hileras cuyo n -ésimo símbolo desde el final es una a . El *ADFN* lo reconoce con $n + 1$ estados adivinando la posición, pero el determinístico debe recordar los últimos n símbolos, lo que exige 2^n estados. Es el ejemplo canónico de que la cota exponencial es **ajustada**.

7.6.73 (★★) **MIXTO** Construya el autómata que acepta exactamente las hileras con tres a y dos b .

SOLUCIÓN

```
In[ ] :=
```

```
AutomataExactamente[{{a, 3}, {b, 2}}];
LenguajeCantidad[G, 5]
```

```
Out[ ] =
```

```
10
```

Son $\binom{5}{3} = 10$ hileras, una por cada elección de las posiciones de las a . El autómata queda almacenado en **G**.

7.6.74 (★★★) **A MANO** ¿Cuántos estados necesita el autómata anterior y por qué?

SOLUCIÓN Necesita llevar la cuenta de cuántas a y cuántas b se han leído, es decir un estado por cada par (i, j) con $0 \leq i \leq 3$ y $0 \leq j \leq 2$: son 12, más un estado de rechazo. Un solo contador no basta, porque hay que controlar las dos cantidades por separado.

7.6.75 (★★★) **A MANO** Diseñe un autómata que acepte las hileras sobre $\{a, b\}$ de longitud par que empiecen y terminen con el mismo símbolo.

SOLUCIÓN Hace falta recordar el primer símbolo, el último leído y la paridad de la longitud: $2 \times 2 \times 2$ combinaciones más el estado inicial. Con nueve estados basta, y se acepta cuando el primero coincide con el último y la longitud es par. Es un buen ejemplo de que la información que hay que recordar (y no la longitud de la hilera) determina el número de estados.

7.6.76 (★★) **A MANO** ¿Puede un autómata de estado finito aceptar el lenguaje de los palíndromos sobre $\{a, b\}$?

SOLUCIÓN No. Habría que recordar toda la primera mitad de la hilera para compararla con la segunda, y eso exige una cantidad **no acotada** de memoria. Con n estados, dos hileras distintas de longitud mayor que $\log_2 n$ acaban en el mismo estado y el autómata ya no puede distinguirlas.

7.6.77 (★★★) **A MANO** Formalice el argumento anterior para $\{a^n b^n\}$.

SOLUCIÓN Supongamos un autómata con k estados. Entre las hileras $a, aa, \dots, a^{k+1}$ hay dos, digamos a^i y a^j con $i \neq j$, que llevan al mismo estado. Entonces $a^i b^i$ y $a^j b^i$ terminan también en el mismo estado, y el autómata acepta ambas o rechaza ambas. Pero la primera pertenece al lenguaje y la segunda no: **contradicción**.

7.6.78 (★★) **A MANO** ¿Cuántas hileras de longitud n acepta un autómata que reconoce $\{a, b\}^*$ completo?

SOLUCIÓN Las 2^n : acepta todas. Ese autómata tiene un solo estado, inicial y aceptado, con lazos para las dos entradas. Es el caso extremo opuesto al autómata que no acepta nada, que también tiene un solo estado pero no aceptado.

7.6.79 (★★★) **A MANO** Describa el autómata que acepta el lenguaje vacío y el que acepta solo la hilera vacía.

SOLUCIÓN El del **lenguaje vacío** tiene un estado, sin aceptados: nada llega a un estado final. El de **solo la hilera vacía** tiene dos: el inicial, que es aceptado, y una trampa a la que se va con cualquier símbolo. La diferencia está en si el estado inicial se declara aceptado.

7.6.80 (★★★) **MIXTO** Compruebe la diferencia anterior.

SOLUCIÓN

In[] :=

```
Vac = Automata[{u0}, {a}, u0, {{u0, a, u0}}, {}];
Eps = Automata[{u0, u1}, {a}, u0, {{u0, a, u1}, {u1, a, u1}}, {u0}];
{LenguajeCantidad[Vac, 2, limite -> True],
LenguajeCantidad[Eps, 2, limite -> True]}
```

Out[] =

{0, 1}

El primero no acepta nada y el segundo acepta exactamente un elemento: la hilera vacía.

7.6.81 (★★) **A MANO** ¿Qué ocurre si en un autómata determinístico se declara aceptado el conjunto vacío de estados?

SOLUCIÓN El lenguaje resultante es **vacío**: ninguna hilera puede terminar en un estado aceptado, porque no hay ninguno. El autómata sigue siendo válido y procesa hileras normalmente; simplemente rechaza todas.

- 7.6.82** (★★★) **A MANO** Si se intercambian los estados aceptados y los no aceptados de un autómata determinístico, ¿qué lenguaje se obtiene?

SOLUCIÓN El **complemento**. Como el autómata es determinístico y completo, toda hilera tiene un recorrido único que termina en exactamente un estado, de modo que invertir la aceptación invierte la respuesta. Con un *ADFN* esto **no funciona**: una hilera puede tener varios recorridos.

- 7.6.83** (★★★) **MIXTO** Compruebe el complemento sobre el autómata de los múltiplos de tres.

SOLUCIÓN

In[] :=

```
D4 = Automata[{q0, q1, q2}, {0, 1}, q0,
  {{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
  {q2, 0, q1}, {q2, 1, q2}}, {q0}];
Dc = Automata[{q0, q1, q2}, {0, 1}, q0,
  {{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
  {q2, 0, q1}, {q2, 1, q2}}, {q1, q2}];
{LenguajeCantidad[D4, 3], LenguajeCantidad[Dc, 3]}
```

Out[] =

```
{3, 5}
```

Tres más cinco son las 8 hileras de longitud tres: los dos lenguajes son complementarios.

- 7.6.84** (★★) **A MANO** ¿Por qué el complemento no funciona intercambiando aceptados en un *ADFN*?

SOLUCIÓN Porque una hilera puede tener **varios** recorridos: unos que terminan en estado aceptado y otros que no. Al intercambiar, esa hilera seguiría aceptándose por el segundo grupo de recorridos. Para complementar hay que determinar primero y después intercambiar.

- 7.6.85** (★★★) **A MANO** Un autómata tiene un estado inalcanzable. ¿Cambia su lenguaje si se elimina?

SOLUCIÓN No. Un estado inalcanzable no participa en ningún recorrido desde el inicial, de modo que eliminarlo (junto con sus transiciones) no altera qué hileras se aceptan. Es lo que hace la opción `reducido->True` de `AutomataToDiagrama` y `onlyreduce->True` de la determinización.

- 7.6.86** (★★★) **A MANO** Dos estados son *equivalentes* si desde ellos se aceptan las mismas hileras. ¿Qué permite hacer eso?

SOLUCIÓN Permite **minimizar** el autómata: los estados equivalentes se fusionan en uno solo sin cambiar el lenguaje. El resultado es único salvo el nombre de los estados, y es el autómata determinístico con el menor número posible de estados para ese lenguaje.

- 7.6.87** (★★) **A MANO** ¿Cuántas transiciones tiene la tabla de un *ADFN* con 4 estados y 3 símbolos?

SOLUCIÓN Tiene $4 \cdot 3 = 12$ filas, igual que un determinístico. La diferencia está en el **contenido**: cada imagen es un subconjunto de estados, posiblemente vacío o con varios elementos. Hay que escribir las doce filas aunque muchas tengan imagen vacía.

- 7.6.88** (★★★) **A MANO** ¿Qué pasa si en un *ADFN* se omite una fila con imagen vacía?

SOLUCIÓN El comando `Automata` deja de procesarlo y devuelve la expresión **sin evaluar**, sin ningún mensaje. Es una de las causas más frecuentes de que un ejercicio de este capítulo no dé resultado, y por eso hay que contar las filas antes de ejecutar.

- 7.6.89** (★★) **A MANO** Resuma qué componente distingue una máquina de estado finito, un autómata determinístico y uno no determinístico.

SOLUCIÓN La **máquina** tiene seis componentes, con símbolos y función de salida. El **autómata determinístico** sustituye esas dos por un conjunto de estados aceptados, y su función de transición da un estado. El **no determinístico** tiene las mismas cinco componentes, pero su función de transición da un **subconjunto** de estados.

- 7.6.90** (★★★) **A MANO** Un autómata acepta una hilera de longitud 20 y tiene 5 estados. ¿Qué se puede afirmar sobre su recorrido?

SOLUCIÓN Que algún estado se repite: el recorrido tiene 21 estados y solo hay cinco distintos. Ese tramo entre dos repeticiones es un **ciclo** que se puede repetir o suprimir sin salir del lenguaje, lo que produce infinitas hileras aceptadas. Es la idea del *lema de bombeo*.

- 7.6.91** (★★★) **A MANO** Use la idea anterior para probar que todo lenguaje regular infinito acepta hileras arbitrariamente largas con estructura repetitiva.

SOLUCIÓN Si el lenguaje es infinito, acepta alguna hilera más larga que el número de estados. Su recorrido repite un estado, de modo que la hilera se descompone como xyz con y no vacía, y **todas** las hileras xy^kz son aceptadas. Un lenguaje regular infinito nunca es *esporádico*: siempre contiene una familia con esa forma.

- 7.6.92** (★★) **A MANO** Un autómata con 5 estados acepta alguna hilera. ¿Cuál es la más corta posible que se puede garantizar?

SOLUCIÓN Si acepta algo, acepta alguna hilera de longitud menor que 5: el camino más corto del inicial a un aceptado no repite estados, de modo que usa a lo sumo 4 transiciones. Buscar entre las hileras cortas basta para decidir si el lenguaje es vacío.

- 7.6.93** (★★★) **MIXTO** Compruebe la afirmación anterior sobre el autómata de aab .

SOLUCIÓN

`In[ ] :=`

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}];
{LenguajeCantidad[N2, 3, limite -> True], LenguajeStrings[N2, 3]}
```

`Out[ ] =`

```
{1, {{a, a, b}}}
```

Con cuatro estados, la hilera aceptada más corta tiene longitud 3, menor que 4 como predice el argumento. Buscar hasta longitud tres bastaba para saber que el lenguaje no es vacío.

- 7.6.94** (★★) **A MANO** Enumere las seis componentes de una máquina de estado finito y las cinco de un autómata, señalando cuáles comparten.

SOLUCIÓN La máquina: σ (estados), τ (entradas), δ (salidas), σ^* (inicial), Δ (transición) y Ω (salida). El autómata: σ , τ , σ^* , Δ y $\hat{A}$ (aceptados). Comparten **cuatro**: estados, entradas, inicial y transición. El autómata cambia δ y Ω por $\hat{A}$.

7.6.95 (★★★) **A MANO** Una máquina tiene n estados y m entradas. ¿Cuántas funciones de transición distintas admite?

SOLUCIÓN Cada una de las nm filas puede apuntar a cualquiera de los n estados, de modo que hay n^{nm} funciones de transición. Para $n = 3$ y $m = 2$ son $3^6 = 729$. Multiplicando por las funciones de salida se obtiene el total de máquinas distintas.

7.6.96 (★★) **A MANO** ¿Cuántos autómatas determinísticos distintos hay con 2 estados y 2 entradas?

SOLUCIÓN La transición admite $2^4 = 16$ formas, el estado inicial 2 elecciones y el conjunto de aceptados $2^2 = 4$. En total $16 \cdot 2 \cdot 4 = 128$. Muchos de ellos reconocen el **mismo** lenguaje: el número de lenguajes distintos es mucho menor.

7.6.97 (★★★) **A MANO** Diseñe un autómata que acepte las hileras sobre $\{a, b\}$ que empiezan y terminan con la misma letra.

SOLUCIÓN Hace falta recordar la primera letra y la última leída: cuatro estados útiles más el inicial. Se acepta en los dos estados donde primera y última coinciden. La hilera de una sola letra también se acepta, porque en ella la primera y la última son la misma.

7.6.98 (★★★) **MIXTO** Compruebe el diseño anterior sobre cuatro casos.

SOLUCIÓN

In[] :=

```
Ai = Automata[{t0, ta, tb, ua, ub}, {a, b}, t0,
{{t0, a, ta}, {t0, b, tb}, {ta, a, ta}, {ta, b, ua},
{tb, a, ub}, {tb, b, tb}, {ua, a, ta}, {ua, b, ua},
{ub, a, ub}, {ub, b, tb}}, {ta, tb}];
[StringAceptadaQ[Ai, {a, b, a}], StringAceptadaQ[Ai, {a, b, b}],
StringAceptadaQ[Ai, {b, a, b}], StringAceptadaQ[Ai, {a}]]
```

Out[] =

```
{True, False, True, True}
```

Se aceptan *aba*, *bab* y *a*; se rechaza *abb*. Los estados t_a y t_b significan *empezó y termina igual*.

7.6.99 (★★) **A MANO** ¿Cuántas hileras de longitud $n \geq 2$ empiezan y terminan con la misma letra sobre un alfabeto de dos símbolos?

SOLUCIÓN Se elige la primera letra (2 formas), quedan libres las $n - 2$ intermedias (2^{n-2}) y la última queda determinada. En total 2^{n-1} : exactamente la mitad. Para $n = 4$ son 8 de las 16 posibles.

7.6.100 (★★★) **A MANO** Diseñe una máquina que emita la *suma acumulada módulo 2* de los bits leídos.

SOLUCIÓN Dos estados, s_0 y s_1 , según la paridad acumulada:

	0	1	0	1
s_0	s_0	s_1	0	1
s_1	s_1	s_0	1	0

La salida coincide siempre con el estado destino. Es la máquina de paridad, y su salida es la suma parcial módulo dos.

7.6.101 (★★★) A MANO Modifique la máquina anterior para que la salida sea la suma acumulada módulo 4. ¿Cuántos estados y cuántas salidas necesita?

SOLUCIÓN Necesita **cuatro** estados (los restos 0, 1, 2, 3) y **cuatro** símbolos de salida. Con entrada binaria, cada 1 avanza un resto y cada 0 lo deja igual. En general, la suma módulo k necesita k estados y k salidas: el tamaño crece linealmente con el módulo.

7.6.102 (★★) A MANO ¿Puede una máquina de estado finito calcular la suma acumulada **sin** módulo?

SOLUCIÓN No. La suma crece sin límite y habría que recordar un valor arbitrariamente grande, lo que exige infinitos estados. Una máquina de estado finito solo puede llevar cuentas **acotadas**: por eso los módulos sí se pueden y las sumas libres no.

7.6.103 (★★★) A MANO Un autómata tiene $\hat{A} = \sigma$, es decir, todos sus estados son aceptados. ¿Qué lenguaje reconoce?

SOLUCIÓN Reconoce τ^* completo: toda hilera termina en algún estado, y todos son aceptados. Es el autómata que acepta todo, y equivale al de un solo estado con lazos. La estructura de las transiciones se vuelve irrelevante.

7.6.104 (★★) A MANO ¿Qué significa que un estado sea una *trampa*?

SOLUCIÓN Que todas sus transiciones apuntan a sí mismo: una vez alcanzado, no se sale. Si es aceptado, la hilera se acepta pase lo que pase después (*trampa de aceptación*); si no lo es, se rechaza siempre (*trampa de rechazo*). Es la forma habitual de modelar condiciones irreversibles.

7.6.105 (★★★) A MANO Diseñe un autómata que acepte las hileras sobre $\{a, b, c\}$ que contienen al menos una de cada letra. ¿Cuántos estados necesita?

SOLUCIÓN Necesita recordar qué subconjunto de letras ya apareció: $2^3 = 8$ estados, uno por subconjunto, con $\{a, b, c\}$ como único aceptado y trampa. El crecimiento es exponencial en el número de letras, y es inevitable: hay que distinguir los 2^3 historiales posibles.

7.6.106 (★★★) A MANO Justifique por qué el autómata anterior no puede tener menos de ocho estados.

SOLUCIÓN Dos subconjuntos distintos exigen respuestas distintas ante alguna continuación: si ya vio $\{a, b\}$, una c lo lleva a aceptar; si solo vio $\{a\}$, no. Como los ocho subconjuntos son distinguibles dos a dos, ningún autómata puede fusionar dos de ellos en un estado. Es el argumento de *Myhill-Nerode*.

7.6.107 (★★) A MANO ¿Cuántas hileras de longitud 4 sobre $\{a, b, c\}$ contienen las tres letras?

SOLUCIÓN Es imposible con longitud 3 tener repeticiones, así que con longitud 4 una letra se repite. Se elige cuál (3 formas) y se cuentan las permutaciones de un multiconjunto con una repetición: $\frac{4!}{2!} = 12$. En total $3 \cdot 12 = 36$ de las 81 posibles.

7.6.108 (★★★) A MANO Un *ADFN* tiene una transición con imagen de tres estados. ¿Cuántos recorridos posibles tiene una hilera que la use una sola vez?

SOLUCIÓN Al menos tres, uno por cada estado de la imagen, y más si hay otras transiciones múltiples más adelante. El número de recorridos puede crecer exponencialmente con la longitud de la hilera, y por eso la simulación directa de un *ADFN* es cara: la determinización la hace lineal.

7.6.109 (★★) A MANO ¿Por qué un *ADFN* y su determinístico equivalente reconocen el mismo len-

guaje?

SOLUCIÓN Porque el estado del determinístico es exactamente el **conjunto** de estados en que el *ADFN* podría estar. Una hilera se acepta en el *ADFN* si algún recorrido termina en estado aceptado, es decir, si ese conjunto final contiene alguno; y esos son justamente los estados declarados aceptados en el determinístico.

7.6.110 (★★★) A MANO ¿Puede un *ADFN* tener menos estados que el determinístico mínimo equivalente?

SOLUCIÓN Sí, y es su principal ventaja. El ejemplo clásico es el lenguaje de las hileras cuya n -ésima letra desde el final es a : el *ADFN* usa $n + 1$ estados y el determinístico mínimo necesita 2^n . El no determinismo no añade poder expresivo, pero sí **concisión**.

7.6.111 (★★) A MANO ¿Qué significa que dos autómatas sean *equivalentes*?

SOLUCIÓN Que reconocen el **mismo lenguaje**, aunque tengan distinto número de estados, distintas transiciones o incluso distinta naturaleza: uno determinístico y otro no. La equivalencia se refiere al conjunto de hileras aceptadas, no a la estructura.

7.6.112 (★★★) A MANO Diseñe un autómata sobre $\{0, 1\}$ que acepte las hileras con un número par de ceros y un número impar de unos.

SOLUCIÓN Se combinan dos paridades: $2 \times 2 = 4$ estados, etiquetados por el par (paridad de ceros, paridad de unos). El inicial es (par, par) y el único aceptado es (par, impar). Cada 0 cambia la primera componente y cada 1 la segunda. Es el *producto* de dos autómatas de dos estados.

7.6.113 (★★★) MIXTO Compruebe el autómata producto del ejercicio anterior.

SOLUCIÓN

In[] :=

```
Ap = Automata[{pp, pi, ip, ii}, {0, 1}, pp,
{{pp, 0, ip}, {pp, 1, pi}, {pi, 0, ii}, {pi, 1, pp},
{ip, 0, pp}, {ip, 1, ii}, {ii, 0, pi}, {ii, 1, ip}}, {pi}];
{LenguajeStrings[Ap, 3], LenguajeCantidad[Ap, 4]}
```

Out[] =

```
{{{0, 0, 1}, {0, 1, 0}, {1, 0, 0}, {1, 1, 1}}, 0}
```

En longitud tres hay cuatro hileras válidas. En longitud cuatro **ninguna**: con cuatro símbolos, si los ceros son pares los unos también lo son, y la condición es contradictoria.

7.6.114 (★★★) A MANO Justifique por qué ese lenguaje es vacío en las longitudes pares.

SOLUCIÓN Si la longitud n es par y el número de ceros es par, el de unos es $n - \text{ceros}$, diferencia de dos pares: también par. Nunca puede ser impar. El lenguaje solo contiene hileras de longitud **impar**, lo que se ve de inmediato en el conteo pero no en el diagrama.

7.6.115 (★★) A MANO Explique la construcción del *autómata producto* para intersecar dos lenguajes.

SOLUCIÓN Se toman como estados los pares (p, q) con p del primero y q del segundo, y se avanzan las dos coordenadas a la vez con cada símbolo. Se acepta cuando **ambas** componentes están en su conjunto de aceptados. Declarando aceptado cuando lo está *alguna* se obtiene la unión.

7.6.116 (★★★) A MANO ¿Cuántos estados tiene el autómata producto de uno con m estados y otro con n ?

SOLUCIÓN Tiene mn , uno por par. No siempre son todos alcanzables, y el mínimo equivalente puede tener menos. Pero la cota mn es **ajustada**: hay lenguajes para los que el producto mínimo necesita exactamente mn estados, por ejemplo los múltiplos de m y de n con m y n primos relativos.

7.6.117 (★★) A MANO Explique por qué los lenguajes regulares son cerrados bajo unión, intersección y complemento.

SOLUCIÓN Para la **intersección** y la **unión** basta el autómata producto, cambiando la condición de aceptación. Para el **complemento** se intercambian aceptados en el determinístico completo. Las tres construcciones producen autómatas finitos, de modo que los lenguajes resultantes siguen siendo regulares.

7.6.118 (★★★) A MANO ¿Es el conjunto de lenguajes regulares cerrado bajo concatenación?

SOLUCIÓN Sí. Dados autómatas para L_1 y L_2 , se construye un *ADFN* que pasa del final de L_1 al inicio de L_2 , y después se determiniza. El no determinismo es justamente la herramienta que hace fácil esta construcción: con autómatas determinísticos habría que adivinar dónde termina la primera parte.

7.6.119 (★★) A MANO ¿Cuántos símbolos tiene la hilera de salida de una máquina al procesar una entrada de longitud n ? ¿Y el recorrido de estados?

SOLUCIÓN La salida tiene n símbolos, uno por transición. El recorrido tiene $n + 1$ estados, porque incluye el inicial. Confundir ambas longitudes es un error frecuente al comparar la salida con la traza.

7.6.120 (★★★) A MANO Una máquina procesa la hilera vacía. ¿Qué devuelve y en qué estado queda?

SOLUCIÓN Devuelve la hilera de salida **vacía** y queda en el estado inicial: no hay transiciones que ejecutar. En un autómata, la hilera vacía se acepta exactamente cuando el estado inicial es aceptado, lo que da un criterio inmediato para decidirlo.

7.6.121 (★★★) A MANO Dos máquinas tienen la misma función de transición y distinta función de salida. ¿Pueden dar la misma salida ante alguna entrada?

SOLUCIÓN Sí, si las dos funciones coinciden en las transiciones que esa entrada recorre. Basta que difieran en una transición nunca usada. Por eso comparar máquinas exige recorrer **todas** las transiciones alcanzables, no una hilera de prueba.

7.6.122 (★★) A MANO Resuma qué información hay que *recordar* para reconocer cada uno de estos lenguajes: terminar en ab , tener un número par de a , y contener aab .

SOLUCIÓN Para **terminar en ab** : los dos últimos símbolos. Para el **número par de a** : solo la paridad, un bit. Para **contener aab** : cuánto del patrón se lleva emparejado. En los tres casos la memoria es **acotada**, y por eso los tres lenguajes son regulares.

7.6.123 (★★★) A MANO ¿Qué habría que recordar para reconocer $\{a^n b^n\}$, y por qué eso lo saca de lo regular?

SOLUCIÓN Habría que recordar el número exacto de a leídas, que no está acotado: para cada n hace falta distinguir a^n de a^m con $m \neq n$. Con un número finito de estados dos de esas hileras acaban en el mismo estado y el autómata deja de distinguirlas. Ese lenguaje necesita

una **pila**, no un número finito de estados.

7.4 Restadora binaria, generadores y salidas en forma de dato

7.6.124 (★★★) **MIXTO** Reste dos binarios con `RestaBinaria` y compruebe el resultado en base diez.

SOLUCIÓN

`In[ ] :=`

```
{RestaBinaria[111001111101, 11100111111],
 2^^111001111101 - 2^^11100111111}
```

`Out[ ] =`

```
{11100111110, 1854}
```

El resultado binario 11100111110 es 1854 en base diez, de modo que la resta es correcta. Lo interesante es **cómo** lo hace: no convierte a decimal, resta y vuelve: lo resuelve con una **máquina de estado finito** que procesa los bits de derecha a izquierda llevando el acarreo en el estado.

7.6.125 (★★★) **VilCretas** Vea la máquina que hay detrás, con `MaquinaRestadoraBinarios` y con la opción `steps` de `RestaBinaria`.

SOLUCIÓN

`In[ ] :=`

```
{MaquinaRestadoraBinarios[],
 RestaBinaria[111001111101, 11100111111, steps -> True]}
```

La máquina tiene dos estados: *sin deuda* y *con deuda*. Cada par de bits produce un bit de salida y decide el estado siguiente. Es el ejemplo más limpio de para qué sirve una máquina de estado finito con salida: la **memoria acotada** (un solo bit de acarreo) basta para restar números de cualquier longitud.

7.6.126 (★★★) **A MANO** ¿Por qué bastan dos estados para restar binarios de cualquier tamaño, y qué pasa si el sustraendo es mayor?

SOLUCIÓN Bastan dos porque lo único que hay que recordar de una columna a la siguiente es **si se pidió prestado o no**: un bit. Si el sustraendo es mayor, la máquina se queda con deuda al terminar y no hay resultado no negativo que devolver; `RestaBinaria` lo rechaza en lugar de inventar un signo. Por eso la documentación habla de **resta no negativa**.

7.6.127 (★★) **MIXTO** Decida con `MaquinaAutomataQ` si una máquina de estado finito es en realidad un autómata.

SOLUCIÓN

`In[ ] :=`

```
MaquinaAutomataQ[{s0, s1, s2, s3}, {a, b}, {0, 1}, s0,
 {{s0, a, s1, 0}, {s0, b, s2, 0}, {s1, a, s0, 1}, {s1, b, s2, 0},
 {s2, a, s3, 0}, {s2, b, s0, 1}, {s3, a, s1, 0}, {s3, b, s3, 0}}]
```

`Out[ ] =`

```
{True, {s0}}
```

Devuelve `True` y, además, **el conjunto de estados de aceptación**: aquí $\{s_0\}$. Una máquina con salidas en $\{0, 1\}$ es un autómata cuando la salida solo depende del estado al que se llega; los estados que emiten 1 son los de aceptación.

7.6.128 (★★★) MIXTO Compruebe que una máquina con tres símbolos de salida no puede ser un autómata.

SOLUCIÓN

`In[ ] :=`

```
MaquinaAutomataQ[{s0, s1}, {a, b}, {0, 1, 2}, s0,
{{s0, a, s1, 2}, {s0, b, s1, 0}, {s1, a, s0, 1}, {s1, b, s1, 0}}]
```

`Out[ ] =`

`False`

Un autómata solo distingue **aceptar o no aceptar**: dos respuestas posibles. Con tres símbolos de salida la máquina transmite más información que un sí o un no, y por tanto ya no es un reconocedor de lenguaje.

7.6.129 (★★) MIXTO Consiga que `StringAceptadaQ` devuelva también la hilera de salida, con salida $\rightarrow$ `True`.

SOLUCIÓN

`In[ ] :=`

```
A = Automata[{s0, s1, s2}, {a, b}, s0,
{{s0, a, s1}, {s0, b, s0}, {s1, a, s2}, {s1, b, s0},
{s2, a, s2}, {s2, b, s2}}, {s2}];
[StringAceptadaQ[A, {a, a}], StringAceptadaQ[A, {a, a}, salida -> True]]
```

`Out[ ] =`

`{True, {True, {0, 1}}}`

Con la opción se obtiene el par $\{\text{aceptación, salida}\}$. La salida $\{0, 1\}$ dice que tras la primera *a* el autómata aún no estaba en un estado de aceptación y tras la segunda sí: es el rastro del recorrido, no solo su veredicto.

7.6.130 (★★) MIXTO Extraiga las cinco componentes de un autómata con `ComponentesAutomata` y conjuntos $\rightarrow$ `True`.

SOLUCIÓN

`In[ ] :=`

```
A = Automata[{s0, s1, s2}, {a, b}, s0,
{{s0, a, s1}, {s0, b, s0}, {s1, a, s2}, {s1, b, s0},
{s2, a, s2}, {s2, b, s2}}, {s2}];
ComponentesAutomata[A, conjuntos -> True][[1, 2, 3, 5]]
```

`Out[ ] =`

`{{s0, s1, s2}, {a, b}, s0, {s2}}`

Se recuperan estados, alfabeto, estado inicial, transiciones y estados de aceptación **como datos**. Sin la opción el comando escribe las componentes en pantalla, y entonces no hay nada que reutilizar: esta es la versión que permite construir un autómata a partir de otro.

7.6.131 (★★★) MIXTO Sobre ese autómata, describa el lenguaje que acepta y compruébelo.

SOLUCIÓN

In[] :=

```
A = Automata[{s0, s1, s2}, {a, b}, s0,
{{s0, a, s1}, {s0, b, s0}, {s1, a, s2}, {s1, b, s0},
{s2, a, s2}, {s2, b, s2}}, {s2}];
{StringAceptadaQ[A, {a, a}], StringAceptadaQ[A, {a, b, a}],
StringAceptadaQ[A, {b, a, a, b}]}
```

Out[] =

```
{True, False, True}
```

Acepta las hileras que contienen **dos a consecutivas**. Una vez en s_2 ya no se sale: es un **estado sumidero de aceptación**, el patrón habitual para reconocer *contiene tal subcadena*. Por eso *aba* se rechaza y *baab* se acepta.

7.6.132 (★★★) MIXTO Verifique sobre un autómata *no* determinístico que acepta lo mismo por más de un camino.

SOLUCIÓN

In[] :=

```
nd = Automata[{s0, s1, s2}, {a, b}, s0,
{{s0, a, {s0, s1}}, {s0, b, {s0}}, {s1, a, {}},
{s1, b, {s2}}, {s2, a, {}}, {s2, b, {}}, {s2}];
{StringAceptadaQ[nd, {a, b}], StringAceptadaQ[nd, {a, a, b}],
StringAceptadaQ[nd, {b, a}]}
```

Out[] =

```
{True, True, False}
```

El autómata acepta las hileras que **terminan en ab**. El no determinismo está en s_0 con la letra *a*: la máquina *adivina* si esa *a* es la que empieza el sufijo final. *ba* se rechaza porque no termina en *ab*.

7.6.133 (★★★) VilCretas Convierta ese autómata a determinístico y compare con la opción `onlyreduce -> True`.

SOLUCIÓN

In[] :=

```
{AutomataDeterministicoEquivalente[nd],
AutomataDeterministicoEquivalente[nd, onlyreduce -> True]}
```

La construcción por subconjuntos genera, en principio, un estado por cada subconjunto de estados: hasta 2^n . Muchos son **inalcanzables**. `onlyreduce -> True` muestra solo la parte accesible, que suele ser muchísimo menor y es la única que importa.

7.6.134 (★★) VilCretas Genere máquinas y autómatas de práctica con los cuatro generadores nuevos.

SOLUCIÓN

`In[ ] :=`

```
{MaquinaRandom[5, 4, 3], MaquinaAutomataRandom[5, 4],
AutomataDRandom[5, 4], AutomataNDRandom[5, 4]}
```

`MaquinaRandom[n, m, h]` pide estados, símbolos de entrada y de salida; `MaquinaAutomataRandom` genera una máquina que **además** es autómata; y los dos últimos generan autómatas determinísticos y no determinísticos. Todos guardan el resultado en la variable `G`.

7.6.135 (★★★) VilCretas Encadene un generador con `DiagramaToMaquina` y la opción `componentes`.

SOLUCIÓN

`In[ ] :=`

```
MaquinaRandom[4, 3, 3];
DiagramaToMaquina[G, componentes -> True]
```

El generador devuelve un **diagrama**; `DiagramaToMaquina` hace el camino inverso y, con `componentes -> True`, entrega las componentes como datos. Generador y lector encajan: se puede producir un enunciado al azar y obtener acto seguido su tabla de transiciones.

7.6.136 (★★★) VilCretas Procese una hilera sobre un diagrama con `StringSalidaDiagrama` y las opciones `trace` y `traceone`.

SOLUCIÓN

`In[ ] :=`

```
maquina = MaquinaRandom[4, 3, 3];
hilera = RandomChoice[{a, b, c}, 12];
{StringSalidaDiagrama[maquina, hilera],
StringSalidaDiagrama[maquina, hilera, traceone -> True]}
```

`StringSalida` trabaja sobre las *componentes* de la máquina; `StringSalidaDiagrama` trabaja directamente sobre el **diagrama**, que es lo que devuelven los generadores. `trace -> True` lista los estados visitados y `traceone -> True` los anima de uno en uno.

7.6.137 (★★) VilCretas Dibuje el diagrama de una máquina con los estados dentro de discos, usando `shape` y `radio`.

SOLUCIÓN

`In[ ] :=`

```
MaquinaToDiagrama[{s0, s1, s2}, {a, b}, {0, 1}, s0,
{{s0, a, s1, 0}, {s0, b, s0, 1}, {s1, a, s2, 0},
{s1, b, s0, 1}, {s2, a, s2, 1}, {s2, b, s2, 0}},
shape -> True, radio -> 0.12]
```

En un diagrama de transiciones las etiquetas de las aristas son largas (entrada y salida separadas por una barra) y se solapan con facilidad. Los discos y un radio bien escogido son la diferencia entre una figura que se proyecta en clase y una que no se lee.

7.6.138 (★★★) MIXTO Procese la hilera *aab* en una máquina de estado finito con `StringSalida`, y luego anímela con `traceone`.

SOLUCIÓN

In[] :=

```
est = {s0, s1, s2}; ent = {a, b}; sal = {0, 1}; ini = s0;
tr = {{s0, a, s1, 0}, {s0, b, s0, 1}, {s1, a, s2, 0},
      {s1, b, s0, 1}, {s2, a, s2, 1}, {s2, b, s2, 0}};
StringSalida[est, ent, sal, ini, tr, {a, a, b}]
```

Out[] =

```
{0, 0, 0}
```

In[] :=

```
StringSalida[est, ent, sal, ini, tr, {a, a, b}, traceone -> True]
```

La salida {0,0,0} se lee columna a columna: cada símbolo de entrada produce uno de salida, de modo que **la hilera de salida tiene siempre la misma longitud que la de entrada**. `traceone -> True` devuelve una animación que avanza **de un paso por vez**, en lugar de la lista completa de estados que da `trace -> True`: es la forma de seguirla en clase sin que el estudiante vea el final antes de tiempo.

CAPÍTULO 8 • COMPLEMENTARIOS

Lenguajes y gramáticas

8.1 Alfabetos, hileras y lenguajes formales

8.5.1 (★) **A MANO** Defina alfabeto, hilera y lenguaje formal.

SOLUCIÓN Un **alfabeto** A es un conjunto finito no vacío de símbolos; una **hilera** sobre A es una sucesión finita de símbolos de A ; y un **lenguaje formal** sobre A es cualquier subconjunto de A^* , el conjunto de todas las hileras que pueden formarse con A , incluida la vacía ε .

8.5.2 (★) **A MANO** Escriba todas las hileras de longitud dos sobre $A = \{a, b\}$.

SOLUCIÓN Son aa , ab , ba y bb : en total $2^2 = 4$. En general hay $|A|^n$ hileras de longitud n .

8.5.3 (★★) **A MANO** ¿Cuántas hileras de longitud menor o igual que tres hay sobre un alfabeto de dos símbolos?

SOLUCIÓN Se suman las de cada longitud, incluida la vacía:

$$2^0 + 2^1 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15$$

8.5.4 (★) **A MANO** ¿Qué es la concatenación de hileras y cuál es su elemento neutro?

SOLUCIÓN Es escribir una hilera a continuación de otra: ab concatenado con ba da $abba$. Su elemento neutro es la hilera vacía ε , pues $\alpha\varepsilon = \varepsilon\alpha = \alpha$. La operación es asociativa pero **no conmutativa**.

8.5.5 (★★) **A MANO** Si $|\alpha| = 3$ y $|\beta| = 4$, ¿cuál es la longitud de $\alpha\beta$?

SOLUCIÓN Es 7: la longitud de una concatenación es la suma de las longitudes. En particular $|\varepsilon| = 0$.

8.5.6 (★★) **A MANO** ¿Es A^* un conjunto finito o infinito? Justifique.

SOLUCIÓN Es infinito numerable, aun cuando A sea finito: contiene hileras de toda longitud. Un lenguaje formal, en cambio, puede ser finito o infinito, porque es cualquier subconjunto de A^* .

8.5.7 (★★) **A MANO** Describa con palabras el lenguaje $L = \{a^n b \mid n \geq 0\}$ y dé tres de sus elementos.

SOLUCIÓN Son las hileras formadas por cero o más a seguidas de una única b . Tres elementos: b , ab y aab . La hilera ba no pertenece, porque la b debe ir al final.

8.5.8 (★★★) **A MANO** Describa el lenguaje $L = \{a^n b^n \mid n \geq 0\}$ y explique por qué no basta con contar símbolos.

SOLUCIÓN Son las hileras con tantas a como b , y con todas las a delante. No basta contar: $abab$ tiene dos de cada una y no pertenece. Hace falta recordar cuántas a se llevan leídas, y por eso este lenguaje **no** es regular, aunque sí libre del contexto.

8.2 Gramáticas y derivaciones

8.5.9 (★) **A MANO** Enumere las cuatro componentes de una gramática en forma normal.

SOLUCIÓN El conjunto de **símbolos no terminales** σ , el de **símbolos terminales** τ , el conjunto P de **producciones** o reglas de composición, y el **símbolo no terminal inicial** σ^* .

8.5.10 (★) **A MANO** ¿Cómo se escribe una gramática en el paquete **VilCretas**?

SOLUCIÓN Como un vector de cadenas de la forma "A->B". Si un mismo no terminal tiene varias producciones se escriben separadas por barras: "S->aS|b". Por convención los no terminales son letras mayúsculas, los terminales cualquier otro carácter, el símbolo inicial es S y la hilera vacía se escribe \[CurlyEpsilon].

8.5.11 (★) **VilCretas** Genere las palabras del lenguaje de la gramática $S \rightarrow aS \mid b$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aS|b"}, 5, 5]
```

Out[] =

```
{aaaab, aaab, aab, ab, b}
```

Los argumentos son la gramática, el número máximo de derivaciones y la longitud máxima de las palabras. El lenguaje es $\{a^n b\}$.

8.5.12 (★) **VilCretas** Repita la generación con a lo sumo tres derivaciones.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aS|b"}, 3, 3]
```

Out[] =

```
{aab, ab, b}
```

Con menos derivaciones se obtienen menos palabras: cada aplicación de $S \rightarrow aS$ agrega una a y consume una derivación.

8.5.13 (★) **VilCretas** Decida si aab y ba pertenecen a ese lenguaje.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aS|b"}, "aab"],  
ElementoLenguajeQ[{"S->aS|b"}, "ba"]}
```

Out[] =

```
{True, False}
```

La segunda tiene la b al principio, y ninguna producción permite colocarla ahí.

8.5.14 (★★) **VilCretas** Muestre las derivaciones que prueban que aab pertenece al lenguaje.

SOLUCIÓN

In[] :=

```
ElementoLenguajeQ[{"S->aS|b"}, "aab", derivaciones -> True]
```

La salida recorre el camino hacia atrás: aaS , aS , S . Leído en el otro sentido,

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$$

La opción **pruebas** fija cuántos intentos internos hace el comando; por defecto son treinta.

8.5.15 (★★) **VilCretas** Escriba la gramática anterior en notación de *Backus-Naur*.

SOLUCIÓN

```
In[ ] :=
```

```
NotationBNF[{"S->aS|b"}]
```

La salida es $\langle S \rangle ::= a \langle S \rangle | b$: los no terminales se encierran entre ángulos y la flecha se sustituye por $::=$. Con **componentes->True** la notación se devuelve como vector.

8.5.16 (★★) **VilCretas** Obtenga las cuatro componentes de la gramática en forma normal.

SOLUCIÓN

```
In[ ] :=
```

```
NotationFNormal[{"S->aS|b"}]
```

El comando informa que los no terminales son $\{S\}$, los terminales $\{a, b\}$, las producciones $S \rightarrow aS$ y $S \rightarrow b$, y el símbolo inicial S . Es la lectura de la gramática como la 4-tupla de la definición.

8.5.17 (★★) **VilCretas** Genere el lenguaje de la gramática $S \rightarrow aA$, $A \rightarrow bS \mid b$ y descríballo.

SOLUCIÓN

```
In[ ] :=
```

```
ElementosLenguaje[{"S->aA", "A->bS|b"}, 6, 6]
```

```
Out[ ] =
```

```
{ab, abab, ababab}
```

El lenguaje es $\{(ab)^n \mid n \geq 1\}$: los dos no terminales se turnan y cada vuelta agrega el bloque ab .

8.5.18 (★★) **VilCretas** Compruebe que ab y $abab$ pertenecen a ese lenguaje.

SOLUCIÓN

```
In[ ] :=
```

```
{ElementoLenguajeQ[{"S->aA", "A->bS|b"}, "ab"],  
ElementoLenguajeQ[{"S->aA", "A->bS|b"}, "abab"]}
```

```
Out[ ] =
```

```
{True, True}
```

Toda palabra del lenguaje tiene longitud par: la producción $A \rightarrow b$ es la única que cierra la derivación.

8.5.19 (★★) VilCretas Genere el lenguaje de la gramática $S \rightarrow aB$, $B \rightarrow bB \mid b$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aB", "B->bB|b"}, 5, 4]
```

Out[] =

```
{ab, abb, abbb}
```

El lenguaje es $\{ab^n \mid n \geq 1\}$: una sola a inicial seguida de al menos una b .

8.5.20 (★★) VilCretas Decida si $abbb$ y aab pertenecen al lenguaje anterior.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aB", "B->bB|b"}, "abbb"],
ElementoLenguajeQ[{"S->aB", "B->bB|b"}, "aab"]}
```

Out[] =

```
{True, False}
```

La segunda tiene dos a , y la única producción que introduce una a se aplica una sola vez, al principio.

8.5.21 (★★★) VilCretas Genere el lenguaje $\{a^n b^n\}$ con la gramática $S \rightarrow aSb \mid \varepsilon$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aSb|[CurlyEpsilon]"}, 6, 6]
```

Out[] =

```
{, aabb, ab}
```

El primer elemento de la lista es la **hilera vacía**, que se ve como un hueco. Cada derivación agrega simultáneamente una a por delante y una b por detrás, lo que garantiza que haya la misma cantidad de cada una.

8.5.22 (★★) VilCretas Compruebe que $aabb$ y $aaabbb$ pertenecen a ese lenguaje, y que aab y $abab$ no.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aSb|[CurlyEpsilon]"}, "aabb"],
ElementoLenguajeQ[{"S->aSb|[CurlyEpsilon]"}, "aaabbb"],
ElementoLenguajeQ[{"S->aSb|[CurlyEpsilon]"}, "aab"],
ElementoLenguajeQ[{"S->aSb|[CurlyEpsilon]"}, "abab"]}
```

Out[] =

```
{True, True, False, False}
```

aab tiene dos a y una b ; $abab$ tiene dos de cada una, pero intercaladas. La gramática exige el bloque de a completo antes del de b .

- 8.5.23 (★★★) **VilCretas** Genere las palabras de longitud a lo sumo tres del lenguaje de los palíndromos sobre $\{a, b\}$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aSa|bSb|a|b|\""}, 5, 3]
```

Out[] =

```
{, a, aa, aaa, aba, b, bab, bb, bbb}
```

Las producciones $S \rightarrow aSa$ y $S \rightarrow bSb$ agregan el mismo símbolo a los dos extremos; las tres restantes cierran la derivación con un centro de longitud cero o uno.

- 8.5.24 (★★) **VilCretas** Genere el lenguaje de $S \rightarrow aaS \mid a$ e interprételo.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aaS|a"}, 4, 5]
```

Out[] =

```
{a, aaa, aaaaa}
```

Son las hileras de a de longitud **impar**: cada aplicación de la primera producción agrega dos, y la segunda cierra con una.

- 8.5.25 (★) **A MANO** Enuncie qué es una derivación.

SOLUCIÓN Es sustituir, dentro de una hilera, el lado izquierdo de una producción por su lado derecho. Se escribe $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$ si $\alpha \rightarrow \beta$ es una producción. Una palabra pertenece al lenguaje si se obtiene del símbolo inicial mediante una sucesión de derivaciones.

- 8.5.26 (★★) **A MANO** Derive a mano la palabra $aaab$ en la gramática $S \rightarrow aS \mid b$.

SOLUCIÓN

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aaaS \Rightarrow aaab$$

Se aplicó tres veces $S \rightarrow aS$ y una vez $S \rightarrow b$. Se necesitan cuatro derivaciones, una más que la cantidad de a .

- 8.5.27 (★★) **A MANO** Clasifique la gramática $S \rightarrow aS \mid b$ según los tres tipos de la definición.

SOLUCIÓN Es **regular**: todas sus producciones tienen la forma $\alpha \rightarrow x\beta$ o $\alpha \rightarrow x$ con α, β no terminales y x terminal. Toda gramática regular es también libre del contexto y sensible al contexto: los tipos están encajados.

- 8.5.28 (★★) **A MANO** ¿Es regular la gramática $S \rightarrow aSb \mid \varepsilon$?

SOLUCIÓN No. La producción $S \rightarrow aSb$ coloca un terminal **después** del no terminal, y la forma regular solo admite $\alpha \rightarrow x\beta$, con el terminal delante. Sí es libre del contexto, porque el lado izquierdo es un único no terminal.

- 8.5.29** (★★★) **A MANO** Escriba una gramática regular para el lenguaje de las hileras sobre $\{a, b\}$ que terminan en b .

SOLUCIÓN

$$S \rightarrow aS \mid bS \mid b$$

Las dos primeras producciones dejan pasar cualquier prefijo y la tercera obliga a cerrar con una b . Todas tienen la forma regular exigida.

- 8.5.30** (★★★) **A MANO** Escriba una gramática para el lenguaje de las hileras sobre $\{a, b\}$ de longitud par.

SOLUCIÓN

$$S \rightarrow aaS \mid abS \mid baS \mid bbS \mid \varepsilon$$

Cada producción agrega dos símbolos, de modo que la longitud siempre crece de dos en dos, y ε cierra con longitud par.

- 8.5.31** (★★) **A MANO** ¿Puede una misma lengua tener dos gramáticas distintas?

SOLUCIÓN Sí, y es lo habitual. Por ejemplo $S \rightarrow aS \mid b$ y $S \rightarrow aA \mid b$, $A \rightarrow aA \mid b$ generan las dos el lenguaje $\{a^n b\}$. Lo que caracteriza al lenguaje es el conjunto de palabras, no las reglas que lo producen.

- 8.5.32** (★★) **VilCretas** Genere el lenguaje $\{a^n b^m \mid n, m \geq 1\}$ con la gramática $S \rightarrow aS \mid aB$, $B \rightarrow bB \mid b$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aS|aB", "B->bB|b"}, 5, 4]
```

Out[] =

```
{aaab, aab, aabb, ab, abb, abbb}
```

Las cantidades de a y de b son ahora independientes: el no terminal S acumula las a y B las b . A diferencia de $\{a^n b^n\}$, este lenguaje sí es regular.

- 8.5.33** (★★) **VilCretas** Genere el lenguaje de las hileras sobre $\{a, b\}$ que empiezan con a .

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA|a", "A->aA|bA|a|b"}, 4, 3]
```

Out[] =

```
{a, aa, aaa, aab, ab, aba, abb}
```

La producción $S \rightarrow a$ cubre la palabra de un solo símbolo y A deja pasar cualquier continuación.

- 8.5.34** (★★) **VilCretas** Genere el lenguaje de las hileras que terminan en b y compruebe dos casos.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aS|bS|b"}, 4, 3]
Out[ ] =
{aab, ab, abb, b, bab, bb, bbb}
In[ ] :=
{ElementoLenguajeQ[{"S->aS|bS|b"}, "bab"],
ElementoLenguajeQ[{"S->aS|bS|b"}, "ba"]}
Out[ ] =
{True, False}
```

Todas las palabras generadas acaban en b , y ba queda fuera por terminar en a .

8.5.35 (★★★) **VilCretas** Compruebe la gramática de las hileras de longitud par que escribió antes.

SOLUCIÓN

```
In[ ] :=
ElementosLenguaje[{"S->aaS|abS|baS|bbS|\""}, 3, 4]
Out[ ] =
{, aa, ab, ba, bb}
```

Con a lo sumo tres derivaciones y longitud máxima cuatro aparecen la hilera vacía y las cuatro de longitud dos. Subiendo los parámetros se obtendrían también las dieciséis de longitud cuatro.

8.5.36 (★★) **VilCretas** Escriba en notación *BNF* una gramática con dos no terminales.

SOLUCIÓN

```
In[ ] :=
NotationBNF[{"S->aB", "B->bB|b"}]
```

La salida trae una línea por cada no terminal: $\langle S \rangle ::= a\langle B \rangle$ y $\langle B \rangle ::= b\langle B \rangle | b$. Es la notación con la que se describen habitualmente los lenguajes de programación.

8.5.37 (★★) **A MANO** ¿Cuántas derivaciones hacen falta para generar $a^n b$ con $S \rightarrow aS \mid b$?

SOLUCIÓN Hacen falta $n + 1$: n aplicaciones de $S \rightarrow aS$ y una de $S \rightarrow b$. Ese es el argumento que hay que tener presente al escoger el segundo parámetro de **ElementosLenguaje**.

8.5.38 (★★) **A MANO** Distinga el lenguaje vacío del lenguaje $\{\varepsilon\}$.

SOLUCIÓN El **lenguaje vacío** no tiene ninguna palabra; el lenguaje $\{\varepsilon\}$ tiene exactamente una, la hilera vacía. El primero tiene cardinalidad cero y el segundo, uno.

8.5.39 (★★★) **A MANO** ¿Cuántos lenguajes formales distintos hay sobre un alfabeto de dos símbolos, si solo se consideran hileras de longitud a lo sumo dos?

SOLUCIÓN Hay $1 + 2 + 4 = 7$ hileras posibles, y un lenguaje es cualquier subconjunto de ellas: $2^7 = 128$. Sobre A^* completo la cantidad de lenguajes ya no es numerable.

8.3 Gramáticas regulares y autómatas

8.5.40 (★) **A MANO** Enuncie la relación entre gramáticas regulares y autómatas de estado finito.

SOLUCIÓN Son dos descripciones del mismo tipo de lenguaje. Dada una gramática regular se construye un autómata no determinístico que reconoce su lenguaje, y dado un autómata determinístico se obtiene una gramática regular que lo genera.

8.5.41 (★★) **VilCretas** Construya el autómata asociado a la gramática $S \rightarrow aS \mid b$.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|b"}]
```

El comando informa los estados $\{EA, S\}$, el estado inicial S , el aceptado EA y la función de transición, y dibuja el diagrama. El estado EA es el *estado de aceptación* que se añade para las producciones de la forma $S \rightarrow b$. Con `dfa->True` se obtiene también un autómata determinístico equivalente, y el resultado queda en G .

8.5.42 (★★) **VilCretas** Compruebe que el autómata obtenido acepta aab y rechaza ba .

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|b"}];
{StringAceptadaQ[G, {a, a, b}], StringAceptadaQ[G, {b, a}]}
```

Out[] =

```
{True, False}
```

Son las mismas respuestas que dio `ElementoLenguajeQ` sobre la gramática: autómata y gramática describen el mismo lenguaje.

8.5.43 (★★) **VilCretas** Construya la gramática regular del autómata que acepta las hileras con una cantidad par de a .

SOLUCIÓN

In[] :=

```
AA = Automata[{S, A}, {a, b}, S,
{S, a, A}, {S, b, S}, {A, a, S}, {A, b, A}}, {S}];
AutomataToRegular[AA, componentes -> True]
```

Out[] =

```
{S, A}, {a, b}, {A->a, A->aS, A->bA, S->aA, S->b, S->bS}, S}
```

Los estados del autómata pasan a ser los no terminales de la gramática, y por eso deben nombrarse con letras mayúsculas siendo S el inicial. Cada transición genera una producción.

8.5.44 (★★) **VilCretas** Obtenga esa misma gramática en notación *BNF*.

SOLUCIÓN

In[] :=

```
AA = Automata[{S, A}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, S}, {A, b, A}}, {S}];
AutomataToRegular[AA, bnf -> True]
```

La salida es $\langle A \rangle ::= a | a \langle S \rangle | b \langle A \rangle$ y $\langle S \rangle ::= a \langle A \rangle | b | b \langle S \rangle$. Sin opciones el comando devuelve la gramática en forma normal.

- 8.5.45 (★★★) **A MANO** Explique de dónde salen las producciones $A \rightarrow a$ y $A \rightarrow aS$ en el ejercicio anterior.

SOLUCIÓN Ambas provienen de la transición de A con entrada a hacia S . La producción $A \rightarrow aS$ traduce el paso al estado siguiente; y como S es un estado **aceptado**, se agrega además $A \rightarrow a$, que permite terminar la derivación justo ahí.

- 8.5.46 (★★) **A MANO** ¿Por qué la gramática que se obtiene de un autómata es siempre regular?

SOLUCIÓN Porque cada producción nace de una transición y tiene la forma $\alpha \rightarrow x\beta$, con α y β estados (no terminales) y x un símbolo de entrada (terminal); y las que corresponden a estados aceptados tienen la forma $\alpha \rightarrow x$. Son exactamente las formas admitidas por la definición de gramática regular.

- 8.5.47 (★★★) **A MANO** El lenguaje $\{a^n b^n\}$ no es regular. ¿Qué implica eso sobre los autómatas de estado finito?

SOLUCIÓN Que ningún autómata de estado finito lo reconoce. Un autómata tiene una cantidad **finita** de estados y no puede recordar un valor de n arbitrariamente grande. Por eso ese lenguaje admite una gramática libre del contexto, pero no una regular.

- 8.5.48 (★★★) **MIXTO** Construya el autómata de la gramática $S \rightarrow aA$, $A \rightarrow bS \mid b$ y compruebe que acepta $abab$.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aA", "A->bS|b"}];
StringAceptadaQ[G, {a, b, a, b}]
```

Out[] =

True

El autómata reconoce $(ab)^n$, el mismo lenguaje que generaba la gramática. Es la comprobación práctica del teorema de equivalencia.

- 8.5.49 (★★) **MIXTO** Compruebe que ese autómata rechaza aba .

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aA", "A->bS|b"}];
StringAceptadaQ[G, {a, b, a}]
```

Out[] =

False

La hilera tiene longitud impar y todas las palabras del lenguaje tienen longitud par, de modo que el rechazo era previsible sin ejecutar nada.

- 8.5.50** (★★★) **A MANO** Dado el autómata que acepta las hileras que empiezan con a , escriba a mano una gramática regular equivalente.

SOLUCIÓN Con S el estado inicial, A el aceptado y B la trampa de rechazo:

$$S \rightarrow aA \mid a \mid bB, \quad A \rightarrow aA \mid bA \mid a \mid b, \quad B \rightarrow aB \mid bB$$

Las producciones de B no llevan a ninguna palabra, de modo que pueden eliminarse sin cambiar el lenguaje.

- 8.5.51** (★★) **A MANO** ¿Qué papel juega el estado EA que agrega `RegularToAutomata`?

SOLUCIÓN Es un estado de aceptación artificial. Las producciones de la forma $\alpha \rightarrow x$, que terminan la derivación sin dejar no terminales, necesitan un estado final al que llegar; ese es EA , y no tiene transiciones de salida.

- 8.5.52** (★★★) **MIXTO** Verifique que la gramática obtenida de un autómata genera el mismo lenguaje que el autómata de partida, en longitud tres.

SOLUCIÓN

In[] :=

```
AA = Automata[{S, A}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, S}, {A, b, A}}, {S}];
LenguajeStrings[AA, 3]
```

Out[] =

```
{a, a, b}, {a, b, a}, {b, a, a}, {b, b, b}
```

In[] :=

```
ElementosLenguaje[{"S->aA|b|bS", "A->a|aS|bA"}, 6, 3]
```

Out[] =

```
{aa, aab, aba, b, baa, bb, bbb}
```

Las cuatro palabras de longitud tres coinciden en ambas listas. La gramática genera además b , aa y bb , de longitudes uno y dos, que el autómata también acepta: todas tienen una cantidad par de a .

- 8.5.53** (★★) **VilCretas** Construya el autómata de la gramática de las hileras que terminan en b y compruébelo.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|bS|b"}];
{StringAcceptadaQ[G, {a, b}], StringAcceptadaQ[G, {b, a}]}
```

Out[] =

```
{True, False}
```

El autómata resultante es no determinístico: desde S con entrada b se puede ir a S o al estado de aceptación EA , según si la b es la última o no.

8.5.54 (★★) **VilCretas** Enumere las hileras de longitud dos que acepta ese autómata.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|bS|b"}];
LenguajeStrings[G, 2]
```

Out[] =

```
{{a, b}, {b, b}}
```

Coinciden con las palabras de longitud dos que generaba la gramática: *ab* y *bb*.

8.5.55 (★★) **VilCretas** Obtenga también el autómata determinístico equivalente.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|b"}, dfa -> True]
```

Con `dfa->True` el comando construye primero el autómata no determinístico y después su determinístico equivalente, mostrando ambos diagramas. `colores->True` los presenta coloreados.

8.5.56 (★★★) **A MANO** Resuma el ciclo completo gramática regular, autómata y de vuelta.

SOLUCIÓN El recorrido tiene tres pasos:

1. de la gramática regular al autómata no determinístico, con `RegularToAutomata`;
2. de ese autómata al determinístico equivalente, con `AutomataDeterministicoEquivalente`;
3. del determinístico a una gramática regular, con `AutomataToRegular`.

La gramática final no tiene por qué ser la de partida, pero **genera el mismo lenguaje**.

8.5.57 (★★) **A MANO** ¿Por qué `AutomataToRegular` exige que los estados sean letras mayúsculas y que el inicial se llame S?

SOLUCIÓN Porque los estados se convierten en símbolos no terminales, y la convención del paquete es que los no terminales son letras mayúsculas y el inicial es S. Con estados llamados *q0* o *q1* el comando se queda **sin evaluar**, sin mensaje de error.

8.5.58 (★★★) **A MANO** Escriba una gramática regular para el lenguaje de las hileras sobre $\{a, b\}$ con una cantidad impar de *a*.

SOLUCIÓN Con *S* (llevo un número par) y *A* (impar), aceptando en *A*:

$$S \rightarrow bS \mid aA \mid a, \quad A \rightarrow bA \mid aS \mid b$$

Es la misma construcción del autómata de la paridad, con las producciones terminales colocadas en el estado que ahora es el aceptado.

8.5.59 (★★) **MIXTO** Genere el lenguaje de $S \rightarrow aaS \mid b$ y descríballo.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aaS|b"}, 5, 7]
```

```
Out[ ] =
```

```
{aaaaaab, aaaab, aab, b}
```

El lenguaje es $\{a^{2n}b \mid n \geq 0\}$: una cantidad **par** de a seguida de una única b . Cada aplicación de la primera producción añade dos a .

8.5.60 (★★) **MIXTO** Decida si *aaaaab* y *aaab* pertenecen a ese lenguaje.

SOLUCIÓN

```
In[ ] :=
```

```
{ElementoLenguajeQ[{"S->aaS|b"}, "aaaaab"],  
ElementoLenguajeQ[{"S->aaS|b"}, "aaab"]}
```

```
Out[ ] =
```

```
{True, False}
```

La segunda tiene tres a , cantidad impar: ninguna sucesión de derivaciones puede producirla, porque las a solo entran de dos en dos.

8.5.61 (★★★) **MIXTO** Genere el lenguaje de $S \rightarrow aS \mid bS \mid aa$ hasta longitud cuatro.

SOLUCIÓN

```
In[ ] :=
```

```
ElementosLenguaje[{"S->aS|bS|aa"}, 4, 4]
```

```
Out[ ] =
```

```
{aa, aaa, aaaa, abaa, baa, baaa, bbaa}
```

El lenguaje es el de las hileras que **terminan** en aa : cualquier prefijo seguido del bloque final. Las dos primeras producciones dejan pasar todo y la tercera cierra.

8.5.62 (★★★) **MIXTO** Genere el lenguaje de $S \rightarrow aB \mid bA$, $A \rightarrow a \mid aS$, $B \rightarrow b \mid bS$ e identifíquelo.

SOLUCIÓN

```
In[ ] :=
```

```
ElementosLenguaje[{"S->aB|bA", "A->a|aS", "B->b|bS"}, 5, 4]
```

```
Out[ ] =
```

```
{ab, abab, abba, ba, baab, baba}
```

Son las hileras con **igual número de a que de b** y longitud par, en las que los dos no terminales llevan la cuenta de cuál letra falta. Es un lenguaje regular pese a parecer un problema de conteo.

8.5.63 (★★★) **A MANO** ¿Por qué el lenguaje anterior sí es regular y $\{a^n b^n\}$ no?

SOLUCIÓN Porque aquí las letras pueden **intercalarse**: basta recordar si van igualadas o falta una de cada tipo, información acotada. En $\{a^n b^n\}$ hay que recordar *cuántas* a se llevan, un valor sin cota. La diferencia está en si la cuenta puede crecer sin límite.

8.5.64 (★★★) **MIXTO** Genere el lenguaje de los palíndromos con centro c : $S \rightarrow aSa \mid bSb \mid c$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aSa|bSb|c"}, 4, 5]
```

Out[] =

```
{aacia, abcba, aca, bacab, bbcbb, bcb, c}
```

Cada derivación añade la misma letra a los dos extremos, de modo que todas las palabras son palíndromos de longitud impar con una c en el centro. Es libre del contexto y no regular.

8.5.65 (★★) **MIXTO** Compruebe si $abcba$ y $abcb$ pertenecen a ese lenguaje.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aSa|bSb|c"}, "abcba"],  
ElementoLenguajeQ[{"S->aSa|bSb|c"}, "abcb"]}
```

Out[] =

```
{True, False}
```

La primera es palíndromo y la segunda no: $abcb$ leída al revés es $bcba$. La c debe estar exactamente en el centro.

8.5.66 (★★★) **MIXTO** Genere el lenguaje de $S \rightarrow 0S1 \mid 01$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->0S1|01"}, 5, 8]
```

Out[] =

```
{00001111, 000111, 0011, 01}
```

Es $\{0^n 1^n \mid n \geq 1\}$, el ejemplo clásico de lenguaje libre del contexto y **no regular**. A diferencia de la versión con ε , la hilera vacía no pertenece.

8.5.67 (★★) **MIXTO** Compruebe dos casos en ese lenguaje.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->0S1|01"}, "000111"],  
ElementoLenguajeQ[{"S->0S1|01"}, "0011"]}
```

Out[] =

```
{True, True}
```

Ambas pertenecen: tienen tantos ceros como unos y todos los ceros delante. La hilera 0101, con la misma cantidad de cada símbolo, no pertenecería.

8.5.68 (★★★) **MIXTO** Genere el lenguaje de $S \rightarrow aA$, $A \rightarrow bA \mid \varepsilon$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA", "A->bA|[CurlyEpsilon]"}, 5, 4]
```

Out[] =

```
{a, ab, abb}
```

Es $\{ab^n \mid n \geq 0\}$: una a seguida de cualquier cantidad de b , incluida ninguna. La producción $A \rightarrow \varepsilon$ es la que permite cerrar sin añadir nada.

8.5.69 (★★) **A MANO** ¿Qué diferencia hay entre $A \rightarrow \varepsilon$ y no escribir ninguna producción para A ?

SOLUCIÓN La primera permite **eliminar** el no terminal y terminar la derivación; la segunda deja a A sin salida y toda derivación que llegue a él se estanca sin producir palabra. En el primer caso el lenguaje puede ser infinito; en el segundo, vacío.

8.5.70 (★★) **VilCretas** Escriba en notación *BNF* la gramática de tres no terminales del ejercicio del conteo equilibrado.

SOLUCIÓN

In[] :=

```
NotationBNF[{"S->aB|bA", "A->a|aS", "B->b|bS"}]
```

La salida trae una línea por no terminal: $\langle S \rangle ::= a \langle B \rangle | b \langle A \rangle$, $\langle A \rangle ::= a | a \langle S \rangle$ y $\langle B \rangle ::= b | b \langle S \rangle$. Es la notación con la que se describen los lenguajes de programación.

8.5.71 (★★) **VilCretas** Obtenga la forma normal de $S \rightarrow aaS \mid b$.

SOLUCIÓN

In[] :=

```
NotationFNormal[{"S->aaS|b"}]
```

Informa que los no terminales son $\{S\}$, los terminales $\{a, b\}$, las producciones $S \rightarrow aaS$ y $S \rightarrow b$, y el símbolo inicial S . Es la 4-tupla de la definición, extraída automáticamente de la notación compacta.

8.5.72 (★★★) **A MANO** ¿Es regular la gramática $S \rightarrow aaS \mid b$?

SOLUCIÓN **No** en sentido estricto: la producción $S \rightarrow aaS$ tiene *dos* terminales antes del no terminal, y la forma regular admite uno solo. Sí genera un lenguaje regular, y se puede reescribir como $S \rightarrow aA$, $A \rightarrow aS$, $S \rightarrow b$. Es necesario distinguir *gramática regular* de *lenguaje regular*.

8.5.73 (★★) **MIXTO** Compruebe que la reescritura anterior genera el mismo lenguaje.

SOLUCIÓN

In[] :=

```
{ElementosLenguaje[{"S->aA|b", "A->aS"}, 6, 7],
```

```
ElementosLenguaje[{"S->aaS|b"}, 5, 7]}
```

Out[] =

$\{\{aaaab, aab, b\}, \{aaaaaab, aaaab, aab, b\}\}$

Las palabras coinciden hasta donde alcanzan las derivaciones permitidas: la reescritura necesita **dos** pasos por cada bloque aa , de modo que con el mismo tope de derivaciones llega menos lejos. El lenguaje generado es el mismo; lo que cambia es el número de derivaciones que hace falta contar.

- 8.5.74 (★★★) **A MANO** Escriba una gramática regular estricta para el lenguaje de las hileras sobre $\{a, b\}$ que terminan en bab .

SOLUCIÓN Se necesita un no terminal por cada prefijo reconocido:

$$S \rightarrow aS \mid bS \mid bA, \quad A \rightarrow aB, \quad B \rightarrow b$$

S deja pasar cualquier prefijo, A recuerda que se vio una b y B que se vio ba . La última producción cierra con un solo terminal.

- 8.5.75 (★★) **MIXTO** Compruebe la gramática anterior.

SOLUCIÓN

`In[ ] :=`

```
ElementosLenguaje[{"S->aS|bS|bA", "A->aB", "B->b"}, 6, 4]
```

`Out[ ] =`

```
{abab, bab, bbab}
```

Todas terminan en bab . Con longitud cuatro solo hay dos, porque el sufijo obligatorio ya ocupa tres símbolos.

- 8.5.76 (★★★) **A MANO** Clasifique: (a) $aA \rightarrow aB$; (b) $S \rightarrow ASB$; (c) $A \rightarrow \varepsilon$ con $A \neq S$; (d) $S \rightarrow abc$.

SOLUCIÓN (a) **Sensible al contexto**: el lado izquierdo no es un solo no terminal. (b) **Libre del contexto**, y no regular: el lado derecho lleva dos no terminales. (c) Libre del contexto; en la definición del libro también cuenta como regular. (d) Libre del contexto y no regular en sentido estricto: tres terminales seguidos.

- 8.5.77 (★★) **A MANO** Ordene los tres tipos de gramática por generalidad y dé un lenguaje que separe cada nivel.

SOLUCIÓN Regular $\subset$ libre del contexto $\subset$ sensible al contexto. El lenguaje $\{a^n b^n\}$ es libre del contexto y no regular; $\{a^n b^n c^n\}$ es sensible al contexto y no libre del contexto. Cada inclusión es **estricta**.

- 8.5.78 (★★★) **A MANO** Escriba una gramática para $\{a^n b^n c^n\}$ y explique por qué no puede ser libre del contexto.

SOLUCIÓN Con producciones sensibles al contexto se logra, por ejemplo generando $a^n (BC)^n$ y reordenando con reglas como $CB \rightarrow BC$ que dependen del entorno. No puede ser libre del contexto porque habría que llevar **dos** cuentas simultáneas (las a frente a las b y las b frente a las c), y una pila solo permite una.

- 8.5.79 (★★) **A MANO** ¿Cuántas hileras hay de longitud menor o igual que dos sobre un alfabeto de tres símbolos, y cuántos lenguajes se pueden formar con ellas?

SOLUCIÓN Hay $3^0 + 3^1 + 3^2 = 13$ hileras y por lo tanto $2^{13} = 8192$ lenguajes. El conteo de lenguajes crece como una exponencial del conteo de hileras, que a su vez es exponencial en la longitud.

8.5.80 (★★) **MIXTO** Compruebe esos dos números.

SOLUCIÓN

In[] :=

```
{3^0 + 3^1 + 3^2, 2^13}
```

Out[] =

```
{13, 8192}
```

Trece hileras y 8192 lenguajes. Sobre A^* completo la cantidad de lenguajes deja de ser numerable, mientras que la de gramáticas sí lo es: hay lenguajes que **ninguna** gramática genera.

8.5.81 (★★★) **A MANO** Justifique la última afirmación: hay lenguajes sin gramática.

SOLUCIÓN Las gramáticas son objetos **finitos** escritos sobre un alfabeto finito, de modo que el conjunto de todas ellas es numerable. Los lenguajes sobre A^* son los subconjuntos de un conjunto numerable infinito, y por el argumento diagonal de *Cantor* son no numerables. Por cardinalidad, casi todos los lenguajes carecen de gramática.

8.5.82 (★★★) **MIXTO** Construya la gramática regular equivalente al autómata con estados $\{S, A, B\}$ que acepta las hileras terminadas en ab .

SOLUCIÓN

In[] :=

```
Aut = Automata[{S, A, B}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, A}, {A, b, B},
  {B, a, A}, {B, b, S}}, {B}];
AutomataToRegular[Aut, componentes -> True]
```

Out[] =

```
{{S, A, B}, {a, b}, {A->aA, A->b, A->bB, B->aA, B->bS, S->aA, S->bS}, S}
```

Cada transición da una producción, y las que llegan al estado aceptado B dan además una producción con un solo terminal: $A \rightarrow b$.

8.5.83 (★★) **MIXTO** Compruebe que ese autómata acepta lo esperado en longitud tres.

SOLUCIÓN

In[] :=

```
Aut = Automata[{S, A, B}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, A}, {A, b, B},
  {B, a, A}, {B, b, S}}, {B}];
LenguajeStrings[Aut, 3]
```

Out[] =

```
{{a, a, b}, {b, a, b}}
```

Las dos hileras de longitud tres que terminan en ab . El primer símbolo puede ser cualquiera de los dos.

- 8.5.84 (★★★) **A MANO** ¿Por qué la conversión de autómeta a gramática exige que los estados se llamen con letras mayúsculas y el inicial sea S ?

SOLUCIÓN Porque los estados se convierten en **símbolos no terminales**, y la convención del paquete es que los no terminales son letras mayúsculas y el inicial es S . Con estados llamados q_0 o s_1 el comando se queda **sin evaluar**, sin ningún mensaje de error.

- 8.5.85 (★★) **VilCretas** Construya el autómeta asociado a la gramática $S \rightarrow aA \mid b, A \rightarrow bS \mid a$.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aA|b", "A->bS|a"}]
```

El comando informa los estados $\{A, EA, S\}$, el inicial S , el aceptado EA y la función de transición. El estado EA se añade para las producciones que terminan la derivación, como $S \rightarrow b$.

- 8.5.86 (★★★) **MIXTO** Compruebe que ese autómeta acepta aa y rechaza ab .

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aA|b", "A->bS|a"}];
{StringAceptadaQ[G, {a, a}], StringAceptadaQ[G, {a, b}]}
```

Out[] =

```
{True, False}
```

Coincide con lo que responde **ElementoLenguajeQ** sobre la gramática: autómeta y gramática describen el mismo lenguaje.

- 8.5.87 (★★★) **A MANO** Resuma el procedimiento para pasar de gramática regular a autómeta.

SOLUCIÓN Cada no terminal se convierte en un estado y el inicial en el estado inicial. Una producción $\alpha \rightarrow x\beta$ da la transición de α a β con entrada x . Una producción $\alpha \rightarrow x$ da una transición de α a un **estado de aceptación** añadido. Y $\alpha \rightarrow \varepsilon$ declara α aceptado.

- 8.5.88 (★★★) **A MANO** ¿Por qué el autómeta que se obtiene de una gramática regular suele ser no determinístico?

SOLUCIÓN Porque un mismo no terminal puede tener varias producciones que empiecen con el **mismo terminal**: $S \rightarrow aA$ y $S \rightarrow aB$ dan dos transiciones desde S con la entrada a . Eso es exactamente el no determinismo, y se elimina después con el método de los subconjuntos.

- 8.5.89 (★★) **A MANO** ¿Qué es una derivación por la izquierda?

SOLUCIÓN Aquella en la que en cada paso se sustituye el **no terminal más a la izquierda**. Toda palabra de un lenguaje libre del contexto admite al menos una derivación por la izquierda, y contar cuántas distintas tiene es la manera de detectar si la gramática es *ambigua*.

- 8.5.90 (★★★) **A MANO** ¿Qué es una gramática ambigua? Dé un ejemplo.

SOLUCIÓN Aquella en la que alguna palabra admite **dos derivaciones por la izquierda distintas**, es decir dos árboles de derivación. El ejemplo clásico es $E \rightarrow E + E \mid E * E \mid a$:

la palabra $a + a * a$ se puede derivar agrupando primero la suma o primero el producto. La ambigüedad es un defecto de la gramática, no del lenguaje.

8.5.91 (★★★) **A MANO** ¿Por qué importa la ambigüedad en un lenguaje de programación?

SOLUCIÓN Porque el árbol de derivación determina el **significado**: en $a + a * a$, agrupar primero la suma da un resultado y agrupar primero el producto da otro. Un compilador necesita una gramática no ambigua, o reglas adicionales de precedencia, para asignar un único sentido a cada programa.

8.5.92 (★★) **A MANO** Escriba una gramática no ambigua para expresiones con suma y producto que respete la precedencia.

SOLUCIÓN Se estratifica por niveles:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow a$$

La suma solo aparece en el nivel superior y el producto en el intermedio, de modo que el producto **siempre** queda más abajo en el árbol y se evalúa antes.

8.5.93 (★★★) **A MANO** ¿Puede una gramática regular ser ambigua?

SOLUCIÓN Sí. La gramática $S \rightarrow aS \mid aS \mid b$ (con la producción repetida) o $S \rightarrow aA \mid aB$, $A \rightarrow b$, $B \rightarrow b$ generan ab por dos caminos distintos. Lo que ocurre es que para lenguajes regulares **siempre existe** una gramática no ambigua: la que se obtiene del autómata determinístico mínimo.

8.5.94 (★★) **A MANO** Defina concatenación de lenguajes y dé un ejemplo.

SOLUCIÓN $L_1 L_2 = \{\alpha\beta \mid \alpha \in L_1, \beta \in L_2\}$: todas las formas de pegar una palabra de cada uno. Con $L_1 = \{a, ab\}$ y $L_2 = \{b, c\}$ resulta $\{ab, ac, abb, abc\}$. Obsérvese que ab aparece una sola vez aunque se pueda formar de dos maneras: es un **conjunto**.

8.5.95 (★★★) **A MANO** ¿Cuántos elementos tiene $L_1 L_2$ si $|L_1| = m$ y $|L_2| = n$?

SOLUCIÓN A lo sumo mn , pero puede tener menos si distintas parejas producen la misma concatenación. Con $L_1 = \{a, aa\}$ y $L_2 = \{a, aa\}$ se obtienen solo tres palabras ($aa, aaa, aaaa$) en lugar de cuatro, porque $a \cdot aa$ y $aa \cdot a$ coinciden.

8.5.96 (★★) **A MANO** Defina la clausura de Kleene L^* y calcule $\{a\}^*$ y $\emptyset^*$.

SOLUCIÓN L^* es la unión de todas las potencias L^k con $k \geq 0$, incluida $L^0 = \{\varepsilon\}$. Así, $\{a\}^* = \{\varepsilon, a, aa, aaa, \dots\}$ y $\emptyset^* = \{\varepsilon\}$: aunque el lenguaje sea vacío, su clausura contiene la hilera vacía por el caso $k = 0$.

8.5.97 (★★★) **A MANO** ¿Es cierto que $(L^*)^* = L^*$?

SOLUCIÓN Sí. Concatenar palabras que ya son concatenaciones de elementos de L vuelve a dar una concatenación de elementos de L . La clausura es **idempotente**, igual que tomar el cierre de un conjunto en topología.

8.5.98 (★★) **A MANO** ¿Cuántas palabras de longitud n tiene $\{a, b\}^*$? ¿Y $\{ab\}^*$?

SOLUCIÓN La primera tiene 2^n . La segunda tiene **una** si n es par (la repetición $(ab)^{n/2}$) y **ninguna** si n es impar. Una clausura puede ser muy grande o muy pequeña según el lenguaje de partida.

8.5.99 (★★★) **A MANO** Escriba una gramática regular para $\{ab\}^*$.

SOLUCIÓN

$$S \rightarrow aA \mid \varepsilon, \quad A \rightarrow bS$$

La producción $S \rightarrow \varepsilon$ genera la hilera vacía y permite cerrar tras cualquier número de bloques ab . Los dos no terminales alternan, garantizando que las letras aparezcan de dos en dos y en el orden correcto.

8.5.100 (★★) **MIXTO** Compruebe la gramática anterior.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA\[CurlyEpsilon]", "A->bS"}, 6, 6]
```

Out[] =

```
{, ab, abab}
```

Genera la hilera vacía y las repeticiones de ab que caben en seis derivaciones. Ninguna palabra de longitud impar aparece, como corresponde.

8.5.101 (★★★) **A MANO** Diseñe una gramática regular para el lenguaje de las hileras sobre $\{0,1\}$ que representan números pares en binario.

SOLUCIÓN Un número binario es par si termina en 0:

$$S \rightarrow 0S \mid 1S \mid 0$$

Las dos primeras producciones dejan pasar cualquier prefijo y la tercera obliga a cerrar con un cero. La hilera 0 sola pertenece, y representa el cero.

8.5.102 (★★★) **MIXTO** Compruebe la gramática de los pares y cuente sus palabras de longitud tres.

SOLUCIÓN

In[] :=

```
{ElementosLenguaje[{"S->0S|1S|0"}, 4, 3],  
Length[ElementosLenguaje[{"S->0S|1S|0"}, 4, 3]]}
```

Out[] =

```
{{0, 00, 000, 010, 10, 100, 110}, 7}
```

De longitud tres hay cuatro: 000, 010, 100 y 110, la mitad de las ocho posibles. La lista incluye además las de longitudes uno y dos.

8.5.103 (★★) **A MANO** ¿Cuántas hileras binarias de longitud n representan números pares?

SOLUCIÓN Son 2^{n-1} : la última cifra queda fijada en 0 y las otras $n-1$ son libres. Es exactamente la mitad, coherente con la intuición de que la mitad de los números son pares.

8.5.104 (★★★) **A MANO** Escriba una gramática para el lenguaje de las hileras binarias sin ceros consecutivos.

SOLUCIÓN

$$S \rightarrow 1S \mid 0A \mid 1 \mid 0 \mid \varepsilon, \quad A \rightarrow 1S \mid 1$$

Tras un 0 es obligatorio un 1, lo que impide la secuencia 00. El número de palabras de longitud n sigue la recurrencia de *Fibonacci*, como se vio en el capítulo de recurrencias.

8.5.105 (★★★) **MIXTO** Compruebe la gramática anterior y el conteo de *Fibonacci*.

SOLUCIÓN

In[] :=

```
{ElementosLenguaje[{"S->1S|0A|1|0|\\[CurlyEpsilon]"}, "A->1S|1"}, 5, 3],
RT[{1, 1}, {2, 3}, 3]}
```

Out[] =

```
{1, 0, 01, 01, 010, 011, 1, 1, 10, 101, 11, 11, 110, 111},
{2, 3, 5}}
```

De longitud tres hay cinco (010, 011, 101, 110, 111), que coincide con el tercer término de la recurrencia $c_n = c_{n-1} + c_{n-2}$ con $c_1 = 2$ y $c_2 = 3$. Algunas palabras aparecen **repetidas** en la lista: la gramática es ambigua, porque 01 y 1 se derivan por dos caminos distintos.

8.5.106 (★★) **A MANO** ¿Qué relación hay entre el número de no terminales de una gramática regular y el número de estados del autómata equivalente?

SOLUCIÓN Coinciden, más un estado de aceptación añadido si hay producciones de la forma $\alpha \rightarrow x$. La correspondencia es directa: no terminal $\leftrightarrow$ estado, producción $\leftrightarrow$ transición. Por eso minimizar el autómata equivale a minimizar la gramática.

8.5.107 (★★★) **A MANO** Un lenguaje regular necesita al menos k estados. ¿Cuántos no terminales necesita como mínimo una gramática regular suya?

SOLUCIÓN Al menos $k - 1$, porque el estado de aceptación puede no corresponder a ningún no terminal. La cota inferior de estados se traslada casi directamente a la gramática: si el autómata mínimo necesita 2^n estados, ninguna gramática regular de ese lenguaje puede tener muchos menos no terminales.

8.5.108 (★★) **A MANO** Defina lenguaje formal y dé tres ejemplos sobre $\{a, b\}$ de naturaleza muy distinta.

SOLUCIÓN Un lenguaje formal sobre A es cualquier subconjunto de A^* . Tres ejemplos: el **finito** $\{a, ab, bb\}$; el **regular infinito** $\{a^n \mid n \geq 0\}$; y el **no regular** $\{a^n b^n\}$. Los tres son lenguajes formales; lo que cambia es qué maquinaria hace falta para reconocerlos.

8.5.109 (★★★) **A MANO** ¿Todo lenguaje finito es regular? Justifique.

SOLUCIÓN Sí. Un lenguaje finito con k palabras se reconoce con un autómata que tiene un camino por palabra, más una trampa de rechazo: el número de estados está acotado por la suma de las longitudes más uno. La finitud garantiza la memoria acotada.

8.5.110 (★★) **A MANO** ¿Y todo lenguaje regular es finito?

SOLUCIÓN No. $\{a\}^*$ es regular e infinito. La regularidad se refiere a la **memoria** que hace falta para reconocerlo, no a su tamaño. De hecho, la mayoría de los lenguajes regulares interesantes son infinitos.

8.5.111 (★★★) **A MANO** ¿Puede una gramática finita generar un lenguaje infinito? ¿Cómo?

SOLUCIÓN Sí, y es lo habitual: basta con que alguna producción sea **recursiva**, es decir que un no terminal se derive en una expresión que lo contenga. $S \rightarrow aS$ genera infinitas palabras con dos producciones. Sin recursión, el lenguaje generado es necesariamente finito.

8.5.112 (★★★) **A MANO** Dé un criterio para decidir si el lenguaje de una gramática es finito.

SOLUCIÓN Es finito si y solo si **ningún** no terminal alcanzable y *útil* se deriva en una expresión que lo vuelva a contener. Formalmente: se construye el grafo de dependencias entre no terminales y se comprueba que no tenga ciclos. Con ciclo, el lenguaje es infinito; sin ciclo, finito.

8.5.113 (★★) **MIXTO** Aplique el criterio a $S \rightarrow aA$, $A \rightarrow bB$, $B \rightarrow c$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA", "A->bB", "B->c"}, 5, 5]
```

Out[] =

```
{abc}
```

El grafo $S \rightarrow A \rightarrow B$ no tiene ciclos, de modo que el lenguaje es **finito**: contiene una única palabra, abc . Añadiendo $B \rightarrow cS$ aparecería un ciclo y el lenguaje pasaría a ser infinito.

8.5.114 (★★★) **MIXTO** Compruebe la afirmación anterior añadiendo la producción recursiva.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA", "A->bB", "B->c|cS"}, 8, 6]
```

Out[] =

```
{abc, abcabc}
```

Con la producción $B \rightarrow cS$ el lenguaje pasa a ser $\{(abc)^n \mid n \geq 1\}$, infinito. Basta cerrar un ciclo en el grafo de dependencias.

8.5.115 (★★) **A MANO** ¿Qué es un no terminal *inútil* y por qué conviene eliminarlo?

SOLUCIÓN Es el que no se alcanza desde el símbolo inicial, o el que no puede derivar en ninguna hilera de terminales. En ambos casos no contribuye al lenguaje. Eliminarlos simplifica la gramática sin cambiar $L(G)$, y es el primer paso de casi todos los algoritmos de normalización.

8.5.116 (★★★) **A MANO** En $S \rightarrow aA \mid b$, $A \rightarrow aA$, $C \rightarrow bb$, señale los no terminales inútiles.

SOLUCIÓN A es inútil porque no puede derivar en terminales: $A \rightarrow aA$ nunca termina. C es inútil porque no se alcanza desde S . Eliminando ambos, y con ellos la producción $S \rightarrow aA$, queda $S \rightarrow b$: el lenguaje es $\{b\}$.

8.5.117 (★★) **MIXTO** Compruebe que ese lenguaje es $\{b\}$.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA|b", "A->aA", "C->bb"}, 6, 5]
```

Out[] =

```
{b}
```

El comando devuelve una sola palabra, confirmando el análisis: las derivaciones que pasan por A nunca terminan y C jamás se alcanza.

8.5.118 (★★) **A MANO** Calcule $|A^3|$ y $|A^0|$ para $A = \{a, b, c\}$.

SOLUCIÓN $|A^3| = 27$ y $|A^0| = 1$: la única hilera de longitud cero es ε . No debe confundirse A^0 , que es $\{\varepsilon\}$, con el lenguaje vacío, que no tiene elementos.

8.5.119 (★★★) **A MANO** Demuestre que $|\alpha\beta| = |\alpha| + |\beta|$ y dé un ejemplo donde $\alpha\beta \neq \beta\alpha$.

SOLUCIÓN La concatenación escribe una hilera tras otra sin añadir ni quitar símbolos, de modo que las longitudes se suman. Con $\alpha = ab$ y $\beta = ba$: $\alpha\beta = abba$ y $\beta\alpha = baab$, distintas pese a tener la misma longitud. La operación es asociativa pero **no conmutativa**.

8.5.120 (★★) **A MANO** ¿Cuántos prefijos y cuántos sufijos tiene una hilera de longitud n ?

SOLUCIÓN Tiene $n + 1$ de cada tipo, contando la hilera vacía y la hilera completa. Para abc los prefijos son ε, a, ab, abc . Las *subhileras* son más: hasta $\frac{n(n+1)}{2} + 1$ distintas.

8.5.121 (★★★) **A MANO** ¿Cuántas subhileras distintas tiene aaa ? ¿Y abc ?

SOLUCIÓN aaa tiene cuatro: ε, a, aa, aaa . abc tiene siete: $\varepsilon, a, b, c, ab, bc, abc$. La cota $\frac{n(n+1)}{2} + 1$ se alcanza solo cuando **no hay repeticiones**: las letras repetidas producen subhileras iguales.

8.5.122 (★★) **A MANO** Si $L = \{a, ab\}$, calcule L^2 y su cardinal.

SOLUCIÓN $L^2 = \{aa, aab, aba, abab\}$, con cuatro elementos. Aquí se alcanza el máximo $2 \cdot 2 = 4$ porque todas las concatenaciones son distintas. Con $L = \{a, aa\}$ el cardinal bajaría a tres.

8.5.123 (★★★) **A MANO** Calcule $\{a, aa\}^2$ y explique por qué tiene tres elementos y no cuatro.

SOLUCIÓN Es $\{aa, aaa, aaaa\}$. Las parejas (a, aa) y (aa, a) producen la **misma** hilera aaa , y en un conjunto los elementos no se repiten. Es la misma razón por la que la ambigüedad de una gramática no cambia el lenguaje generado.

8.5.124 (★★) **A MANO** ¿Qué es L^+ y en qué se diferencia de L^* ?

SOLUCIÓN L^+ es la unión de las potencias con $k \geq 1$, es decir L^* sin el caso $k = 0$. Se cumple $L^* = L^+ \cup \{\varepsilon\}$, y ambos coinciden exactamente cuando $\varepsilon \in L$. Con $L = \{a\}$: L^+ no contiene ε y L^* sí.

8.5.125 (★★★) **A MANO** Describa con palabras el lenguaje $\{aa, b\}^*$ y dé sus palabras de longitud tres.

SOLUCIÓN Son las hileras que se pueden partir en bloques aa y b , es decir aquellas en las que las a aparecen siempre en **grupos de tamaño par**. De longitud tres: aab, baa y bbb . La hilera aba no pertenece, porque las a quedan sueltas.

8.5.126 (★★★) **MIXTO** Escriba una gramática regular para $\{aa, b\}^*$ y compruébela.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA|bS|[CurlyEpsilon]", "A->aS"}, 5, 3]
```

Out[] =

```
{, aa, b, bb}
```

La gramática es $S \rightarrow aA \mid bS \mid \varepsilon$, $A \rightarrow aS$: el no terminal A obliga a que cada a venga acompañada de otra. Los dos parámetros no significan lo mismo: el primero acota el **número**

de derivaciones y el segundo la longitud, de modo que con 5 derivaciones solo salen ε , aa , b y bb ; para ver aab y baa hay que subir el primer parámetro.

8.5.127 (★★) **A MANO** ¿Cuántas palabras de longitud n tiene $\{aa, b\}^*$?

SOLUCIÓN Sigue la recurrencia $c_n = c_{n-1} + c_{n-2}$ (una palabra termina en b , con prefijo de longitud $n-1$, o en aa , con prefijo de longitud $n-2$) con $c_0 = 1$ y $c_1 = 1$. Son los números de *Fibonacci*: 1, 1, 2, 3, 5, 8, ...

8.5.128 (★★★) **A MANO** Demuestre que la unión de dos lenguajes regulares es regular, sin usar autómatas.

SOLUCIÓN Si G_1 y G_2 son gramáticas regulares con símbolos iniciales S_1 y S_2 y **no terminales disjuntos**, basta añadir un nuevo inicial S con las producciones $S \rightarrow S_1$ y $S \rightarrow S_2$... salvo que esas no son regulares. La forma correcta es copiar las producciones de S_1 y de S_2 sobre S : si $S_1 \rightarrow x\beta$, se añade $S \rightarrow x\beta$. Así S genera la unión y la gramática sigue siendo regular.

8.5.129 (★★) **A MANO** ¿Por qué hace falta que los no terminales sean disjuntos en esa construcción?

SOLUCIÓN Porque si G_1 y G_2 comparten un no terminal A con producciones distintas, al unir las gramáticas ese A podría usarse con reglas de una y de otra en la misma derivación, generando palabras que no están en ninguno de los dos lenguajes. Renombrar es un paso **obligatorio**, no cosmético.

8.5.130 (★★★) **MIXTO** Construya la gramática de la unión de $\{a^n b\}$ y $\{b^n a\}$ y compruébela.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA|b|bB|a", "A->aA|b", "B->bB|a"}, 5, 4]
```

Out[] =

```
{a, aaab, aab, ab, b, ba, bba, bbba}
```

El nuevo inicial S copia las producciones de los dos iniciales originales. La lista mezcla palabras de los dos lenguajes: $a^n b$ por un lado y $b^n a$ por el otro, en perfecta simetría.

8.5.131 (★★★) **A MANO** ¿Es la intersección de dos lenguajes libres del contexto siempre libre del contexto?

SOLUCIÓN **No**. $\{a^n b^n c^m\}$ y $\{a^m b^n c^n\}$ son libres del contexto, y su intersección es $\{a^n b^n c^n\}$, que no lo es. Los lenguajes *regulares* sí son cerrados bajo intersección; los libres del contexto, no.

8.5.132 (★★) **A MANO** Resuma qué operaciones conservan la regularidad y cuáles conservan la libertad de contexto.

SOLUCIÓN Los **regulares** son cerrados bajo unión, intersección, complemento, concatenación y clausura. Los **libres del contexto** lo son bajo unión, concatenación y clausura, pero **no** bajo intersección ni complemento. La diferencia se explica porque una pila no se puede duplicar.

8.5.133 (★★★) **A MANO** Escriba una gramática libre del contexto para el lenguaje de los paréntesis bien balanceados.

SOLUCIÓN

$$S \rightarrow (S) \mid SS \mid \varepsilon$$

La primera producción anida, la segunda yuxtapone y la tercera cierra. La gramática es **ambigua** $((())())$ se deriva de varias formas) pero genera exactamente el lenguaje correcto.

8.5.134 (★★★) A MANO ¿Por qué el lenguaje de los paréntesis balanceados no es regular?

SOLUCIÓN Habría que recordar cuántos paréntesis quedan abiertos, un valor sin cota. Formalmente, las hileras $(, ((), (((, \dots$ deben quedar en estados distintos, y con un número finito de estados dos de ellas coinciden, tras lo cual el autómata no puede distinguir cuántos cierres faltan.

8.5.135 (★★) A MANO ¿Qué estructura de datos reconoce los paréntesis balanceados y cómo se relaciona con las gramáticas libres del contexto?

SOLUCIÓN Una **pila**: se apila con cada apertura y se desapila con cada cierre; la hilera es válida si la pila queda vacía y nunca se desapila de más. Los autómatas con pila reconocen exactamente los lenguajes libres del contexto, igual que los autómatas finitos reconocen los regulares.

8.5.136 (★★★) A MANO Diseñe una gramática para las hileras sobre $\{a, b\}$ con el doble de a que de b . ¿De qué tipo es?

SOLUCIÓN Una posible es

$$S \rightarrow SaSaSbS \mid SaSbSaS \mid SbSaSaS \mid \varepsilon$$

que intercala cada b con dos a en todas las posiciones relativas. Es **libre del contexto** y no regular: hay que llevar una cuenta sin cota.

8.5.137 (★★) A MANO ¿Cuántas palabras de longitud 6 tiene ese lenguaje?

SOLUCIÓN Debe haber cuatro a y dos b , de modo que son $\binom{6}{2} = 15$ palabras, una por cada elección de las posiciones de las b . La longitud debe ser **múltiplo de tres**: con longitud 5 no hay ninguna.

8.5.138 (★★★) A MANO Escriba una gramática regular para el lenguaje vacío y otra para $\{\varepsilon\}$.

SOLUCIÓN Para el **lenguaje vacío**: $S \rightarrow aS$ y nada más; ninguna derivación termina, de modo que no se genera ninguna palabra. Para $\{\varepsilon\}$: la única producción $S \rightarrow \varepsilon$. Los dos casos extremos se distinguen por si existe alguna derivación que llegue a terminales.

8.5.139 (★★) MIXTO Compruebe los dos casos anteriores.

SOLUCIÓN

In[] :=

```
{ElementosLenguaje[{"S->aS"}, 5, 5], ElementosLenguaje[{"S->[CurlyEpsilon]"}, 3, 3]}
```

Out[] =

```
{ {}, {} }
```

La primera lista es vacía: ninguna palabra. La segunda contiene un solo elemento, la hilera vacía, que se imprime como una lista con un hueco. La diferencia entre **ninguna palabra** y **la palabra vacía** es sutil pero esencial.

8.5.140 (★★★) A MANO ¿Cuántas gramáticas regulares distintas se pueden escribir con un solo no terminal S y alfabeto $\{a\}$?

SOLUCIÓN Las producciones posibles son $S \rightarrow a$, $S \rightarrow aS$ y $S \rightarrow \varepsilon$: tres, de las que se puede elegir cualquier subconjunto, es decir $2^3 = 8$ gramáticas. Generan solo cuatro lenguajes distintos: $\emptyset$, $\{\varepsilon\}$, $\{a\}^+$ y $\{a\}^*$.

8.5.141 (★★★) A MANO Justifique que esos cuatro son todos los lenguajes posibles en ese caso.

SOLUCIÓN Con un solo no terminal y un solo terminal, el lenguaje generado queda determinado por dos decisiones: si se admite la palabra vacía (producción $S \rightarrow \varepsilon$) y si se puede seguir generando: producción $S \rightarrow aS$ combinada con alguna que cierre. Dos decisiones binarias dan cuatro lenguajes.

8.5.142 (★★) A MANO Explique por qué dos gramáticas distintas pueden generar el mismo lenguaje.

SOLUCIÓN Porque el lenguaje es el **conjunto de palabras**, no el conjunto de reglas. $S \rightarrow aS \mid b$ y $S \rightarrow aA \mid b$, $A \rightarrow aS \mid b$ generan lenguajes distintos, pero $S \rightarrow aaS \mid b$ y su reescritura regular generan el mismo. Decidir si dos gramáticas son equivalentes es, en general, un problema **indecidible** para las libres del contexto.

8.5.143 (★★★) A MANO Para gramáticas *regulares*, cómo se decide si generan el mismo lenguaje?

SOLUCIÓN Se convierten en autómatas determinísticos mínimos y se comparan: dos lenguajes regulares son iguales si y solo si sus autómatas mínimos son idénticos salvo el nombre de los estados. A diferencia del caso libre del contexto, aquí el problema **sí** es decidable.

8.5.144 (★★) A MANO Resuma la correspondencia entre los tres tipos de gramática y su maquinaria de reconocimiento.

SOLUCIÓN **Regular**: autómata de estado finito, memoria acotada. **Libre del contexto**: autómata con pila, memoria en forma de pila. **Sensible al contexto**: autómata linealmente acotado, memoria proporcional a la entrada. Cada nivel de gramática corresponde exactamente a un tipo de máquina.

8.5.145 (★★★) MIXTO Cierre el capítulo comprobando el ciclo completo: gramática regular, autómata y de vuelta a gramática.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aS|b"}];
{StringAceptadaQ[G, {a, a, b}], StringAceptadaQ[G, {b, a}]}
```

Out[] =

```
{True, False}
```

In[] :=

```
Ac = Automata[{S, A}, {a, b}, S,
{{S, a, S}, {S, b, A}, {A, a, A}, {A, b, A}}, {A}];
AutomataToRegular[Ac, componentes -> True]
```

Out[] =

```
{{S, A}, {a, b}, {A->a, A->aA, A->b, A->bA, S->aS, S->b, S->bA}, S}
```

La gramática final no es idéntica a la inicial: tiene un no terminal más y siete producciones en lugar de dos. La razón es que A es estado de aceptación, así que cada transición que sale

de él genera **dos** producciones, una que continúa ($A \rightarrow aA$) y otra que cierra ($A \rightarrow a$). El lenguaje generado sí es el mismo: $a^n b \{a, b\}^*$.

8.4 Clasificar, reducir y generar gramáticas

8.5.146 (★★) **MIXTO** Decida con `GramaticaQ` si dos objetos son gramáticas.

SOLUCIÓN

`In[ ] :=`

```
{GramaticaQ[{"S->aS|b"}], GramaticaQ[{"S", "aa"}]}
```

`Out[ ] =`

```
{True, False}
```

Una gramática es un vector de *strings* de la forma "A->B". El segundo objeto no tiene flechas: son dos hileras sueltas, no producciones. Este comando sirve como **primer filtro** antes de pasar una gramática a cualquier otra instrucción: si no lo es, los demás comandos fallan en silencio.

8.5.147 (★★★) **VilCretas** Clasifique con `TipoGramatica` la gramática $S \rightarrow aS \mid b$ y explique el resultado.

SOLUCIÓN

`In[ ] :=`

```
TipoGramatica[{"S->aS|b"}]
```

Responde que sí a las tres preguntas: sensible al contexto, libre del contexto **y** regular. No es un error: la jerarquía de *Chomsky* está **anidada**. Toda gramática regular es libre del contexto, y toda libre del contexto es sensible al contexto. Lo informativo es la respuesta **más restrictiva** que sale `True`.

8.5.148 (★★★) **VilCretas** Clasifique ahora $S \rightarrow aSb \mid ab$ y la gramática con la producción $aSb \rightarrow aab$.

SOLUCIÓN

`In[ ] :=`

```
{TipoGramatica[{"S->aSb|ab"}],  
TipoGramatica[{"aSb->aab", "S->b"}]}
```

La primera es libre del contexto pero **no regular**: el lado derecho aSb tiene el no terminal en medio, y una gramática regular lo exige en un extremo. La segunda no es ni siquiera libre del contexto: su lado izquierdo tiene más de un símbolo, que es precisamente lo que significa **sensible al contexto**: la sustitución solo vale en cierto entorno.

8.5.149 (★★★) **A MANO** ¿Por qué $S \rightarrow aSb \mid ab$ genera un lenguaje que ninguna gramática regular puede generar?

SOLUCIÓN Genera $\{a^n b^n : n \geq 1\}$, y ese lenguaje **no es regular**: haría falta recordar cuántas a se leyeron, un valor sin cota, y un autómata finito tiene memoria acotada. Con un número finito de estados, dos prefijos a^i y a^j distintos acaban en el mismo estado, y a partir de ahí la máquina ya no puede distinguirlos.

8.5.150 (★★) **MIXTO** Use `GramaticaSensibleContextoQ` y compare su respuesta con la de `TipoGramatica`.

SOLUCIÓN

In[] :=

```
{GramaticaSensibleContextoQ[{"S->aS|b"}],
GramaticaSensibleContextoQ[{"aSb->aab", "S->b"}]}
```

Out[] =

```
{True, True}
```

Las dos son sensibles al contexto, la primera por ser regular (y por tanto también de los dos tipos superiores) y la segunda por serlo en sentido estricto. El comando responde a **una sola** de las tres preguntas que responde `TipoGramatica`, y devuelve un valor lógico que sí se puede encadenar.

8.5.151 (★★) **MIXTO** Compacte con `ReduceReglas` un conjunto de producciones escritas una a una.

SOLUCIÓN

In[] :=

```
ReduceReglas[{"S->aS", "S->b", "A->a", "A->bA"}]
```

Out[] =

```
{S->aS|b, A->a|bA}
```

Agrupar en una sola línea todas las producciones que empiezan por el mismo no terminal, separadas por la barra. Es la forma **reducida**, la que esperan `NotationBNF` y `NotationFNormal`, y la que se usa al escribir una gramática a mano.

8.5.152 (★★) **MIXTO** Pase esa gramática a notación de *Backus-Naur* con `NotationBNF`.

SOLUCIÓN

In[] :=

```
NotationBNF[{"S->aS|b"}, componentes -> True]
```

Out[] =

```
{<S>::=a<S>|b, S}
```

En *BNF* los no terminales van entre ángulos y la flecha se escribe `::=`. Con `componentes -> True` el comando devuelve además el símbolo inicial, y todo como **datos**: sin la opción solo lo escribe en pantalla.

8.5.153 (★★★) **MIXTO** Obtenga las cuatro componentes formales de $S \rightarrow aSb \mid ab$ con `NotationFNormal`.

SOLUCIÓN

In[] :=

```
NotationFNormal[{"S->aSb|ab"}, componentes -> True]
```

Out[] =

$\{\{S\}, \{a, b\}, \{S \rightarrow ab, S \rightarrow aSb\}, S\}$

Son las cuatro de la definición: **no terminales**, **terminales**, **producciones** y **símbolo inicial**. Obtenerlas por separado permite comprobar cosas que a simple vista se escapan, como que ningún símbolo aparezca en las dos primeras listas a la vez.

- 8.5.154 (★★★) **VilCretas** Genere gramáticas de cada tipo con `GramaticaRandom` y clasifíquelas en el acto.

SOLUCIÓN

In[] :=

```
{GramaticaRandom[4, 4, regular -> True],
GramaticaRandom[4, 4, libre -> True],
GramaticaRandom[4, 4, sensible -> True, valorn -> 3]}
```

Los dos primeros parámetros son el número de no terminales y de terminales. Sin opción de tipo, el comando escoge uno al azar: útil para practicar la clasificación a ciegas. `valorn` acota la longitud del contexto en las reglas sensibles.

- 8.5.155 (★★★) **VilCretas** Compruebe que lo generado es del tipo pedido, encadenando con `TipoGramatica`.

SOLUCIÓN

In[] :=

```
g = GramaticaRandom[3, 3, libre -> True];
{GramaticaQ[g], TipoGramatica[g]}
```

Encadenar generador y clasificador es la forma de **comprobar el generador**, no solo de usarlo. Y sirve de ejercicio: primero se clasifica a mano mirando los lados izquierdos y derechos, y solo después se contrasta con el comando.

- 8.5.156 (★★★) **VilCretas** Obtenga la gramática regular de un autómata generado al azar, en las dos notaciones que ofrece `AutomataToRegular`.

SOLUCIÓN

In[] :=

```
AutomataDRandom[4, 2];
comp = ComponentesAutomata[G, conjuntos -> True];
aut = Automata[comp[[1]], comp[[2]], comp[[3]], comp[[4]], comp[[5]]];
{AutomataToRegular[aut, componentes -> True],
AutomataToRegular[aut, bnf -> True]}
```

`componentes -> True` devuelve la gramática como datos y `bnf -> True` la escribe en notación de *Backus-Naur*. El encadenado completo (generar autómata, extraer componentes, reconstruirlo y traducirlo a gramática) recorre de un tirón la equivalencia entre autómatas finitos y gramáticas regulares.

CUARTA PARTE

Preguntas tipo examen

CAPÍTULO 1 • PREGUNTAS TIPO EXAMEN

Recursividad

1.7.1 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-Rec. Prog. de pila

Complete el siguiente código, que calcula $f(n) = 3^n$ mediante una recursividad de pila:

```
f[n_] := If[n == ____(i), ____(ii), ____(iii)*f[____(iv)]]
```

SOLUCIÓN El caso raíz es $n = 0$, donde $3^0 = 1$; el paso recursivo multiplica por 3 el resultado para $n - 1$. Los huecos son (i) 0, (ii) 1, (iii) 3 y (iv) $n - 1$:

```
In[ ] :=
```

```
f[n_] := If[n == 0, 1, 3*f[n - 1]];
f[5]
```

```
Out[ ] =
```

```
243
```

En efecto $3^5 = 243$.

1.7.2 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-Rec. Prog. de sumatorias con Rec. de pila

Complete el código siguiente para calcular $\sum_{i=3}^{n-1} (3i + 2)$ mediante una recursividad de pila:

```
f[n_] := If[n == ____(i), ____(ii),
  f[____(iii)] + ((3 i + 2) /. i -> ____ (iv))]
```

SOLUCIÓN La suma se detiene cuando el límite superior $n - 1$ alcanza el inferior 3, es decir cuando $n = 4$; en ese caso vale $3 \cdot 3 + 2 = 11$. Cada paso agrega el sumando evaluado en $i = n - 1$. Los huecos son (i) 4, (ii) 11, (iii) $n - 1$ y (iv) $n - 1$:

```
In[ ] :=
```

```
f[n_] := If[n == 4, 11, f[n - 1] + ((3 i + 2) /. i -> n - 1)];
f[6]
```

```
Out[ ] =
```

```
42
```

Comprobación directa: $11 + 14 + 17 = 42$.

1.7.3 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-Rec. Prog. de sumatorias con Rec. de pila

Para la recursividad del ejercicio anterior, calcule $f(10)$ y compruébelo con el comando **Sumatoria** del paquete **VilCretas**.

SOLUCIÓN

```
In[ ] :=
```

```
f[n_] := If[n == 4, 11, f[n - 1] + ((3 i + 2) /. i -> n - 1)];
f[10]
```

Out[] =

140

In[] :=

```
Sumatoria[{3, n, 3 i + 2}, 9]
```

Out[] =

140

Sumatoria recibe el límite inferior, la variable del límite superior y el término general; el segundo argumento es el valor en que se evalúa. Ambos caminos dan 140.

1.7.4 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 3-Rec. Prog. de productorias con Rec. de pila

Complete el código siguiente para calcular $\prod_{i=2}^{n-3} \left(1 + \frac{2}{i}\right)$ mediante una recursividad de pila:

```
f[n_] := If[n == ____(i), ____ (ii),
          f[____(iii)]*((1 + 2/i) /. i -> ____ (iv))]
```

SOLUCIÓN El límite superior $n - 3$ alcanza al inferior 2 cuando $n = 5$, y ahí el producto vale $1 + \frac{2}{2} = 2$. Los huecos son (i) 5, (ii) 2, (iii) $n - 1$ y (iv) $n - 3$:

In[] :=

```
f[n_] := If[n == 5, 2, f[n - 1]*((1 + 2/i) /. i -> n - 3)];
f[8]
```

Out[] =

7

Comprobación: $2 \cdot \frac{5}{3} \cdot \frac{3}{2} \cdot \frac{7}{5} = 7$.

1.7.5 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 3-Rec. Prog. de productorias con Rec. de pila

Compruebe la productoria anterior con **Productoria**.

SOLUCIÓN

In[] :=

```
Productoria[{2, n, 1 + 2/i}, 5]
```

Out[] =

7

El comando despliega además la recursividad que construye y el desarrollo paso a paso, lo que permite contrastarlo con el código escrito a mano.

1.7.6 (★★) TIPO EXAMEN A MANO

Categoría del banco: 5-Rec. Prog. de cola

Complete el siguiente código, que calcula $f(n) = 3^n$ mediante una recursividad de cola:

```
f[n_] := Module[{g}, g[i_, p_] := If[i == ____, ____,  
    g[i + 1, ____]]; g[____, 1]]
```

SOLUCIÓN El acumulador p arranca en 1 con $i = 0$ y se multiplica por 3 en cada paso; el proceso se detiene cuando i alcanza n y devuelve el acumulador. Los huecos son (i) n , (ii) p , (iii) $3 * p$ y (iv) 0:

In[] :=

```
f[n_] := Module[{g}, g[i_, p_] := If[i == n, p, g[i + 1, 3*p]]; g[0, 1];  
f[5]
```

Out[] =

243

A diferencia de la versión de pila, aquí no queda ninguna operación pendiente al regresar de la llamada.

1.7.7

(★★★)

TIPO EXAMEN

A MANO

Categoría del banco: 6-Rec. Prog. de sumatorias con Rec. de cola

Complete el código necesario para calcular $\sum_{i=7}^{n+2} (i^2 - 1)$ mediante una recursividad de cola:

```
f[n_] := Module[{g}, g[i_, s_] := If[i == n + ____, ____,  
    g[i + 1, s + ____]]; g[____, 0]]
```

SOLUCIÓN El índice recorre de 7 hasta $n+2$, de modo que el ciclo se detiene cuando $i = n+3$; el acumulador arranca en 0 y suma $i^2 - 1$ en cada paso. Los huecos son (i) 3, (ii) s , (iii) $i^2 - 1$ y (iv) 7:

In[] :=

```
f[n_] := Module[{g}, g[i_, s_] := If[i == n + 3, s,  
    g[i + 1, s + i^2 - 1]]; g[7, 0];  
f[8]
```

Out[] =

290

Comprobación: $48 + 63 + 80 + 99 = 290$.

1.7.8

(★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 6-Rec. Prog. de sumatorias con Rec. de cola

Para la misma sumatoria, calcule $f(5)$ y explique el resultado.

SOLUCIÓN

In[] :=

```
f[n_] := Module[{g}, g[i_, s_] := If[i == n + 3, s,  
    g[i + 1, s + i^2 - 1]]; g[7, 0];  
f[5]
```

Out[] =

48

Con $n = 5$ el límite superior es 7, igual al inferior, de modo que la suma tiene un solo sumando: $7^2 - 1 = 48$.

1.7.9 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-Rec. Problemas de desarrollo con Rec. de pila

Escriba un programa recursivo de pila que calcule $S = \sum_{i=1}^{n+1} i \left(\prod_{j=2}^{i+1} (2j+1) \right)$. Siga estos

pasos: (a) construya una recursividad de pila f_1 que calcule $\prod_{j=2}^{n+1} (2j+1)$, cuidando que valga

1 cuando el límite superior sea menor que el inferior; (b) use f_1 para escribir una recursividad de pila f_2 que calcule S . Compruebe que $f_1(2) = 35$ y que $f_2(3) = 14\,880$.

SOLUCIÓN (a) El producto empieza en $j = 2$, de modo que el caso raíz se alcanza cuando $n + 1 = 2$, es decir $n = 1$, y ahí vale $2 \cdot 2 + 1 = 5$. Para $n < 1$ el límite superior queda por debajo del inferior y el producto vacío vale 1:

In[] :=

```
f1[n_] := Which[n < 1, 1, n == 1, 5, n > 1, f1[n - 1]*((2 j + 1) /. j -> n + 1)];
f1[2]
```

Out[] =

35

(b) La suma va de $i = 1$ a $i = n + 1$; el caso raíz es $n = 0$, donde vale $1 \cdot f_1(1) = 5$:

In[] :=

```
f1[n_] := Which[n < 1, 1, n == 1, 5, n > 1, f1[n - 1]*((2 j + 1) /. j -> n + 1)];
f2[n_] := If[n == 0, 5, f2[n - 1] + (n + 1)*f1[n + 1]];
f2[3]
```

Out[] =

14880

Los dos valores de control coinciden: $f_1(2) = 5 \cdot 7 = 35$ y $f_2(3) = 5 + 2 \cdot 35 + 3 \cdot 315 + 4 \cdot 3465 = 14\,880$.

1.7.10 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rec. Problemas de desarrollo con Rec. de cola

Resuelva con recursividad de cola: la población de una ciudad en 2026 es de 5 millones de habitantes y crece un 2% anual. (a) ¿Cuál será la población en 2035? (b) Construya la tabla de los diez primeros años.

SOLUCIÓN (a) Cada año la población se multiplica por 1,02. Con una recursividad de cola, el acumulador guarda la población y el contador los años transcurridos:

In[] :=

```
pob[n_] := Module[{g}, g[i_, p_] := If[i == n, p, g[i + 1, 1.02*p]];
  g[0, 5000000]];
Round[pob[9]]
```

Out[] =

5975463

En 2035 han pasado nueve años, de modo que la población será de unos 5 975 463 habitantes.

(b) La tabla completa se obtiene con RT, que genera los términos de la relación de recurrencia $a_n = 1,02 a_{n-1}$:

In[] :=

```
Round[RT[{1.02}, {5000000}, 10]]
```

Out[] =

```
{5000000, 5100000, 5202000, 5306040, 5412161, 5520404,
5630812, 5743428, 5858297, 5975463}
```

El primer valor corresponde a 2026 y el último a 2035.

1.7.11 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rec. Prog. de pila / 5-Rec. Prog. de cola

Compare la versión de pila y la de cola del factorial que trae el paquete y verifique que coinciden.

SOLUCIÓN

In[] :=

```
{Factoriales[10], FactorialesCola[10]}
```

Out[] =

```
{3628800, 3628800}
```

Las dos devuelven $10! = 3\,628\,800$. La diferencia no está en el resultado sino en el consumo de pila: la versión de cola no deja multiplicaciones pendientes. Con PruebaADA2 puede compararse su tiempo de ejecución.

1.7.12 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-Rec. Prog. de sumatorias con Rec. de pila

Complete el código siguiente para calcular $\sum_{i=4}^{n-1} (i^2 - 2i)$ mediante una recursividad de pila:

```
s[n_] := If[n == ____(i), ____ (ii),
  s[____ (iii)] + ((i^2 - 2 i) /. i -> ____ (iv))]
```

SOLUCIÓN El límite superior $n - 1$ alcanza al inferior 4 cuando $n = 5$, y ahí el sumando vale $4^2 - 2 \cdot 4 = 8$. Los huecos son (i) 5, (ii) 8, (iii) $n - 1$ y (iv) $n - 1$:

In[] :=

```
s[n_] := If[n == 5, 8, s[n - 1] + ((i^2 - 2 i) /. i -> n - 1)];
{s[9], Sum[i^2 - 2 i, {i, 4, 8}]}
```

Out[] =

{130, 130}

1.7.13 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 3-Rec. Prog. de productorias con Rec. de pila

Complete el código siguiente para calcular $\prod_{i=3}^{n-2} \left(1 + \frac{3}{i}\right)$ mediante una recursividad de pila:

```
q[n_] := If[n == ____(i), ____(ii),
  q[____(iii)]*((1 + 3/i) /. i -> ____ (iv))]
```

SOLUCIÓN El límite superior $n-2$ alcanza al inferior 3 cuando $n = 5$, y el factor vale $1 + \frac{3}{3} = 2$. Los huecos son (i) 5, (ii) 2, (iii) $n - 1$ y (iv) $n - 2$:

In[] :=

```
q[n_] := If[n == 5, 2, q[n - 1]*((1 + 3/i) /. i -> n - 2)];
{q[9], Product[1 + 3/i, {i, 3, 7}]}
```

Out[] =

{12, 12}

El producto es telescópico: $\prod \frac{i+3}{i}$ deja $\frac{(n-2)(n-1)n}{3 \cdot 4 \cdot 5}$ tras cancelar.

1.7.14 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 6-Rec. Prog. de sumatorias con Rec. de cola

Complete el código necesario para calcular $\sum_{i=6}^{n+4} (2i + 3)$ mediante una recursividad de cola:

```
s[n_] := Module[{g}, g[i_, t_] := If[i == n + ____(i), ____(ii),
  g[i + 1, t + ____ (iii)]]; g[____(iv), 0]]
```

SOLUCIÓN El índice va de 6 a $n + 4$, de modo que el ciclo se detiene en $i = n + 5$. Los huecos son (i) 5, (ii) t , (iii) $2i + 3$ y (iv) 6:

In[] :=

```
s[n_] := Module[{g}, g[i_, t_] := If[i == n + 5, t, g[i + 1, t + 2 i + 3]];
  g[6, 0]];
{s[4], s[10]}
```

Out[] =

{51, 207}

Comprobación directa: $\sum_{i=6}^8 (2i + 3) = 15 + 17 + 19 = 51$.

1.7.15 (★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 5-Rec. Prog. de cola

Complete el siguiente código, que calcula $f(n) = 5^n$ mediante una recursividad de cola:

```
f[n_] := Module[{g}, g[i_, a_] := If[i == ____(i), ____(ii),
  g[i + 1, ____ (iii)]]; g[____(iv), 1]]
```

SOLUCIÓN Los huecos son (i) n , (ii) a , (iii) $5 * a$ y (iv) 0:

In[] :=

```
f[n_] := Module[{g}, g[i_, a_] := If[i == n, a, g[i + 1, 5 a]]; g[0, 1]];
{f[7], 5^7}
```

Out[] =

```
{78125, 78125}
```

1.7.16 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-Rec. Problemas de desarrollo con Rec. de pila

Escriba un programa recursivo de pila que calcule $S = \sum_{i=1}^{n+2} i \left(\prod_{j=1}^i (4j - 2) \right)$. (a) Construya

f_1 para la productoria, cuidando que valga 1 cuando el límite superior sea menor que el inferior. (b) Use f_1 para escribir f_2 . Compruebe que $f_1(3) = 120$ y que $f_2(4) = 7106$.

SOLUCIÓN (a) El producto empieza en $j = 1$, de modo que el caso raíz es $n = 1$ con valor $4 \cdot 1 - 2 = 2$, y para $n < 1$ el producto vacío vale 1:

In[] :=

```
g1[n_] := Which[n < 1, 1, n == 1, 2, n > 1, g1[n - 1]*((4 j - 2) /. j -> n)];
{g1[3], g1[5]}
```

Out[] =

```
{120, 30240}
```

(b) La suma va de $i = 1$ a $i = n + 2$; se toma como caso raíz $n = 0$, que suma hasta $i = 2$:

In[] :=

```
g1[n_] := Which[n < 1, 1, n == 1, 2, n > 1, g1[n - 1]*((4 j - 2) /. j -> n)];
g2[n_] := If[n == 0, 0, g2[n - 1] + n*g1[n]];
g2[4]
```

Out[] =

```
7106
```

Los dos valores de control coinciden: $f_1(3) = 2 \cdot 6 \cdot 10 = 120$ y $f_2(4) = 2 + 2 \cdot 12 + 3 \cdot 120 + 4 \cdot 1680 = 7106$.

1.7.17 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rec. Problemas de desarrollo con Rec. de cola

Resuelva con recursividad de cola: un capital de \$250 000 se invierte al 7,5 % anual compues-
to. (a) ¿Cuál es el capital al cabo de siete años? (b) Construya la tabla de los ocho primeros
valores, redondeados a la unidad.

SOLUCIÓN (a) Cada año el capital se multiplica por 1,075:

In[] :=

```
cap[n_] := Module[{g}, g[i_, c_] := If[i == n, c, g[i + 1, 1.075*c]];
g[0, 250000]];
```

```
Round[cap[7]]
```

Out[] =

414762

Al cabo de siete años el capital es de unos \$414 762. (b)

In[] :=

```
Round[RT[{1.075}, {250000}, 8]]
```

Out[] =

```
{250000, 268750, 288906, 310574, 333867, 358907, 385825, 414762}
```

El primer valor es el capital inicial y el último corresponde al séptimo año.

1.7.18 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rec. Prog. de pila

Evalúe `PotenciaPos[7, 8]` e indique cuántas llamadas recursivas realiza.

SOLUCIÓN

In[] :=

```
PotenciaPos[7, 8]
```

Out[] =

5764801

El resultado es $7^8 = 5\,764\,801$ y hace 8 llamadas, una por unidad del exponente. La potenciación rápida haría solo 4: la diferencia entre $\Theta(n)$ y $\Theta(\ln n)$.

1.7.19 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 2-Rec. Prog. de sumatorias con Rec. de pila

Calcule $\sum_{i=3}^{10} (4i + 1)$ con `Sumatoria`.

SOLUCIÓN

In[] :=

```
Sumatoria[{3, n, 4 i + 1}, 10]
```

Out[] =

216

El comando construye la recursividad, la despliega paso a paso y devuelve 216. A mano: son ocho sumandos en progresión aritmética, de 13 a 41, con suma $\frac{8(13+41)}{2} = 216$.

1.7.20 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 3-Rec. Prog. de productorias con Rec. de pila

Calcule $\prod_{i=1}^8 \frac{i+2}{i}$ con `Productoria` y explique el resultado.

SOLUCIÓN

In[] :=


```
Productoria[{1, n, (i + 2)/i}, 8]
```

Out[] =

45

Es un producto **telescópico**: sobreviven los numeradores 9 y 10 y los denominadores 1 y 2, de modo que vale $\frac{9 \cdot 10}{1 \cdot 2} = 45$.

1.7.21 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-Rec. Prog. de pila / 5-Rec. Prog. de cola

Considere los dos métodos siguientes, que calculan lo mismo:

```
f1[n_] := If[n == 1, 1, n*f1[n - 1]]
```

```
f2[n_] := Module[{g}, g[i_, a_] := If[i > n, a, g[i + 1, a*i]]; g[1, 1]]
```

(a) El valor para $n = 25$ es _____. (i). (b) El que usa recursividad de cola es _____. (ii). (c) ¿Cuál agotaría antes la pila del sistema?

SOLUCIÓN (a) Los dos calculan $25!$:

In[] :=

```
FactorialesCola[25]
```

Out[] =

15511210043330985984000000

(b) Es `f2`: su llamada recursiva es la última operación y el acumulador `a` ya lleva el producto parcial. (c) `f1`, la de pila: guarda un marco por nivel y deja pendiente una multiplicación en cada uno. Con n del orden de mil el sistema alcanza su límite de recursión.

1.7.22 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 1-Rec. Prog. de pila

El comando `Divisores` recorre de forma recursiva todos los enteros menores o iguales que n . Sabiendo que el límite de recursión de *Mathematica* es 1024, hasta qué valores de n puede usarse?

SOLUCIÓN Hasta $n \approx 1020$. El comando hace una llamada por cada entero, de modo que la profundidad de la pila crece **linealmente** con n ; al pasar de mil llamadas el sistema interrumpe la evaluación con `$RecursionLimit::reclim`. Para n mayores hay que usar una versión iterativa o la instrucción `Divisors` de *Mathematica*. Es un recordatorio de que la elegancia de una recursividad de pila tiene un costo concreto.

1.7.23 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rec. Prog. de pila

Liste los divisores de 1000 e indique cuántos son.

SOLUCIÓN

In[] :=

```
{Divisores[1000], Length[Divisores[1000]]}
```

Out[] =

{1, 2, 4, 5, 8, 10, 20, 25, 40, 50, 100, 125, 200, 250, 500, 1000}, 16}

Son 16. Como $1000 = 2^3 \cdot 5^3$, la fórmula da $(3 + 1)(3 + 1) = 16$ sin necesidad de recorrer nada.

1.7.24 (★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 1-Rec. Prog. de pila

Calcule la suma de los dígitos de 12 345 678 901 234 567 890.

SOLUCIÓN

In[] :=

SumaDigi[12345678901234567890]

Out[] =

90

Son 20 cifras y la suma es 90. La recursividad hace 20 llamadas, una por cifra: el coste es $\Theta(\ln n)$ respecto del valor del número, no $\Theta(n)$.

1.7.25 (★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 1-Rec. Prog. de pila

Determine el máximo común divisor de 8633 y 3094 e interprete el resultado.

SOLUCIÓN

In[] :=

MCD[8633, 3094]

Out[] =

1

Vale 1: los dos números son **primos relativos**, aunque ninguno de los dos sea primo ($8633 = 89 \cdot 97$ y $3094 = 2 \cdot 7 \cdot 13 \cdot 17$). El algoritmo de *Euclides* llega a este resultado en pocas llamadas sin factorizar nada.

1.7.26 (★★) **TIPO EXAMEN** **VilCretas**

Categoría del banco: 1-Rec. Prog. de pila

Ordene la lista {5, 12, 3, 9, 1} de forma descendente con **Quicksort**.

SOLUCIÓN

In[] :=

Quicksort[{5, 12, 3, 9, 1}, ascendente -> False]

Out[] =

{12, 9, 5, 3, 1}

La opción **ascendente->False** invierte el criterio de comparación. La estructura recursiva no cambia: sigue partiendo por un pivote y llamando sobre las dos sublistas.

1.7.27 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 4-Rec. Problemas de desarrollo con Rec. de pila

Una recursividad de pila calcula $P(n) = \prod_{i=1}^n \frac{2i}{2i-1}$. Escriba su definición base, su caso raíz, y diga cuántas llamadas hace para $n = 50$.

SOLUCIÓN El caso raíz es $P(1) = \frac{2}{1} = 2$ y la definición base $P(n) = P(n-1) \cdot \frac{2n}{2n-1}$. Para $n = 50$ hace 50 llamadas, una por factor. El producto tiende a infinito muy despacio: está relacionado con la fórmula de *Wallis* para π .

1.7.28 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rec. Problemas de desarrollo con Rec. de cola

Una población de bacterias se duplica cada tres horas y parte de 400 individuos. (a) Escriba una recursividad de cola que dé la población tras n períodos de tres horas. (b) ¿Cuántas hay al cabo de un día?

SOLUCIÓN (a) El acumulador se duplica en cada período:

In[] :=

```
bac[n_] := Module[{g}, g[i_, c_] := If[i == n, c, g[i + 1, 2 c]]; g[0, 400]];
Table[bac[k], {k, 0, 8}]
```

Out[] =

```
{400, 800, 1600, 3200, 6400, 12800, 25600, 51200, 102400}
```

(b) Un día son ocho períodos de tres horas, de modo que hay $400 \cdot 2^8 = 102\,400$ bacterias.

1.7.29 (★★) TIPO EXAMEN A MANO

Categoría del banco: 5-Rec. Prog. de cola

En una recursividad de cola con acumulador, ¿con qué valor debe inicializarse el acumulador y por qué?

SOLUCIÓN Con el **elemento neutro** de la operación que se acumula: 0 para una suma, 1 para un producto, la cadena vacía para una concatenación. Así el primer paso no altera el resultado. Cuando la operación no tiene neutro (el máximo de una lista, por ejemplo) hay que inicializar con el **primer elemento**, y la función deja de estar definida sobre la lista vacía.

1.7.30 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 6-Rec. Prog. de sumatorias con Rec. de cola

En el código de cola de una sumatoria, un estudiante escribe `If[i == n, s, ...]` cuando el límite superior es n . ¿Qué error comete y cómo se detecta?

SOLUCIÓN Se detiene **antes** de sumar el término $i = n$, de modo que devuelve la suma hasta $n - 1$. Es el error de desplazamiento por uno, y no produce ningún mensaje: el programa entrega un número equivocado. Se detecta comparando con un caso pequeño calculado a mano, o con `Sum`. La condición correcta es $i > n$.

1.7.31 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-Rec. Prog. de sumatorias con Rec. de pila / 6-Rec. de cola

Programe $\sum_{i=1}^n i(i+1)$ en pila y en cola, compruebe que coinciden para $n = 12$ y diga cuál conviene para $n = 5000$.

SOLUCIÓN**In[] :=**

```
sp[n_] := If[n == 1, 2, sp[n - 1] + n*(n + 1)];  
sc[n_] := Module[{g}, g[i_, t_] := If[i > n, t, g[i + 1, t + i*(i + 1)]]; g[1, 0];  
{sp[12], sc[12], Sum[i (i + 1), {i, 1, 12}]}
```

Out[] =

```
{728, 728, 728}
```

Los tres valores coinciden. Para $n = 5000$ conviene la **de cola**: la de pila necesitaría 5000 marcos anidados y superaría el límite de recursión del sistema, que está en 1024.

CAPÍTULO 2 • PREGUNTAS TIPO EXAMEN

Relaciones de recurrencia**2.5.1** (★★) **TIPO EXAMEN** VilCretas

Categoría del banco: 1-Rr. Evaluación de recurrencias

Al calcular la suma de los diez primeros términos de la sucesión recurrente $a_n = a_{n-1} + 2a_{n-2} + n$, con $a_1 = 1$ y $a_2 = 2$, se obtiene _____.(i). *Ayuda:* el comando **Total** suma los valores de una lista.

SOLUCIÓN**In[] :=****RT[{1, 2, n}, {1, 2}, 10]****Out[] =**

{1, 2, 7, 15, 34, 70, 145, 293, 592, 1188}

In[] :=**Total[RT[{1, 2, n}, {1, 2}, 10]]****Out[] =**

2347

El hueco vale 2347. En RT el vector de coeficientes va en orden descendente y la parte no homogénea, aquí n , ocupa la última componente.

2.5.2 (★★) **TIPO EXAMEN** VilCretas

Categoría del banco: 2-Rr. Hallar recurrencia dada una lista

Al buscar una recurrencia para la sucesión $\{3, 7, 17, 41, 99, 239\}$ se obtiene $a_n = \text{____}(i) a_{n-1} + \text{____}(ii) a_{n-2}$, con $a_1 = \text{____}(iii)$ y $a_2 = \text{____}(iv)$.

SOLUCIÓN**In[] :=****FindRRHL[{3, 7, 17, 41, 99, 239}, a, n]**

La salida entrega $a[n] == a[-2 + n] + 2*a[-1 + n]$, con $a[1] == 3$ y $a[2] == 7$, además de una fórmula explícita que en este caso involucra $\sqrt{2}$. Los huecos son (i) 2, (ii) 1, (iii) 3 y (iv) 7. Comprobación: $17 = 2 \cdot 7 + 3$ y $41 = 2 \cdot 17 + 7$.

2.5.3 (★★) **TIPO EXAMEN** MIXTO

Categoría del banco: 3-Rr. Gráfica de una recurrencia

Los primeros ocho términos de cierta recurrencia son $\{1, 2, 7, 20, 61, 182, 547, 1640\}$. Indique a cuál de estas relaciones corresponden: (a) $a_n = 3a_{n-1}$; (b) $a_n = 2a_{n-1} + 3a_{n-2}$; (c) $a_n = a_{n-1} + a_{n-2}$.

SOLUCIÓN Es la opción (b). El crecimiento es casi triple en cada paso, lo que descarta (c), y no es exactamente triple, lo que descarta (a):

In[] :=

```
RT[{2, 3}, {1, 2}, 8]
```

```
Out[ ] =
```

```
{1, 2, 7, 20, 61, 182, 547, 1640}
```

```
In[ ] :=
```

```
RR[{2, 3}, {1, 2}, n, inicio -> 0]
```

```
Out[ ] =
```

```
((-1)^n + 3^(1 + n))/4
```

La fórmula explícita $a_n = \frac{3^{n+1} + (-1)^n}{4}$ explica el comportamiento: domina la potencia de 3. Con `GraficaRRL[{2, 3}, {1, 2}]` se obtiene la representación gráfica.

2.5.4

(★★)

TIPO EXAMEN

A MANO

Categoría del banco: 4-Rr. Dada la fórmula explícita hallar una recurrencia

Encuentre una relación de recurrencia cuya fórmula explícita es $a_n = 5n - 2$ para $n \geq 1$.

SOLUCIÓN Al restar dos términos consecutivos,

$$a_n - a_{n-1} = (5n - 2) - (5(n - 1) - 2) = 5$$

de donde $a_n = a_{n-1} + 5$, con $a_1 = 3$. Es una progresión aritmética de diferencia 5:

```
In[ ] :=
```

```
RT[{1, 5}, {3}, 6]
```

```
Out[ ] =
```

```
{3, 8, 13, 18, 23, 28}
```

```
In[ ] :=
```

```
RR[{1, 5}, {3}, n, inicio -> 1]
```

```
Out[ ] =
```

```
-2 + 5*n
```

2.5.5

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 5-Rr. Caso raíces diferentes

Complete. Al resolver $a_n - 6a_{n-1} + 11a_{n-2} - 6a_{n-3} = 0$ con $a_0 = 1$, $a_1 = 2$ y $a_2 = 3$ se obtiene $a_n = \frac{1}{2} (\text{---}_{(i)} \cdot 2^n - 3^n - \text{---}_{(ii)})$.

SOLUCIÓN La ecuación característica es $t^3 - 6t^2 + 11t - 6 = 0$:

```
In[ ] :=
```

```
Solve[t^3 - 6 t^2 + 11 t - 6 == 0, t]
```

```
Out[ ] =
```

```
{{t -> 1}, {t -> 2}, {t -> 3}}
```

Las tres raíces son distintas, de modo que la solución tiene la forma $b_1 + b_2 2^n + b_3 3^n$. Resolviendo con las condiciones iniciales:

In[] :=

```
RR[{6, -11, 6}, {1, 2, 3}, n, inicio -> 0]
```

Out[] =

$(-1 + 2^{2+n} - 3^n)/2$

Es decir $a_n = \frac{4 \cdot 2^n - 3^n - 1}{2}$: los huecos son (i) 4 y (ii) 1. Comprobación con los primeros términos:

In[] :=

```
RT[{6, -11, 6}, {1, 2, 3}, 6]
```

Out[] =

{1, 2, 3, 2, -9, -58}

2.5.6

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 6-Rr. Caso raíces repetidas

Considere para $n \geq 3$ la recurrencia $a_n - 5a_{n-1} + 8a_{n-2} - 4a_{n-3} = 0$ con $a_0 = 1$, $a_1 = 3$ y $a_2 = 9$. Complete: la forma explícita es $a_n = \underline{\hspace{1cm}}_{(i)} + \underline{\hspace{1cm}}_{(ii)} \cdot 2^n$, y además $a_5 = \underline{\hspace{1cm}}_{(iii)}$.

SOLUCIÓN La ecuación característica tiene una raíz simple y una doble:

In[] :=

```
Solve[t^3 - 5 t^2 + 8 t - 4 == 0, t]
```

Out[] =

{{t -> 1}, {t -> 2}, {t -> 2}}

Con la raíz 2 repetida, la solución general es $b_1 + b_2 2^n + b_3 n 2^n$:

In[] :=

```
RR[{5, -8, 4}, {1, 3, 9}, n, inicio -> 0]
```

Out[] =

$1 + 2^{n*n}$

Los huecos son (i) 1 y (ii) n , es decir $a_n = 1 + n 2^n$. Para a_5 :

In[] :=

```
RT[{5, -8, 4}, {1, 3, 9}, 6]
```

Out[] =

{1, 3, 9, 25, 65, 161}

El sexto término corresponde a $a_5 = 161 = 1 + 5 \cdot 32$, de modo que (iii) = 161.

2.5.7 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rr. Indicar conjunto Sol. de la ecuación auxiliar

Considere para $n \geq 2$ la recurrencia $a_n - 7a_{n-1} + 12a_{n-2} = 0$. El conjunto solución, ordenado de menor a mayor, de la ecuación auxiliar o característica es $\{\text{---(i)}, \text{---(ii)}\}$.

SOLUCIÓN Se sustituye $a_n = t^n$ y se divide entre t^{n-2} , lo que da $t^2 - 7t + 12 = 0$:

In[] :=

Solve[t^2 - 7 t + 12 == 0, t]

Out[] =

{{t -> 3}, {t -> 4}}

Los huecos son (i) 3 y (ii) 4. El signo de los coeficientes de la ecuación característica es el mismo del lado izquierdo de la recurrencia igualada a cero.

2.5.8 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-Rr. Sistema de ecuaciones asociado

Sean $b_1, b_2 \in \mathbb{R}$. Para la recurrencia $a_n = 7a_{n-1} - 12a_{n-2}$ con $a_0 = 2$ y $a_1 = 7$, el sistema que hay que resolver para hallar la fórmula explícita es

$$\text{---(i)} b_1 + \text{---(ii)} b_2 = 2, \quad \text{---(iii)} b_1 + \text{---(iv)} b_2 = 7$$

SOLUCIÓN Con raíces 3 y 4 la solución general es $a_n = b_1 3^n + b_2 4^n$. Evaluando en $n = 0$ y $n = 1$ se obtiene $b_1 + b_2 = 2$ y $3b_1 + 4b_2 = 7$, de modo que los huecos son (i) 1, (ii) 1, (iii) 3 y (iv) 4:

In[] :=

Solve[{b1 + b2 == 2, 3 b1 + 4 b2 == 7}, {b1, b2}]

Out[] =

{{b1 -> 1, b2 -> 1}}

In[] :=

RR[{7, -12}, {2, 7}, n, inicio -> 0]

Out[] =

3^n + 4^n

El comando MetodoRRHL recorre este mismo procedimiento paso a paso.

2.5.9 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-Rr. Recurrencias lineales no homogéneas

Considere para $n \geq 1$ la recurrencia $a_n = 3a_{n-1} + 4$ con $a_0 = 2$. I. Calcule $a_5 = \text{---(i)}$. II. La forma explícita es $a_n = \text{---(ii)} \cdot 3^n - \text{---(iii)}$.

SOLUCIÓN

In[] :=

RT[{3, 4}, {2}, 6]

Out[] =

{2, 10, 34, 106, 322, 970}

El sexto término de la lista es $a_5 = 970$.

In[] :=

RR[{3, 4}, {2}, n, inicio -> 0]

Out[] =

2*(-1 + 2*3^n)

Es decir $a_n = 4 \cdot 3^n - 2$. Los huecos son (i) 970, (ii) 4 y (iii) 2. Comprobación: $4 \cdot 243 - 2 = 970$.

2.5.10 (★★) TIPO EXAMEN A MANO

Categoría del banco: 10-Rr. Clasificar relación

Señale cuáles de las siguientes relaciones son lineales, homogéneas y con coeficientes constantes: (a) $a_n = 2a_{n-1} - 3a_{n-2}$; (b) $a_n = n a_{n-1}$; (c) $a_n = a_{n-1} + a_{n-2} + 5$; (d) $a_n = (a_{n-1})^2$; (e) $a_n = 4a_{n-3}$.

SOLUCIÓN Solo (a) y (e). En (b) el coeficiente n no es constante; en (c) el sumando 5 es un término independiente, de modo que la relación es no homogénea; y en (d) el término anterior aparece elevado al cuadrado, lo que la hace no lineal. Que en (e) falten a_{n-1} y a_{n-2} no impide la clasificación: sus coeficientes valen cero.

2.5.11 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-Rr. Recurrencias reducibles a lineales

Considere $(a_n)^3 = 8(a_{n-1})^3$ con $a_0 = 2$. (a) Halle una fórmula explícita mediante un cambio de variable. (b) Construya la tabla de los seis primeros términos.

SOLUCIÓN (a) Con $b_n = (a_n)^3$ la relación se vuelve lineal homogénea: $b_n = 8b_{n-1}$ con $b_0 = 8$, cuya solución es $b_n = 8^{n+1}$. Deshaciendo el cambio, $a_n = \sqrt[3]{8^{n+1}} = 2^{n+1}$. (b) Los términos de b_n y su raíz cúbica:

In[] :=

RT[{8}, {8}, 6]

Out[] =

{8, 64, 512, 4096, 32768, 262144}

In[] :=

Table[2^(n + 1), {n, 0, 5}]

Out[] =

{2, 4, 8, 16, 32, 64}

La segunda lista es la sucesión buscada: 2, 4, 8, 16, 32, 64. El cambio de variable es la técnica estándar para reducir a lineal una recurrencia en la que los términos aparecen elevados a una misma potencia.

2.5.12 (★★) TIPO EXAMEN A MANO

Categoría del banco: 10-Rr. Clasificar relación

Indique el orden de cada relación y cuántas condiciones iniciales necesita: (a) $a_n = 4a_{n-1} - a_{n-4}$; (b) $a_n = a_{n-2}$; (c) $a_n = 2a_{n-1} + n^2$.

SOLUCIÓN (a) Orden 4, cuatro condiciones iniciales, pues el término más lejano es a_{n-4} . (b) Orden 2, dos condiciones. (c) Orden 1, una condición; el sumando n^2 no altera el orden, solo hace la relación no homogénea.

2.5.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-Rr. Caso raíces diferentes / 7-Rr. Ecuación auxiliar

Para $a_n = 5a_{n-1} - 6a_{n-2}$ con $a_0 = 1$ y $a_1 = 2$: (a) escriba la ecuación característica y resuélvala; (b) halle la fórmula explícita; (c) verifique los primeros ocho términos.

SOLUCIÓN (a) La ecuación es $t^2 - 5t + 6 = 0$:

In[] :=

```
Solve[t^2 - 5 t + 6 == 0, t]
```

Out[] =

```
{{t -> 2}, {t -> 3}}
```

(b) Con raíces distintas, $a_n = b_1 2^n + b_2 3^n$; las condiciones dan $b_1 = 1$ y $b_2 = 0$:

In[] :=

```
RR[{5, -6}, {1, 2}, n, inicio -> 0]
```

Out[] =

```
2^n
```

(c) Los términos son potencias de dos:

In[] :=

```
RT[{5, -6}, {1, 2}, 8]
```

Out[] =

```
{1, 2, 4, 8, 16, 32, 64, 128}
```

Que una de las constantes salga nula es perfectamente posible: la raíz 3 no interviene en esta solución particular.

2.5.14 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rr. Evaluación de recurrencias

Para la recurrencia de *Fibonacci* $a_n = a_{n-1} + a_{n-2}$ con $a_1 = a_2 = 1$, indique el décimo término y la suma de los diez primeros.

SOLUCIÓN

In[] :=

```
RT[{1, 1}, {1, 1}, 10]
```

Out[] =

```
{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}
```

In[] :=

```
Total[RT[{1, 1}, {1, 1}, 10]]
```

```
Out[ ] =
```

```
143
```

El décimo término es 55 y la suma vale 143. Se cumple la identidad $\sum_{i=1}^n F_i = F_{n+2} - 1$: en efecto, $F_{12} - 1 = 144 - 1 = 143$.

2.5.15 (★★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rr. Evaluación de recurrencias

Al calcular la suma de los diez primeros términos de $a_n = 2a_{n-1} - a_{n-2} + 2a_{n-3}$, con $a_1 = 1$, $a_2 = 1$ y $a_3 = 2$, se obtiene ____ (i).

SOLUCIÓN

```
In[ ] :=
```

```
RT[{2, -1, 2}, {1, 1, 2}, 10]
```

```
Out[ ] =
```

```
{1, 1, 2, 5, 10, 19, 38, 77, 154, 307}
```

```
In[ ] :=
```

```
Total[RT[{2, -1, 2}, {1, 1, 2}, 10]]
```

```
Out[ ] =
```

```
614
```

El hueco vale 614. A partir del cuarto término la sucesión casi duplica en cada paso, porque la raíz dominante de la ecuación característica es 2.

2.5.16 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-Rr. Hallar recurrencia dada una lista

Para la sucesión $\{1, 5, 19, 77, 307, 1229\}$, halle $a_n = \text{____} (i) a_{n-1} + \text{____} (ii) a_{n-2}$ y sus condiciones iniciales.

SOLUCIÓN Se plantean $19 = 5c_1 + c_2$ y $77 = 19c_1 + 5c_2$; resolviendo, $c_1 = 3$ y $c_2 = 4$.

```
In[ ] :=
```

```
{RT[{3, 4}, {1, 5}, 8], RR[{3, 4}, {1, 5}, n, inicio -> 0]}
```

```
Out[ ] =
```

```
{1, 5, 19, 77, 307, 1229, 4915, 19661}, (-(-1)^n + 3*2^(1 + 2*n))/5}
```

Los huecos son (i) 3 y (ii) 4, con $a_0 = 1$ y $a_1 = 5$. La fórmula explícita mezcla 4^n con una alternancia, porque las raíces son 4 y -1 .

2.5.17 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rr. Indicar conjunto Sol. de la ecuación auxiliar

El conjunto solución de la ecuación auxiliar de $a_n = 3a_{n-1} + 4a_{n-2}$, ordenado de menor a mayor, es $\{\text{____} (i), \text{____} (ii)\}$.

SOLUCIÓN**In[] :=**`Solve[t^2 - 3 t - 4 == 0, t]`**Out[] =**`{{t -> -1}, {t -> 4}}`

Los huecos son -1 y 4 . Los signos piden cuidado: la ecuación se arma pasando todo al lado izquierdo, de modo que $+4a_{n-2}$ produce -4 en el término independiente.

2.5.18 (★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 6-Rr. Caso raíces repetidas

Considere $a_n = 4a_{n-1} - 5a_{n-2} + 2a_{n-3}$ con $a_0 = 1$, $a_1 = 3$ y $a_2 = 8$. I. Las raíces de la ecuación auxiliar son _____. II. La forma explícita es $a_n = \text{____} \cdot 2^n - n - \text{_____}$.

SOLUCIÓN**In[] :=**`Solve[t^3 - 4 t^2 + 5 t - 2 == 0, t]`**Out[] =**`{{t -> 1}, {t -> 1}, {t -> 2}}`**In[] :=**`RR[{4, -5, 2}, {1, 3, 8}, n, inicio -> 0]`**Out[] =**`-2 + 3*2^n - n`

Hay raíz doble $t = 1$ y simple $t = 2$, de modo que la forma general es $b_1 + b_2n + b_32^n$. Los huecos: (i) $\{1, 1, 2\}$, (ii) 3 y (iii) 2 .

2.5.19 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 6-Rr. Caso raíces repetidas

¿Por qué la raíz doble $t = 1$ del ejercicio anterior aporta un término lineal en n ?

SOLUCIÓN Porque con raíz repetida t de multiplicidad dos, las dos soluciones independientes son t^n y nt^n . Con $t = 1$ eso da 1 y n : una constante y una recta. Si no se añadiera el factor n , las dos soluciones serían la misma y el sistema quedaría indeterminado.

2.5.20 (★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 9-Rr. Recurrencias lineales no homogéneas

Considere $a_n = 2a_{n-1} + 7$ con $a_0 = 5$. I. Calcule $a_6 = \text{_____}$. II. La forma explícita es $a_n = \text{_____} \cdot 2^n - \text{_____}$.

SOLUCIÓN**In[] :=**`RT[{2, 7}, {5}, 7]`**Out[] =**

{5, 17, 41, 89, 185, 377, 761}

In[] :=

RR[{2, 7}, {5}, n, inicio -> 0]

Out[] =

-7 + 3*2^(2 + n)

El séptimo valor de la lista es $a_6 = 761$, y la fórmula es $12 \cdot 2^n - 7$. Los huecos: (i) 761, (ii) 12 y (iii) 7. El -7 es el punto fijo de la relación.

2.5.21 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 8-Rr. Sistema de ecuaciones asociado

Sean $b_1, b_2, b_3 \in \mathbb{R}$. Para una relación con raíces 1, 2 y 3 y condiciones $a_0 = 1$, $a_1 = 3$, $a_2 = 11$, escriba el sistema y resúelvalo.

SOLUCIÓN La forma general es $a_n = b_1 + b_2 2^n + b_3 3^n$, de donde

$$b_1 + b_2 + b_3 = 1, \quad b_1 + 2b_2 + 3b_3 = 3, \quad b_1 + 4b_2 + 9b_3 = 11$$

In[] :=

Solve[{b1 + b2 + b3 == 1, b1 + 2 b2 + 3 b3 == 3,
b1 + 4 b2 + 9 b3 == 11}, {b1, b2, b3}]

Out[] =

{{b1 -> 1, b2 -> -2, b3 -> 2}}

La solución es $a_n = 1 - 2 \cdot 2^n + 2 \cdot 3^n$.

2.5.22 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-Rr. Recurrencias lineales no homogéneas

Resuelva $a_n = a_{n-1} + 3n$ con $a_0 = 2$ y dé su fórmula explícita.

SOLUCIÓN

In[] :=

RT[{1, 3 n}, {2}, 7, inicio -> 0]

Out[] =

{2, 5, 11, 20, 32, 47, 65}

In[] :=

RR[{1, 3 n}, {2}, n, inicio -> 0]

Out[] =

(4 + 3*n + 3*n^2)/2

Es decir $a_n = \frac{3n^2 + 3n + 4}{2}$. La parte no homogénea es de grado uno y la solución sube a grado dos, como corresponde a una relación acumulativa. Nótese el `inicio -> 0` en RT: sin él la lista no correspondería a la fórmula.

2.5.23 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-Rr. Gráfica de una recurrencia

Los primeros ocho términos de una recurrencia son $\{2, 1, 11, -5, 71, -101, 527, -1133\}$. ¿Cuál de estas la genera? (a) $a_n = 6a_{n-2} - a_{n-1}$; (b) $a_n = a_{n-1} + 6a_{n-2}$; (c) $a_n = -a_{n-1} - 6a_{n-2}$.

SOLUCIÓN Es la (a). El cambio de signo alternante delata una raíz negativa:

In[] :=

```
{RT[{-1, 6}, {2, 1}, 8], RR[{-1, 6}, {2, 1}, n, inicio -> 0]}
```

Out[] =

```
{2, 1, 11, -5, 71, -101, 527, -1133}, (7*2^n + (-1)^n*3^(1 + n))/5}
```

Las raíces son 2 y -3 ; como $|-3| > 2$, el término alternante domina y por eso los signos se alternan a partir del tercer valor.

2.5.24 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-Rr. Caso raíces diferentes

Complete. Al resolver $a_n = 4a_{n-1} - 3a_{n-2}$ con $a_0 = 2$ y $a_1 = 4$ se obtiene $a_n = \underline{\hspace{1cm}}_{(i)} + \underline{\hspace{1cm}}_{(ii)}^n$.

SOLUCIÓN

In[] :=

```
{RT[{4, -3}, {2, 4}, 8], RR[{4, -3}, {2, 4}, n, inicio -> 0]}
```

Out[] =

```
{2, 4, 10, 28, 82, 244, 730, 2188}, 1 + 3^n}
```

Las raíces son 1 y 3, y las condiciones dan coeficientes ambos iguales a uno: $a_n = 1 + 3^n$. Los huecos son (i) 1 y (ii) 3.

2.5.25 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rr. Indicar conjunto Sol. de la ecuación auxiliar

Halle las raíces de la ecuación auxiliar de $a_n = 2a_{n-1} - 5a_{n-2}$ y describa la sucesión.

SOLUCIÓN

In[] :=

```
Solve[t^2 - 2 t + 5 == 0, t]
```

Out[] =

```
{{t -> 1 - 2*I}, {t -> 1 + 2*I}}
```

Las raíces son complejas conjugadas $1 \pm 2i$, de módulo $\sqrt{5} > 1$. La sucesión **oscila con amplitud creciente**: los valores cambian de signo sin periodo fijo y crecen como $(\sqrt{5})^n$.

2.5.26 (★★) TIPO EXAMEN A MANO

Categoría del banco: 10-Rr. Clasificar relación

Señale cuáles son lineales homogéneas con coeficientes constantes: (a) $a_n = a_{n-1} + 2^n$; (b) $5a_n = 4a_{n-2} - 5a_{n-1}$; (c) $a_n = a_{n-1}a_{n-3}$; (d) $a_n = \sqrt{2}a_{n-1}$; (e) $a_n = \frac{a_{n-1}}{n}$.

SOLUCIÓN Solo (b) y (d). En (a) el sumando 2^n es una parte no homogénea; en (c) los términos anteriores se multiplican, lo que la hace no lineal; en (e) el coeficiente $1/n$ **depende de n** . Que el coeficiente sea irracional, como en (d), no es obstáculo: lo que se exige es que sea constante.

2.5.27 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-Rr. Recurrencias reducibles a lineales

Considere $(a_n)^3 = 27(a_{n-1})^3$ con $a_0 = 2$. (a) Halle la fórmula explícita mediante un cambio de variable. (b) Tabule cinco términos de $(a_n)^3$.

SOLUCIÓN (a) Con $b_n = (a_n)^3$ queda $b_n = 27b_{n-1}$ con $b_0 = 8$, de solución $b_n = 8 \cdot 27^n$. Desahaciendo, $a_n = \sqrt[3]{8 \cdot 27^n} = 2 \cdot 3^n$. (b)

In[] :=

RT[{27}, {8}, 5]

Out[] =

{8, 216, 5832, 157464, 4251528}

Las raíces cúbicas son 2, 6, 18, 54, 162, es decir $2 \cdot 3^n$.

2.5.28 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 11-Rr. Recurrencias reducibles a lineales

¿Qué cambio de variable conviene en $a_n = \sqrt{a_{n-1}}$ con $a_0 = 16$, y cuál es el límite de la sucesión?

SOLUCIÓN Tomando logaritmos, $b_n = \log_2 a_n$ cumple $b_n = \frac{b_{n-1}}{2}$ con $b_0 = 4$, de solución $b_n = 4/2^n = 2^{2-n}$. Por lo tanto $a_n = 2^{2^{2-n}}$, y como el exponente tiende a cero, el límite es $2^0 = 1$. La raíz se convierte en división al pasar al logaritmo.

2.5.29 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-Rr. Evaluación de recurrencias

Un capital de \$800 000 crece al 9 % anual. Indique el capital al cabo de cinco años, redondeado a la unidad.

SOLUCIÓN

In[] :=

Round[RT[{1.09}, {800000}, 6]]

Out[] =

{800000, 872000, 950480, 1036023, 1129265, 1230899}

El sexto valor corresponde al quinto año: \$1 230 899. El capital supera el millón en el tercer año.

2.5.30 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 4-Rr. Dada la fórmula explícita hallar una recurrencia

Encuentre una relación de recurrencia cuya fórmula explícita sea $a_n = n^2 + n$ para $n \geq 0$.

SOLUCIÓN Se calcula la diferencia:

$$a_n - a_{n-1} = (n^2 + n) - ((n-1)^2 + (n-1)) = 2n$$

de donde $a_n = a_{n-1} + 2n$ con $a_0 = 0$. La relación es lineal no homogénea de orden uno con parte no homogénea de grado uno, coherente con que la solución sea de grado dos.

2.5.31 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-Rr. Dada la fórmula explícita hallar una recurrencia

Compruebe la recurrencia del ejercicio anterior.

SOLUCIÓN

In[] :=

```
{RT[{1, 2 n}, {0}, 7, inicio -> 0], Table[k^2 + k, {k, 0, 6}]}
```

Out[] =

```
{{0, 2, 6, 12, 20, 30, 42}, {0, 2, 6, 12, 20, 30, 42}}
```

Las dos listas coinciden. Es imprescindible el `inicio -> 0`: con el valor por defecto la recurrencia arrancaría en otro índice y las listas no coincidirían.

2.5.32 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-Rr. Gráfica de una recurrencia

Los primeros términos de una recurrencia son $\{5, 30, 180, 1080, 6480\}$. Identifíquela y dé su fórmula.

SOLUCIÓN Cada término es seis veces el anterior:

In[] :=

```
{RT[{6}, {5}, 8], RR[{6}, {5}, n, inicio -> 0]}
```

Out[] =

```
{{5, 30, 180, 1080, 6480, 38880, 233280, 1399680}, 5*6^n}
```

Es geométrica de razón 6: $a_n = 5 \cdot 6^n$. En una gráfica, una progresión geométrica se reconoce porque su representación en escala logarítmica es una recta.

2.5.33 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 8-Rr. Sistema de ecuaciones asociado

Al plantear el sistema de una relación de orden tres con raíces 2, 2 y 5, ¿qué forma tiene la solución general y qué columnas tiene el sistema?

SOLUCIÓN La raíz 2 es doble, de modo que la forma general es

$$a_n = b_1 2^n + b_2 n 2^n + b_3 5^n$$

El sistema se obtiene evaluando en $n = 0, 1, 2$ y sus columnas son los valores de 2^n , $n 2^n$ y 5^n : $\{1, 2, 4\}$, $\{0, 2, 8\}$ y $\{1, 5, 25\}$. Sin el factor n en la segunda, dos columnas serían iguales y el sistema no tendría solución única.

2.5.34 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-Rr. Clasificar relación / 7-Rr. Ecuación auxiliar

Una relación lineal homogénea de orden dos tiene por solución $a_n = 3 \cdot 5^n - 2 \cdot (-1)^n$. Reconstruyala.

SOLUCIÓN Las raíces son 5 y -1 , de modo que la ecuación característica es $(t - 5)(t + 1) = t^2 - 4t - 5 = 0$ y la relación

$$a_n = 4a_{n-1} + 5a_{n-2}$$

Las condiciones iniciales salen de la fórmula: $a_0 = 3 - 2 = 1$ y $a_1 = 15 + 2 = 17$. El camino de la solución a la relación es exactamente el inverso del habitual.

CAPÍTULO 3 • PREGUNTAS TIPO EXAMEN

Análisis de algoritmos

3.7.1 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-AI. Comparación por experimento de 2 algoritmos

Considere los dos métodos siguientes, que realizan la misma tarea:

```
f1[n_] := Which[n == 1, 3, n == 2, -2, n > 2, 2 f1[n - 1] + f1[n - 2]]
f2[n_] := Module[{g}, g[i_, a_, ta_] := Which[n == 1, 3, n == 2, -2,
    i == n, 2 a + ta, True, g[i + 1, 2 a + ta, a]]; g[3, -2, 3]]
```

(a) El valor de estos métodos cuando $n = 20$ es _____. (i). (b) El método que usa una recursividad de cola es _____. (ii). (c) En un experimento con n entre 5 y 25, el método más rápido es _____. (iii).

SOLUCIÓN (a) Los dos calculan la misma sucesión:

In[] :=

```
f1[n_] := Which[n == 1, 3, n == 2, -2, n > 2, 2 f1[n - 1] + f1[n - 2]];
f2[n_] := Module[{g}, g[i_, a_, ta_] := Which[n == 1, 3, n == 2, -2,
    i == n, 2 a + ta, True, g[i + 1, 2 a + ta, a]]; g[3, -2, 3]];
{f1[20], f2[20]}
```

Out[] =

```
{-5017588, -5017588}
```

(b) Es f2: su llamada recursiva no deja ninguna operación pendiente, porque arrastra los dos términos anteriores en los parámetros a y ta . (c) También f2. La versión de pila recalcula las mismas ramas una y otra vez, con un costo exponencial, mientras que la de cola hace un número de pasos proporcional a n . El experimento se monta con `PruebaADA2[{f1, f2}, 20, 5, 1]`.

3.7.2 (★★★) TIPO EXAMEN ViCretas

Categoría del banco: 2-AI. Comparación por experimento de 4 métodos

Ordene de más rápido a más lento los cuatro métodos de ordenamiento disponibles: *quicksort*, burbuja, inserción y selección, sobre listas pseudoaleatorias.

SOLUCIÓN El experimento se monta con `PruebaADA2` por parejas o con `PruebaADA3` para tres a la vez:

In[] :=

```
PruebaADA2[{Quicksort, Burbuja}, 20, 10, 2, lista -> True]
```

Quicksort es el más rápido: su orden es $O(n \ln n)$ en el caso promedio, frente a $O(n^2)$ de los otros tres. Entre burbuja, inserción y selección las diferencias son de constante y no de orden, de modo que el conteo varía de una corrida a otra y `PruebaADA3` suele reportar empates. La conclusión firme la da el enfoque teórico, no el experimento.

3.7.3 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-AI. Análisis O grande para un procedimiento en código

Considere el siguiente método:

```
Programa[n_] := Module[{p = 1}, For[i = 0, i <= n - 2,
    For[j = 1, j <= Floor[i/2], p = p + 1; j++]; i++]; Return[p]]
```

Al realizar un análisis O grande se obtiene $O(f(n))$ con $f(n) = \underline{\hspace{1cm}}_{(i)}$.

SOLUCIÓN Para cada valor de i el ciclo interno da $\lfloor i/2 \rfloor$ pasos, y i recorre de 0 a $n - 2$. El total es aproximadamente

$$\sum_{i=0}^{n-2} \frac{i}{2} = \frac{(n-1)(n-2)}{4}$$

un polinomio de grado dos.

In[] :=

```
Limit[((n - 1) (n - 2)/2)/n^2, n -> Infinity]
```

Out[] =

1/2

El límite es real y positivo, de modo que $f(n) = n^2$: el programa es $O(n^2)$, de hecho $\Theta(n^2)$.

3.7.4 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-AI. Análisis O grande para un procedimiento en código

Determine la notación O grande del siguiente método:

```
g[n_] := Module[{s = {}}, For[i = 1, i <= n, i = i*5,
    s = Append[s, i]]; s]
```

SOLUCIÓN El contador no crece sumando sino **multiplicando** por 5, de modo que toma los valores 1, 5, 25, y en general 5^k . El ciclo termina cuando $5^k > n$, es decir cuando $k > \log_5 n$. El esfuerzo es $\lfloor \log_5 n \rfloor + 1$, y como cambiar de base solo multiplica por una constante,

In[] :=

```
Limit[(Log[n]/Log[5])/Log[n], n -> Infinity]
```

Out[] =

Log[5]^(-1)

el límite es la constante $1/\ln 5$, positiva, de modo que el método es $\Theta(\ln n)$.

3.7.5 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-AI. Simplificar productorias

Elija la notación asintótica que mejor se ajusta a $\prod_{i=1}^n 2i$: (a) $O(n!)$; (b) $\Omega(n!)$; (c) $\Theta(n!)$; (d) ninguna de las anteriores.

SOLUCIÓN La respuesta es (b). El producto vale $2^n n!$:

In[] :=

```
Product[2 i, {i, 1, n}]
```

Out[] =

$2^n \text{Gamma}[1 + n]$

In[] :=

```
Limit[Product[2 i, {i, 1, n}]/n!, n -> Infinity]
```

Out[] =

Infinity

El cociente tiende a infinito, lo que da $\Omega(n!)$ y descarta $O(n!)$ y por lo tanto $\Theta(n!)$. El factor 2^n no es despreciable frente a $n!$.

3.7.6

(★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 5-AI. Simplificar productos

Elija la notación asintótica que mejor se ajusta a $(n^n + 3^n + 1)(n! + n^2 + 2^n)$.

SOLUCIÓN Se acota cada factor por su término dominante: el primero es $\Theta(n^n)$ y el segundo $\Theta(n!)$, de modo que el producto es $\Theta(n! n^n)$.

In[] :=

```
Limit[(n^n + 3^n + 1) (n! + n^2 + 2^n))/(n! n^n), n -> Infinity]
```

Out[] =

1

El límite es 1, lo que confirma la respuesta. Hay que acotar factor por factor y no distribuir.

3.7.7

(★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 6-AI. Simplificar sumas cortas

Halle la notación asintótica que mejor se ajusta a $7^n + 10n^2 + 8n^3 + 3^n$: _____(i) (_____(ii)).

SOLUCIÓN En una suma manda el sumando de mayor orden, y 7^n supera tanto a los polinomios como a 3^n .

In[] :=

```
Limit[(7^n + 10 n^2 + 8 n^3 + 3^n)/7^n, n -> Infinity]
```

Out[] =

1

El límite es 1, real y positivo, de modo que la respuesta es $\Theta(7^n)$: los huecos son (i) Θ y (ii) 7^n .

3.7.8

(★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 7-AI. Simplificar sumatorias

Halle la notación asintótica que mejor se ajusta a $\sum_{i=1}^n (2i)^3$.

SOLUCIÓN

In[] :=

```
Sum[(2 i)^3, {i, 1, n}]
```

Out[] =

$2n^2(1 + n)^2$

La suma vale $2n^2(n+1)^2$, un polinomio de grado cuatro.

In[] :=

```
Limit[Sum[(2 i)^3, {i, 1, n}]/n^4, n -> Infinity]
```

Out[] =

2

El límite es 2, de modo que la respuesta es $\Theta(n^4)$, en concordancia con la regla $1^j + \dots + n^j = \Theta(n^{j+1})$ para $j = 3$.

3.7.9

(★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 7-AI. Simplificar sumatorias

Halle la notación asintótica de $\sum_{i=1}^n i(i+1)$.

SOLUCIÓN

In[] :=

```
Sum[i (i + 1), {i, 1, n}]
```

Out[] =

```
(n*(1 + n)*(2 + n))/3
```

In[] :=

```
Limit[Sum[i (i + 1), {i, 1, n}]/n^3, n -> Infinity]
```

Out[] =

1/3

El polinomio es de grado tres y el límite vale $\frac{1}{3}$, de modo que la suma es $\Theta(n^3)$.

3.7.10

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 8-AI. Conjetura de parámetros para un análisis asintótico

Sean $f(n) = \prod_{i=1}^n 23i$ y $g(n) = n!j^n$. El valor de j para el cual $f(n) = \Theta(g(n))$ es $j = \underline{\hspace{1cm}}_{(i)}$.

SOLUCIÓN

In[] :=

```
Product[23 i, {i, 1, n}]
```

Out[] =

```
23^n*Gamma[1 + n]
```

El producto vale $23^n n!$, de modo que coincide con g exactamente cuando $j = 23$:

In[] :=

```
Limit[Product[23 i, {i, 1, n}]/(n! 23^n), n -> Infinity]
```

Out[] =

1

El límite es 1, real y positivo, lo que confirma Θ . Con cualquier otro valor de j el cociente tendería a cero o a infinito, y se obtendría O u Ω pero no Θ .

3.7.11 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 8-AI. Conjetura de parámetros para un análisis asintótico

Sean $f(n) = n^4 + \ln n$ y $g(n) = n^j$ con j natural. Conjeture para cuáles j se cumple O , Ω y Θ .

SOLUCIÓN El orden de f es n^4 , así que basta comparar n^4 con n^j :

$$1 \leq j \leq 3 \Rightarrow f(n) = \Omega(g(n)), \quad j = 4 \Rightarrow f(n) = \Theta(g(n)), \quad j \geq 5 \Rightarrow f(n) = O(g(n))$$

El criterio es el límite de n^4/n^j , que vale $+\infty$, 1 o 0 según j sea menor, igual o mayor que 4.

3.7.12 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 3-AI. Análisis O grande para un procedimiento en código

El comando `CalculaSuma` del paquete implementa dos ciclos anidados. Evalúelo en $n = 12$ y deduzca su notación asintótica.

SOLUCIÓN

`In[ ] :=`

`CalculaSuma[12]`

`Out[ ] =`

288

El resultado es $2 \cdot 12^2$. Como $f(n) = 2n^2$, el método es $\Theta(n^2)$. Con `CalculaSuma[12, code -> True]` se obtiene el código interno y se puede confirmar el conteo leyendo los dos ciclos.

3.7.13 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-AI. Simplificar sumas cortas

Halle la notación asintótica que mejor se ajusta a $n^n + n! + 2^n + n^{100}$: (i) ((ii)).

SOLUCIÓN

`In[ ] :=`

`Limit[(n^n + n! + 2^n + n^100)/n^n, n -> Infinity]`

`Out[ ] =`

1

Manda n^n , que supera incluso al factorial. Los huecos son (i) Θ y (ii) n^n . La jerarquía es $n^{100} \prec 2^n \prec n! \prec n^n$: el exponente 100 impresiona pero es constante.

3.7.14 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-AI. Simplificar sumatorias

Halle la notación asintótica de $\sum_{i=1}^n (3i)^2$.

SOLUCIÓN

`In[ ] :=`

```
Sum[(3 i)^2, {i, 1, n}]
```

```
Out[ ] =
```

```
(3*n*(1 + n)*(1 + 2*n))/2
```

```
In[ ] :=
```

```
Limit[Sum[(3 i)^2, {i, 1, n}]/n^3, n -> Infinity]
```

```
Out[ ] =
```

```
3
```

Es $\Theta(n^3)$. El factor 3 dentro del cuadrado se convierte en un 9 constante que no altera el orden.

3.7.15 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-AI. Simplificar productorias

Elija la notación que mejor se ajusta a $\prod_{i=1}^n \frac{i}{2}$: (a) $O(n!)$; (b) $\Omega(n!)$; (c) $\Theta(n!)$; (d) ninguna.

SOLUCIÓN La respuesta es (a). El producto vale $\frac{n!}{2^n}$:

```
In[ ] :=
```

```
{Product[i/2, {i, 1, n}], Limit[Product[i/2, {i, 1, n}]/n!, n -> Infinity]}
```

```
Out[ ] =
```

```
{Gamma[1 + n]/2^n, 0}
```

El límite es cero, de modo que es $O(n!)$ pero no $\Omega(n!)$. Dividir cada factor entre dos sí cambia el orden, al contrario de lo que ocurre con un factor constante fuera del producto.

3.7.16 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-AI. Simplificar productos

Elija la notación que mejor se ajusta a $(n! + 2^n)(n^3 + \ln n)$.

SOLUCIÓN

```
In[ ] :=
```

```
Limit[((n! + 2^n) (n^3 + Log[n]))/(n! n^3), n -> Infinity]
```

```
Out[ ] =
```

```
1
```

Es $\Theta(n!n^3)$: se acota cada factor por su término dominante y se multiplican. Distribuir daría cuatro sumandos y habría que volver a comparar.

3.7.17 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-AI. Análisis O grande para un procedimiento en código

Considere el siguiente método:

```
Programa[n_] := Module[{p = 0}, For[i = 1, i <= n,
  For[j = 1, j <= i, p = p + 1; j++]; i++]; Return[p]]
```

Al realizar un análisis O grande se obtiene $O(f(n))$ con $f(n) = \underline{\hspace{1cm}}(i)$.

SOLUCIÓN El ciclo interno hace i pasos, de modo que el total es $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

In[] :=

```
Limit[(n (n + 1)/2)/n^2, n -> Infinity]
```

Out[] =

1/2

El límite es real y positivo: $f(n) = n^2$, y el programa es $\Theta(n^2)$.

3.7.18 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-AI. *Análisis O grande para un procedimiento en código*

Determine la notación de este método:

```
Programa[n_] := Module[{p = 0, i = n},
  While[i >= 1, For[j = 1, j <= n, p = p + 1; j++];
  i = Floor[i/2]]; Return[p]]
```

SOLUCIÓN El ciclo exterior divide i entre dos, de modo que da $\lfloor \log_2 n \rfloor + 1$ vueltas; el interior hace n pasos en cada una. El total es $n(\log_2 n + 1)$.

In[] :=

```
Limit[(n (Log[2, n] + 1))/(n Log[n]), n -> Infinity]
```

Out[] =

Log[2]^(-1)

El límite es la constante $1/\ln 2$, positiva: el método es $\Theta(n \ln n)$.

3.7.19 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-AI. *Conjetura de parámetros para un análisis asintótico*

Sean $f(n) = \prod_{i=1}^n \frac{i}{j}$ y $g(n) = \frac{n!}{7^n}$. El valor de j para el cual $f(n) = \Theta(g(n))$ es $j = \underline{\hspace{1cm}}_{(i)}$.

SOLUCIÓN El producto vale $\frac{n!}{j^n}$, de modo que coincide con g cuando $j = 7$:

In[] :=

```
Limit[Product[i/7, {i, 1, n}]/(n!/7^n), n -> Infinity]
```

Out[] =

1

Con cualquier otro valor el cociente sería $(7/j)^n$, que tiende a cero o a infinito y por lo tanto da O u Ω , nunca Θ .

3.7.20 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 8-AI. *Conjetura de parámetros para un análisis asintótico*

Sean $f(n) = n^3 \ln n$ y $g(n) = n^j$. Conjeture para cuáles j naturales se cumple O , Ω y Θ .

SOLUCIÓN Como $\ln n$ crece más despacio que cualquier potencia pero más rápido que una constante, $n^3 \prec n^3 \ln n \prec n^4$. Por lo tanto

$$1 \leq j \leq 3 \Rightarrow f(n) = \Omega(g(n)), \quad j \geq 4 \Rightarrow f(n) = O(g(n))$$

y **nunca** se cumple Θ : no hay ningún j entero que atrape el factor logarítmico.

3.7.21 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-AI. Comparación por experimento de 2 algoritmos

Considere estos dos métodos, que calculan lo mismo:

```
h1[n_] := Module[{s = 0, i}, For[i = 1, i <= n, i++, s = s + i^2]; s]
h2[n_] := n (n + 1) (2 n + 1)/6
```

(a) El valor para $n = 200$ es _____. (b) ¿Cuál es más rápido y por qué?

SOLUCIÓN

In[] :=

```
h1[n_] := Module[{s = 0, i}, For[i = 1, i <= n, i++, s = s + i^2]; s];
h2[n_] := n (n + 1) (2 n + 1)/6;
{h1[200], h2[200]}
```

Out[] =

```
{2686700, 2686700}
```

(b) $h2$ es incomparablemente más rápido: es $\Theta(1)$ frente a $\Theta(n)$. El experimento lo confirmaría, pero aquí la teoría basta: una fórmula cerrada no depende del tamaño de la entrada.

3.7.22 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-AI. Comparación por experimento de 4 métodos

Cuatro métodos resuelven el mismo problema con órdenes $\Theta(n)$, $\Theta(n \ln n)$, $\Theta(n^2)$ y $\Theta(2^n)$. Ordénelos por rapidez para $n = 10$ y para $n = 10^6$, suponiendo la misma constante.

SOLUCIÓN Para $n = 10$: $2^{10} = 1024$, $10^2 = 100$, $10 \ln 10 \approx 23$, 10 . El orden es lineal, $n \ln n$, cuadrático y exponencial. Para $n = 10^6$ el orden relativo es el mismo, pero las distancias son abismales: 10^6 , $1,4 \cdot 10^7$, 10^{12} y un número de más de trescientas mil cifras. **El orden relativo no cambia**; lo que cambia es que con n grande las diferencias dejan de ser tolerables.

3.7.23 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-AI. Comparación por experimento de 4 métodos

¿Puede un método $\Theta(2^n)$ ganarle a uno $\Theta(n)$ en un experimento?

SOLUCIÓN Sí, si el rango de tamaños es pequeño y las constantes lo favorecen. Con $T_1(n) = 2^n$ y $T_2(n) = 10^6 n$, el exponencial gana hasta $n \approx 25$. Un experimento con n entre 5 y 20 concluiría lo contrario de lo que dice la teoría, y ambas observaciones serían correctas en su ámbito.

3.7.24 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-AI. Simplificar sumas cortas

Halle la notación de $\frac{n^4 + 3n}{n^2 + 1}$.

SOLUCIÓN

In[] :=

```
Limit[(n^4 + 3 n)/(n^2 + 1) n^2, n -> Infinity]
```

Out[] =

1

Es $\Theta(n^2)$: el grado del numerador menos el del denominador. El límite 1 confirma que la cota es ajustada.

3.7.25 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-A1. Simplificar sumatorias

Halle la notación de $\sum_{i=1}^n \frac{1}{\sqrt{i}}$.

SOLUCIÓN

In[] :=

```
Limit[Sum[1/Sqrt[i], {i, 1, n}]/Sqrt[n], n -> Infinity]
```

Out[] =

2

Es $\Theta(\sqrt{n})$. La suma **diverge**, a diferencia de $\sum 1/i^2$, pero mucho más despacio que n : la comparación con la integral $\int_1^n x^{-1/2} dx = 2\sqrt{n} - 2$ da el mismo resultado.

3.7.26 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-A1. Simplificar sumatorias

Halle la notación de $\sum_{i=1}^n \frac{1}{i^2}$ y explique la diferencia con la serie armónica.

SOLUCIÓN

In[] :=

```
Limit[Sum[1/i^2, {i, 1, n}], n -> Infinity]
```

Out[] =

$\text{Pi}^2/6$

La suma **converge** a $\frac{\pi^2}{6}$, de modo que es $\Theta(1)$. La armónica $\sum 1/i$, en cambio, es $\Theta(\ln n)$ y **diverge**. El exponente 2 frente al 1 marca la frontera entre acotado y no acotado.

3.7.27 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-A1. Simplificar productorias

Determine la notación de $\prod_{i=1}^n \left(1 + \frac{1}{i}\right)$.

SOLUCIÓN

In[] :=

```
Product[1 + 1/i, {i, 1, n}]
```

Out[] =

$1 + n$

Es $\Theta(n)$. El producto es telescópico y vale exactamente $n + 1$: aunque tenga n factores, cada uno tiende a uno y el crecimiento resulta apenas lineal.

3.7.28 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 3-AI. Análisis O grande para un procedimiento en código

Un procedimiento tiene dos partes independientes, la primera $\Theta(n^2)$ y la segunda $\Theta(n \ln n)$. ¿De qué orden es el procedimiento?

SOLUCIÓN Es $\Theta(n^2)$. Por la regla de la suma, el orden del total es el del sumando mayor, y $n \ln n \prec n^2$. Si en cambio la segunda parte estuviera **dentro** de un ciclo de la primera habría que multiplicar, y el resultado sería $\Theta(n^3 \ln n)$: hay que distinguir si las partes se suceden o se anidan.

3.7.29 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 6-AI. Simplificar sumas cortas

Ordene por orden asintótico creciente: $n^{\ln n}$, $(\ln n)^n$, 2^n , n^2 .

SOLUCIÓN Tomando logaritmos: $\ln^2 n$, $n \ln \ln n$, $n \ln 2$ y $2 \ln n$. Ordenando esos, $2 \ln n \prec \ln^2 n \prec n \ln 2 \prec n \ln \ln n$, de donde

$$n^2 \prec n^{\ln n} \prec 2^n \prec (\ln n)^n$$

Comparar los logaritmos es la técnica estándar cuando las funciones son exponenciales con exponentes distintos.

3.7.30 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-AI. Comparación por experimento de 2 algoritmos

Use `CompLimit` para decidir la relación entre $n \ln n$ y n^2 .

SOLUCIÓN`ln[] :=``CompLimit[{n Log[n], n^2}]`

Responde notación O grande, con límite cero: $n \ln n = O(n^2)$ y no al revés. Es la diferencia de orden que separa *quicksort* de los métodos de ordenamiento elementales.

3.7.31 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 5-AI. Simplificar productos

Simplifique $(\sum_{i=1}^n i) (\sum_{i=1}^n i^2)$ en notación asintótica.

SOLUCIÓN La primera suma es $\Theta(n^2)$ y la segunda $\Theta(n^3)$, de modo que el producto es $\Theta(n^5)$. No hace falta desarrollar el producto de los dos polinomios: basta multiplicar los órdenes, por la regla del producto.

3.7.32 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-AI. Comparación por experimento de 4 métodos

Un experimento sobre cuatro métodos reporta 20 empates de 20 invocaciones. ¿Qué conclusiones son legítimas?

SOLUCIÓN Que en el rango probado los cuatro tardan lo mismo **dentro de la resolución del reloj**. No se puede concluir que tengan el mismo orden: puede que las entradas sean tan pequeñas que todo termine en microsegundos. La respuesta correcta es **repetir con entradas mayores**, y si el empate persiste, recurrir al análisis teórico.

3.7.33 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-AI. *Análisis O grande para un procedimiento en código*

Un procedimiento recorre una lista de n elementos y, por cada uno, recorre otra de m . Exprese su orden y diga qué ocurre si $m = n^2$.

SOLUCIÓN El orden es $\Theta(nm)$, con **dos** parámetros independientes: no se puede reducir a uno solo sin saber cómo se relacionan. Si $m = n^2$ entonces el procedimiento es $\Theta(n^3)$; si m es constante, es $\Theta(n)$. Escribir $\Theta(n^2)$ sin justificar la relación entre n y m es un error frecuente.

CAPÍTULO 4 • PREGUNTAS TIPO EXAMEN

Relaciones binarias

4.7.1 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-Rel. Dominio de una relación

Sea A el conjunto de los ocho primeros números primos y R la relación sobre A dada por $a R b$ si y solo si $\frac{a+b}{2} < 8$. Indique el dominio de R . Ayuda: en *Mathematica*, `Prime[n]` da el primo número n .

SOLUCIÓN`In[] :=``A = Table[Prime[k], {k, 1, 8}]``Out[] =``{2, 3, 5, 7, 11, 13, 17, 19}``In[] :=`

```
A = Table[Prime[k], {k, 1, 8}];
R = RelBin["(a + b)/2 < 8", A, A];
Union[First /@ R]
```

`Out[] =``{2, 3, 5, 7, 11, 13}`

El dominio es $\{2, 3, 5, 7, 11, 13\}$: los primos 17 y 19 quedan fuera porque sumados incluso con el menor elemento, 2, dan un promedio mayor o igual que 8. El ámbito coincide con el dominio, porque la condición es simétrica en a y b .

4.7.2 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-Rel. Dominio de una relación

Para la relación anterior, indique cuántos pares ordenados tiene.

SOLUCIÓN`In[ ] :=`

```
Length[RelBin["(a + b)/2 < 8", Table[Prime[k], {k, 1, 8}],
Table[Prime[k], {k, 1, 8}]]]
```

`Out[] =``22`

Son 22 de los $8 \cdot 8 = 64$ pares posibles del producto cartesiano.

4.7.3 (★★) TIPO EXAMEN A MANO

Categoría del banco: 3-Rel. Producto cartesiano

Sea $A = \{e^2, \{1\}, \emptyset\}$ y $B = \{\emptyset, \{\emptyset\}\}$. Seleccione los dos elementos que pertenecen a $A \times B$:
 (a) $(\emptyset, \{\emptyset\})$ y $(e^2, \emptyset)$; (b) $(\{\emptyset\}, \emptyset)$ y $(\emptyset, e^2)$; (c) $(1, \emptyset)$ y $(\emptyset, \{1\})$.

SOLUCIÓN La respuesta es (a). En $A \times B$ la primera coordenada debe estar en A y la segunda en B . En (b) el par $(\emptyset, e^2)$ falla porque e^2 no pertenece a B ; en (c) el elemento 1 no está en A (lo que está es el **conjunto** $\{1\}$, que no es lo mismo). Obsérvese que $\emptyset$ es elemento de los dos conjuntos, de modo que $(\emptyset, \emptyset)$ también pertenece al producto.

4.7.4

(★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 3-Rel. Producto cartesiano

Determine el número de elementos de $A \times B$ si A tiene cuatro elementos y B tiene tres, y escriba $A \times B$ para $A = \{1, 2, 3\}$ y $B = \{4, 5\}$.

SOLUCIÓN

In[] :=

```
Length[PC[{1, 2, 3, 4}, {5, 6, 7}]]
```

Out[] =

12

Por la fórmula del cardinal, $4 \cdot 3 = 12$.

In[] :=

```
PC[{1, 2, 3}, {4, 5}]
```

Out[] =

```
{{1, 4}, {1, 5}, {2, 4}, {2, 5}, {3, 4}, {3, 5}}
```

4.7.5

(★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 4-Rel. Producto cartesiano de intervalos (Desarrollo)

Haga, en un plano cartesiano, el dibujo del producto cartesiano de los intervalos $[-1, 2]$ y $[-3, 3]$, y escriba el código que lo genera.

SOLUCIÓN El producto cruz de dos intervalos reales es el **rectángulo** de base $[-1, 2]$ sobre el eje x y altura $[-3, 3]$ sobre el eje y : contiene todos los puntos (x, y) con $-1 \leq x \leq 2$ y $-3 \leq y \leq 3$. Su área es $3 \cdot 6 = 18$.

In[] :=

```
GraficaPC[{-1, 2}, {-3, 3}]
```

Si algún extremo fuera infinito, el comando grafica hasta 100 o -100 según corresponda.

4.7.6

(★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 5-Rel. Relaciones y su gráfico (conjunto de pares)

Sean $A = \{5, 8, 11, 14\}$ y $B = \{4, 6, 9, 10\}$. Considere la relación R de A en B tal que $a R b$ si el residuo de $a + b$ entre 3 es cero. Determine R explícitamente.

SOLUCIÓN

In[] :=

```
RelBin["Mod[a + b, 3] == 0", {5, 8, 11, 14}, {4, 6, 9, 10}]
```

Out[] =

$\{\{5, 4\}, \{5, 10\}, \{8, 4\}, \{8, 10\}, \{11, 4\}, \{11, 10\}, \{14, 4\}, \{14, 10\}\}$

Todos los elementos de A dejan residuo 2 al dividirse entre 3, de modo que se relacionan exactamente con los elementos de B de residuo 1, que son 4 y 10. El dominio es todo A y el ámbito es $\{4, 10\}$.

4.7.7 (★★) TIPO EXAMEN A MANO

Categoría del banco: 2-Rel. Graficación de relaciones en el plano cartesiano (Desarrollo)

Sea $A = \{3, 6, 9, 12\}$ y R la relación sobre A dada por $a R b$ si y solo si $\frac{a+b}{3}$ es un entero par. (a) Halle R explícitamente. (b) Explique cómo se ve en el plano cartesiano.

SOLUCIÓN (a) Se recorre $A \times A$ y se conservan los pares cuya suma dividida entre 3 es par:

$$R = \{(3, 3), (3, 9), (6, 6), (6, 12), (9, 3), (9, 9), (12, 6), (12, 12)\}$$

En efecto, $(3+3)/3 = 2$, $(3+9)/3 = 4$, $(6+6)/3 = 4$, $(6+12)/3 = 6$, y sus simétricos. (b) Son ocho puntos aislados del plano, simétricos respecto de la recta $y = x$, lo que refleja que la relación es simétrica. La diagonal completa aparece porque $(a+a)/3 = 2a/3$ es par para los cuatro elementos.

In[] :=

```
GraficaRelBinPares[{{3, 3}, {3, 9}, {6, 6}, {6, 12},
{9, 3}, {9, 9}, {12, 6}, {12, 12}}]
```

4.7.8 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 6-Rel. Representación gráfica de algunas relaciones en el plano

Represente en el plano la relación real definida por $a \leq b$, con ambos ejes entre -5 y 5 .

SOLUCIÓN

In[] :=

```
GraficaRelBin["a <= b", 5, 5, xmin -> -5, ymin -> -5]
```

La región es el semiplano que queda por encima de la recta $y = x$, incluida la recta. Los argumentos tercero y cuarto son los extremos máximos de cada eje, y `xmin` e `ymin` cambian los mínimos, que por defecto valen -10 .

4.7.9 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rel. De digrafo a matriz

Sea $A = \{1, 2, 3\}$ y suponga que el digrafo de la relación R sobre A tiene las flechas $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$ y el lazo $3 \rightarrow 3$. Complete la matriz M_R asociada.

SOLUCIÓN La entrada (i, j) vale 1 si hay flecha de i a j :

$$M_R = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

In[] :=

```
MatrizRelBin[{{1, 2}, {2, 3}, {3, 1}, {3, 3}}, {1, 2, 3}, {1, 2, 3},
table -> False]
```

Out[] =

```
{{0, 1, 0}, {0, 0, 1}, {1, 0, 1}}
```

El lazo se traduce en el único 1 de la diagonal. La matriz no es simétrica, de modo que la relación no lo es.

4.7.10 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 8-Rel. De digrafo a pares

Con base en la matriz del ejercicio anterior, determine los pares ordenados de la relación.

SOLUCIÓN

In[] :=

```
RelBinMatriz[{{0, 1, 0}, {0, 0, 1}, {1, 0, 1}}, {1, 2, 3}, {1, 2, 3}]
```

Out[] =

```
{{1, 2}, {2, 3}, {3, 1}, {3, 3}}
```

Se recuperan exactamente las cuatro flechas del digrafo. **MatrizRelBin** y **RelBinMatriz** son operaciones inversas.

4.7.11 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-Rel. Operaciones unión, intersección, inversa, complemento con matrices

Sean $M_{R_1} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$ y $M_{R_2} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{pmatrix}$. Determine la matriz asociada a $R = \overline{R_1 \cup R_2^{-1}}$.

SOLUCIÓN Se procede de dentro hacia fuera: primero el complemento de M_{R_1} , luego la transpuesta de M_{R_2} (que es la inversa), luego la unión booleana y por último el complemento del resultado.

In[] :=

```
M1 = {{0, 1, 0}, {1, 0, 1}, {0, 0, 1}};
M2 = {{1, 0, 0}, {0, 1, 1}, {1, 0, 0}};
ComplementoBooleano[UnionBooleana[ComplementoBooleano[M1, lista -> True],
Transpose[M2], lista -> True], lista -> True]
```

Out[] =

```
{{0, 1, 0}, {1, 0, 1}, {0, 0, 1}}
```

El resultado coincide con M_{R_1} . La opción **lista->True** es imprescindible: sin ella cada comando devuelve un objeto con formato de matriz que el siguiente no puede procesar.

4.7.12 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 9-Rel. Operaciones unión, intersección, inversa, complemento con matrices

Con las mismas matrices, calcule la unión y la intersección booleanas.

SOLUCIÓN

In[] :=

```
M1 = {{0, 1, 0}, {1, 0, 1}, {0, 0, 1}};
M2 = {{1, 0, 0}, {0, 1, 1}, {1, 0, 0}};
UnionBooleana[M1, M2, lista -> True]
InterseccionBooleana[M1, M2, lista -> True]
```

Out[] =

```
{{1, 1, 0}, {1, 1, 1}, {1, 0, 1}}
{{0, 0, 0}, {0, 0, 1}, {0, 0, 0}}
```

La unión marca las posiciones en que al menos una matriz tiene un 1; la intersección, solo aquellas en que ambas lo tienen.

4.7.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-Rel. Operaciones composición, unión, intersección, inversa, complemento

Sean $R_2 = \{(1,2), (1,3), (4,2)\}$ de A en B y $R_1 = \{(2,7), (3,8)\}$ de B en C . Determine cuáles de estos pares están en $R_1 \circ R_2$: $(1,7)$, $(1,8)$, $(4,7)$, $(4,8)$, $(2,7)$.

SOLUCIÓN

In[] :=

```
RelacionComposicion[{{2, 7}, {3, 8}}, {{1, 2}, {1, 3}, {4, 2}}]
```

Out[] =

```
{{1, 7}, {1, 8}, {4, 7}}
```

Están $(1,7)$, $(1,8)$ y $(4,7)$. No está $(4,8)$ porque desde 4 solo se llega a 2, y de 2 solo a 7; y $(2,7)$ tampoco, porque 2 no pertenece al dominio de R_2 . La composición se lee de derecha a izquierda.

4.7.14 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-Rel. Operaciones composición con matrices

Calcule el producto booleano $M_{R_1} \odot M_{R_2}$ con las matrices del ejercicio de operaciones e interprételo.

SOLUCIÓN

In[] :=

```
M1 = {{0, 1, 0}, {1, 0, 1}, {0, 0, 1}};
M2 = {{1, 0, 0}, {0, 1, 1}, {1, 0, 0}};
ProductoBooleano[M1, M2, lista -> True]
```

Out[] =

```
{{0, 1, 1}, {1, 0, 0}, {1, 0, 0}}
```

El producto booleano de las matrices corresponde a la **composición** de las relaciones: la entrada (i,j) vale 1 cuando existe un k tal que i se relaciona con k en la primera y k con j en la segunda.

4.7.15 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-Rel. Clasificación de relaciones

Sea A el conjunto de los enteros pares entre 2 y 10 inclusive, y R la relación sobre A dada por $a R b$ si y solo si a y b dejan el mismo residuo al dividirse entre 4. Determine si R es reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial.

SOLUCIÓN

In[] :=

```
Cj = {2, 4, 6, 8, 10};
R = RelBin["Mod[a, 4] == Mod[b, 4]", Cj, Cj];
ClasificacionRelBin[R, Cj]
```

Out[] =

```
{True, True, False, True}
```

En ese orden: reflexiva **sí**, simétrica **sí**, antisimétrica **no**, transitiva **sí**.

In[] :=

```
Cj = {2, 4, 6, 8, 10};
R = RelBin["Mod[a, 4] == Mod[b, 4]", Cj, Cj];
{RelacionEquivalenciaQ[R, Cj], RelacionOrdenParcialQ[R, Cj]}
```

Out[] =

```
{True, False}
```

Es de equivalencia y no de orden parcial: falla la antisimetría, pues por ejemplo $(2, 6)$ y $(6, 2)$ están ambos y $2 \neq 6$.

4.7.16 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 12-Rel. Relaciones de equivalencia

Para la relación del ejercicio anterior, determine las clases de equivalencia y la partición que inducen sobre A .

SOLUCIÓN

In[] :=

```
Cj = {2, 4, 6, 8, 10};
R = RelBin["Mod[a, 4] == Mod[b, 4]", Cj, Cj];
ParticionReEquivalencia[R, Cj]
```

Out[] =

```
{{2, 6, 10}, {4, 8}}
```

Hay dos clases: la de los que dejan residuo 2 al dividirse entre 4, que es $\{2, 6, 10\}$, y la de los que dejan residuo 0, que es $\{4, 8\}$. Se cumple $[2] = [6] = [10]$ y $[4] = [8]$, y las dos clases son disjuntas y cubren todo A .

4.7.17 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 12-Rel. Relaciones de equivalencia

Recorra el camino inverso: reconstruya la relación de equivalencia a partir de la partición $\{\{2, 6, 10\}, \{4, 8\}\}$.

SOLUCIÓN

In[] :=

```
ReEquivalenciaParticion[{{2, 6, 10}, {4, 8}}, {2, 4, 6, 8, 10}]
```

Out[] =

```
{{2, 2}, {2, 6}, {2, 10}, {4, 4}, {4, 8}, {6, 2}, {6, 6}, {6, 10},
{8, 4}, {8, 8}, {10, 2}, {10, 6}, {10, 10}}
```

Se obtienen los trece pares de la relación original: 9 dentro del primer bloque y 4 dentro del segundo. Las relaciones de equivalencia y las particiones se corresponden una a una.

4.7.18 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-Rel. Dominio de una relación

Sea A el conjunto de los primeros diez números primos y R la relación sobre A dada por $a R b$ si y solo si $a + b < 20$. Indique el dominio de R y su cardinal.

SOLUCIÓN

In[] :=

```
A = Table[Prime[k], {k, 1, 10}];
R = RelBin["a + b < 20", A, A];
{Union[First /@ R], Length[R]}
```

Out[] =

```
{{2, 3, 5, 7, 11, 13, 17}, 32}
```

El dominio llega hasta 17, porque $17 + 2 = 19 < 20$; los primos 19, 23 y 29 quedan fuera. La relación tiene 32 pares de los 100 posibles.

4.7.19 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-Rel. Relaciones y su gráfico

Sean $A = \{10, 15, 21, 28, 33\}$ y $B = \{6, 9, 14, 22\}$. Determine $R = \{(a, b) \mid \gcd(a, b) > 1\}$.

SOLUCIÓN

In[] :=

```
RelBin["GCD[a, b] > 1", {10, 15, 21, 28, 33}, {6, 9, 14, 22}]
```

Out[] =

```
{{10, 6}, {10, 14}, {10, 22}, {15, 6}, {15, 9}, {21, 6}, {21, 9}, {21, 14},
{28, 6}, {28, 14}, {28, 22}, {33, 6}, {33, 9}, {33, 22}}
```

Son 14 pares. El 22 solo se relaciona con los múltiplos de 2 o de 11, y por eso aparece con 10, 28 y 33.

4.7.20 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-Rel. Clasificación de relaciones

Sea $A = \{1, \dots, 8\}$ y R dada por $a R b$ si a y b dejan el mismo residuo al dividirse entre 4. Determine si es reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial, y cuántas clases tiene.

SOLUCIÓN

```
In[ ] :=
```

```
A = Range[8];
R = RelBin["Mod[a, 4] == Mod[b, 4]", A, A];
{ClasificacionRelBin[R, A], RelacionEquivalenciaQ[R, A],
RelacionOrdenParcialQ[R, A], ParticionReEquivalencia[R, A]}
```

```
Out[ ] =
```

```
{{True, True, False, True}, True, False,
{{1, 5}, {2, 6}, {3, 7}, {4, 8}}}
```

Reflexiva, simétrica, transitiva y de equivalencia; no antisimétrica ni de orden parcial. Tiene **cuatro** clases, una por residuo, con dos elementos cada una.

4.7.21 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 12-Rel. Relaciones de equivalencia

Sea $A = \{2,5, 4,5, 6,5, 3, 7, 1\}$ y $\mathcal{R}$ dada por $a \mathcal{R} b$ si $a - b \in \mathbb{Z}$. Señale las proposiciones verdaderas: (a) $2,5 = 6,5$; (b) $\bar{1} = \{1\}$; (c) $\bar{3} = \bar{7}$; (d) hay tres clases.

SOLUCIÓN Son verdaderas (a), (c) y ninguna más: hay **dos** clases, la de los que terminan en ,5 ($\{2,5, 4,5, 6,5\}$) y la de los enteros ($\{3, 7, 1\}$), de modo que (d) es falsa. La (b) también lo es: $\bar{1} = \{3, 7, 1\}$, no $\{1\}$.

```
In[ ] :=
```

```
A = {5/2, 9/2, 13/2, 3, 7, 1};
ParticionReEquivalencia[RelBin["IntegerQ[a - b]", A, A], A]
```

```
Out[ ] =
```

```
{{5/2, 9/2, 13/2}, {1, 3, 7}}
```

Cuidado: escritos como decimales de máquina (2.5 en vez de 5/2) la relación **deja de ser de equivalencia**, porque `IntegerQ[0.]` devuelve `False` y se pierde hasta la reflexividad. Con fracciones exactas el cálculo es correcto.

4.7.22 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-Rel. Operaciones unión, intersección, inversa, complemento

Sean $M_{R_1} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}$ y $M_{R_2} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{pmatrix}$. Determine la matriz de $R = \overline{R_1 \cap R_2^{-1}}$.

SOLUCIÓN

```
In[ ] :=
```

```
M1 = {{0, 1, 1}, {1, 0, 0}, {0, 1, 1}};
M2 = {{1, 0, 1}, {0, 1, 1}, {1, 0, 0}};
ComplementoBooleano[InterseccionBooleana[M1, Transpose[M2], lista -> True],
lista -> True]
```

```
Out[ ] =
```

```
{{1, 1, 0}, {1, 1, 1}, {1, 0, 1}}
```

Se opera de dentro hacia fuera: primero la transpuesta, luego la intersección y por último el complemento. La opción `lista->True` es obligatoria en cada paso para poder encadenar.

4.7.23 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-Rel. Operaciones composición

Sean $R_1 = \{(1, 4), (2, 5), (3, 4), (3, 6)\}$ de A en B y $R_2 = \{(4, 7), (5, 8), (6, 7), (6, 9)\}$ de B en C . Determine cuáles de estos pares están en $R_2 \circ R_1$: $(1, 7)$, $(2, 8)$, $(3, 9)$, $(1, 9)$, $(2, 7)$.

SOLUCIÓN

`In[ ] :=`

```
RelacionComposicion[{ {4, 7}, {5, 8}, {6, 7}, {6, 9} },
{ {1, 4}, {2, 5}, {3, 4}, {3, 6} }]
```

`Out[ ] =`

```
{ {1, 7}, {2, 8}, {3, 7}, {3, 9} }
```

Están $(1, 7)$, $(2, 8)$ y $(3, 9)$. No están $(1, 9)$ (desde 1 solo se llega a 4, y de 4 solo a 7) ni $(2, 7)$.

4.7.24 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-Rel. De digrafo a matriz

Un digrafo sobre $A = \{1, 2, 3, 4\}$ tiene las flechas $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $4 \rightarrow 1$ y los lazos $2 \rightarrow 2$ y $4 \rightarrow 4$. Complete la matriz M_R .

SOLUCIÓN

`In[ ] :=`

```
MatrizRelBin[{ {1, 2}, {2, 3}, {3, 4}, {4, 1}, {2, 2}, {4, 4} },
{1, 2, 3, 4}, {1, 2, 3, 4}, table -> False]
```

`Out[ ] =`

```
{ {0, 1, 0, 0}, {0, 1, 1, 0}, {0, 0, 0, 1}, {1, 0, 0, 1} }
```

Los dos unos de la diagonal corresponden a los lazos. La matriz no es simétrica: el digrafo es realmente dirigido.

4.7.25 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 8-Rel. De digrafo a pares

Recupere los pares de la relación a partir de la matriz del ejercicio anterior.

SOLUCIÓN

`In[ ] :=`

```
RelBinMatriz[{ {0, 1, 0, 0}, {0, 1, 1, 0}, {0, 0, 0, 1}, {1, 0, 0, 1} },
{1, 2, 3, 4}, {1, 2, 3, 4}]
```

`Out[ ] =`

```
{ {1, 2}, {2, 2}, {2, 3}, {3, 4}, {4, 1}, {4, 4} }
```

Se recuperan los seis pares. La lista sale ordenada, aunque el digrafo se describiera en otro orden.

4.7.26 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 3-Rel. Producto cartesiano

Sea $A = \{\emptyset, \{\emptyset\}, \{\{\emptyset\}\}$ y $B = \{1, \emptyset\}$. Indique cuántos elementos tiene $A \times B$ y señale cuáles de estos le pertenecen: $(\emptyset, 1)$, $(1, \emptyset)$, $(\{\emptyset\}, \emptyset)$, $(\emptyset, \{\emptyset\})$.

SOLUCIÓN Tiene $3 \cdot 2 = 6$ elementos. Pertenecen $(\emptyset, 1)$ y $(\{\emptyset\}, \emptyset)$. No pertenece $(1, \emptyset)$ porque 1 no está en A , ni $(\emptyset, \{\emptyset\})$ porque $\{\emptyset\}$ no está en B . Recuérdese que $\emptyset$ y $\{\emptyset\}$ son **objetos distintos**: el segundo es un conjunto con un elemento.

4.7.27 (★★) TIPO EXAMEN A MANO

Categoría del banco: 4-Rel. Producto cartesiano de intervalos

Describa y dibuje el producto cartesiano de $[-2, 3]$ y $[0, +\infty)$.

SOLUCIÓN Es la **banda vertical** de puntos (x, y) con $-2 \leq x \leq 3$ y $y \geq 0$: un rectángulo infinito hacia arriba, apoyado sobre el eje x entre -2 y 3 . Con `GraficaPC` $\{-2, 3\}$, $\{0, \text{Infinity}\}$ el comando dibuja hasta 100 en el eje vertical, porque el infinito no es representable.

4.7.28 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 2-Rel. Graficación de relaciones en el plano cartesiano

Sea $A = \{4, 8, 12, 16, 20\}$ y R dada por $a R b$ si $\frac{a+b}{8}$ es entero. (a) Halle R explícitamente. (b) Describa su gráfica.

SOLUCIÓN (a) Los pares cuya suma es múltiplo de 8:

$$R = \{(4, 4), (4, 12), (4, 20), (8, 8), (8, 16), (12, 4), (12, 12), (12, 20), \\ (16, 8), (16, 16), (20, 4), (20, 12), (20, 20)\}$$

(b) Son trece puntos, simétricos respecto de la recta $y = x$. Se agrupan en dos bloques según el residuo módulo 8: $\{4, 12, 20\}$ y $\{8, 16\}$, lo que da un patrón de dos rejillas rectangulares.

4.7.29 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-Rel. Graficación de relaciones

Compruebe la relación del ejercicio anterior.

SOLUCIÓN

`In[ ] :=`

```
A = {4, 8, 12, 16, 20};
R = RelBin["Mod[a + b, 8] == 0", A, A];
{Length[R], ParticionReEquivalencia[R, A]}
```

`Out[ ] =`

`{13, {{4, 12, 20}, {8, 16}}}`

Trece pares y dos clases, de tamaños 3 y 2: el conteo cuadra con $3^2 + 2^2 = 13$.

4.7.30 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 11-Rel. Clasificación de relaciones

Una relación sobre $\{1, 2, 3, 4\}$ tiene matriz $\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}$. Clasifíquela sin usar el software.

SOLUCIÓN Reflexiva: sí, la diagonal es de unos. **Simétrica:** no, hay un uno en $(1, 2)$ y un cero en $(2, 1)$. **Antisimétrica:** sí, no hay ningún par simétrico fuera de la diagonal. **Transitiva:** sí; se comprueba que todo camino de longitud dos ya está: por ejemplo $(3, 4)$ y $(4, 2)$ exigen $(3, 2)$, que está. Es un **orden parcial**.

4.7.31 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-Rel. Clasificación de relaciones

Compruebe la clasificación anterior.

SOLUCIÓN

In[] :=

```
Mrel = {{1, 1, 0, 0}, {0, 1, 0, 0}, {1, 1, 1, 1}, {0, 1, 0, 1}};
R = RelBinMatriz[Mrel, {1, 2, 3, 4}, {1, 2, 3, 4}];
{ClasificacionRelBin[R, {1, 2, 3, 4}],
RelacionOrdenParcialQ[R, {1, 2, 3, 4}]}
```

Out[] =

```
{{True, False, True, True}, True}
```

Confirma el análisis a mano: reflexiva, antisimétrica, transitiva y por lo tanto de orden parcial.

4.7.32 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 12-Rel. Relaciones de equivalencia

Una relación de equivalencia sobre un conjunto de 9 elementos tiene 29 pares. Determine los tamaños de sus clases.

SOLUCIÓN Hay que escribir 29 como suma de cuadrados cuyas bases sumen 9. Probando: $4^2 + 3^2 + 2^2 = 16 + 9 + 4 = 29$ con $4 + 3 + 2 = 9$. Es la única descomposición, de modo que las clases tienen 4, 3 y 2 elementos. El número de pares de una relación de equivalencia es siempre $\sum n_i^2$.

4.7.33 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 9-Rel. Operaciones con matrices

Demuestre que $\overline{R_1 \cup R_2} = \overline{R_1} \cap \overline{R_2}$.

SOLUCIÓN Un par (a, b) está en el lado izquierdo si y solo si no está en $R_1 \cup R_2$, es decir si no está ni en R_1 ni en R_2 . Eso equivale a estar a la vez en $\overline{R_1}$ y en $\overline{R_2}$, o sea en su intersección. Es la ley de *De Morgan*, que en matrices booleanas se escribe $\overline{M_1 \vee M_2} = \overline{M_1} \wedge \overline{M_2}$.

4.7.34 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-Rel. Operaciones con matrices

Compruebe la ley anterior sobre dos matrices concretas.

SOLUCIÓN

In[] :=

```
M1 = {{1, 0}, {1, 1}}; M2 = {{0, 1}, {1, 0}};
{ComplementoBooleano[UnionBooleana[M1, M2, lista -> True], lista -> True],
InterseccionBooleana[ComplementoBooleano[M1, lista -> True],
```

```
ComplementoBooleano[M2, lista -> True], lista -> True]}
```

Out[] =

```
{{{0, 0}, {0, 0}}, {{0, 0}, {0, 0}}}
```

Los dos lados coinciden. Aquí la unión cubre todas las posiciones, de modo que el complemento es la matriz nula.

4.7.35 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-Rel. Operaciones composición

Sea R una relación sobre A . Demuestre que R es transitiva si y solo si $RoR \subseteq R$.

SOLUCIÓN Si R es transitiva y $(a, c) \in RoR$, existe b con $(a, b), (b, c) \in R$, luego $(a, c) \in R$: la inclusión se cumple. Recíprocamente, si $RoR \subseteq R$ y se tienen $(a, b), (b, c) \in R$, entonces $(a, c) \in RoR \subseteq R$, que es la transitividad. En matrices es exactamente $M \odot M \leq M$.

4.7.36 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-Rel. Representación gráfica de relaciones en el plano

Describa la región del plano de la relación real $a^2 + b^2 \leq 9$ y decida si es reflexiva sobre $\mathbb{R}$.

SOLUCIÓN Es el **disco cerrado** de centro el origen y radio 3. **No** es reflexiva sobre $\mathbb{R}$: el par $(4, 4)$ debería pertenecer y no cumple $32 \leq 9$. Sí lo sería restringida al intervalo $\left[-\frac{3}{\sqrt{2}}, \frac{3}{\sqrt{2}}\right]$, donde $2a^2 \leq 9$.

In[] :=

```
GraficaRelBin["a^2 + b^2 <= 9", 4, 4, xmin -> -4, ymin -> -4]
```

Es simétrica, porque la condición no distingue a de b .

CAPÍTULO 5 • PREGUNTAS TIPO EXAMEN

Teoría de grafos

5.7.1 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-TG. Conceptos básicos de grafos

Sea G el grafo con nodos $\{1, 2, 3, 4, 5, 6\}$ y aristas $\{1, 2\}, \{1, 3\}, \{1, 5\}, \{2, 3\}, \{2, 4\}, \{2, 6\}, \{3, 5\}, \{4, 5\}, \{4, 6\}$ y $\{5, 6\}$. Indique el número de vértices, el de aristas y si es un grafo simple.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
G = Grafo[Ar];
{CantNodos[G], CantAristas[G]}
```

Out[] =

{6, 10}

Tiene seis vértices y diez aristas. **Sí** es simple: no hay lazos ni aristas repetidas, lo que se comprueba observando que las diez parejas son distintas y ninguna repite vértice. Este grafo G se usa en varias de las preguntas que siguen.

5.7.2 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-TG. Valencias de grafos no dirigidos

Complete la tabla de valencias de G :

vértice	1	2	3	4	5	6
valencia	—(i)	—(ii)	—(iii)	—(iv)	—(v)	—(vi)

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Valencias[Grafo[Ar], table -> False]
```

Out[] =

{3, 4, 3, 4, 3, 3}

Los huecos son 3, 4, 3, 4, 3 y 3. La suma vale $20 = 2 \cdot 10$, como exige el teorema de las valencias, y hay cuatro vértices de grado impar, una cantidad par.

5.7.3 (★★) TIPO EXAMEN A MANO

Categoría del banco: 3-TG. Valencias internas/externas

Sea D el digrafo con vértices $\{1, 2, 3, 4\}$ y aristas dirigidas $(1, 2), (1, 3), (2, 3), (3, 1)$ y $(4, 1)$. Calcule la suma de las valencias internas y la de las externas.

SOLUCIÓN La **valencia externa** de un vértice cuenta las flechas que salen y la **interna** las que entran:

	1	2	3	4
externa	2	1	1	1
interna	2	1	2	0

Ambas sumas valen 5, el número de aristas. Es un resultado general: cada flecha aporta exactamente uno a cada suma, de modo que las dos coinciden con $|E|$.

5.7.4 (★) TIPO EXAMEN MIXTO

Categoría del banco: 4-TG. Grafos conexos

¿Es conexo el grafo de referencia G ? ¿Y el grafo con las mismas aristas salvo las que tocan el vértice 6?

SOLUCIÓN G sí es conexo: desde 1 se alcanza cualquier otro vértice. Al eliminar las aristas $\{2, 6\}$, $\{4, 6\}$ y $\{5, 6\}$, el vértice 6 queda aislado y el grafo deja de ser conexo, aunque el resto siga conectado entre sí.

In[] :=

```
ConnectedGraphQ[Graph[{1 <-> 2, 1 <-> 3, 1 <-> 5, 2 <-> 3, 2 <-> 4,
3 <-> 5, 4 <-> 5}, VertexList -> {1, 2, 3, 4, 5, 6}]]
```

Out[] =

False

La instrucción devuelve **False** porque el vértice 6, declarado en la lista de nodos, no tiene ninguna arista.

5.7.5 (★★) TIPO EXAMEN A MANO

Categoría del banco: 5-TG. Número de aristas

Ana asiste a una reunión con 100 personas en total, ella incluida. Cada asistente estrecha la mano de todos los demás exactamente una vez. (a) ¿A cuántas personas les estrechó la mano Ana? (b) ¿Cuántos apretones hubo en total?

SOLUCIÓN (a) A 99: todos menos ella misma. (b) El modelo es el grafo completo K_{100} , y cada apretón es una arista:

$$\frac{100 \cdot 99}{2} = 4950$$

Se divide entre dos porque cada apretón involucra a dos personas y se contaría dos veces al sumar las valencias.

5.7.6 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-TG. Grafos notables

Sea K_n un grafo completo con 153 aristas. Determine cuántos nodos tiene.

SOLUCIÓN Se resuelve $\frac{n(n-1)}{2} = 153$:

In[] :=

```
Solve[n (n - 1)/2 == 153 && n > 0, n]
```

Out[] =

$\{\{n \rightarrow 18\}\}$

El grafo es K_{18} . Comprobación: $\frac{18 \cdot 17}{2} = 153$.

5.7.7

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 7-TG. Circuitos de Euler

Para el grafo de referencia G , decida si posee circuito de *Euler* y si posee ruta de *Euler*, justificando con las valencias.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
G = Grafo[Ar];
{CircuitoEulerQ[G], RutaEulerQ[G]}
```

Out[] =

{False, False}

Las valencias son 3, 4, 3, 4, 3, 3: hay **cuatro** vértices de grado impar. El criterio exige que todas sean pares para el circuito y que haya exactamente dos impares para la ruta, de modo que G no tiene ninguno de los dos.

5.7.8

(★★)

TIPO EXAMEN

A MANO

Categoría del banco: 8-TG. Trayectorias de Euler en K_n

Falso o verdadero: el grafo completo K_{93} posee un circuito de *Euler*; posee una ruta de *Euler*; es conexo; y su número de aristas es ____ (i).

SOLUCIÓN En K_n todas las valencias valen $n - 1$. Para $n = 93$ son 92, todas **pares**, de modo que **sí** hay circuito de *Euler*. Existiendo circuito, también existe recorrido que usa todas las aristas, pero la pregunta por la *ruta* en el sentido estricto del criterio (exactamente dos vértices impares) se responde **falso**. Es conexo, como todo grafo completo. Y el número de aristas es

$$\frac{93 \cdot 92}{2} = 4278$$

5.7.9

(★★)

TIPO EXAMEN

A MANO

Categoría del banco: 9-TG. Trayectorias de Euler en K_{mn}

Falso o verdadero: el grafo bipartito completo $K_{100,501}$ posee un circuito de *Euler*.

SOLUCIÓN Falso. En $K_{n,m}$ los vértices del lado izquierdo tienen valencia m y los del derecho, n . Aquí $m = 501$ es impar, de modo que los cien vértices del primer lado tienen grado impar y el criterio falla. Un $K_{n,m}$ tiene circuito de *Euler* exactamente cuando n y m son ambos pares.

5.7.10

(★)

TIPO EXAMEN

A MANO

Categoría del banco: 10-TG. Circuitos de Hamilton en K_n

Falso o verdadero: el grafo completo K_{99} posee un circuito de *Hamilton*.

SOLUCIÓN Verdadero. En un grafo completo cualquier orden de los vértices define un circuito de *Hamilton*, porque todos los pares están unidos. Basta $n \geq 3$. De hecho K_n tiene $\frac{(n-1)!}{2}$

circuitos distintos.

5.7.11 (★★) TIPO EXAMEN A MANO

Categoría del banco: 11-TG. Circuitos de Hamilton Kmn

Falso o verdadero: $K_{500,500}$ posee un circuito de *Hamilton*. ¿Y $K_{500,501}$?

SOLUCIÓN El primero, **verdadero**; el segundo, **falso**. En un grafo bipartito todo circuito alterna entre los dos lados, de modo que solo puede cerrarse si ambos tienen la **misma cantidad** de vértices. Con 500 y 501 sobraría un vértice sin visitar.

5.7.12 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 12-TG. Análisis de trayectorias y circuitos

Para el grafo de referencia G determine: si posee circuito de *Euler*; si todas sus valencias son pares; si es conexo; y si posee circuito de *Hamilton*.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
G = Grafo[Ar];
{CircuitoEulerQ[G], Divisible[Total[Valencias[G, table -> False]], 2],
 ConnectedGraphQ[G], CircuitoHamiltonQ[G]}
```

Out[] =

```
{False, True, True, True}
```

Circuito de *Euler*: no, por los cuatro vértices impares. La suma de valencias sí es par, pero eso ocurre **siempre** y no aporta nada: lo que importa es que cada valencia lo sea. Es conexo y sí tiene circuito de *Hamilton*, por ejemplo $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 3 \rightarrow 1$.

5.7.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 13-TG. Matriz de adyacencia (Desarrollo)

Escriba la matriz de adyacencia de G en el orden $\{1, 2, 3, 4, 5, 6\}$ y explique cómo se leen las valencias en ella.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True]
```

Out[] =

```
{{0, 1, 1, 0, 1, 0}, {1, 0, 1, 1, 0, 1}, {1, 1, 0, 0, 1, 0},
 {0, 1, 0, 0, 1, 1}, {1, 0, 1, 1, 0, 1}, {0, 1, 0, 1, 1, 0}}
```

Es simétrica y de diagonal nula, como toda matriz de adyacencia de un grafo simple no dirigido. La valencia de cada vértice es la **suma de su fila**: 3, 4, 3, 4, 3, 3.

5.7.14 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 14-TG. Matriz de adyacencia de pesos

Los pesos de las aristas de G , en el orden en que se escribieron, son 4, 2, 6, 3, 5, 7, 1, 2, 4, 3. Indique el peso de la arista $\{2, 6\}$ y el de la arista $\{3, 5\}$, y calcule el peso de la ruta $1 \rightarrow 2 \rightarrow 6$.

SOLUCIÓN La sexta arista es $\{2, 6\}$, con peso 7, y la séptima es $\{3, 5\}$, con peso 1.

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
PesoRuta[Grafo[Ar, pesos -> Pe], {{1, 2}, {2, 6}}]
```

Out[] =

11

El peso de la ruta es $4 + 7 = 11$. En la matriz de adyacencia de pesos, la entrada (i, j) guarda el peso de la arista y un cero significa que no hay arista, motivo por el cual el paquete no admite pesos nulos.

5.7.15 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 15-TG. Matriz de incidencia

Complete la matriz de incidencia del triángulo con vértices $\{1, 2, 3\}$ y aristas $\{1, 2\}$, $\{1, 3\}$ y $\{2, 3\}$.

SOLUCIÓN

In[] :=

```
MIGrafo[Grafo[{{1, 2}, {1, 3}, {2, 3}}], {1, 2, 3}, matriz -> True]
```

Out[] =

```
{{1, 1, 0}, {1, 0, 1}, {0, 1, 1}}
```

Las filas son vértices y las columnas aristas. Cada columna suma 2, porque toda arista tiene dos extremos, y cada fila suma la valencia del vértice, que aquí vale 2 para los tres.

5.7.16 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 16-TG. Rutas de longitud k

Determine el número máximo de rutas de longitud 3 que hay del nodo 1 al nodo 4 en el grafo de referencia G .

SOLUCIÓN La entrada $(1, 4)$ de M_A^3 cuenta esos recorridos:

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
MatrixPower[MGrafo[Grafo[Ar], {1, 2, 3, 4, 5, 6}, matriz -> True], 3][[1, 4]]
```

Out[] =

4

Son cuatro. Estas rutas admiten repetir aristas y vértices, a diferencia de las que cuenta `CantRutas`, que son simples.

5.7.17 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 16-TG. Rutas de longitud k

Cuente todas las rutas simples del nodo 1 al nodo 6 en G .

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
CantRutas[Grafo[Ar], 1, 6]
```

Out[] =

16

Son 16 recorridos que no repiten aristas. El conteo crece muy rápido con el tamaño del grafo, y por eso `GeneraRutas` pide un tope máximo de rutas por devolver.

5.7.18 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 17-TG. Camino más corto

Determine la longitud de un camino más corto entre los nodos 1 y 6 del grafo ponderado, y exhiba una ruta que la alcance.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
LongitudCaminoMC[Grafo[Ar, pesos -> Pe], 1, 6]
```

Out[] =

```
{6., {{1, 3}, {3, 5}, {5, 6}}}
```

La longitud mínima es 6, alcanzada por $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$, con pesos $2 + 1 + 3$. Nótese que la ruta $1 \rightarrow 2 \rightarrow 6$, con solo dos aristas, cuesta 11: el camino con menos aristas no es el más corto.

5.7.19 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 17-TG. Camino más corto

Repita el cálculo entre los nodos 1 y 4, y compárelo con el resultado sin pesos.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
LongitudCaminoMC[Grafo[Ar, pesos -> Pe], 1, 4]
```

Out[] =

```
{5., {{1, 3}, {3, 5}, {5, 4}}}
```

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
LongitudCaminoMC[Grafo[Ar], 1, 6]
```

Out[] =

```
{2, {{1, 2}, {2, 6}}}
```

Con pesos, la longitud de 1 a 4 es 5 por la ruta $1 \rightarrow 3 \rightarrow 5 \rightarrow 4$. Sin pesos la longitud cuenta aristas, y de 1 a 6 bastan dos.

5.7.20 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 18-TG. Camino más largo

Determine la longitud de un camino más largo entre los nodos 1 y 6 del grafo ponderado, y la ruta que la alcanza.

SOLUCIÓN No hay un comando directo: se enumeran todas las rutas simples y se toma la de mayor peso.

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
w[e_] := Pe[[Position[Ar, Sort[e]][[1, 1]]]];
gr = Graph[UndirectedEdge @@@ Ar];
misRutas = FindPath[gr, 1, 6, Infinity, All];
misCostos = Map[Total[Map[w, Partition[#, 2, 1]]] &, misRutas];
{Max[misCostos], misRutas[[Ordering[misCostos, -1][[1]]]}
```

Out[] =

```
{20, {1, 5, 4, 2, 6}}
```

El camino más largo mide 20 y es $1 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6$, con pesos $6 + 2 + 5 + 7$. Hay 16 rutas simples en total, la más barata de peso 6 y la más cara de peso 20. Obsérvese que las variables no se llaman pesos ni rutas: son **símbolos protegidos** de VilCretas, y asignarles un valor hace fallar el cálculo sin previo aviso.

5.7.21 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 12-TG. Análisis de trayectorias y circuitos

Resuelva el problema del agente viajero sobre el grafo ponderado y verifique el resultado.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
AgenteViajero[Grafo[Ar, pesos -> Pe]][[1]]
```

Out[] =

19

La longitud mínima de un circuito de *Hamilton* es 19. Un circuito que la alcanza es $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 3 \rightarrow 1$, y se comprueba con

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pe = {4, 2, 6, 3, 5, 7, 1, 2, 4, 3};
PesoRuta[Grafo[Ar, pesos -> Pe], {{1, 2}, {2, 4}, {4, 6}, {6, 5}, {5, 3}, {3, 1}}]
```

Out[] =

19

Advertencia: la lista de aristas que devuelve **AgenteViajero** en su segunda componente no siempre corresponde a aristas existentes del grafo: en este caso incluye $\{2, 5\}$, que no es arista. De ese comando debe tomarse la **longitud** y obtener el circuito con **CircuitosHamilton**, contrastando el peso con **PesoRuta**.

5.7.22 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 1-TG. Conceptos básicos de grafos

Obtenga un circuito de *Hamilton* concreto del grafo de referencia.

SOLUCIÓN

In[] :=

```
Ar = {{1, 2}, {1, 3}, {1, 5}, {2, 3}, {2, 4}, {2, 6},
      {3, 5}, {4, 5}, {4, 6}, {5, 6}};
CircuitosHamilton[Grafo[Ar], 3]
```

El comando devuelve hasta tres circuitos distintos; el primero es $1 - 2 - 4 - 6 - 5 - 3 - 1$, el mismo que alcanza el peso mínimo del agente viajero. Con **ruta->True** se obtiene la animación del recorrido.

5.7.23 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-TG. Valencia de grafos no dirigidos

Sea $A = \{6, 10, 14, 15, 21, 22\}$. Dos vértices son adyacentes si y solo si $\gcd(a, b) > 1$. Complete la tabla de valencias.

SOLUCIÓN

In[] :=

```
A = {6, 10, 14, 15, 21, 22};
R = Select[RelBin["GCD[a, b] > 1", A, A], #[[1]] < #[[2]] &];
{Length[R], Table[Count[Flatten[R], v], {v, A}]}
```

Out[] =

{11, {5, 4, 4, 3, 3, 3}}

Hay 11 aristas y las valencias son 5, 4, 4, 3, 3, 3, que suman $22 = 2 \cdot 11$. El 6 es adyacente a todos los demás por ser divisible entre 2 y entre 3.

5.7.24 (★★) TIPO EXAMEN A MANO

Categoría del banco: 5-TG. Número de aristas

En un torneo de ajedrez cada participante juega exactamente una vez con cada uno de los demás. Si se jugaron 66 partidas, ¿cuántos participantes hubo?

SOLUCIÓN Se resuelve $\frac{n(n-1)}{2} = 66$, es decir $n^2 - n - 132 = 0$, con raíz positiva $n = 12$. El modelo es K_{12} : cada partida es una arista.

5.7.25 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-TG. Grafos notables

Compruebe el resultado anterior con el software.

SOLUCIÓN

In[] :=

```
{Solve[n (n - 1)/2 == 66 && n > 0, n], CantAristas[GrafoCompleto[12]]}
```

Out[] =

```
{{n -> 12}}, 66}
```

Doce participantes y 66 partidas. Cada uno juega 11 veces, y $12 \cdot 11/2$ evita contar dos veces cada partida.

5.7.26 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-TG. Circuitos de Euler

Considere el grafo G con nodos $\{1, \dots, 5\}$ y aristas $\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 5\}, \{3, 4\}, \{4, 5\}$. Complete la tabla de valencias y decida si tiene circuito de *Euler*.

SOLUCIÓN

In[] :=

```
Ge = Grafo[{{1, 2}, {1, 3}, {1, 4}, {2, 3}, {2, 5}, {3, 4}, {4, 5}}];
{Valencias[Ge, table -> False], CircuitoEulerQ[Ge], RutaEulerQ[Ge]}
```

Out[] =

```
{{3, 3, 3, 3, 2}, False, False}
```

Las valencias son 3, 3, 3, 3, 2: hay **cuatro** vértices impares, de modo que no hay ni circuito ni ruta de *Euler*.

5.7.27 (★★) TIPO EXAMEN A MANO

Categoría del banco: 8-TG. Trayectorias de Euler en K_n

Falso o verdadero para K_{57} : tiene circuito de *Euler*; tiene circuito de *Hamilton*; es conexo; el número de aristas es _____(i).

SOLUCIÓN Todas las valencias valen 56, par: **sí** hay circuito de *Euler*. **Sí** hay circuito de *Hamilton*, como en todo K_n con $n \geq 3$. **Sí** es conexo. Y el número de aristas es

$$\frac{57 \cdot 56}{2} = 1596$$

5.7.28 (★★) TIPO EXAMEN A MANO

Categoría del banco: 9-TG. Trayectorias de Euler en K_{mn}

Falso o verdadero: $K_{34,50}$ tiene circuito de *Euler*; $K_{34,51}$ lo tiene.

SOLUCIÓN El primero **sí**: las valencias son 50 y 34, ambas pares. El segundo **no**: los 34 vértices del primer lado tendrían valencia 51, impar, y además serían más de dos. La regla es que $K_{n,m}$ tiene circuito de *Euler* exactamente cuando n y m son ambos pares.

5.7.29 (★★) TIPO EXAMEN A MANO

Categoría del banco: 11-TG. Circuitos de *Hamilton* K_{mn}

Falso o verdadero: $K_{40,40}$ tiene circuito de *Hamilton*; $K_{40,41}$ lo tiene.

SOLUCIÓN El primero **sí** y el segundo **no**. En un grafo bipartito el circuito alterna lados, de modo que solo puede cerrarse con la misma cantidad de vértices en cada uno. Obsérvese que la condición para *Hamilton* (lados iguales) es distinta de la de *Euler*: ambos pares.

5.7.30 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 12-TG. Análisis de trayectorias y circuitos

Sea G el grafo con 8 vértices y aristas

$\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}, \{5, 6\}, \{5, 7\}, \{6, 7\}, \{6, 8\}, \{7, 8\}$

Decida si tiene circuito de *Euler*, ruta de *Euler*, si es conexo y si tiene circuito de *Hamilton*.

SOLUCIÓN

In[] :=

```
Gg = Grafo[{{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 4}, {4, 5},
{5, 6}, {5, 7}, {6, 7}, {6, 8}, {7, 8}}];
{Valencias[Gg, table -> False], CircuitoEulerQ[Gg], RutaEulerQ[Gg],
ConnectedGraphQ[Gg], CircuitoHamiltonQ[Gg]}
```

Out[] =

```
{{2, 3, 3, 3, 3, 3, 3, 2}, False, False, True, False}
```

Es conexo pero tiene seis vértices impares: no hay *Euler*. Tampoco hay *Hamilton*: la arista $\{4, 5\}$ es un **punto** y su eliminación separa el grafo en dos mitades.

5.7.31 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 12-TG. Análisis de trayectorias y circuitos

Explique por qué un grafo con un puente no puede tener circuito de *Hamilton*.

SOLUCIÓN Un circuito de *Hamilton* visita todos los vértices y vuelve al inicio, de modo que entra y sale de cada lado del puente. Pero el puente es la única conexión entre las dos partes: habría que cruzarlo **dos veces**, y un circuito no repite aristas. Por lo tanto no existe.

5.7.32 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 13-TG. Matriz de adyacencia

Sea G el grafo cuya matriz de adyacencia en el orden $\{1, \dots, 5\}$ es
$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}.$$
 De-

termine sus valencias, su número de aristas y si tiene circuito de *Euler*.

SOLUCIÓN

In[] :=

```
Ma = {{0, 1, 1, 0, 1}, {1, 0, 1, 1, 0}, {1, 1, 0, 1, 0},
{0, 1, 1, 0, 1}, {1, 0, 0, 1, 0}};
Gm = GrafoM[Ma, {1, 2, 3, 4, 5}];
{Total /@ Ma, CantAristas[Gm], CircuitoEulerQ[Gm]}
```

Out[] =

```
{{3, 3, 3, 3, 2}, 7, False}
```

Las valencias son las sumas de las filas. Hay 7 aristas y cuatro vértices impares, de modo que no hay circuito de *Euler*.

5.7.33 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 15-TG. Matriz de incidencia

Complete la matriz de incidencia del grafo con aristas $\{1,2\}$, $\{1,3\}$, $\{2,4\}$ y $\{3,4\}$ sobre $\{1,2,3,4\}$.

SOLUCIÓN

In[] :=

```
MIGrafo[Grafo[{{1, 2}, {1, 3}, {2, 4}, {3, 4}}], {1, 2, 3, 4},
matriz -> True]
```

Out[] =

```
{{1, 1, 0, 0}, {1, 0, 1, 0}, {0, 1, 0, 1}, {0, 0, 1, 1}}
```

Cada fila suma 2, la valencia de su vértice, y cada columna también 2: el grafo es el ciclo C_4 , regular de grado dos.

5.7.34 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 16-TG. Rutas de longitud k

Determine el número máximo de rutas de longitud 4 del nodo 1 al nodo 3 en el grafo de la matriz del ejercicio anterior.

SOLUCIÓN

In[] :=

```
Mb = {{0, 1, 1, 0}, {1, 0, 0, 1}, {1, 0, 0, 1}, {0, 1, 1, 0}};
MatrixPower[Mb, 4][[1, 3]]
```

Out[] =

```
0
```

Son **cero**. El grafo es el ciclo C_4 , que es bipartito: los vértices 1 y 3 están en lados opuestos y toda ruta entre ellos tiene longitud **impar**. Con exponente par la entrada se anula, y con exponente impar no.

5.7.35 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 17-TG. Camino más corto

Sea G con 6 vértices, aristas $\{1,2\}, \{1,3\}, \{2,4\}, \{3,4\}, \{3,5\}, \{4,6\}, \{5,6\}$ y pesos 4, 2, 5, 1, 7, 3, 2. Determine la longitud de un camino más corto entre 1 y 6.

SOLUCIÓN

In[] :=

```
Ac = {{1, 2}, {1, 3}, {2, 4}, {3, 4}, {3, 5}, {4, 6}, {5, 6}};
Pc = {4, 2, 5, 1, 7, 3, 2};
LongitudCaminoMC[Grafo[Ac, pesos -> Pc], 1, 6]
```

Out[] =

```
{6., {{1, 3}, {3, 4}, {4, 6}}}
```

La longitud mínima es 6, por la ruta $1 - 3 - 4 - 6$ con pesos $2 + 1 + 3$. La ruta $1 - 3 - 5 - 6$ costaría $2 + 7 + 2 = 11$.

5.7.36 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 18-TG. Camino más largo

Para el mismo grafo, determine la longitud de un camino más largo entre 1 y 6 y la ruta que la alcanza.

SOLUCIÓN

In[] :=

```
Ac = {{1, 2}, {1, 3}, {2, 4}, {3, 4}, {3, 5}, {4, 6}, {5, 6}};
Pc = {4, 2, 5, 1, 7, 3, 2};
w[e_] := Pc[[Position[Ac, Sort[e]][[1, 1]]]];
misRutas = FindPath[Graph[UndirectedEdge @@@ Ac], 1, 6, Infinity, All];
misCostos = Map[Total[Map[w, Partition[#, 2, 1]]] &, misRutas];
{Max[misCostos], misRutas[[Ordering[misCostos, -1][[1]]]]}
```

Out[] =

```
{19, {1, 2, 4, 3, 5, 6}}
```

El camino más largo mide 19: $1 - 2 - 4 - 3 - 5 - 6$ con pesos $4 + 5 + 1 + 7 + 2$. Se enumeran todas las rutas simples porque no hay un algoritmo eficiente para el camino más largo.

5.7.37 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 18-TG. Camino más largo

¿Por qué no se puede adaptar *Dijkstra* al camino más largo cambiando mínimos por máximos?

SOLUCIÓN Porque se pierde el invariante: al fijar un vértice con la etiqueta *mayor* no hay garantía de que no exista un rodeo aún más largo. De hecho, sin la restricción de no repetir vértices el camino más largo no existe (basta dar vueltas), y con ella el problema es tan difícil como el del agente viajero.

5.7.38 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-TG. Valencias internas/externas

Sea D el digrafo con vértices $\{1, \dots, 5\}$ y aristas dirigidas $(1, 2), (1, 3), (2, 3), (3, 4), (4, 1), (4, 5), (5, 1)$. Calcule la suma de las valencias internas y la de las externas.

SOLUCIÓN

In[] :=

```
Ad = {{1, 2}, {1, 3}, {2, 3}, {3, 4}, {4, 1}, {4, 5}, {5, 1}};
{Table[Count[First /@ Ad, v], {v, 1, 5}],
Table[Count[Last /@ Ad, v], {v, 1, 5}]}
```

Out[] =

```
{{2, 1, 1, 2, 1}, {2, 1, 2, 1, 1}}
```

Ambas suman 7, el número de aristas. Cada flecha aporta exactamente uno a cada total, de modo que las dos sumas siempre coinciden.

5.7.39 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-TG. Grafos conexos

Decida si el grafo con aristas $\{1, 2\}, \{2, 3\}, \{4, 5\}$ sobre $\{1, \dots, 5\}$ es conexo, e indique su número de componentes.

SOLUCIÓN

In[] :=

```
Gc = Graph[{1 <-> 2, 2 <-> 3, 4 <-> 5}];
{ConnectedGraphQ[Gc], Length[ConnectedComponents[Gc]]}
```

Out[] =

```
{False, 2}
```

No es conexo: tiene **dos** componentes, $\{1, 2, 3\}$ y $\{4, 5\}$. Con cinco vértices y solo tres aristas era imposible que fuera conexo, porque hacen falta al menos cuatro.

5.7.40 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-TG. Circuitos de Hamilton en K_n

¿Cuántos circuitos de *Hamilton* distintos tiene K_9 ?

SOLUCIÓN Son $\frac{(9-1)!}{2} = \frac{40320}{2} = 20160$. Se fija un vértice de partida, se permutan los ocho restantes y se divide entre dos porque recorrer el circuito al revés da el mismo conjunto de aristas.

5.7.41 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 1-TG. Conceptos básicos de grafos

Un grafo tiene 7 vértices y su secuencia de valencias es 6, 5, 4, 3, 2, 1, 1. ¿Es posible? Justifique.

SOLUCIÓN La suma es 22, par, de modo que la condición necesaria del teorema de las valencias se cumple, y habría 11 aristas. Pero el vértice de grado 6 es adyacente a los otros seis, de modo que **ninguno** puede tener grado 0 y todos tienen al menos 1: eso sí se cumple. La secuencia es realizable, por ejemplo con un vértice universal y el resto formando la secuencia 4, 3, 2, 1, 0, 0 entre ellos.

5.7.42 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 16-TG. Rutas de longitud k

En el ciclo C_4 con vértices $\{1, 2, 3, 4\}$, determine el número de recorridos de longitud 5 del nodo 1 al nodo 3 y explique el patrón.

SOLUCIÓN

In[] :=

```
Mb = {{0, 1, 0, 1}, {1, 0, 1, 0}, {0, 1, 0, 1}, {1, 0, 1, 0}};  
{MatrixPower[Mb, 3][[1, 3]], MatrixPower[Mb, 4][[1, 3]],  
MatrixPower[Mb, 5][[1, 3]]}
```

Out[] =

```
{0, 8, 0}
```

Con exponente **impar** el resultado es cero y con exponente par no. La razón es la bipartición: en el ciclo C_4 escrito en este orden, 1 y 3 quedan en el **mismo** lado, de modo que toda ruta entre ellos usa una cantidad par de aristas. Con 1 y 2, que están en lados opuestos, el patrón se invierte.

CAPÍTULO 6 • PREGUNTAS TIPO EXAMEN

Teoría de árboles

6.8.1 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-TAr. Info. de un árbol

Considere el árbol con raíz 1 y aristas $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$, $\{2, 5\}$, $\{3, 6\}$, $\{5, 7\}$ y $\{5, 8\}$. Indique: el número de nodos, los nodos terminales, la altura, si es un árbol binario completo y si el nodo 5 es un vértice interno.

SOLUCIÓN

In[] :=

```
Tarb = Arbol[{{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {5, 7}, {5, 8}}];
{CantNodos[Tarb], Hojas[Tarb], Altura[Tarb, 1]}
```

Out[] =

{8, {4, 6, 7, 8}, 3}

Tiene ocho nodos; las hojas son 4, 6, 7 y 8; y la altura es 3, contando la raíz en el nivel cero. **No** es un árbol binario completo: el nodo 3 tiene un solo hijo, y en un binario completo cada vértice interno debe tener exactamente dos. El nodo 5 **sí** es interno, porque tiene los hijos 7 y 8.

6.8.2 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-TAr. Verificar que es un árbol

Un grupo de 100 personas se toma de las manos formando un círculo cerrado. Pensando en cada persona como un nodo y en cada par de brazos entrelazados como una arista, ¿se puede afirmar que el grafo es un árbol?

SOLUCIÓN No. El grafo es un ciclo de 100 vértices y 100 aristas, mientras que un árbol con n vértices tiene exactamente $n - 1$; además, el círculo cerrado es en sí mismo un circuito, y un árbol no tiene ninguno. Si una sola pareja se soltara, quedaría un camino, que sí es un árbol.

In[] :=

ArbolJQ[GrafoCiclo[6], vectorial -> True]

Out[] =

{False, True, True}

El comando lo confirma sobre un ciclo pequeño: no es árbol, aunque sí es conexo, y la tercera componente advierte que **tiene circuitos**.

6.8.3 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 2-TAr. Dibujar árboles en software (Desarrollo)

Escriba el código de *Mathematica*, con **VilCretas**, que dibuja el árbol del primer ejercicio tomando 1 como raíz, y que resalta la ruta de la raíz a la hoja 7.

SOLUCIÓN

In[] :=

```
Tarb = {{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {5, 7}, {5, 8}};
ArbolR[Tarb, 1]
AnimarGrafo[Arbol[Tarb], {{1, 2}, {2, 5}, {5, 7}}]
```

`ArbolR` recibe las aristas y la raíz, y organiza el dibujo por niveles; `AnimarGrafo` resalta paso a paso las aristas de la lista, que aquí es la ruta $1 \rightarrow 2 \rightarrow 5 \rightarrow 7$. La opción `shape` de `ArbolR` cambia la forma de los nodos.

6.8.4 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-TAr. Códigos de Huffman

Considere la palabra *matematica*. (a) Indique la frecuencia de cada carácter. (b) ¿Cuántos bits se necesitan con una codificación de longitud fija? (c) ¿Cuántos con un código de *Huffman* optimizado?

SOLUCIÓN (a) Las frecuencias:

In[] :=

```
ArbolHuffmanN["matematica", frecuencias -> True][[1]]
```

Out[] =

```
{{a, 3}, {m, 2}, {t, 2}, {e, 1}, {i, 1}, {c, 1}}
```

(b) Hay seis caracteres distintos, de modo que un código de longitud fija necesita $\lceil \log_2 6 \rceil = 3$ bits por carácter:

In[] :=

```
10*Ceiling[Log[2, 6]]
```

Out[] =

30

(c) El algoritmo de *Huffman* combina repetidamente las dos frecuencias menores y suma cada combinación:

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
huff[{3, 2, 2, 1, 1, 1}]
```

Out[] =

25

Bastan 25 bits, cinco menos que con longitud fija. El árbol correspondiente se obtiene con `ArbolHuffman["matematica"]`.

6.8.5 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 5-TAr. Expresiones algebraicas y conversión a polaca

Para la expresión $\frac{x+y}{x-2z}$, dibuje el árbol que la representa e indique su notación polaca y su polaca inversa. *Nota:* la respuesta no debe contener comas, comillas ni espacios.

SOLUCIÓN La raíz es la división, con hijos el subárbol de la suma y el de la resta:

$$/ \longrightarrow \{+ \longrightarrow \{x, y\}, \quad - \longrightarrow \{x, * \longrightarrow \{2, z\}\}\}$$

El recorrido prefijo da la **notación polaca**

$$/ + xy - x * 2z$$

y el postfijo, la **polaca inversa**

$$xy + x2z * - /$$

El código que dibuja el árbol es

In[] :=

```
ArbolExpAlgebraica[(x + y)/(x - 2 z)]
```

y `Polaca` y `PolacaInversa` devuelven las dos notaciones. Hay que revisar el árbol que construye el software: internamente representa la división como un producto por una potencia -1 .

6.8.6

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 6-TAr. Almacenamiento de datos numéricos y los tres recorridos

Mediante un árbol binario de búsqueda almacene los datos $\{8, 3, 12, 1, 6, 10, 14, 4, 7\}$ e indique los recorridos prefijo, interfijo y postfijo.

SOLUCIÓN

In[] :=

```
datos = {8, 3, 12, 1, 6, 10, 14, 4, 7};
```

```
Ab = ArbolBinarioBusqueda[datos];
```

```
Prefijo[Ab, 8]
```

Out[] =

```
{8, 3, 1, 6, 4, 7, 12, 10, 14}
```

In[] :=

```
datos = {8, 3, 12, 1, 6, 10, 14, 4, 7};
```

```
Postfijo[ArbolBinarioBusqueda[datos], 8]
```

Out[] =

```
{1, 4, 7, 6, 3, 10, 14, 12, 8}
```

El recorrido **interfijo** de un árbol binario de búsqueda devuelve siempre los datos en orden creciente, de modo que basta ordenarlos:

In[] :=

```
Sort[{8, 3, 12, 1, 6, 10, 14, 4, 7}]
```

Out[] =

```
{1, 3, 4, 6, 7, 8, 10, 12, 14}
```

6.8.7 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-TAr. Almacenamiento de datos numéricos

Para el árbol del ejercicio anterior indique su altura y sus hojas.

SOLUCIÓN

In[] :=

```
Ab = ArbolBinarioBusqueda[{8, 3, 12, 1, 6, 10, 14, 4, 7}];
{Altura[Ab, 8], Hojas[Ab], CantNodos[Ab]}
```

Out[] =

```
{3, {1, 4, 7, 10, 14}, 9}
```

La altura es 3 y hay cinco hojas. Con nueve datos, la altura mínima posible sería 3, de modo que el árbol está bien equilibrado: la búsqueda requiere a lo sumo cuatro comparaciones.

6.8.8 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 7-TAr. Almacenamiento de hileras y los tres recorridos

Almacene la frase *la teoria de arboles* en un árbol de búsqueda y realice un recorrido interfijo.

SOLUCIÓN

In[] :=

```
ArbolBinarioBusqueda["la teoria de arboles"]
```

El comando toma las **palabras** como datos y las compara en orden alfabético. El recorrido interfijo devuelve entonces las palabras ordenadas: *arboles, de, la, teoria*. La raíz es *la*, la primera palabra de la frase.

6.8.9 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-TAr. Árboles generadores (BFS/DFS)

Sea G el grafo con nodos $\{1, \dots, 6\}$ y aristas $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}, \{4, 6\}$ y $\{5, 6\}$. Halle el árbol generador que produce la búsqueda a lo ancho y el que produce la búsqueda a lo largo.

SOLUCIÓN

In[] :=

```
Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
BuscarPrimeroAncho[Grafo[Arb]]
```

Out[] =

```
{{1, 2}, {1, 3}, {2, 4}, {3, 5}, {4, 6}}
```

In[] :=

```
Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
BuscarPrimeroLargo[Grafo[Arb]]
```

Out[] =

$\{\{1, 2\}, \{2, 3\}, \{3, 5\}, \{5, 4\}, \{4, 6\}\}$

Ambos tienen cinco aristas, las $n - 1$ de un árbol con seis vértices; se eliminaron tres de las ocho, justo las que cerraban circuitos. El primero se abre en abanico desde el 1 y el segundo avanza en cadena.

6.8.10 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 8-TAr. Árboles generadores (BFS/DFS)

Repita la búsqueda a lo ancho sobre el mismo grafo con el orden de vértices $\{6, 5, 4, 3, 2, 1\}$.

SOLUCIÓN

In[] :=

```
Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
BuscarPrimeroAncho[Grafo[Arb], orden -> {6, 5, 4, 3, 2, 1}]
```

Out[] =

$\{\{6, 5\}, \{6, 4\}, \{5, 3\}, \{4, 2\}, \{3, 1\}\}$

El árbol generador cambia por completo, aunque conserva las cinco aristas. El orden de exploración determina cuáles aristas entran y cuáles se descartan.

6.8.11 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-TAr. Árboles de expansión mínima

Los pesos de las aristas de G , en el orden en que se escribieron, son 4, 6, 2, 5, 3, 7, 1, 8. (a) Indique el peso del árbol de expansión mínima. (b) Indique cuál es la tercera arista que toma el algoritmo de *Prim*.

SOLUCIÓN (a) Con *Kruskal* o con *Prim* el peso resulta ser 15:

In[] :=

```
Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
Peb = {4, 6, 2, 5, 3, 7, 1, 8};
gg = Graph[UndirectedEdge @@@ Arb, EdgeWeight -> Peb];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gg]]]
```

Out[] =

15

(b) Ejecutando *Prim* sobre el grafo ponderado, el orden en que entran las aristas es $\{1, 2\}$, $\{2, 3\}$, $\{3, 5\}$, $\{2, 4\}$ y $\{4, 6\}$, de modo que la **tercera** es $\{3, 5\}$, de peso 3. *Kruskal* llega al mismo árbol tomando las aristas de menor peso primero: $\{4, 6\}$ con 1, $\{2, 3\}$ con 2, $\{3, 5\}$ con 3, $\{1, 2\}$ con 4 y $\{2, 4\}$ con 5.

6.8.12 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 9-TAr. Árboles de expansión mínima

Ejecute el algoritmo de *Kruskal* sobre ese grafo y observe el procedimiento.

SOLUCIÓN

In[] :=

```

Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
Peb = {4, 6, 2, 5, 3, 7, 1, 8};
Kruskal[Grafo[Arb, pesos -> Peb], all -> True]

```

La salida informa el árbol $T = \{\{4, 6\}, \{2, 3\}, \{3, 5\}, \{1, 2\}, \{2, 4\}\}$ y su peso, 15, además de la animación arista por arista. Con `all->True` se despliega el procedimiento completo.

6.8.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-TAr. Árboles de expansión máxima

Para el mismo grafo ponderado, indique el peso del árbol de expansión máxima.

SOLUCIÓN La estrategia es la misma cambiando el signo de los pesos: el árbol de expansión mínima del grafo con pesos negados es el de expansión máxima del original.

In[] :=

```

Arb = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5},
{4, 6}, {5, 6}};
Peb = {4, 6, 2, 5, 3, 7, 1, 8};
gg = Graph[UndirectedEdge @@@ Arb, EdgeWeight -> Peb];
gm = Graph[UndirectedEdge @@@ Arb, EdgeWeight -> (-Peb)];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gm]]]

```

Out[] =

30

El peso máximo es 30, alcanzado por las aristas $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$, $\{4, 5\}$ y $\{5, 6\}$, con pesos $4 + 6 + 5 + 7 + 8$. Aplicar *Kruskal* tomando las aristas de mayor peso primero da el mismo resultado.

6.8.14 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-TAr. Info. de un árbol

Un árbol binario completo tiene 17 vértices. ¿Cuántas hojas y cuántos vértices internos tiene?

SOLUCIÓN De $2x + 1 = 17$ sale $x = 8$ vértices internos y $x + 1 = 9$ hojas. El total de vértices de un árbol binario completo es siempre impar, de modo que un enunciado con 18 vértices sería imposible.

6.8.15 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 1-TAr. Info. de un árbol

Considere el árbol con raíz 1 y aristas

$$\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 5\}, \{2, 6\}, \{3, 7\}, \{5, 8\}, \{5, 9\}, \{7, 10\}$$

Indique el orden, el número de nodos terminales, la altura, si es completo, si el nodo 2 es interno y cuántos nodos hay en el nivel 1.

SOLUCIÓN

In[] :=

```
T1 = Arbol[{{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {3, 7},
{5, 8}, {5, 9}, {7, 10}}];
{CantNodos[T1], Length[Hojas[T1]], Altura[T1, 1]}
```

Out[] =

```
{10, 5, 3}
```

Orden: 3, porque el nodo 1 tiene tres hijos y ninguno tiene más. **Terminales:** 5. **Altura:** 3. **Completo:** no, el nodo 3 tiene un solo hijo. El nodo 2 **sí** es interno. En el nivel 1 hay **tres** nodos: 2, 3 y 4.

6.8.16 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 3-TAr. Verificar que es un árbol

Un grupo de 60 personas se organiza en cadena, cada una dando la mano a la siguiente, y la última da la mano a la primera. ¿Es un árbol?

SOLUCIÓN

In[] :=

```
{CantNodos[GrafoCiclo[60]], CantAristas[GrafoCiclo[60]],
ArbolJQ[GrafoCiclo[60], vectorial -> True]}
```

Out[] =

```
{60, 60, {False, True, True}}
```

No: tiene 60 aristas y un árbol con 60 vértices tendría 59. El vector confirma que es conexo pero tiene circuitos. Si una sola pareja se soltara, quedaría un camino, que sí es árbol.

6.8.17 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-TAr. Almacenamiento de datos numéricos y los tres recorridos

Mediante un árbol binario de búsqueda almacene {25, 10, 40, 5, 15, 35, 50, 12, 20, 45} e indique los recorridos prefijo, interfijo y postfijo.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{25, 10, 40, 5, 15, 35, 50, 12, 20, 45}];
{Prefijo[A, 25], Postfijo[A, 25]}
```

Out[] =

```
{{25, 10, 5, 15, 12, 20, 40, 35, 50, 45},
{5, 12, 20, 15, 10, 35, 45, 50, 40, 25}}
```

In[] :=

```
Sort[{25, 10, 40, 5, 15, 35, 50, 12, 20, 45}]
```

Out[] =

```
{5, 10, 12, 15, 20, 25, 35, 40, 45, 50}
```

El interfijo es la lista ordenada. La raíz 25 abre el prefijo y cierra el postfijo.

6.8.18 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-TAr. Almacenamiento de datos numéricos

Para ese árbol indique la altura, las hojas y si está balanceado.

SOLUCIÓN

In[] :=

```
A = ArbolBinarioBusqueda[{25, 10, 40, 5, 15, 35, 50, 12, 20, 45}];
Altura[A, 25]
```

Out[] =

3

Altura 3; las hojas son 5, 12, 20, 35 y 45. Con diez datos la altura mínima es 3, de modo que **sí** está balanceado: no hay forma de acomodarlos en menos niveles.

6.8.19 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-TAr. Almacenamiento de hileras y los tres recorridos

Almacene la frase *la teoria de grafos y arboles* en un árbol de búsqueda y dé su recorrido interfijo.

SOLUCIÓN

In[] :=

```
ArbolBinarioBusqueda["la teoria de grafos y arboles"]
```

El comando toma las **palabras** como datos. El recorrido interfijo las devuelve en orden alfabético: *arboles, de, grafos, la, teoria, y*. La raíz es *la*, la primera palabra de la frase.

6.8.20 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-TAr. Códigos de Huffman

Para la palabra *circunferencia*, obtenga las frecuencias y calcule cuántos bits usa un código de *Huffman* optimizado.

SOLUCIÓN

In[] :=

```
ArbolHuffmanN["circunferencia", frecuencias -> True][[1]]
```

Out[] =

```
{{c, 3}, {i, 2}, {r, 2}, {n, 2}, {e, 2}, {u, 1}, {f, 1}, {a, 1}}
```

In[] :=

```
huff[fr_] := Module[{L = Sort[fr], tot = 0, a, b},
  While[Length[L] > 1, a = L[[1]]; b = L[[2]]; tot += a + b;
  L = Sort[Append[Drop[L, 2], a + b]]; tot];
{huff[{3, 2, 2, 2, 2, 1, 1, 1}], 14*3}
```

Out[] =

```
{41, 42}
```

Son 41 bits frente a los 42 de un código fijo de tres bits: un solo bit de ahorro. Con ocho símbolos de frecuencias tan parejas *Huffman* apenas puede mejorar. La compresión rinde solo cuando las frecuencias son desiguales.

6.8.21 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 5-TAr. Expresiones algebraicas y conversión a polaca

Para $\frac{x(y+z)}{x-2w}$ dibuje el árbol e indique su notación polaca. La respuesta no debe llevar comas, comillas ni espacios.

SOLUCIÓN La raíz es la división; su hijo izquierdo es el producto de x por la suma $y+z$, y el derecho la resta $x-2w$. El recorrido prefijo da

$$/ * x + yz - x * 2w$$

y el postfijo, $xyz + *x2w * - /$. El árbol se construye antes de escribir la cadena: el orden de los operadores refleja **qué se evalúa de último**.

6.8.22 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-TAr. Árboles generadores (BFS/DFS)

Sea G con aristas $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}, \{4, 6\}, \{5, 6\}$. Al hacer una búsqueda a lo ancho con el orden $\{6, 5, 4, 3, 2, 1\}$, ¿qué árbol se obtiene?

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
BuscarPrimeroAncho[Grafo[Ag], orden -> {6, 5, 4, 3, 2, 1}]
```

Out[] =

```
{{6, 5}, {6, 4}, {5, 3}, {4, 2}, {3, 1}}
```

Cinco aristas, como todo árbol generador de seis vértices. El árbol resultante cuelga del 6 y es completamente distinto del que se obtiene con el orden natural.

6.8.23 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-TAr. Árboles de expansión mínima

Para ese grafo con pesos 5, 3, 8, 2, 6, 4, 7, 1: (a) indique el peso del árbol de expansión mínima; (b) indique la tercera arista que toma *Kruskal*.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
gg = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> Pg];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gg]]]
```

Out[] =

15

(a) El peso es 15. (b) *Kruskal* ordena por peso: $\{5, 6\}$ con 1, $\{2, 4\}$ con 2 y en **tercer** lugar $\{1, 3\}$ con 3. La cuarta sería $\{4, 5\}$ y la quinta $\{1, 2\}$.

6.8.24 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-TAr. Árboles de expansión máxima

Para el mismo grafo, indique el peso del árbol de expansión máxima y la primera arista que tomaría *Kruskal* adaptado.

SOLUCIÓN

In[] :=

```
Ag = {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}, {4, 6}, {5, 6}};
Pg = {5, 3, 8, 2, 6, 4, 7, 1};
gg = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> Pg];
gm = Graph[UndirectedEdge @@@ Ag, EdgeWeight -> (-Pg)];
Total[PropertyValue[{gg, #}, EdgeWeight] & /@ EdgeList[FindSpanningTree[gm]]]
```

Out[] =

30

El peso máximo es 30. *Kruskal* adaptado toma primero la arista más **cara**: {2, 3} con peso 8, después {4, 6} con 7, y así sucesivamente.

6.8.25 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-TAr. Info. de un árbol

Un árbol binario completo tiene 31 vértices. ¿Cuántas hojas, cuántos internos y cuál es su altura mínima?

SOLUCIÓN De $2x+1 = 31$ sale $x = 15$ internos y 16 hojas. La altura mínima es $\lceil \log_2 32 \rceil - 1 = 4$, y con 31 nodos el árbol queda **lleno**: todos los niveles completos. Es el único caso en que altura mínima y árbol binario completo coinciden exactamente.

6.8.26 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 3-TAr. Verificar que es un árbol

Un grafo tiene 20 vértices y 19 aristas. ¿Es necesariamente un árbol?

SOLUCIÓN **No**. La condición $p = n - 1$ es necesaria pero no suficiente: hace falta además que sea **conexo**. Un triángulo más un camino de 17 vértices tiene 20 vértices y 19 aristas y no es árbol. Con conexión *o* con ausencia de circuitos, la condición sí basta.

6.8.27 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-TAr. Dibujar árboles en software

Escriba el código con **VilCretas** que dibuja el árbol de la primera pregunta con raíz 1 y resalta la ruta de la raíz a la hoja 9.

SOLUCIÓN

In[] :=

```
Ar1 = {{1, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}, {3, 7},
{5, 8}, {5, 9}, {7, 10}};
ArbolR[Ar1, 1]
AnimarGrafo[Arbol[Ar1], {{1, 2}, {2, 5}, {5, 9}}]
```

ArbolR organiza el dibujo por niveles desde la raíz y **AnimarGrafo** resalta la ruta $1 \rightarrow 2 \rightarrow 5 \rightarrow 9$ paso a paso. La opción **shape** de **ArbolR** cambia la forma de los nodos.

6.8.28 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 6-TAr. Almacenamiento de datos numéricos

Un árbol binario de búsqueda con n datos tiene altura $n - 1$. ¿Qué orden de inserción lo produjo y cuántas comparaciones exige la peor búsqueda?

SOLUCIÓN Los datos llegaron **ya ordenados**, de forma ascendente o descendente: cada uno se colgó siempre del mismo lado. La peor búsqueda recorre desde la raíz hasta la única hoja, con n comparaciones, exactamente como una lista sin ordenar.

6.8.29 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-TAr. Árboles generadores (BFS/DFS)

Compare el árbol generador que produce BFS y el que produce DFS sobre el ciclo C_5 .

SOLUCIÓN

$\text{In}[] :=$

`{BuscarPrimeroAncho[GrafoCiclo[5]], BuscarPrimeroLargo[GrafoCiclo[5]]}`

$\text{Out}[] =$

`{{{1, 2}, {1, 5}, {2, 3}, {5, 4}}, {{1, 2}, {2, 3}, {3, 4}, {4, 5}}}`

BFS abre hacia los dos vecinos del 1; DFS recorre el ciclo en un solo sentido. Ambos descartan exactamente una arista, la que cerraba el circuito.

6.8.30 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 9-TAr. Árboles de expansión mínima

En un grafo ponderado con todos los pesos distintos, puede haber dos árboles de expansión mínima?

SOLUCIÓN No: con pesos todos distintos el árbol de expansión mínima es **único**. Si hubiera dos, tomando la arista de menor peso en que difieren y aplicando la propiedad del corte se llega a una contradicción. Con empates sí puede haber varios, y por eso *Kruskal* y *Prim* pueden devolver árboles distintos del mismo peso.

6.8.31 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-TAr. Árboles de expansión máxima

Un grafo ponderado tiene todos sus pesos negativos. ¿Qué relación hay entre su árbol de expansión mínima y el máximo?

SOLUCIÓN Se intercambian respecto del grafo con pesos opuestos: el mínimo del original es el máximo del negado. Lo importante es que ambos existen y tienen $n - 1$ aristas: el signo de los pesos no afecta a *Kruskal* ni a *Prim*, porque solo comparan, no suman. Sí afecta a *Dijkstra*, que necesita pesos no negativos.

6.8.32 (★★) TIPO EXAMEN A MANO

Categoría del banco: 4-TAr. Códigos de Huffman

¿Por qué el árbol de *Huffman* coloca todos los caracteres en las hojas y nunca en un nodo interno?

SOLUCIÓN Para que el código sea **prefijo**: si un carácter estuviera en un nodo interno, su código sería prefijo del de sus descendientes y la decodificación sería ambigua. Al leer bit a bit, se sabe que el carácter terminó exactamente cuando se llega a una hoja.

6.8.33 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 5-TAr. Expresiones algebraicas y conversión a polaca

Reconstruya la expresión cuya polaca inversa es $ab + cd - *e/$.

SOLUCIÓN Se recorre apilando: a, b ; el $+$ da $(a + b)$; c, d ; el $-$ da $(c - d)$; el $*$ da $(a + b)(c - d)$; se apila e ; el $/$ da

$$\frac{(a + b)(c - d)}{e}$$

El último operador de la polaca inversa es siempre la **raíz** del árbol, es decir, la operación que se evalúa de último.

6.8.34 (★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 1-TAr. Info. de un árbol

Un árbol tiene 12 hojas y todos sus vértices internos tienen exactamente tres hijos. ¿Cuántos vértices internos y cuántos nodos tiene en total?

SOLUCIÓN Cada vértice interno aporta tres aristas hacia abajo, de modo que $3t = n - 1$ con $n = t + 12$. Sustituyendo, $3t = t + 11$, de donde $t = 5,5$: **no existe** tal árbol. Con 13 hojas sí: $3t = t + 12$ da $t = 6$ internos y 19 nodos. La condición es que el número de hojas sea congruente con 1 módulo 2 en el caso ternario.

CAPÍTULO 7 • PREGUNTAS TIPO EXAMEN

Máquinas de estado finito y autómatas

7.7.1 (★) TIPO EXAMEN A MANO

Categoría del banco: 1-MqAu. Conceptos básicos de MEF

Responda falso o verdadero: (a) una máquina de estado finito produce una hilera de salida; (b) un autómata tiene seis componentes; (c) la hilera de salida de una máquina tiene la misma longitud que la de entrada; (d) el recorrido de estados tiene un elemento más que la hilera de entrada.

SOLUCIÓN (a) **Verdadero**. (b) **Falso**: tiene cinco, porque en lugar de símbolos de salida y función de salida lleva un conjunto de estados aceptados. (c) **Verdadero**: se emite un símbolo por cada transición. (d) **Verdadero**: el recorrido incluye el estado inicial, antes de leer nada.

7.7.2 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-MqAu. DT de una MEF

Considere la máquina $M = (\sigma, \tau, \delta, \sigma^*, \Delta, \Omega)$ con $\sigma = \{s_0, s_1, s_2\}$, $\tau = \{a, b\}$, $\delta = \{0, 1\}$, estado inicial s_0 y las tablas

	Δ		Ω	
	a	b	a	b
s_0	s_1	s_0	0	1
s_1	s_2	s_0	1	0
s_2	s_2	s_1	0	1

Escriba el código de **VilCretas** que genera su diagrama de transición.

SOLUCIÓN Las dos funciones se escriben juntas, con cuatro columnas por fila: estado, entrada, estado siguiente y salida.

In[] :=

```
FE = {{s0, a, s1, 0}, {s0, b, s0, 1}, {s1, a, s2, 1},
      {s1, b, s0, 0}, {s2, a, s2, 0}, {s2, b, s1, 1}};
MaquinaToDiagrama[{s0, s1, s2}, {a, b}, {0, 1}, s0, FE]
```

En el diagrama cada arista se rotula *entrada/salida*. Esta máquina se usa en los dos ejercicios siguientes.

7.7.3 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-MqAu. Procesando una hilera en una MEF

Procese la hilera $\alpha = abbaa$ en la máquina anterior e indique la hilera de salida.

SOLUCIÓN

In[] :=

```
FE = {{s0, a, s1, 0}, {s0, b, s0, 1}, {s1, a, s2, 1},
      {s1, b, s0, 0}, {s2, a, s2, 0}, {s2, b, s1, 1}};
StringSalida[{s0, s1, s2}, {a, b}, {0, 1}, s0, FE, {a, b, b, a, a}]
```

Out[] =

```
{0, 0, 1, 0, 1}
```

La salida es 00101. El recorrido de estados es $s_0 \rightarrow s_1 \rightarrow s_0 \rightarrow s_0 \rightarrow s_1 \rightarrow s_2$, que se obtiene añadiendo la opción `trace->True`.

7.7.4 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-MqAu. Procesando una hilera en una MEF

Procese ahora *aab* y *bbb* en la misma máquina.

SOLUCIÓN

`In[ ] :=`

```
FE = {{s0, a, s1, 0}, {s0, b, s0, 1}, {s1, a, s2, 1},
      {s1, b, s0, 0}, {s2, a, s2, 0}, {s2, b, s1, 1}};
StringSalida[{s0, s1, s2}, {a, b}, {0, 1}, s0, FE, {a, a, b}]
StringSalida[{s0, s1, s2}, {a, b}, {0, 1}, s0, FE, {b, b, b}]
```

`Out[ ] =`

```
{0, 1, 1}
{1, 1, 1}
```

Con *bbb* la máquina nunca sale de s_0 y emite siempre 1; con *aab* recorre $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_1$.

7.7.5 (★★) TIPO EXAMEN A MANO

Categoría del banco: 3-MqAu. MEF dado el DT

De un diagrama de transición se leen las aristas $s_0 \xrightarrow{a/0} s_1$, $s_0 \xrightarrow{b/1} s_0$, $s_1 \xrightarrow{a/1} s_2$, $s_1 \xrightarrow{b/0} s_0$, $s_2 \xrightarrow{a/0} s_2$ y $s_2 \xrightarrow{b/1} s_1$. Escriba las tablas de Δ y Ω .

SOLUCIÓN Cada arista aporta una entrada a las dos tablas:

	Δ		Ω	
	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>
s_0	s_1	s_0	0	1
s_1	s_2	s_0	1	0
s_2	s_2	s_1	0	1

Hay $3 \cdot 2 = 6$ filas, una por cada par (estado, entrada): si faltara alguna, la máquina quedaría incompleta y el paquete no la procesaría.

7.7.6 (★★) TIPO EXAMEN A MANO

Categoría del banco: 5-MqAu. Determinar si una MEF es un autómata

¿Se puede ver la máquina anterior como un autómata? ¿Con qué conjunto de estados aceptados?

SOLUCIÓN Sí, siempre que las salidas sean 0 y 1 y se interpreten como *rechaza* y *acepta*. Se toma como aceptados los estados a los que se llega emitiendo un 1: aquí s_0 (desde s_0 con *b*), s_2 (desde s_1 con *a*) y s_1 (desde s_2 con *b*). Formalmente se descarta la función de salida y se declara $\hat{A}$; la estructura del diagrama no cambia.

7.7.7 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-MqAu. DT de un autómata

Construya el autómata determinístico sobre $\{a, b, c\}$ que acepta las hileras que terminan en *abc*, y dibuje su diagrama de transición.

SOLUCIÓN Los estados recuerdan cuánto del sufijo se lleva reconocido: s_0 nada, s_1 una a , s_2 un ab y s_3 un abc completo.

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
AutomataToDiagrama[Aut, colores -> True]
```

El estado aceptado s_3 se dibuja con doble círculo. Este autómata se reutiliza en los ejercicios siguientes.

7.7.8 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-MqAu. Procesando una hilera en un autómata

Decida si el autómata anterior acepta abc , $abcabc$ y $abca$.

SOLUCIÓN

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
{StringAceptadaQ[Aut, {a, b, c}], StringAceptadaQ[Aut, {a, b, c, a, b, c}],
StringAceptadaQ[Aut, {a, b, c, a}]}
```

Out[] =

```
{True, True, False}
```

Las dos primeras terminan en abc ; la tercera termina en a y el autómata queda en s_1 , que no es aceptado.

7.7.9 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 8-MqAu. Procesando una hilera en un autómata

Indique el recorrido de estados que sigue la hilera abc .

SOLUCIÓN

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
StringAceptadaQ[Aut, {a, b, c}, trace -> True]
```

Out[] =

```
{True, {s0, s1, s2, s3}}
```

El recorrido pasa por los cuatro estados, uno por cada símbolo leído más el inicial.

7.7.10 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-MqAu. Hilas generales aceptadas por un autómata

Para ese mismo autómata: (a) enumere las hileras de longitud tres que acepta; (b) cuente las de longitud cuatro; (c) cuente los elementos del lenguaje de longitud menor o igual que cuatro.

SOLUCIÓN

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
LenguajeStrings[Aut, 3]
```

Out[] =

```
{{a, b, c}}
```

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
{LenguajeCantidad[Aut, 4], LenguajeCantidad[Aut, 4, limite -> True]}
```

Out[] =

```
{3, 4}
```

(a) Solo abc . (b) Tres: las de la forma $xabc$ con $x \in \{a, b, c\}$. (c) Cuatro en total: la de longitud tres y las tres de longitud cuatro; no hay ninguna más corta, porque el sufijo obligatorio ya mide tres.

7.7.11 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-MqAu. Diseño de autómatas

Diseñe un autómata determinístico sobre $\{a, b\}$ cuyo lenguaje sean las hileras con una cantidad de a múltiplo de tres. Indique el total de estados y el total de estados aceptados.

SOLUCIÓN Se necesitan **tres** estados, uno por cada resto de la división entre tres, y **uno** aceptado, el del resto cero, que es a la vez el inicial:

Δ	a	b
s_0	s_1	s_0
s_1	s_2	s_1
s_2	s_0	s_2

Las b no cambian de estado. La hilera vacía se acepta, porque cero es múltiplo de tres.

7.7.12 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 10-MqAu. Diseño de autómatas

Construya con **AutomataExactamente** el autómata que acepta las hileras con exactamente dos a y una b , y cuente su lenguaje de longitud tres.

SOLUCIÓN

In[] :=

```
AutomataExactamente[{{a, 2}, {b, 1}}];
LenguajeCantidad[G, 3]
```

Out[] =

3

Son aab , aba y baa . El comando dibuja el diagrama y deja el autómata almacenado en la variable G , de modo que hay que consultarlo desde ahí y no desde el resultado de la llamada.

7.7.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-MqAu. DT de un Auto No Det

Construya un autómata **no determinístico** sobre $\{a, b\}$ que acepte las hileras que *contienen* la subhilera ab , y dibuje su diagrama.

SOLUCIÓN El estado q_0 lee cualquier prefijo y *adivina* dónde empieza el ab ; q_2 es una trampa de aceptación.

In[] :=

```
FN = {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}},
      {q1, b, {q2}}, {q2, a, {q2}}, {q2, b, {q2}}};
Nd = Automata[{q0, q1, q2}, {a, b}, q0, FN, {q2}];
AutomataToDiagrama[Nd, colores -> True]
```

El no determinismo se ve en q_0 : con la entrada a salen dos aristas, una hacia sí mismo y otra hacia q_1 . Hay que escribir **todas** las filas, incluida $\{q_1, a, \{\}\}$, cuya imagen es el conjunto vacío.

7.7.14 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 13-MqAu. Hilera aceptada por un AND

Decida si ese autómata no determinístico acepta ab , bba y $babb$.

SOLUCIÓN

In[] :=

```
FN = {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}},
      {q1, b, {q2}}, {q2, a, {q2}}, {q2, b, {q2}}};
Nd = Automata[{q0, q1, q2}, {a, b}, q0, FN, {q2}];
{StringAceptadaQ[Nd, {a, b}], StringAceptadaQ[Nd, {b, b, a}],
 StringAceptadaQ[Nd, {b, a, b, b}]}
```

Out[] =

{True, False, True}

La segunda no contiene ab en ningún lugar; las otras dos sí. Basta con que **alguna** trayectoria termine en un estado aceptado.

7.7.15 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 12-MqAu. Auto No Det dado el DT / 13-MqAu.

Obtenga la trayectoria que sigue la hilera *babb* en ese autómata.

SOLUCIÓN

In[] :=

```
FN = {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}},
      {q1, b, {q2}}, {q2, a, {q2}}, {q2, b, {q2}}};
Nd = Automata[{q0, q1, q2}, {a, b}, q0, FN, {q2}];
StringAceptadaQ[Nd, {b, a, b, b}, trace -> True]
```

Out[] =

```
{True, {{q0}, {q0}, {q0, q1}, {q0, q2}, {q0, q2}}}
```

La traza son **conjuntos** de estados, porque el recorrido se sigue sobre el autómata determinístico equivalente. Al leer la *a* el conjunto se desdobra, y al leer la *b* siguiente aparece *q2*, que ya no se abandona.

7.7.16 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 13-MqAu. Hilera aceptada por un AND

Enumere y cuente las hileras de longitud tres que acepta ese autómata.

SOLUCIÓN

In[] :=

```
FN = {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}},
      {q1, b, {q2}}, {q2, a, {q2}}, {q2, b, {q2}}};
Nd = Automata[{q0, q1, q2}, {a, b}, q0, FN, {q2}];
{LenguajeStrings[Nd, 3], LenguajeCantidad[Nd, 3]}
```

Out[] =

```
{{{a, a, b}, {a, b, a}, {a, b, b}, {b, a, b}}, 4}
```

Son las cuatro hileras de longitud tres que contienen *ab*. De las $2^3 = 8$ posibles quedan fuera *aaa*, *bab* invertido y las que no tienen esa subhilera.

7.7.17 (★★★) TIPO EXAMEN VilCretas

Categoría del banco: 14-MqAu. Auto Det equivalente a un AND

Construya el autómata determinístico equivalente a ese no determinístico.

SOLUCIÓN

In[] :=

```
FN = {{q0, a, {q0, q1}}, {q0, b, {q0}}, {q1, a, {}},
      {q1, b, {q2}}, {q2, a, {q2}}, {q2, b, {q2}}};
Nd = Automata[{q0, q1, q2}, {a, b}, q0, FN, {q2}];
AutomataDeterministicoEquivalente[Nd, onlyreduce -> True]
```

El método de los subconjuntos toma como estados nuevos los conjuntos de estados alcanzables: $\{q_0\}$, $\{q_0, q_1\}$ y $\{q_0, q_2\}$. Con `onlyreduce->True` se descartan los subconjuntos inalcanzables, y el resultado queda en la variable **G**. En el peor de los casos un autómata no determinístico de *n* estados da uno determinístico de hasta 2^n .

7.7.18 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-MqAu. Conceptos básicos de MEF

Una máquina de estado finito tiene cuatro estados y tres símbolos de entrada. ¿Cuántas filas tiene la matriz con que se define en el paquete, y cuántas columnas cada fila?

SOLUCIÓN Tiene $4 \cdot 3 = 12$ filas, una por cada par (estado, entrada), y cada fila tiene **cuatro** columnas: estado, entrada, estado siguiente y salida. Para un autómata las filas serían las mismas pero con tres columnas, porque no hay salida.

7.7.19 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-MqAu. Autómata dado el DT

De un diagrama de transición se leen las aristas $s_0 \xrightarrow{a} s_1$, $s_0 \xrightarrow{b} s_0$, $s_0 \xrightarrow{c} s_0$, $s_1 \xrightarrow{a} s_1$, $s_1 \xrightarrow{b} s_2$, $s_1 \xrightarrow{c} s_0$, $s_2 \xrightarrow{a} s_1$, $s_2 \xrightarrow{b} s_0$, $s_2 \xrightarrow{c} s_3$, $s_3 \xrightarrow{a} s_1$, $s_3 \xrightarrow{b} s_0$ y $s_3 \xrightarrow{c} s_0$, con s_3 dibujado en doble círculo. Indique el estado inicial, el total de estados aceptados y complete la tabla Δ .

SOLUCIÓN El **estado inicial** es s_0 , el único sin flecha de entrada desde fuera del recorrido y al que apunta la marca de inicio; hay **un** estado aceptado, s_3 , señalado por el doble círculo. La tabla de transición recoge las doce aristas, una por cada par (estado, entrada):

Δ	a	b	c
s_0	s_1	s_0	s_0
s_1	s_1	s_2	s_0
s_2	s_1	s_0	s_3
s_3	s_1	s_0	s_0

Se comprueba que el autómata así reconstruido acepta abc y rechaza $abca$, que es el lenguaje de las hileras terminadas en abc :

In[] :=

```
FD = {{s0, a, s1}, {s0, b, s0}, {s0, c, s0},
      {s1, a, s1}, {s1, b, s2}, {s1, c, s0},
      {s2, a, s1}, {s2, b, s0}, {s2, c, s3},
      {s3, a, s1}, {s3, b, s0}, {s3, c, s0}};
Aut = Automata[{s0, s1, s2, s3}, {a, b, c}, s0, FD, {s3}];
[StringAceptadaQ[Aut, {a, b, c}], StringAceptadaQ[Aut, {a, b, c, a}]]
```

Out[] =

{True, False}

Faltaría una sola fila para que el autómata dejase de estar completo, y varios comandos del paquete se quedarían **sin evaluar**.

7.7.20 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 4-MqAu. Procesando una hilera en una MEF

Considere la máquina M con $\sigma = \{s_0, s_1, s_2\}$, $\tau = \{0, 1\}$, $\delta = \{0, 1\}$, inicial s_0 y las transiciones del código. Procese la hilera $\alpha = 01001$.

SOLUCIÓN

In[] :=

```
FM = {{s0, 0, s1, 1}, {s0, 1, s0, 0}, {s1, 0, s2, 0},
      {s1, 1, s0, 1}, {s2, 0, s0, 1}, {s2, 1, s2, 0}};
StringSalida[{s0, s1, s2}, {0, 1}, {0, 1}, s0, FM, {0, 1, 0, 0, 1}]
```

Out[] =

```
{1, 1, 1, 0, 0}
```

La salida es 11100. El recorrido de estados se obtiene añadiendo `trace -> True`, y tiene seis elementos: uno más que la entrada.

7.7.21 (★★) TIPO EXAMEN A MANO

Categoría del banco: 3-MqAu. MEF dado el DT

De un diagrama se leen las aristas $s_0 \xrightarrow{0/1} s_1$, $s_0 \xrightarrow{1/0} s_0$, $s_1 \xrightarrow{0/0} s_2$, $s_1 \xrightarrow{1/1} s_0$, $s_2 \xrightarrow{0/1} s_0$, $s_2 \xrightarrow{1/0} s_2$. Escriba las tablas Δ y Ω .

SOLUCIÓN

	Δ		Ω	
	0	1	0	1
s_0	s_1	s_0	1	0
s_1	s_2	s_0	0	1
s_2	s_0	s_2	1	0

Son $3 \cdot 2 = 6$ filas, una por par (estado, entrada). Es la misma máquina del ejercicio anterior.

7.7.22 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 5-MqAu. Determinar si una MEF es un autómata

¿Bajo qué condiciones puede verse la máquina anterior como un autómata, y cuál sería su conjunto de estados aceptados?

SOLUCIÓN Se descarta la función de salida y se declara $\hat{A}$. Si se interpreta el 1 como *aceptar*, los estados a los que se llega emitiendo 1 son s_1 (desde s_0 con 0), s_0 (desde s_1 con 1 y desde s_2 con 0), de modo que $\hat{A} = \{s_0, s_1\}$. La estructura del diagrama no cambia: solo se reinterpreta.

7.7.23 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-MqAu. DT de un autómata / 9-MqAu. Hileras generales

Construya el autómata que acepta las hileras sobre $\{a, b\}$ con al menos dos a consecutivas y describa su lenguaje en longitud tres.

SOLUCIÓN

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
  {{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
   {q2, a, q2}, {q2, b, q2}}, {q2}];
{LenguajeStrings[D3, 3], LenguajeCantidad[D3, 4]}
```

Out[] =

```
{{{a, a, a}, {a, a, b}, {b, a, a}}, 8}
```

Los estados representan cuántas a consecutivas se llevan: cero, una o **dos o más**. El último es una trampa de aceptación de la que no se sale.

7.7.24 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 8-MqAu. Procesando una hilera en un autómata

Decida si el autómata anterior acepta *baab*, *abab* y *aaab*, e indique el recorrido de la primera.

SOLUCIÓN

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
{q2, a, q2}, {q2, b, q2}}, {q2}];
StringAceptadaQ[D3, {b, a, a, b}], StringAceptadaQ[D3, {a, b, a, b}],
StringAceptadaQ[D3, {a, a, a, b}]
```

Out[] =

{True, False, True}

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
{q2, a, q2}, {q2, b, q2}}, {q2}];
StringAceptadaQ[D3, {b, a, a, b}, trace -> True]
```

Out[] =

{True, {q0, q0, q1, q2, q2}}

Una vez en q_2 el autómata ya no sale: la b final no lo saca de la trampa de aceptación.

7.7.25 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 10-MqAu. Diseño de autómatas

Diseñe un autómata determinístico que acepte las hileras binarias cuyo valor sea múltiplo de 3. Indique el total de estados, el total de aceptados y compruebe su lenguaje en longitud tres.

SOLUCIÓN Se necesitan **tres** estados (los restos módulo 3) y **uno** aceptado, el resto cero, que es también el inicial.

In[] :=

```
D4 = Automata[{q0, q1, q2}, {0, 1}, q0,
{{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
{q2, 0, q1}, {q2, 1, q2}}, {q0}];
LenguajeStrings[D4, 3], LenguajeCantidad[D4, 5]}
```

Out[] =

{{{0, 0, 0}, {0, 1, 1}, {1, 1, 0}}, 11}

En longitud tres se aceptan 000, 011 y 110, que valen 0, 3 y 6.

7.7.26 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 11-MqAu. DT de un Auto No Det / 13-MqAu.

Construya el *ADFN* que acepta las hileras que contienen *aab* y enumere su lenguaje de longitud cuatro.

SOLUCIÓN

In[] :=

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}}];
{StringAceptadaQ[N2, {a, a, b}], LenguajeStrings[N2, 4]}
```

Out[] =

```
{True, {{a, a, a, b}, {b, a, a, b}}}
```

Hay que escribir las ocho filas de la función de transición, incluidas las de imagen vacía: omitir una deja el autómata **sin evaluar**.

7.7.27 (★★) **TIPO EXAMEN** VilCretas

Categoría del banco: 14-MqAu. Auto Det equivalente a un AND

Construya el determinístico equivalente a ese *ADFN* e indique cuántos estados alcanzables tiene.

SOLUCIÓN

In[] :=

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}}];
AutomataDeterministicoEquivalente[N2, onlyreduce -> True]
```

De los $2^4 = 16$ subconjuntos posibles solo cuatro son alcanzables: $\{r_0\}$, $\{r_0, r_1\}$, $\{r_0, r_2\}$ y $\{r_0, r_3\}$. La opción `onlyreduce->True` descarta el resto y el resultado queda en G.

7.7.28 (★★★) **TIPO EXAMEN** A MANO

Categoría del banco: 12-MqAu. Auto No Det dado el DT

De un diagrama de *ADFN* se leen las aristas: desde q_0 con a hacia q_0 y q_1 ; desde q_0 con b hacia q_0 ; desde q_1 con b hacia q_2 ; y ninguna otra. Escriba la función de transición completa.

SOLUCIÓN Hay que escribir las seis filas, incluidas las vacías:

$$\{q_0, a, \{q_0, q_1\}\}, \{q_0, b, \{q_0\}\}, \{q_1, a, \{\}\}, \\ \{q_1, b, \{q_2\}\}, \{q_2, a, \{\}\}, \{q_2, b, \{\}\}$$

El lenguaje es el de las hileras que terminan en ab , con q_2 como único estado aceptado.

7.7.29 (★★) **TIPO EXAMEN** MIXTO

Categoría del banco: 1-MqAu. Conceptos básicos de MEF

Responda falso o verdadero: (a) un *ADFN* puede aceptar una hilera aunque alguno de sus recorridos muera; (b) intercambiar aceptados complementa el lenguaje de un *ADFN*; (c) toda MEF con salidas $\{0, 1\}$ puede verse como autómata; (d) un autómata con cero estados aceptados reconoce el lenguaje vacío.

SOLUCIÓN (a) **Verdadero**: basta que **alguno** llegue a estado aceptado. (b) **Falso**: eso solo vale para determinísticos; hay que determinar primero. (c) **Verdadero**, reinterpretando la salida como aceptación. (d) **Verdadero**: ninguna hilera puede terminar en un estado aceptado.

7.7.30 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 9-MqAu. Hilas generales aceptadas por un autómata

Señale cuáles de estas familias pertenecen al lenguaje del autómata de múltiplos de tres:
 (a) 0^n ; (b) $(11)^n$; (c) $1(01)^n$; (d) $(110)^n$.

SOLUCIÓN

In[] :=

```
D4 = Automata[{q0, q1, q2}, {0, 1}, q0,
{{q0, 0, q0}, {q0, 1, q1}, {q1, 0, q2}, {q1, 1, q0},
{q2, 0, q1}, {q2, 1, q2}}, {q0}];
{StringAceptadaQ[D4, {0, 0, 0, 0}], StringAceptadaQ[D4, {1, 1, 1, 1}],
StringAceptadaQ[D4, {1, 0, 1, 0, 1}], StringAceptadaQ[D4, {1, 1, 0, 1, 1, 0}]}
```

Out[] =

{True, True, True, True}

Pertenecen las **cuatro**. Los valores son 0, 15, 21 y 54, todos múltiplos de tres. Es un buen recordatorio de que hay que **calcular el valor** antes de conjeturar: $1111_2 = 15$ parece un contraejemplo y no lo es.

7.7.31 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 10-MqAu. Diseño de autómatas

Diseñe un autómata determinístico sobre $\{a, b, c\}$ que acepte las hileras que **no** contienen la letra c . ¿Cuántos estados necesita?

SOLUCIÓN Dos: el inicial, que es aceptado y tiene lazos con a y b , y una trampa de rechazo a la que se va con c y de la que no se sale. Es el autómata más pequeño posible para este lenguaje: con uno solo no se podría distinguir el antes y el después de ver una c .

7.7.32 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-MqAu. Autómata dado el DT

Un autómata tiene estados $\{s_0, s_1, s_2\}$, alfabeto $\{a, b\}$, inicial s_0 , aceptado s_1 , y transiciones $s_0 \rightarrow s_1$ con a , $s_0 \rightarrow s_0$ con b , $s_1 \rightarrow s_1$ con a , $s_1 \rightarrow s_2$ con b , $s_2 \rightarrow s_2$ con ambos. Describa su lenguaje.

SOLUCIÓN

In[] :=

```
Ad = Automata[{s0, s1, s2}, {a, b}, s0,
{{s0, a, s1}, {s0, b, s0}, {s1, a, s1}, {s1, b, s2},
{s2, a, s2}, {s2, b, s2}}, {s1}];
{LenguajeStrings[Ad, 3], LenguajeCantidad[Ad, 4]}
```

Out[] =

{{{a, a, a}, {b, a, a}, {b, b, a}}, 4}

El lenguaje es b^*a^+ : cualquier cantidad de b seguida de al menos una a , sin ninguna b posterior. El estado s_2 es una trampa de rechazo.

7.7.33 (★★) TIPO EXAMEN A MANO

Categoría del banco: 2-MqAu. DT de una MEF

Una máquina tiene 4 estados, 3 símbolos de entrada y 2 de salida. ¿Cuántas aristas tiene su diagrama de transición y cómo se rotulan?

SOLUCIÓN Tiene $4 \cdot 3 = 12$ aristas, una por par (estado, entrada), rotuladas *entrada/salida*. Algunas pueden ser lazos, si el estado siguiente coincide con el actual, y varias pueden unir el mismo par de estados con rótulos distintos.

7.7.34 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 5-MqAu. Determinar si una MEF es un autómata

¿Puede una máquina con tres símbolos de salida verse como autómata?

SOLUCIÓN No directamente: la aceptación es binaria. Habría que agrupar los tres símbolos en dos clases (*aceptar* y *rechazar*) y esa elección no es única: con tres símbolos hay $2^3 - 2 = 6$ formas no triviales de hacerlo, y cada una da un lenguaje distinto.

7.7.35 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 14-MqAu. Auto Det equivalente a un AND

Un *ADFN* tiene 6 estados. Acote el número de estados de su determinístico equivalente y diga cuándo se alcanza la cota.

SOLUCIÓN La cota superior es $2^6 = 64$, uno por subconjunto. Se alcanza solo en lenguajes especialmente contruidos, como el de *la n-ésima letra desde el final es una a*. En la práctica la mayoría de los subconjuntos son inalcanzables y el autómata reducido tiene muchos menos estados.

7.7.36 (★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 13-MqAu. Hilera aceptada por un AND

Para el *ADFN* de *aab*, siga la trayectoria de *abaab*.

SOLUCIÓN

In[] :=

```
N2 = Automata[{r0, r1, r2, r3}, {a, b}, r0,
{{r0, a, {r0, r1}}, {r0, b, {r0}}, {r1, a, {r2}}, {r1, b, {}},
{r2, a, {}}, {r2, b, {r3}}, {r3, a, {}}, {r3, b, {}}, {r3}];
StringAceptadaQ[N2, {a, b, a, a, b}, trace -> True]
```

Out[] =

```
{True, {{r0}, {r0, r1}, {r0}, {r0, r1}, {r0, r1, r2}, {r0, r3}}}
```

La traza son conjuntos de estados, porque se sigue sobre el determinístico equivalente. El conjunto final contiene r_3 , que es aceptado.

7.7.37 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 9-MqAu. Hilera generales aceptadas por un autómata

Describa con una expresión general el lenguaje del autómata que acepta las hileras sobre $\{a, b\}$ con al menos dos *a* consecutivas.

SOLUCIÓN Es $\{a, b\}^*aa\{a, b\}^*$: cualquier prefijo, la subhilera *aa*, cualquier sufijo. El número de hileras de longitud n en ese lenguaje es $2^n - c_n$ con c_n el conteo de las libres de *aa*, que sigue la recurrencia de *Fibonacci*.

7.7.38 (★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 9-MqAu. Hileras generales aceptadas por un autómata

Compruebe la relación anterior para $n = 3$ y $n = 4$.

SOLUCIÓN

In[] :=

```
D3 = Automata[{q0, q1, q2}, {a, b}, q0,
{{q0, a, q1}, {q0, b, q0}, {q1, a, q2}, {q1, b, q0},
{q2, a, q2}, {q2, b, q2}}, {q2}];
{LenguajeCantidad[D3, 3], LenguajeCantidad[D3, 4],
2^3 - 5, 2^4 - 8}
```

Out[] =

```
{3, 8, 3, 8}
```

Coinciden: $2^3 - c_3 = 8 - 5 = 3$ y $2^4 - c_4 = 16 - 8 = 8$, con c_n el número de hileras **libres** de aa , que sigue la recurrencia de *Fibonacci* $c_n = c_{n-1} + c_{n-2}$ con $c_1 = 2$ y $c_2 = 3$. Contar el complemento suele ser más fácil que contar directamente.

CAPÍTULO 8 • PREGUNTAS TIPO EXAMEN

Lenguajes y gramáticas

8.6.1 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-LG. Conceptos básicos sobre LyG

Responda falso o verdadero: (a) el lenguaje de un autómata es un ejemplo de lenguaje natural; (b) toda gramática regular es libre del contexto; (c) A^* es finito cuando A lo es; (d) un lenguaje formal puede ser infinito.

SOLUCIÓN (a) **Falso**: es un lenguaje **formal**, definido por reglas precisas, no una lengua humana. (b) **Verdadero**: los tres tipos están encajados, y toda producción regular tiene un único no terminal a la izquierda. (c) **Falso**: A^* contiene hileras de toda longitud, de modo que es infinito aunque A tenga un solo símbolo. (d) **Verdadero**: es cualquier subconjunto de A^* .

8.6.2 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-LG. Verificar que una hilera es elemento de $L(G)$

Considere la gramática G con no terminales $\{S, A\}$, terminales $\{a, b\}$ y producciones $S \rightarrow aA \mid b$, $A \rightarrow bS \mid a$. Decida si pertenecen a $L(G)$ las hileras aa , ab , $abab$ y b .

SOLUCIÓN**In[] :=**

```
G1 = {"S->aA|b", "A->bS|a"};
{ElementoLenguajeQ[G1, "aa"], ElementoLenguajeQ[G1, "ab"],
ElementoLenguajeQ[G1, "abab"], ElementoLenguajeQ[G1, "b"]}
```

Out[] =

```
{True, False, False, True}
```

Pertenecen aa y b . La hilera ab no: tras aplicar $S \rightarrow aA$ hay que seguir con $A \rightarrow bS$, que deja pendiente el no terminal S , o con $A \rightarrow a$, que daría aa . La derivación de aa es $S \Rightarrow aA \Rightarrow aa$.

8.6.3 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 5-LG. Lenguaje asociado a una G libre de contexto

Genere las palabras del lenguaje de esa gramática con a lo sumo cinco derivaciones y longitud máxima cinco.

SOLUCIÓN**In[] :=**

```
ElementosLenguaje[{"S->aA|b", "A->bS|a"}, 5, 5]
```

Out[] =

```
{aa, abaa, ababb, abb, b}
```

Las palabras alternan bloques ab y terminan en a o en b según cuál producción cierre la derivación. Subiendo los dos parámetros aparecen más elementos: el lenguaje es infinito.

8.6.4 (★★) TIPO EXAMEN VilCretas

Categoría del banco: 3-LG. Notación BNF

Escriba esa gramática en notación de *Backus-Naur*.

SOLUCIÓN

In[] :=

```
NotationBNF[{"S->aA|b", "A->bS|a"}]
```

La salida es $\langle S \rangle ::= a \langle A \rangle | b$ y $\langle A \rangle ::= b \langle S \rangle | a$: los no terminales van entre ángulos y la flecha se sustituye por $::=$. Con `NotationFNormal` se obtiene en cambio la 4-tupla $(\sigma, \tau, P, \sigma^*)$ de la forma normal.

8.6.5

(★★★)

TIPO EXAMEN

A MANO

Categoría del banco: 4-LG. Tipos de gramática

Clasifique cada gramática en regular, libre del contexto o sensible al contexto, tomando en cada caso el tipo más restrictivo que le corresponda: (a) $S \rightarrow aS \mid b$; (b) $S \rightarrow aSb \mid \varepsilon$; (c) $cA \rightarrow cB$; (d) $S \rightarrow abS$.

SOLUCIÓN (a) **Regular**: cada producción tiene la forma $\alpha \rightarrow x\beta$ o $\alpha \rightarrow x$ con x un **único** terminal. (b) **Libre del contexto** y no regular: el terminal b queda a la derecha del no terminal. (c) **Sensible al contexto**: el lado izquierdo no es un solo no terminal, sino que lleva el contexto c . (d) **Libre del contexto** y no regular según la definición estricta, porque el lado derecho empieza con **dos** terminales. Reescrita como $S \rightarrow aA$, $A \rightarrow bS$ sí sería regular y generaría el mismo lenguaje.

8.6.6

(★★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 7-LG. Diseñar gramáticas (Desarrollo)

Diseñe una gramática regular sobre $\{a, b, c\}$ cuyo lenguaje sean las hileras que terminan en abc , y compruebe su lenguaje de longitud cuatro.

SOLUCIÓN Se necesitan estados intermedios que reconozcan el sufijo un símbolo a la vez, porque una gramática regular solo admite un terminal por producción:

$$S \rightarrow aS \mid bS \mid cS \mid aA, \quad A \rightarrow bB, \quad B \rightarrow c$$

In[] :=

```
G2 = {"S->aS|bS|cS|aA", "A->bB", "B->c"};
ElementosLenguaje[G2, 6, 4]
```

Out[] =

```
{aabc, abc, babc, cabc}
```

Con longitud máxima cuatro se obtienen abc y las tres hileras de la forma $xabc$. Escribir $S \rightarrow abc$ sería más corto pero **no sería regular**: la forma admitida lleva un solo terminal.

8.6.7

(★★)

TIPO EXAMEN

MIXTO

Categoría del banco: 2-LG. Verificar que una hilera es elemento de $L(G)$

Para la gramática anterior, decida si abc , $ababc$ y $abca$ pertenecen al lenguaje.

SOLUCIÓN

In[] :=

```
G2 = {"S->aS|bS|cS|aA", "A->bB", "B->c"};
{ElementoLenguajeQ[G2, "abc"], ElementoLenguajeQ[G2, "ababc"],
ElementoLenguajeQ[G2, "abca"]}
```

Out[] =

```
{True, True, False}
```

Las dos primeras terminan en *abc*; la tercera termina en *a* y por eso se rechaza.

8.6.8 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-LG. $L(G')$ mediante un AEF

Construya el autómata de estado finito asociado a esa gramática y compruebe que reconoce el mismo lenguaje.

SOLUCIÓN

In[] :=

```
G2 = {"S->aS|bS|cS|aA", "A->bB", "B->c"};
RegularToAutomata[G2];
{StringAceptadaQ[G, {a, b, c}], StringAceptadaQ[G, {a, b, c, a}]}
```

Out[] =

```
{True, False}
```

In[] :=

```
G2 = {"S->aS|bS|cS|aA", "A->bB", "B->c"};
RegularToAutomata[G2];
LenguajeStrings[G, 4]
```

Out[] =

```
{{a, a, b, c}, {b, a, b, c}, {c, a, b, c}}
```

Los estados del autómata son los no terminales más un estado de aceptación *EA*, al que llevan las producciones que terminan la derivación, como $B \rightarrow c$. Las respuestas coinciden con las de la gramática: los dos modelos describen el mismo lenguaje.

8.6.9 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-LG. Lenguaje asociado a una G libre de contexto

Considere $S \rightarrow aSbb \mid \varepsilon$. (a) Describa su lenguaje. (b) Genere sus elementos de longitud a lo sumo ocho. (c) Decida si *aabbbb* y *aabb* pertenecen.

SOLUCIÓN (a) Cada derivación agrega una *a* por delante y **dos** *b* por detrás, de modo que el lenguaje es $\{a^n b^{2n} \mid n \geq 0\}$. (b)

In[] :=

```
ElementosLenguaje[{"S->aSbb|[CurlyEpsilon]"}, 6, 8]
```

Out[] =

```
{, aabbbb, abb}
```

El primer elemento es la hilera vacía. (c)

In[] :=

```
{ElementoLenguajeQ[{"S->aSbb|[CurlyEpsilon]"}, "aabbbb"],
ElementoLenguajeQ[{"S->aSbb|[CurlyEpsilon]"}, "aabb"]}
```

Out[] =

{True, False}

aabbbb sí, con $n = 2$; *aabb* no, porque necesitaría cuatro *b*. Este lenguaje es libre del contexto pero **no regular**: ningún autómata de estado finito puede llevar la cuenta de n .

8.6.10 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-LG. Conceptos básicos sobre LyG

Sobre un alfabeto de tres símbolos: (a) ¿cuántas hileras de longitud dos hay? (b) ¿Cuántas de longitud menor o igual que dos? (c) ¿Cuántos lenguajes distintos pueden formarse con esas hileras?

SOLUCIÓN (a) $3^2 = 9$. (b) $3^0 + 3^1 + 3^2 = 1 + 3 + 9 = 13$. (c) Un lenguaje es cualquier subconjunto de esas 13 hileras, de modo que hay $2^{13} = 8192$. Sobre A^* completo la cantidad de lenguajes deja de ser numerable.

8.6.11 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 7-LG. Diseñar gramáticas (Desarrollo)

Diseñe una gramática regular en notación *BNF* cuyo lenguaje sean las hileras sobre $\{a, b\}$ con una cantidad par de a .

SOLUCIÓN Con dos no terminales, S para *par* y A para *impar*, aceptando cuando la derivación termina estando en S :

$$\langle S \rangle ::= b\langle S \rangle \mid a\langle A \rangle \mid b, \quad \langle A \rangle ::= b\langle A \rangle \mid a\langle S \rangle \mid a$$

La producción $S \rightarrow b$ permite cerrar con cero a , y $A \rightarrow a$ cierra con una cantidad par en total. Es la traducción directa del autómata de la paridad: cada transición da una producción, y las que llegan a un estado aceptado dan además una producción con un solo terminal.

8.6.12 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-LG. Lenguaje asociado a una G libre de contexto

Genere el lenguaje de $S \rightarrow aSa \mid bSb \mid c$ hasta longitud cinco y descríbalos.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aSa|bSb|c"}, 4, 5]
```

Out[] =

{aaca, abcba, aca, bacab, bbcbb, bcb, c}

Son los palíndromos de longitud impar con una única c en el centro. El lenguaje es libre del contexto y **no regular**: habría que recordar toda la primera mitad.

8.6.13 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-LG. Verificar que una hilera es elemento de $L(G)$

Para esa gramática, decida si *abcba*, *abcb* y *bbcbb* pertenecen al lenguaje.

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aSa|bSb|c"}, "abcba"],
ElementoLenguajeQ[{"S->aSa|bSb|c"}, "abcbab"],
ElementoLenguajeQ[{"S->aSa|bSb|c"}, "bbcbb"]}
```

Out[] =

```
{True, False, True}
```

La segunda no es palíndromo. Verificar es tan simple como leer la hilera al revés y comparar.

8.6.14 (★★) **TIPO EXAMEN** VilCretas

Categoría del banco: 3-LG. Notación BNF

Escriba en notación *BNF* la gramática $S \rightarrow aB \mid bA$, $A \rightarrow a \mid aS$, $B \rightarrow b \mid bS$.

SOLUCIÓN

In[] :=

```
NotationBNF[{"S->aB|bA", "A->a|aS", "B->b|bS"}]
```

La salida es $\langle S \rangle ::= a \langle B \rangle \mid b \langle A \rangle$, $\langle A \rangle ::= a \mid a \langle S \rangle$ y $\langle B \rangle ::= b \mid b \langle S \rangle$. Los no terminales van entre ángulos y la flecha se sustituye por $::=$.

8.6.15 (★★★) **TIPO EXAMEN** A MANO

Categoría del banco: 4-LG. Tipos de gramática

Clasifique en el tipo más restrictivo posible: (a) $S \rightarrow 0S1 \mid 01$; (b) $bAc \rightarrow bBc$; (c) $S \rightarrow 0S \mid 1S \mid 0$; (d) $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$.

SOLUCIÓN (a) **Libre del contexto**: un no terminal a la izquierda, pero el terminal 1 queda a la derecha del no terminal. (b) **Sensible al contexto**: el lado izquierdo lleva contexto. (c) **Regular**: cada producción es $\alpha \rightarrow x\beta$ o $\alpha \rightarrow x$. (d) **Libre del contexto**: dos no terminales en el lado derecho.

8.6.16 (★★★) **TIPO EXAMEN** MIXTO

Categoría del banco: 5-LG. Lenguaje asociado a una *G* libre de contexto

Genere el lenguaje de $S \rightarrow 0S1 \mid 01$ y decida si 000111 y 0101 pertenecen.

SOLUCIÓN

In[] :=

```
{ElementosLenguaje[{"S->0S1|01"}, 5, 8],
ElementoLenguajeQ[{"S->0S1|01"}, "000111"],
ElementoLenguajeQ[{"S->0S1|01"}, "0101"]}
```

Out[] =

```
{{00001111, 000111, 0011, 01}, True, False}
```

El lenguaje es $\{0^n 1^n \mid n \geq 1\}$. La hilera 0101 tiene la misma cantidad de cada símbolo pero **intercalados**, y no pertenece.

8.6.17 (★★★) **TIPO EXAMEN** MIXTO

Categoría del banco: 6-LG. $L(G)$ mediante un AEF

Construya la gramática regular del autómata que acepta las hileras terminadas en ab , con estados $\{S, A, B\}$.

SOLUCIÓN

In[] :=

```
Aut = Automata[{S, A, B}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, A}, {A, b, B},
  {B, a, A}, {B, b, S}}, {B}];
AutomataToRegular[Aut, componentes -> True]
```

Out[] =

```
{{S, A, B}, {a, b}, {A->aA, A->b, A->bB, B->aA, B->bS, S->aA, S->bS}, S}
```

Siete producciones: una por transición, más $A \rightarrow b$ porque la transición de A con b llega al estado aceptado.

8.6.18 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-LG. $L(G)$ mediante un AEF

Compruebe que ese autómata y su gramática reconocen el mismo lenguaje en longitud tres.

SOLUCIÓN

In[] :=

```
Aut = Automata[{S, A, B}, {a, b}, S,
  {{S, a, A}, {S, b, S}, {A, a, A}, {A, b, B},
  {B, a, A}, {B, b, S}}, {B}];
LenguajeStrings[Aut, 3]
```

Out[] =

```
{{a, a, b}, {b, a, b}}
```

Las dos hileras de longitud tres terminadas en ab . La gramática genera exactamente esas.

8.6.19 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 7-LG. Diseñar gramáticas (Desarrollo)

Diseñe una gramática regular en notación BNF para las hileras sobre $\{a, b, c\}$ que terminan en bca .

SOLUCIÓN Se necesita un no terminal por cada prefijo del sufijo reconocido:

$$\langle S \rangle ::= a\langle S \rangle \mid b\langle S \rangle \mid c\langle S \rangle \mid b\langle A \rangle$$

$$\langle A \rangle ::= c\langle B \rangle, \quad \langle B \rangle ::= a$$

S deja pasar cualquier prefijo; A recuerda que se vio b y B que se vio bc . Escribir $S \rightarrow bca$ sería más corto pero **no sería regular**.

8.6.20 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-LG. Diseñar gramáticas

Compruebe la gramática anterior en longitud cuatro.

SOLUCIÓN

In[] :=

ElementosLenguaje[{"S->aS|bS|cS|bA", "A->cB", "B->a"}, 6, 4]

Out[] =

{abca, bbca, bca, cbca}

Las tres hileras de longitud cuatro terminadas en *bca*, más la propia *bca* de longitud tres. Con tres letras en el alfabeto hay tres prefijos posibles.

8.6.21 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 1-LG. Conceptos básicos sobre LyG

Responda falso o verdadero: (a) toda gramática finita genera un lenguaje finito; (b) $\emptyset^* = \emptyset$; (c) existen lenguajes sin gramática; (d) toda gramática regular es libre del contexto.

SOLUCIÓN (a) **Falso**: $S \rightarrow aS \mid b$ es finita y genera infinitas palabras. (b) **Falso**: $\emptyset^* = \{\varepsilon\}$, por el caso $k = 0$. (c) **Verdadero**: las gramáticas son numerables y los lenguajes no. (d) **Verdadero**: los tipos están encajados.

8.6.22 (★★★) **TIPO EXAMEN** **MIXTO**

Categoría del banco: 5-LG. Lenguaje asociado a una G libre de contexto

Genere el lenguaje de $S \rightarrow aS \mid bS \mid aa$ y descríballo.

SOLUCIÓN

In[] :=

ElementosLenguaje[{"S->aS|bS|aa"}, 4, 4]

Out[] =

{aa, aaa, aaaa, abaa, baa, baaa, bbaa}

Son las hileras que **terminan** en *aa*. De longitud cuatro hay cuatro: *aaaa*, *abaa*, *baaa* y *bbaa*, una por cada prefijo de dos símbolos.

8.6.23 (★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 1-LG. Conceptos básicos sobre LyG

Sobre un alfabeto de cuatro símbolos, ¿cuántas hileras hay de longitud exactamente tres, y cuántas de longitud menor o igual que tres?

SOLUCIÓN De longitud tres hay $4^3 = 64$. Acumulando: $1 + 4 + 16 + 64 = 85$. La suma es geométrica de razón 4, y en general vale $\frac{k^{n+1} - 1}{k - 1}$ con k símbolos.

8.6.24 (★★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 4-LG. Tipos de gramática

¿Por qué $S \rightarrow abS$ no es regular en sentido estricto, y cómo se reescribe para que lo sea?

SOLUCIÓN Porque el lado derecho tiene **dos** terminales antes del no terminal, y la forma regular admite uno solo. Se reescribe introduciendo un auxiliar:

$$S \rightarrow aA, \quad A \rightarrow bS$$

El lenguaje generado es el mismo. La técnica general añade un no terminal por cada terminal sobrante.

8.6.25 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 2-LG. Verificar que una hilera es elemento de $L(G)$

Para $S \rightarrow aaS \mid b$, decida si pertenecen aab , $aaab$ y b .

SOLUCIÓN

In[] :=

```
{ElementoLenguajeQ[{"S->aaS|b"}, "aab"],
ElementoLenguajeQ[{"S->aaS|b"}, "aaab"],
ElementoLenguajeQ[{"S->aaS|b"}, "b"]}
```

Out[] =

```
{True, False, True}
```

El lenguaje es $\{a^{2n}b\}$: la cantidad de a debe ser par. La hilera b corresponde a $n = 0$.

8.6.26 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 7-LG. Diseñar gramáticas (Desarrollo)

Diseñe una gramática regular para las hileras sobre $\{0, 1\}$ que representan múltiplos de 4 en binario.

SOLUCIÓN Un binario es múltiplo de 4 si termina en 00:

$$S \rightarrow 0S \mid 1S \mid 0A, \quad A \rightarrow 0$$

Las dos primeras dejan pasar cualquier prefijo y las dos últimas obligan a cerrar con dos ceros. La hilera 00 pertenece y representa el cero.

8.6.27 (★★) TIPO EXAMEN MIXTO

Categoría del banco: 7-LG. Diseñar gramáticas

Compruebe la gramática de los múltiplos de cuatro.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->0S|1S|0A", "A->0"}, 5, 4]
```

Out[] =

```
{00, 000, 0000, 0100, 100, 1000, 1100}
```

Todas terminan en 00. De longitud cuatro hay cuatro, la cuarta parte de las dieciséis posibles, coherente con que uno de cada cuatro números es múltiplo de cuatro.

8.6.28 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 6-LG. $L(G)$ mediante un AEF

Resuma el ciclo completo entre gramáticas regulares y autómatas, y diga qué se conserva en cada paso.

SOLUCIÓN

1. De gramática regular a *ADFN*, con `RegularToAutomata`;

2. de *ADFN* a determinístico, con `AutomataDeterministicoEquivalente`;
3. de determinístico a gramática regular, con `AutomataToRegular`.

En los tres pasos se conserva el **lenguaje**, no la gramática ni el número de estados: la gramática final puede ser muy distinta de la inicial.

8.6.29 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 6-LG. $L(G)$ mediante un AEF

Construya el autómata de $S \rightarrow aA \mid b$, $A \rightarrow bS \mid a$ y compruebe dos hileras.

SOLUCIÓN

In[] :=

```
RegularToAutomata[{"S->aA|b", "A->bS|a"}];
{StringAcceptadaQ[G, {a, a}], StringAcceptadaQ[G, {a, b}]}
```

Out[] =

```
{True, False}
```

El autómata queda en `G` y añade un estado de aceptación *EA* para las producciones que terminan la derivación. Acepta *aa* y rechaza *ab*, igual que la gramática.

8.6.30 (★★) TIPO EXAMEN A MANO

Categoría del banco: 1-LG. Conceptos básicos sobre LyG

¿Qué significa que un lenguaje sea *regular* en términos de la memoria necesaria para reconocerlo?

SOLUCIÓN Que basta una cantidad **acotada** de memoria: un número finito de estados, fijado de antemano e independiente de la longitud de la entrada. Los lenguajes libres del contexto necesitan además una pila, cuya altura crece con la entrada, y por eso $\{a^n b^n\}$ no es regular.

8.6.31 (★★★) TIPO EXAMEN A MANO

Categoría del banco: 4-LG. Tipos de gramática

Una gramática tiene las producciones $S \rightarrow aSb$, $S \rightarrow ab$, $aS \rightarrow aa$. ¿De qué tipo es y por qué?

SOLUCIÓN Es **sensible al contexto**, por la tercera producción: su lado izquierdo *aS* no es un solo no terminal. Las otras dos son libres del contexto, pero el tipo de una gramática lo determina la producción **más general** que contenga.

8.6.32 (★★★) TIPO EXAMEN MIXTO

Categoría del banco: 5-LG. Lenguaje asociado a una *G* libre de contexto

Genere el lenguaje de $S \rightarrow aA$, $A \rightarrow bA \mid \varepsilon$ y descríbalos con una expresión general.

SOLUCIÓN

In[] :=

```
ElementosLenguaje[{"S->aA", "A->bA|"[CurlyEpsilon]}, 5, 4]
```

Out[] =

```
{a, ab, abb}
```


Es $\{ab^n \mid n \geq 0\}$: una a seguida de cualquier cantidad de b . La producción $A \rightarrow \varepsilon$ permite el caso $n = 0$, que da la palabra a .

8.6.33 (★★) **TIPO EXAMEN** **A MANO**

Categoría del banco: 3-LG. Notación BNF

Traduzca a notación *BNF* la gramática $S \rightarrow 0S \mid 1S \mid 0$ e indique cuántas alternativas tiene su única regla.

SOLUCIÓN

$$\langle S \rangle ::= 0\langle S \rangle \mid 1\langle S \rangle \mid 0$$

Tiene **tres** alternativas separadas por barras. La notación *BNF* agrupa en una sola línea todas las producciones de un mismo no terminal, lo que la hace mucho más compacta que la forma normal.

APÉNDICES

Recursos, funciones y mapa de estudio

APÉNDICE A

Los recursos del curso

El curso no vive solo en el libro. A su alrededor hay cuadernos de Mathematica, videos de las clases, presentaciones y documentos de apoyo que llevan años acumulándose. Este apéndice los reúne en un solo lugar y dice, de cada uno, qué es y a qué capítulo sirve, para que el estudiante no tenga que adivinarlo.

A.1 Los cuadernos y documentos de *Materiales importantes ED*

Son 49 archivos, y viajan con el cuadernillo: están en la subcarpeta **Materiales** de **Entregables**. Los enlaces de esta sección abren cada archivo directamente, y funcionan mientras el PDF y esa subcarpeta sigan juntos. Si alguno no abriera, ahí está la [carpeta completa](#).

Cuadernos con las soluciones de los ejercicios del libro. Uno por capítulo, con el mismo patrón siempre: una celda de texto con el número del ejercicio, una celda de entrada con la definición y una celda de comprobación del tipo `Table[f[n] == ...]` que debe devolver `True` en todo el rango. De ahí sale la primera parte de los archivos de la carpeta **Wolfram**.

Archivo	Tema	Cap.	Abrir
Exercises recursividad.nb	Recursividad	1	abrir
Exercises recurrencias.nb	Relaciones de recurrencia	2	abrir
Exercises algoritmos.nb	Análisis de algoritmos	3	abrir
Exercises relaciones binarias.nb	Relaciones binarias	4	abrir
Exercises grafos.nb	Teoría de grafos	5	abrir
Exercises arboles.nb	Teoría de árboles	6	abrir
Exercises maquinas.nb	Máquinas y autómatas	7	abrir
Exercises lenguajes.nb	Lenguajes y gramáticas	8	abrir

Cuadernos de las clases. El material que se ejecutó en cada sesión, en el mismo orden en que aparece en el video de esa clase. La numeración coincide con la de los videos de la sección A.2.

1 abrir	2 abrir	3 abrir	4 abrir	5 abrir	6 abrir
7 abrir	8 abrir	9 abrir	10 abrir	11 abrir	12 abrir
13 abrir	14 abrir	15 abrir	16 abrir	17 abrir	18 abrir
19 abrir	20 abrir	21 abrir	22 abrir	23 abrir	24 abrir
25 abrir	26 abrir	27 abrir			

Los capítulos del libro en PDF. Para consultar la teoría sin abrir el libro entero.

1 abrir	2 abrir	3 abrir	4 abrir	5 abrir	6 abrir	7 abrir	8 abrir
-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------

Exámenes, listas y el paquete.

Archivo	Qué es	Abrir
Compilacion 1 Examenes Discretas.pdf	Exámenes de períodos anteriores	abrir
Compilacion 2 Examenes Discretas.pdf	Más exámenes de períodos anteriores	abrir
Listas de ejercicios.pdf	Las listas repartidas en clase	abrir
Libro Vilcretas.pdf	El manual completo del paquete	abrir
Reinstalar el paquete.txt	Cómo instalar VilCretas desde cero	abrir
VilCretas.m	El paquete que carga todo el código del curso	abrir

A.2 Los videos de las clases

Son **31 videos** agrupados en diez bloques temáticos. La columna «Cap.» indica el capítulo del libro al que corresponde cada bloque; se verificó contrastando el título del video con el contenido del cuaderno de la clase del mismo número. Cuatro videos son de transición y cubren dos capítulos: son justamente las clases 14, 18, 22 y 26.

Clase	Título en Drive	Cap.	Enlace
RECURSIVIDAD			
1	Presentación del curso y recursividad	1	ver
2	Recursividad	1	ver
3	Recursividad	1	ver
RELACIONES DE RECURRENCIA			
4	Relaciones de recurrencia	2	ver
5	Relaciones de recurrencia	2	ver
6	Relaciones de recurrencia	2	ver
ANÁLISIS DE ALGORITMOS			
7	Análisis de algoritmos	3	ver
8	Análisis de algoritmos	3	ver
9	Análisis de algoritmos	3	ver
10	Análisis de algoritmos	3	ver
RELACIONES BINARIAS			
11	Relaciones binarias	4	ver
12	Relaciones binarias	4	ver
13	Relaciones binarias	4	ver
14	Relaciones binarias y grafos	4 y 5	ver
TEORÍA DE GRAFOS			
15	Teoría de grafos	5	ver
16	Teoría de grafos	5	ver
17	Teoría de grafos	5	ver
18	Teoría de grafos y árboles	5 y 6	ver
TEORÍA DE ÁRBOLES			
19	Teoría de árboles	6	ver
20	Teoría de árboles	6	ver
21	Teoría de árboles	6	ver
22	Teoría de árboles y máquinas de estado finito	6 y 7	ver
MÁQUINAS Y AUTÓMATAS DE ESTADO FINITO			
23	Máquinas y autómatas de estado finito	7	ver
24	Máquinas y autómatas de estado finito	7	ver
25	Máquinas y autómatas de estado finito	7	ver
26	Máquinas y autómatas, lenguajes y gramáticas	7 y 8	ver
LENGUAJES Y GRAMÁTICAS			
27	Lenguajes y gramáticas	8	ver
REPASO DE EXAMEN			
28	Repaso de examen	todos	ver
29	Repaso de examen	todos	ver
30	Repaso de examen	todos	ver
31	Repaso de examen	todos	ver

Los cuatro videos de «Repaso de examen» (clases 28 a 31) no pertenecen a un capítulo: repasan todo el curso antes de la evaluación. Se recomienda verlos con la cuarta parte de este cuadernillo delante.

A.3 Las presentaciones del curso

Las 26 presentaciones cubren el curso completo, una por sesión. Sus PDF están en la subcarpeta **Presentaciones de Entregables**, y los enlaces de la última columna los abren directamente. Como en la sección anterior, funcionan mientras el PDF y esa subcarpeta viajen juntos; si hiciera falta, ahí está la [carpeta completa](#).

Clase	Presentación	Cap.	Archivo
01	Recursividad	1	abrir
02	Propiedades de una recursividad	1	abrir
03	Dígitos, MCD y recursividad de cola	1	abrir
04	Relaciones de recurrencia	2	abrir
05	El método iterativo	2	abrir
06	Recurrencias homogéneas lineales	2	abrir
07	Análisis de algoritmos	3	abrir
08	Demostrar con O grande	3	abrir
09	Analizar ciclos	3	abrir
10	Comparar funciones con límites	3	abrir
11	Relaciones binarias	4	abrir
12	Representaciones y operaciones	4	abrir
13	Tipos de relaciones	4	abrir
14	Equivalencia, particiones y grafos	4 y 5	abrir
15	Representaciones de un grafo	5	abrir
16	Incidencia y circuitos de Euler	5	abrir
17	Circuitos de Hamilton	5	abrir
18	Camino más corto y árboles	5 y 6	abrir
19	Huffman y árboles binarios de búsqueda	6	abrir
20	Recorridos y árboles generadores	6	abrir
21	Árboles generadores y de expansión mínima	6	abrir
22	Máquinas de estado finito	7	abrir
23	Autómatas de estado finito	7	abrir
24	Diseño de autómatas y no determinismo	7	abrir
25	Determinización, y empiezan los lenguajes	7 y 8	abrir
26	Gramáticas, tipos y autómatas	8	abrir

La correspondencia con los videos no es uno a uno: las presentaciones reorganizan en unidades temáticas lo que en el semestre grabado ocupó 27 sesiones más cuatro de repaso. Por eso hay que guiarse por la columna «Cap.» y no por el número de clase cuando se combinan las dos fuentes.

A.4 Recursos generales

No pertenecen a ningún capítulo en particular; sirven a todo el curso.

Recurso	Para qué sirve	Enlace
El paquete VilCretas	Instalarlo es el primer paso del curso: sin él no corre casi ningún bloque de código de este cuadernillo.	abrir
Lista de reproducción en YouTube	Videos adicionales del autor sobre los mismos temas, útiles como segunda explicación.	abrir
Carpeta «Materiales importantes ED»	Los archivos de la sección A.1, en Google Drive.	abrir
Carpeta «Clases ED»	Los 31 videos de la sección A.2, en Google Drive.	abrir

A ellos se suman los ocho archivos **Exercises.zip** que el propio libro enlaza al comienzo de cada capítulo de ejercicios, en la primera parte de este cuadernillo.

APÉNDICE B

Índice de funciones de VilCretas

Las 197 funciones del paquete que aparecen en algún momento del curso: en los ejercicios del libro, en los complementarios o en las preguntas tipo examen. La columna “Cap.” dice en qué capítulos se usa cada una (varias reaparecen en dos o tres) y la última columna remite a un ejercicio concreto donde verla trabajando. La descripción es la primera frase de su propia documentación: para leerla completa, ?NombreFunción en Mathematica.

Hay que tener presente que el paquete exporta más de cuatrocientos símbolos y que la mayoría están **protegidos**: nombres tan corrientes como *ruta*, *lista*, *texto* o *analizar* no pueden usarse como variable. Si una asignación responde `Set::wrsym`, es esto.



El autor con Stephen Wolfram, creador de *Wolfram Mathematica*, en la *Wolfram Technology Conference* de 2016, donde participó como ponente. La presentación que expuso estuvo dedicada precisamente a **VilCretas**, el paquete que cataloga este apéndice.

Función	Cap.	Qué hace	Véase
AgenteViajero	5	Resuelve el problema del agente viajero sobre un grafo “G” simple, no dirigido creado en el “Wolfram System”, o bien, con el paquete “Combinatorica”	5.6.77
AlDijkstra	5	Ejecuta paso a paso el algoritmo de Dijkstra o de la longitud del camino más corto con respecto a dos vértices distintos “a” y “b”, sobre un grafo. . .	5.6.130
AlDijkstraMax	5	Ejecuta paso a paso una modificación del algoritmo de Dijkstra (seleccionando la marca máxima y actualizando marcas con el valor mayor) en el. . .	5.6.158
AlDijkstraMaxValor	5	Ejecuta una modificación del algoritmo de Dijkstra (seleccionando la marca máxima y actualizando marcas con el valor mayor) en el contexto de la. . .	5.6.148
AlDijkstraValor	5	Ejecuta el algoritmo de Dijkstra o de la longitud del camino más corto con respecto a dos vértices distintos “a” y “b”, sobre un grafo “G” conexo y. . .	5.6.147
Altura	6	Devuelve la altura de un árbol “T” recibido como parámetro (creado en el “Wolfram System” o con el paquete “Combinatorica”)	6.7.109

Función	Cap.	Qué hace	Véase
AnimarGrafo	5,6	Construye una animación sobre un grafo "G" y una lista "L" de aristas, remarcando paso a paso las aristas sobre el grafo "G" en el orden de "L"	5.6.34
Arbol	6	Construye un árbol a través del "Wolfram System" dadas sus aristas como una matriz de pares ordenados	6.7.105
ArbolBinario	6	Genera un árbol binario con "n" vértices usando el "Wolfram System" de Mathematica	6.7.12
ArbolBinarioBusqueda	6	Construye un árbol binario de búsqueda para almacenar un conjunto de datos numéricos o caracteres recibidos como parámetro, o bien, una frase. . .	6.7.109
ArbolBinarioCompletoQ	6	Función booleana utilizada para indicar si un árbol "T" recibido como parámetro es o no binario completo, retornando únicamente un valor lógico	6.7.152
ArbolBinarioRandom	6	Construye un árbol binario pseudoaleatorio con "n" vértices en el "Wolfram System" de Mathematica	6.7.12
ArbolBinarioReturnQ	6	Función booleana utilizada para indicar si un árbol "T" recibido como parámetro es o no binario, retornando únicamente un valor lógico	6.7.151
ArbolBinarioRRandom	6	Construye un árbol binario pseudoaleatorio con "n" vértices, retornando en un vector el árbol y su raíz	6.7.159
ArbolDirection	6	Al recibir un árbol binario "T" como parámetro (creado en el "Wolfram System") retorna un árbol binario con direcciones pseudoaleatorias. . .	6.7.160
ArbolEtiquetado	6	Construye un árbol binario para una expresión algebraica	6.7.148
ArbolExpAlgebraica	6	Construye un árbol para una expresión algebraica "ExpAlg" recibida como parámetro	6.7.31
ArbolGeneradorKruskal	6	Retorna las aristas de un árbol generador de peso mínimo y su ponderación, obtenido al emplear el algoritmo de Kruskal sobre un grafo "G" no. . .	6.7.153
ArbolGeneradorMaximo	6	Función que recibe como parámetro un grafo conexo, no dirigido, ponderado "G" y retorna un árbol generador (o de expansión) de peso máximo	libro
ArbolGeneradorPrim	6	Retorna las aristas de un árbol generador de peso mínimo y su ponderación, obtenido al emplear el algoritmo de Prim sobre un grafo "G" no dirigido, . . .	6.7.153
ArbolHuffman	6	Construye un árbol de códigos de Huffman optimizado, dado el "string" que se desea codificar	6.7.38
ArbolHuffmanN	6	Construye un árbol de códigos de Huffman no optimizado, dado el "string" que se desea codificar	6.7.37
ArbolJQ	6	Retorna "True" si el argumento "T" recibido como parámetro es un grafo en Mathematica que corresponde a un árbol o "False", en caso contrario	6.7.14
ArbolR	6	Construye un árbol a través del "Wolfram System" dadas sus aristas como una matriz de pares ordenados y un vértice "Raiz", correspondiente al nodo. . .	6.7.150
ArbolRandom	6	Construye un árbol pseudoaleatorio con "n" vértices, por defecto lo genera usando el "Wolfram System" de Mathematica	6.7.162
ArbolRBinario	6	Construye un árbol binario con direcciones izquierda/derecha en los lados a través del "Wolfram System" dadas sus aristas como una matriz de pares. . .	6.7.152
ArbolRBinarioBlancos	6	Construye un árbol binario con direcciones izquierda/derecha en los lados a través del "Wolfram System" dadas sus aristas como una matriz de pares. . .	6.7.163
ArbolRRandom	6	Construye un árbol pseudoaleatorio con "n" vértices, retornando en un vector el árbol y su raíz	6.7.159
AristasCombinatoriaToWolframSystem	6	Recibe los lados de un grafo construido mediante el paquete "Combinatoria" y retorna el conjunto como las aristas de un grafo creado en el "Wolfram. . .	libro

Función	Cap.	Qué hace	Véase
Automata	7,8	Crea un autómata de estado finito (determinístico (DFA) o no determinístico (NDFA)) dadas las cinco componentes que lo definen: el conjunto de...	7.6.113
AutomataDeterministicoEquivalente	7	Construye un autómata de estado finito determinístico (DFA) equivalente a otro no determinístico "A" (NDFA), ingresado como parámetro	7.6.133
AutomataDRandom	7,8	Construye de forma pseudoaleatoria un autómata de estado finito determinístico con "n" estados y "m" símbolos de entrada	7.6.134
AutomataExactamente	7	Construye un autómata de estado finito determinístico que acepta hileras con exactamente cierta cantidad de "a's", "b's", o cualquier otro carácter...	7.6.28
AutomataNDRandom	7	Construye de forma pseudoaleatoria un autómata de estado finito no determinístico con "n" estados y "m" símbolos de entrada	7.6.134
AutomataToDiagrama	7,8	Genera el diagrama de transición de un autómata de estado finito "A" recibido como parámetro (determinístico (DFA) o no determinístico (NDFA))	7.6.22
AutomataToRegular	8	Construye una gramática regular con el mismo lenguaje, de un autómata de estado finito determinístico "A" recibido como parámetro	8.5.145
Burbuja	3	Ordena una lista de datos de forma ascendente o descendente utilizando el "algoritmo de la burbuja"	3.6.13
BuscarPrimeroAncho	6	Aplica el algoritmo buscar primero a lo ancho sobre un grafo "G", mostrando como salida una lista ordenada de aristas transitadas durante el proceso	6.7.126
BuscarPrimeroLargo	6	Aplica el algoritmo buscar primero a lo largo sobre un grafo "G", mostrando como salida una lista ordenada de aristas transitadas durante el proceso	6.7.43
CalculaOtroProducto	3	Ejercicio para analizar un algoritmo pudiendo visualizar su código de programación con este objetivo	3.6.122
CalculaProducto	3	Ejercicio para analizar un algoritmo brindando la posibilidad de observar el código de programación con esta finalidad	3.6.27
CalculaSuma	3	Ejercicio para analizar un algoritmo brindando la posibilidad de observar el código de programación con esta finalidad	3.6.123
CantAristas	5,6	Retorna el número de lados que contiene un grafo "G" dirigido o no, creado con o sin el paquete "Combinatorica"	5.6.10
CantMRutas	5	Recibe un grafo simple "G" devolviendo una matriz donde cada una de sus entradas cuenta la cantidad máxima de rutas de longitud "k" (con repetición...	5.6.31
CantNodos	5,6	Retorna el número de vértices que contiene un grafo "G" dirigido o no, creado con o sin el paquete "Combinatorica"	5.6.3
CantRutas	5	Cuenta la cantidad total de rutas de cualquier longitud, de un nodo "a" a un nodo "b", sobre un grafo "G" simple (sin aristas múltiples, ni lazos)	5.6.116
CDFGraficaNA	3	Recibe dos funciones donde una se encuentra en el orden notacional asintótico de la otra (O grande u omega), representando visualmente la notación	3.6.87
CDFGraficaNAP	3	Recibe dos funciones donde una se encuentra en el orden notacional asintótico de la otra (O grande u omega), dependiendo de un parámetro "j"	3.6.138
CDFGraficaNOG	3	Representa gráficamente la relación asintótica "O grande" de las funciones: $\ln(\ln(n))$, $\ln(n)$, n (función identidad), $n\ln(n)$, n^2 , n^3 y 2^n	3.6.86

Función	Cap.	Qué hace	Véase
CDFMaquinaEF	7	Recibe una máquina de estado finito a través de las seis componentes que la definen: el conjunto de estados, el conjunto de símbolos de entrada, el...	7.6.9
CDFNRegions	2	Muestra sobre un plano el número de regiones en las que queda dividido al emplear "n" rectas que se cortan dos a dos en un único punto con $1 \leq n \leq 6$...	2.4.15
CDFOperaciones MBooleanas	4	Realiza las operaciones de unión, intersección, complemento y producto booleano sobre dos matrices booleanas	4.6.88
CircuitoEulerQ	5	Función booleana que determina si un grafo "G" posee un circuito de Euler	5.6.105
CircuitoHamiltonQ	5	Función booleana que determina si un grafo "G" posee un circuito de Hamilton	5.6.105
CircuitosEuler	5	Encuentra "n" circuitos de Euler distintos sobre un grafo "G", si es que existen	5.6.44
CircuitosHamilton	5	Encuentra "n" circuitos de Hamilton distintos sobre un grafo "G", si es que existen	5.6.111
ClasesEquivalencia	4	Retorna todas las clases de equivalencia de una relación de equivalencia definida sobre un conjunto "A" finito distinto de vacío	4.6.105
ClasificacionRelBin	4	Clasifica una relación binaria en reflexiva, simétrica, antisimétrica y transitiva	4.6.126
ColorearGrafo	5	Colorea los nodos de un grafo "G" recibido como parámetro	5.6.154
ComplementoBooleano	4	Calcula el complemento booleano sobre una matriz booleana	4.6.36
CompLimit	3	Compara en el límite dos funciones para determinar si la primera es "theta", "O grande" u "Omega" de la segunda	3.6.139
ComponentesAutomata	7,8	Retorna las cinco componentes que definen a un autómata de estado finito "A" recibido como parámetro (determinístico (DFA) o no determinístico (NDA))	7.6.130
Dato	1	Realiza la búsqueda de forma recursiva de un dato "a" sobre una lista "L"	1.6.25
DiagramaToMaquina	7	Construye una máquina de estado finito con sus seis componentes respectivas, a través de su diagrama de transición	7.6.135
Divisores	1	Determina mediante un recorrido recursivo todos los divisores de un número natural "n"	1.6.104
DominioRelBin	4	Retorna el conjunto dominio de una relación binaria	4.6.149
ElementoLenguajeQ	8	Función booleana encargada de determinar si una hilera de símbolos terminales "string" en una gramática libre del contexto "G", es un elemento del...	8.5.13
ElementosLenguaje	8	Recibe una gramática libre del contexto "G" y retorna todas las palabras del lenguaje de longitud máxima "m" obtenidas por "n" o menos derivaciones	8.5.100
ElementRelBinQ	4	Función booleana que determina si un par ordenado "{a, b}" es un elemento de una relación binaria dada de manera explícita, o bien, mediante una...	4.6.11
ExisteCircuitoHamilton	5	Determina la existencia de un circuito de Hamilton en un grafo "G" no dirigido, simple, conexo con una cantidad de vértices mayor o igual a tres, por...	5.6.107
Factoriales	1	Calcula de forma recursiva el factorial de un número natural o cero	1.6.11
FactorialesCola	1	Calcula de forma recursiva el factorial de un número natural o cero, usando una recursividad de cola	1.6.122
FindRRHL	2	Comando que encuentra si existe, una relación de recurrencia homogénea lineal con coeficientes constantes y su solución, dada una sucesión de números...	2.4.100

Función	Cap.	Qué hace	Véase
Fleury	5	Genera un circuito de Euler sobre un grafo "G" simple, conexo no dirigido, con todas sus valencias pares, utilizando como base el algoritmo de Fleury	5.6.126
GeneraRutas	5	Retorna una cantidad máxima de "n" rutas de un nodo "a" a un nodo "b" sobre un grafo "G", en caso de lograr encontrar los caminos	5.6.33
GraficaPC	4	Grafica en el plano cartesiano el producto cruz entre dos intervalos del conjunto de los números reales recibidos como parámetros	4.6.15
GraficaRelBin	4	Grafica los pares ordenados de una relación binaria definida por una o varias ecuaciones, o bien, una o varias inecuaciones o su combinación	4.6.16
GraficaRelBinPares	4	Grafica en el plano cartesiano los pares ordenados de una relación binaria dada de forma explícita	4.6.17
GraficaRRL	2	Instrucción que grafica una relación de recurrencia lineal permitiendo al usuario las opciones de especificar la cantidad de puntos a representar y...	2.4.105
Grafo	5,6	Construye un grafo a través del "Wolfram System" dadas sus aristas como una matriz de pares ordenados	5.6.100
GrafoBipartitoCompleto	5	Traza un grafo bipartito completo con "n" nodos izquierdos y "m" vértices derechos	5.6.13
GrafoCamino	5,6	Crea un grafo "path" o camino con "n" nodos	6.7.13
GrafoCiclo	5,6	Traza un grafo ciclo o circuito con "n" nodos	5.6.109
GrafoCompleto	5,6	Traza un grafo completo con "n" nodos	5.6.10
GrafoEstrella	5,6	Construye un grafo estrella con "n" nodos	5.6.68
GrafoM	5	Crea un grafo por medio del "Wolfram System" generado por su matriz de adyacencia de acuerdo con un conjunto "Nodos", que representa el orden de los...	5.6.157
GrafoMI	5	Crea un grafo por medio del "Wolfram System" generado por su matriz de incidencia de acuerdo con un conjunto "Nodos", que representa el orden de los...	5.6.157
GrafoMP	5,6	Construye un grafo por medio del "Wolfram System" generado por su matriz de adyacencia de pesos de acuerdo con un conjunto "Nodos", que representa el...	5.6.157
GrafoRandom	5	Construye si es posible, un grafo pseudoaleatorio no dirigido con "n" vértices y un mínimo de "m" aristas	5.6.152
GrafoRandomConexo	5	Construye si es posible, un grafo conexo pseudoaleatorio no dirigido con "n" vértices y un mínimo de "m" aristas	5.6.153
GrafoRandomEuler	5	Construye si es posible, un grafo pseudoaleatorio con "n" vértices y "m" aristas que posee un circuito o una ruta euleriana	5.6.150
GrafoRandomHamilton	5	Construye si es posible, un grafo pseudoaleatorio con "n" vértices y "m" aristas que posee un circuito o una ruta hamiltoniana	5.6.151
GrafoRed	5	Traza un grafo red o cuadrícula con "n" filas y "m" columnas ("n*m" nodos)	libro
GrafoRelBin	4	Representa mediante un grafo una relación binaria sobre un conjunto finito no vacío "A", recibida como parámetro a través de los pares ordenados que...	4.6.152
GrafoRueda	5,6	Genera un grafo rueda con "n" nodos	5.6.103
GrafoString	5	Construye un grafo que muestra una cadena "string" ingresada por el usuario	5.6.156
GramaticaQ	8	Retorna "True" si el parámetro recibido es una gramática, o "False", en caso contrario	8.5.146
GramaticaRandom	8	Construye de manera pseudoaleatoria una gramática regular, libre del contexto, sensible al contexto o de otro tipo con "n" símbolos no terminales y...	8.5.154

Función	Cap.	Qué hace	Véase
GramaticaSensibleContextoQ	8	Retorna "True" si al recibir como parámetro una gramática "G" ésta es sensible al contexto, o "False", en caso contrario	8.5.150
HileraArbolHuffman	6	Codifica o decodifica una hilera "string1" (string1 = letras o binarios, siendo excluyentes) a partir de un árbol de códigos de Huffman, dada una...	6.7.155
Hojas	6	Retorna los vértices finales u hojas de un árbol "T" recibido como parámetro (creado en el "Wolfram System" o con el paquete "Combinatorica")	6.7.105
HojasR	6	Retorna los vértices finales u hojas de un árbol "T" recibido como parámetro (creado en el "Wolfram System" o con el paquete "Combinatorica")	6.7.150
Insercion	3	Ordena una lista de datos de forma ascendente o descendente utilizando el "algoritmo de inserción"	3.6.15
Interfijo	6	Recorre un k-árbol "T" en orden intermedio (interorden/interfijo) mostrando como salida una lista ordenada de los nodos transitados durante el proceso	6.7.144
Interorden	6	Recorre un árbol binario "T" en orden intermedio (interorden/interfijo) mostrando como salida una lista ordenada de los nodos transitados durante el...	6.7.145
InterseccionBooleana	4	Determina la intersección booleana entre dos matrices booleanas	4.6.35
KArbol	6	Genera un árbol de orden "k" con "n" vértices usando el "Wolfram System" de Mathematica	6.7.11
KnCircuitoEulerQ	5	Función booleana que determina si un grafo completo Kn posee un circuito de Euler	5.6.137
KnCircuitoHamiltonQ	5	Función booleana que determina si un grafo completo Kn posee un circuito de Hamilton	5.6.139
KnmCircuitoEulerQ	5	Función booleana que determina si un grafo bipartito completo Kn,m posee un circuito de Euler	5.6.140
KnmCircuitoHamiltonQ	5	Función booleana que determina si un grafo bipartito completo Kn,m posee un circuito de Hamilton	5.6.141
KnmRutaEulerQ	5	Función booleana que determina si un grafo bipartito completo Kn,m posee una ruta de Euler	5.6.142
KnRutaEulerQ	5	Función booleana que determina si un grafo completo Kn posee una ruta de Euler	5.6.138
KnRutaHamiltonQ	5	Función booleana que determina si un grafo completo Kn posee una ruta de Hamilton	5.6.139
Kruskal	6	Muestra la ejecución del algoritmo de Kruskal sobre un grafo "G" no dirigido, conexo y ponderado, generando adicionalmente una animación arista por...	6.7.50
LenguajeCantidad	7	Devuelve el número de cadenas de símbolos de entrada de longitud "n" que son aceptadas por un autómata de estado finito "A" recibido como parámetro...	7.6.113
LenguajeStrings	7,8	Devuelve las cadenas de símbolos de entrada de longitud "n" que son aceptadas por un autómata de estado finito "A" recibido como parámetro...	7.6.113
ListaAristas	5,6	Retorna la lista de todos los lados de un grafo "G" creado en el "Wolfram System" de Mathematica, o bien, mediante el uso del paquete "Combinatorica"	5.6.4
ListaVertices	5	Retorna la lista de todos los vértices de un grafo "G" creado en el "Wolfram System" de Mathematica, o bien, mediante el uso del paquete...	5.6.4
LongitudCaminoMC	5,6	Retorna la longitud y una ruta de un camino más corto entre dos vértices "a" y "b" distintos en un grafo "G", creado con o sin "Combinatorica"	5.6.101
MaquinaAutomataQ	7	Retorna "True" si al recibir las seis componentes de una máquina ésta es un autómata de estado finito determinístico y "False", en caso contrario	7.6.127

Función	Cap.	Qué hace	Véase
MaquinaAutomataRandom	7	Construye de forma pseudoaleatoria una máquina de estado finito con "n" estados y "m" símbolos de entrada que corresponde a un autómata	7.6.134
MaquinaRandom	7	Construye de forma pseudoaleatoria una máquina de estado finito con "n" estados, "m" símbolos de entrada y "h" símbolos de salida	7.6.134
MaquinaRestadora Binarios	7	Muestra el diagrama de transición de una máquina de estado finito cuya función es restar dos números binarios cuyo resultado es no negativo	7.6.125
MaquinaSumadoraBinarios	7	Muestra el diagrama de transición de una máquina de estado finito cuya función es sumar dos números binarios	7.6.10
MaquinaToDiagrama	7	Genera el diagrama de transición de una máquina de estado finito dadas las seis componentes que la definen: el conjunto de estados, el conjunto de...	7.6.137
MatrizRelBin	4	Construye la matriz asociada a una relación binaria definida sobre dos conjuntos finitos "A" y "B" distintos de vacío	4.6.151
MCD	1	Calcula el máximo común divisor entre dos números enteros positivos o cero usando una lógica recursiva	1.6.71
MetodoI	2	Muestra sobre una relación de recurrencia lineal de orden uno, sus evaluaciones en "k" iteraciones de forma ascendente o descendente, según lo escoja...	2.4.45
MetodoRRHL	2	Muestra paso a paso la aplicación de uno de los métodos clásicos para resolver una relación de recurrencia homogénea lineal con coeficientes...	2.4.104
MGrafo	5	Recibe un grafo "G" y sus vértices en un conjunto "Nodos", retornando la correspondiente matriz de adyacencia de acuerdo con el orden de "Nodos"	5.6.114
MIGrafo	5	Recibe un grafo "G" y sus vértices en un conjunto "Nodos", retornando la correspondiente matriz de incidencia de acuerdo con el orden de "Nodos"	5.6.145
MPGrafo	5	Recibe un grafo ponderado "G" y sus vértices en un conjunto "Nodos", devolviendo la correspondiente matriz de adyacencia de pesos de acuerdo con el...	5.6.146
Multipli	1	Realiza de forma recursiva la multiplicación entre dos números enteros positivos	1.6.116
Multiplos	1	Muestra mediante un recorrido recursivo todos los múltiplos de un número natural "n" menor que otro número natural "m", visualizando si el usuario lo...	1.6.60
NFibonacci	1	Encuentra de forma recursiva un número de la sucesión de Fibonacci, además a través de la opción "code->True" muestra el código de Mathematica que...	1.6.4
NFibonacciCola	1	Encuentra de forma recursiva un número de la sucesión de Fibonacci, usando una recursividad de cola, además a través de la opción "code->True"...	1.6.125
NotationBNF	8	Recibe una gramática libre del contexto "G" y la retorna en notación de Backus-Naur	8.5.15
NotationFNormal	8	Recibe una gramática libre del contexto "G" y la retorna en notación "forma normal"	8.5.153
ParticionReEquivalencia	4	Retorna una partición sobre un conjunto finito no vacío "A", construida a través de una relación de equivalencia "R" sobre "A"	4.6.113
PCGeneral	4	Calcula el producto cartesiano entre "n" conjuntos dados distintos de vacío	4.6.144
PesoRuta	5,6	Calcula el peso asociado a una ruta o camino "L" sobre un grafo "G" (con o sin "Combinatorica")	5.6.100
Polaca	6	Genera la notación polaca de una expresión algebraica "ExpAlg" recibida como parámetro	6.7.32

Función	Cap.	Qué hace	Véase
PolacaInversa	6	Genera la notación polaca inversa de una expresión algebraica "ExpAlg" recibida como parámetro	6.7.33
Postfijo	6	Recorre un k-árbol "T" en orden final (postorden/postfijo) mostrando como salida una lista ordenada de los nodos transitados durante el proceso	6.7.142
PotenciaPos	1	Determina de manera recursiva la potencia de un número real, siendo el exponente un número natural o cero	1.6.35
Prefijo	6	Recorre un k-árbol "T" en orden inicial (preorden/prefijo) mostrando como salida una lista ordenada de los nodos transitados durante el proceso	6.7.109
Prim	6	Muestra la ejecución del algoritmo de Prim sobre un grafo "G" no dirigido, conexo y ponderado, generando adicionalmente una animación arista por...	6.7.130
ProductoBooleano	4	Realiza la multiplicación booleana	4.6.103
Productoria	1	Construye de manera automática un programa recursivo para calcular una productoria ingresada por el usuario, la instrucción además, facilita el...	1.6.112
ProductoriaCola	1	Construye de manera automática un programa recursivo de cola para calcular una productoria ingresada por el usuario, la instrucción además, facilita...	1.6.152
ProductoriaColaAux	1	Construye de manera automática un programa recursivo de cola con una función auxiliar para calcular una productoria ingresada por el usuario	1.6.153
ProductoriaGeneral	1	Construye de manera automática un programa recursivo para calcular una productoria ingresada por el usuario, donde el límite superior podría no ser...	1.6.157
ProductoriaRandom	1	Genera de forma pseudoaleatoria una productoria	1.6.159
PruebaADA2	3	Analiza la eficiencia de dos algoritmos que resuelven el mismo problema, ejecutando un experimento con "k" invocaciones y retornando el número de...	3.6.19
PruebaADA2Valores	3	Analiza la eficiencia de dos algoritmos que resuelven el mismo problema, ejecutando un experimento y retornando el número de veces en el que cada...	3.6.142
PruebaADA3	3	Analiza la eficiencia de tres algoritmos que resuelven el mismo problema, ejecutando un experimento con "k" invocaciones y retornando el número de...	3.6.21
PruebaADA3Valores	3	Analiza la eficiencia de tres algoritmos que resuelven el mismo problema, ejecutando un experimento y retornando el número de veces en el que cada...	3.6.144
PruebaADAGrafica	3	Analiza gráficamente la eficiencia de dos o más algoritmos que resuelven el mismo problema, en términos de los tiempos de salida registrados por el...	3.6.23
PruebaADAGraficaValores	3	Analiza gráficamente la eficiencia de dos o más algoritmos que resuelven el mismo problema, en términos de los tiempos de salida registrados por el...	3.6.145
Quicksort	1,3	Implementación del algoritmo "quicksort" desarrollado por el científico británico C.A.R Hoare en 1960 (ganador del premio Turing en 1980), para...	1.6.114
RangoRelBin	4	Retorna el conjunto rango de una relación binaria	4.6.149
ReduceReglas	8	Retorna las reglas de composición de una gramática "G" en forma reducida, es decir, si existen "n" producciones que inician con "A", las devuelve...	8.5.151
ReEquivalenciaParticion	4	Construye una relación de equivalencia, dada una partición "P" definida sobre un conjunto finito no vacío "A"	4.6.137
RegularToAutomata	8	Recibe una gramática regular "Gr" y a través de ella construye un autómata no determinístico con su mismo lenguaje	8.5.145

Función	Cap.	Qué hace	Véase
RelacionComposicion	4	Retorna la composición entre dos relaciones binarias R1 y R2. Sintaxis: RelacionComposicion[R1, R2], o bien, RelacionComposicion[R1, R2, code->True]...	4.6.102
RelacionEquivalenciaQ	4	Retorna "True" si la relación binaria recibida como pares ordenados es de equivalencia o "False", en caso contrario	4.6.134
RelacionOrdenParcialQ	4	Retorna "True" si la relación binaria recibida como parámetro de manera explícita es de orden parcial o "False", en caso contrario	4.6.130
RelBin	4,5	Retorna de forma explícita los elementos de una relación binaria finita, dada la condición que la define y los conjuntos sobre los cuáles se construye	4.6.10
RelBinMatriz	4	Retorna los pares ordenados de una relación binaria definida sobre dos conjuntos finitos "A" y "B" distintos de vacío, dada su matriz de...	4.6.20
RelBinRandom	4	Construye de forma pseudoaleatoria una relación binaria definida de un conjunto "A" a un conjunto "B"	4.6.153
RelBinRandomConjuntos	4	Construye de forma pseudoaleatoria una relación binaria definida de un conjunto "A" a un conjunto "B"	4.6.153
RelBinRandomCriterios	4	Construye de forma pseudoaleatoria una relación binaria definida de un conjunto "A" a un conjunto "B" especificando el número "n" de criterios a...	4.6.154
RelBinRandomCriterios Reales	4	Proporciona de forma pseudoaleatoria el o los criterios para la construcción de una relación binaria definida sobre el conjunto de los números reales	4.6.155
ResaltarRuta	5	Resalta sobre un grafo "G" una ruta obtenida de una lista "L" de aristas	libro
RestaBinaria	7	Realiza la resta no negativa de dos números binarios "a" y "b" usando como base una máquina de estado finito	7.6.124
RRCP	2	Resuelve distintos casos de una relación de recurrencia lineal no homogénea, donde la parte no homogénea depende de un parámetro "j"	2.4.113
RRHLRandom	2	Genera de forma pseudoaleatoria una relación de recurrencia homogénea lineal con coeficientes constantes recibiendo como parámetro el orden "n"	2.4.140
RRLRandom	2	Genera de forma pseudoaleatoria una relación de recurrencia lineal de orden "n"	2.4.147
RutaEuler	5	Retorna una ruta de Euler sobre un grafo "G" simple (sin aristas múltiples, ni lazos) no dirigido que la contenga	5.6.123
RutaEulerQ	5	Función booleana que retorna "True" si un grafo "G" no dirigido recibido como parámetro posee una ruta de Euler o "False", en caso contrario	5.6.123
RutaHamiltonQ	5	Función booleana que retorna "True" si un grafo simple no dirigido "G" recibido como parámetro posee una ruta de Hamilton o "False", en caso contrario	5.6.151
Seleccion	3	Ordena una lista de datos de forma ascendente o descendente utilizando el "algoritmo de selección"	3.6.16
StringAceptadaQ	7,8	Retorna "True" si una hilera de símbolos de entrada es aceptada por un autómata de estado finito "A" recibido como parámetro (determinístico (DFA) o...	7.6.129
StringSalida	7	Construye la hilera de símbolos de salida que se obtiene al procesar una hilera de símbolos de entrada en una máquina de estado finito	7.6.138
StringSalidaDiagrama	7	Construye la hilera de símbolos de salida que se obtiene al procesar una hilera de símbolos de entrada en una máquina de estado finito	7.6.136
SumaDigi	1	Suma de manera recursiva los dígitos de un número natural	1.6.135

Función	Cap.	Qué hace	Véase
Sumatoria	1	Construye de manera automática un programa recursivo para calcular una sumatoria ingresada por el usuario, la instrucción además, facilita el cálculo. . .	1.6.110
SumatoriaCola	1	Construye de manera automática un programa recursivo de cola para calcular una sumatoria ingresada por el usuario, la instrucción además, facilita el. . .	1.6.150
SumatoriaColaAux	1	Construye de manera automática un programa recursivo de cola con una función auxiliar para calcular una sumatoria ingresada por el usuario	1.6.151
SumatoriaGeneral	1	Construye de manera automática un programa recursivo para calcular una sumatoria ingresada por el usuario, donde el límite superior podría no ser un. . .	1.6.154
SumatoriaRandom	1	Genera de forma pseudoaleatoria una sumatoria	1.6.158
TipoGramatica	8	Determina si una gramática "G" recibida como parámetro es "sensible al contexto", "libre del contexto", o bien, "regular"	8.5.147
TipoMRelacion	4	Clasifica una relación binaria en reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial, dada de manera explícita	4.6.128
TipoRelacion	4	Clasifica una relación binaria en reflexiva, simétrica, antisimétrica, transitiva, de equivalencia y de orden parcial, dados sus pares ordenados de. . .	4.6.124
UnionBooleana	4	Calcula la unión booleana entre dos matrices booleanas	4.6.35
Valencias	5,6	Retorna las valencias o grados de todos los vértices de un grafo "G" dirigido o no, creado con o sin el paquete "Combinatorica"	5.6.103

El manual completo del paquete es [Libro Vilcretas.pdf](#), en la carpeta descrita en el apéndice A.

APÉNDICE C

Mapa de estudio y simulacros

Este apéndice responde tres preguntas que todo estudiante se hace antes de un examen: “qué hay en este capítulo”, “qué tengo que saber hacer” y “cómo sé si ya estoy listo”. Hay una entrada por capítulo, siempre con la misma estructura.

El **simulacro** no trae preguntas nuevas: reúne diez de la cuarta parte, de categorías distintas, con el tiempo que debería bastar para cada una. La forma de usarlo es resolverlo entero, cronometrado y sin mirar, y solo después comparar con las soluciones.

C.1 Capítulo 1. Recursividad

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (1.5)	22	<i>sin clasificar: son los del autor</i>					
Complementarios	160	13	78	69	68	24	68
Tipo examen	31	0	14	17	13	7	11
Total	213	13	92	86	81	31	79

Lo que hay que saber hacer

- ☐ Escribir una función recursiva de pila para una sumatoria o una productoria, identificando bien el caso raíz.
- ☐ Explicar por qué una recursividad sin caso raíz alcanzable no termina.
- ☐ Convertir una recursividad de pila en una de cola, hacia adelante con contador y hacia atrás, y decir cuándo cada forma es posible.
- ☐ Contar cuántas llamadas genera una definición recursiva y estimar su costo.
- ☐ Usar **Factoriales**, **NFibonacci** y sus versiones de cola, y leer el código que devuelven con `code -> True`.
- ☐ Reconocer una definición recursiva mal planteada leyendo solo el enunciado.
- ☐ Traducir a recursividad una definición dada por una fórmula cerrada.
- ☐ Justificar por escrito por qué una recursividad calcula lo que dice calcular.

Simulacro de examen · 71 minutos

#	Ejercicio	Categoría del banco	min
1	1.7.1 ★★	1-Rec. Prog. de pila	5
2	1.7.2 ★★★	2-Rec. Prog. de sumatorias con Rec. de pila	8
3	1.7.2 ★★★	2-Rec. Prog. de sumatorias con Rec. de pila	8
4	1.7.4 ★★★	3-Rec. Prog. de productorias con Rec. de pila	8
5	1.7.6 ★★	5-Rec. Prog. de cola	5
6	1.7.7 ★★★	6-Rec. Prog. de sumatorias con Rec. de cola	8
7	1.7.9 ★★★	4-Rec. Problemas de desarrollo con Rec. de pila	8
8	1.7.10 ★★★	7-Rec. Problemas de desarrollo con Rec. de cola	8
9	1.7.11 ★★	1-Rec. Prog. de pila / 5-Rec. Prog. de cola	5
10	1.7.31 ★★★	2-Rec. Prog. de sumatorias con Rec. de pila / 6-Rec. de cola	8
Total			71

C.2 Capítulo 2. Relaciones de recurrencia

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (2.3)	27	<i>sin clasificar: son los del autor</i>					
Complementarios	149	7	76	66	67	25	57
Tipo examen	34	0	17	17	10	4	20
Total	210	7	93	83	77	29	77

Lo que hay que saber hacer

- ☐ Plantear la relación de recurrencia de un problema enunciado en palabras.
- ☐ Resolver una recurrencia por el método iterativo y comprobar el resultado.
- ☐ Resolver una homogénea lineal por la ecuación característica, con raíces distintas, repetidas y complejas.
- ☐ Distinguir un problema planteado en a_0 de uno planteado en a_1 y ajustar la opción inicio en consecuencia.
- ☐ Usar RR, MetodoRRHL, RT y RRCP y contrastar la fórmula cerrada con los primeros términos.
- ☐ Reconocer cuándo FindRRHL devuelve NaD y explicar por qué.
- ☐ Resolver una no homogénea sencilla y verificar la solución particular.
- ☐ Interpretar la solución: crecimiento, oscilación, convergencia.

Simulacro de examen · 59 minutos

#	Ejercicio	Categoría del banco	min
1	2.5.1 ★★	1-Rr. Evaluación de recurrencias	5
2	2.5.2 ★★	2-Rr. Hallar recurrencia dada una lista	5
3	2.5.3 ★★	3-Rr. Gráfica de una recurrencia	5
4	2.5.4 ★★	4-Rr. Dada la fórmula explícita hallar una recurrencia	5
5	2.5.5 ★★★	5-Rr. Caso raíces diferentes	8
6	2.5.6 ★★★	6-Rr. Caso raíces repetidas	8
7	2.5.7 ★★	7-Rr. Indicar conjunto Sol. de la ecuación auxiliar	5
8	2.5.8 ★★★	8-Rr. Sistema de ecuaciones asociado	8
9	2.5.9 ★★	9-Rr. Recurrencias lineales no homogéneas	5
10	2.5.10 ★★	10-Rr. Clasificar relación	5
Total			59

C.3 Capítulo 3. Análisis de algoritmos

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (3.5)	26	<i>sin clasificar: son los del autor</i>					
Complementarios	147	12	71	64	66	25	56
Tipo examen	33	1	11	21	8	3	22
Total	206	13	82	85	74	28	78

Lo que hay que saber hacer

- ☐ Contar operaciones elementales de un algoritmo dado en pseudocódigo.
- ☐ Demostrar $f \in O(g)$ exhibiendo la constante y el umbral.
- ☐ Distinguir O , Ω y Θ y decir cuándo no existe Θ .
- ☐ Analizar ciclos anidados traduciendo los a sumatorias.
- ☐ Usar la propiedad del límite para comparar dos órdenes.
- ☐ Ordenar una lista de funciones por orden de crecimiento y justificarlo.
- ☐ Usar PruebaADA y sus variantes, y leer con cuidado los empates.

- ☐ Estimar el efecto práctico de un orden: cuánto tarda si la entrada se duplica.

Simulacro de examen · 63 minutos

#	Ejercicio	Categoría del banco	min
1	3.7.1 ★★★	1-Al. Comparación por experimento de 2 algoritmos	8
2	3.7.1 ★★★	1-Al. Comparación por experimento de 2 algoritmos	8
3	3.7.2 ★★★	2-Al. Comparación por experimento de 4 métodos	8
4	3.7.2 ★★★	2-Al. Comparación por experimento de 4 métodos	8
5	3.7.4 ★★	3-Al. Análisis O grande para un procedimiento en código	5
6	3.7.5 ★★	4-Al. Simplificar productorias	5
7	3.7.6 ★★	5-Al. Simplificar productos	5
8	3.7.7 ★	6-Al. Simplificar sumas cortas	3
9	3.7.8 ★★	7-Al. Simplificar sumatorias	5
10	3.7.10 ★★★	8-Al. Conjetura de parámetros para un análisis asintótico	8
Total			63

C.4 Capítulo 4. Relaciones binarias

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (4.5)	21	<i>sin clasificar: son los del autor</i>					
Complementarios	155	25	68	62	70	42	43
Tipo examen	36	1	15	20	10	6	20
Total	212	26	83	82	80	48	63

Lo que hay que saber hacer

- ☐ Escribir una relación binaria por extensión, por comprensión y como matriz.
- ☐ Clasificar una relación: reflexiva, simétrica, antisimétrica y transitiva, con un contraejemplo cuando falle.
- ☐ Traducir cada propiedad a una condición sobre la matriz booleana.
- ☐ Calcular unión, intersección, complemento, inversa y composición.
- ☐ Reconocer una relación de equivalencia y obtener sus clases y la partición.
- ☐ Reconocer un orden parcial y dibujar el diagrama de Hasse.
- ☐ Usar `RelBin`, `MatrizRelBin`, `ClasificacionRelBin` y los operadores booleanos con la opción `lista -> True`.
- ☐ Contar relaciones con una propiedad dada sobre un conjunto pequeño.

Simulacro de examen · 65 minutos

#	Ejercicio	Categoría del banco	min
1	4.7.1 ★★★	1-Rel. Dominio de una relación	8
2	4.7.3 ★★	3-Rel. Producto cartesiano	5
3	4.7.5 ★★	4-Rel. Producto cartesiano de intervalos (Desarrollo)	5
4	4.7.6 ★★★	5-Rel. Relaciones y su gráfico (conjunto de pares)	8
5	4.7.7 ★★	2-Rel. Graficación de relaciones en el plano cartesiano (Desarrollo)	5
6	4.7.8 ★★	6-Rel. Representación gráfica de algunas relaciones en el plano	5
7	4.7.9 ★★★	7-Rel. De digrafo a matriz	8
8	4.7.10 ★★	8-Rel. De digrafo a pares	5
9	4.7.11 ★★★	9-Rel. Operaciones unión, intersección, inversa, complemento con matrices	8
10	4.7.13 ★★★	10-Rel. Operaciones composición, unión, intersección, inversa, complemento	8
Total			65

C.5 Capítulo 5. Teoría de grafos

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (5.5)	22	<i>sin clasificar: son los del autor</i>					
Complementarios	158	20	73	65	60	39	59
Tipo examen	42	2	21	19	14	3	25
Total	222	22	94	84	74	42	84

Lo que hay que saber hacer

- ☐ Distinguir grafo simple, multigrafo, dirigido y ponderado, y contar sus aristas.
- ☐ Calcular valencias y usar el teorema de la suma de los grados.
- ☐ Decidir la existencia de circuitos de Euler y construirlos con Fleury.
- ☐ Discutir la existencia de circuitos de Hamilton y aplicar las condiciones suficientes conocidas.
- ☐ Aplicar Dijkstra a mano y con `AlDijkstra`, y leer el resultado.
- ☐ Pasar de grafo a matriz de adyacencia y volver, e interpretar sus potencias.
- ☐ Reconocer isomorfismo entre dos grafos o justificar que no lo hay.
- ☐ Usar `Grafo`, `Valencias`, `MGrafo`, `CircuitosHamilton` y `PesoRuta` sin confiar a ciegas en `AgenteViajero`.

Simulacro de examen · 57 minutos

#	Ejercicio	Categoría del banco	min
1	5.7.1 ★★	1-TG. Conceptos básicos de grafos	5
2	5.7.2 ★★	2-TG. Valencias de grafos no dirigidos	5
3	5.7.3 ★★	3-TG. Valencias internas/externas	5
4	5.7.4 ★	4-TG. Grafos conexos	3
5	5.7.5 ★★	5-TG. Número de aristas	5
6	5.7.6 ★★	6-TG. Grafos notables	5
7	5.7.7 ★★★	7-TG. Circuitos de Euler	8
8	5.7.8 ★★	8-TG. Trayectorias de Euler en Kn	5
9	5.7.12 ★★★	12-TG. Análisis de trayectorias y circuitos	8
10	5.7.13 ★★★	13-TG. Matriz de adyacencia (Desarrollo)	8
Total			57

C.6 Capítulo 6. Teoría de árboles

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (6.6)	26	<i>sin clasificar: son los del autor</i>					
Complementarios	163	15	71	77	81	33	49
Tipo examen	34	0	12	22	10	4	20
Total	223	15	83	99	91	37	69

Lo que hay que saber hacer

- ☐ Reconocer un árbol y contar sus aristas, hojas y niveles.
- ☐ Construir un árbol binario de búsqueda y recorrerlo en preorden, inorden y posorden.
- ☐ Reconstruir un árbol a partir de dos de sus recorridos.
- ☐ Construir el código de Huffman de una lista de frecuencias y calcular la longitud media.
- ☐ Aplicar Prim y Kruskal y comparar los árboles obtenidos.
- ☐ Usar búsqueda en anchura y en profundidad para obtener un árbol generador.

- ☐ Usar `ArbolBinarioBusqueda`, `Huffman`, `AlPrim` y `AlKruskal`, recordando que la orientación con que se imprime una arista no altera el árbol.
- ☐ Justificar por qué un árbol de expansión mínima no tiene por qué ser único.

Simulacro de examen · 68 minutos

#	Ejercicio	Categoría del banco	min
1	6.8.1 ★★	1-TAr. Info. de un árbol	8
2	6.8.2 ★★	3-TAr. Verificar que es un árbol	5
3	6.8.3 ★★	2-TAr. Dibujar árboles en software (Desarrollo)	5
4	6.8.4 ★★	4-TAr. Códigos de <i>Huffman</i>	8
5	6.8.5 ★★	5-TAr. Expresiones algebraicas y conversión a polaca	8
6	6.8.6 ★★	6-TAr. Almacenamiento de datos numéricos y los tres recorridos	8
7	6.8.7 ★★	6-TAr. Almacenamiento de datos numéricos	5
8	6.8.8 ★★	7-TAr. Almacenamiento de hileras y los tres recorridos	5
9	6.8.9 ★★	8-TAr. Árboles generadores (BFS/DFS)	8
10	6.8.11 ★★	9-TAr. Árboles de expansión mínima	8
Total			68

C.7 Capítulo 7. Máquinas de estado finito y autómatas

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (7.5)	39	<i>sin clasificar: son los del autor</i>					
Complementarios	138	9	63	66	78	33	27
Tipo examen	38	1	13	24	13	5	20
Total	215	10	76	90	91	38	47

Lo que hay que saber hacer

- ☐ Leer y construir la tabla de transiciones de una máquina de estado finito.
- ☐ Dibujar el diagrama de estados a partir de la tabla y al revés.
- ☐ Decidir si un autómata acepta una hilera, siguiendo el recorrido paso a paso.
- ☐ Describir con palabras el lenguaje que acepta un autómata.
- ☐ Construir un autómata que reconozca un lenguaje descrito en palabras.
- ☐ Distinguir determinístico de no determinístico y convertir uno en otro.
- ☐ Usar `Automata`, `StringAceptadaQ` y `AutomataToDiagrama`, cuidando que los nombres de estados no choquen con símbolos protegidos.
- ☐ Minimizar un autómata identificando estados equivalentes.

Simulacro de examen · 66 minutos

#	Ejercicio	Categoría del banco	min
1	7.7.1 ★	1-MqAu. Conceptos básicos de MEF	3
2	7.7.2 ★★	2-MqAu. DT de una MEF	8
3	7.7.4 ★★	4-MqAu. Procesando una hilera en una MEF	5
4	7.7.5 ★★	3-MqAu. MEF dado el DT	5
5	7.7.6 ★★	5-MqAu. Determinar si una MEF es un autómata	5
6	7.7.7 ★★	6-MqAu. DT de un autómata	8
7	7.7.8 ★★	8-MqAu. Procesando una hilera en un autómata	8
8	7.7.10 ★★	9-MqAu. Hileras generales aceptadas por un autómata	8
9	7.7.11 ★★	10-MqAu. Diseño de autómatas	8
10	7.7.13 ★★	11-MqAu. DT de un Auto No Det	8
Total			66

C.8 Capítulo 8. Lenguajes y gramáticas

	Total	★	★★	★★★	a mano	VilCretas	mixto
Del libro (8.4)	22	<i>sin clasificar: son los del autor</i>					
Complementarios	156	10	77	69	89	34	33
Tipo examen	33	0	13	20	14	3	16
Total	211	10	90	89	103	37	49

Lo que hay que saber hacer

- ☐ Manejar alfabeto, hilera, longitud, concatenación y potencias de un lenguaje.
- ☐ Distinguir L^* de L^+ y el lenguaje vacío de $\{\varepsilon\}$.
- ☐ Escribir la derivación de una hilera a partir de una gramática, paso a paso.
- ☐ Clasificar una gramática en la jerarquía de Chomsky.
- ☐ Escribir una gramática regular para un lenguaje descrito en palabras.
- ☐ Pasar de gramática regular a autómatas y al revés, y explicar por qué el regreso no devuelve la gramática original.
- ☐ Reconocer una gramática ambigua exhibiendo dos derivaciones de la misma hilera.
- ☐ Usar `ElementosLenguaje`, `ElementoLenguajeQ`, `RegularToAutomata` y `AutomataToRegular`, sabiendo que el primer parámetro acota las derivaciones y no la longitud.

Simulacro de examen · 68 minutos

#	Ejercicio	Categoría del banco	min
1	8.6.1 ★★	1-LG. Conceptos básicos sobre LyG	5
2	8.6.2 ★★★	2-LG. Verificar que una hilera es elemento de $L(G)$	8
3	8.6.2 ★★★	2-LG. Verificar que una hilera es elemento de $L(G)$	8
4	8.6.3 ★★	5-LG. Lenguaje asociado a una G libre de contexto	5
5	8.6.4 ★★	3-LG. Notación BNF	5
6	8.6.5 ★★★	4-LG. Tipos de gramática	8
7	8.6.5 ★★★	4-LG. Tipos de gramática	8
8	8.6.6 ★★★	7-LG. Diseñar gramáticas (Desarrollo)	8
9	8.6.8 ★★★	6-LG. $L(G)$ mediante un AEF	8
10	8.6.20 ★★	7-LG. Diseñar gramáticas	5
Total			68

Los ocho simulacros suman 517 minutos de trabajo cronometrado. Si al resolverlos aparece una categoría que se resiste, el camino es volver a la lista de arriba, localizar la línea correspondiente y repasar los ejercicios complementarios de ese apartado antes de intentarlo de nuevo.